

Custom Compiler™ Layout Editor User Guide for Connecting Objects

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	14
Related Products, Publications, and Trademarks	14
Conventions	15
Customer Support	15
Statement on Inclusivity and Diversity	16
1. Understanding Connectivity	17
About the Layout Editor Connectivity Features	17
About Connectivity	18
Connectivity Engine	18
Connectivity Concepts	18
Layer Ordering and Adjacency	19
Mask Layer Ordering and Adjacency	19
Material-Based Connections	19
Propagating Layer Identification	23
Cut Layer Identification	23
Layer Attribute Configuration	23
Propagating Connectivity	24
Implicit Net Assignments	24
Explicit Net Assignments	25
Connectivity Anchors and Islands	25
Connectivity Anchor	26
Connectivity Island	28
Non-propagating Layers	28
Short Circuits	28
Short Circuit at the Top Level (0x) of the Design	29
Short Circuit Through the Hierarchy of the Design	29
Enabling the Connectivity Engine	35
Defining and Changing Pin Relationships	35
Types of Pin Relationships	36
Changing Pin Relationships	37
How the Pin Relationship Command Affects Pins and Terminals	37
Strong	38
Weak	38

Contents

Must Join	39
Using Pins and Terminals for Hierarchical Connections	40
Pins and Terminals in Hierarchical Instances	40
Pin Connection Relationships	41
Strongly Connected	42
Weakly Connected	42
Must Join	43
Examples of Pin Connection Relationships	44
Understanding Flight Lines	45
About Flight Lines	45
Flight Lines and Connectivity Extraction	46
Displaying and Hiding Flight Lines	46
Working With PCells and Feedthrough Shapes	48
2. Controlling Route Placement	49
Ways to Control Route Placement	49
Creating and Editing Blockages	50
About Blockages	50
Creating Blockages	50
Editing Blockages	51
Blockage Toolbar and Create Blockage Dialog Box Reference	54
Blockage Shortcut Menu and Shortcut Keys	56
Creating and Editing Route Corridors	57
About Route Corridors	58
Before You Start	58
Creating New Route Corridors	58
Appending to Existing Route Corridors	60
Viewing Route Corridor Area Information	61
Changing Route Corridor Attributes	61
Creating and Editing Track Region PCells	62
About Track Region PCells	63
Before You Start	64
Creating Track Region PCells Using Presets	64
Creating New Track Region PCells	66
Copying Track Groups	73
Deleting Track Groups	74
Editing Track Region PCells	74

Contents

Displaying and Hiding Track Region PCells	76
Track Region PCell Toolbar and Dialog Box Reference	77
Create Rectangle Toolbar and Dialog Box (Track Region PCells)	77
Edit Tracks Dialog Box	78
Apply Preset Dialog Box	84
Crossing Track Regions	86
Setting allowCrossBoundary	86
Setting leCompatibleTracks	87
Considering Net Support and Track Types	89
Crossing Boundary Types	91
Boundary Crossing and Track Grids	91
Crossing Constraint Group Regions	92
Considerations When Crossing Constraint Group Regions	92
Considerations When Regions Overlap	93
Setting Up Region-Based Constraint Groups	94
Setting Up Track Grids	94
3. Creating Pins and Terminals	96
About Pins and Terminals	96
Before You Start	98
Track Setup	99
Pin Generation for Invisible Layers	99
Creating and Modifying Terminals	99
Creating Terminals	100
Define Terminals Dialog Box	100
Creating Individual Pins Automatically	102
Creating Pin Shapes (Auto Mode)	102
Pin Toolbar and Create Pin Dialog Box (Auto Mode)	104
Attributes Section	105
Create Pin Label Section	108
Pin Shortcut Menu and Shortcut Keys (Auto Mode)	110
Creating Individual Pins Manually	111
Creating a Single Pin Shape on a Single Terminal (Manual Mode)	112
Creating Multiple Pin Shapes on One Terminal (Manual Mode)	114
Creating Identical Pins on Different Terminals (Manual Mode)	115
Pin Toolbar and Create Pin Dialog Box (Manual Mode)	115
Attributes Section	116

Contents

Create Pin Label Section	118
Pin Shortcut Menu and Shortcut Keys (Manual Mode)	120
Creating Individual Pins From Existing Shapes	121
About the Create > Pin > From Selection Command	122
Creating Pins From Existing Shapes	123
Pins From Selection Toolbar and Create Pins From Selection Dialog Box	123
General Options	124
New Shapes	125
Create Pin Label	126
Generating Multiple Pins Simultaneously	128
About Multiple-Pin Generation	128
Understanding Automatic Pin Placement	129
Preparing the Layer Mapping File	131
Before You Start to Generate Pins	132
Ignoring Labels	132
Ignoring User-Defined Blockage Layers	132
Generating Pins Within the Layout	132
Generating Pins for Specified Top Layers	133
Generating Layout Pins at the Library Level	135
Generate Pins Toolbar and Dialog Box Reference	137
Area	139
From Text	139
From Net	141
Other Options	145
Generate Layout Pins Dialog Box Reference	146
Editing Instance Pins	151
Before You Start	151
Selecting and Editing Instance Pins	153
Deleting Generated Pins	155
4. Creating and Editing Interconnects	156
About Interconnects	157
Before You Start	158
Setting Up Tracks for Routing	158
Changing Interconnect Colors During Track Snapping	158
Enforcing Route Direction	159
Displaying Track Widths as You Route	159
Constraining Maximum Path Widths	160

Contents

Changing Same-Layer Path Widths as You Route	160
Extending Path Ends to Cover Vias	161
Displaying Net Names	162
Creating Interconnects	163
About the Create > Interconnect Command	164
Paths and Path Segments	164
Using the Create > Interconnect Command	165
Interconnect Toolbar and Create Interconnect Dialog Box	167
Interconnect Options	172
Auto Options	177
Via Justification	178
Create Interconnect Menu and Shortcut Keys	178
Create Interconnect Context-Sensitive Toolbar	183
Selecting Layer Purpose Pairs for Interconnects	185
Connecting Routes on the Same Layer	185
Changing Interconnect Layers	187
Avoiding Overlap	188
Choosing End Styles	189
Adding Vias Between Interconnects	192
Before You Start	192
Adding Default Vias	192
Adding Custom Vias or a Via Stack	193
Adding Default Bridge Vias	196
Adding Custom Bridge Vias	197
Adding Vias Between Overlapping Interconnects	197
Positioning the Vias	199
Add Via Dialog Box Reference	201
Add Bridge Via Dialog Box	204
Editing Paths and Path Segments Using the Property Editor	204
Cutting Corners of Paths	208
Reshaping Interconnects	208
Commands for Reshaping Paths or Path Segments	209
Using the Reshape Interconnect Command	209
Saving and Deleting Dangling Path Sections	211
Inserting Vias When Reshaping Interconnects	212
Examples of Reshaping Interconnects	212
Reshape Interconnect Toolbar and Dialog Box	213

Contents

Reshape Interconnect Shortcut Menu and Shortcut Keys	215
Add Via Dialog Box for Reshape Interconnect	217
Layer Table	217
Define Via Size Manually	219
Bridging Interconnects	220
About Bridging Interconnects	220
Bridging Leading Interconnects in Bus Configurations	222
Using the Bridge Interconnect Command	223
Bridge Toolbar	223
Bridge Interconnect Menu and Shortcut Keys	224
Stacking Interconnects	225
About Stacking Interconnects	225
Before You Start	226
Adding Stacked Interconnects	226
Striping Wires	227
About Striped Wires	227
What is Striping	228
Why is Striping Needed	228
Wire Color, Width, and Spacing	228
Limitations	229
Examples Of Wire Striping	229
Striping Wires Manually	230
Before You Start	231
Procedure for Striping Wires	231
Editing Striped Wire Attributes	232
Stacking Specified Striped Wires	234
Converting Striped Wires Back to Fat Wires	234
Stripe Toolbar Reference	235
Chopping Striped Wires	236
Trimming Wires	237
About Trimming Wires	237
Using the Edit > Wire > Trim Wire Command	239
Trim Wire Toolbar	241
Trim Wire Examples	242
Stretching Interconnects	244
Stretching Paths and Path Segments	244
Stretching Interconnects and Adding Vias Automatically	248
Stretching Interconnects on Track Grids	249

Stretching Striped Wires	249
Moving Paths and Interconnects	250
Adding Vias Between Overlapping Interconnects	250
Changing LPPs When Moving Paths	251
Merging Paths and Path Segments Automatically	251
About Merging Paths and Path Segments	252
Requirements for Merging Paths and Path Segments Automatically	252
Results of Path and Path Segment Merging	253
Stretching the Wire Width	254
About Stretching Wire Widths	254
Procedure for Stretching Wire Widths	255
5. Creating and Editing Bus Routes	258
About Bus Routes	258
Before You Start	259
Creating Paths or Path Segments in Bus Routes	260
Displaying Detailed Information About the Bus Routes	260
Starting Bus Routes From 0x Pins	260
Extending Path Ends to Cover Vias	260
Creating Bus Routes	262
About the Create > Bus Command	262
Bus Routing Across Multiple LPPs	262
Creating New Bus Routes	264
Bus Toolbar and Create Bus Dialog Box Reference	266
Manual Mode	267
Extend Mode	274
Create Bus Shortcut Menu and Shortcut Keys	276
Creating Bus Context-Sensitive Toolbar	280
Extending Existing Bus Routes	282
About the Create Bus Extend Mode	282
Extending Existing Bus Bits	288
Adding Vias During Bus Creation	295
Connecting Bus Bits on the Same Layer	298
Continuing Bus Routes	300
About Continuing Bus Routes	300
Using the Continue Bus Command	300

Contents

Continue Bus Toolbar and Dialog Box	302
Continue Bus Shortcut Menu	303
Compressing and Expanding Bus Routes	303
About Compressing and Expanding Bus Routes	303
Compact Mode	304
Fan-In/Out Mode	305
Ladder Mode	306
Compacting Bus Bits	308
Using the Fan-In/Out Mode	309
Adding Jogs in Bus Bits (Ladder Mode)	311
Compress/Expand Bus Toolbar and Dialog Box Reference	314
Compact Mode	314
Fan-In/Out Mode	315
Ladder Mode	317
Cutting Corners of Bus Routes	319
6. Creating and Editing Vias	320
About Vias	320
Via Description	321
Types of Vias	322
Methods of Adding Vias	323
Via Justification	324
Bypassing the System's Standard Via Parameter Calculations	327
Creating Individual Vias Manually	329
Creating Individual Vias Manually at Specific Locations	329
Via Toolbar and Create Via Dialog Box (Manual Non-stacked Mode)	330
Via Shortcut Menu and Shortcut Keys	335
Creating Individual Vias Automatically	337
About Creating Individual Vias Automatically	337
About the Use Active Object and All Overlapped Areas Options	338
Before You Start	341
Setting Up for Rule Compliance Checks Within the Surrounding Area	341
Setting Up for Automatic Bridge Via Creation	342
Creating Individual Vias Automatically by Object Selection	343
Creating Individual Vias Automatically Within Specified Region	344
Creating Vias in the Intersection Mode	344
Via Toolbar and Create Via Dialog Box Reference (Auto Mode)	347
General Options	348

Contents

Options Tab	349
Rules Tab	357
Layer Table	359
Via Shortcut Menu and Shortcut Keys	361
Creating Arrays of Vias	363
Creating Stacked Vias	364
Creating Stacked Vias Manually Within Shape Boundary	364
Creating Stacked Vias Manually Within Specified Region	367
Creating Stacked Vias Automatically	370
Via Toolbar and Create Via Dialog Box Reference (Manual Stacked Mode) ..	372
Stretching Vias	377
Stretching 1x Shapes in Standard Vias	378
Stretching 1x Shapes in Built-in Custom Vias	381
Replacing Vias	384
About Replacing Vias	385
L-connection	385
T-connection	387
Other Cases	387
Using the Replace Via Command	388
Replace Via Toolbar	388
Adjusting Vias	389
Modifying Via Attributes and Properties	390
Freezing and Unfreezing Vias	395
Freezing and Unfreezing Vias Using the Property Editor	396
Freezing and Unfreezing Vias Using Tcl Commands	398
Creating and Editing Built-In Custom Vias	398
Creating and Deleting Built-In Custom Vias	399
Modifying Built-in Custom Via Attributes and Properties	399
Editing Built-in Custom Vias	400
Chopping Operations That Result in Built-in Custom Vias	400
Healing Built-in Custom Vias	402
Defining Cut LPPs for Built-In Custom Vias	403
 7. Creating and Editing Transmission Lines	 404
About Creating Transmission Lines	404
Creating Transmission Lines	405

Editing Transmission Lines	410
Using Editing Commands	410
Using the Property Editor	410
Create Transmission Line Toolbar and Dialog Box Reference	412
8. Working With Cut Metal Shapes and Erasing Layers	417
Generating and Deleting Cut Metal Shapes	417
About Cut Metal Shapes	417
Before You Start	419
Generating Cut Metal Shapes	420
Deleting Cut Metal Shapes	423
Erasing Layers	424
About Erasing Layers	424
About Hierarchical Erase-Layer Operations	425
Before You Start	427
Defining the eraseLayer Technology Constraint	427
Turning On the leAutoEraseLayer Tcl Preference	428
Setting the leSmartEraseLayer Tcl Preference	428
Setting Up Hierarchical Erase-Layer Operations	428
Defining Feedthrough Layers	428
Setting Up Tcl Functions to Erase Layers in PCells	429
Creating the Erasing Layer User-Defined Tcl Function	429
Setting the leAutoEraseLayer Tcl preference	431
Using the Edit > Other > Erase Layer Command	431
9. Creating Abstract Views From Designs	433
About Abstract Views	433
Requirements	434
Exclusions	436
Objects Supported in Abstract Creation	436
Attributes Used to Create Abstract Views	438
Blockage Shapes	439
Generating Abstract Views From Console	439
Generating Abstract Views From Layout Editor	441
Managing Abstract Generator Option Sets	442
Displaying the Abstract Generator Options Dialog Box	443
Creating New Option Sets	444

Contents

Abstract Configurations Tab	446
Layer Options Tab	453
Saving Option Sets to Files	456
Loading Option Sets From Files	456
Copying Existing Option Sets	457
Modifying Existing Option Sets	457
Removing Option Sets	458
Generating Cellviews With Specific Options to P/G Pins and Signal Pins	458
Extracting Antenna With GDS Mode	460
Setting Run Directory for PVE	461
Running Abstract Generator in Batch Mode	462
Specifying the Routing Layers to Extract	463
10. Reporting and Locating Short Circuits	464
Methods for Reporting and Locating Short Circuits	464
Checking and Reporting Shorts Using the <code>le::checkShorts</code> Tcl Command	465
Tracing Path Segments Using Highlight Path	467
About Highlight Path	467
Ignoring Instances	471
Highlighting Paths	472
Highlight Path Toolbar and Dialog Box	475

About This User Guide

This user guide explains how to use the Custom Compiler™ Layout Editor tool to manually connect objects in design layouts.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® Custom Compiler™

Synopsys Fusion Compiler™

Synopsys IC Compiler™ II

Synopsys IC Compiler

Synopsys IC Validator™

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Understanding Connectivity

This chapter describes various aspects of layout connectivity in the Custom Compiler environment.

The various aspects of physical connectivity are defined and described in the following sections:

- [About the Layout Editor Connectivity Features](#)
- [About Connectivity](#)
- [Enabling the Connectivity Engine](#)
- [Defining and Changing Pin Relationships](#)
- [Using Pins and Terminals for Hierarchical Connections](#)
- [Understanding Flight Lines](#)
- [Working With PCells and Feedthrough Shapes](#)

About the Layout Editor Connectivity Features

You use the Custom Compiler Layout Editor to:

- Create and edit physical designs (layout entry and routing).
- Create and edit advanced node designs. For example, you can assign colors to shapes for multiple patterning.
- Verify the physical layout through the use of design rule checks (DRC), layout versus schematic (LVS) checks, and layout parasitic extraction (LPE) checks.
- Simulate the design.

For detailed information about the Layout Editor and how to create and edit basic objects, see [About the Layout Editor](#).

This guide explains how to use the Layout Editor tool to manually connect objects in design layouts. To automate the connection or routing tasks, see [About the Custom](#)

Router. For verification instructions, see Introduction to Design Checking and Physical Verification. For simulation instructions, see the Setting Up and Invoking the PrimeWave Design Environment.

About Connectivity

The physical realization of a logical net in a layout design is a collection of one or more objects which are laid out in such a way as to make a series of physical connections and complete an electrical path from a source location to one or more target locations.

Connectivity Engine

Understanding how shapes connect can help you troubleshoot issues in your design. This topic describes the Custom Compiler Connectivity Engine.

The Connectivity Engine is a component of the Custom Compiler that:

- Models the connectivity of the physical data of a layout design.
- Propagates connectivity from pins and instance terminals (instTerms).
- Propagates connectivity through 1x instances that have no pin shapes (such as floating M0 without pins).

The M0 layer should be defined as a feedthrough layer in the CDF file. Shapes on the M0 layer should not have a net assigned or should not be connected to netted objects.

- Identifies shorts.
- Synchronizes instance terminals for mustJoin terminals and multi-terminal nets. See [Defining and Changing Pin Relationships](#).
- Provides connectivity anchoring. See [Connectivity Island](#).

Connectivity Concepts

Objects that have the correct electrical properties required to make physical connections as part of a net are said to *physically touch*. Two objects are considered to be physically touching if they satisfy any or all of the following conditions:

- The objects overlap or abut.
- The objects are on the same layer.
- The layer material is not non-propagating.
- The shapes are 0x or 1x pin figures.

Note:

Objects can also connect by overlap with cut shape (for example, metal1 connects to poly via contact). The Connectivity Engine looks at shapes above and below cut layers by maskNumber.

Layer Ordering and Adjacency

Connectivity propagation and touching depend on:

- [Mask Layer Ordering and Adjacency](#)
- [Material-Based Connections](#)
- [Propagating Layer Identification](#)
- [Cut Layer Identification](#)
- [Layer Attribute Configuration](#)

Mask Layer Ordering and Adjacency

The rules for touching (see [Connectivity Concepts](#)) depend on the properties of the layer that the shape is associated with. Those properties include:

- The `oaPhysicalLayer` attributes for the layer:
 - mask number
 - material type
- Custom Compiler environment-defined attributes:
 - Constraints that define the routing layers
 - Groups of layers that are defined to be shorting layers
 - The `nonPropagating` attribute

Material-Based Connections

When connecting layers below the first metal layer, you can turn on the `leMaterialBasedConnection` Tcl preference to consider material types in addition to layer mask numbers. Using material-based connections better reflects manufacturing processes and is therefore optimal.

This topic describes the following:

- [Layers Used for Material-Based Connections](#)
- [Diff - Implant - Well Connections](#)

- [Lowest Cut-Layer Connections](#)
- [Tub-Well Connections](#)
- [IsoTub - IsoTubBulkShort - Well Connections](#)

Layers Used for Material-Based Connections

When `leMaterialBasedConnection` is turned on, the following materials can be used for connections:

- poly
- diffusion, nDiff, pDiff
- nImplant, pImplant
- nWell, pWell
- nTub, pTub
- nIsoTub, pIsoTub
- isoTubBulkShort

The layers are valid routing layers.

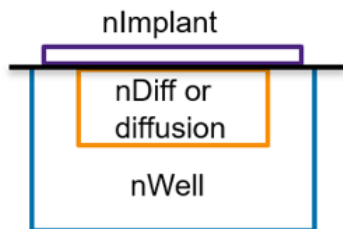
Diff - Implant - Well Connections

When working with guard rings, you typically connect diff layers to well layers. Diff layers can connect to well layers if the overlapping implant layers are of the same layer type (n or p) as the diff or well layers. For example, the following connections would work:

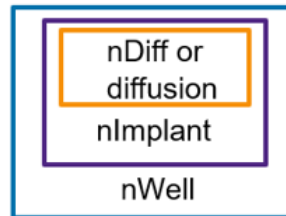
- Diffusion - nImplant - nWell
- Diffusion - pImplant - pWell
- nDiff - nImplant - nWell
- pDiff - pImplant - pWell

The following example shows that the nImplant layer can connect the nDiff and nWell layers successfully. The nImplant layer is of the same type (n) as the nDiff and nWell layers.

Cross-Section View

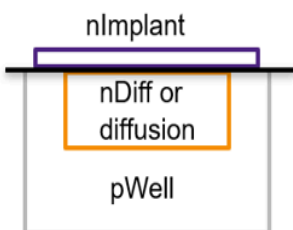


Top View

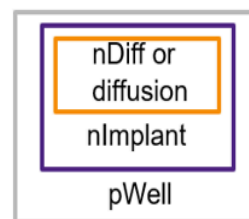


Overlapping implant layers that are of different types would break the connection. In the following examples, the connections are not successful because the nImplant layer is of a different type than the pWell layer.

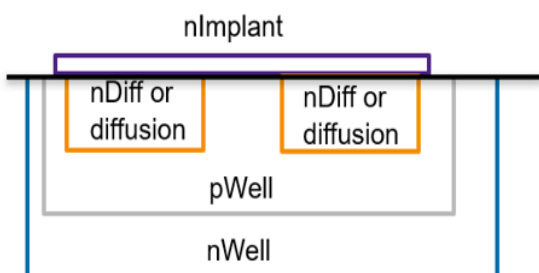
Cross-Section View



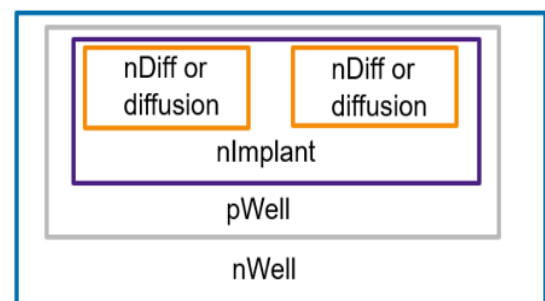
Top View



Cross-Section View



Top View



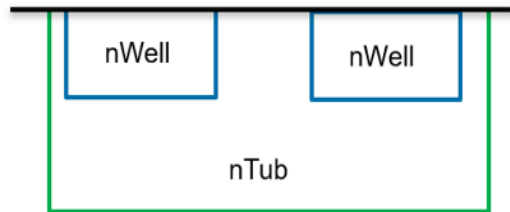
Lowest Cut-Layer Connections

Lowest cut layers can only connect to diff or poly layers, including poly, diffusion, nDiff, and pDiff material types. You cannot connect lowest cut layers to implant or well layers directly.

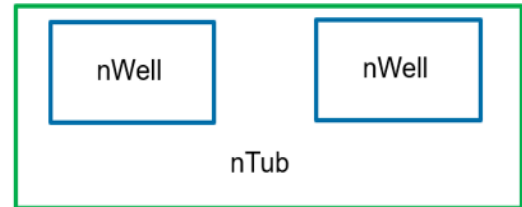
Tub-Well Connections

Tub layers can connect to well layers if they are of the same type (n or p). In the following example, the nTub layer can connect to the nWell layer because they are of the same type (n).

Cross-Section View



Top View



IsoTub - IsoTubBulkShort - Well Connections

Isolating tub layers can form a different type tub to connect with different-type well layers. For example:

- nlsoTub layer can form an isolated p-type tub.

The nlsoTub layer itself is an n-type. However, because it can form an isolated p-type tub, it can connect with all enclosed pWells.

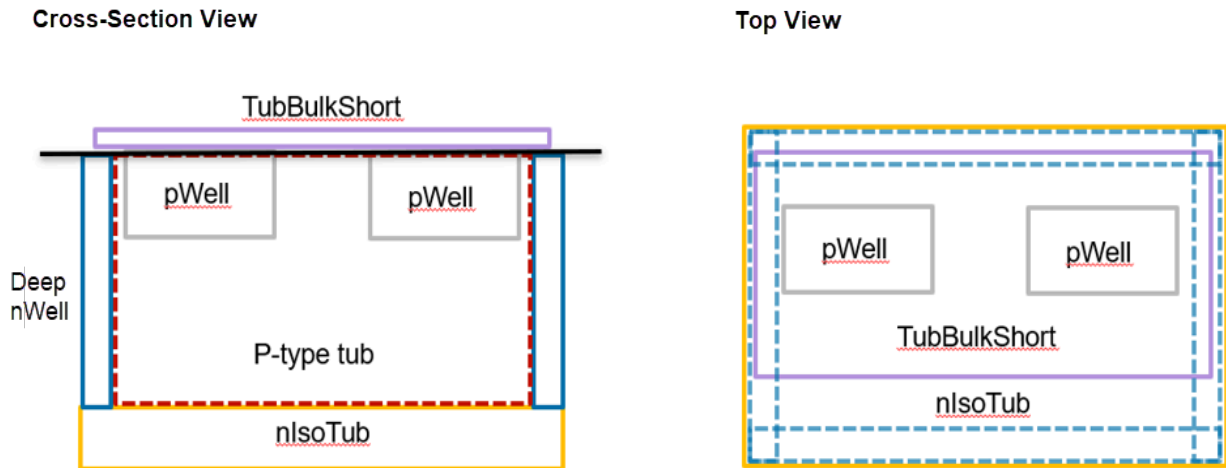
- plsoTub layer can form an isolated n-type tub.

The plsoTub layer itself is a p-type. However, because it can form an isolated n-type tub, it can connect with all enclosed nWells.

Isolating tub layers do not participate in the tub connection themselves because they are of different types than the tubs. To form the connection with well layers, you need the isoTubBulkShort layer.

The isoTubBulkShort layers can connect to:

- pWells if overlapping with nlsoTub. The pWell shape should be contained by isoTubBulkShort and nlsoTub.
- nWell if overlapping with plsoTub. The nWell shape should be contained by isoTubBulkShort and plsoTub.



Propagating Layer Identification

To determine which layers to consider for use in connectivity extraction, the Connectivity Engine searches for a `validRoutingLayers` constraint that applies to the edit design (the top design being analyzed for connectivity). The search looks for the `validRoutingLayers` constraint in the `leCEConstraintGroups` preference, which is a space separated list of constraint group names. The default value for `leCEConstraintGroups` is `leCEConstraintGroups virtuosoDefaultExtractorSetup`.

Cut Layer Identification

The Connectivity Engine looks for the `validRoutingLayers` constraint first in the edit design. If the constraint is not found, the search continues in the technology file.

- When a `validRoutingLayers` constraint is found
 - All layers listed in that constraint, and all existing layers with a *material type* of "cut" are used in the connectivity analysis. However, layers of type `cut` in the `validRoutingLayers` constraint are ignored.
 - The `leNonPropagatingMaterials` preference is ignored.
- If a `validRoutingLayers` constraint is not found, then the Connectivity Engine uses all existing layers whose material type is not listed in the `leNonPropagatingLayerMaterials` preference.

Layer Attribute Configuration

The following table shows an example configuration of layer attributes in a technology file. The ordering of the layers is the order of their mask number.

Applying the rules for touching (see [Connectivity Concepts](#)), the cut layers divide the other layers into groups of layers between a given pair of cut layers, or groups of layers below

the lowest cut layer. For example, the table shows a single metal layer between cont and via1, and a single metal layer between via1 and via2, but the entire set of layers with mask numbers under 10 will be grouped together.

Table 1 *Layer Attributes in a Technology File*

Layer Name	Material	Mask Number	Propagating
nWell	nWell	1	Y
pWell	pWell	2	Y
nDiff	nDiff	3	Y
pDiff	pDiff	4	Y
nPlus	nImplant	5	Y
pPlus	pImplant	6	Y
poly	poly	7	Y
cont	cut	10	Y
metal1	metal	12	Y
via1	cut	15	Y
metal2	metal	16	Y
via2	cut	20	Y
metal3	metal	22	Y
passivation	passivationCut	46	N

Propagating Connectivity

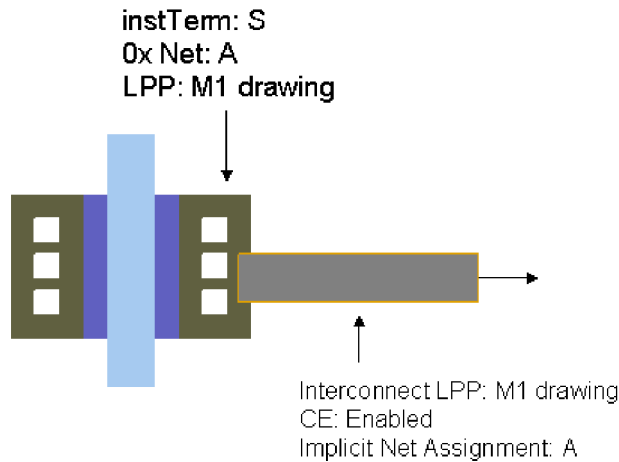
There are two ways to propagate connectivity to an object:

- [Implicit Net Assignments](#)
- [Explicit Net Assignments](#)

Implicit Net Assignments

You do not have to explicitly assign a net name to every new object. Instead, you can let the unassigned object inherit a net name through a physical connection to an assigned object on the same layer.

For example, if you place the first point of a new path on a terminal that is on the same layer, and the terminal already has a net name, then the path inherits the net name for as long as the two objects physically touch. If the two objects become separated, then the path object would again become unassigned.



Note:

If you make a connection that creates a net name conflict while drawing an object, the tool marks the connection as a short. See [Short Circuits](#).

When the object you are drawing inherits a net name, the inherited name appears in the **Net Names** box of the command options toolbar. You cannot edit the toolbar to change an inherited name. If an object inherits the wrong name, remove the point that caused the inheritance and then type the correct name in the **Net Names** box. To change net names on existing objects, use the Property Editor.

Note:

When the Connectivity Engine option is disabled, objects cannot inherit net names. If that were allowed, you could accidentally create a short by specifying the wrong net name for a new object that would have inherited the correct net name.

Explicit Net Assignments

If you assign a net name to an object, then the object retains the net name even when physically touching other assigned objects.

Connectivity Anchors and Islands

The persistence of logical connectivity is governed by connectivity anchors.

This topic describes the following:

- [Connectivity Anchor](#)
- [Connectivity Island](#)

Connectivity Anchor

A *connectivity anchor* is any object that has been explicitly assigned to a net. Do not add anchor automatically.

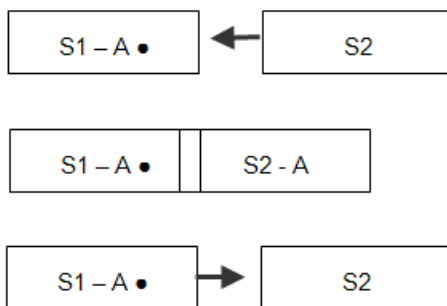
For example,

- If you create a rectangle and specify a net name, then the object is automatically anchored to the specified net. However, if you later delete the net, then the object becomes unnetted and will no longer be an anchor.
- Pin shapes are always created with a terminal, which requires a net, so pin shapes are always connectivity anchors.

When an object is anchored, it cannot be moved to another net. Anchoring is controlled by the Anchor attribute, which you can edit in the Connectivity region of the Property Editor assistant.

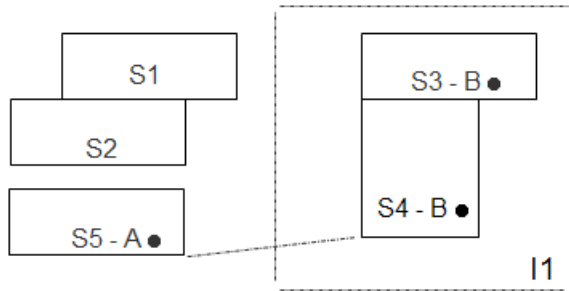
When the Connectivity Engine is enabled, any object that physically touches an anchored object becomes anchored itself, and remains anchored until the physical connection is broken.

The following figure shows an explicitly named object (S1 - A), which is anchored to net A, and an unnetted object (S2). If S2 were to overlap S1 - A, it would become part of net A (S2 - A). Similarly, if you were to then move S2 away, it would become unnetted again.

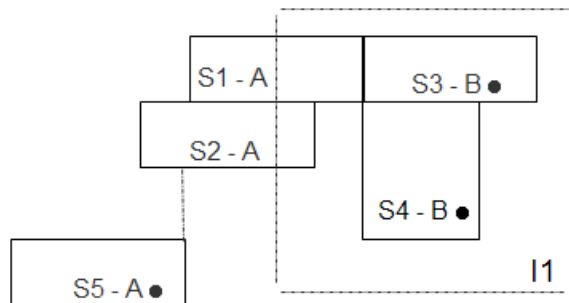


The following three figures demonstrate propagation from a lower level of hierarchy. It is assumed that the lower level objects are `oaPinFig` with `hasPin=1` and `oaInstTerm` is on `oaNet` at `0x`.

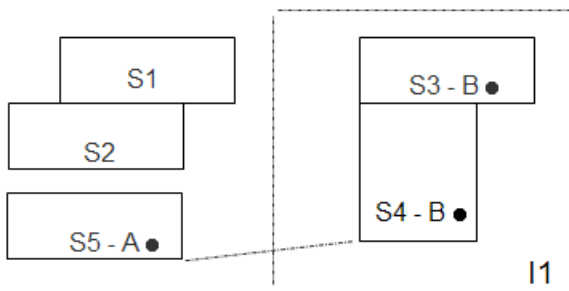
In the first figure, 1x level net B, which exists inside instance I1, contains a terminal for which the instTerm at 0x has been assigned to connect to 0x level net A, as shown by the flight line.



In the following figure, object S1 overlaps object S3, resulting in implicit net propagation.



In the following figure, the overlap is eliminated, resulting in the implicit net assignment being removed.



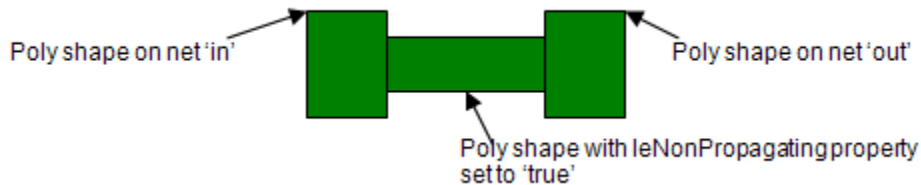
Connectivity Island

A *connectivity island* consists of a logical grouping of objects (such as shapes), on the same net, that are connected, either implicitly or explicitly. If a net has more than one island, then there is a connection missing.

Check connectivity island status before doing operations where connectivity information is needed.

Non-propagating Layers

You can control the connectivity properties of interconnect layers by designating them in the technology file as non-propagating. Typically, this is how technologies handle bulk layers.



Short Circuits

A short circuit is an illegal physical connection between two or more objects on different nets. No short occurs when a netted [Connectivity Island](#) and un-netted connectivity island overlap.

This section describes the conditions under which short circuits occur and how you can detect and fix the short circuits:

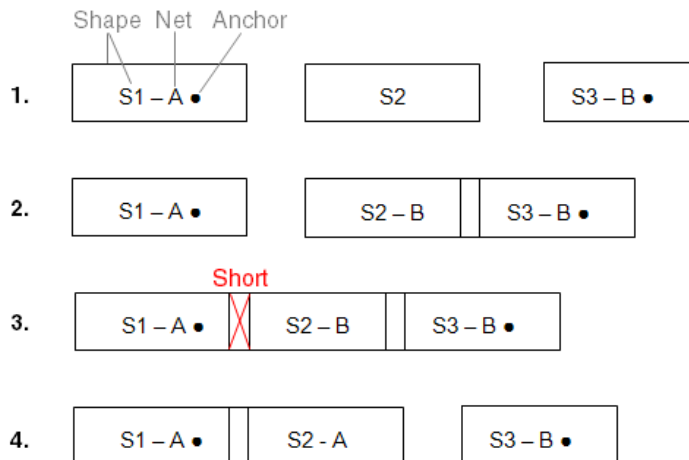
- [Short Circuit at the Top Level \(0x\) of the Design](#)
- [Short Circuit Through the Hierarchy of the Design](#)

To report and locate short circuits in your design, see [Reporting and Locating Short Circuits](#).

Short Circuit at the Top Level (0x) of the Design

If two 0x objects on different nets physically touch each other, the result is a short. The following example demonstrates this concept:

1. Three objects are created: S1, S2, and S3.
 - S1 is anchored to net A.
 - S2 is not assigned a net.
 - S3 is anchored to net B.
2. When S2 overlaps S3-B, it is assigned to net B.
3. If S2 then overlaps S1-A, a short results.
4. If S3-B is moved, S2 is assigned to net A (S2-A).



Short Circuit Through the Hierarchy of the Design

When checking for short circuits across the design hierarchy, the Layout Editor looks at overlapping objects at the top level to see if the objects are on the same effective net. No short circuit occurs when:

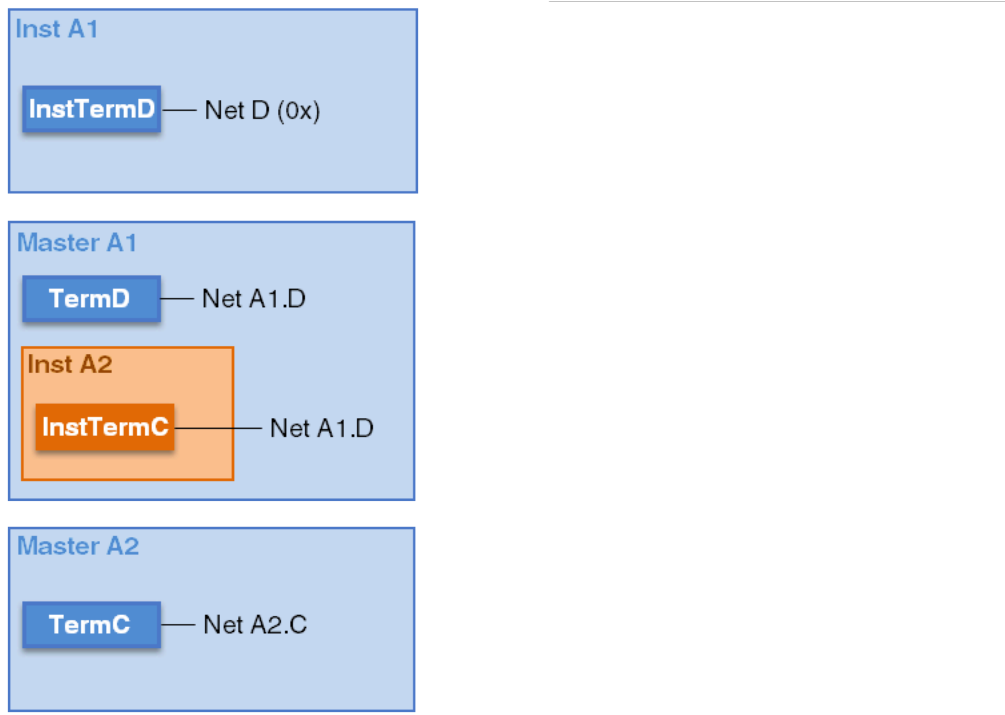
- Two connection islands on the same net overlap.
- A netted connection island overlaps with a non-netted connection island.

The following conditions cause short circuits in hierarchical designs:

- Netted 0x object touching a lower-level object that:
 - Is on an instTerm that propagates up to 0x with a different 0x net than the 0x object.
 - Is on a net with no terminal and thus no instTerm that propagates up to 0x.
 - Has no net.
- Unnetted 0x object touching a lower-level object that:
 - Is on a net with no instTerm that propagates up to 0x.
 - Has no net assigned at the level in which it resides.
- Any touching between objects in different instances is a short unless both objects are on nets that have instTerms that propagate all the way up to the lowest common parent of the instances containing the objects, on the same net in that parent.
- Shorts across the hierarchy which have a common parent at a lower level of hierarchy will only be detected and marked in the common parent. This avoids duplication of short markers in many designs due to nesting of the hierarchy.

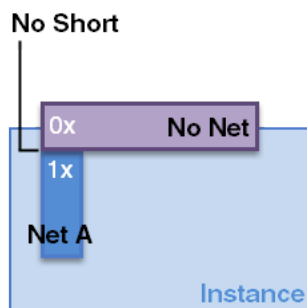
To determine whether a short circuit would occur in a hierarchical design, the Layout Editor traces the connectivity to ascertain the effective net relative to the top-level design. In the following example, the Layout Editor would trace the connection from TermC on net A2.C as follows:

Net A.2.C > TermC > instTermC > Net A1.D > TermD > instTermD > Net D



Example 1:

- The 0x shape is not assigned a net.
- The 1x shape that is within an instance is on net A.
- When the 0x object touches the 1x object, no short occurs.
- No connectivity is propagated to the 0x shape, because the instance does not contain a terminal or an instTerm.



If the instance contains an instTerm that has a net assignment, then the net assignment will be propagated to the 0x shape.

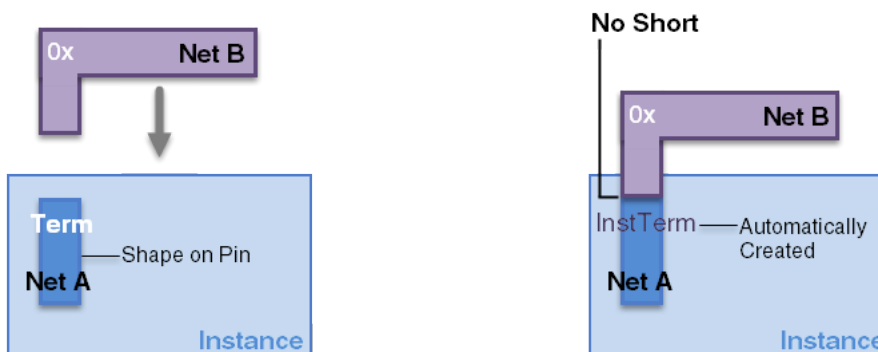


If:

- The instance contains a shape on a pin that corresponds to a terminal that is on net A.
- The 0x shape is on net B.

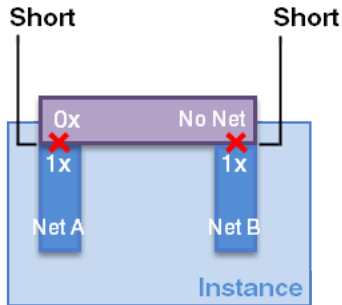
Then:

- When the 0x shape touches the shape in the instance, an instTerm is automatically created.
- No short occurs because the two shapes are effectively on the same net island (B) relative to the top-level design.



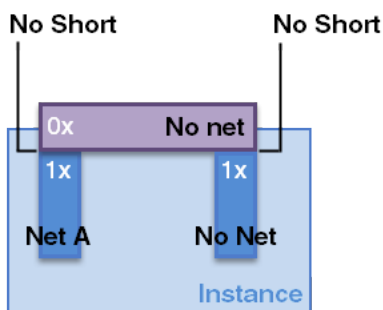
Example 2:

- The 0x shape is not assigned a net.
- The two 1x shapes are within an instance, and each shape is assigned a different net (A and B).
- When the 0x shape touches the two 1x shapes, a short occurs at both abutment locations.



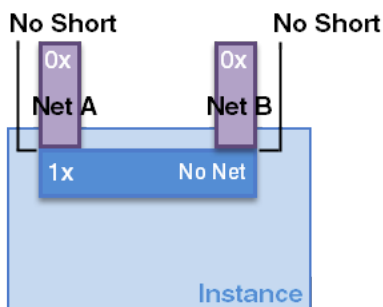
Example 3:

- The 0x shape is not assigned a net.
- One of the two 1x shapes in the instance is on net A. The second 1x shape is not assigned a net.
- When the 0x shape touches the two 1x shapes, no short occurs.



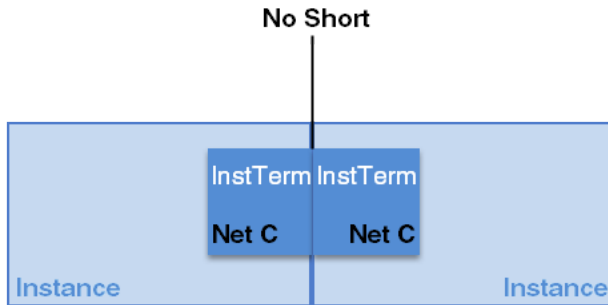
Example 4:

- The two 0x shapes are on different nets (A and B).
- The 1x shape in the instance is not assigned a net.
- When the two 0x shapes touch the 1x shape, no short occurs.



Example 5:

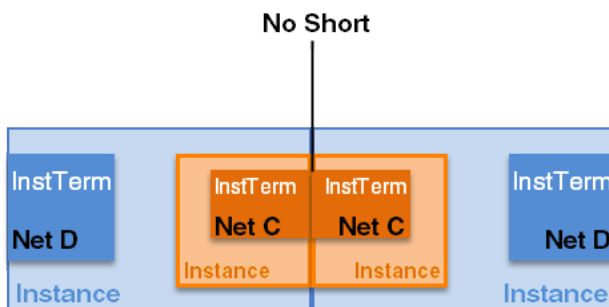
- Each of the two instances contain an instTerm that is on top-level net C.
- When the two instances touch as shown, no short occurs.



Example 6:

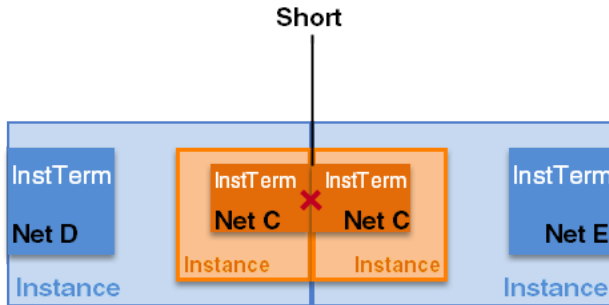
- The two blue instTerms are on top-level net D.
- The two orange instTerms are on net C.
- When the two orange instTerms touch, no short occurs because they are effectively on the same net island (D) relative to the top-level design.

Effective net island is determined by tracing the logical connection between net C and top-level net D. To prevent a short in the example shown, the trace must include corresponding terminals for the instTerms on net C and the instTerms on top-level net D.



Example 7:

- The two blue instTerms are on different top-level nets (D and E).
- The two orange instTerms are on net C.
- When the two orange instTerms touch, a short occurs because they are effectively on different net islands (D and E) relative to the top-level design.



Enabling the Connectivity Engine

The Connectivity Engine provides specific connectivity services in design layouts. See [About Connectivity](#).

To enable the Connectivity Engine:

1. Select **Options > Design**.

The **Layout Design Options** dialog box opens.

2. Select the **Connectivity** tab.
3. Select the **Connectivity Engine** option.

Note:

When **Connectivity Engine** is disabled, objects cannot inherit net names. If that were allowed, you could accidentally create a short by specifying the wrong net name for a new object that would have inherited the correct net name.

Defining and Changing Pin Relationships

This topic describes the following:

- [Types of Pin Relationships](#)
- [Changing Pin Relationships](#)
- [How the Pin Relationship Command Affects Pins and Terminals](#)

Types of Pin Relationships

Pin relationships (or pin connections) describe how instance pins must be connected to meet the electrical requirements of the instance. The types of pin relationships are:

Type of Pin Relationship	Description
Weak	Multiple pins are associated with a single instance terminal, but only one pin needs to be connected for all pins to be considered fully connected. Pins that have weak relationships cannot provide connectivity in the parent design. You use weak pins to connect routes of high resistance within the instance. Do not connect weak pins using load and driver routes unless the pins are explicitly connected by a wire in the parent.
Strong	Multiple pin shapes are on a single pin, but only one pin shape is required to be connected in the parent to connect the net hierarchically into the instance. Strong pin connections provide multiple valid hierarchical connections and are the most common connection type. You use strong connection types for feedthroughs.
Must (Must Join)	Multiple terminals on an instance must all be connected in the parent as a single net. Must Join connections indicate that the internal block is incomplete, and you must connect the terminals at the hierarchy level above the block. Use this connection type when layout constraints prevent you from completing the connections within the instance, and electrical constraints require loads within the instance to be connected to an external driver using more than one wire.

You can use the **Change Pin Relationship** command to change the existing connection relationship among pin shapes. Because the command only changes existing connection relationships, you can safely select multiple sets of unconnected pin shapes (see [Strong](#) pin relationships).

See [Using Pins and Terminals for Hierarchical Connections](#) for more information about pin relationships.

Note:

If you select any shape on a pin with multiple shapes, the tool automatically selects all shapes on the pin. Completing the command changes the connection relationship for all of the shapes on the pin.

Caution:

While this command is active, all pins are selectable and therefore, regardless of the settings in the Object/Layer Panel, they could be modified if selected. After the command completes, the selection setting of the pins reverts to the settings in the Object/Layer Panel.

The pin access direction is determined as follows:

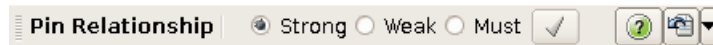
- For modified existing pins, the pin access direction is the logical AND of the access directions of all previously existing pins in the connection relationship.
- For new pins, the pin access direction is the logical OR of the access directions of all selected pin shapes on the same terminal or must-join terminals.

New terminals inherit their type from the original terminals.

Changing Pin Relationships

To change the relationship among a set of pins:

1. Choose **Edit > Pin > Change Pin Relationship**. The **Pin Relationship** toolbar opens.



2. Select a pin relationship: **Strong**, **Weak**, or **Must**. See [Types of Pin Relationships](#).
3. If you did not preselect suitable groups of pin shapes, use the mouse to select them now.
4. Set the new relationship value.
5. Click **Apply**

For information on using the `ile::changePinRelationship` and `le::changePinRelationship` Tcl commands, see “Layout Editing Tcl” in the *Custom Compiler Tcl Reference Manual*.

How the Pin Relationship Command Affects Pins and Terminals

This section describes how the **Pin Relationship** command impacts existing pin and terminal relationships:

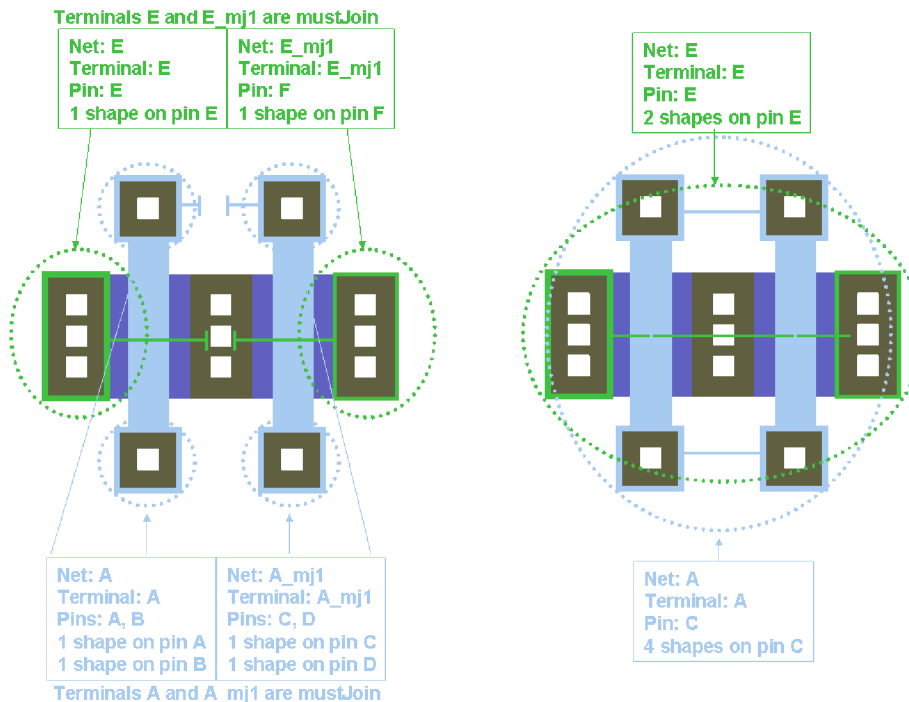
- [Strong](#)
- [Weak](#)
- [Must Join](#)

Strong

If **Connection** is set to **Strong**, and the selected shapes belong to different weakly-connected pins and must-join terminals:

- The tool removes all but one of the weakly-connected pins.
- All shapes involved in the previous weak connection become shapes on the remaining pin.
- The tool removes all but one of the must-join terminals.
- Shapes from pins on the original must-join terminals become shapes on the appropriate pin object of the remaining terminal.

The following diagram shows the concept of changing weak or must-join pin groups to strongly-connected groups.

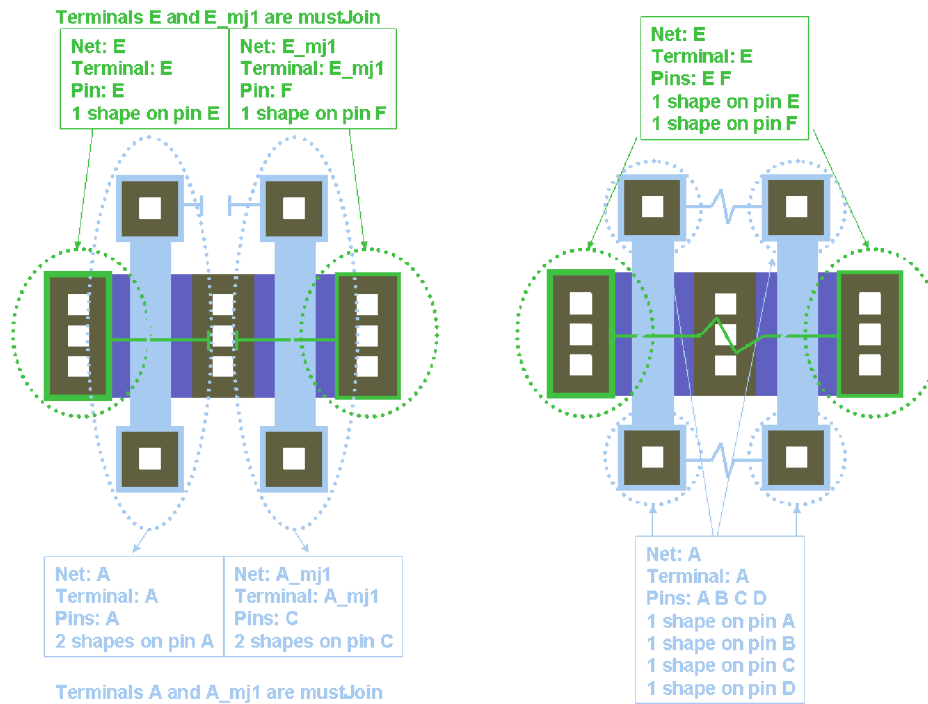


Weak

If **Connection** is set to **Weak**:

- Strongly-connected shapes on the same pin become shapes with new, unique pins.
- The tool removes all but one of the must-join terminals.
- Pins on the original must-join terminals become pins of the remaining terminal.

The following diagram shows the concept of changing strong and must-join pin groups to weak connections.

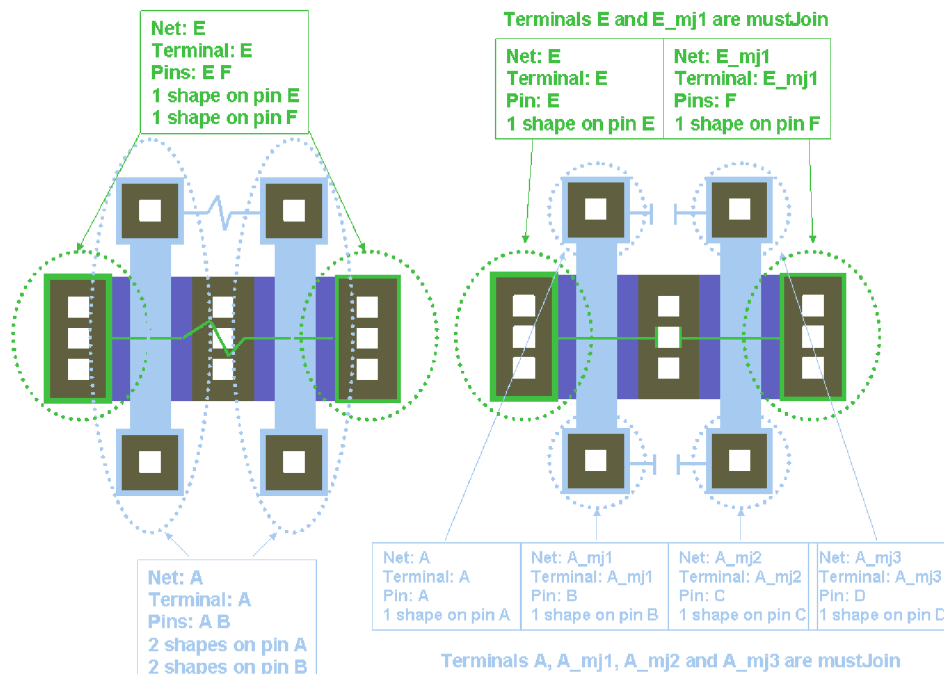


Must Join

If **Connection** is set to **Must Join**:

- The tool preserves the terminal and net of one of the selected pin shapes.
- The tool creates a new pin, terminal, and net for each remaining pin shape.
- The new terminals have a must-join relationship.

The following diagram shows the concept of changing strong and weak pin groups to must-join connections.



Using Pins and Terminals for Hierarchical Connections

This topic describes the following:

- [Pins and Terminals in Hierarchical Instances](#)
- [Pin Connection Relationships](#)

Pins and Terminals in Hierarchical Instances

Hierarchical instances use pins and terminals to:

- Specify the allowed connection points
- Alias higher-level net names to lower-level names

Automatic routing tools use this information to make connections to the instances.

When you create an instance master, there are two ways to specify the allowed net connection locations to placed instances:

- By creating terminals, you make all shapes and instance terminals on the net available as connection points.
- Alternatively, you can create pin shapes to define specific net connection points for terminals.

When you place an instance, the terminals are exposed for connections, but the terminal name remains undefined. Making a connection to the terminal sets the terminal name; there is no requirement or expectation that the net name at the current level of hierarchy matches the net name inside the master.

While creating a design, it is generally a good practice to define net names, terminals, and pins in this order:

1. Net names
2. Terminals
3. Pins

Note:

The manual routing commands (Create Path and Create Interconnect) ignore terminal types.

Pin Connection Relationships

Pins in the Custom Compiler environment may share the three standard connection relationships:

- [Strongly Connected](#)
- [Weakly Connected](#)
- [Must Join](#)

By default, pins of the same shape share a *weakly-connected* relationship. The tool does not try to automatically assign a connection relationship based on the layer type (metal or high-resistance). You must explicitly specify any required *strongly-connected* or *must-join* relationships among pins.

To change pin connection relationships, see [Defining and Changing Pin Relationships](#). For information of examples, see [Examples of Pin Connection Relationships](#).

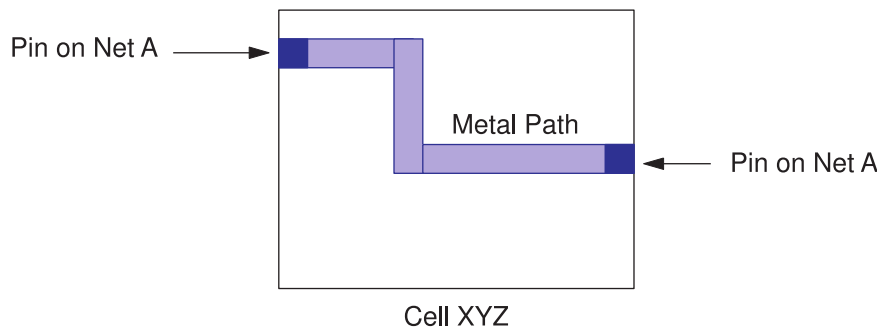
Strongly Connected

Routers are free to use two pins in a strongly-connected group as a pass-through. Strongly-connected pins are typically connected inside the instance by a low-resistance layer.

For example, you might choose to define a pair of pins on the same Metal1 shape as strongly-connected. More generally, when there are several pins on a single net of an instance and the design only requires a connection to one of those pins, they could share a strongly-connected relationship. Any net designed to have multiple valid hierarchical connection points on low-resistance layers should have a group of strongly-connected pins.

The OpenAccess database represents groups of strongly-connected pins as multiple pin shapes (pin figures) on a single pin.

The following diagram shows an example of a metal path through a cell that provides an appropriate feedthrough for automatic routing. The two pins should have a strongly-connected relationship.

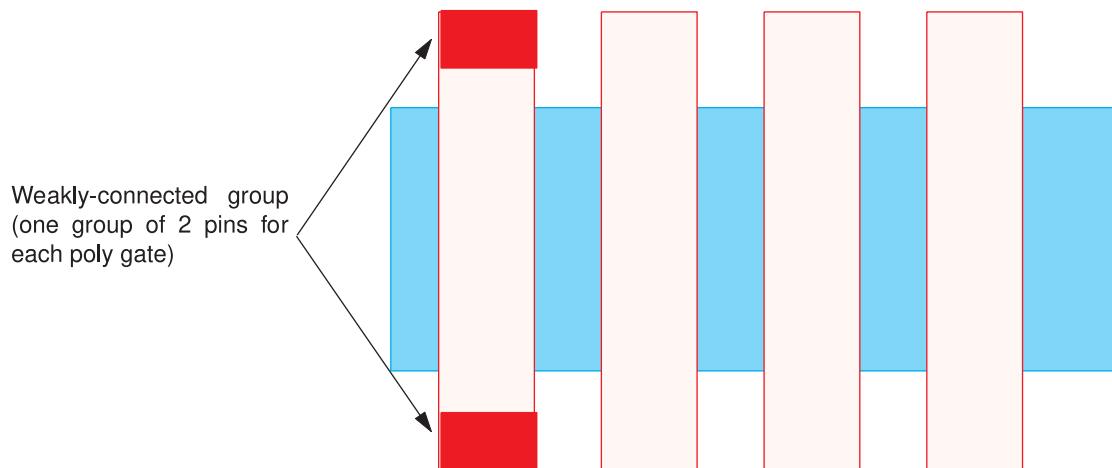


Weakly Connected

Routers should not use two pins in a weakly-connected group as a pass-through. Weakly-connected pins are typically connected inside the instance by a high-resistance layer, such as polysilicon. The two pins on a polysilicon transistor gate are a typical case where you would define a weakly-connected pin group. More generally, if you have two or more pins on a single net of an instance that cannot be used for connections at a higher level, you should define a weakly-connected group for those pins.

The OpenAccess database represents weakly-connected pin groups as multiple pins on a single logical terminal.

The following diagram shows an example of a folded transistor. The pins on each poly gate should have a weakly-connected relationship because, although the pins are on the same net, the automatic router should not use the gate as a feedthrough.

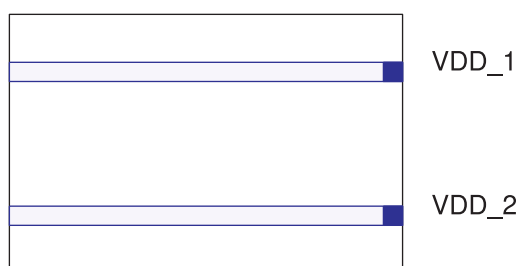


Must Join

Must-join terminals represent several pins on different nets in an instance that must be connected at a higher level of the design hierarchy. Two common situations that require a must-join relationship are:

- Layout constraints that prevent the completion of a required connection inside an instance
- Electrical constraints such as electromigration tolerance or power requirements, which demand that the loads within the instance be connected to an external driver with more than one connection

For example, a cell with multiple power and ground rails might specify that the pins on the supply rails form a must-join group. The following diagram shows an example of two power rails on separate nets that the automatic router must connect together at a higher level.

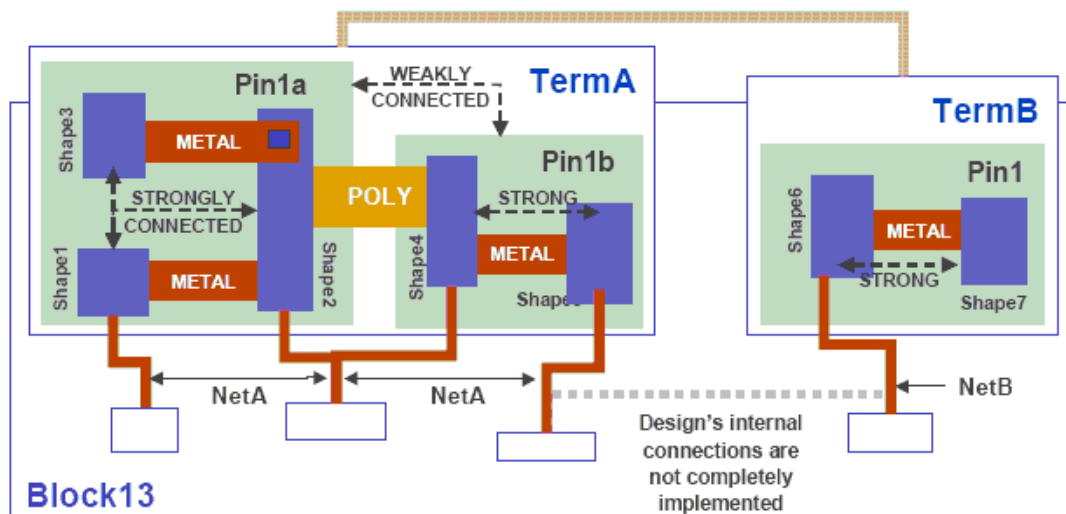


The OpenAccess database represents must-join relationships as a relationship among several terminals. Two cannot exist on the same net or equivalent nets. Consequently, a pair of must-join pins must be on two different non-equivalent nets.

Two terminals in a must-join relationship might contain pins and pin shapes which have weak or strong relationships.

Examples of Pin Connection Relationships

The pin connection relationships needed to properly constraint a router include three must-connect groups, each containing two weakly-connected pins having one shape. The following diagram shows examples of all three types of pin relationships.



In the example, shapes 1, 2, and 3 are strongly-connected pin shapes in the same pin, Pin1a. Connecting NetA to Shape1 makes a valid connection to all of the other pin shapes within the pin.

Pin1a is weakly-connected to pin1b. If you connect a driver to the segment of NetA that connects to Shape1 and connect a load to the segment of netA on Shape5, the segment of NetA that connects Shape2 and Shape4 is necessary to make a proper connection, because the poly link between Pin1a and Pin1b cannot carry the load.

TermA shares a must-join relationship with TermB. The designer could not connect NetB with any part of NetA within Block13, so the designer passed the requirement up through two terminals on distinct nets to any parent design that instantiates Block13.

To change pin connection relationships, see [Defining and Changing Pin Relationships](#).

Understanding Flight Lines

This topic describes the following:

- [About Flight Lines](#)
- [Flight Lines and Connectivity Extraction](#)
- [Displaying and Hiding Flight Lines](#)

About Flight Lines

The Layout Editor displays flight lines to identify required connections between physical objects on the same net. There are two types of flight lines: static and dynamic.

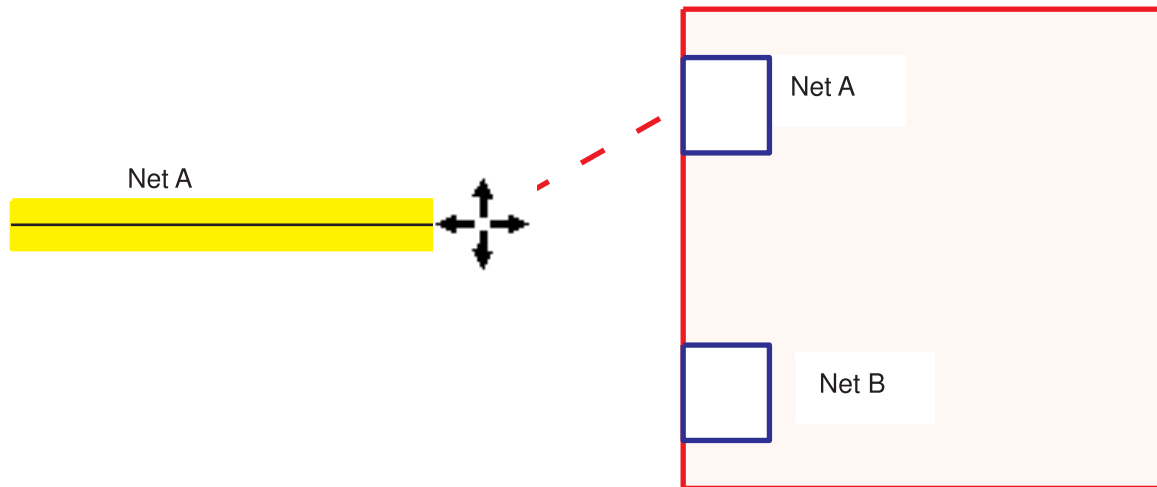
- Static flight lines appear between objects on any net that is not being actively edited.
- Dynamic flight lines appear between objects being interactively edited.

Flight line display is dynamic and configurable. There are four ways to set Flight line display preferences:

- Globally in your startup files, to control display for all designs that you edit during a Custom Compiler session
- Individually for each design
- On individual nets or groups of nets in each design
- Separately for nets you are manipulating with the active command (dynamic Flight lines), to help distinguish them from all of the other nets in the design (static Flight lines)

During object creation and object editing, as you move the mouse, the tool continuously updates the dynamic flight lines for all nets that the command affects. If you are dragging an object outline on the cursor (for example, moving a polygon or drawing a path) a flight line shows the closest incomplete connection. This feature is particularly helpful while you are drawing interconnect.

The following example shows a flight line between the end of a partially completed path and the nearest unconnected object on the same net: in this case, the pin to which this path must connect.



Flight Lines and Connectivity Extraction

The Layout Editor needs connectivity information to display and update flight lines. This creates some dependencies between your connectivity extraction preferences and flight line display:

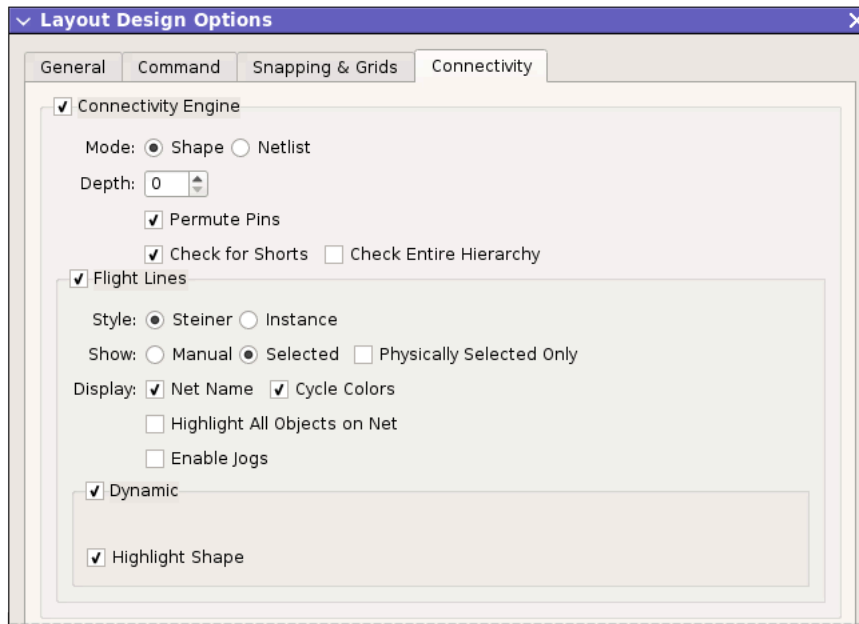
- If you completely disable connectivity extraction, you cannot display any flight lines.
- If you disable connectivity extraction for specific nets, you cannot display flight lines for those nets.
- If you do not extract connectivity for the complete design hierarchy, you might see misleading flight lines for nets that are connected at lower, unextracted levels.

Displaying and Hiding Flight Lines

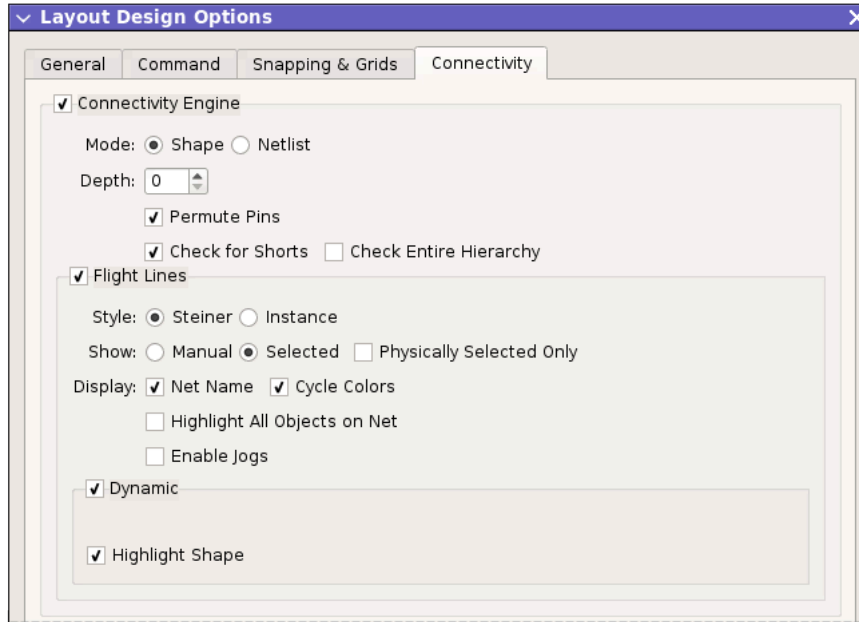
To display flight lines, you must first set options to specify how the flight lines are displayed. After the options are set, you use the **Query > Flight Lines** selections to show or hide the flight lines.

To set up flight line display options:

1. Choose **Options > Design**. The **Layout Design Options** dialog box opens.



2. Click the **Connectivity** tab and set the following options:
 - a. Select **Flight Lines**.
 - b. Select the **Flight Line** options. See Connectivity Tab for details.
 - c. Click **OK** to apply the changes globally and close the dialog box.



3. Display the Design Navigator, and click **Set Flight Line** , and then choose the flight line display setting. See Setting Flight Line Display Options.

4. To hide flight lines, see Showing and Hiding Flight Lines From the Canvas.

To display all flight lines in your design, choose **Query > Flight Lines** and select **Enable**. To hide all flight lines, choose **Query > Flight Lines** and clear the **Enable** check box.

To display flight lines of pin connections, choose **Query > Flight Lines** and select **Pin Connection**. To hide the pin connection flight lines, choose **Query > Flight Lines** and clear the **Pin Connection** check box.

Working With PCells and Feedthrough Shapes

You can use the `leUseLocalNetShapeAsFeedthrough` Tcl preference to control whether to treat shapes in PCells as feedthrough shapes.

The settings for `leUseLocalNetShapeAsFeedthrough` is as follows:

- `true`: Treats shapes with local nets in PCells as a feedthrough shapes.
- `false`: Treats only unnetted shapes in PCells as feedthrough shapes.

2

Controlling Route Placement

This chapter describes how to control the placement of route shapes in the designs.

This chapter describes the following:

- [Ways to Control Route Placement](#)
- [Creating and Editing Blockages](#)
- [Creating and Editing Route Corridors](#)
- [Creating and Editing Track Region PCells](#)
- [Crossing Track Regions](#)
- [Crossing Constraint Group Regions](#)
- [Setting Up Track Grids](#)

Ways to Control Route Placement

Controlling route placement can help you meet various application requirements. For example, you can add route corridors to ensure that time-critical nets have priority through optimal routes.

You can use the following methods to control route placement:

Method	Description
Blockage	Prevents certain objects from being placed on specific layers or within the defined regions. See Creating and Editing Blockages .
Route corridor	Restricts routes that can be placed within defined areas. See Creating and Editing Route Corridors .
Tracks	Ensures that routes are placed correctly. Tracks can be especially helpful in FinFET designs. See Creating and Editing Track Region PCells , Crossing Track Regions , and Setting Up Track Grids .

Creating and Editing Blockages

In the Custom Compiler environment, you use the **Create > Blockage** command to create pin, routing, placement, wiring, fill, slot, screen, or feedthrough blockages.

This topic describes the following:

- [About Blockages](#)
- [Creating Blockages](#)
- [Editing Blockages](#)
- [Blockage Toolbar and Create Blockage Dialog Box Reference](#)
- [Blockage Shortcut Menu and Shortcut Keys](#)

About Blockages

Blockages are used to prevent certain objects from being placed on specific layers or within defined regions. You use the **Create > Blockage** command to create pin, routing, placement, wiring, fill, slot, screen, or feedthrough blockages.

Note:

You can use other commands to create shapes on blockage layers or purposes, but the tool cannot recognize those shapes as blockages. To create blockages, you must use the **Create > Blockage** command.

Note:

The Layout Editor does not support creation of halo blockages (oaAreaHalo objects), and ignores them during most operations. The only exception is abstract creation: if a halo boundary is attached to the prBoundary layer of the design, the tool copies the boundary to the abstract.

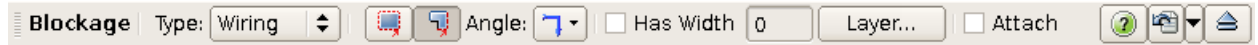
You can optionally attach blockages to a parent object, in which case editing the parent object with commands such as **Move** or **Delete** also affects the blockage.

Creating Blockages

You use the **Create > Blockage** command to create pin, routing, placement, wiring, fill, slot, screen, or feedthrough blockages in layouts.

To create a blockage:

1. Choose **Create > Blockage**. The Blockage toolbar appears.



2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Create Blockage** dialog box. The dialog box contains additional options, such as the **Color** option.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

3. On the toolbar or in the dialog box:
 - Choose the **Type** of blockage you want to create. For example, choose Routing to prevent all routing shapes (path, path segments, and vias) from being placed on the specified layer or within the defined region.
 - Specify the **Layer** that is blocked. For example, if you do not want any routing shape on m8, specify m8.
4. (Optional) Set the remaining options on the toolbar or the dialog box. See [Blockage Toolbar and Create Blockage Dialog Box Reference](#).
5. On the layout canvas, draw the rectangular or polygonal blockage shape. For polygonal blockages, double-click to enter the last point.
6. If you selected **Attach** in [Step 4](#), select the parent object (instance) on the layout canvas.
7. When you are finished, press Esc to quit the command.

See Also

- [About Blockages](#)
- [Editing Blockages](#)
- [Blockage Toolbar and Create Blockage Dialog Box Reference](#)
- [Blockage Shortcut Menu and Shortcut Keys](#)

Editing Blockages

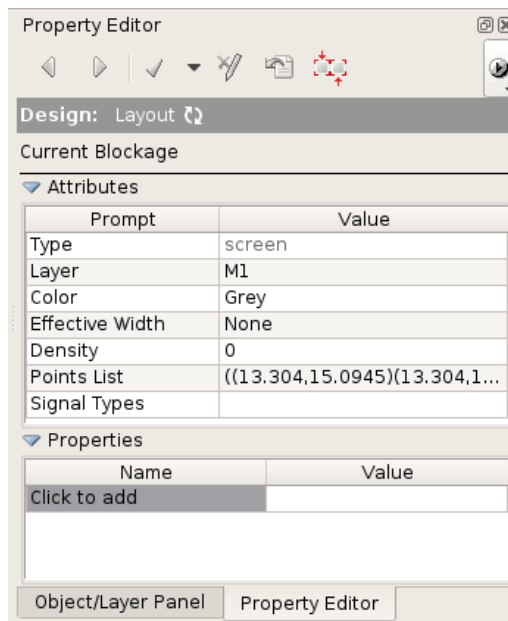
You use the Property Editor to edit pin, routing, placement, wiring, fill, slot, screen, or feedthrough blockages in layouts.

To edit blockage properties:

1. Display the Property Editor.
2. On the canvas, locate and select the blockage you want to modify. The **Attributes** of the selected blockage appear in the Property Editor.

The list of attributes vary by blockage type:

- Placement blockages only have the Points List attribute.
- Only Fill and Screen blockages have the Density attribute.



3. View or modify the Blockage attributes:

Attribute	Description
Type	(Read-only) Type of blockage. The type was specified when you created the blockage. See Creating Blockages .
Layer	Layer for which the blockage applies. Choose the layer from the list.

Attribute	Description
Color	Color of the blockage. Choose the color from the list: ◦ Multi: DRD will skip Multi blockages during design-rule checking. ◦ Black: Colored shapes will observe <code>minSpacing</code> rules when approaching Black blockages. ◦ Grey: Ordinary blockages. The color attributes of Grey blockages are ignored, and other rules, such as <code>minSpacing</code>, are observed. ◦ <color_n>: <i>Color_<n></i> shapes must observe <code>minSpacing</code> rules when approaching <i><color_n></i> blockages.
Effective Width	Width of the blockage.
Density	(Fill and Screen blockages only) Percent of the blockage area that can be used by nets. Specify a percent between 0 and 100, inclusive. Default: 0
Points List	List of coordinates that specify the vertices of the block. To change the vertices, click  and update the coordinates in the dialog box.
Signal Types	Signal type for which the blockage applies: ◦ Analog ◦ Clock ◦ Ground ◦ Power ◦ Reset ◦ Scan ◦ Signal ◦ TieHi ◦ TieLo ◦ Tieoff

4. When you are finished, click **Apply Changes** () to save the modifications.

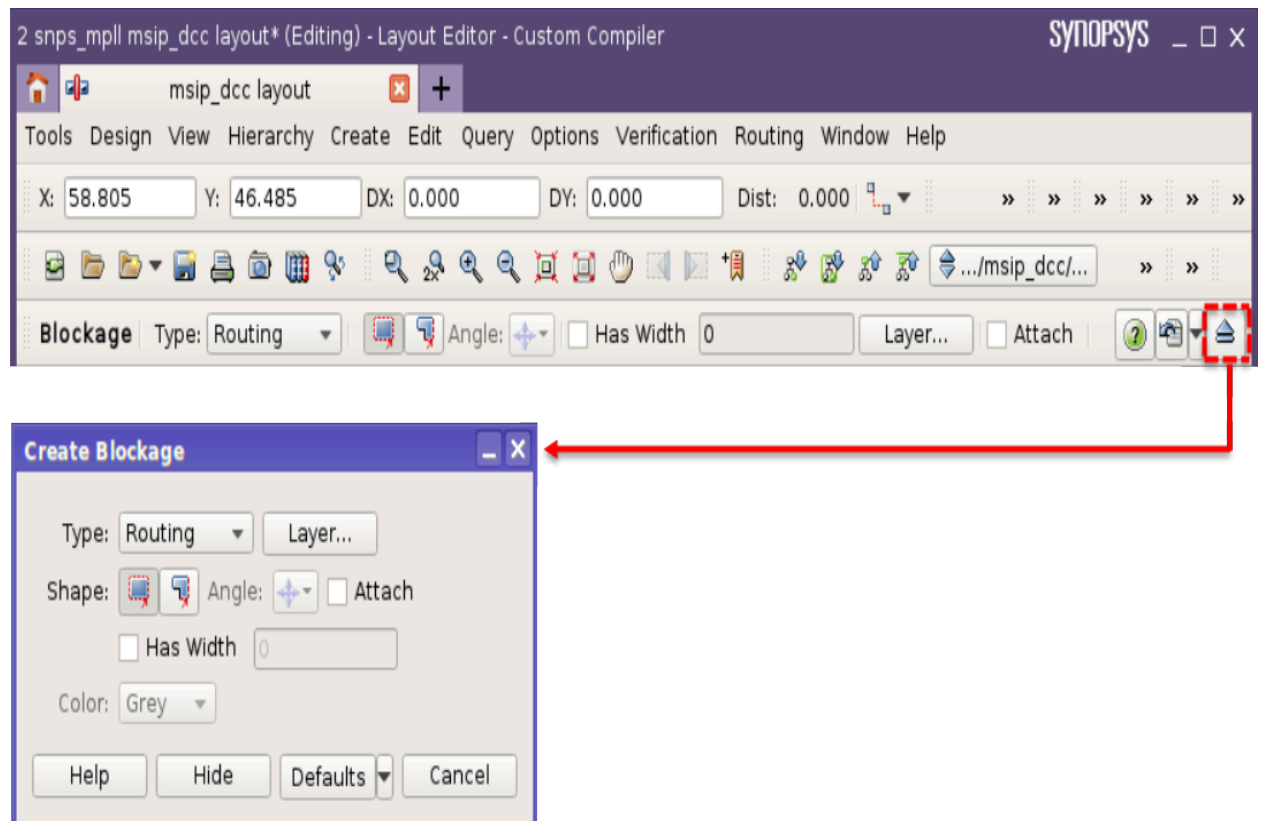
See Also

- [About Blockages](#)
- [Creating Blockages](#)
- [Blockage Toolbar and Create Blockage Dialog Box Reference](#)
- [Blockage Shortcut Menu and Shortcut Keys](#)

Blockage Toolbar and Create Blockage Dialog Box Reference

The **Blockage** toolbar and its corresponding **Create Blockage** dialog box contain mostly the same options. The dialog box also lets you set the color of the blockage shape.

This section provides a quick reference of the **Blockage** toolbar and the **Create Blockage** dialog box. For instructions on how to create blockages, see [Creating Blockages](#).



Option	Description
Type	Specifies the type of objects to block. The available blockage types are Routing, Placement, Pin, Wiring, Fill, Slot, Screen, and Feedthru. The default is Placement. • Routing blockage specifies the region on a corresponding layer where no routing object (path, path segment, and via) can be created on a given layer. • Placement blockage specifies the region where no instances may be placed. • Pin blockage specifies the region on a corresponding layer where no pin shape may be created on a given layer. • Wiring blockage specifies the region on a corresponding layer, where no wiring object may be created on a given layer. • Fill, Slot, Screen, and Feedthru blockages are similar and they specify the region on a corresponding layer, where no specific purpose shapes may be created on a given layer. Tcl Preference: <code>leBlockageType</code> • To be more specific about layer materials that each type of blockage supports: • <i>Routing</i>: metal, poly, diff, localInterconnect, cut • <i>Wiring</i>, <i>Pin</i>, Fill, Slot, Screen: metal, poly, diff, localInterconnect • Feedthru: metal
Layer	• Opens a dialog box that lets you select the blockage layers. In the dialog box, select Show All to display the list of blockage layers. • For routing, pin, wiring, fill, slot, screen and feedthru blockages, you must select at least one layer. The tool selects the active layer purpose pair automatically if it is a routing layer purpose pair. • If you select multiple layers, the tool creates blockage shapes for each layer. • This option is not applicable to placement blockages. Tcl Preference: <code>leBlockageLayer</code>
Shape	Create either a rectangular or polygonal blockage shape. Tcl Preference: <code>leCreateShapeType</code>  
Angle	Specifies an angle mode for drawing polygonal blockages. The available angle modes are Orthogonal, Diagonal, Any Angle, X First, and Y First. The default is Orthogonal. Tcl Preference: <code>leCreateAngleMode</code>

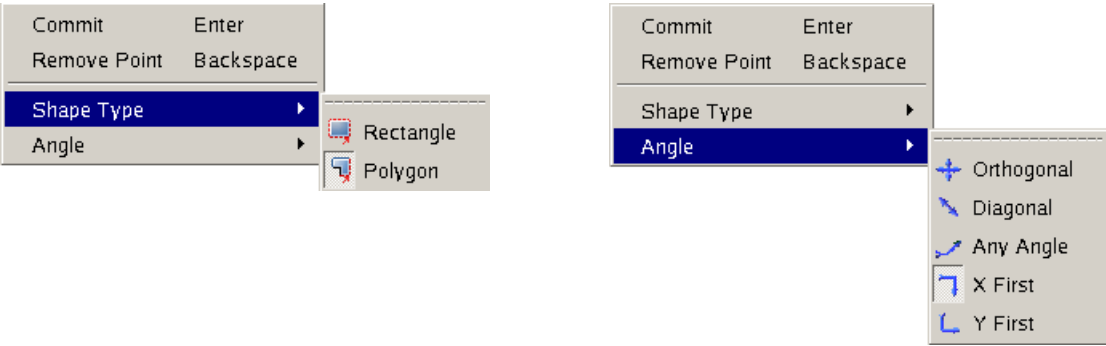
Option	Description
Attach	When enabled, prompts you, after completing the shape, to select a parent instance to which to attach the blockage. You must select an instance, not a shape. Use Attach to place blockages over specific instances at the top level to prevent further placement of instances or routings (depending on blockage type) over that instance. When the instance is moved, the blockage is moved along automatically. Tcl Preference: <code>leAttach</code>
Has Width	When enabled, activates the Width text box for editing. • When the Width value is 0 (zero), the tool allows blocked objects to abut the blockage. • When the Width value is positive, the tool uses the value to determine the minimum spacing between the blockage and blocked objects. • This option is disabled for placement blockages. Tcl Preferences: <code>leBlockageHasWidth</code> and <code>leBlockageWidth</code>
Color	Color of the blockage. Choose the color from the list: • Multi: DRD will skip Multi blockages during design-rule checking. • Black: Colored shapes will observe <code>minSpacing</code> rules when approaching Black blockages. • Grey: Ordinary blockages. The color attributes of Grey blockages are ignored, and other rules, such as <code>minSpacing</code>, are observed. • <color_n>: <i>Color_<n></i> shapes must observe <code>minSpacing</code> rules when approaching <i><color_n></i> blockages.

See Also

- [Blockage Shortcut Menu and Shortcut Keys](#)

Blockage Shortcut Menu and Shortcut Keys

While the **Create > Blockage** command is active, you can right-click to display the Create Blockage shortcut menu. This section provides a quick reference of the shortcut menu and shortcut keys.



Command	Purpose	Default Shortcut Key
Commit	Completes the blockage if the shape is valid.	Enter
Remove Point	Deletes the previously entered point.	Backspace
Shape Type	Selects a rectangular or polygonal blockage	
Angle	Controls the orientation of polygonal segments.	

See Also

- [Creating Blockages](#)
- [Editing Blockages](#)
- [Blockage Toolbar and Create Blockage Dialog Box Reference](#)

Creating and Editing Route Corridors

This topic describes the following:

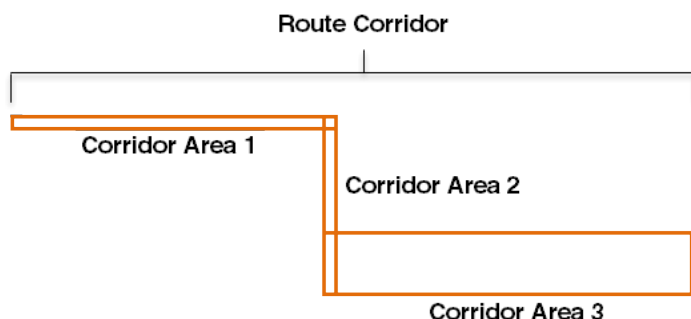
- [About Route Corridors](#)
- [Before You Start](#)
- [Creating New Route Corridors](#)
- [Appending to Existing Route Corridors](#)
- [Viewing Route Corridor Area Information](#)
- [Changing Route Corridor Attributes](#)

About Route Corridors

The speed of integrated circuits depends on the critical nets in your design. Because critical nets have shorter time budgets than non-critical nets, you should carefully control the routes of critical nets to ensure optimal performance.

Routing corridors provide a way for you to control the routing of critical nets or non-critical nets. You can associate nets to a corridor to restrict the routes permitted in the corridor.

Route corridors consist of one or more corridor areas, and the areas within a corridor can have different widths. You can create a routing corridor, and then append to it later. When you append to an existing corridor, you are adding a new corridor area to an existing route corridor.



You can move, copy, stretch, or delete a corridor. For information about these editing commands, see [Editing Objects - Basic Operations](#).

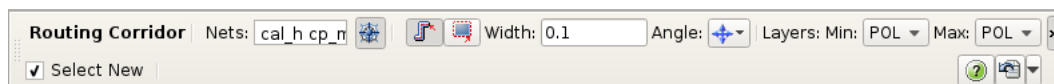
Before You Start

Review the design and plan the routes for the critical nets.

Creating New Route Corridors

To create a routing corridor:

1. In the Layout Editor, choose **Create > Routing Corridor**. The **Routing Corridor** toolbar appears.

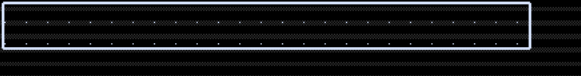


2. (Optional) In the **Nets** box, type the names of the nets to be associated with the routing corridor. Alternatively, click  to select the nets using the Design Navigator Assistant.

If you do not specify a net now, you can always add the net later using the Property Editor. See [Changing Route Corridor Attributes](#).

If route corridors do not have net assignments, the Custom Router will determine the routes to place inside the corridor.

3. Select one of the following ways to create the corridor:

Button	Description
 (line)	Allows you to draw corridors that have a fixed Width. A temporary centerline appears while you are in this mode to provide visual guidance. The centerline is at the midpoint of the width. When you are finished drawing, the centerline is removed. Before  After 
 (rectangle)	Allows you to create rectangular-shaped corridors. The width and height of the corridor remain dynamic until you commit the corridor.

4. ( only) Specify the fixed **Width** of the corridor.
5. (Optional) Select the **Angle** of the corridor: **Orthogonal**, **X-First**, or **Y-First**. For the description of the options, see Angle Mode for Edges and Centerlines.
6. Specify the range of **Layers** to which the corridor applies:
 - **Min**: The lowest layer.
 - **Max**: The highest layer.
7. Enable **Select New** to select the corridor immediately after you create it. Enable this option if you want to append to the current corridor immediately after you create it. If you want to create a new corridor, disable the option.

For example, after creating Corridor_1, if you want to continue to add corridor areas to Corridor_1, enable Select New. However, if you want to create a new corridor, Corridor_2, disable Select New.

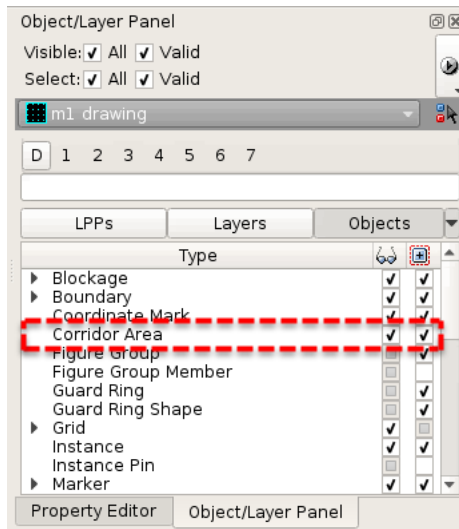
8. On the canvas, draw the corridor. Make sure the corridor covers pins.

9. When you are finished, press Enter. The corridor is committed to the design.

If you enabled **Select New** in [Step 7](#), the corridor borders are highlighted to indicate that it is selected.

Appending to Existing Route Corridors

Before you append to corridors, make sure corridor areas are selectable. To do this, in the **Objects** tab of the Object/Layer Panel Assistant, turn on **Selectability** for **Corridor Area**.



You can append to an existing route corridor as follows:

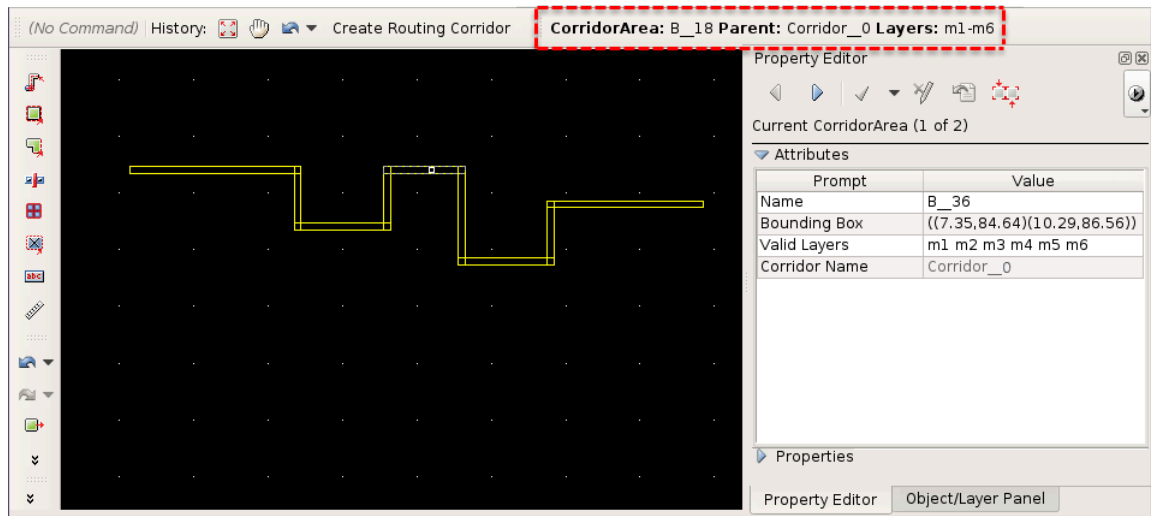
- On the canvas, select the existing corridor (through direct selection or area selection), and then use the **Create > Routing Corridor** command to append to the selected corridor.
- When you create the first corridor, enable the **Select New** option on the **Routing Corridor** toolbar. The newly created corridor is automatically selected so that you can proceed to append to it.

When appending to existing corridors, the new corridor will apply to the same nets as the existing corridor. You will not be permitted to specify a different net.

Viewing Route Corridor Area Information

To view the details about a route corridor area:

1. On the canvas, rest the pointer on the route corridor area. Information about the selected corridor area displays on the Object Info toolbar.

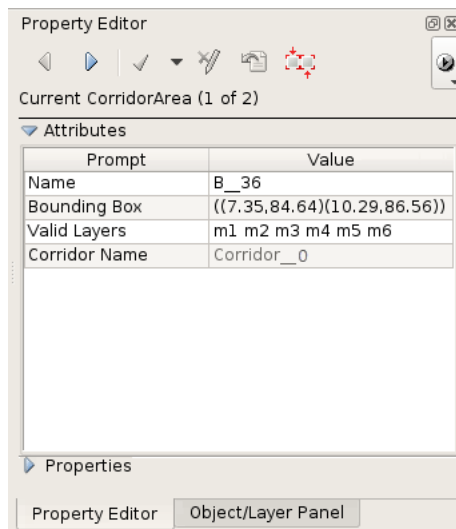


2. To view the information in the Property Editor, display the Property Editor, and then select the corridor or corridor area on the canvas.

Changing Route Corridor Attributes

To change the attributes of an existing route corridor or corridor area:

1. In the Layout Editor, display the Property Editor.
2. On the canvas, select the route corridor or corridor area using direct selection or area selection. Information about the selected corridor or corridor area displays in the Property Editor.



3. Modify any of the corridor attributes.

4. When you are finished, click .

Creating and Editing Track Region PCells

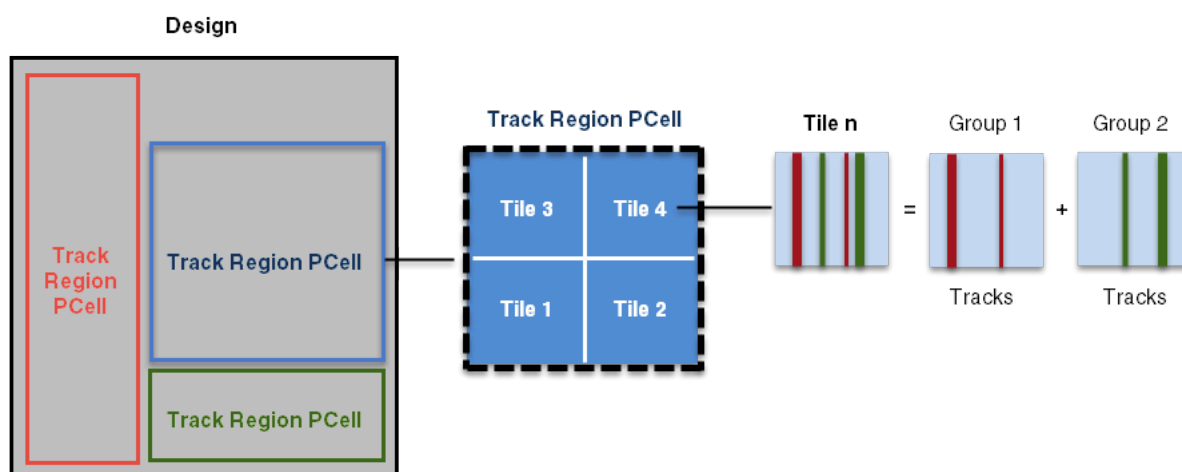
This topic describes the following:

- [About Track Region PCells](#)
- [Before You Start](#)
- [Creating Track Region PCells Using Presets](#)
- [Creating New Track Region PCells](#)
- [Copying Track Groups](#)
- [Deleting Track Groups](#)
- [Editing Track Region PCells](#)
- [Displaying and Hiding Track Region PCells](#)
- [Track Region PCell Toolbar and Dialog Box Reference](#)

About Track Region PCells

Tracks in the design ensure that routes are properly placed and correct by construction. During route creation and editing, routes are snapped to the nearest track, thus facilitating the routing process and reducing design time. Using tracks are especially helpful in advanced node designs, such as those containing FinFET components.

To accommodate different requirements within the same design, you can create multiple track region PCells. A track region PCell is an editable PCell that contains a set of tiles that always starts from the bottom-left corner of the PCell. Each tile contains groups of tracks.



You can create track region PCells using one of the following methods:

- **Create > Rectangle** command, which requires that you use the Object/Layer Panel to select a track boundary LPP, such as the system-reserved **border track** LPP.
- Track Region Assistant **Create Region PCell** command, which automatically detects and uses the available track boundary LPPs.

This topic describes how to use the **Create > Rectangle** command to create and edit track region PCells. For instructions on using the Track Region Assistant, see Track Region Assistant.

Note:

The Track Region Assistant also lets you create non-PCell track regions. This method requires that you first set up track region definitions in the technology database. Routes you create in the Layout Editor will recognize the tracks regardless of the method you use.

Before You Start

You can create track region PCells only on specific layer-purpose pairs (LPPs). The system-reserved **border track** LPP is available for this purpose. When you use the **border track** LPP, you can set the track properties as you create the track region PCells.

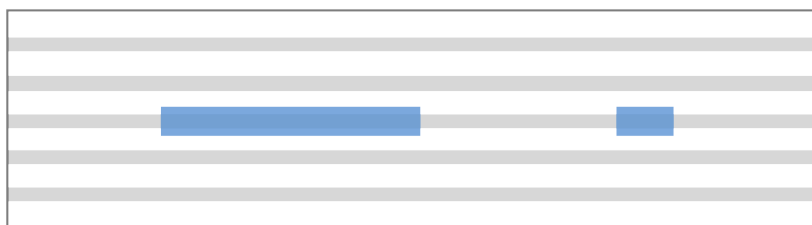
If you prefer to create track region PCells that have predefined track properties, you can add `boundaryTrackRegion` definitions in the Layer Rules section of the technology database. In addition to track properties, you can define the LPP names, which will be listed in the Object/Layer Panel.

If you want interconnects to start and end at specific locations on the tracks, be sure to plan and define grids containing permissible starting and stopping points in the track region PCell. See the [Track Parameters Table](#) in the Edit Track dialog box.

Note:

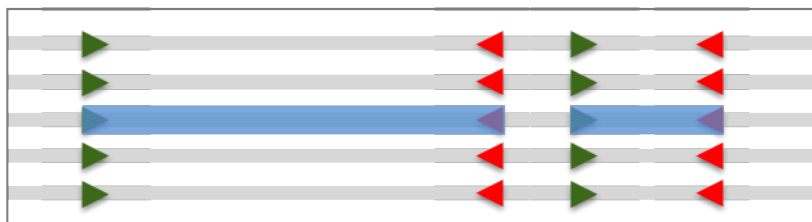
If a gridded track region overlaps with a track region without grid, the gridded track region takes priority.

No Grid



-  Tracks
-  Interconnects/Routes
-  Start Points
-  Stop Points

With Grid



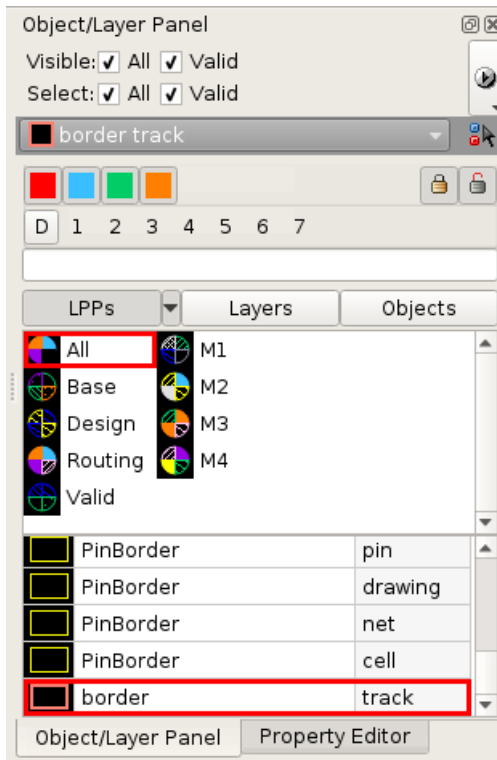
Creating Track Region PCells Using Presets

Presets are definitions of track region PCells that you created previously in the same design or in another design. You can reuse these definitions to create new track region PCells.

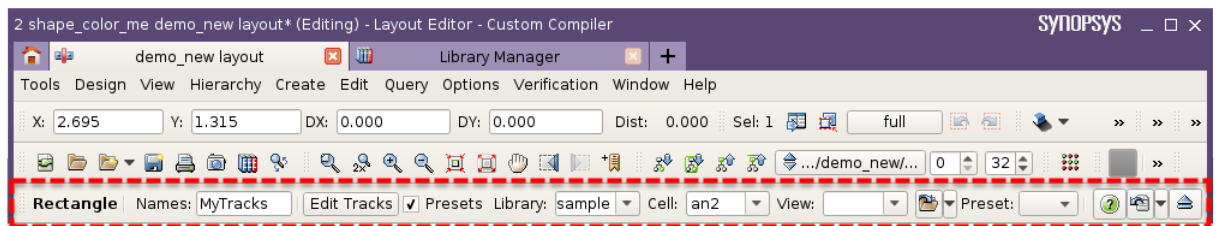
To create a track region PCell using a preset:

1. In the Object/Layer Panel, choose a layer-purpose pair (LPP) that is defined for track region PCells. See [Before You Start](#).

To use the system-reserved **border track** LPP, in the Object/Layer Panel, click, the **LPPs** tab, select **All**, and select **border track**.

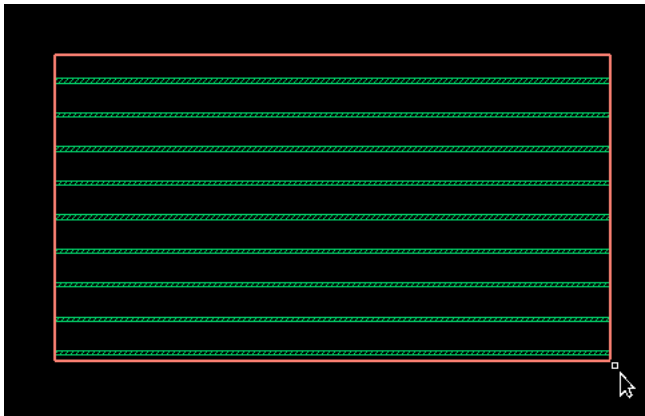


2. Choose **Create > Rectangle**. The **Rectangle** toolbar appears.



3. On the **Rectangle** toolbar, set the options:
 - **Names:** Type the names of the new track region PCells. A name is applied to each track region PCell that you create consecutively while the command is active.
 - **Presets:** Select Presets to enable the preset options.

- **Library, Cell, View:** Choose the cellview (design) that contains the preset (track region PCell definitions that you want to reuse).
 - **Preset:** Choose the name of the preset that you want to use. The preset names are names of the track region PCells you created previously.
4. On the layout canvas, draw the track region PCell.



5. When you are finished, press Esc to quit the command.

See Also

- [Defining Track Presets](#)

Creating New Track Region PCells

When you create new track region PCells, you can define the tracks (width, spacing, and direction). You can also define the size of the tiles, the offset from the region boundary, and other options.

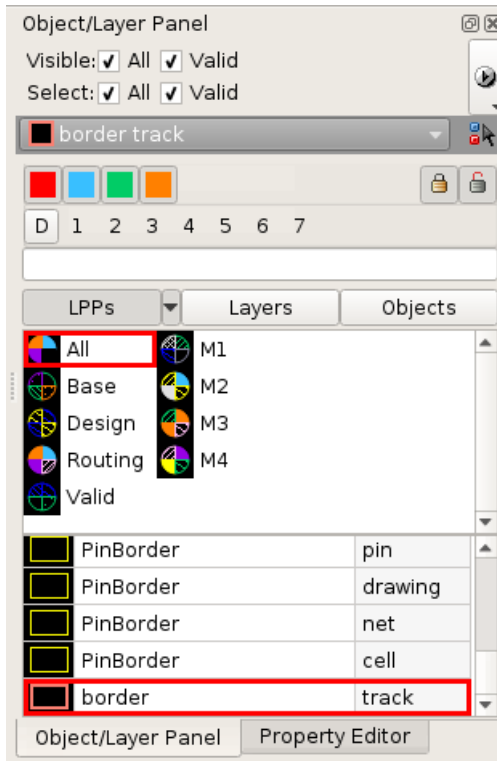
Track region PCells are stored as instances in layout cellviews (or designs). Although the track region PCell you create is stored in the current design, its definitions are available for reuse as presets in other designs. See [Creating Track Region PCells Using Presets](#).

As you create track region PCells, check the Console for messages that might help correct missing or invalid entries.

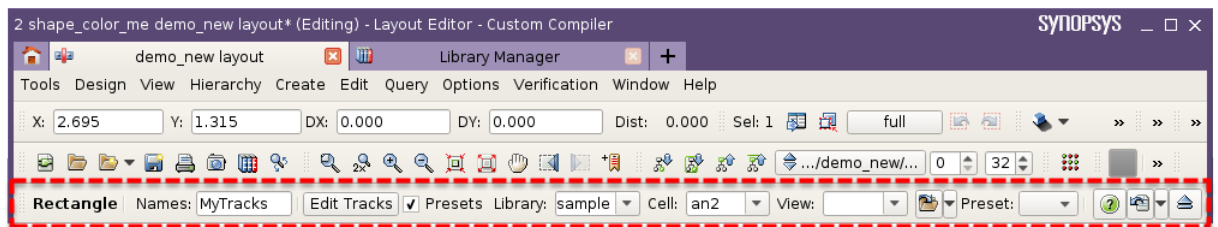
 [Show me](#) an example of how to create track region PCells.

To create a new track region PCell:

1. In the Object/Layer Panel, choose a layer-purpose pair (LPP) that is defined for track region PCells. See [Before You Start](#).

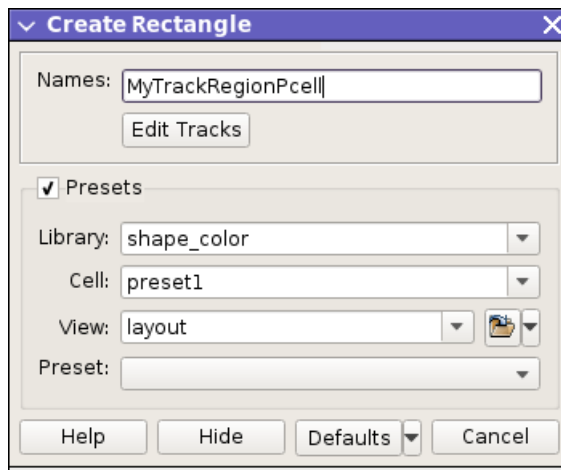


2. Choose **Create > Rectangle**. The **Rectangle** toolbar appears.



3. (Optional) On the **Rectangle** toolbar, click Eject  to display the **Create Rectangle** dialog box. The toolbar and the dialog box contain the same options.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.



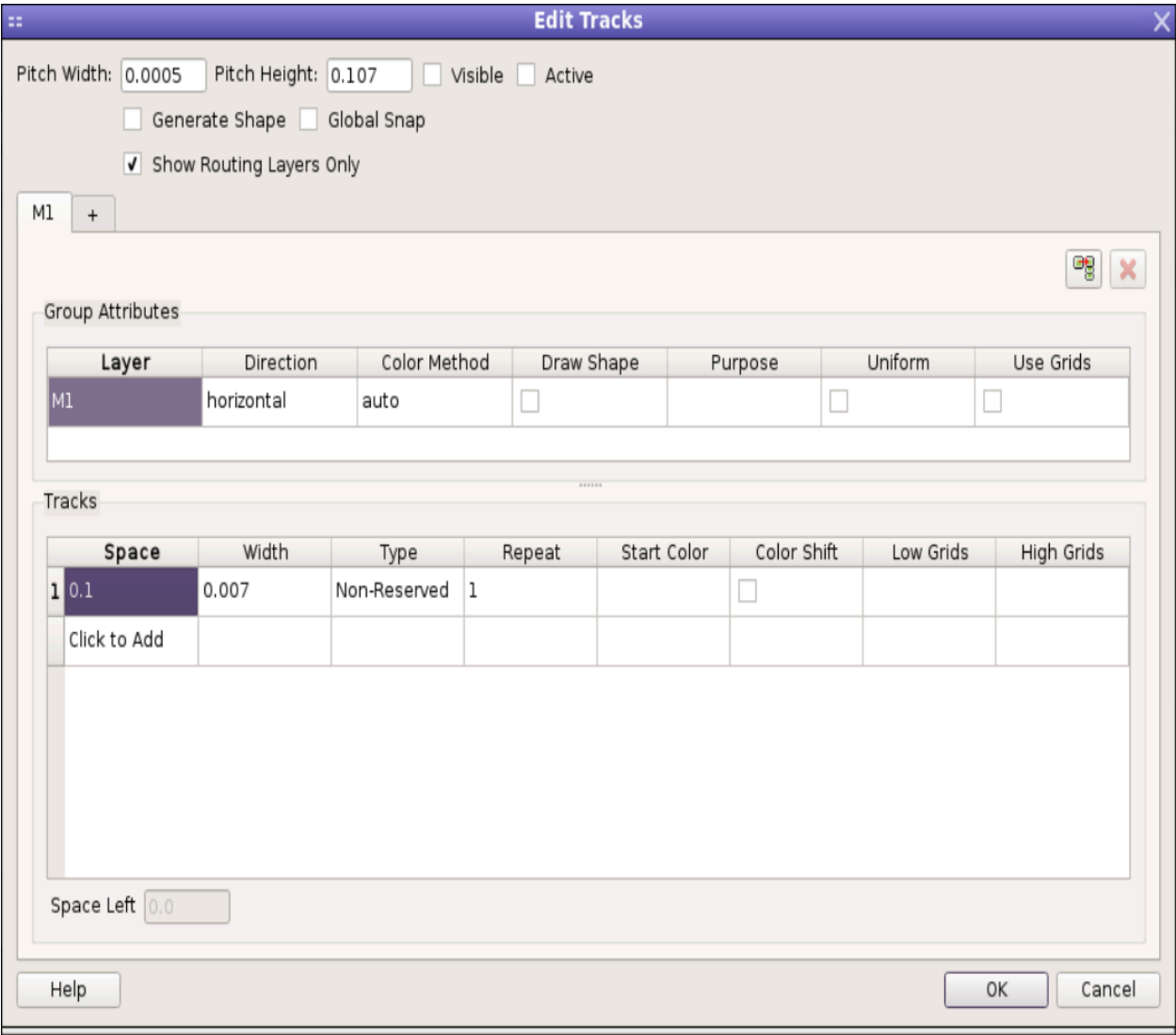
4. On the toolbar or in the dialog box, type the **Names** of the new track region PCells. A name is applied to each track region PCell that you create consecutively while the command is active.

Note:

The names will appear in the Preset list so that you can reuse the track region PCell definitions later. See [Creating Track Region PCells Using Presets](#).

5. Click **Edit Tracks**. The **Edit Tracks** dialog box opens.

At the top of the dialog box are general options that apply to the entire track region PCell. Below the general parameters are tabs that contain data for tracks. Each tab represents a track group. Within each tab are data for individual tracks.



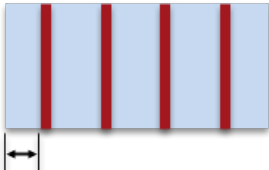
6. At the top of the dialog box, set the general options that are applicable to the entire track region PCell:

Option	Description
Pitch Width	Tile width.
Pitch Height	Tile height.
Visible	Turns on or off the visibility of the track region PCell.
Active	Turns on or off the active state of the track region PCell.

Option	Description
Generate Shape	Draws the track region PCell on the layout canvas.
Global Snap	Specifies whether the track region PCell snaps globally relative to the design axis (on) or locally (off). The default local snapping is relative to the manufacturing grid.
Show Routing Layers Only	Displays only the routing layers in the Layers list. To display all layers, clear the Show Routing Layers Only check box.

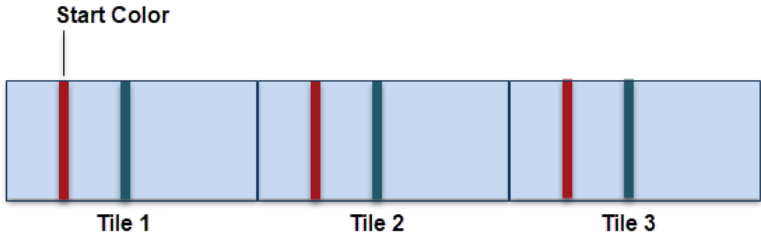
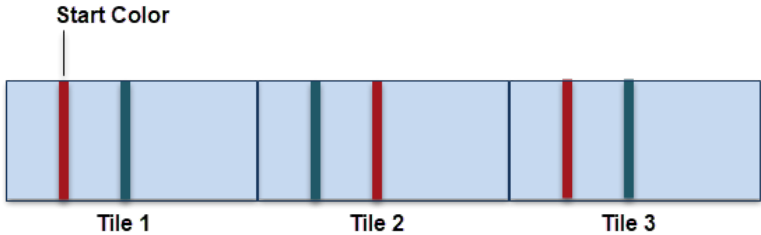
7. In the tab, set the following in the **Layer** table (the first table). Required fields have a red background.

Option	Description
Layer	The layer on which the tracks are created. The tab name refreshes to match the layer name you choose.
Direction	The direction of the tracks: Horizontal or Vertical .
Colors	The colors of the tracks: ◦ auto: Automatically assigns the color. ◦ manual: Lets you manually assign the color. ◦ no color: Does not assign any color.
Draw Shape	Draws physical shapes inside the PCell master to represent the tracks.
Purpose	The corresponding purpose of the layer you chose.

Option	Description
Uniform	Specifies whether the properties of a group of tracks are uniform. In the uniform mode, each row in a tab provides the definition for a set of tracks in the tile. The set of tracks have the same track width and spacing. In the example shown, one row (or one track definition) results in the set of four uniform tracks in the tile. In the non-uniform mode, each row in a tab defines a single track. In the example shown, three rows in the same tab results in three tracks with different widths and spacing. If you want a track to appear more than once in the tile, set the Repeat option in the Track Parameters Table. Uniform  Same: - o Track Widths - o Track-to-Track Spacing Non-uniform  Different: - o Track Widths - o Track-to-Track Spacing

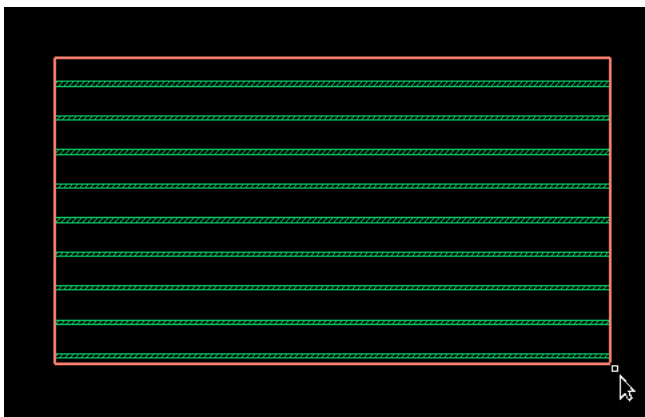
8. In the tab, set the following in the track parameters table (the second table in the tab). The parameters that are displayed depend on whether you specified **Uniform** tracks in [Step 7](#). Required fields have a red background.

Option	Description
Offset (Uniform only)	Distance between the first track and the PCell boundary. The offset always measured from the left or bottom side of the PCell.
Width	Track width.
Space	Distance between adjacent tracks.

Option	Description
Type	Type of track that paths will snap to: ◦ default: Wire snapping has no wire width requirements. Wires of any width can snap to the track. with wider wires consuming more available area. ◦ reserved: Wire snapping depends on the <code>leReservedTrackWidthCheck</code> Tcl preference setting. If the preference is set to <code>false</code>, the track operates like a default track, and wires of any width can route on the tracks. When the preference is set to <code>true</code>, only wires with non-default-rule nets can route on the tracks, and the wires must match the width of the reserved tracks. ◦ nonReserved: Only wires that have the same width as the tracks can snap to the tracks. Tcl Preference: <code>leTrackPcellTrackType</code>
Repeat (Non-uniform only)	Number of times to repeat the track. When the tracks are uniform, the repeat value is automatically calculated, so you do not need to specify it.
Start Color	Color of the first track in the tile. In the following example, Start Color is red.  The diagram shows three adjacent rectangular tiles labeled 'Tile 1', 'Tile 2', and 'Tile 3'. Each tile contains two vertical lines: a red line on the left and a blue line on the right. A label 'Start Color' with a pointer indicates the red line in Tile 1.
Color Shift	Cycles the Start Color in subsequent tiles. In the following example, the available colors are red and blue. Start Color is red, so blue becomes the First Color in tile 2.  The diagram shows three adjacent rectangular tiles labeled 'Tile 1', 'Tile 2', and 'Tile 3'. In Tile 1, there is a red line on the left and a blue line on the right. In Tile 2, the positions are swapped: a blue line on the left and a red line on the right. In Tile 3, it returns to a red line on the left and a blue line on the right. A label 'Start Color' with a pointer indicates the red line in Tile 1.
Low Grids	Specifies the starting point information for gridded tracks: <offset> <lowPoint1> ... <lowPointn> The first value is the offset distance for the grid starting points. The subsequent values are the starting points for the grid. Add a character space between the values. You must select Use Grids before setting this option.

Option	Description
High Grids	Specifies the ending point information for gridded tracks : <offset> <highPoint1> ... <highPointn> The first value is the offset distance for the grid ending points. The subsequent values are the ending points for the grid. Add a character space between the values. You must select Use Grids before setting this option.
Space Left (Non-uniform only)	(Read-only) Displays the amount of space left in the region after adding the tracks you defined. You can use the information to determine if you can add more tracks in the region.

9. To add a new tab, click the **+** tab.
10. Repeat [Step 7](#) through [Step 9](#) to define new track groups.
11. When you are finished, click **OK** to return to the **Create Rectangle** dialog box.
12. On the layout canvas, draw the track region PCell.



13. When you are finished, press Esc to quit the command.

Copying Track Groups

You can create a new track group by copying an existing layer tab and modify some of the settings.

To copy a track group (layer tab):

1. Click the layer tab that you want to copy so that the tab's contents are displayed.
2. Click **Copy group to a new tab** . A duplicate copy of the tab appears.

You can change the **Layer** name and other settings.

Deleting Track Groups

To delete a track group (a layer tab):

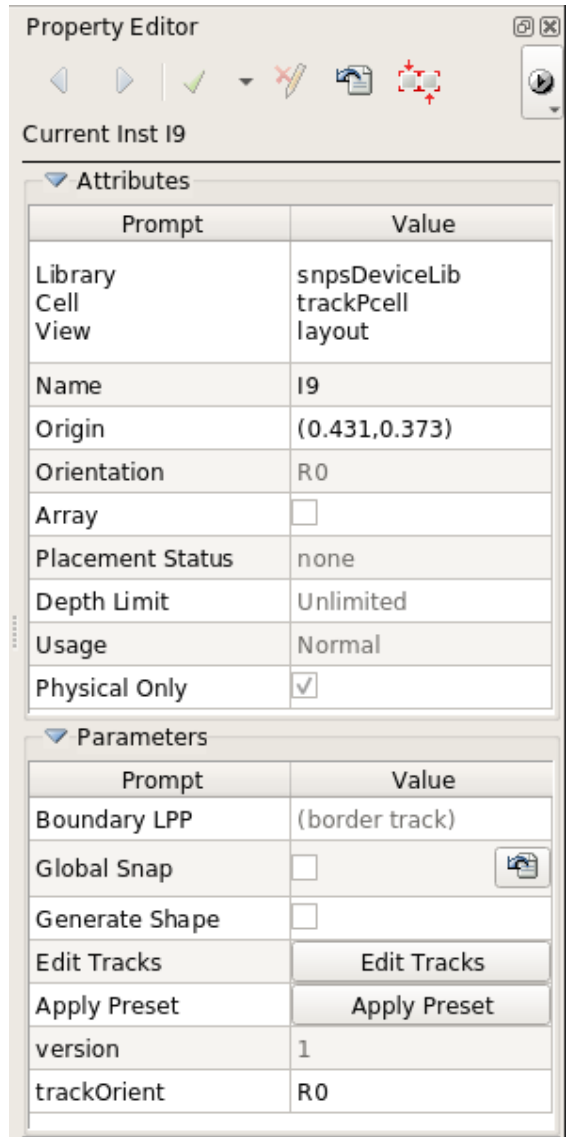
1. Click the layer tab that you want to delete.
2. Click **Delete group** . The tab is deleted.

Editing Track Region PCells

You can modify aspects of track region PCells. For example, you can adjust the track spacing, offset, and width. You can also switch to and apply another preset.

To edit track region PCells:

1. Display the Property Editor.



2. On the layout canvas, select the track region PCell. The properties of the track region PCell appear in the Property Editor.

Track Region PCell supports Orientation.

The orientation of the track regions created by the track region Pcell has the same orientation as the CDF parameter of the track region PCell. In addition, the `shapeData` parameter is updated by the callback function if the orientation changes.

3. Modify the track region PCell **Parameters**. For the description of the parameters, see [Create Rectangle Toolbar and Dialog Box \(Track Region PCells\)](#).

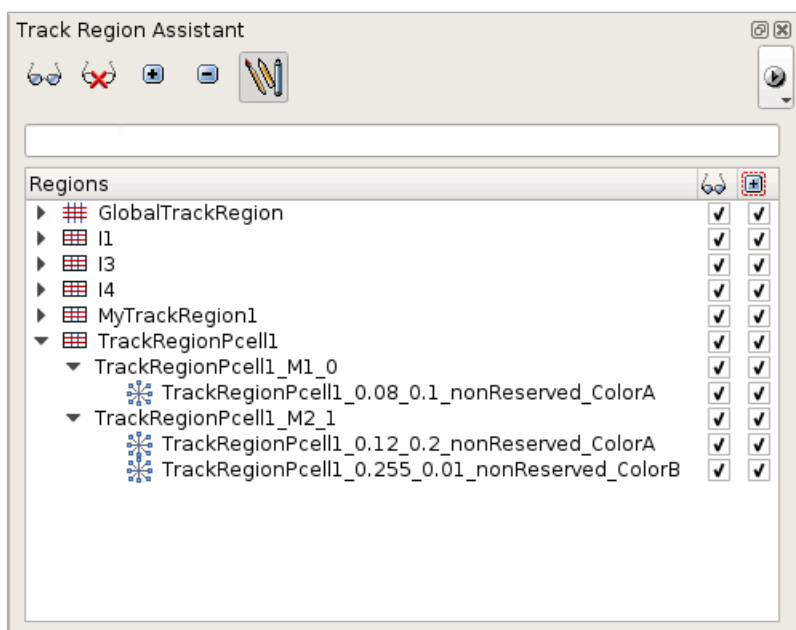
If you want to edit the tracks, click **Edit Tracks**.

If you want to select and apply a different preset, click **Apply Preset**.

4. When you are finished, click **Apply Changes**  at the top of the Property Editor. The Layout Editor saves the changes.

Displaying and Hiding Track Region PCells

You can use the Track Region Assistant to display or hide entire track region PCells or specific tracks in the PCells. See Track Region Assistant for instructions.



To facilitate viewing, when you select a track region PCell in the Track Region Assistant, the corresponding track region is also selected on the canvas. Conversely, if you select a track region PCell on the canvas, it is also selected in the Track Region Assistant. See Synchronizing Assistants.

Track Region PCell Toolbar and Dialog Box Reference

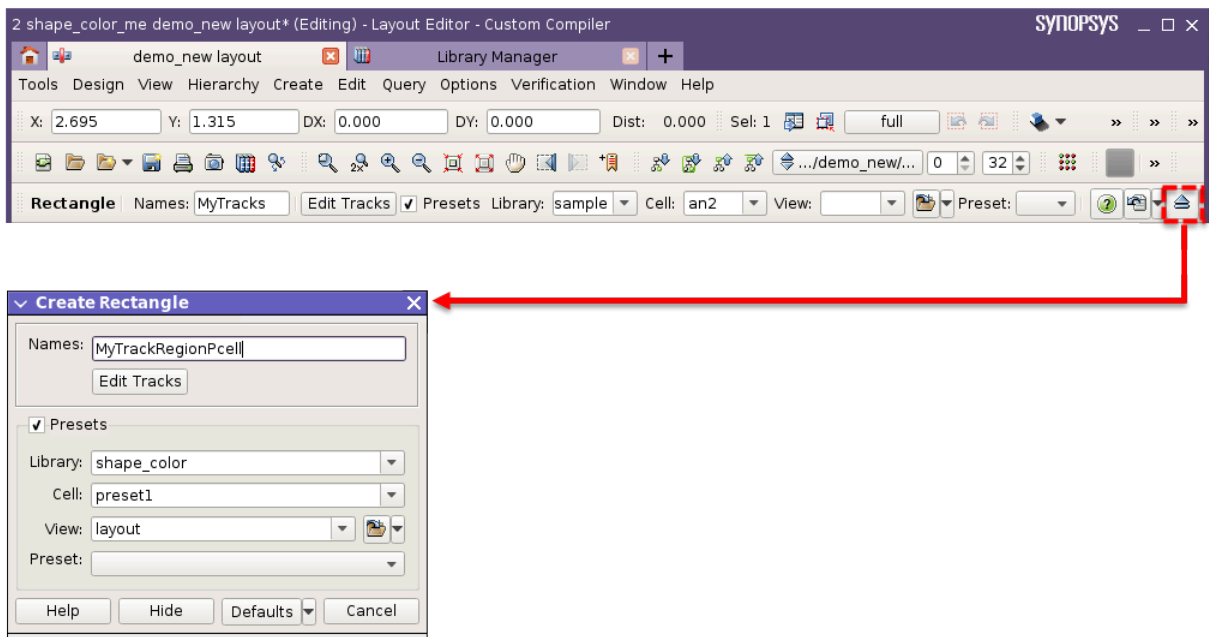
This section provides a quick reference of the following:

- [Create Rectangle Toolbar and Dialog Box \(Track Region PCells\)](#)
- [Edit Tracks Dialog Box](#)
- [Apply Preset Dialog Box](#)

For instructions on how to create and edit track region PCells, see [Creating and Editing Track Region PCells](#).

Create Rectangle Toolbar and Dialog Box (Track Region PCells)

This section provides a quick reference of the **Rectangle toolbar** and **Create Rectangle** dialog box for track region PCells. For information about the basic **Create Rectangle** command or the **Create Rectangle** dialog box, see [Creating Rectangles](#).



Dialog Box Content	Description
Names	The names of the new track region PCells. A name is applied to each track region PCell that you create consecutively while the command is active.
Edit Tracks	Opens the Edit Tracks dialog box. See Edit Tracks Dialog Box .

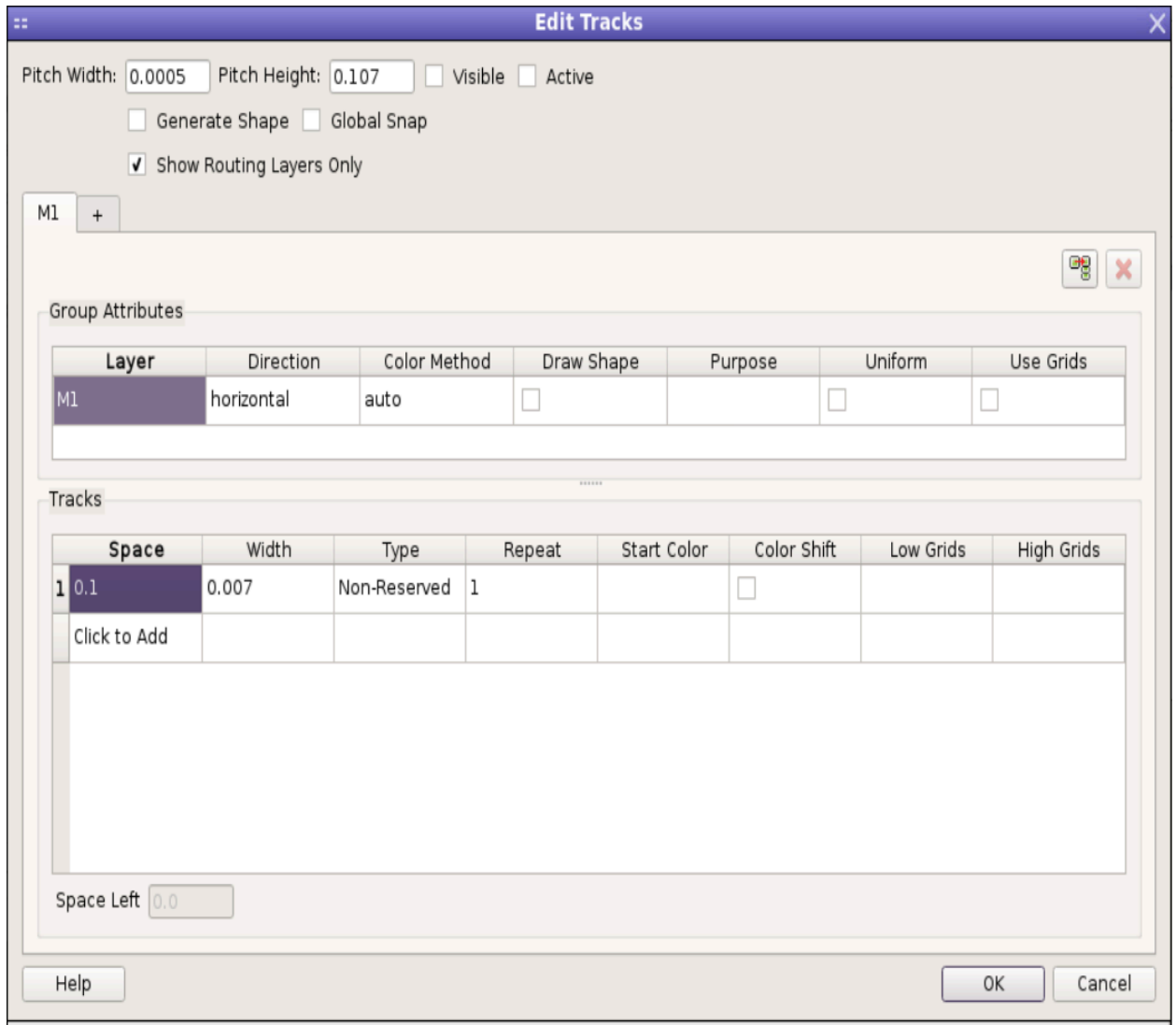
Dialog Box Content	Description
Presets	Enables the options for preset selection.
Library/Cell/View	The library, cell, and view (or the design) that contains the preset.
Preset	The list of existing track region PCells whose definitions you can reuse to create new track region PCells.

Edit Tracks Dialog Box

This section provides a quick reference of the **Edit Tracks** dialog box. For instructions on how to create and edit track region PCells, see [Creating and Editing Track Region PCells](#).

To open the Edit Tracks dialog box, click **Edit Tracks** from one of the following:

- Rectangle toolbar
- Create Rectangle dialog box
- Property Editor



The **Edit Tracks** dialog box consists of the following:

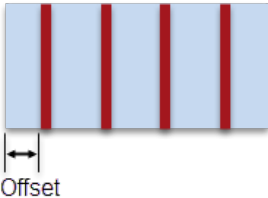
Dialog Box Content	Description
General options	The General Options area contains options that apply to the entire track region PCell.
Tabs	Each tab represents the data for a single track group. For each tab or track group, you can set parameters in the Layer Table and the Track Parameters Table .

General Options

Option	Description
Pitch Width	Tile width.
Pitch Height	Tile height.
Visible	Turns on or off the visibility of the track region PCell.
Active	Turns on or off the active state of the track region PCell.
Generate Shape	Draws the track region PCell on the layout canvas. This option does not appear in the Edit Track dialog box that opens from the Property Editor.
Global Snap	Specifies whether the track region PCell snaps globally relative to the design axis (on) or locally (off). The default local snapping is relative to the manufacturing grid. This option does not appear in the Edit Track dialog box that opens from the Property Editor.
Show Routing Layers Only	Displays only the routing layers in the Layers list. To display all layers, clear the Show Routing Layers Only check box.

Layer Table

Option	Description
Layer	The layer on which the tracks are created.
Direction	The direction of the tracks: Horizontal or Vertical .
Colors	The colors of the tracks: • auto: Automatically assigns the color. • manual: Lets you manually assign the color. • no color: Does not assign any color.
Draw Shape	Draws physical shapes inside the PCell master to represent the tracks. Tcl Preference: <code>leTrackPcellDrawShape</code>
Purpose	The corresponding purpose of the layer you chose.

Option	Description
Uniform	Specifies whether the properties of a group of tracks are uniform. In the uniform mode, each row in a tab provides the definition for a set of tracks in the tile. The set of tracks have the same track width and spacing. In the example shown, one row (or one track definition) results in the set of four uniform tracks in the tile. In the non-uniform mode, each row in a tab defines a single track. In the example shown, three rows in the same tab results in three tracks with different widths and spacing. If you want a track to appear more than once in the tile, set the Repeat option in the Track Parameters Table. Uniform  Same: - o Track Widths - o Track-to-Track Spacing Non-uniform  Different: - o Track Widths - o Track-to-Track Spacing
Use Grids	Creates tracks in intervals, with starting points and ending points. If you select Use Grid, you must also specify the Low Grids and High Grids. See Before You Start for information about track grids.

Track Parameters Table

The second table in the tab lets you specify the track width, track spacing, and other parameters.

Track Parameters (Non-uniform)

Group Attributes

Layer	Direction	Color Method	Draw Shape	Purpose	Uniform	Use Grids
M1	horizontal	auto	☐		☐	☐

Tracks

Space	Width	Type	Repeat	Start Color	Color Shift	Low Grids	High Grids
1 0.1	0.007	Non-Reserved	1		☐		
Click to Add							

Track Parameters (Uniform)

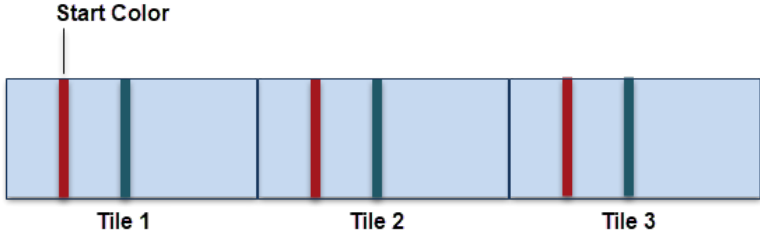
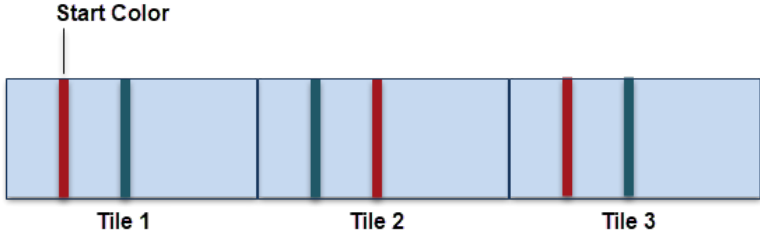
Group Attributes

Layer	Direction	Color Method	Draw Shape	Purpose	Uniform	Use Grids
M1	horizontal	auto	☒	boundary	☒	☐

Tracks

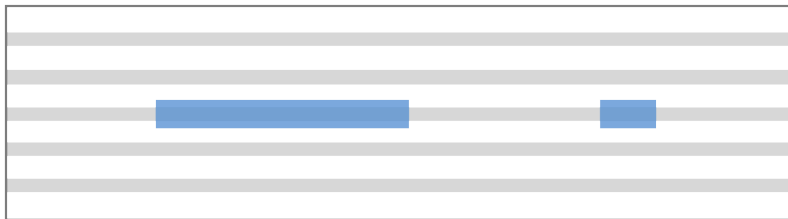
Offset	Width	Space	Type	Start Color	Color Shift	Low Grids	High Grids
1 0.1	0.007	0.05	Non-Reserved		☐		
Click to Add							

Option	Description
Offset (Uniform only)	Distance between the first track and the PCell boundary. The offset always measured from the left or bottom side of the PCell.
Width	Track width.
Space	Distance between adjacent tracks.

Option	Description
Type	Type of track that paths will snap to: • default: Wire snapping has no wire width requirements. Wires of any width can snap to the track. with wider wires consuming more available area. • reserved: Wire snapping depends on the <code>leReservedTrackWidthCheck</code> Tcl preference setting. If the preference is set to <code>false</code>, the track operates like a default track, and wires of any width can route on the tracks. When the preference is set to <code>true</code>, only wires with non-default-rule nets can route on the tracks, and the wires must match the width of the reserved tracks. • nonReserved: Only wires that have the same width as the tracks can snap to the tracks. Tcl Preference: <code>leTrackPcellTrackType</code>
Repeat (Non-uniform only)	Number of times to repeat the track. When the tracks are uniform, the repeat value is automatically calculated, so you do not need to specify it.
Start Color	Color of the first track in the tile. In the following example, Start Color is red. 
Color Shift	Cycles the Start Color in subsequent tiles. In the following example, the available colors are red and blue. Start Color is red, so blue becomes the First Color in tile 2. 
Low Grids	Specifies the starting point information for gridded tracks: <code><offset> <lowPoint1> ... <lowPointn></code> The first value is the offset distance for the grid starting points. The subsequent values are the starting points for the grid. Add a character space between the values. You must select Use Grids before setting this option.

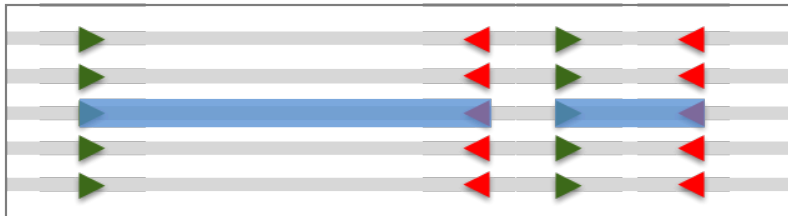
Option	Description
High Grids	Specifies the ending point information for gridded tracks : <offset> <highPoint1> ... <highPointn> The first value is the offset distance for the grid ending points. The subsequent values are the ending points for the grid. Add a character space between the values. You must select Use Grids before setting this option.
Space Left (Non-uniform only)	(Read-only) Displays the amount of space left in the region after adding the tracks you defined. You can use the information to determine if you can add more tracks in the region.

No Grid



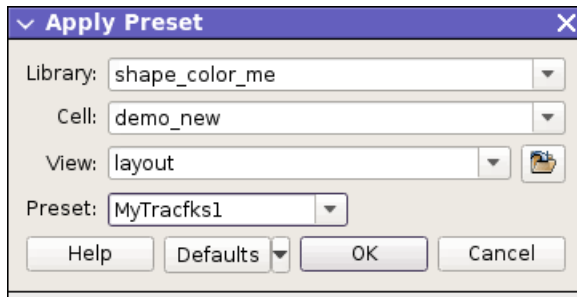
- Tracks
- Interconnects/Routes
- Start Points
- Stop Points

With Grid



Apply Preset Dialog Box

This section provides a quick reference of the **Apply Preset** dialog box. To open the dialog box, click **Apply Preset** in the Property Editor when editing track region PCells. See [Editing Track Region PCells](#).



Option	Description
Library Cell View	The library, cell, and view (or the design) that contains the preset.
Preset	List of presets contained in the specific design. Choose a preset to apply to the selected track region PCell.

Crossing Track Regions

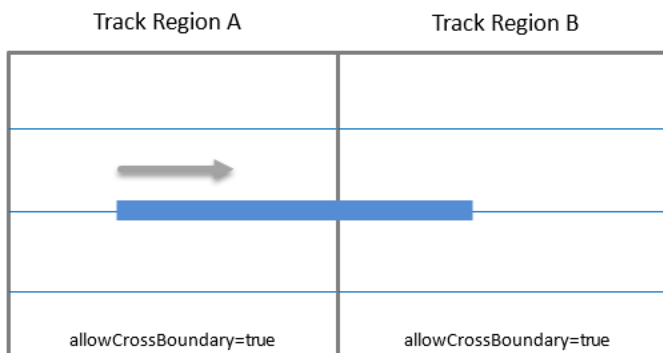
This topic describes the following:

- [Setting allowCrossBoundary](#)
- [Setting leCompatibleTracks](#)
- [Considering Net Support and Track Types](#)
- [Crossing Boundary Types](#)
- [Boundary Crossing and Track Grids](#)

Setting allowCrossBoundary

To permit interconnects to cross global and local track regions, the following requirements must be met:

- The `trackDef() > groups()` definition for the two adjacent track groups must both have the `allowCrossBoundary = true` setting.
- You can also select **Allow Interconnect to Cross Region Boundary** in the **Create Track Region Group Definition** and the **Edit Track Region Group Definition** dialog boxes. See Track Region Assistant. Alternatively, you can specify `allowCrossBoundary = true` when using the `le::createTrackGroupDef Tcl` command.





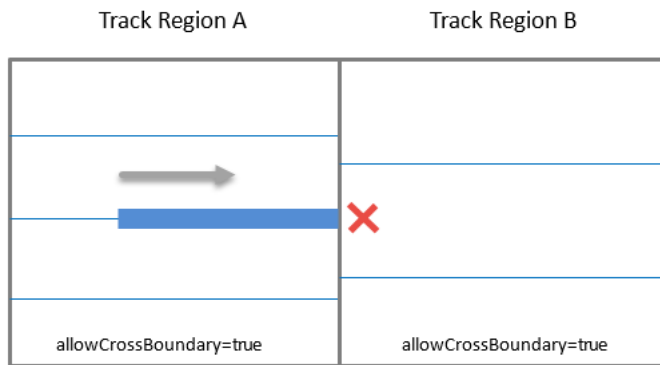
Setting leCompatibleTracks

Even if `allowCrossBoundary = true` for the two adjacent track groups, an interconnect can only continue from one local track region to another local track region or to a global track if tracks in the different regions are available and compatible. You can set the `leCompatibleTracks` Tcl preference to specify the compatibility criteria:

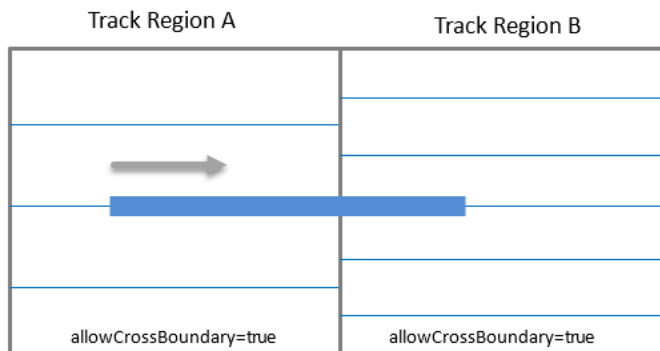
- `sameAttr`: The adjacent track groups must have the same attributes: track direction, layer, width, color, and signal type.
- `sameGroup`: The adjacent track groups must have the same group definition and attributes (track direction, layer, width, color, signal type).

In addition, the center of the compatible track must align with the centerline of the interconnect. If no compatible track is available, try to use a different routing layer.

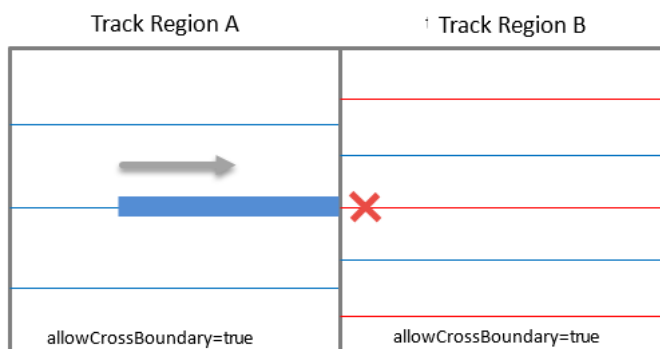
In the following example, `leCompatibleTracks = sameGroup`. Because Track Region A and Track Region B have different track groups, the interconnect cannot cross the boundary.



In the following example, `leCompatibleTracks = sameAttr`. Because Track Region A and Track Region B have the same attributes, the interconnect is able to cross the boundary.



In the following example, `leCompatibleTracks = sameAttr`. Because Track Region A and Track Region B have the different attributes, the interconnect cannot cross the boundary.



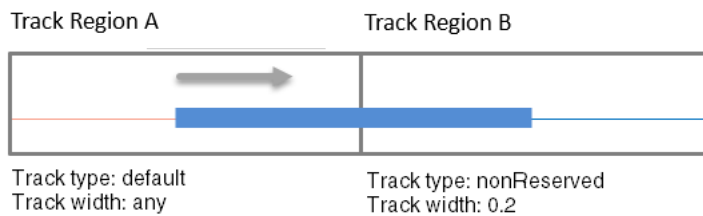
Considering Net Support and Track Types

The following factors also affect track-region compatibility:

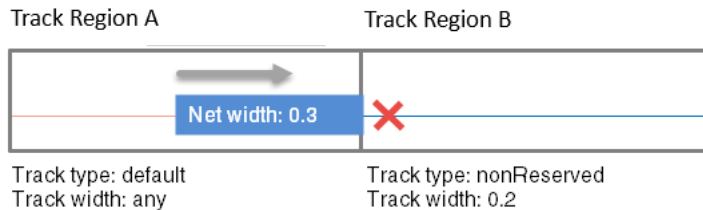
- Adjacent track regions must support the same physical nets. For example, if both track region A and track region B support net P, then net P interconnects can cross from track region A to track region B.
- Adjacent track regions can have different track types (default, reserved, or nonReserved), as long as the attributes remain the same. For track type descriptions, see the [Layer Table](#) in [Track Region PCell Toolbar and Dialog Box Reference](#).

For reserved track types, the Layout Editor checks the `leReservedTrackWidthCheck` Tcl preference setting for track attributes. For nonReserved track types, the Layout Editor checks the interconnect width and track width. The default track type does not have requirements, so it can be compatible with the other track types.

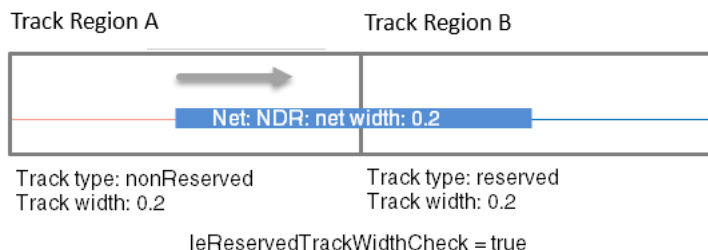
In the following example, Track Region A consists of default track types. Because default track types do not have requirements, the interconnect can cross the boundary to Track Region B.



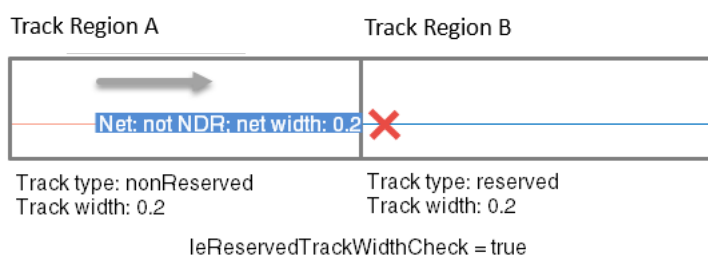
In the following example, Track Region A consists of default track types. However, because the net (interconnect) width is not compatible with the track width in Track Region B, the interconnect cannot cross the boundary.



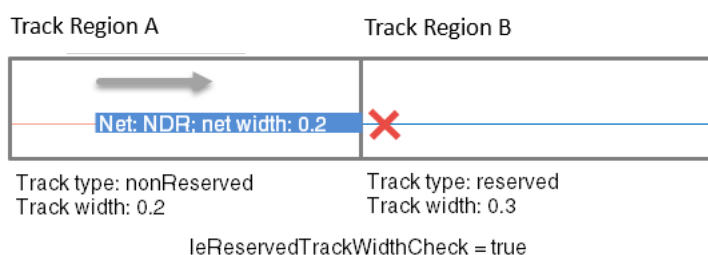
In the following example, the interconnect is on a non-default-rule (NDR) net. Track Region A consists of nonReserved track types, so only interconnects that have the same width as the tracks can be placed in the region. Track Region B consists of reserved track types, and `leReservedTrackWidth` is true, so only NDR nets can route on the tracks and the widths must match. Because these conditions are met, the interconnect can cross the boundary.



The conditions of the following example are similar to the previous example, except the interconnect is not on a non-default-rule (NDR) net. Therefore, the interconnect cannot cross the boundary.



In the following example, the interconnect is on a non-default-rule (NDR) net. Track Region A consists of nonReserved track types, so only interconnects that have the same width as the tracks can be placed in the region. Track Region B consists of reserved track types, and `leReservedTrackWidth` is true, so only NDR nets can route on the tracks and the widths must match. Because the interconnect net width does not match the Track Region B width, the interconnect cannot cross the boundary.

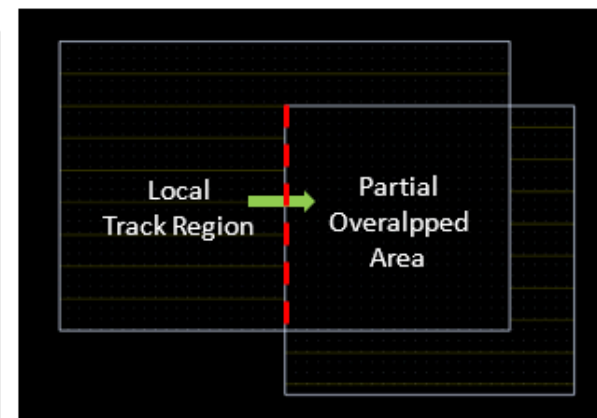


If you are unable to find compatible tracks in a new track region, manually change to another layer and try crossing the boundary.

Crossing Boundary Types

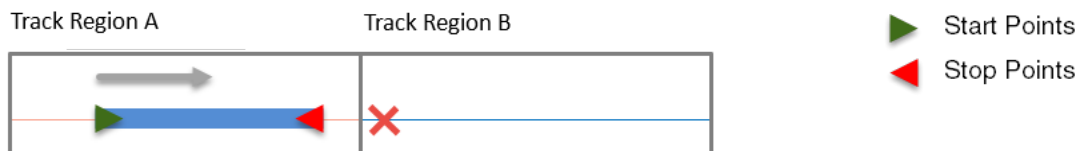
As long as cross-boundary conditions are met, tracks can cross boundaries between the following:

- Local track region and global track region
- Two abutting local track regions
- Nested local track regions (one local track region within another local track region)
- Partially overlapping local track regions



Boundary Crossing and Track Grids

If your design contains track grids, and boundary-crossing is not permitted, the interconnect will stop at the nearest possible end point before the boundary.



If your design contains track enclosures (`-groupEnclosure` in `regions()`), and boundary-crossing is not permitted, the interconnect and vias will be placed within the enclosures.

Crossing Constraint Group Regions

Region-based constraints are foundry or alternate-foundry rules that apply to a specific region (physical layer or layer-purpose pair). When creating routes, you should be aware of the compatibility of the rules across adjacent regions and how they impact route placement.

This topic describes the following:

- [Considerations When Crossing Constraint Group Regions](#)
- [Considerations When Regions Overlap](#)
- [Setting Up Region-Based Constraint Groups](#)

Considerations When Crossing Constraint Group Regions

When routing through different constraint regions, consider the following:

- You can cross constraint group regions when using the **Create Interconnect** and **Create Bus** commands.
- Routes can cross constraint group regions if the rules in the adjacent regions can be honored. The cross-region rules that are considered are:
 - Minimum width (`minWidth`)
 - Width range (`allowedWidthRange`)
 - Minimum spacing (`minSpacing`)
 - Routing direction (`minSpacing`, `preferredRoutingDirection`)
 - Valid routing layers (`preferredRoutingDirection`)
 - Rectangle only (`OnlyRectangleAllowed`)
- The vias you add must also honor the same rules.
- Region-based rules have higher priority over non-region-based rules.

In the following example, a route with 0.3um width can cross into a region that requires a minimum width of 0.2um. However, a route with 0.2um width cannot cross into a region that requires a minimum width of 0.3um.



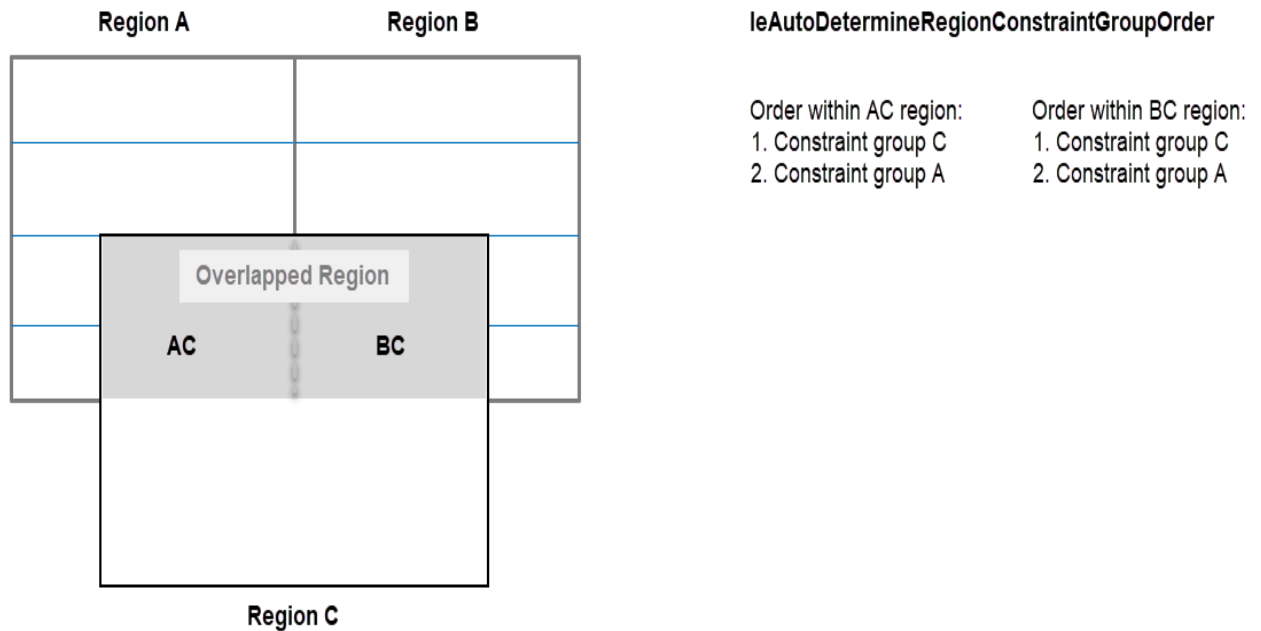
Considerations When Regions Overlap

If a route enters an area where two regions overlap, and the region definitions of the constraint groups match, the rules of both regions can apply, and the order depends on the `leAutoDetermineRegionConstraintGroupOrder` Tcl preference setting:

- `true`: Automatically arranges the order from the highest number of region definition matches to the lowest number of matches.

In the example, in regions AC and BC, constraint group C is first, because it matches two layers.

- `false`: Uses the order defined in the technology file.



Setting Up Region-Based Constraint Groups

To set up region-based constraint groups::

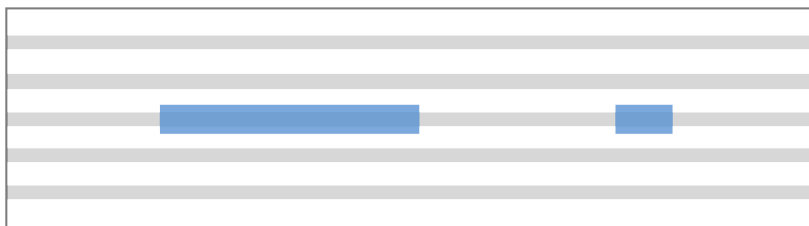
- When defining region-based constraints, use the `-insideLayers` and `-outsideLayers` options. See [Specifying User-Defined Constraint Groups and Constraint Groups](#) Section.
- Set the `leTechFoundryConstraintGroups` Tcl preference to enable region-based constraint groups. Be sure to add the region-based constraints before foundry or alternate foundry constraint groups.

Setting Up Track Grids

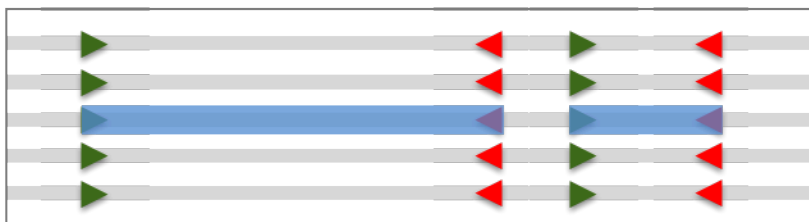
If you want the interconnects to start and end at specific locations on the tracks, be sure to define grids containing permissible starting and stopping points in the track region or track pattern. For track regions, specify the `-grid*` arguments when defining `tracks()`. For track patterns, specify the `-grid*` arguments when using the `le::createTrack` and `le::createTrackPattern` Tcl commands.

By default, if a gridded track region overlaps with a track region without grid, the gridded track region takes priority. (The `leSnapToGrids` Tcl preference is true by default.)

No Grid



With Grid



- Tracks
- Interconnects/Routes
- Start Points
- Stop Points

3

Creating Pins and Terminals

This chapter describes how to create and modify pins and terminals in your designs.

This chapter describes the following:

- [About Pins and Terminals](#)
- [Before You Start](#)
- [Creating and Modifying Terminals](#)
- [Creating Individual Pins Automatically](#)
- [Creating Individual Pins Manually](#)
- [Creating Individual Pins From Existing Shapes](#)
- [Generating Multiple Pins Simultaneously](#)
- [Editing Instance Pins](#)
- [Deleting Generated Pins](#)

About Pins and Terminals

Pins are shapes that are associated with nets and are physical connection points. When creating routes, you are connecting all pins that are assigned the same net.

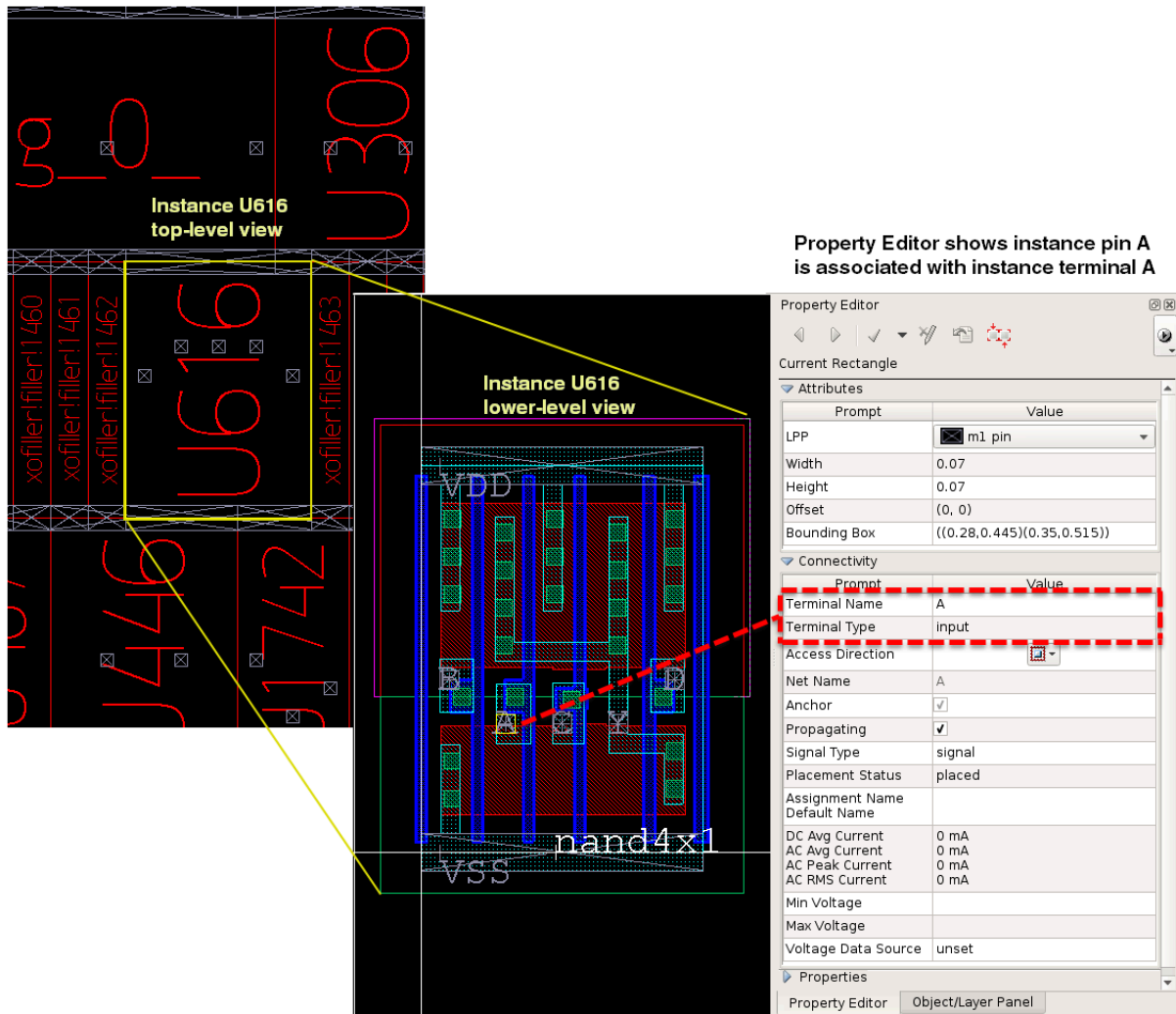


A terminal is a logical group of pins in an instance. Terminals are also associated with nets, and can be one of the following types (or directions):

- input
- output
- inputOutput
- switch
- jumper
- unused
- tristate

For information about the terminal types (or directions), see [Editing Instance Pins](#).

Because terminals are logical and not physical, they are not represented visually on the layout canvas. To view a pin's terminal assignment, you must select the pin and view its terminal assignment in the Property Editor. See [Editing Instance Pins](#).



When you create pins and associate them with terminals, pin connection relationships are automatically determined. To change the pin relationships, see [Defining and Changing Pin Relationships](#).

Before You Start

This topic describes preparations for pin creation:

- [Track Setup](#)
- [Pin Generation for Invisible Layers](#)

Track Setup

If you plan to place pins on tracks, make sure the design contains track regions. In addition, make sure the tracks are visible to facilitate pin placement. For information, see [Track Region Assistant](#).

If you plan to place pins on gridded tracks, make sure the design contains track grid definitions. See [Setting Up Track Grids](#). When placing or stretching pins on gridded tracks, pin shapes will be extended or shortened to fit the defined track grids, as long as the `leSnapRectToTrack` Tcl preference is true.

During pin creation, the pin will snap to the nearest candidate track and inherit the track orientation. If the nearest candidate tracks are horizontal, the pin is snapped to the horizontal track and becomes a horizontal pin. If the nearest candidate tracks are vertical tracks, the pin is snapped to the nearest vertical track and becomes a vertical pin.

If both horizontal and vertical tracks are available, the pin creation command will check the direction in which the pointer is moving when you create the pin. If the X direction is greater than the Y direction, the pin will snap to the nearest horizontal track. If the Y direction is greater than the X direction, the pin will snap to the nearest vertical track.

Pin Generation for Invisible Layers

You can use the `leGeneratePinForInvisibleLayers` Tcl preference to generate pins for objects of invisible layers when the layers are set invisible in the Object/Layer Panel, or to delete generated pins for objects of invisible layers.

Creating and Modifying Terminals

You can define terminals using one of the following methods:

- Define the terminals before you create pins.
- Define terminals as you create pins.

This topic describes how to create terminals before you create pins. The topics are:

- [Creating Terminals](#)
- [Define Terminals Dialog Box](#)

For instructions on how to create terminals as you create pins, see [Creating Individual Pins Automatically](#).

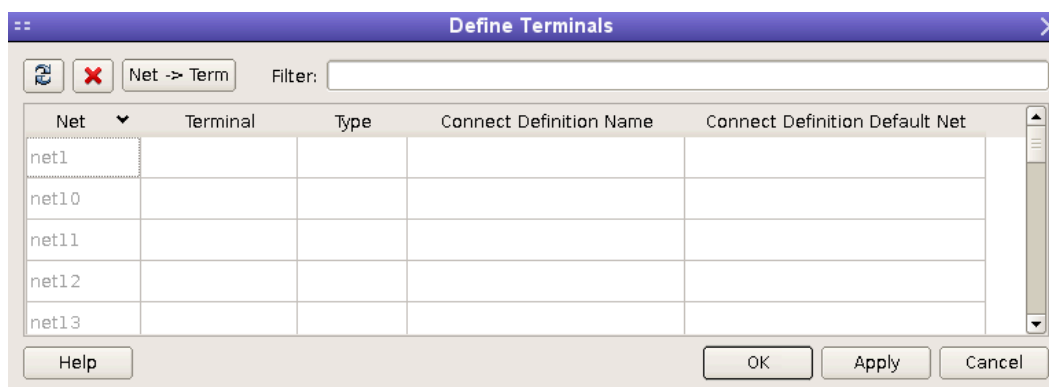
Creating Terminals

This section describes how to create terminals. You can use the same procedure when modifying the terminals.

Make sure you define nets (net names) before creating or modifying terminals.

To create or modify terminals:

1. With your design open in the Layout Editor, choose **Edit > Pin > Define Terminals**. The **Define Terminals** dialog box opens.



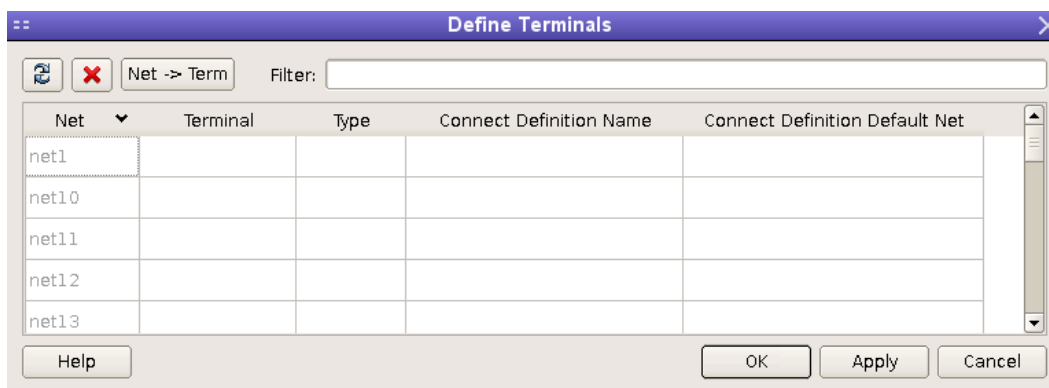
2. In the **Terminal** column, type a scalar name for the terminal. Adding a name to the **Terminal** column creates a terminal for the net.
3. In the **Type** column, choose the terminal's electrical direction.
4. In the **Connect Definition Name** column, type the variable name for the terminal.
5. In the **Connect Definition Default Net** column, type the default net name of the connection definition on the terminal.
6. Click **OK** or **Apply** to save your changes.

Note:

You might be saving changes to nets that are currently hidden by the Filter expression.

Define Terminals Dialog Box

This section provides a quick reference of the **Define Terminals** dialog box. For instructions on creating terminals, see [Creating and Modifying Terminals](#).



Option	Description
 Refresh	Resets the contents of the selected rows (or resets all rows, if none are selected) to the last saved values.
 Delete	Deletes the terminals for the selected rows (or deletes all terminals, if none are selected).
Net -> Term	Sets the terminal name to the Net name for the selected rows. You must select one or more rows first.
Filter	Simplifies the table by displaying only the rows with text that matches the Filter expression.
Net	Displays the existing net names in the design.
Terminal	Displays the name of the terminal for the corresponding net. Entering a terminal name defines a terminal for the net. Type a scalar name for each net that should have a terminal.
Type	Defines the terminal direction: • input • output • inputOutput (default) • switch • jumper • unused • tristate
Connection Definition Name	Defines the connection definition variable name for the terminal.
Connection Definition Default Net	Defines the default net name of connection definition on terminal.

Creating Individual Pins Automatically

You use the **Create > Pin > By Shape** command to create individual pin shapes and the associated terminals.

The command provides two ways to create individual pins:

- **Auto** mode: You select a 0x shape on the canvas and the command creates a new pin based on the selected shape. You can also area-select multiple shapes and create multiple pins simultaneously.
- **Manual** mode: You draw a pin shape manually.

This topic describes how to create individual pins and associated terminals automatically. The topics are:

- [Creating Pin Shapes \(Auto Mode\)](#)
- [Pin Toolbar and Create Pin Dialog Box \(Auto Mode\)](#)
- [Pin Shortcut Menu and Shortcut Keys \(Auto Mode\)](#)

Creating Pin Shapes (Auto Mode)

When you use the **Create > Pin > By Shape** command in the **Auto** mode, you select a 0x shape or area-select multiple pins on the canvas, and create a new pin shape based on the selected shape. During the pin creation process, a new terminal is also created and is automatically associated with the pin. The name of the terminal and the pin are identical.

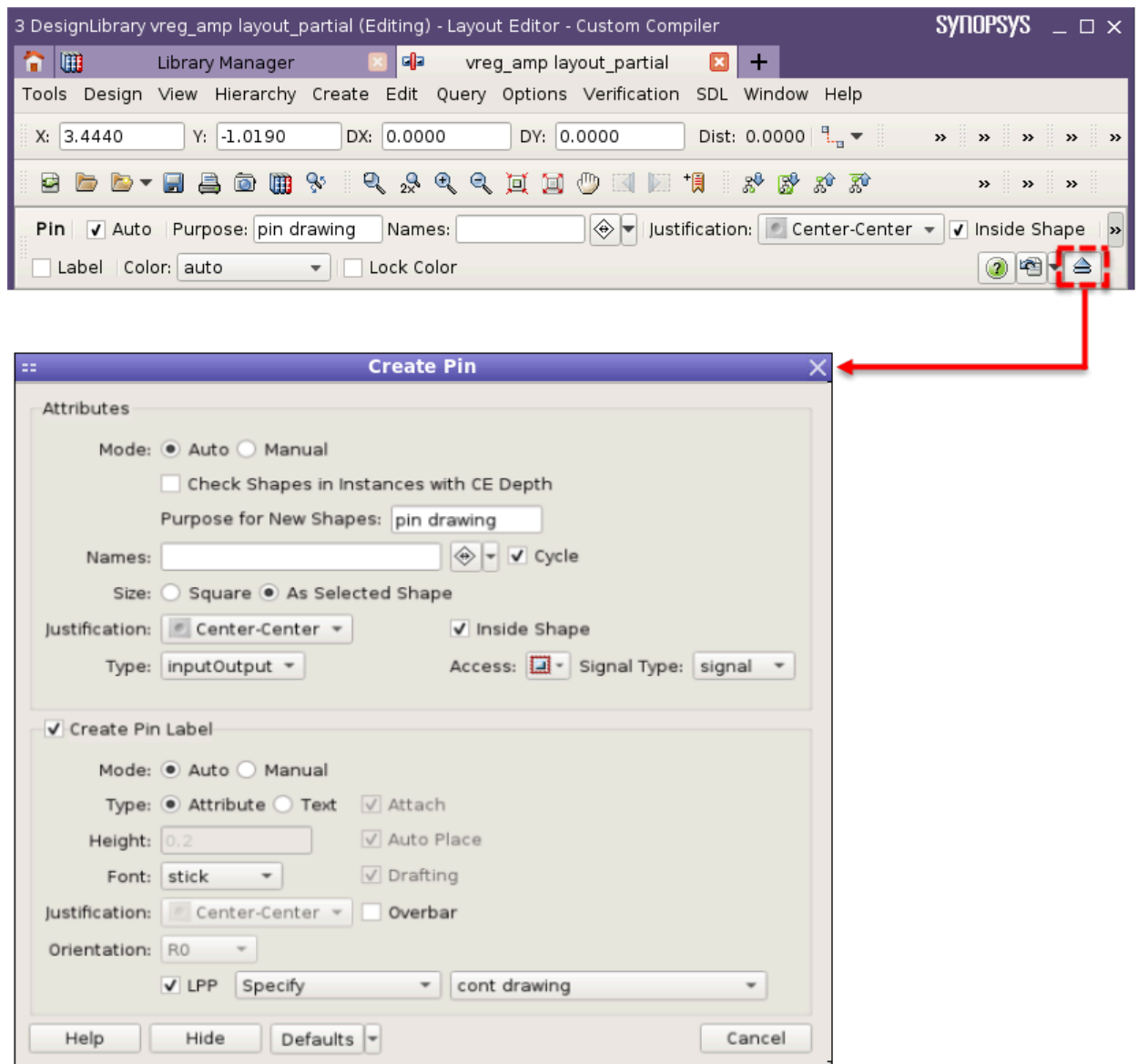
 [Show me](#) an example of creating pins automatically as part of a device routing task.

To create a pin shape and an associated terminal in the Auto mode:

1. With your design open in the Layout Editor, choose **Create > Pin > By Shape** or press Ctrl+P. The **Pin** toolbar appears.
2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Create Pin** dialog box. The dialog box contains additional options.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

You can set the `leCreatePinDialogMode` preference to control whether the dialog box or the toolbar opens by default.



3. Select **Auto** (default).
4. (Optional) Specify the remaining options on the toolbar or in the dialog box. For example, you can select the terminal's electrical type.
5. On the canvas, select a shape or area-select multiple shapes in the design. One or more pin shapes appear under the cursor.

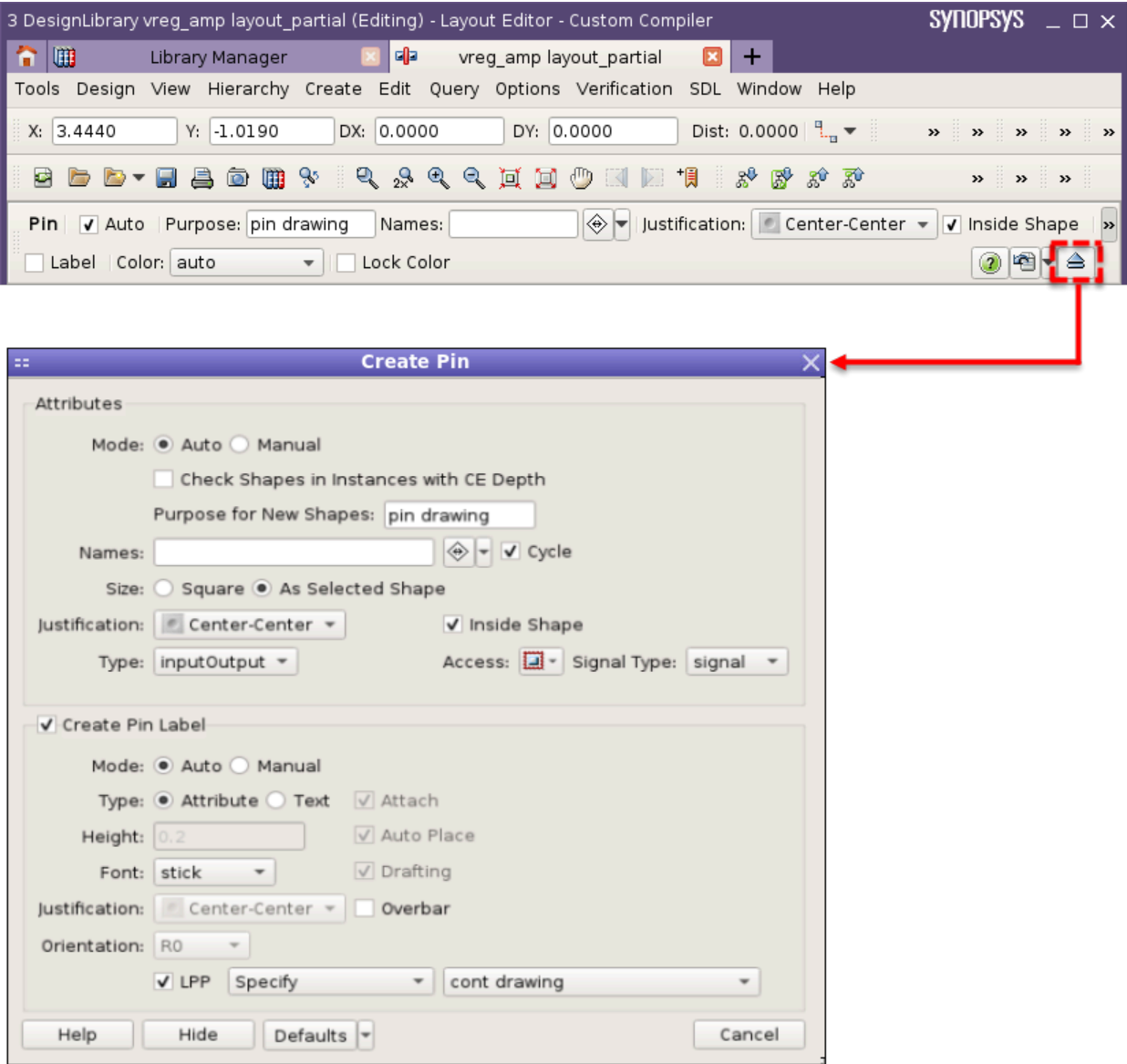
Note:

The selected shapes must be rectangles, polygons, or a path segments.
The shapes must also be associated with a net and is on a routing
(interconnect) layer-purpose pair.

6. Click the location on the canvas to create the pin shapes.
7. (Optional) Right-click the pin shapes and choose a command to rotate, flip, modify the pins' shape, or change the pins' angle.
8. When you are finished, press Esc to quit the command.

Pin Toolbar and Create Pin Dialog Box (Auto Mode)

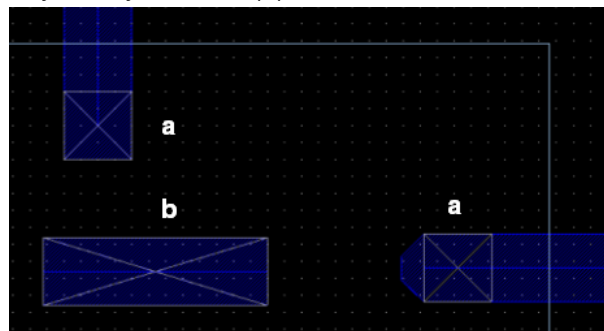
This section provides a quick reference of the **Pin** toolbar and the **Create Pin** dialog box in the **Auto** mode.



Attributes Section

Option	Description
Mode	Pin creation mode: • Auto: Creates pins based on the shapes you select. • Manual: Lets you draw pin shapes manually. When you select Auto , the Justification , Align , Label , and Color options are available.

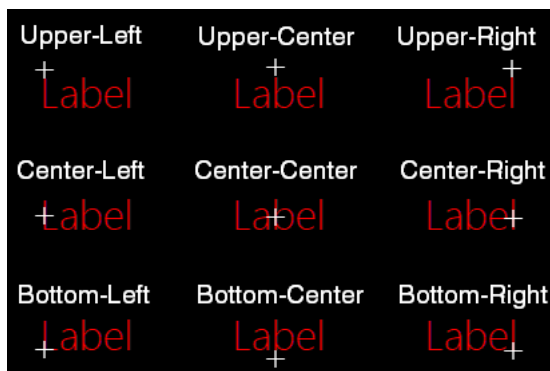
Option	Description
Check Shapes in Instances with CE Depth	Determines if auto pin applies to an active figure in the hierarchy. The depth depends on the depth of the Connectivity Engine(CE) and view level. - False (Default): Only 0x active figures are applied. - True: nx active figures are applied to create auto pin if it connects to net in the top design. Note: This option is hidden if <code>lePinAuto=false</code>. TcL Preference: <code>lePinCheckShapesInInstances</code>
Purpose for New Shapes	Specifies the pin layer's purpose. You can specify multiple purpose names. The order of the names indicate the order in which the Layout Editor looks for the layer-purpose pair during pin generation. For example, suppose you specify <code>pin drawing</code>, and the existing shape is on <code>m2</code>. During pin creation, the Layout Editor will look for <code>m2 pin</code> first. If <code>m2 pin</code> does not exist, it will look for <code>m2 drawing</code>. If <code>m2 drawing</code> does not exist, no pin will be created.
Names	Specifies the net names. If you already selected nets in the Design Navigator, click  to synchronize the selections with the Create Pin toolbar or dialog box. If you selected multiple nets, select Cycle to cycle through the consecutive nets as you place pins one by one on the canvas. Use this option to place pins on unnetted shapes. You can specify a net name, set the remaining options, and then place a pin on an unnetted shape on the layout canvas.
Size	Specifies the size of the new pins: Square: Creates square-shaped pins at path ends when you select the path end points (a). As Selected Shape: Creates pins that are identical to the shapes you fully selected (b).



Option	Description
Color	Assigns a color to the selected shape as follows: • Auto: Automatically assigns a color to the new shape based on the color propagation rule. If no propagation can be applied, then the shape or via is assigned the default color. • <colorName>: Assigns the selected color, <colorName>, to the newly created shape. The available color names are defined in the technology file. • Same: Assumes that nearby shapes have the same color and applies the <code>multiPatternSpacing</code> constraint. This selection is available only if you set the <code>leIncludeColorSame</code> preference to <i>true</i>. • No Color: Does not apply any color to the newly created shape. This selection is available only if you set the <code>leIncludeColorNone</code> preference to <i>true</i>. This option is available only if your design and technology supports color.
Lock Color	Locks the color assignment so that it cannot be changed during color propagation. This option is available only if your design and technology database supports color.
Justification	Determines justification of the pin shape relative to the cursor by Upper-Left, Center-Left, Lower-Left, Upper-Center, Center-Center, Lower-Center, Upper-Right, Center-Right, or Lower-Right. The default is Center-Center.
Inside Shape	Determines whether the pin shape is aligned with the active figure.
Type	(Dialog box only) Sets the terminal electrical type: input, output, inputOutput, switch, jumper, unused, or tristate. The default is inputOutput. If the terminal already exists, changing the direction while creating the pin modifies the terminal.
Access	(Dialog box only) Specifies the directions from which you can route a connection to the pin. The access direction can be All, None, Top, Bottom, Left, or Right.
Signal Type	(Dialog box only) Specifies the pin signal type (signal type for the net associated with the pin). The available signal types are signal, power, ground, clock, tieoff, tieHi, tieLo, analog, scan, or reset.

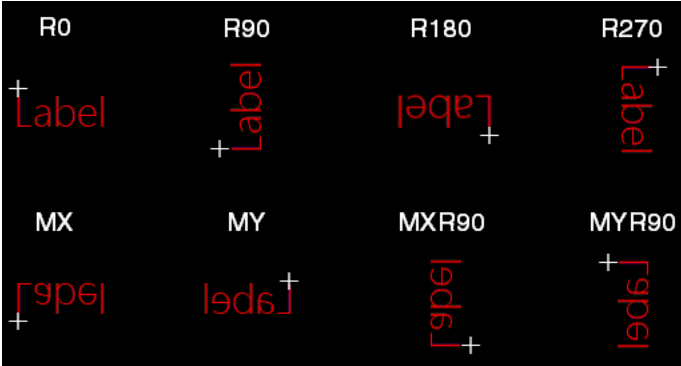
Create Pin Label Section

Option	Description
Label/ Create Pin Label	Creates pin labels that show the terminal name. When selected, you can specify one of the following methods for applying the label style: • Auto: Allows the tool to determine the label style. • Manual: Allows you to specify the label style. The label styles include label type, font height, addition of overbar, orientation, and justification. Default: not selected Tcl preferences: <code>leGeneratePinCreatePinLabel</code>, <code>leGeneratePinLabelStyle</code> The preference <code>leGeneratePinLabelTopMetalOnly</code> creates a pin label on top metal layer for a connected island. The connected island is extracted and pin shapes with the same LPP are merged to generate one pin label.
Type	Label type: Text , or Attribute .
Height	Sets the text height in user units.
Font	Sets the font face. The available font types are euroStyle , gothic , math , roman , script , stick , fixed , swedish , and milSpec .
Justification	(Dialog box only) Positions the text relative to the origin. The default is Center-Center. To adjust the label position after you place the pin, use the Edit > Move command.

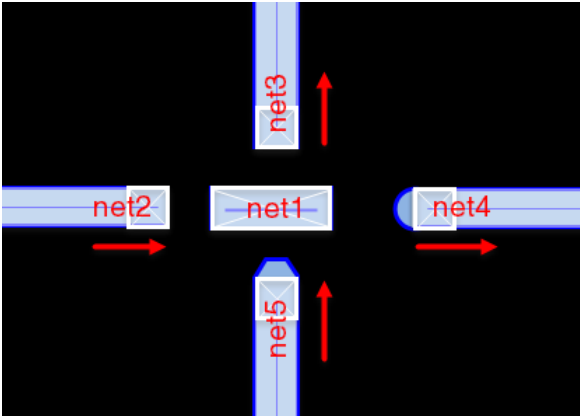


Option	Description
--------	-------------

Orientation Rotates the label counterclockwise about the origin, flips the label about the x- or y- axis, or rotates and flips the label. Choose one of the following orientations:



The default is **R0**.
For pins on paths or path segments, the label text size is ½ of the path width. The label orientation is determined by the path's centerline direction. The justification (or position) is affected when the label is created on the path's end.



Auto Place When enabled, creates the attribute label automatically with the origin at the center of the pin shape bounding box, snapped to the closest manufacturing grid using current label options.

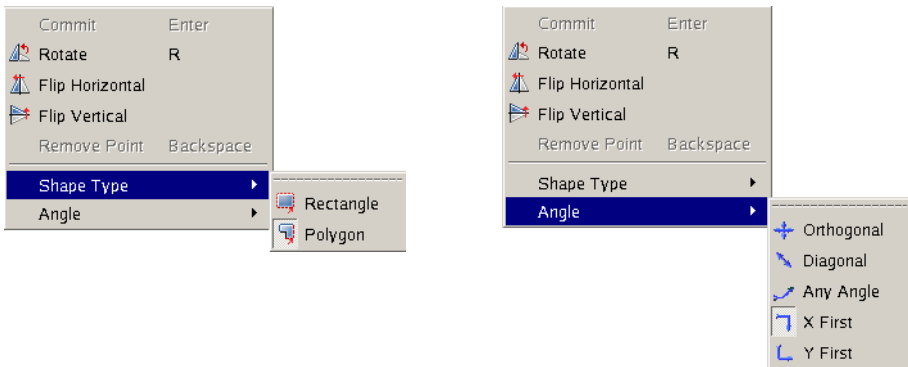
Drafting (Dialog box only) Limits the possible orientations to R0 or R90 to ensure consistent labeling in the design.

- Choosing **R0**, **R180**, **MX**, or **MY** will always display labels in the R0 orientation.
- Choosing **R90**, **R270**, **MXR90**, or **MYR90** will always display labels in the R90 orientation.

Option	Description
Overbar	(Dialog box only) When enabled, displays an overline above the text. You might want to add an overbar to indicate reverse logic in your design. To create the overbar, you must type an underscore (<code>_</code>) in the Names box. For example, if you type <code>AB_CD</code> in the Names box, the label will appear as follows: 
LPP	When enabled, creates the label based on one of the following options: • Specify: The specified layer-purpose pair. • Technology Library: The associated technology library. • Map: The file that defines the mapping of layer-purpose pairs for shapes and text labels. See Defining the LPP Mapping File. • Purpose: The list of purpose names. The command will look for LPPs in the technology library that have the pin shape layer and specified purpose names in this list. If no valid LPP is found, the LPP of the pin shape is used. Tcl preference: <code>leTextPurposeList</code> When disabled, creates the label on the same layer as the pin shape, using the purpose defined by the <code>lePinPurposeList</code> option. The default purpose is <code>text</code>. If no valid purposes in <code>lePinPurposeList</code>, use the shape LPP for pin labels.

Pin Shortcut Menu and Shortcut Keys (Auto Mode)

While the **Create > Pin > By Shape** command is active, you can right-click to display the Pin shortcut menu. This section provides a quick reference of the Pin shortcut menu and shortcut keys. For instructions on how to create individual pins automatically, see [Creating Pin Shapes \(Auto Mode\)](#).



Command	Purpose	Default Shortcut Key
Commit	Completes the pin (available when the shape is Polygon).	Enter
Rotate	Rotates the shape 90° counter-clockwise. To rotate 180° or 270°, repeat the command.	R
Flip Horizontal	Mirrors the shape around the X-axis before placement.	
Flip Vertical	Mirrors the shape around the Y-axis before placement.	
Remove Point	Deletes the previous point you entered to define the pin shape (available when the shape is Polygon).	Backspace
Shape Type	Specifies a rectangular or polygonal pin shape.	
Angle	Controls the orientation of edges (available when the shape is Polygon). See Angle Mode for Edges and Centerlines.	

Creating Individual Pins Manually

You use the **Create > Pin > By Shape** command to create individual pin shapes and the associated terminals. The command provides two ways to create individual pins:

- **Auto** mode: You select a 0x shape on the canvas and the command creates a new pin based on the selected shape.
- **Manual** mode: You draw a pin shape manually.

This topic describes how to create individual pins manually. The topics are:

- [Creating a Single Pin Shape on a Single Terminal \(Manual Mode\)](#)
- [Creating Multiple Pin Shapes on One Terminal \(Manual Mode\)](#)
- [Creating Identical Pins on Different Terminals \(Manual Mode\)](#)
- [Pin Toolbar and Create Pin Dialog Box \(Manual Mode\)](#)
- [Pin Shortcut Menu and Shortcut Keys \(Manual Mode\)](#)

To create individual pin shapes automatically, see [Creating Individual Pins Automatically](#).

To create multiple pins simultaneously, see [Generating Multiple Pins Simultaneously](#).

Creating a Single Pin Shape on a Single Terminal (Manual Mode)

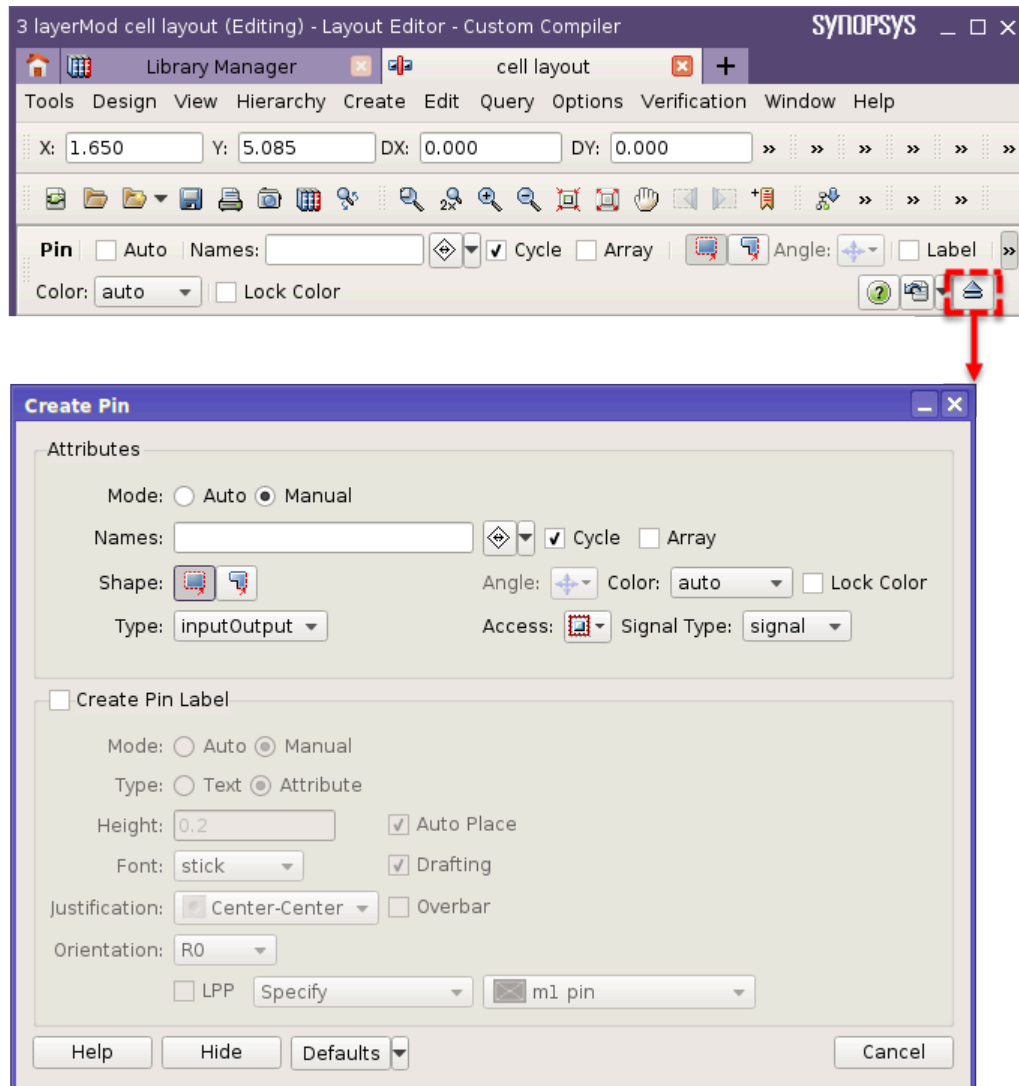
When you use the **Create > Pin > By Shape** command in the manual mode (**Auto** mode is turned off), you can draw a rectangle or polygon pin shape on the canvas. You can also specify an existing terminal name to associate the new shape to that terminal. If the terminal does not exist, the command will create the terminal during the pin creation process.

To create a pin shape on a terminal:

1. With your design open in the Layout Editor, choose **Create > Pin > By Shape** or press Ctrl+P. The **Pin** toolbar appears.
2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Create Pin** dialog box. The dialog box contains additional options.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

You can set the `leCreatePinDialogMode` preference to control whether the dialog box or the toolbar opens by default.



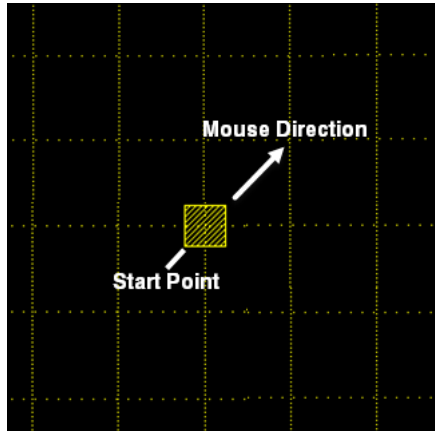
3. Clear the **Auto** check box.
4. In the **Name** box, type a terminal name. If the terminal does not exist, the command will create it.

Note:

The terminal name must comply with CDBA naming standards.

5. (Optional) Specify the remaining options on the toolbar or in the dialog box. For example, you can select the terminal's electrical type. See [Pin Toolbar and Create Pin Dialog Box \(Manual Mode\)](#).
6. On the layout canvas, draw the pin shape.

- Click and drag to draw a rectangular pin shape. Release the mouse button to complete the shape. As you do so, the Layout Editor automatically snaps the pin to the nearest applicable track.



If only horizontal or vertical tracks are available, the center point of the pin will snap to the nearest applicable horizontal or vertical track.

- Click multiple points to draw a polygon pin shape. Double-click to complete a polygon pin shape.
7. (Optional) Right-click the pin shape and choose a command to rotate, flip, modify the pin's shape, or change the pin's angle. See [Pin Shortcut Menu and Shortcut Keys \(Manual Mode\)](#).
 8. When you are finished, press Esc to quit the command.

Creating Multiple Pin Shapes on One Terminal (Manual Mode)

You can create multiple pins on the same terminal. To do this:

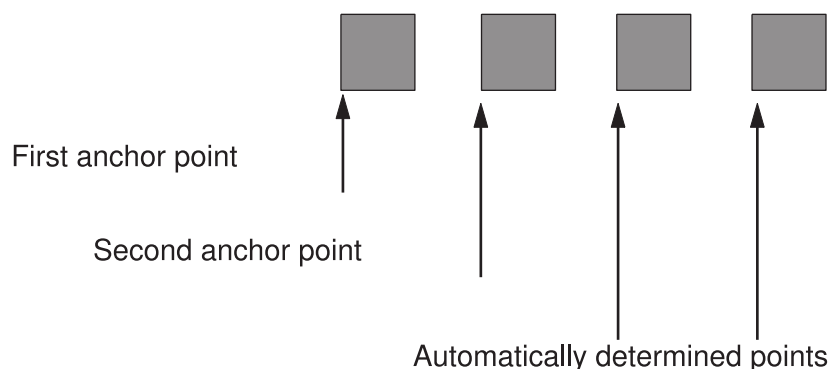
- Follow the instructions in [Creating a Single Pin Shape on a Single Terminal \(Manual Mode\)](#).
- Repeat [Step 6](#) and [Step 7](#) to draw additional shapes on the same terminal.

The new pin shapes share a strongly connected relationship. To change this relationship, use the **Edit > Connectivity > Pin Relationship** command.

Creating Identical Pins on Different Terminals (Manual Mode)

You can create identical pins on different terminals. To do this:

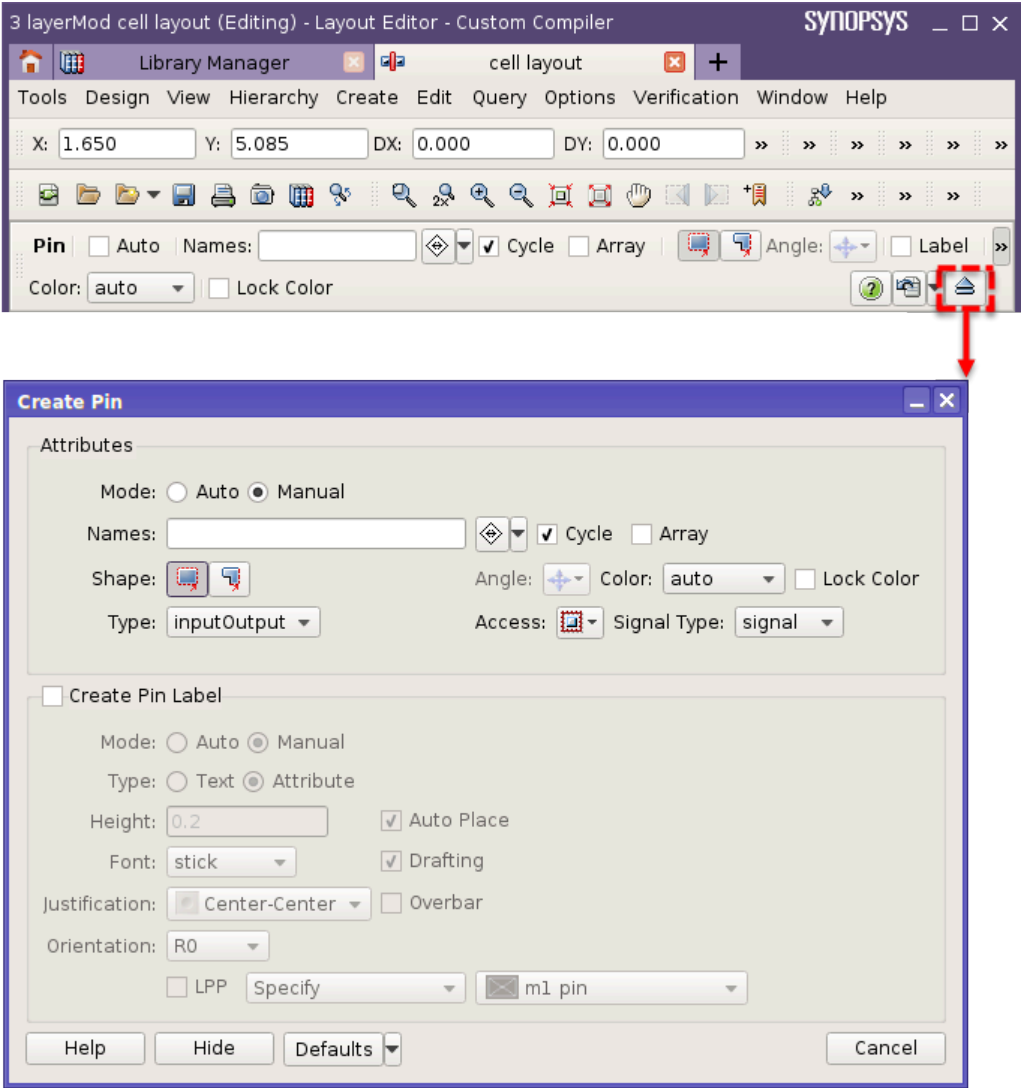
- Follow the instructions in [Creating a Single Pin Shape on a Single Terminal \(Manual Mode\)](#).
- At [Step 4](#), type multiple terminal names in the **Names** box. Add a character space after each terminal name.
- After you draw the first pin shape ([Step 6](#) and [Step 7](#)), notice that the command removes the first terminal name in the **Names** box. Click another location on the canvas to place the next pin. Repeat until the last terminal name is removed from the **Names** box.
- If you selected **Array**, click a location on the canvas to place the lower-left corner of the first pin. The command then prompts you for a vector and automatically places the remaining pin shapes in an array.



Pin Toolbar and Create Pin Dialog Box (Manual Mode)

This section provides a quick reference of the **Pin** toolbar and the **Create Pin** dialog box in the manual mode. For instructions on how to create individual pins manually, see one of the following sections:

- [Creating a Single Pin Shape on a Single Terminal \(Manual Mode\)](#)
- [Creating Multiple Pin Shapes on One Terminal \(Manual Mode\)](#)



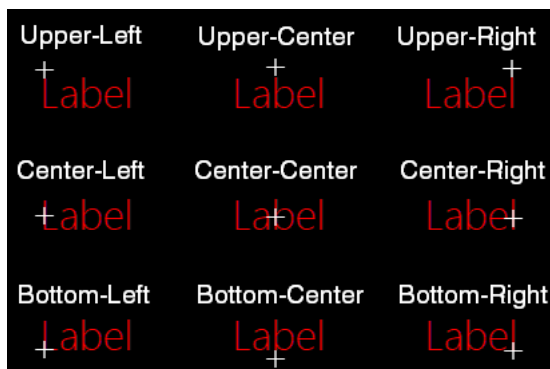
Attributes Section

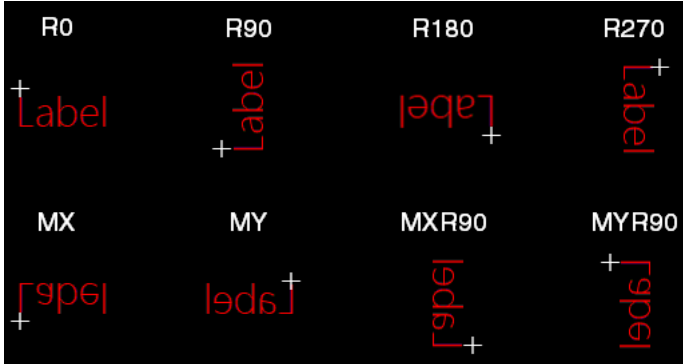
Option	Description
Mode	The pin creation mode: • Auto: Creates pins based on the shapes you select. • Manual: Lets you draw pin shapes manually. When you select Manual, the Names, Cycle, Array, Shape, Angle, and Color options are available. For information about the Auto mode, see Pin Toolbar and Create Pin Dialog Box (Auto Mode).

Option	Description
Names	Specifies one or more existing terminals in the design. You can use bus, bundle, or complex notation to quickly specify several related terminals. Separate the names with character spaces.
Cycle	When enabled, deletes each name from the Names box as you create a corresponding pin shape.
Array	After you place the first pin shape, automatically places the remaining named shapes in an array.
Shape	Specifies a rectangular or polygonal pin shape.
Angle	For polygonal pins only, sets the orientation of the polygon edges. See Angle Mode for Edges and Centerlines.
Color	Assigns a color to the selected shape as follows: • Auto: Automatically assigns a color to the new shape based on the color propagation rule. If no propagation can be applied, then the shape or via is assigned the default color. • <colorName>: Assigns the selected color, <colorName>, to the newly created shape. The available color names are defined in the technology file. • Same: Assumes that nearby shapes have the same color and applies the <code>multiPatternSpacing</code> constraint. This selection is available only if you set the <code>leIncludeColorSame</code> preference to <i>true</i>. • No Color: Does not apply any color to the newly created shape. This selection is available only if you set the <code>leIncludeColorNone</code> preference to <i>true</i>.
Lock Color	Locks the color assignment so that it cannot be changed during color propagation. This option is available only if your design and technology database supports color.
Type	(Dialog box only) Sets the terminal electrical type by input, output, inputOutput, switch, jumper, unused, or tristate. The default is inputOutput. If the terminal already exists, changing the direction while creating the pin modifies the terminal.
Access	(Dialog box only) Specifies the directions from which you can route a connection to the pin. The access direction can be All, None, Top, Bottom, Left, or Right.
Signal Type	(Dialog box only) Specifies the pin signal type (signal type for the net associated with the pin). The available signal types are signal, power, ground, clock, tieoff, tieHi, tieLo, analog, scan, or reset.

Create Pin Label Section

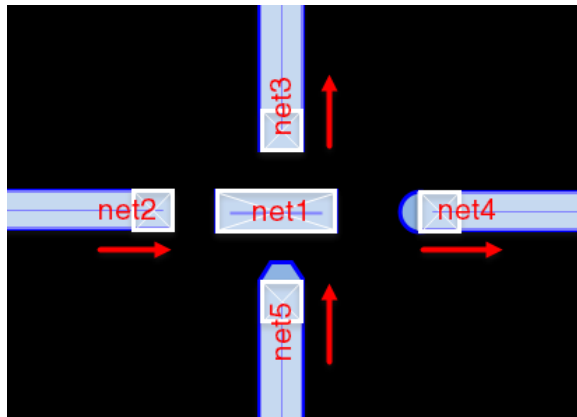
Option	Description
Label/ Create Pin Label	Creates pin labels that show the terminal name. When selected, you can specify one of the following methods for applying the label style: • Auto: Allows the tool to determine the label style. • Manual: Allows you to specify the label style. The label styles include label type, font height, addition of overbar, orientation, and justification. Default: not selected Tcl preferences: <code>leGeneratePinCreatePinLabel</code>, <code>leGeneratePinLabelStyle</code> The preference <code>leGeneratePinLabelTopMetalOnly</code> creates a pin label on top metal layer for a connected island. The connected island is extracted and pin shapes with the same LPP are merged to generate one pin label.
Type	Label type: Text , or Attribute .
Height	Sets the text height in user units.
Font	Sets the font face. The available font types are euroStyle , gothic , math , roman , script , stick , fixed , swedish , and milSpec .
Justification	(Dialog box only) Positions the text relative to the origin. The default is Center-Center. To adjust the label position after you place the pin, use the Edit > Move command.



Option	Description
Orientation	Rotates the label counterclockwise about the origin, flips the label about the x- or y- axis, or rotates and flips the label. Choose one of the following orientations: 

The default is **R0**.

For pins on paths or path segments, the label text size is $\frac{1}{2}$ of the path width. The label orientation is determined by the path's centerline direction. The justification (or position) is affected when the label is created on the path's end.



Auto Place	When enabled, creates the attribute label automatically with the origin at the center of the pin shape bounding box, snapped to the closest manufacturing grid using current label options.
Drafting	(Dialog box only) Limits the possible orientations to R0 or R90 to ensure consistent labeling in the design. - Choosing R0, R180, MX, or MY will always display labels in the R0 orientation. - Choosing R90, R270, MXR90, or MYR90 will always display labels in the R90 orientation.

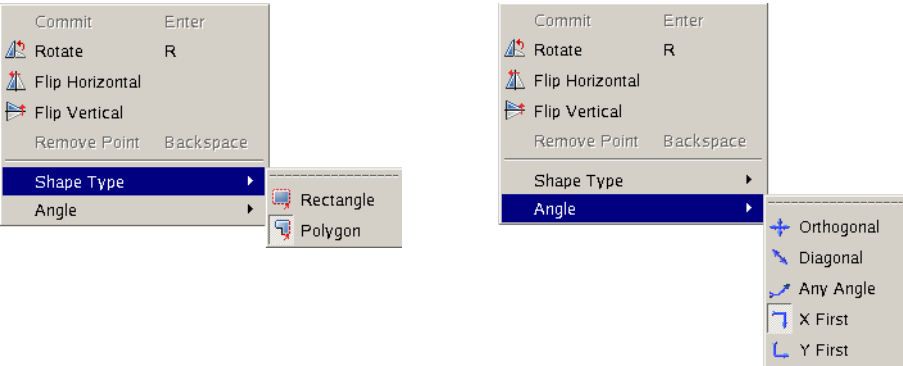
Option	Description
Overbar	(Dialog box only) When enabled, displays an overline above the text. You might want to add an overbar to indicate reverse logic in your design. To create the overbar, you must type an underscore (<code>_</code>) in the Names box. For example, if you type <code>AB_CD</code> in the Names box, the label will appear as follows: 
LPP	When enabled, creates the label based on one of the following options: • Specify: The specified layer-purpose pair. • Technology Library: The associated technology library. • Map: The file that defines the mapping of layer-purpose pairs for shapes and text labels. See Defining the LPP Mapping File. • Purpose: The list of purpose names. The command will look for LPPs in the technology library that have the pin shape layer and specified purpose names in this list. If no valid LPP is found, the LPP of the pin shape is used. Tcl preference: <code>leTextPurposeList</code> When disabled, creates the label on the same layer as the pin shape, using the purpose defined by the <code>lePinPurposeList</code> option. The default purpose is <code>text</code>. If no valid purposes in <code>lePinPurposeList</code>, use the shape LPP for pin labels.

Pin Shortcut Menu and Shortcut Keys (Manual Mode)

While the **Create > Pin > By Shape** command is active, you can right-click to display the Pin shortcut menu. This section provides a quick reference of the Pin shortcut menu and shortcut keys.

For instructions on how to create individual pins manually, see one of the following sections:

- [Creating a Single Pin Shape on a Single Terminal \(Manual Mode\)](#)
- [Creating Multiple Pin Shapes on One Terminal \(Manual Mode\)](#)



Command	Purpose	Default Shortcut key
Commit	Completes the pin (available when the shape is Polygon).	Enter
Rotate	Rotates the shape 90 degrees counter-clockwise. To rotate 180 or 270 degrees, repeat the command.	R
Flip Horizontal	Mirrors the shape around the X-axis before placement.	
Flip Vertical	Mirrors the shape around the Y-axis before placement.	
Remove Point	Deletes the previous point you entered to define the pin shape (available when the shape is Polygon).	Backspace
Shape Type	Specifies a rectangular or polygonal pin shape.	
Angle	Controls the orientation of edges (available when the shape is Polygon). See Angle Mode for Edges and Centerlines.	

Creating Individual Pins From Existing Shapes

This topic describes how to create individual pins from existing shapes. The topics are:

- [About the Create > Pin > From Selection Command](#)
- [Creating Pins From Existing Shapes](#)
- [Pins From Selection Toolbar and Create Pins From Selection Dialog Box](#)

To create multiple pins simultaneously, see [Generating Multiple Pins Simultaneously](#).

About the Create > Pin > From Selection Command

The **Create > Pin > from Selection** command lets you select existing shapes on the layout canvas and convert them into new strongly connected pins and terminals. The shapes must already have nets assigned.

You must fully select the shapes while the command is active. The command ignores partially selected shapes.

If the net already has a terminal, the command preserves that terminal. Otherwise, the command generates a new terminal with the same name as the net. If that terminal name would not be unique, the command appends a unique numeric suffix to the net name, such as `mynet_1`.

You can create pins from the following existing shapes:

- Rectangles
- Polygons
- Ellipses
- Paths
- Path Segments

You cannot create pins from the following shapes:

- Vias
- Shapes without a net name
- Existing pins shapes
- Shapes that are not on a routing layer-purpose pair

Note:

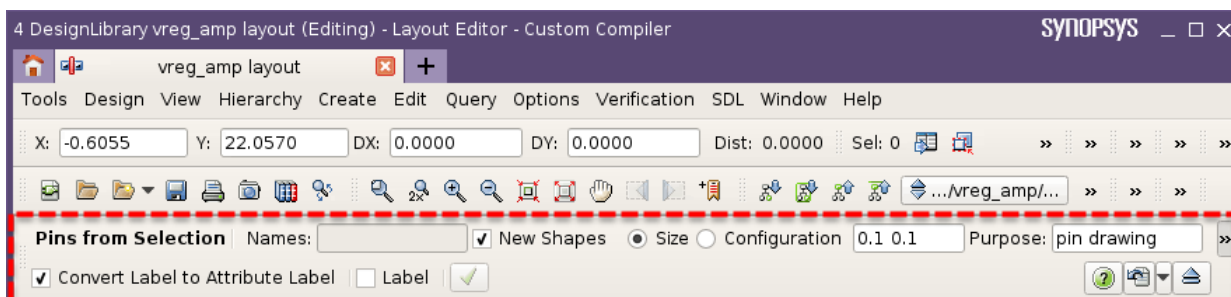
This command does not support infix mode.

The newly created pin shapes will share a strongly connected relationship. To modify a connection relationship, use the **Edit > Pin > Change Pin Relationship** command after you create the pins. See [Defining and Changing Pin Relationships](#).

Creating Pins From Existing Shapes

To create pins from existing shapes:

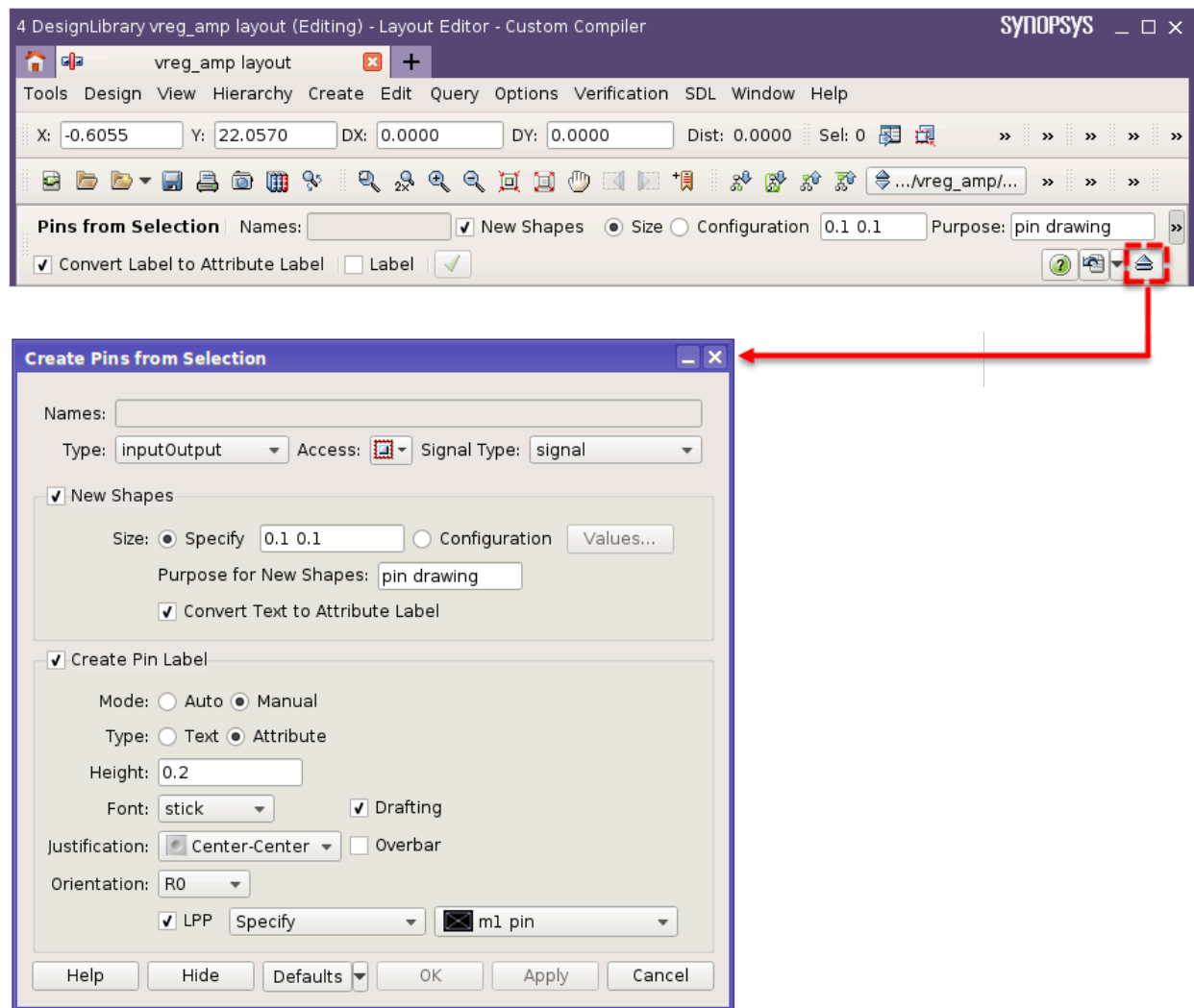
1. With your design open in the Layout Editor, choose **Create > Pin > From Selection**. The **Pins from Selection** toolbar appears.



2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Create Pins from Selection** dialog box. The dialog box contains additional options.
To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.
3. On the layout canvas, select one or more shapes that are assigned nets. You can select shapes that belong to different nets. As you select a shape, the shape's net name appears in the **Names** box.
4. (Optional) Specify the remaining options on the toolbar or in the dialog box. For example, you can select the terminal's electrical type. See [Pins From Selection Toolbar and Create Pins From Selection Dialog Box](#).
5. Click **Apply**  on the toolbar or in the dialog box to create the pins and create (or modify) the terminals.
6. (Optional) Repeat [Step 3](#) through [Step 5](#) to create new pins from selected shapes.
7. When you are finished, press Esc to quit the command.

Pins From Selection Toolbar and Create Pins From Selection Dialog Box

This section provides a quick reference of the **Pins from Selection** toolbar and the **Create Pins from Selection** dialog box. For instructions on how to create pins from selected shapes, see [Creating Pins From Existing Shapes](#).



General Options

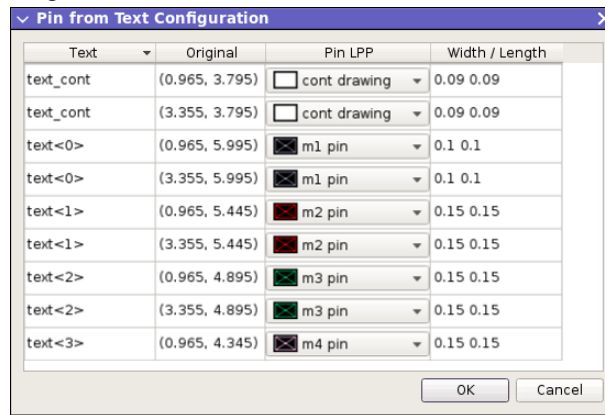
Option	Description
Names	Shows a mix of the existing terminal names and the terminal names to be created (based on the net names of the selected set). This is a read-only text box for providing additional information.
Type	(Dialog box only) Sets the terminal electrical type by input , output , inputOutput , switch , jumper , unused , or tristate . The default is inputOutput . If the terminal already exists, changing the direction while creating the pin modifies the terminal.

Option	Description
Access	(Dialog box only) Specifies the directions from which you can route a connection to the pin. The access direction can be All , None , Top , Bottom , Left , or Right .
Signal	(Dialog box only) Specifies the pin signal type (signal type for the net associated with the pin). The available signal types are signal , power , ground , clock , tieoff , tieHi , tieLo , analog , scan , or reset .

New Shapes

Specifies whether to create new oaRect shapes when a pin is created from a text.

Option	Description
Size: Specify	Specifies the width and height for new shapes.
Size: Configuration	Lets you specify the pin layer-purpose pair (LPP), width, and length. On the toolbar, select Configuration, type the width and length values, and type the name of the layer-purpose pair. The width and length values must be separated by a character space. In the dialog box, click Values to open the Pin from Text Configuration dialog box and specify the pin layer, width, and length information.

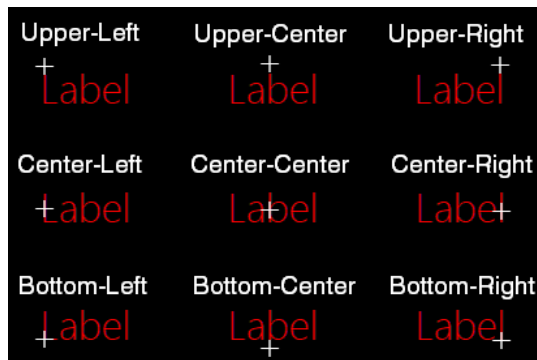


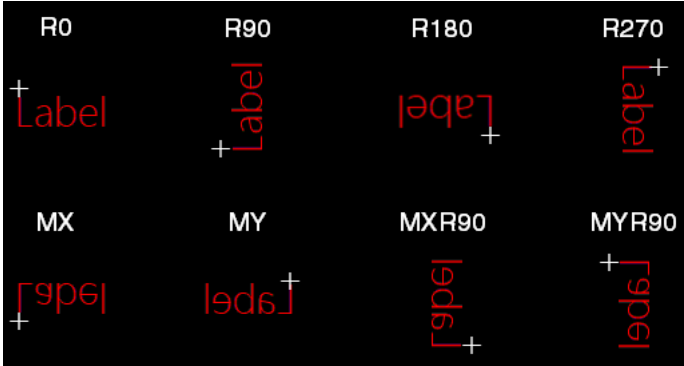
Purpose for New Shapes Specifies the pin layer's purpose. You can specify multiple purpose names. The order of the names indicate the order in which the Layout Editor looks for the layer-purpose pair during pin creation. For example, suppose you specify `pin drawing`, and the existing shape is on `m2`. During pin creation, the Layout Editor will look for `m2 pin` first. If `m2 pin` does not exist, it will look for `m2 drawing`. If `m2 drawing` does not exist, no pin will be created.

Option	Description
Convert Label to Attribute Label	Converts labels to attribute labels. See Creating Attribute Labels. Tcl preference: <code>lePinLabelTextType</code>

Create Pin Label

Option	Description
Label/Create Pin Label	A toggle that enables the text attributes you can apply. When enabled, automatically creates an attribute label showing the terminal name. The label origin is the center of the pin shape.
Height	(Dialog box only) Sets the text height in user units.
Font	(Dialog box only) Sets the font face. The available font types are euroStyle , gothic , math , roman , script , stick , fixed , swedish , and milSpec .
Justification	(Dialog box only) Positions the text relative to the origin. The default is Center-Center . To adjust the label position after you place the pin, use the Edit > Move command.



Option	Description
Orientation	Rotates the label counterclockwise about the origin, flips the label about the x- or y- axis, or rotates and flips the label. Choose one of the following orientations:  The default is R0.
Drafting	(Dialog box only) Limits the possible orientations to R0 or R90 to ensure consistent labeling in the design. - Choosing R0, R180, MX, or MY will always display labels in the R0 orientation. - Choosing R90, R270, MXR90, or MYR90 will always display labels in the R90 orientation.
Overbar	(Dialog box only) When enabled, displays an overline above the text. You might want to add an overbar to indicate reverse logic in your design. To create the overbar, you must type an underscore (_) in the Names box. For example, if you type AB_CD in the Names box, the label will appear as follows: 
LPP	(Dialog box only) When enabled, creates the label based on one of these options: Specify: The specified layer purpose pair. Technology Library: The associated technology library. Map: The file that defines the mapping of layer purpose pairs for shapes and text labels. See Defining the LPP Mapping File. When disabled, creates the label on the same layer as the pin shape, using the purpose defined by the <code>lePinPurposeList</code> option. The default purpose is <code>text</code>.

Generating Multiple Pins Simultaneously

Instead of creating pins individually, you can create and place multiple pins simultaneously throughout the design or within a specified region.

This topic describes the following:

- [About Multiple-Pin Generation](#)
- [Understanding Automatic Pin Placement](#)
- [Preparing the Layer Mapping File](#)
- [Before You Start to Generate Pins](#)
- [Generating Pins Within the Layout](#)
- [Generating Pins for Specified Top Layers](#)
- [Generating Layout Pins at the Library Level](#)
- [Generate Pins Toolbar and Dialog Box Reference](#)
- [Generate Layout Pins Dialog Box Reference](#)

For the description of pins and the instructions for creating pins individually, see [Creating Individual Pins Automatically](#).

About Multiple-Pin Generation

You can create and place (or generate) multiple pins simultaneously throughout the design or within a specified region.

Use one of the following methods to generate multiple pins:

Method	Description
Layout Editor: Create > Pin > Generate Pins	Generates pins while you are working in a design layout. See Generating Pins Within the Layout .
Library Manager: File > Create Layout Pins	Generates pins at the library level. See Generating Layout Pins at the Library Level .

The pin generation operation relies on nets or labels.

- *When using nets:* (Layout Editor method only) The Layout Editor looks for 0x shapes and will generate pin shapes for 0x shapes that belong to nets but are not terminals or instance pins with net. For instance pins, the Layout Editor will create a pin shape.

When in the SDL mode, the operation generates pins for nets that are terminals in the schematic.

- *When using labels:* (Both methods) The Layout Editor looks for shapes that overlap with 0x labels. If a shape is not on 0x, or if the [Assign to Shape](#) option is not selected, the Layout Editor will generate a pin shape on 0x.

You can select **For Logic Terms Only** to generate pins based on existing layout or schematic terminals. Alternatively, set the `leGeneratePinLECheckLogicTerms` (layout) or `leGeneratePinCheckLogicTerms` (schematic) Tcl preference to `true`. When the preference is `false`, pins will be generated from all text and nets.

Pin generation supports valid bus bit names and scalar names for nets and terminals. The bus bit label must have the format `BaseName<n>`, where `n` is the non-negative bit index. For example, `A<0>` is the label for bus bit 0, and the base name is `A`.

In both cases, the newly created pin shape uses the same layer name as the existing shape and uses the purpose name you specify in the Generate Pins or Generate Layout Pins dialog box.

When two newly created pin shapes on the same net overlap (1), and the shapes have the same layer-purpose pair, the two shapes are merged (2).

1. Overlapping Shapes



2. Merged Shapes



Understanding Automatic Pin Placement

When you set up for pin generation in the layout, you can select the **Auto Place Pin** option to facilitate pin placement on large array of shapes. The following priority is used to determine pin placement:

1. Shapes that are on cell boundaries will automatically be treated as pins.
2. If the cell boundaries do not have any shapes, the command will check the **Limit Search Top** option and search those layers for shapes. If **Limit Search Top** is not specified, the command will search all layers and treat shapes near boundaries as pins.
3. If a net is not available in the top cellview, `instTerm` pin shapes will be treated as pins if you selected the **Assign to Shape** option. If the option is not selected, the command will create pins on top of `instTerm`.

For more information about the **Auto Place Pin**, **Limit Search Top**, and **Assign to Shape** options, see [Generate Layout Pins Dialog Box Reference](#).

In the following example:

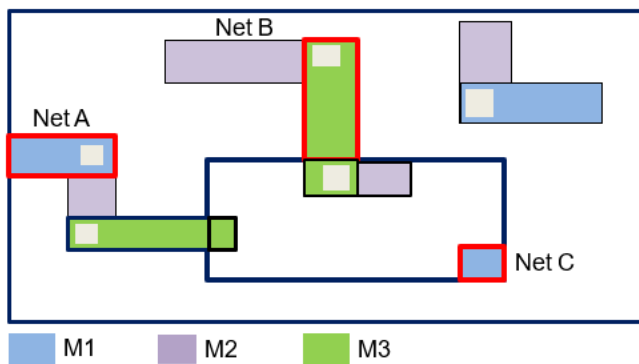
- The **Assign to Shape** option is selected in the **Generate Pins** dialog box, so pin shape candidates will be treated as pins.
- Net A has an m1 shape on the cell boundary, so the m1 shape is automatically treated as a pin.
- Net B does not have any shape on the cell boundary.

If **Limit Search Top** is not specified in the **Generate Pins** dialog box, the command will search all layers and treat shapes near the cell boundary as pins.

If **Limit Search Top** is set to 1, and Net B has shapes on m2 and m3, then m3 is the top layer, and near-boundary shapes on m3 are treated as pins.

If **Limit Search Top** is set to 2, and Net B has shapes on m2 and m3, then the command will search shapes on m3 and m2 for near-boundary shapes and treat them as pins.

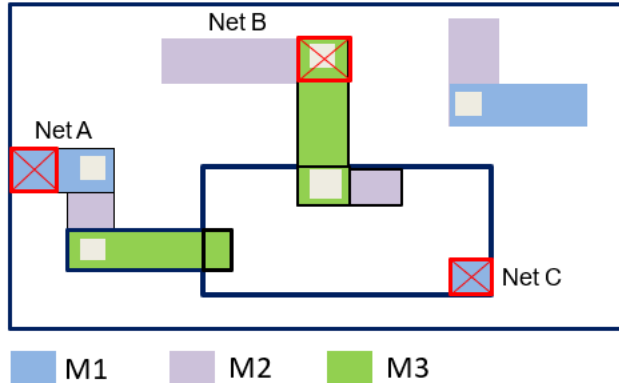
- Net C does not have any shapes at the top level, but it has an instance pin. The command will copy the instance pin to the top level and treat it as a pin.



In the following example:

- The **Assign to Shape** option is not selected in the **Generate Pins** dialog box, so the command will create square pin shapes using the following rules:
 - If the shape is path, path segment, or orthogonal shape, the command will use the path or shape's width or short edge to determine the pin size. Otherwise, the command will treat the original shape as a pin.
 - If the pin candidate is touching the cell boundary, the square pin will be placed on the boundary. Otherwise, the square pin will be placed near the boundary.
- On net A, the square pin is placed on the boundary.

- On net B, the square pin is placed on the top-most layer (m3) near the boundary.
- On net C, the instance pin is treated as a pin.



Preparing the Layer Mapping File

If you plan to generate pins from labels, you must supply the layer mapping file. The Layout Editor will only generate pins for shapes whose layer-purpose pair are mapped to the layer-purpose pair of overlapping labels in the mapping file.

The mapping file must be a space-delimited ASCII file. Each row in the file must contain the following in the sequence shown:

- Shape layer-purpose pair
- Label layer-purpose pair
- (Optional) Pin layer-purpose pair

If you do not provide the shape layer-purpose pair, one of the following occurs:

leIgnoreUnmappedLPP Tcl Preference setting	Result
true	No pin will be generated.
false	Pin will be generated only if the shape layer-purpose pair is the same as the overlapped text layer-purpose pair.

The `leIgnoreUnmappedLPP` Tcl preference setting also applies when you use the Technology Library extraction rule.

If you do not provide the pin layer-purpose pair information, the Layout Editor will use the existing shape's layer name and use the purpose name you supply in the **Purpose for New Shapes** field. See [Generate Layout Pins Dialog Box Reference](#).

The following example shows a space-delimited mapping file that contains three mappings for shapes and labels on layers m1, m2, and m3. In this example, the pin layer-purpose pair information is available in the last column.

```
m 1 drawing m 1text drawing m 1 pin
m 2 drawing m 2text drawing m 2 pin
m 3 drawing m 3text drawing m 3 pin
```

Before You Start to Generate Pins

- [Ignoring Labels](#)
- [Ignoring User-Defined Blockage Layers](#)

Ignoring Labels

You can ignore certain labels during pin generation by specifying the label layer information. To do this, use the `leGeneratePinIgnoreLabel` Tcl preference, as follows:

```
leGeneratePinIgnoreLabel = <layerNames>
```

When specifying multiple layer names:

- Separate the names using character spaces. For example, you can type `labelName1 labelName2 labelName3`.
- Use the `*` wildcard character to include all layer names that contain the specified characters.

Ignoring User-Defined Blockage Layers

The pin generation operation will ignore layers whose purposes are listed in the `leIgnorePurposes` Tcl preference. Before you start to generate pins, review the preference setting to make sure the correct layers will be ignored.

Generating Pins Within the Layout

This section describes how to generate multiple pins while you are working in a design layout. For instructions on generating multiple pins at the library level, see [Generating Layout Pins at the Library Level](#).

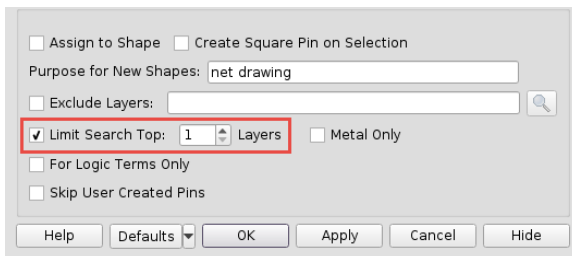
To generate multiple pins in your design:

1. In the Layout Editor, choose **Create > Pin > Generate Pins**. The **Generate Pins** toolbar appears.
2. Specify the options you want to use for the operation. See [Generate Layout Pins Dialog Box Reference](#).
3. To specify additional options, click **Eject** . The **Generate Pins** dialog box opens. See [Generate Layout Pins Dialog Box Reference](#).
4. When you are finished, click **Apply** . When the pin generation operation is finished, the Console shows a report that lists the pins that were successfully generated. The report also shows the list of terminals for which pins were not generated.

Generating Pins for Specified Top Layers

While working in a design layout, during the generation of multiple pins, you can restrict the pin generation to a specified number of top layers.

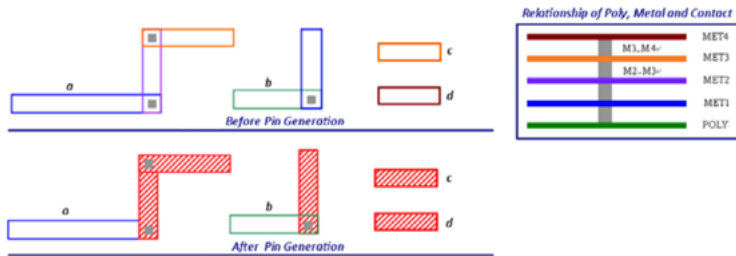
To do this, in the Generate Layout Pins dialog box, select **Limit Search Top** and specify the number of top layers for the pin generation in the **Layers** list. The tool then restricts the pin generation to the specified top layers of the individual net islands. See the following figure:



For example, on selecting **Limit Search Top** and entering a value 2 in the **Layers** list, the tool uses only the top two layers. If **Metal Only** is also selected, only the top two metal layers are used.

The pin figures are generated depending on the nets within a pin area. In the following scenario where the shapes within a pin area are connected with different nets a, b, c and

d, the tool generates the following pin figures (for the option settings in the preceding paragraph):

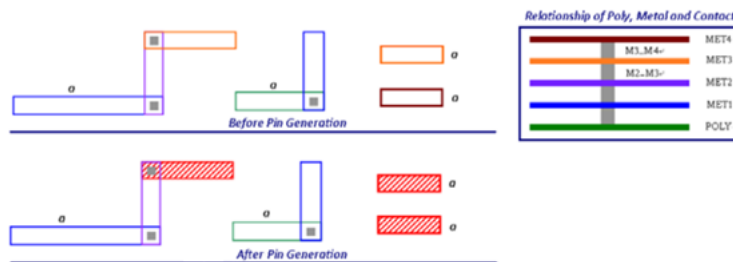


- For net a, the MET3 and MET2 pin shapes are created, since the top two metal layers are MET3 and MET2.
- For net b, the MET1 pin shape is created since the top metal layer is MET1, and the poly shape is excluded.
- For net c, the MET3 pin shape is created since the top metal layer is MET3.
- For net d, the MET4 pin shape is created since the top metal layer is MET4.

If all shapes within a pin area are connected with the same net, on selecting **Limit Search Top**, the pin figures are generated differently from net and from the net text on several connection islands.

- Generating pin from net:

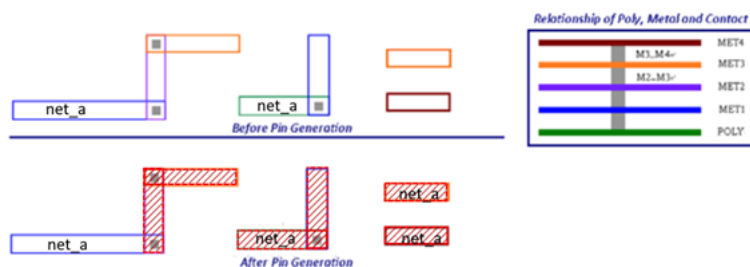
Consider the following example where all shapes within a pin area are connected with the same net a.



For a value 2 in the **Layers** list, for net a, the MET4 and MET3 pin shapes are created since the top two metal layers are MET4 and MET3; and the POLY shapes are excluded if the **Metal Only** option is selected.

- Generating pin from text:

Consider the following example where all shapes within a pin area are connected with the same net net_a.



For net_a texts on several connection islands, the tool considers top layers for each island. For a value 2 in the **Layers** list, the tool considers only the top two layers to generate pin for each island. The POLY layer is included if **Metal Only** is not selected.

Note:

When **Extract Shapes Through Via** is not selected (see [Generate Layout Pins Dialog Box Reference](#)), generating pins does not trace the connection islands. Net texts with the same text string are treated as individual connection islands and pins are generated individually.

Generating Layout Pins at the Library Level

This section describes how to generate multiple pins at the library level. For instructions on generating pins while you are working in a design layout, see [Generating Pins Within the Layout](#).

Note:

The **Generate Layout Pins** command does not check the logic terminals for the cellviews that have associated schematic views. If you want to check the logic terminal information during the operation, use the `le::generatePins Tcl` command.

To generate multiple layout pins at the library level:

1. In the Library Manager, choose **File > Generate Layout Pins**. The **Generate Layout Pins** dialog box opens.

Generate Layout Pins

Library:
Cell:
View:

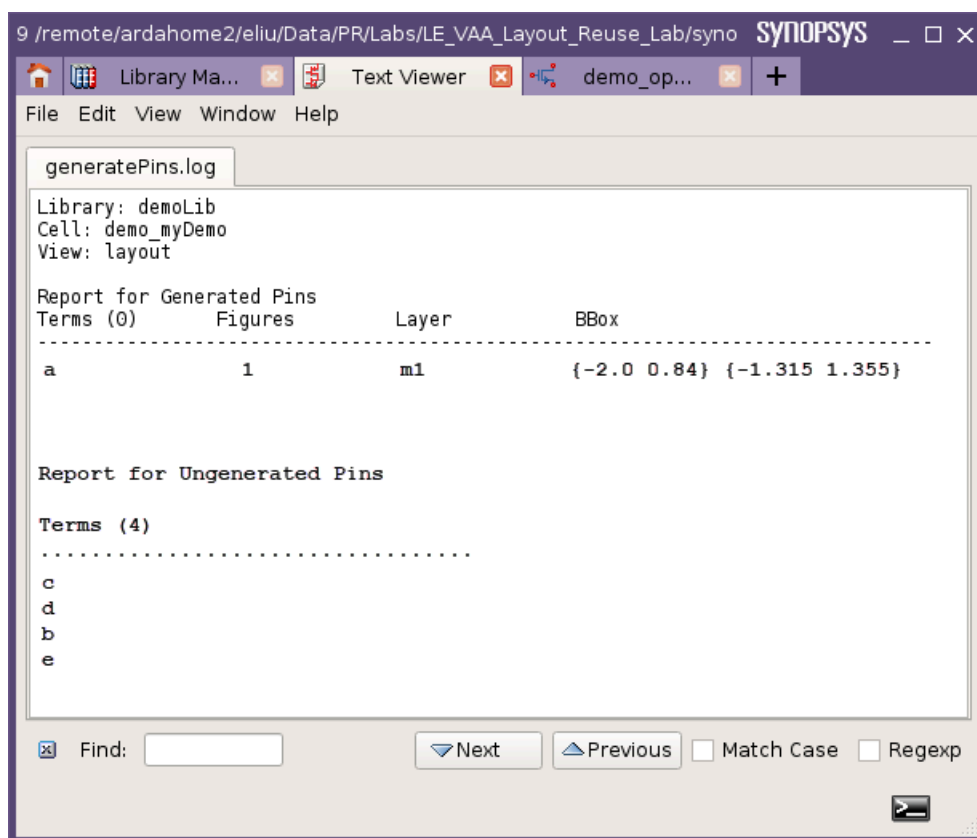
From Text

Extract Rule:
Extract Depth:
☒ Use Virtual Connection
☐ Extract Shapes Through Via
☐ Size for New Shapes:

☒ Assign to Shape
Purpose for New Shapes:
☐ Exclude Layers:
☐ Limit Search Top: Metal Layers
☒ Show Report

2. Choose the **Library**, **Cell**, and **View**.
3. Specify the options you want to use for the generate-pin operation. See [Generate Layout Pins Dialog Box Reference](#).
4. When you are finished, click **Apply**.

When the operation is finished, the Text Viewer opens and displays the pin generation report. The report shows the list of terminals for which pins were generated. The report also shows any pins that were not generated during the operation.



Generate Pins Toolbar and Dialog Box Reference

This section provides a quick reference of the **Generate Pins** toolbar and dialog box. For instructions on how to generate multiple pins simultaneously, see [Generating Pins Within the Layout](#).

Generate Pins

Area: ☒ Design ☐ Viewport ☐ Rectangle ☐ Polygon ☐ Selection

☒ From Text

Extract Rule: Technology Library

☐ Honor Purpose for New Shapes

Extract Depth: Shape 0 Text 0

☐ Extract Shapes Through Via

☐ Size for New Shapes: 0.1

☒ From Net

Nets: ☒ All ☐ Selected

☐ Ignore Nets:

Shape Depth: ☒ Top Level Only ☐ Include Instance Pins ☐ Instance Pins Only

☐ Attach Pins to Instances

☒ Create Pin Label

Type: ☒ Attribute ☐ Text ☒ Attach

Mode: ☐ Manual ☒ Auto ☐ Follow Instance Pin Label

Font: stick

☐ Overbar

LPP: Specify cont drawing

General

☒ Assign to Shape ☐ Create Square Pin on Selection

Purpose for New Shapes: pin drawing

☐ Exclude Layers:

☐ Limit Search Top: 1 Layers ☒ Metal Only

☒ Use Virtual Connection

☐ For Logic Terms Only

☐ Skip User Created Pins

Help Hide Defaults OK Apply Cancel

The **Generate Pins** toolbar and dialog box consist of the following sections:

- [Area](#)
- [From Text](#)
- [From Net](#)
- [Other Options](#)

Area

Specifies the area within which pins are to be generated. Select one of the following:

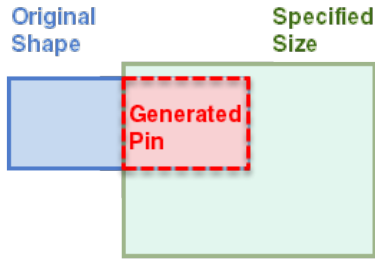
Option	Description
Area	• Design: Generate pins across the entire design. • Viewport: Generate pins within the viewport region. • Rectangle: Generate pins within the specified rectangle. Select Rectangle, and then draw a rectangle on the canvas. • Polygon: Generate pins within the specified polygon. Select Polygon, and then draw the polygon on the canvas. • Selection: Generates pins for selected objects only. If you select Generate From Text, only selected labels are considered during pin generation. If you select Generate From Net, only partially and fully selected objects on the net will be considered during pin generation. Tcl preference: <code>leGeneratePinArea</code>

From Text

Generates pins using text labels.

Option	Description
Extract Rule	Generates pin labels using one of the following: • Same LPP: The shapes that have the same layer-purpose pair (LPP) as the overlapping text label. • Technology Library: (default) The <code>pinTextLayers</code> section of the technology library. • Map: The file that defines the mapping of layer-purpose pairs for shapes and text labels. Click , and then select the mapping file you want to use. For information about mapping files, see Preparing the Layer Mapping File. If you select Technology Library: • Shapes, whose LPPs are mapped with the overlapping text LPPs as defined in the technology, are used to generate pins. • If the pin LPP is defined in the technology, pin LPP is used to create new shapes. • If the pin LPP is not defined in the technology, the Honor Purpose for New Shapes option is used. Tcl preference: <code>leGeneratePinExtractRule</code>
Extract Depth	(Dialog box only) Searches for Shape and Text up to the specified depth when generating pins using text labels. Tcl preference: <code>leGeneratePinExtractShapeLevel</code>

Option	Description
Extract Shapes Through Via	(Dialog box only) Traces shapes that are connected by vias. In the following examples, a via connects shapes on m1 and m2. If you clear the Extract Shapes Through Via check box, only the m1 shape is used to generate the pin. ☐ Extract shapes through Via  m1 via m2 If you select Extract Shapes Through Via, shapes on both m1 and m2 are used to generate the pin. ☒ Extract shapes through Via  m1 via m2 Tcl preference: <code>leGeneratePinExtractThroughVia</code>

Option	Description
Size for New Shapes	(Dialog box only) Specifies the size of the pin shapes, in user database units. If you specify only one value (n), the generated shape will be n x n. If you specify two values (n, m), the generated pin will be n x m, where n is the width and m is the height. The two values must be separated by a comma or a character space. Note: The pin size cannot be smaller than the mfgGridResolution specification. The center of the generated pin aligns with the origin of the overlapped text. The final pin size and position is determined by the intersection of the original shape and the pin size you specified.  The diagram illustrates the process of generating a pin. It shows a blue rectangle labeled 'Original Shape' and a green rectangle labeled 'Specified Size'. The intersection of these two rectangles is highlighted with a red dashed border and labeled 'Generated Pin'. Specify a pin shape size if you want to generate pins of uniform size and shape. If you do not specify a size, the Layout Editor will automatically generate pins that fit the overlapping shapes. Tcl preference: leGeneratePinSpecifyPinSize, leGeneratePinPinSize

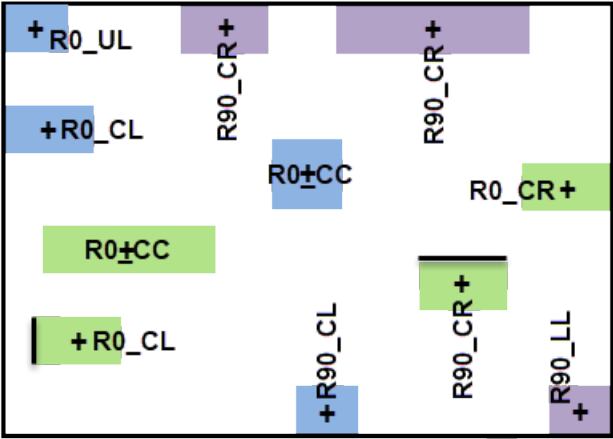
From Net

Generates pins using net information.

Option	Description
Nets	Nets for which pins are generated: • All: (Default) Generates pins for all nets. • Selected: Generates pins for selected nets. Tcl preference: leGeneratePinNetMode, leGeneratePinSelectedNetNames
Ignore Nets	Ignores the specified nets when generating pins.

Option	Description
Shape Depth	Hierarchy depth at which pins are generated: • Top Level Only: Generates pins only at the top level of the design. • Include Instance Pins: Generates instance pins in addition to generating pins at the top level. • Instance Pins Only: Generates instance pins only.
Auto Place Pin	(Dialog box only) Places pins automatically during pin generation. Select this option to facilitate pin placement on large arrays of shapes. See Understanding Automatic Pin Placement. When you select this option, the Shape Depth option becomes unavailable. Default: not selected Tcl preference: <code>leGeneratePinAutoPlacePin</code>
Attach Pins to Instances	Creates and attaches pins on the top level to instances. The selection does not available when you select Shape Depth: Top Level Only. Default: Not selected Tcl preference: <code>leGeneratePinAttachToInst</code>
Create Pin Label	Creates pin labels for the generated pins. When selected, you can specify the method and label styles to use. Creating a pin label for every pin shape is enhanced to create a pin label on top metal layer for a connected island. Connected islands are further extracted, and pin shapes with the same LPP are merged to generate a single pin label. Tcl preference: <code>leGeneratePinLabelTopMetalOnly</code> (Default is true). If the preference is set to false, no connected island is extracted and pin labels are created for all pin shapes.
Type	Pin label type to create: • Attribute: Uses pin attributes as labels. • Text: Uses text as the labels. Select Attach if you want to attach the text to the pin shapes. Tcl preference: <code>lePinLabelTextType</code>, <code>lePinLabelTextAttach</code>
Mode	Method of pin creation: • Manual: Allows you to specify the label style: Font, Height, Overbar, LPP, Drafting, Orientation, and Justification. • Auto: Allows the tool to determine the following label styles: Height, Drafting, Orientation, and Justification. You can specify the Font, LPP, and Overbar. For 1x pins, select Follow Instance Pin Label if you want the command to apply existing 1x pin label styles if 1x instance pin labels are found. Otherwise, applies the label styles automatically. Default: Auto Tcl preferences: <code>leGeneratePinCreatePinLabel</code>, <code>leGeneratePinLabelStyle</code>, <code>leGeneratePinFollowInstPinLabel</code>

Option	Description
Height:	(Manual only) The height of the label text, in micrometers. Default: 0.2 Tcl preference: <code>leTextHeight</code>
Drafting:	(Manual only) Always draws horizontal label text from the left, always draws vertical label text from the top, or follow the label orientation. Tcl preference: <code>leTextDrafting</code>
Font:	The font style of the label text. Default: stick Tcl preference: <code>leTextFont</code>
Overbar:	Adds a short horizontal line (overbar) above the label text.Default: not selected Tcl preference: <code>leTextOverbar</code>

Option	Description
Orientation:	(Manual only) The orientation and mirroring of the label text: R0, R90, R180, R270, MX, MY, MXR90, or MYR90. The default orientation is determined by the following priority: • Cell boundary: For example, pins placed on the vertical edge of the cell boundaries likely indicate that routes are horizontal and moving from left to right. In these cases, the default label orientation will be R0. • Routing direction in the technology library: If a pin has multiple directions, the command will use the layer's routing direction specified in the technology library. • Shape boundary box orientation: The label orientation will be based on the shape boundary box orientation (horizontal or vertical). • Pin placement in a cell: The following diagram shows possible pin placements in a cell and the resulting label text placement and orientation. For example, if the pin is on the left vertical edge of the cell boundary, the label will be placed to the right of the pin, its orientation is R0, and its justification is center-left (CL).
	 M1 horizontal M2 vertical M3 no orientation Pin direction left Pin direction top UL = upper-left CL = center-left CC = center-center CR = center-right LL = lower-left
	Tcl preference: <code>leCreateOrientation</code>
Justification	(Manual only) The position of the label anchor (origin) relative to the label text: Upper-Left, Center-Left, Lower-Left, Upper-Center, Center-Center, Lower-Center, Upper-Right, Center-Right, or Lower-Right. See the example in the Orientation description. Default: Center-Center Tcl preference: <code>leTextJust</code>

Option	Description
LPP	The layer-purpose pair on which the label text is placed. Select one of the following: • Inherit: Inherits the LPP specification from shapes. • Specify: Allows you to select the LPP to use. • Technology Library: Uses the LPP information from the technology library. • Map: Uses the LPP information in the mapping file. Select the mapping file to use. Default: Inherit Tcl preference: <code>leTextLPPMethod</code>

Other Options

Option	Description
Assign to Shape	(Dialog box only) Assigns pin information to original 0x shapes directly. Clear the check box to create new pin shapes and assign the pin information to the newly created shapes. The option is available only if you turn off Create Square Pin on Selection.
Create Square Pin on Selection	Creates square-shaped pins on selection. The option is available only if you select Area: Selection and turn off Assign to Shape.
Purpose for New Shapes	Specifies the pin layer's purpose. See also Preparing the Layer Mapping File. You can specify multiple purpose names. The order of the names indicate the order in which the Layout Editor looks for the layer-purpose pair during pin generation. For example, suppose you specify <code>pin drawing</code>, and the existing shape is on <code>m2</code>. During pin generation, the Layout Editor will look for <code>m2 pin</code> first. If <code>m2 pin</code> does not exist, it will look for <code>m2 drawing</code>. If <code>m2 drawing</code> does not exist, no pin will be generated.
Exclude Layers	(Dialog box only) Excludes the specified layers during pin generation. Click  to select the layers to exclude.
Limit Search Top	(Dialog box only) Restricts pin generation to the specified layers. Select Metal if you want to restrict pin generation to the specified metal layers.
Use Virtual Connection	(Dialog box only) Treats colons (:) in the label as virtual connection indicators. Tcl preference: <code>leGeneratePinUseVirtualConnect</code>

Option	Description
For Logic Terms Only	(Dialog Box only) Generates pins based only on existing terminals in the Layout Editor or SDL mode. Clear the check box if you want to generate pins from all text and nets. See also About Multiple-Pin Generation .
Skip User Created Pins	(Dialog Box only) Ignores pins you created using other pin creation commands.

Generate Layout Pins Dialog Box Reference

This section provides a quick reference of the **Generate Layout Pin** dialog box. For instructions on how to generate multiple layout pins for a library, see [Generating Layout Pins at the Library Level](#).

Generate Layout Pins

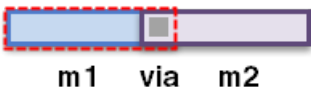
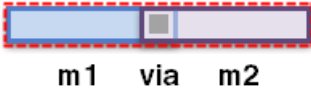
Library:
 Cell:
 View:

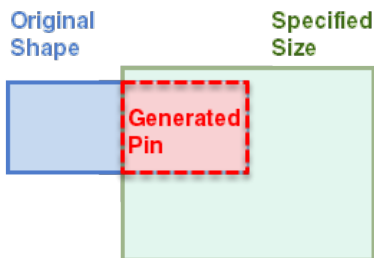
From Text

Extract Rule:
 Extract Depth:
☒ Use Virtual Connection
☐ Extract Shapes Through Via
☐ Size for New Shapes:

☒ Assign to Shape
 Purpose for New Shapes:
☐ Exclude Layers:
☐ Limit Search Top: Metal Layers
☒ Show Report

Option	Description
Library Cell View	Specifies the library, cell, and view of the design.
From Text	Generates pins using text labels.

Option	Description
Extract Rule	Generates pin labels using one of the following: • Same LPP: The shapes that have the same layer-purpose pair (LPP) as the overlapping text label. • Technology Library: (default) The associated technology library. • Map: The file that defines the mapping of layer-purpose pairs for shapes and text labels. Click , and then select the mapping file you want to use. For information about mapping files, see Preparing the Layer Mapping File.
Extract Depth	(Dialog box only) Searches for shapes up to the specified depth when generating pins using text labels.
Use Virtual Connection	(Dialog box only) Treats colons (:) in the label as virtual connection indicators.
Extract Shapes Through Via	(Dialog box only) Traces shapes that are connected by vias. In the following examples, a via connects shapes on m1 and m2. If you clear the Extract Shapes Through Via check box, only the m1 shape is used to generate the pin. ☐ Extract shapes through Via  m1 via m2 If you select Extract Shapes Through Via, shapes on both m1 and m2 are used to generate the pin. ☒ Extract shapes through Via  m1 via m2

Option	Description
Size for New Shapes	(Dialog box only) Specifies the size of the pin shapes, in user database units. If you specify only one value (n), the generated shape will be n x n. If you specify two values (n, m), the generated pin will be n x m, where n is the width and m is the height. The two values must be separated by a comma or a character space. Note: The pin size cannot be smaller than the <code>mfgGridResolution</code> specification. The center of the generated pin aligns with the origin of the overlapped text. The final pin size and position is determined by the intersection of the original shape and the pin size you specified.  The diagram illustrates the process of generating a pin. It shows a blue rectangle labeled 'Original Shape' and a green rectangle labeled 'Specified Size'. The intersection of these two rectangles is highlighted with a red dashed border and labeled 'Generated Pin'. The 'Generated Pin' is a smaller rectangle that fits within the intersection of the 'Original Shape' and the 'Specified Size'. Specify a pin shape size if you want to generate pins of uniform size and shape. If you do not specify a size, the Layout Editor will automatically generate pins that fit the overlapping shapes.
Assign to Shape	(Dialog box only) Assigns pin information to original 0x shapes directly. Clear the check box to create new pin shapes and assign the pin information to the newly created shapes.
Purpose for New Shapes	Specifies the Pin layer's purpose. You can specify multiple purpose names. The order of the names indicate the order in which the Layout Editor will look for the layer-purpose pair during pin generation. For example, suppose you specify <code>pin drawing</code>, and the existing shape is on <code>m2</code>. During pin generation, the Layout Editor will look for <code>m2 pin</code> first. If <code>m2 pin</code> does not exist, it will look for <code>m2 drawing</code>. If <code>m2 drawing</code> does not exist, no pins will be generated.
Exclude Layers	(Dialog box only) Excludes the specified layers during pin generation. Click  to select the layers to exclude.
Limit Search Top	(Dialog box only) Restricts pin generation to the specified metal layers.

Option	Description
Metal Layers	(Dialog box only) Generates pins for the specified metal layers within the net. For example, if you specify 2, the Layout Editor will generate pins for the top two metal layers within the net. The top metal layers that you specify can vary from net to net. For example, net A has shapes on metal layers m3, m2, and m1, and net B has shapes on m4, m3, and m2. The top metal layers in net A are m3 and m2, and the top layers in net B are m4 and m3. Limit Search Top works with Exclude Layers to determine the top metal layers. For example, if you excluded m3 from net A, then the only top layer remaining is m2, so the Layout Editor generates pins on m2.
Show Report	Automatically displays the pin-generation report after the operation finishes.

Editing Instance Pins

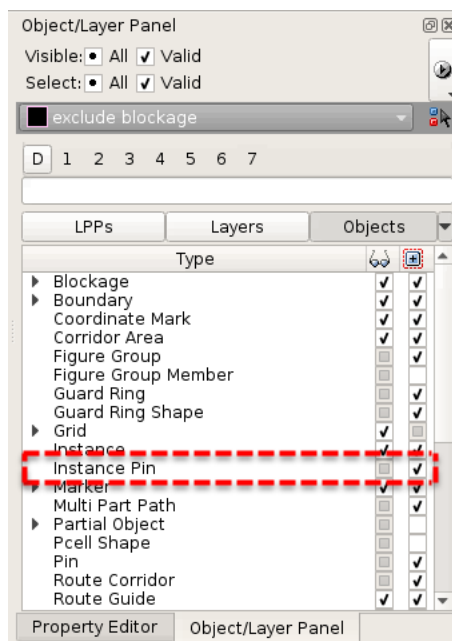
This topic describes the following:

- [Before You Start](#)
- [Selecting and Editing Instance Pins](#)

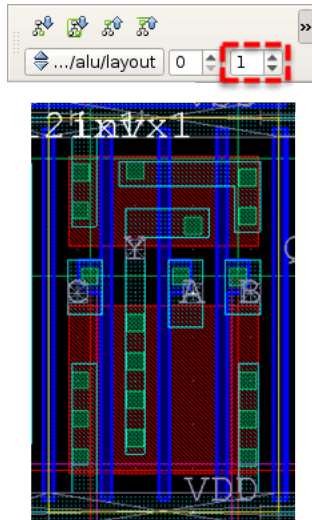
Before You Start

Before you start, make sure the instance pins are selectable, and set up the design so that you can best view the pins when working with them:

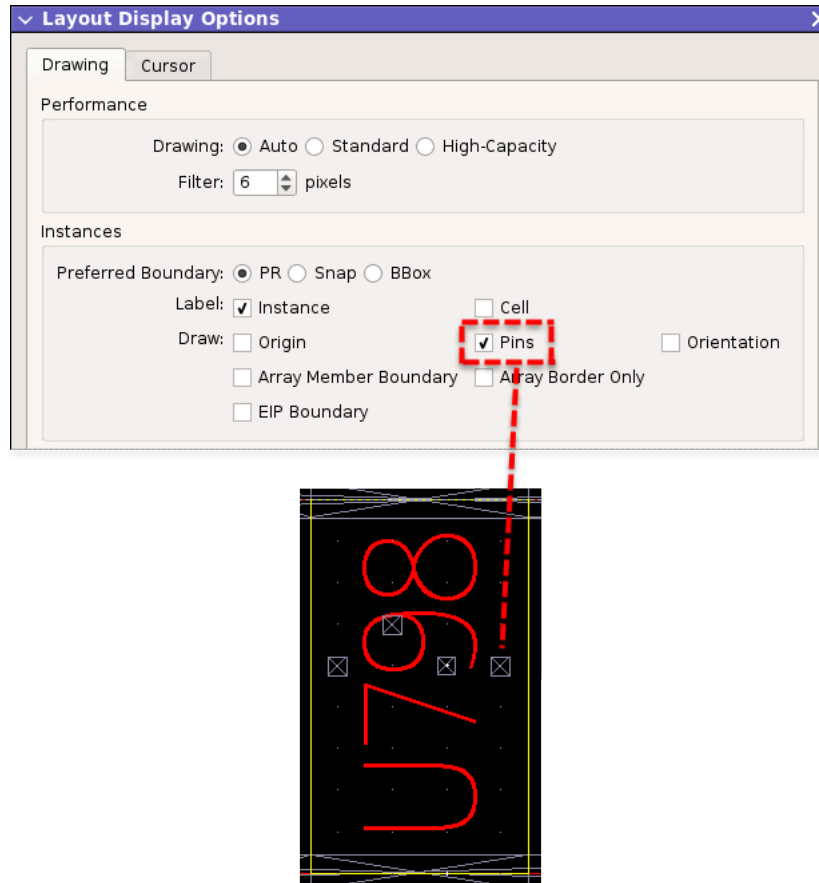
1. In the Object/Layer Panel, turn on the selectability of the **Instance Pin**.



2. Set up the design for viewing the instances and instance pins. For example, you can use one or both of the following methods:
 - On the Hierarchy toolbar, set the Stop Level to view the pins at the specified level.



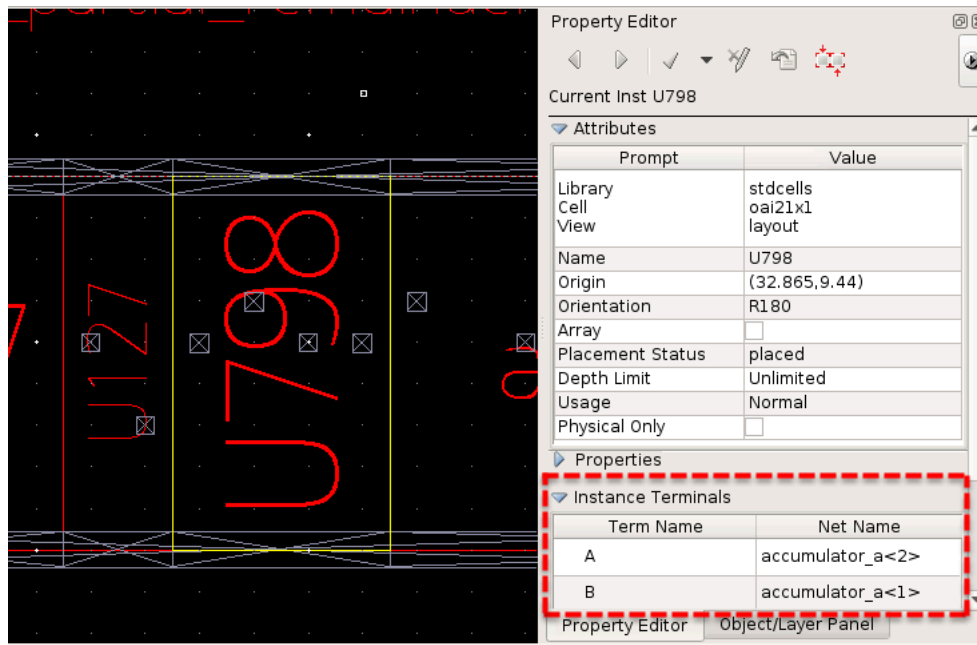
- If you want the instance pins to be rendered even if they are not in the current depth level, in the **Layout Display Options** dialog box, click the **Drawing** tab, and select **Pins** in the **Instance** area.



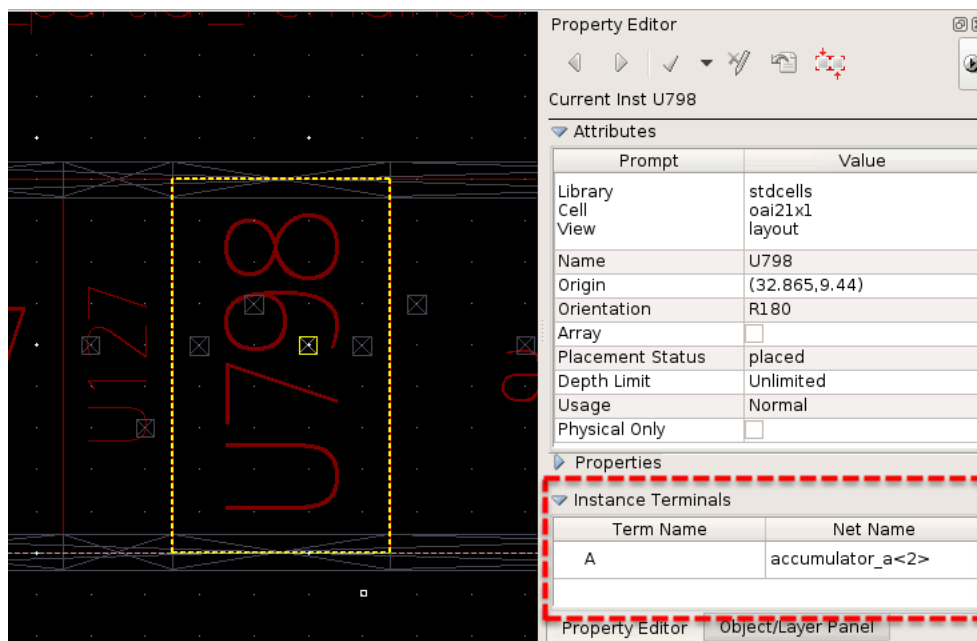
Selecting and Editing Instance Pins

To select and edit instance pins:

1. Display the Property Editor.
2. On the canvas, select the instance. Information about the instance appears in the Property Editor. The **Instance Terminals** section lists all the pins associated with the instance.



3. (Optional) To display only one of the instance pins in the Property Editor, on the canvas, select the instance pin of interest. In the Property Editor, the **Instance Terminals** section updates and displays only the selected pin.



4. Edit the **Net Name** for the instance terminal or terminals.
5. When you are finished, click **Apply Changes**  on the Property Editor toolbar.

Deleting Generated Pins

You can delete the pins that were generated from the Generate Pins command. For information about using the Generate Pins command, see [Generating Multiple Pins Simultaneously](#).

To delete generated pins:

- In the Layout Editor, choose **Edit > Pin > Delete Generated Pins**. The Layout Editor deletes the pins that were generated from the Generate Pins command.

You can use the `leGeneratePinForInvisibleLayers` preference to specify whether you want to delete generated pins for invisible objects or objects of invisible layers.

4

Creating and Editing Interconnects

This chapter describes how to create and edit interconnects in the designs.

This chapter describes the following.

- [About Interconnects](#)
- [Before You Start](#)
- [Creating Interconnects](#)
- [Selecting Layer Purpose Pairs for Interconnects](#)
- [Connecting Routes on the Same Layer](#)
- [Changing Interconnect Layers](#)
- [Avoiding Overlap](#)
- [Choosing End Styles](#)
- [Adding Vias Between Interconnects](#)
- [Editing Paths and Path Segments Using the Property Editor](#)
- [Cutting Corners of Paths](#)
- [Reshaping Interconnects](#)
- [Bridging Interconnects](#)
- [Stacking Interconnects](#)
- [Striping Wires](#)
- [Chopping Striped Wires](#)
- [Trimming Wires](#)
- [Stretching Interconnects](#)
- [Moving Paths and Interconnects](#)

- [Merging Paths and Path Segments Automatically](#)
- [Stretching the Wire Width](#)

About Interconnects

After you place circuit components in your design, you can add wires (routes or interconnects) to connect these components.



You can use one of the following methods to add routes:

Routing Method	Description
Manually	Use the Create > Interconnect command to manually add routes. See Creating Interconnects . Use the Create Bus command to manually add a group of parallel routes simultaneously. See Creating Bus Routes .
Automatically, with user guidance	Use the Custom Router to automate the routing process. Although the process is automated, the Custom Router lets you provide some guidance, giving you control over route placement. See the Custom Router User Guide.

To add interconnects or routes for conducting high-frequency alternating currents (AC), such as radio and microwave signals, see [Creating and Editing Transmission Lines](#).

Before You Start

Before you start to create interconnects, review the applicable prerequisites and be aware of default settings that might impact the results.

This topic describes the following:

- [Setting Up Tracks for Routing](#)
- [Changing Interconnect Colors During Track Snapping](#)
- [Enforcing Route Direction](#)
- [Displaying Track Widths as You Route](#)
- [Constraining Maximum Path Widths](#)
- [Changing Same-Layer Path Widths as You Route](#)
- [Extending Path Ends to Cover Vias](#)
- [Displaying Net Names](#)

Setting Up Tracks for Routing

If you plan to route on tracks, make sure the design contains tracks. In addition, make sure the tracks are visible and active to facilitate route placement. For information, see [Creating and Editing Track Region PCells](#), Track Pattern Assistant, or Track Region Assistant.

The track region definitions that are available for global and local track region creation are filtered by the `leTrackRegionAllowedDefNames` Tcl preference. If you do not see the definition when creating track regions, check the Tcl preference setting.

Changing Interconnect Colors During Track Snapping

When snapping to default track types, if the width of the interconnect is greater than the `minWidth` of the layer, the interconnect can automatically change color to permit neighboring wires to be placed closer. To enable the automatic color change, you must set the `leFlipWideWireColor` Tcl preference to `true`. In addition, you must choose the **Color > Auto** on the [Interconnect Toolbar and Create Interconnect Dialog Box](#) when you create the interconnect.



Enforcing Route Direction

The route direction is affected by the following:

- `preferredRoutingOnly` technology constraint
- `leHonorRoutingDirection` Tcl preference
- `leHonorPreferredRoutingOnly` Tcl preference

The following table presents the possible settings of the two Tcl preferences and the resulting routing directions:

<code>leHonorRoutingDirection</code>	<code>leHonorPreferredRoutingOnly</code>	Result
true	NA	All layers honor preferred route direction.
true	false	All layers honor preferred route direction.
false	true	Only the layers defined in the <code>preferredRoutingOnly</code> constraint (true) honor the preferred route direction.
false	false	All layers can route in both directions.

Displaying Track Widths as You Route

When you create an interconnect, the Layout Editor automatically displays the width value of the current track and adjacent parallel tracks. The width information is a visual aid that helps you place the interconnects on the correct tracks.



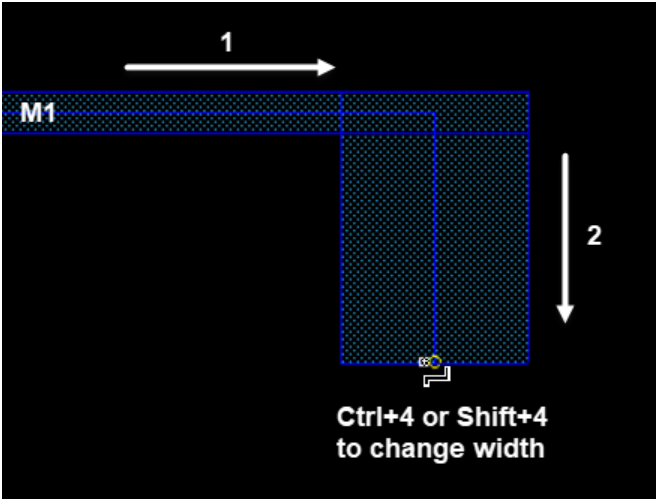
Constraining Maximum Path Widths

By default, the **Create Interconnect** command permits widths that are greater than the `maxWidth` constraint or the maximum width in the `allowedWidthRange` constraint. For example, if the current path has a width of 0.5 μm , the path width remains at 0.5 μm after you change to a layer that requires a width of 0.3 μm .

If you want the command to comply with the width constraints, set the `leInterconnectAllowExceedMaxWidth` Tcl preference to false.

Changing Same-Layer Path Widths as You Route

To change same-layer path widths after changing route direction, make sure you define the `allowedWidthRange` layer constraint and specify the permitted track widths. In the following example, the permitted M1 vertical track widths are 0.2, 0.4, and 0.6 μm . The M1 horizontal interconnect starts with a width of 0.2. After changing the direction and moving vertically on M1, you can press Ctrl+4 or Shift+4 to change the width to 0.4 or 0.6. Track width compliance can limit the available width selection.



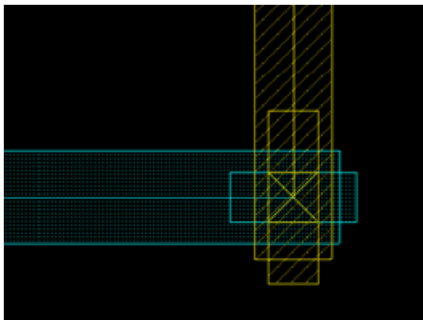
See [Using the Create > Interconnect Command](#).

Extending Path Ends to Cover Vias

Under some circumstances, the added via metal layers can be longer than the path ends, causing short-edge violations. To ensure the paths fully cover the vias, the `leInterconnectExtendPathEndToViaEnd` Tcl preference to set to true by default.

The following diagrams show the results of the `leInterconnectExtendPathEndToViaEnd` Tcl preference settings:

Preference Setting	Result
true (default)	

Preference Setting	Result
false	

This preference applies to vias that are added during the following command operations:

- **Create Interconnect**
- **Reshape Interconnect**
- **Create Bus**
- **Continue Bus**

In addition, the preferences is only applied when the pathend style is one of the following:

- **Truncate**
- **Extend**
- **Variable**

The preference does not apply to the **Round** pathend style.

See Also

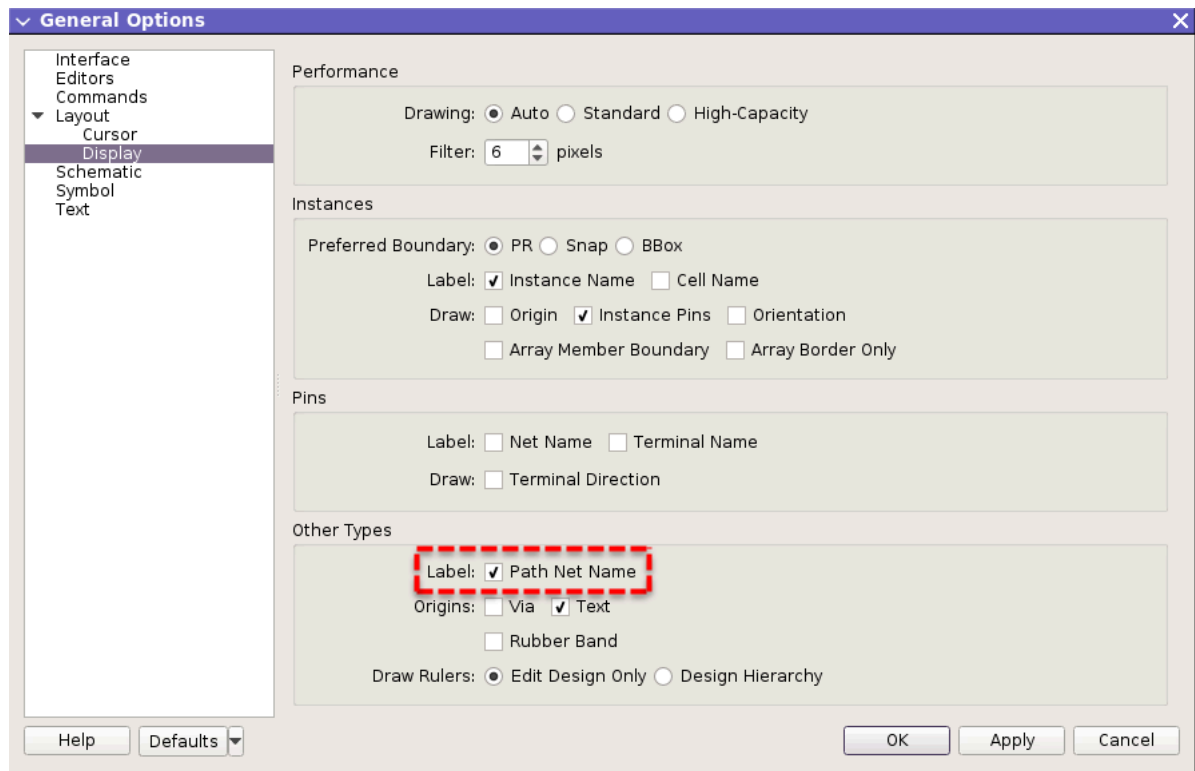
- [Choosing End Styles](#)

Displaying Net Names

To display net names on the path and path segments:

1. In the **General Options** dialog box:
 - a. Select **Layout > Display**.
 - b. In the **Other Types** area, select **Label: Path Net Name**.

By default, turning on **Path Net Name** displays the net names on 0x paths and path segments.



2. To display net names on paths and path segments on hierarchy levels up to the **Stop Level**, set the `leDrawPathLabelFullDepth` Tcl preference to `true`.

Creating Interconnects

You use the **Create > Interconnect** command to create bus routes manually.

This topic describes the following:

- [About the Create > Interconnect Command](#)
- [Paths and Path Segments](#)
- [Using the Create > Interconnect Command](#)
- [Interconnect Toolbar and Create Interconnect Dialog Box](#)
- [Create Interconnect Menu and Shortcut Keys](#)
- [Create Interconnect Context-Sensitive Toolbar](#)

Make sure you review the task requirements before you proceed. See [Before You Start](#).

About the Create > Interconnect Command

The Layout Editor **Create > Interconnect** command places route paths in the layout. This topic describes the features of the command.

The **Create > Interconnect** command lets you:

- Manually create routes in your design.
- Add vias as you create the routes, and then continue routing on a layer above or below the current route.
- Display the resistance, capacitance, and length of the route.

If you enable **Dynamic Measurement** on the **Cursor** tab of the **Layout Display Options** dialog box, the tool displays the resistance, capacitance, and length of the path as cursor text while you create interconnects. For information, see [Setting Layout Display, Cursor, and Selection Options](#).

You can only create interconnects or routes on layer purpose pairs that your technology file identifies as valid routing layers.

In advanced node designs (such as those containing FinFETs), you can also assign colors to the newly created routes. Color names must be defined in the technology library. When you create routes, be aware of the color propagation rules. For information, see [Color Propagation](#). To change the color of routes or to set the color lock, see [Locking and Changing Colors](#).

If you want the interconnect you create to automatically merge with overlapping collinear paths and path segments, set the `leAutoMergePaths` Tcl preference to `true`. See [Merging Paths and Path Segments Automatically](#).

Instead of using the **Interconnect** toolbar, you can use the `ile::createInterconnect`, `ile::createPath`, and `ile::createPathSeg` Tcl commands to create routes manually. For more information, see the *Custom Compiler Tcl Reference Manual*.

Paths and Path Segments

Interconnects consist of path and path segments. When you use the **Create > Interconnect** command, you can select whether you want to create a path or a path segment.

The following table lists the differences between paths and path segments.

Paths (path objects)	Path Segments (pathSeg objects)
Can be oriented at any angle	Must be oriented orthogonally or diagonally

Paths (path objects)	Path Segments (pathSeg objects)
Are defined by two or more points	Are defined by exactly two points
Do not change their width when rotated or when drawn at non-orthogonal orientations Interactive editing commands keep the centerlines of non-orthogonal paths on-grid, but edge vertices can move off-grid.	Automatically adjust their width to stay on grid when stretched, when rotated, and when drawn at 45° orientations

To control whether you use path or path segments, use the **Interconnect Object** option in the **Command** tab of the **Layout Design Options** dialog box. See [Setting Layout Design Options](#).

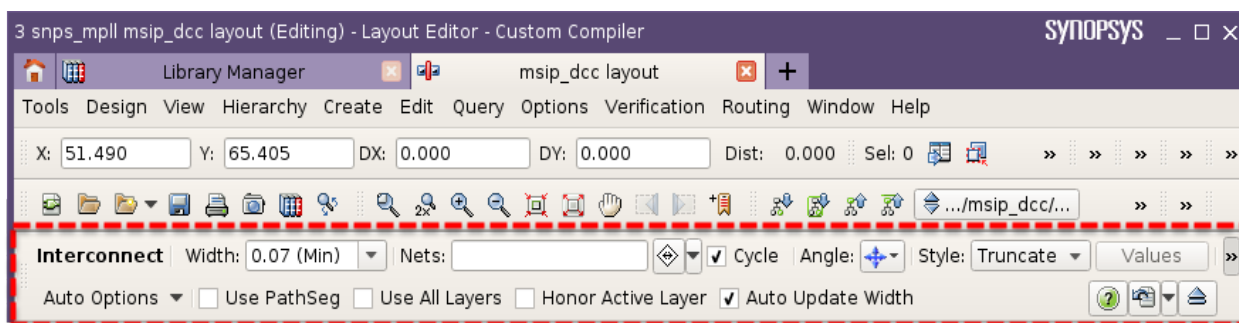
Using the Create > Interconnect Command

You use the **Create > Interconnect** command to manually place routes in the design layout. This topic describes the procedure for using the command.

 [Show me](#) an example of creating interconnects.

To create an interconnect path:

1. Choose **Create > Interconnect** or press P. The **Interconnect** toolbar appears.



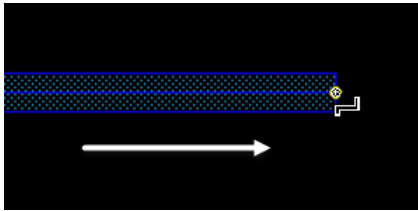
2. Specify the options you want to apply. See [Interconnect Toolbar and Create Interconnect Dialog Box](#).

3. On the canvas, click the starting location of the route.

If you are starting from a pin and want the first path segment to align with the pin, right-click the pin and choose **Toggle Start/End Alignment** (or press /) to cycle through the different alignment positions. See [Create Interconnect Menu and Shortcut Keys](#).

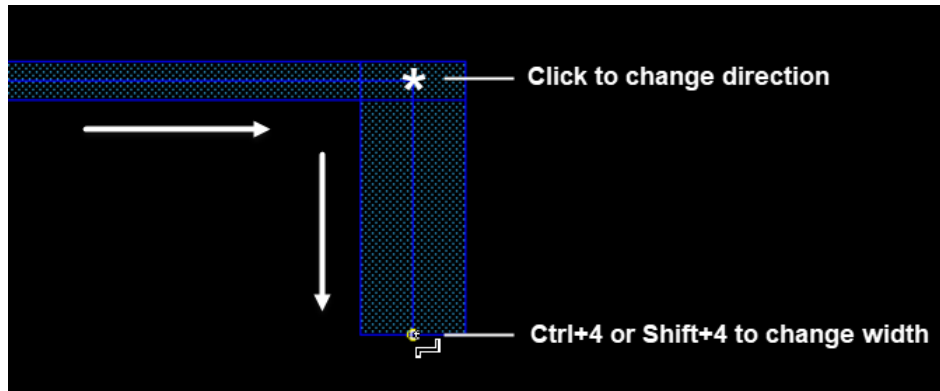
4. Move the cursor across the canvas to create the route.

Notice that a small ring appears at the end of the route's centerline so that you can see where the centerline ends.



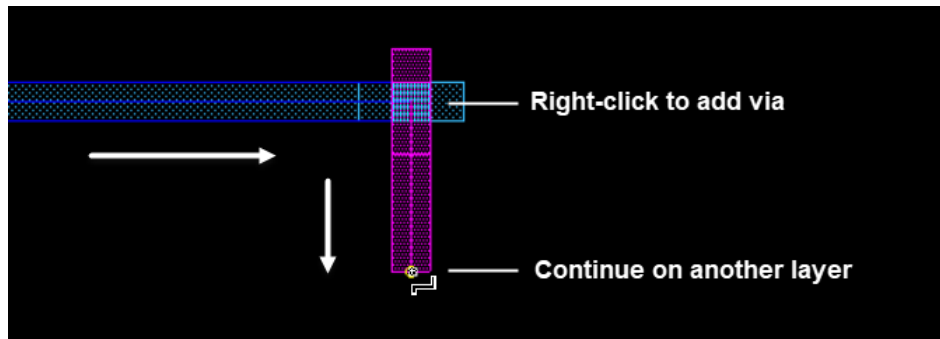
5. Click to add a waypoint and change the direction of the route while remaining on the same layer.

If you want to change the path width after changing the direction, press Ctrl+4 (next width value) or Shift+4 (previous width value) to cycle through and select the permitted width. The permitted widths are specified by the `allowedWidthRange` layer constraint. Track width compliance can limit the available width selection. See [Before You Start](#).



6. To continue to another layer and add a via, right-click the location where you want to add the via. In the menu that appears, choose one of the following:
 - **Layer Up** or **Layer Down** to add a default (standard) via, and then click to place the via.
 - **Add Via** to add a custom via or a via stack. In the **Add Via** dialog box that appears, select the desired values. For instructions, see [Adding Custom Vias or a Via Stack](#).

For information about standard and custom vias, see [About Vias](#).



7. Double-click to enter the last point and complete the path.

If you enabled the **Auto Terminate** option, making a connection to an object on the same net and on an equivalent layer completes the path. The new interconnect and the object must overlap at the point you click.

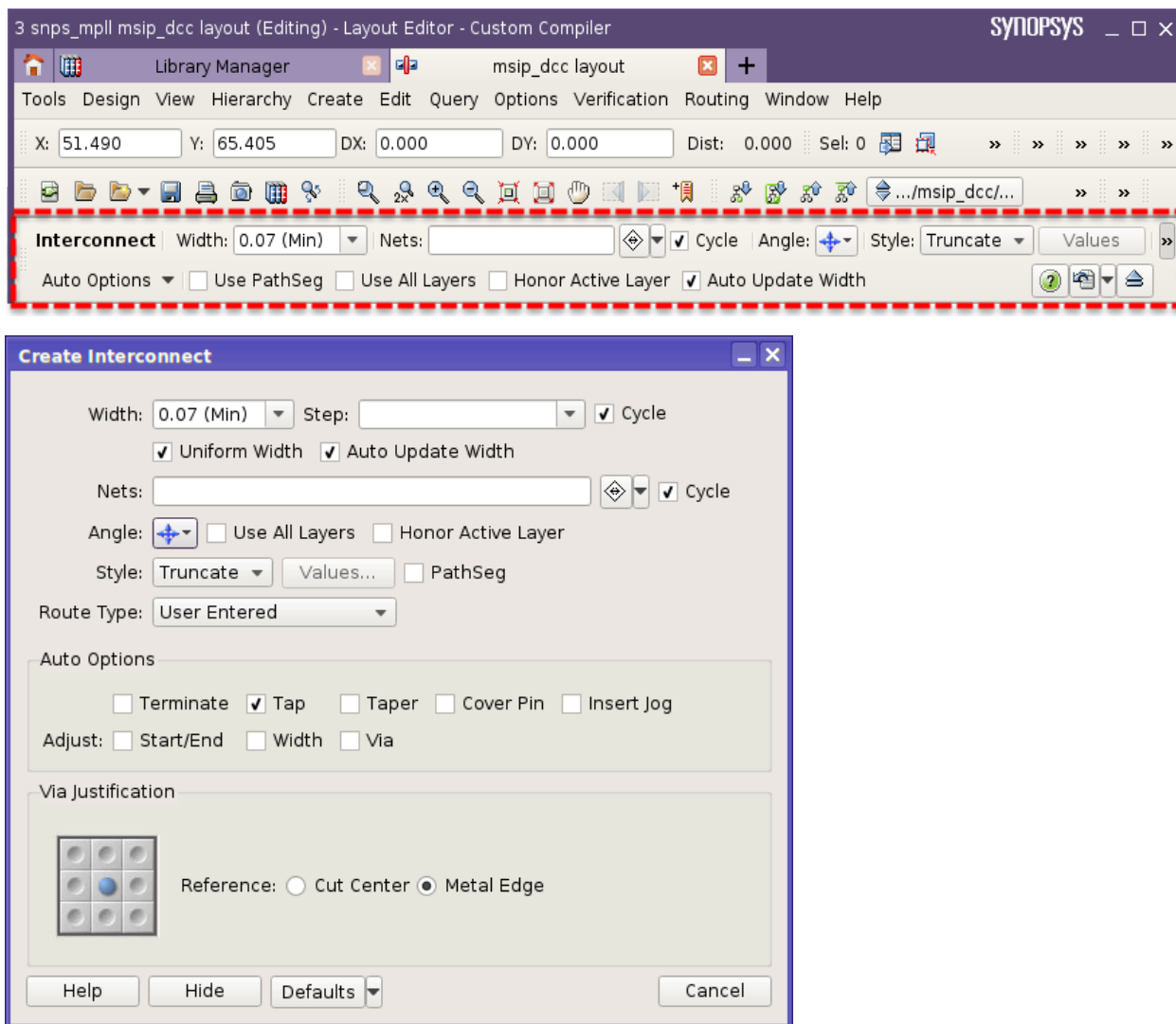
If you are ending at a pin and want the last path segment to align with the pin, right-click the pin and choose **Toggle Start/End Alignment** (or press /) to cycle through the different alignment positions. See [Create Interconnect Menu and Shortcut Keys](#). To accommodate the changing alignment position, the via and the perpendicular path segment at the adjoining junction will automatically adjust position and length, respectively.



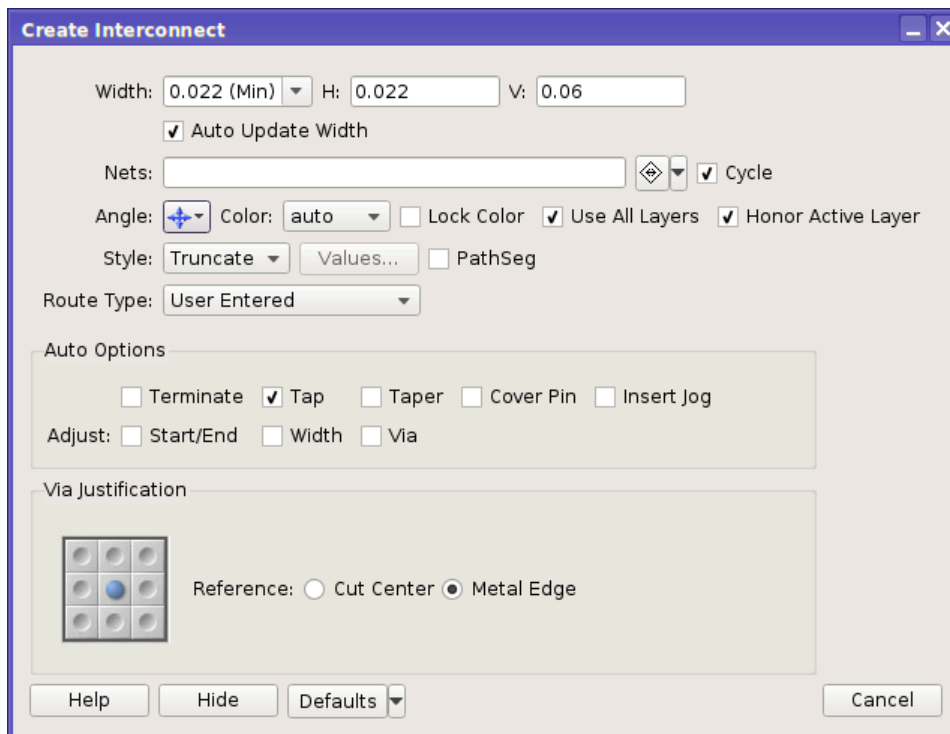
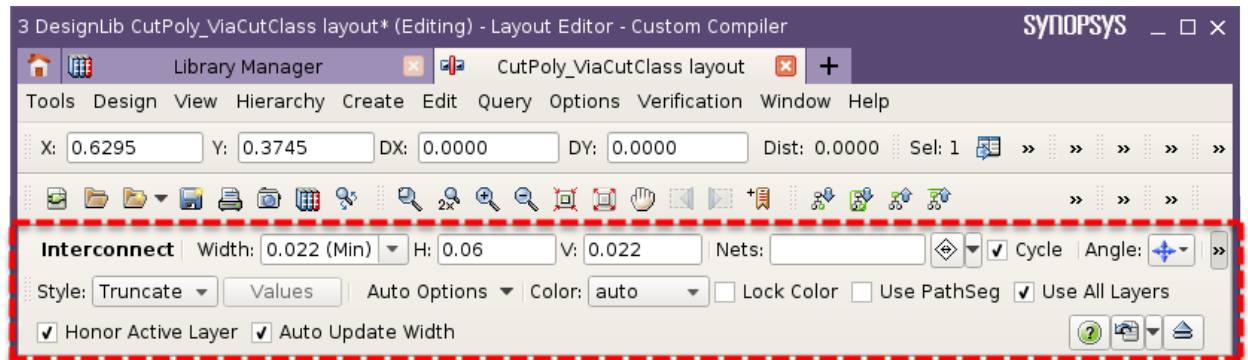
Interconnect Toolbar and Create Interconnect Dialog Box

This section provides a quick reference of the **Interconnect** toolbar and the **Create Interconnect** dialog box. For instructions on how to create interconnects, see [Creating Interconnects](#).

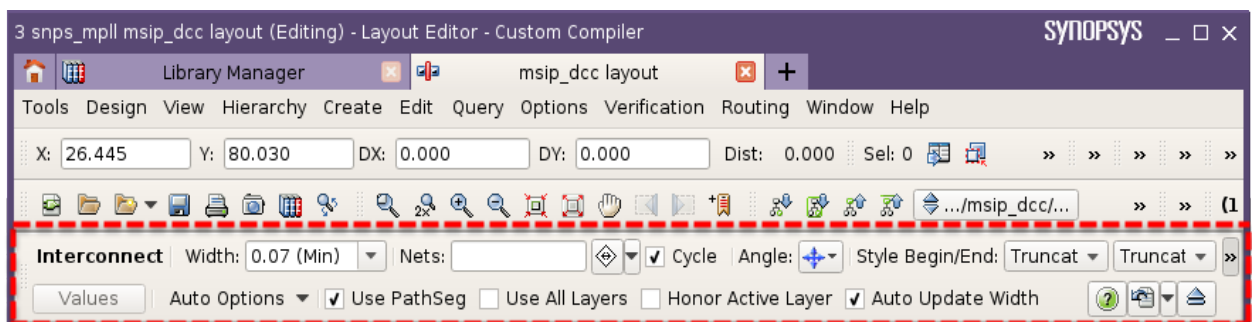
The **Interconnect** toolbar and **Create Interconnect** dialog box can vary depending on the interconnect object you want to draw. The toolbar and dialog box for paths are:

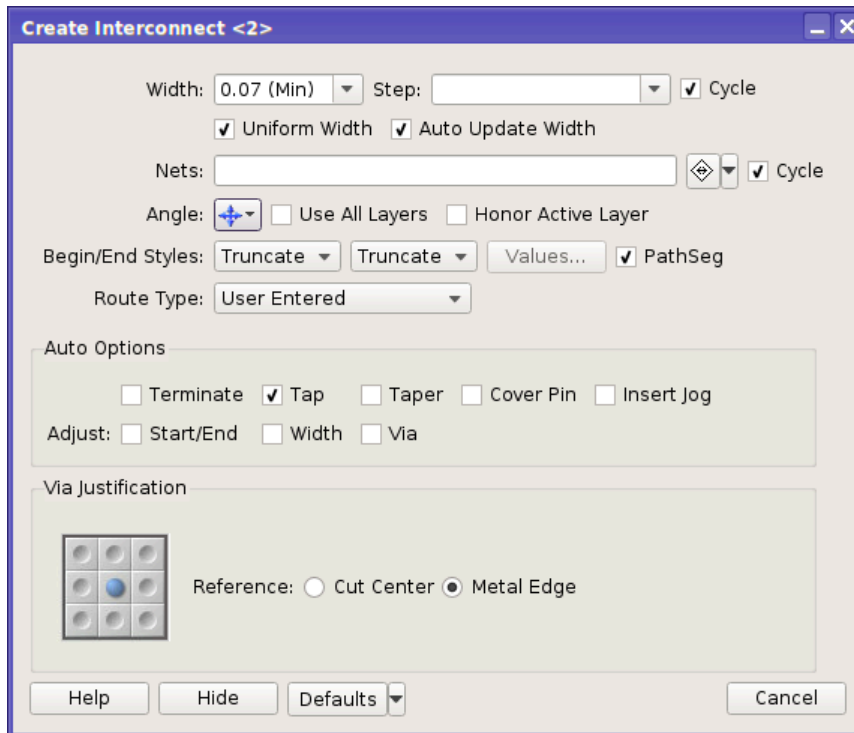


If you specified the `spacingDirection` argument for the `minWidth` or `allowedWidthRange` layer constraints in the technology library, the **H** and **V** options appear on the toolbar.

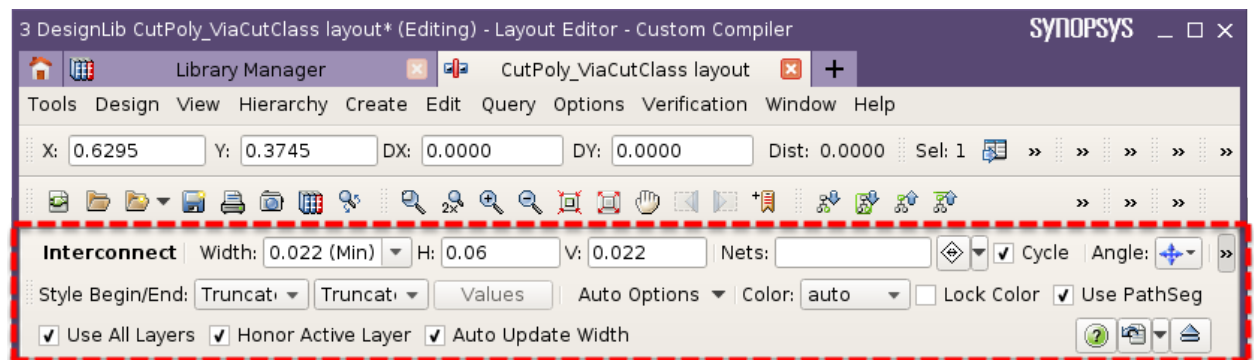


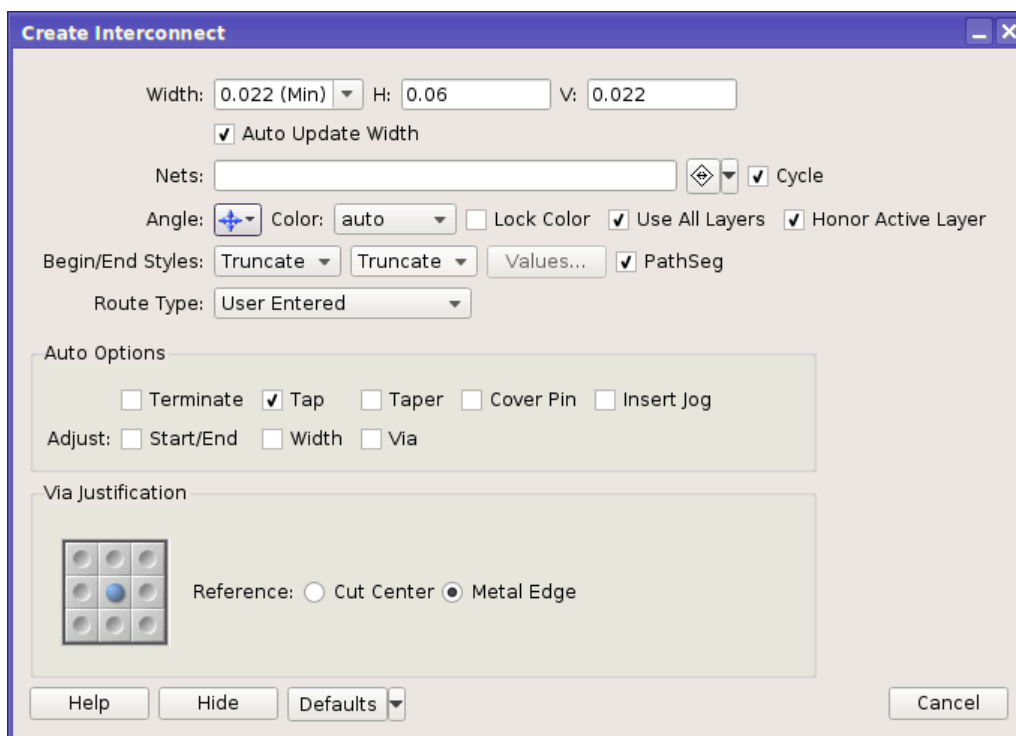
The toolbar and dialog box for path segments are:





If you specified the `spacingDirection` argument for the `minWidth` or `allowedWidthRange` layer constraints in the technology library, the **H** and **V** options appear on the toolbar.





The "Create Interconnect" dialog box is used for configuring interconnect parameters. It includes fields for Width (0.022 Min), H (0.06), and V (0.022), with an "Auto Update Width" checkbox. The "Nets" field is empty, and the "Cycle" checkbox is checked. The "Angle" is set to a default value, and the "Color" is set to "auto". The "Lock Color" checkbox is unchecked, while "Use All Layers" and "Honor Active Layer" are checked. The "Begin/End Styles" are set to "Truncate", and the "PathSeg" checkbox is checked. The "Route Type" is set to "User Entered". The "Auto Options" section includes checkboxes for "Terminate", "Tap" (checked), "Taper", "Cover Pin", and "Insert Jog". The "Adjust" section includes checkboxes for "Start/End", "Width", and "Via". The "Via Justification" section includes a 3x3 grid of icons and a "Reference" section with radio buttons for "Cut Center" and "Metal Edge" (selected). The dialog box has "Help", "Hide", "Defaults", and "Cancel" buttons at the bottom.

Create Interconnect

Width: 0.022 (Min) H: 0.06 V: 0.022

☒ Auto Update Width

Nets:

Angle: Color: auto ☐ Lock Color ☒ Use All Layers ☒ Honor Active Layer

Begin/End Styles: Truncate Truncate Values... ☒ PathSeg

Route Type: User Entered

Auto Options

☐ Terminate ☒ Tap ☐ Taper ☐ Cover Pin ☐ Insert Jog

Adjust: ☐ Start/End ☐ Width ☐ Via

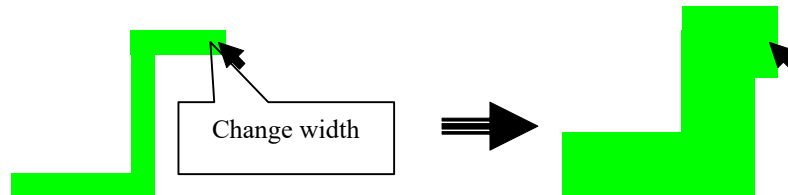
Via Justification

☐ Cut Center ☒ Metal Edge

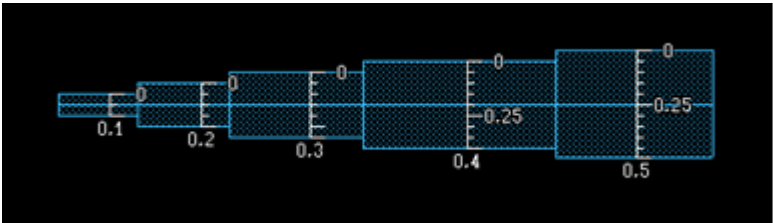
Help Hide Defaults Cancel

Interconnect Options

Option	Description
Width	Sets the width of the current path or path segment. The default value is derived from the <code>minWidth</code> layer constraint in the technology library. If <code>minWidth</code> is not defined in the technology library, the Layout Editor will use the value set by the <code>leDefaultPathWidth</code> Tcl preference. You can set the <code>lePathPresetWidths</code> Tcl preference to define the list of the custom default widths. The defined values will appear as Width selections. The Layout Editor keeps track of the most recently used Width value for each layer purpose pair during the current session. When you begin creating a new interconnect on a previously used layer purpose pair, the toolbar defaults to the previous Width value. However, if you change the Width value, begin drawing on one layer, and then continue drawing on another layer, the changed Width value is retained. If you have not yet created an interconnect on this layer purpose pair, the default width is the minimum width for the layer purpose pair. If you change the Width value after starting to draw the interconnect, the new value applies to all the connected path segments you have created during the command (or to the entire path if you are creating a path).



You can use Shift+4 or Ctrl+4 to toggle between the default and changed **Width** values as you draw.
Tcl preference: `leWidth`

Option	Description
Step	(Not available if you specified the <code>spacingDirection</code> argument for the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraints in the technology library.) Specifies widths to apply to the path or path segment. You must select Cycle to apply the different widths. When you click the layout canvas to start path creation, the first width is applied. To apply the subsequent widths, click the canvas. You can click and add bends in the path on the same layer. You can also layer up or down during the operation. For example, if Step is 0.1 0.2 0.3 0.4 0.5, then the resulting path would appear as shown. The last width value is applied to the remaining path or path segments.  The order of the widths do not have to be in numeric sequence. The following are examples of valid entries: • 0.5 0.4 0.3 • 0.8 0.3 0.6 0.9
Cycle	(Not available only if you specified the <code>spacingDirection</code> argument for the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraints in the technology library.) Cycles through the specified widths in the order shown in the Step box.
Width H	(Available only if you specified the <code>spacingDirection</code> argument for the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraints in the technology library.) Specifies the width of the current horizontal path or path segment.  The default value is derived from the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraints in the technology library, where the <code>spacingDirection</code> argument is vertical. If the width is not defined in the technology library, the Layout Editor will use the value set by the <code>leDefaultPathWidth</code> Tcl preference. The Layout Editor keeps track of the most recently used value for each layer purpose pair during the current session. When you begin creating a new interconnect on a previously used layer purpose pair, the toolbar defaults to the previous value. Tcl preference: <code>leHorizontalWidth</code>

Option	Description
Width V	(Available only if you specified the <code>spacingDirection</code> argument for the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraints in the technology library.) Specifies the width of the current vertical path or path segment.  Width V The default value is derived from the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraint in the technology library, where the <code>spacingDirection</code> argument is <code>horizontal</code>. If the width is not defined in the technology library, the Layout Editor will use the value set by the <code>leDefaultPathWidth</code> Tcl preference. The Layout Editor keeps track of the most recently used value for each layer purpose pair during the current session. When you begin creating a new interconnect on a previously used layer purpose pair, the toolbar defaults to the previous value. Tcl preference: <code>leVerticalWidth</code>
Uniform Width	(Taper is not selected) Requires that all interconnect segments have uniform width. Clear the check box in designs where segments can have different widths, such as fanout designs. By default, the option is selected.
Auto Update Width	When enabled (default), updates the width to match the minimum width of the selected layer purpose pair after changing the active layer purpose pair. If the width has been modified, retains the last modified width of the selected layer purpose pair when changing layers. When disabled, applies the last modified width to the selected layer purpose pair when changing the active layer purpose pair. Tcl preference: <code>leInterconnectAutoUpdateWidth</code>
Nets	Sets the net names of paths and vias. See Naming Nets. Tcl preference: <code>leNetName</code>
 (Synchronize)	Copies the names of selected nets in the Design Navigator and pastes them in the Nets box. The order of the net names in the Nets box matches the order of the selected nets in the Design Navigator.

Option	Description
Cycle	When enabled, deletes the first net name from the Nets box as you create a corresponding object. When disabled, remains the net name after the object is committed. Tcl preference: <code>leCycleName</code>
Angle	Specifies the angle of the interconnect when it changes direction: • Orthogonal: The new segment is 90° of the current segment. • Diagonal: The new segment is 45° of the current segment. • Any Angle: The new segment can be at any angle of the current segment. • X First: As you move diagonally from the previous point, a horizontal segment is added first, followed by a vertical segment. • Y First: As you move diagonally from the previous point, a vertical segment is added first, followed by a horizontal segment. If you are creating path segments, the Diagonal option automatically replaces the Any Angle option. With X First or Y First selected, adding a single point can create two path segments. See Angle Mode for Edges and Centerlines. Tcl preference: <code>leCreateAngleMode</code>
Style	(Paths only) Specifies one style for both endpoints of paths. The available styles are Truncate, Extend, Round, and Variable. The tool adjusts the end styles of the interconnect that you extend if necessary. See Choosing End Styles. Tcl preference: <code>lePathStyle</code>
Begin/End Styles	(Path Segments only) Sets the begin/end styles for path segments. The available styles are Truncate, Extend, Chamfer, Custom, and Variable. When creating a series of path segment objects, the end style setting only affects the first and last endpoints. The tool adjusts the begin/end styles of the interconnect that you extend if necessary. See Choosing End Styles. Tcl preference: <code>lePathSegBeginStyle</code> and <code>lePathSegEndStyle</code>
Values	Enabled when Style is set to Variable (for paths) or Begin/End Styles is set to Custom/Variable (for path segments). Pressing the Values button opens the Path End Style (for paths) or PathSeg End Style (for path segments) dialog box so you can set the beginning and ending extension distance in user units. The default is 0.0.

Option	Description
Color	Assigns a color to the selected shape as follows: • Auto: Automatically assigns a color to the new shape based on the color propagation rule. If no propagation can be applied, then the shape or via is assigned the default color. This selection is available only if you set the <code>leColorName</code> preference to <i>auto</i>. • <colorName>: Assigns the selected color, <colorName>, to the newly created shape. The available color names are defined in the technology file. • Same: Assumes that nearby shapes have the same color and applies the <code>multiPatternSpacing</code> constraint. This selection is available only if you set the <code>leIncludeColorSame</code> preference to <i>true</i>. • No Color: Does not apply any color to the newly created shape. This selection is available only if you set the <code>leIncludeColorNone</code> preference to <i>true</i>.
Lock Color	Locks or unlocks the assigned color. Locking and unlocking a shape's color changes its ability to propagate its color to or inherit color from overlapping shapes. For more information, see Color Propagation.
Use PathSeg	When enabled, creates interconnect objects with path segments. When disabled, create interconnect objects with paths. This option automatically updates the Interconnect Object option status in the Command tab of the Layout Design Options dialog box (see Setting Layout Design Options). Tcl preference: <code>leUsePathSeg</code>
Use All Layers	Uses all layers defined in the technology database.
Honor Active Layer	When enabled, gets the layer purpose pair (LPP) from the active layer on the Object/Layer Panel. When disabled (default), gets the layer purpose pair from the active object when Taper is selected. Tcl preference: <code>leInterconnectHonorActiveLayer</code>
Route Type	The type of route (for example, signal, PG, clock, and so on). Choose the route type that is applicable to the path you are creating. This options is used primarily in analog and digital co-designing.

Auto Options

Option	Description
Auto Options	Sets the auto options. • Terminate: Commits the route if you complete a connection by adding a point that overlaps a shape or via having an electrically equivalent layer purpose pair. • Tap: Queries the shape under the first point to set the layer purpose pair, width, and end style. See Tapping. • Taper: Automatically adjusts the width of the interconnect to match the connecting terminal or wire. Taper permits tapping of objects on other hierarchy levels while in the Edit in Place mode. Tapping sets the layer-purpose pair (LPP) of the new object to match the LPP of the existing object. If Taper is enabled, the Terminate, Tap, Adjust Width, and Adjust Start/End options are grayed out. The Width value remains unchanged. If Taper and Adjust Width are both enabled, the Width value changes with the interconnect snapping to a pin or rectangle. • Cover Pin: Ensures that the interconnect fully covers the connecting pin. Cover Pin works with Taper to ensure that the tapering will fully cover the pin. Clear the check box if you want the tapering to start at the selected edge of the pin. Cover Pin works with Tap to ensure that the interconnect fully covers the connecting pin, and the interconnect changes its layer-purpose pair to match the pin. • Insert Jog: Inserts a jog at the end of a path segment. • Drop Via: Adds vias and connects to layers or pins above or below the routing layer. The connecting layers or pins must be on the same net as the routing layer. To prevent extra vias from being created during editing operations such as Copy or Move, turn off Vias and End Styles: Auto Update, or turn on Vias and End Styles: Auto Update and select End Styles. See the Command Tab in the Layout Design Options dialog box. If the <code>leTaperFromSelectedEdge</code> Tcl preference is set to <code>true</code>, the width of the interconnect will taper to match the width of the selected pin or its edge when you start to create the interconnect. If you set the preference to <code>false</code>, the width of the interconnect is adjusted as you move the cursor. Tcl preferences: <code>leAutoTerminate</code>, <code>leInterconnectTap</code>, <code>leInterconnectTaper</code>, <code>leTaperToCoverPin</code>, and <code>leInterconnectInsertJog</code>

Option	Description
Adjust	(Dialog box only) When creating an interconnect, the cursor is tied to the interconnect centerline. Consequently, snapping to reference points would result in overlap. To align interconnect properly, select one or more of the following preferences: • Width: Sets the interconnect width equal to the width of the object over which the interconnect starts in the direction of the routing. • Start/End: Aligns the interconnect with the edges of the object over which the interconnect starts or finishes. • Via: Fills the overlap of the interconnect and the shape under the start or end point when the Terminate option is selected. Tcl preferences: <code>leInterconnectAdjustStartEnd</code>, <code>leInterconnectAdjustWidth</code>, and <code>leInterconnectAdjustVia</code>

Via Justification

Option	Description
Via Justification	(Dialog box only) Specifies the justification for via origin by Upper Left, Center Left, Lower Left, Upper Center, Center Center, Lower Center, Upper Right, Center Right, or Lower Right. The default is Center Center. See Positioning the Vias. This option works with the Reference option and only applies for cases with one via between adjacent metal layers; it does not affect via stacks. Tcl preference: <code>leViaJustification</code>
Reference	(Dialog box only) Specifies whether Via Justification is based on the Cut Center (default) or the Metal Edge. See Positioning the Vias. This option works with the Via Justification option and only applies for cases with one via between adjacent metal layers; it does not affect via stacks. Tcl preference: <code>leViaJustificationMaterial</code>

Create Interconnect Menu and Shortcut Keys

While the **Create > Interconnect** command is active, you can right-click to display the Create Interconnect shortcut menu. This section provides a quick reference of the shortcut menu and shortcut keys. For instructions on how to create individual interconnects, see [Creating Interconnects](#).

Note:

The bridge via commands are only available if you have defined bridge vias in the technology database.

CommitNum+Enter

Add PointSpace

Remove PointBackspace

Toggle Start/End Alignment/

Angle

Auto

Use Path

Use PathSeg

Via Justification

Reference

Rotate Via/Enclosure

Layer UpShift+V

Layer DownCtrl+V

Bridge Via UpShift+S

Bridge Via DownCtrl+S

Add Via ...V

Add Bridge Via ...S

Orthogonal

Diagonal

Any Angle

X First

Y First

CommitNum+Enter

Add PointSpace

Remove PointBackspace

Toggle Start/End Alignment/

Angle

Auto

Use Path

Use PathSeg

Via Justification

Reference

Rotate Via/Enclosure

Layer UpShift+V

Layer DownCtrl+V

Bridge Via UpShift+S

Bridge Via DownCtrl+S

Add Via ...V

Add Bridge Via ...S

Terminate

Tap

Taper

Adjust Width

Adjust Start/End

Adjust Via

Cover Pin

Insert Jog

Drop Via

CommitNum+Enter

Add PointSpace

Remove PointBackspace

Toggle Start/End Alignment/

Angle

Auto

Use Path

Use PathSeg

Via Justification

Reference

Rotate Via/Enclosure

Layer UpShift+V

Layer DownCtrl+V

Bridge Via UpShift+S

Bridge Via DownCtrl+S

Add Via ...V

Add Bridge Via ...S

Upper Left

Center Left

Lower Left

Upper Center

Center Center

Lower Center

Upper Right

Center Right

Lower Right

CommitNum+Enter

Add PointSpace

Remove PointBackspace

Toggle Start/End Alignment/

Angle

Auto

Use Path

Use PathSeg

Via Justification

Reference

Rotate Via/Enclosure

Layer UpShift+V

Layer DownCtrl+V

Bridge Via UpShift+S

Bridge Via DownCtrl+S

Add Via ...V

Add Bridge Via ...S

Cut Center

Metal Edge

CommitNum+Enter

Add PointSpace

Remove PointBackspace

Toggle Start/End Alignment/

Angle

Auto

Use Path

Use PathSeg

Via Justification

Reference

Rotate Via/Enclosure

Layer UpShift+V

Layer DownCtrl+V

Add Via ...V

ViaR

Lower Layer EnclosureCtrl+R

Upper Layer EnclosureShift+R

Command	Purpose	Default Shortcut Keys
Commit	Completes the operation if it is valid.	Enter
Add Point	Inserts another point along the interconnect.	Space
Remove Point	Deletes the previously inserted point.	Backspace

Command	Purpose	Default Shortcut Keys
Toggle Start/End Alignment	Cycles through the different interconnect-pin alignment positions: - Aligns the interconnect with the left edge of the pin - Aligns the interconnect with the right edge of the pin - Aligns the interconnect with the center of pin - No alignment (the original position of the interconnect) To cycle through the alignment styles, you must disable Insert Jogs.	/
Angle	Specifies the drawing angle by Orthogonal (default), Diagonal, Any Angle, X First, or Y First. If you are creating path segments, the Diagonal option automatically replaces the Any Angle option. With X First or Y First selected, adding a single point can create two path segments. See Angle Mode for Edges and Centerlines.	Right-click to toggle the X First or Y First angle mode only if one of these modes is active.

Command	Purpose	Default Shortcut Keys
Auto	Specifies the automatic options to apply when creating interconnects: • Terminate: Commits the route if you complete a connection by adding a point that overlaps a shape or via having an electrically equivalent layer purpose pair. • Tap: Queries the shape under the first point to set the layer purpose pair, width, and end style. See Tapping. • Taper: Automatically adjusts the width of the interconnect to match the connecting terminal or wire. Taper permits tapping of objects on other hierarchy levels while in the Edit in Place mode. Tapping sets the layer-purpose pair (LPP) of the new object to match the LPP of the existing object. If Taper is enabled, the Terminate, Tap, Adjust Width, and Adjust Start/End options are grayed out. The Width value remains unchanged. If Taper and Adjust Width are both enabled, the Width value changes with the interconnect snapping to a pin or rectangle. • Adjust Width: Sets the interconnect width equal to the width of the object over which the interconnect starts in the direction of the routing. • Adjust Start/End: Aligns the interconnect with the edges of the object over which the interconnect starts or finishes. • Adjust Via: Fills the overlap of the interconnect and the shape under the start- or endpoint when the Terminate option is selected. • Cover Pin: Ensures that the interconnect fully covers the connecting pin. Cover Pin works with Taper to ensure that the tapering will fully cover the pin. Clear the check box if you want the tapering to start at the selected edge of the pin. Cover Pin works with Tap to ensure that the interconnect fully covers the connecting pin, and the interconnect changes its layer-purpose pair to match the pin. • Insert Jog: Inserts a jog at the end of a path segment. • Drop Via: Adds vias and connects to layers or pins above or below the routing layer. The connecting layers or pins must be on the same net as the routing layer. To prevent extra vias from being created during editing operations such as Copy or Move, the tool automatically turns off Vias and End Styles: Auto Update, or automatically turns on Vias and End Styles: Auto Update and End Styles. See the Command Tab in the Layout Design Options dialog box.	

Command	Purpose	Default Shortcut Keys
Use Path	Creates interconnect objects with paths.	
Use PathSeg	Creates interconnect objects with path segments.	
Via Justification	Specifies the justification for via origin by Upper Left , Center Left , Lower Left , Upper Center , Center Center (default), Lower Center , Upper Right , Center Right , or Lower Right . This option only applies for cases with one via between adjacent metal layers; it does not affect via stacks.	
Reference	Specifies whether Via Justification is based on Cut Center (default) or Metal Edge . See Positioning the Vias . This option only applies for cases with one via between adjacent metal layers; it does not affect via stacks.	
Rotate Via/Enclosure	Rotates one of the following: • Via: Rotates the entire via. • Lower Layer Enclosure: Rotates only the metal layer below the cuts. • Upper Layer Enclosure: Rotates only the metal layer above the cuts. See Adding Vias Manually for how the vias and enclosures are rotated.	R Ctrl+R Shift+R
Layer Up	Inserts a single via at a layer above the current layer. For example, if you want insert a via two layers above the current layer, press Shift+V two times.	Shift+V
Layer Down	Inserts a single via at a layer below the current layer. For example, if you want to add a via three layers below the current layer, press Ctrl+V three times.	Ctrl+V
Bridge Via Up	Inserts a bridge via at a layer above the current layer.	Shift+S
Bridge Via Down	Inserts a bridge via at a layer below the current layer.	Ctrl+S
Add Via	Opens the Add Via dialog box which you can use to place a via or via stack. See Adding Custom Vias or a Via Stack on page 193 .	V
Add Bridge Via	Opens the Add Bridge Via dialog box. See Adding Custom Bridge Vias on page 197 .	S

Create Interconnect Context-Sensitive Toolbar

The context-sensitive toolbar is displayed when the `ile::createInterconnect` command is activated followed by right-clicking.



Table 2 *ile::createInterconnect options*

Options	Description
Width	Defines width of the current path or pathSeg object. Preference: <code>leWidth</code> Note: The width text input is automatically in focus after the CST is displayed.
Nets	Defines net name(s) to be associated with the routing objects that are to be created. Preference: <code>leNetName</code>
Sync button	Synchronizes the net name(s) selected in Design Navigator. Preference: <code>leDNButton</code>
Cycle	Defines if first net name in the net name field is removed or remains after object is committed. Preference: <code>leCycleName</code>
Use All Layers	Enables all layers for interconnect creation. Preference: <code>leUseAllLayers</code>
PathSeg	Enables pathSeg object for interconnect creation. Preference: <code>leUsePathSeg</code>
Taper	Defines whether interconnect supports auto taper or not. Preference: <code>leInterconnectTaper</code>
Drop Via	Supports auto drop via in ON/OFF state. Preference: <code>leInterconnectDropVia</code>

Table 2 *ile::createInterconnect options (Continued)*

Options	Description
Color	Defines the color name and the color enumerator to be associated with interconnect paths / pathSegs. Preference: <code>leColorName</code> Note: - This widget is enabled only when the selected layer has more than one color. In addition to tech-defined colors, “auto”, “none” or “same” can also be shown in the option list. “None” and “same” are controlled by preferences <code>leIncludeColorNone</code> and <code>leIncludeColorSame</code> respectively. “Auto” is always shown. It means LE commands determine the color automatically by tap or colored track. If both cannot determine color, tech defined default color is used. The only exception is, if no default color is defined and “none” is prohibited by preference, command uses the first legal color. - This option is invisible if the current layer has no color.
Lock Color	Defines if interconnect color is locked. Note: - If this option is set to true, the color on interconnect is locked. - If the current layer has no color, this option is invisible.
Stack commands tool button	The Set Cursor action is disabled when you have not specified the first point.

Selecting Layer Purpose Pairs for Interconnects

You can select layer purpose pairs (LPPs) for interconnect in the Object/Layer Panel or by tapping an existing object. For more information about tapping, see [Tapping](#).

The layer purpose pair must:

- Be visible and selectable in the Object/Layer Panel
- Be an interconnect layer purpose pair.

Sometimes you do not want a router to connect to a shape on an interconnect layer. For example, you might want a router to connect to the pins of a poly resistor but not the interior. You can designate specific shapes on interconnect layers as highly resistive to indicate that they are unsuitable for routing. To do so, assign the `ceHighlyResistiveLayer` attribute with the Property Editor.

Note:

If you start drawing interconnect while the purpose of the active layer is not a routing purpose, the tool automatically selects a routing purpose based on the priority in the technology file.

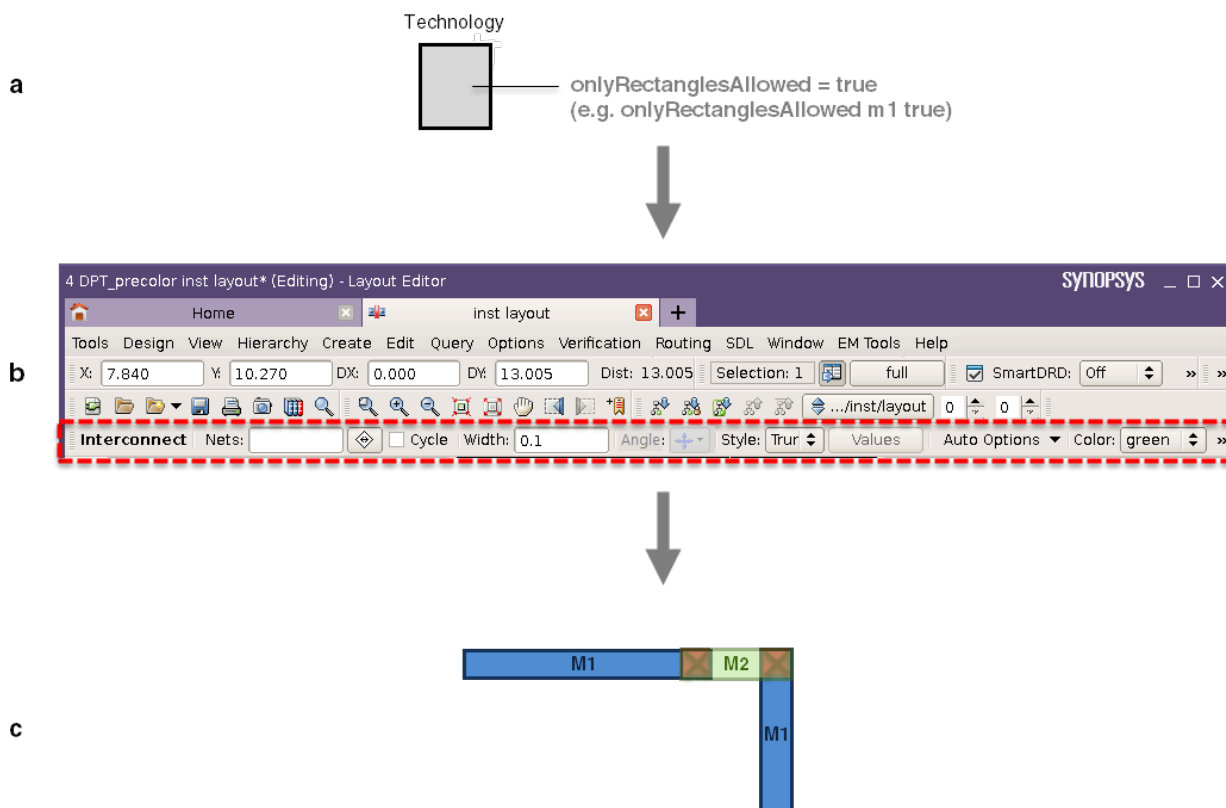
Connecting Routes on the Same Layer

If the `onlyRectanglesAllowed` layer constraint in the technology is set to *true* for specified layers ([Figure 1a](#)):

- The **Angle** option is disabled on the **Interconnect** toolbar ([Figure 1b](#)).
- The interconnects that you create on the specified layers can only be placed horizontally or vertically. To connect orthogonal shapes on the same layer, you must add vias and connecting shapes on layers above or below the orthogonal shapes ([Figure 1c](#)).

Note that a path cannot have more than one segment in this mode.

Figure 1 Connecting Orthogonal Routes on the Same Layer



If one of the following interconnect position and **Via Justification** pairings occurs, the Layout Editor adjusts the position, end style, and width of the interconnects to ensure that the via is fully enclosed:

Interconnect Position	Via Justification
Horizontal	Center-Left Center-Center Center-Right
Vertical	Lower-Center Center-Center Upper-Center

For information about setting the **Via Justification** option, see [Interconnect Toolbar and Create Interconnect Dialog Box](#).

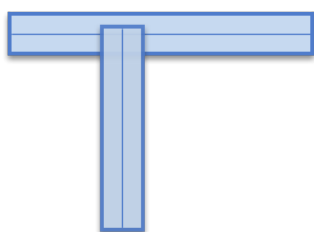
For information about the `onlyRectanglesAllowed` layer constraint, see the *Custom Compiler Technology and Display Resource Reference Manual*.

Changing Interconnect Layers

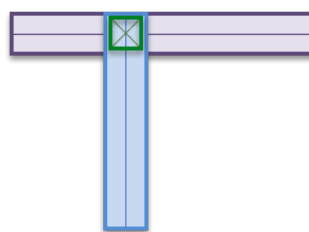
You can modify existing route components to optimize the design. For example, you can change the layers on which the interconnects are placed. During the layer change operation:

- If the layer change causes two connecting shapes on the same layer to be on different layers, the Layout Editor will automatically add vias to ensure that connectivity is retained.

Before Layer Change

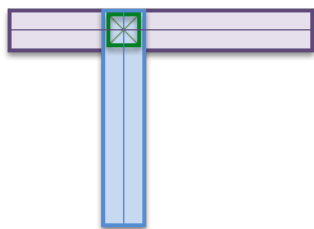


After Layer Change

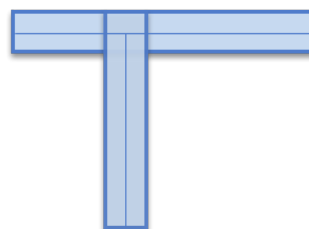


- If the layer change causes connecting shapes on different layers to reside on the same layer, the Layout Editor will remove any existing vias that are no longer necessary.

Before Layer Change



After Layer Change



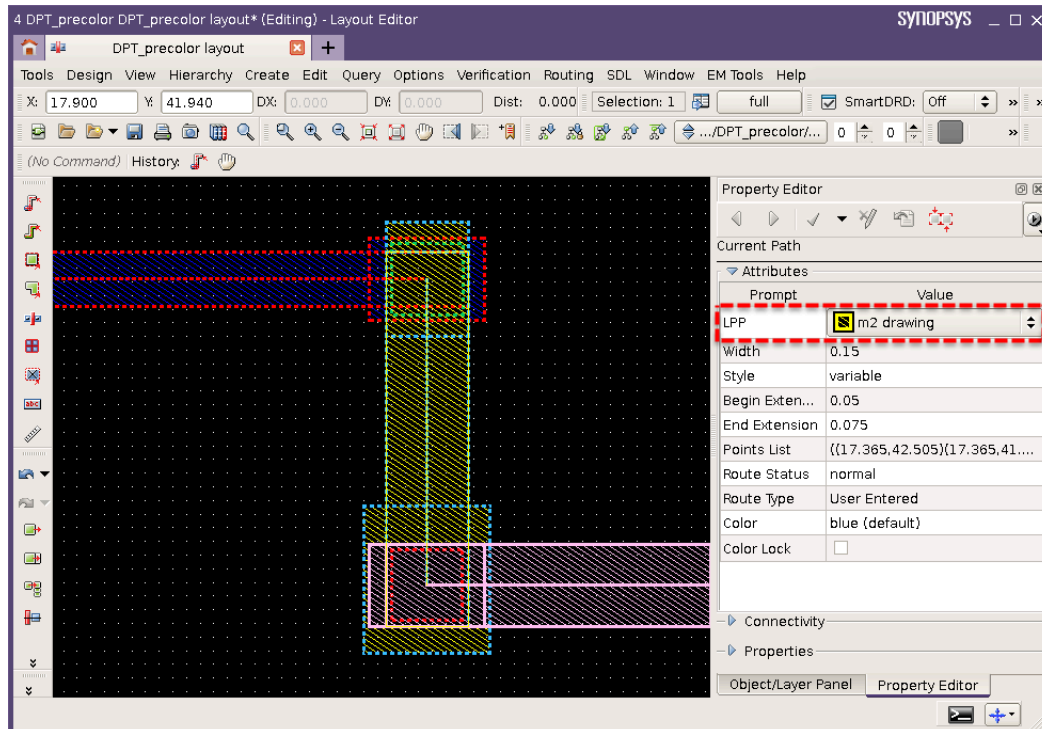
- Before vias are added or removed, the Layout Editor can make the necessary modifications, such as the wire width and end styles, to ensure that vias are properly enclosed and the wires remain well formed.

Note:

To enable the automatic addition and removal of vias during layer changes, you must set the `leUpdateConnectedViasAndEndStyles` Tcl preference to `true`. For more information, see the *Custom Compiler Tcl Reference Manual*.

To change the layer of an interconnect:

1. In the Layout Editor, display the Property Editor.
2. On the canvas, select the interconnect whose layer you want to change. The properties of the selected interconnect appears in the Property Editor.
3. In the Property Editor, locate the **Attribute** section, and then select the desired layer from the **LPP** list.



Avoiding Overlap

When creating an interconnect, the cursor is tied to the interconnect centerline. Consequently, snapping to reference points would result in overlap. To align interconnect properly, set one or more of the following preferences to *true*.

- `leInterconnectAdjustWidth`

When creating an interconnect, this preference sets the interconnect width equal to the width of the object over which the interconnect starts or finishes.

- `leInterconnectAdjustStart`

When starting or ending an interconnect, this preference aligns the interconnect with the edges of the object over which the interconnect starts or finishes.

- `leInterconnectAdjustVia`

When starting or ending interconnect with a via, this preference fills the overlap of the interconnect and the shape under the start or end point.

- `leInterconnectJog`

When creating a single segment interconnect and simultaneously aligning start or end points, this preference inserts a jog if one is required to complete the alignment.

Choosing End Styles

The **Style** list on the **Interconnect** toolbar lets you control the default end styles for paths and path segments by one of the following options: **Truncate**, **Extend**, **Round**, and **Variable**.

Alternatively, you can specify values for the following preferences:

- `lePathStyle`

Sets the style of both the first and the last end points on paths.

- `lePathSegBeginStyle` and `lePathSegEndStyle`

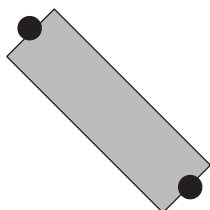
Set the style of the first and last points along a series of one or more connected path segments.

Although OpenAccess defines different sets of end styles for paths and path segments, the Layout Editor supports only the styles shown in [Figure 2](#) and [Figure 3](#).

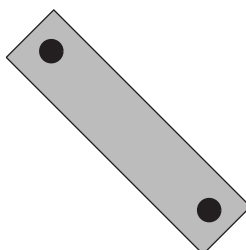
When creating interconnects, ensure that the endpoints of paths are automatically adjusted to be well-formed and DRC-clean at ends when touching vias.

Figure 2 Path End Styles

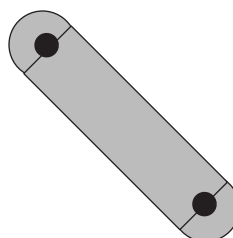
Truncate



Extend



Round



Variable

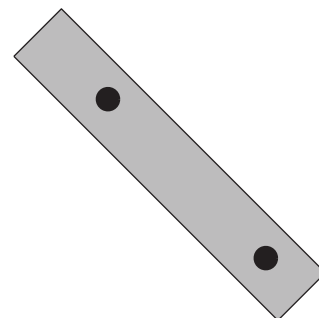


Figure 3 Path Segment End Styles

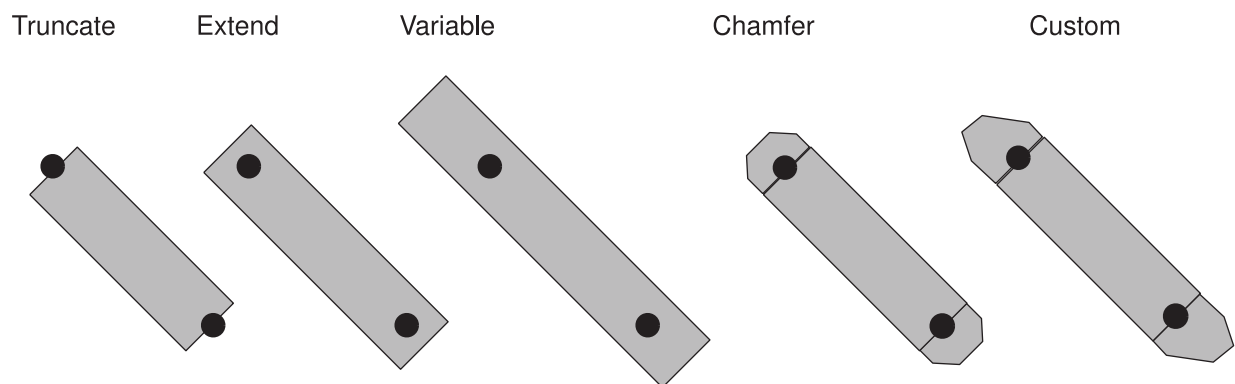
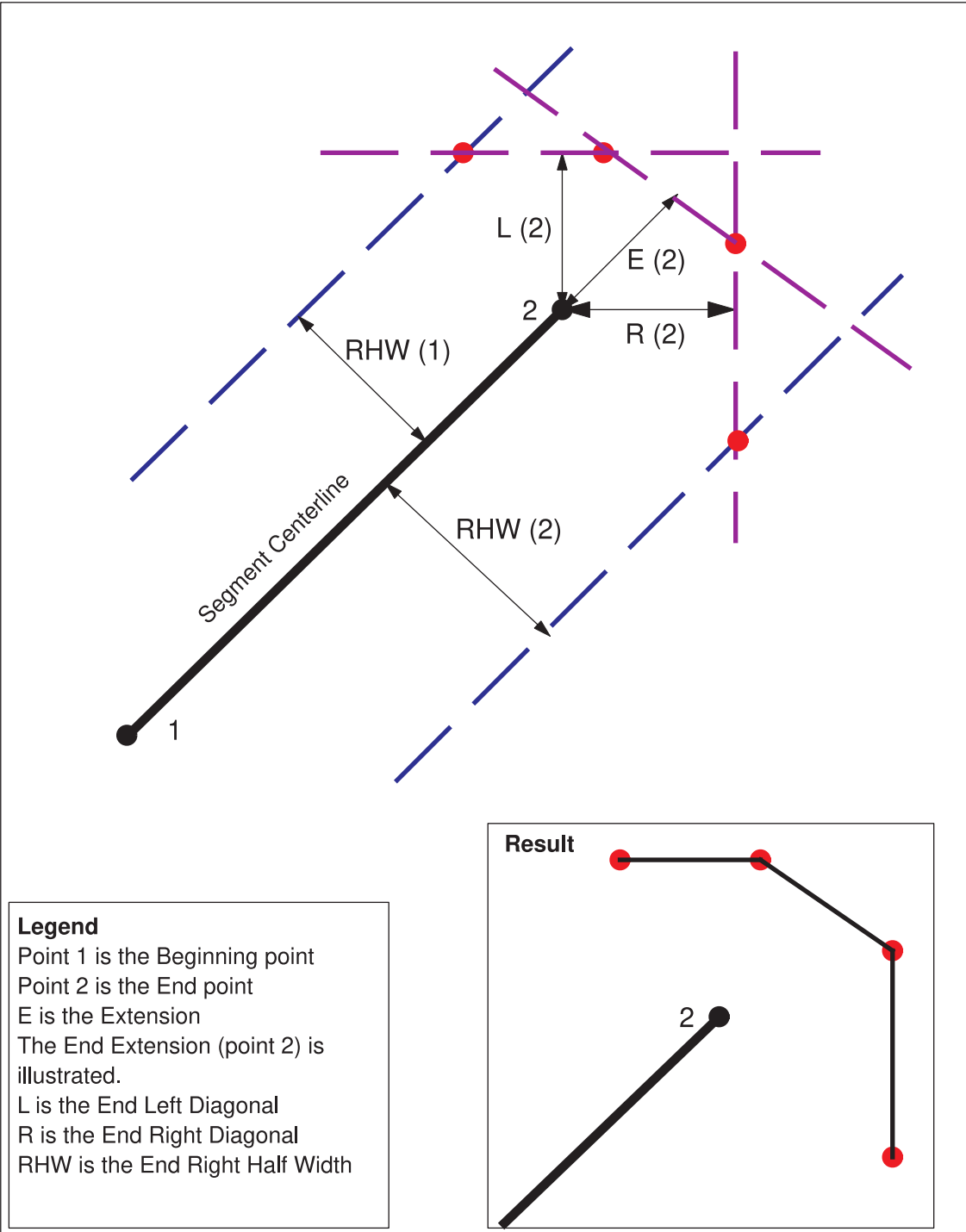


Figure 4 Specifying the Parameters for the Custom End Style



Adding Vias Between Interconnects

You can add vias as you create interconnects. This topic describes the following:

- [Before You Start](#)
- [Adding Default Vias](#)
- [Adding Custom Vias or a Via Stack](#)
- [Adding Default Bridge Vias](#)
- [Adding Custom Bridge Vias](#)
- [Adding Vias Between Overlapping Interconnects](#)
- [Positioning the Vias](#)
- [Add Via Dialog Box Reference](#)
- [Add Bridge Via Dialog Box](#)

For general information about vias, see [About Vias](#). To modify the vias, see [Modifying Via Attributes and Properties](#).

Before You Start

The vias you add must be defined in the technology database. See Via Definitions Section in the technology database.

Adding Default Vias

To add default vias while creating interconnects:

1. While creating the interconnect, at the point where you want to add the default via on the layout canvas, right-click and choose one of the following from the shortcut menu that appears:
 - **Layer Up:** Adds a default via at a layer above the current layer.
 - **Layer Down:** Adds a default via at a layer above the current layer.

See [Create Interconnect Menu and Shortcut Keys](#).

2. On the layout canvas, click to add the via.

3. Continue creating interconnects, or double-click to end the interconnect at the via.
If you continue the interconnect from the added via, the layer-purpose pair changes appropriately.
4. When you are finished, press Esc to quit the command.

Adding Custom Vias or a Via Stack

When adding a custom via or via stack, consider the following:

- If you add a custom via, all of the parameters have the default values. To change the values, use the Property Editor.
- In cases when via stack rules are defined in the technology file and a rule violation occurs, the via stack staggers automatically on certain layer-purpose pairs to avoid the violation.

For information about defining via stacks and specifying staggering rules, see the `viaStackingAllowed`, `viaStackLimit`, and `minViaClearance` descriptions in the *Custom Compiler Technology and Display Resource Reference Manual*.

To add a custom via or a via stack while creating interconnects:

1. While creating interconnects, at the point where you want to add the custom via or via stack on the layout canvas, right-click and choose **Add Via**. The **Add Via** dialog box opens.

Layer	Via Definition	Cut Class	Spacing	Layer1 Enc	Layer2 Enc	Metal Color	Cut Color
POLY			auto	auto			
M0POLY	POLY	auto	auto	auto	auto	Not Colorable	Not Colorable
MODIFF1	M0POLY	auto	auto	auto	auto	Not Colorable	Not Colorable
MODIFF2	auto	auto	auto	auto	auto	Not Colorable	Not Colorable
M1	auto	auto	auto	auto	auto	auto	Not Colorable
M2	auto	auto	auto	auto	auto	auto	auto
M3	auto	auto	auto	auto	auto	auto	auto

☐ Define Via Size Manually

☒ Cols/Rows: 1 1

☐ Width/Height: 0.01 0.01 ☐ Keep Value

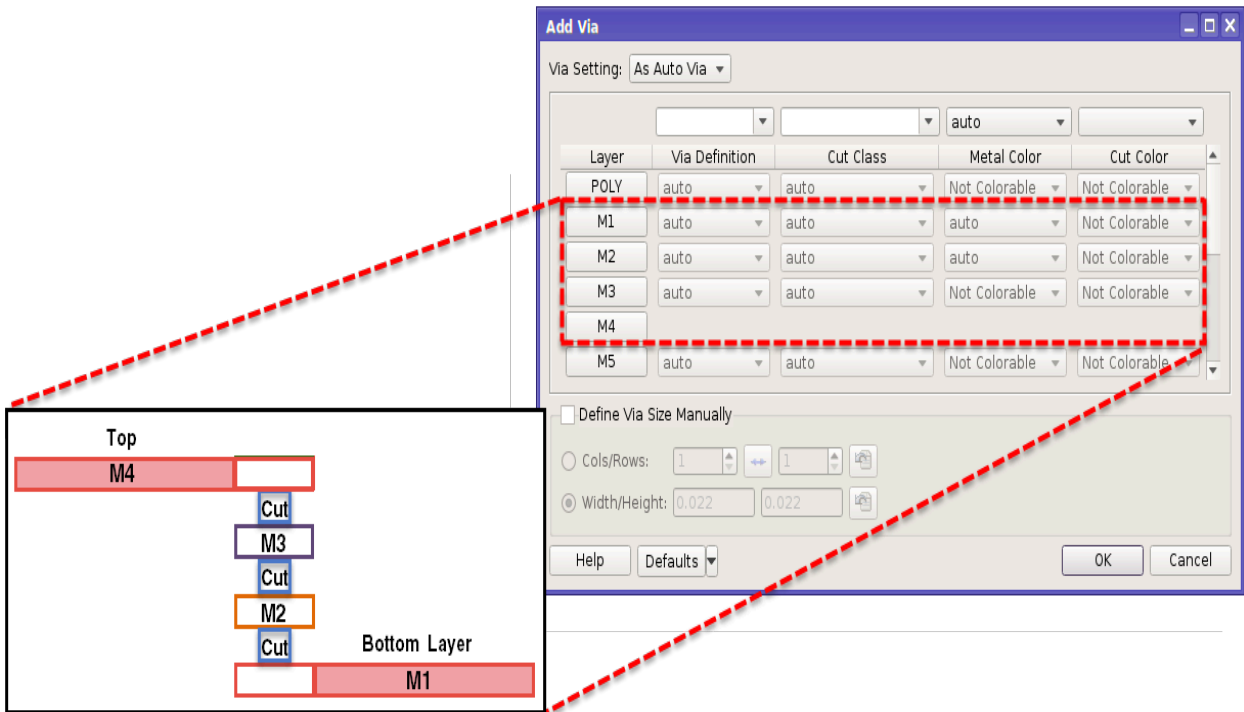
Help Defaults OK Cancel

2. In the **Via Setting** list, choose one of the following:

Argument	Description
Local	The Via Definition and Cut Class settings you specify in this dialog box apply only to the Create Interconnect > Add Via command.
As Auto Via	The Via Definition and Cut Class settings you specify in this dialog box are synchronized with the Create Via (Auto mode) command.

3. In the **Layer** column, click the layer to which you want to connect.

In the example shown, an interconnect is created on layer m4. To connect to m1, you would click **m1**. The **Metal Color** label changes to **Bottom Metal Color**. (If you created the interconnect on the m1 layer and you click m4 to connect to it, the Metal Color label would change to Top Metal Color.)



4. (Optional) Set the remaining options in the dialog box. See [Add Via Dialog Box Reference](#).

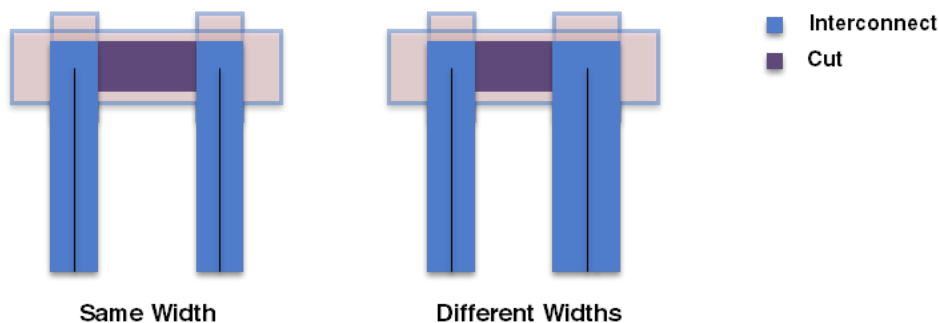
5. When you are finished, click **OK**.

6. On the layout canvas, click to place the via. The active layer purpose pair changes to match the other routing layer of the via.

7. Continue drawing interconnects, or double-click to end the interconnect at the via.
8. When you are finished, press Esc to quit the command. The Layout Editor fills the overlap with the maximum legal number of vias.

Adding Default Bridge Vias

Instead of using two vias and a rectangle to connect two wires on adjacent tracks, you can use bridge vias to define and create a single via for the same purpose. Bridge vias must be defined in the technology database. If the bridge via definition contains a second interconnect width value (second leg width), then the width of the second interconnect will change when you place the bridge via. See the Custom Compiler Technology and Display Resource File Reference Manual.



To insert default bridge vias while creating interconnects:

1. While creating the interconnect, at the point where you want to add the bridge via, right-click and choose one of the following commands from the menu that appears:
 - **Bridge Via Up:** Inserts a bridge via at a layer above the current layer.
 - **Bridge Via Down:** Inserts a bridge via at a layer below the current layer.

See [Create Interconnect Menu and Shortcut Keys](#).

2. Click to place the via.
3. Continue drawing the interconnect or double-click to end the interconnect at the bridge via.

If you continue the interconnect on the other end of the bridge via, the layer purpose pair changes appropriately.

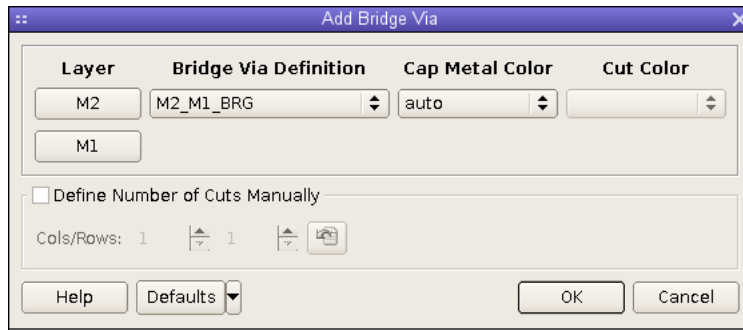
4. Press Esc to end the operation.

Adding Custom Bridge Vias

To insert custom bridge vias while creating interconnects:

1. While creating the interconnect, at the point where you want to add the custom bridge via, right-click and choose **Add Bridge Via**. See [Create Interconnect Menu and Shortcut Keys](#).

The **Add Bridge Via** dialog box opens.



2. In the **Layer** column, click the layer to which you want to connect.
3. (Optional) Set the remaining options in the dialog box. See [Add Bridge Via Dialog Box](#).
4. When you are finished, click **OK**.
5. On the canvas, click to place the custom bridge via. The active layer purpose pair changes to match the other routing layer of the via.
6. Continue creating interconnects, or double-click to end the interconnect at the custom bridge via.
7. When you are finished, press Esc to quit the operation.

Adding Vias Between Overlapping Interconnects

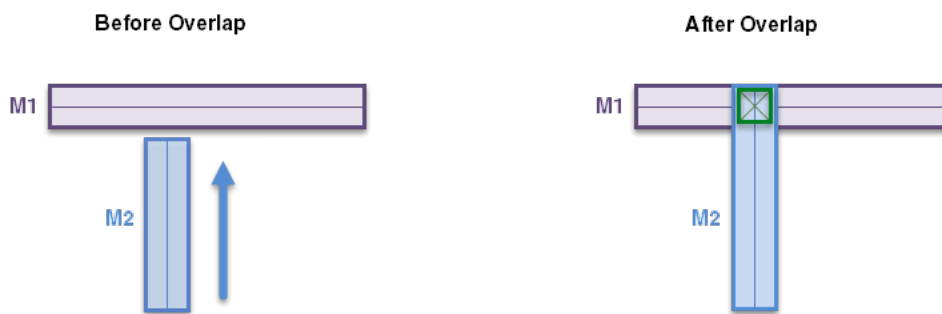
Vias are automatically added when you create an interconnect that overlaps with an existing interconnect. Stacked vias are added if the interconnects are not on adjacent layers. For example, if the existing interconnect is on m3 and the new interconnect is on m1, then via1 is added between layers m1 and m2, and via2 is added between m2 and m3.

The requirements for adding the vias automatically between overlapping interconnects are:

- The overlapping interconnects must be on the same net.

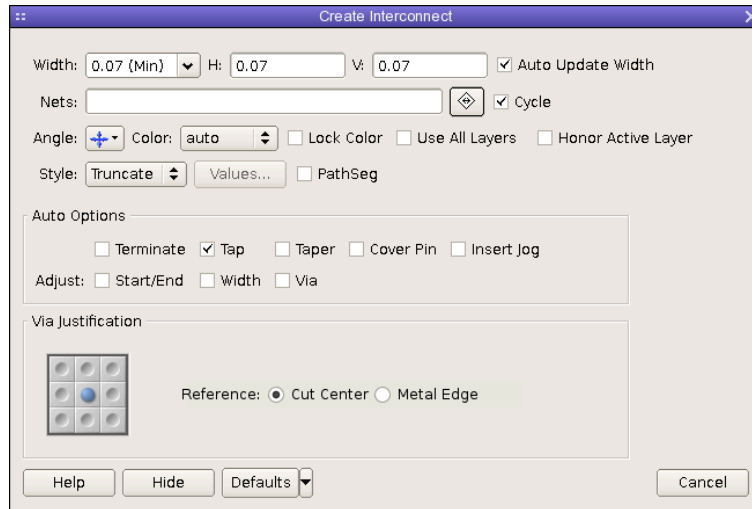
For example, suppose the existing interconnect is on netA. When you create the overlapping interconnect, make sure you type netA in the **Nets** box on the **Interconnect** toolbar. For more information, see [Interconnect Toolbar and Create Interconnect Dialog Box](#).

- You must set the `leUpdateConnectedViasAndEndStyles` Tcl preference to `true`. Doing so preserves the original via cut number. For more information, see the *Custom Compiler Tcl Reference Manual*.

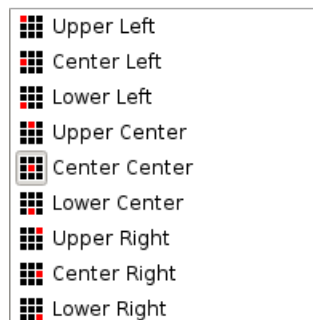


Positioning the Vias

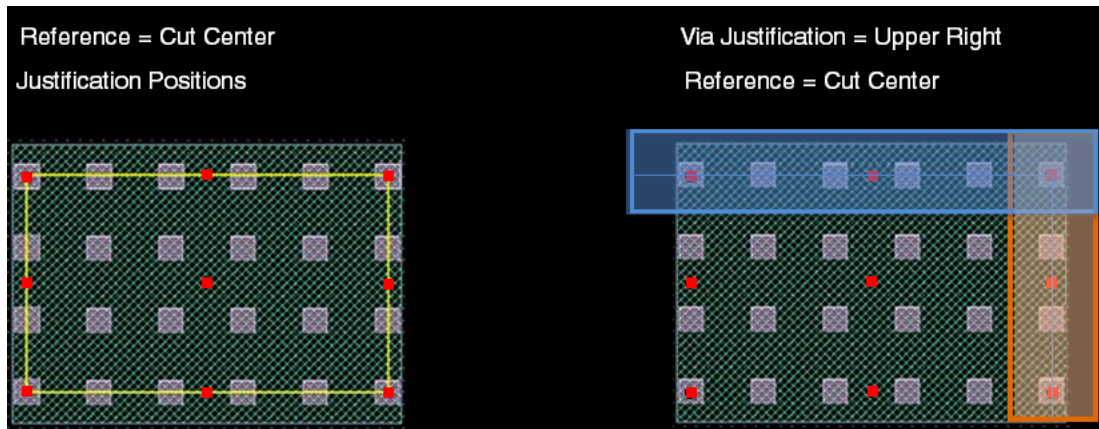
When you add vias, you can specify how the vias should be positioned using the **Via Justification** and **Reference** options in the **Create Interconnect** dialog box. See [Interconnect Toolbar and Create Interconnect Dialog Box](#), [Create Interconnect Menu and Shortcut Keys](#), [Bus Toolbar and Create Bus Dialog Box Reference](#) and [Create Bus Shortcut Menu and Shortcut Keys](#).



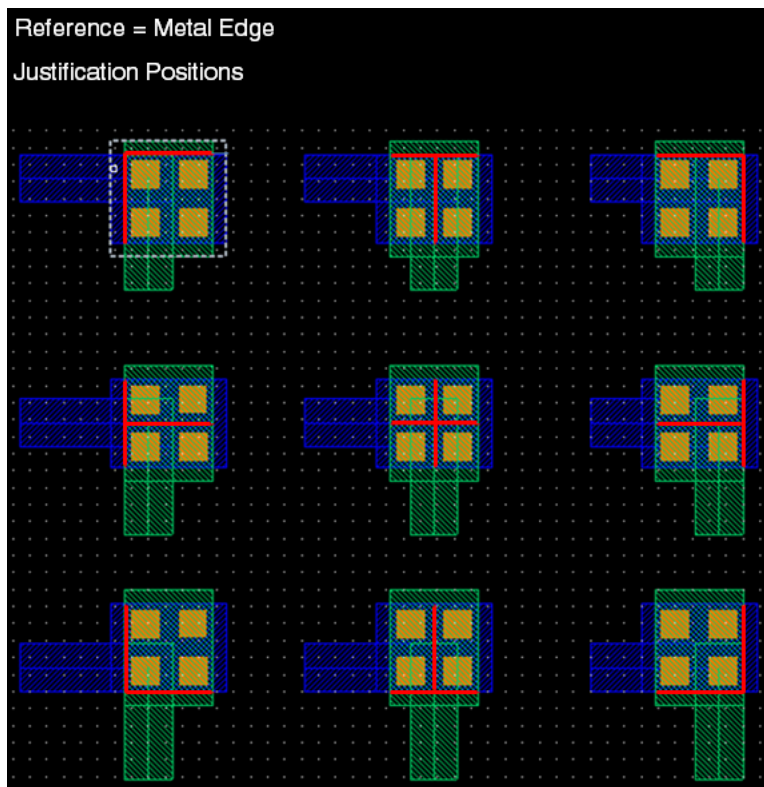
Via Justification and **Reference** work together to produce different results. **Via Justification** specifies the nine possible positions in a via array. See [Via Justification](#).



When **Reference** is **Cut Center**, the nine positions are determined by the box whose corners intersect with the centers of the four corner cuts. In the following example, the red dots mark the nine Via Justification positions in a 6 x 4 via array. (The yellow box and the red dots are only used to determine the nine position and are not actually displayed on the canvas.) The perpendicular interconnects will align with one of nine positions that you select. The following example shows the result when Via Justification is Upper Right.



When **Reference** is **Metal Edge**, the edges of perpendicular interconnects will align with the edges of the corresponding metal via layers, as shown. In the following example, the red lines are added to show the justification positions.



Add Via Dialog Box Reference

This section provides a quick reference of the **Add Via** dialog box. For instructions on how to add vias as you create interconnects, see [Adding Default Vias](#) and [Adding Custom Vias or a Via Stack](#).

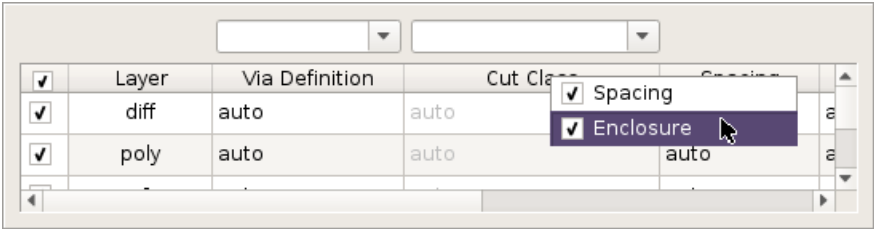
Layer	Via Definition	Cut Class	Spacing	Layer1 Enc	Layer2 Enc	Metal Color	Cut Color
POLY							
M0POLY	POLY	auto	auto	auto	auto	Not Colorable	Not Colorable
MODIFF1	M0POLY	auto	auto	auto	auto	Not Colorable	Not Colorable
MODIFF2	auto	auto	auto	auto	auto	Not Colorable	Not Colorable
M1	auto	auto	auto	auto	auto	auto	Not Colorable
M2	auto	auto	auto	auto	auto	auto	auto
M3	auto	auto	auto	auto	auto	auto	auto

Via Setting

Option dialog	Description
Via Setting	Applies the Via Definition and Cut Class settings you specify in this dialog box to: • Local: The Create Interconnect > Add Via command only. • As Auto Via: The Create Via (Auto mode) command, to ensure consistency between the two commands without having to re-enter the settings..

Layer Table

To display additional columns in the table, right-click the table header, and select the columns you want to add to the table: **Spacing, Enclosure**.



Option	Description
--------	-------------

Filter boxes
(above the Layer table)

Filters the list of vias and cut classes based on the criteria you provide. The filters you specify are remembered within the same design session.

By default, the predefined **auto** filter is used. However, you can change the criteria by typing one or more strings in the box. Use a character space to separate the strings. For example:

bar diff po

After you type the filter strings, press Enter. The list of names that meet the criteria appear in the column. An ellipse (...) in a result indicates that multiple definitions match the filter criteria. To view the definitions, rest the pointer on the result.

	bar	auto
Layer	Via Definition	Cut Class
M4	M4_M5xHBAR	auto
M5	M5_M6xHBAR ...	auto
M6	M6_M7xHBA	M5_M6xHBAR M5_M6xVBAR
M7	M7_M8xHBAR ...	auto

The filter results depends on the **Filter** setting you selected in the **General Options** dialog box.

Layer

(Dialog box only) Displays only the bottom layer. You can specify whether to enabled each bottom layer. By default, all the bottom layers are enabled for via creation.

If **Stacked** mode is enabled, the table only shows the layers between bottom and top layers.

Via Definition

(Dialog box only) Lists via, via variant, and bridge via definitions. A via variant name contains the name of the via definition on which it is based.

The definitions are sorted in alphabetical order. The first name in the list is the default via.

This list might include oaViaDefArray objects. Selecting a predefined oaViaDefArray from the list is the only way to place an array of vias (a regular mosaic).

You can select the available definitions by clicking on the field. The list includes an **auto** definition when **Auto** mode is enabled.

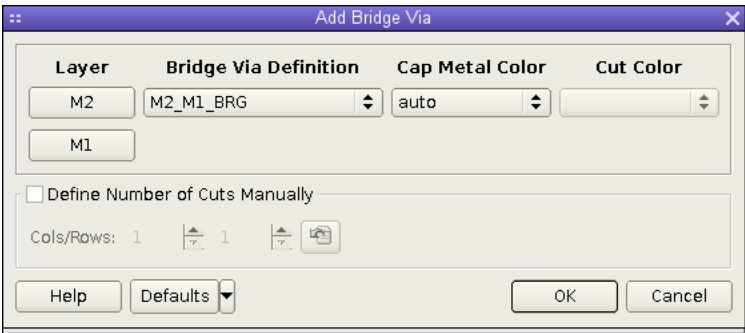
Option	Description
Cut Class	(Dialog box only) Lists cut shapes defined in your technology database. You can select the available cut classes by clicking on the field. The list includes an <i>auto</i> cut class when Auto mode is enabled. When Manual non-stacked mode is enabled, you can modify the parameters of the selected via in the Parameters table. When Manual stacked mode is enabled, you can modify the spacing and the enclosure values for Layer1 and Layers by clicking on the field or by entering the desired values.
Spacing	Spacing between cuts in standard and custom built-in vias. The spacing applies to neighboring cut shapes that have positive parallel run lengths. You can specify one value, which applies to both the X and Y spacing directions. Alternatively, you can specify two values, the first value applies to the X direction and the second value applies to the Y direction. The spacing values you provide must be larger than the design-rule values. If the spacing values you provide create violations, the Layout Editor will automatically correct them. For all standard vias, if the <code>leViaHonorViaDefDefaultSpacing</code> Tcl preferences is <code>true</code>, the Layout Editor will automatically correct spacing values that are smaller than the defined cut spacing value. If a via variant is placed, the Layout Editor will ensure the placed via variant complies with the cut spacing value you specify and the via variant cut spacing value.
Layer <i>n</i> Enc	Via enclosure specifications. You can specify one, two, or four values: • One value specifies that the same distance is applied on all four sides of the enclosure. • When you specify two values, the first value is applied to the left and right sides of the enclosure, the second value applies to the top and bottom sides of the enclosure. • When you specify four values, the values are applied to the left, right, top, and bottom sides in the order encountered.

Define Via Size Manually

Cols/Rows	Uses the number of cut columns and rows that you specify to determine the via size (width and length).
Width/Height	Uses the width and length values that you specify to calculate the number of cuts that can fit within the area.

Add Bridge Via Dialog Box

This section provides a quick reference of the **Add Bridge Via** dialog box. For instructions on how to add bridge vias as you create interconnects, see [Adding Default Bridge Vias](#) and [Adding Custom Bridge Vias](#).



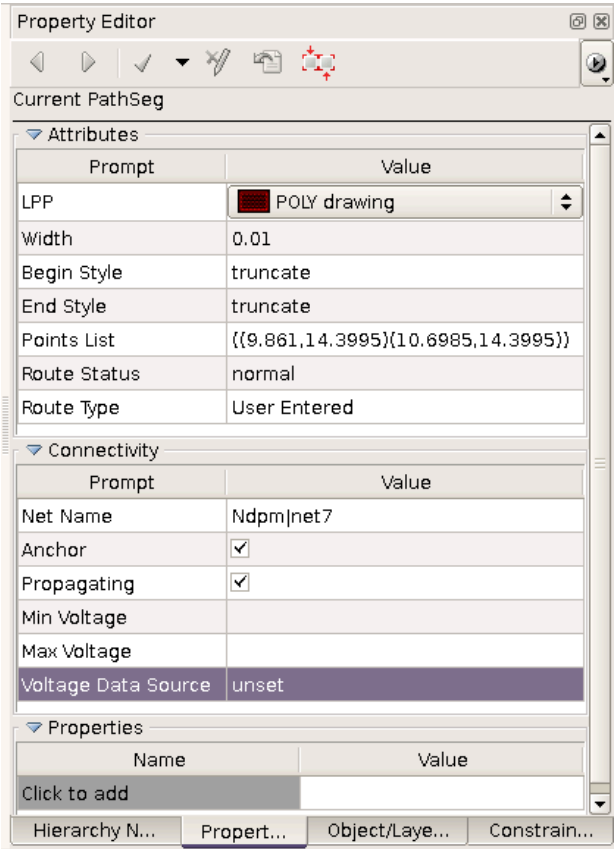
Option dialog	Description
Layer	Layer to which you want to connect. For example, if you just created an interconnect on m1 and you want to connect to layer m3, you would click m3.
Bridge Via Definition	Bridge via definition for the layer you selected. You can choose an available bridge via definition from the list. The list contains bridge vias defined in the technology database. The definitions are sorted in alphabetical order. The first name in the list is the default bridge via.
Cap Metal Color	Color to assign to the colorable bridge shape layer. If the layer is not colorable, the Cap Metal Color list is unavailable.
Cut Color	Color to assign to the cut layer.
Define Number of Cuts Manually	Dimensions (rows and columns) of the via array. If you do not supply the information, the Layout Editor calculates the values from the design rules in the technology database.

Editing Paths and Path Segments Using the Property Editor

You can edit a path or path segment to change its width, style, color, and extensions. For example, you might want to change the width of an end segment to match the width of the connecting pin.

To edit a path or path segment:

1. In the Layout Editor, display the Property Editor.



2. On the canvas, select the path segment you want to edit. The information about the selected path segment appear in the Property Editor.
3. Change one or more of the **Attributes**:

Attribute	Description
LPP	The layer-purpose pair of the selected path segment.

Attribute	Description
Width	The width of the path segment. Use the <code>lePropertyEditorPathWidthMethod</code> Tcl preference to specify whether the width is rounded to the nearest database unit (DBU) or two times the minimum manufacturing grid. To automatically add or remove vias when you change the path width, you must set the <code>leUpdateConnectedViasAndEndStyles</code> Tcl preference to <code>true</code>. For more information, see the <i>Custom Compiler Tcl Reference Manual</i>. Depending on the Vias and End Style selections in the Layout Design Options dialog box Command tab, the vias and path end styles can be updated automatically or independently. See Layout Design Options - Command Tab.
Style	(Paths only) The end styles of the selected path segment. For details, see Choosing End Styles.
Begin/End Style	(Path segments only) The style at the beginning and end of the path segment: ◦ truncate ◦ extend ◦ chamfer ◦ custom ◦ variable For the descriptions of the styles, see Choosing End Styles.
Begin Extension	The length to extend the beginning of the path.
Begin Left Diagonal Extension	(Custom end styles only) The diagonal on the left side of the path's starting point.
Begin Right Diagonal Extension	(Custom end styles only) The diagonal on the right side of the path's starting point.
Begin Right Half Width	(Custom end styles only) The right-half width of the path's starting point.
End Extension	The length to extend the end of the path.
End Left Diagonal Extension	(Custom end styles only) The diagonal on the left side of the path's ending point.
End Right Diagonal Extension	(Custom end styles only) The diagonal on the right side of the path's ending point.
End Right Half Width	(Custom end styles only) The right-half width of the path's ending point.
Points List	The coordinates of the vertices of the selected path segment.

Attribute	Description
Route Status	Indicator of whether the route can be modified: ◦ normal: The route or via can be modified. ◦ fixed: The route or via can be modified manually using the Edit commands or by direct manipulation on the canvas. The route or via cannot be modified by an application such as the router. ◦ locked: The route cannot be modified.
Route Type	How the route or via will be treated by the Fusion Compiler or IC Compiler II router: ◦ User Entered ◦ Detail Routing ◦ Global Routing ◦ Partial Parallel Route ◦ P/G Ring ◦ P/G Strap ◦ P/G Macro to IO Pin ◦ P/G Std Cell to IO Pin ◦ Clock Ring ◦ Clock Strap ◦ Clock Zero Skew Route ◦ Bus ◦ Fixed Shield ◦ Dynamic Shield ◦ Fill Track ◦ User Trunk For information about these route types, see the IC Compiler user documentation. Custom Route indicates the route shape is created by the Auto Router, the Interactive Router, or the Pattern Router process. This attribute value is used by the Delete Custom Route Shape command. See Deleting Routes.
Color	The color assignment of the path segment. This attribute is available only if the object is on a colored layer.
Color Lock	The indicator that the color assignment cannot be changed. This attribute is available only if the color assignment can be edited.

4. View or change the following **Connectivity** properties:

Connectivity Property	Description
Net Name	The name of the net that is associated with the path segment.
Anchor	The indicator that the path segment is explicitly assigned to the specified net.

Connectivity Property	Description
Propagating	The indicator that the path segment propagates connectivity.
Signal Type	Type of signal.

For more information about path connectivity, see [About Connectivity](#).

5. When you are finished, click **Apply Changes**  at the top of the Property Editor. The Layout Editor saves the changes.

Cutting Corners of Paths

When using the **Edit > Other > Cut Corner** command on interconnects, paths might be converted to path segments so that they can remain on the track. You can optionally change the objects back to paths if having off-track paths is acceptable.

For information about the **Cut Corner** command, see Cutting Object Corners.

Reshaping Interconnects

The Layout Editor provides multiple commands for editing (or reshaping) existing interconnects.

This section describes the following:

- [Commands for Reshaping Paths or Path Segments](#)
- [Using the Reshape Interconnect Command](#)
- [Saving and Deleting Dangling Path Sections](#)
- [Inserting Vias When Reshaping Interconnects](#)
- [Examples of Reshaping Interconnects](#)
- [Reshape Interconnect Toolbar and Dialog Box](#)
- [Reshape Interconnect Shortcut Menu and Shortcut Keys](#)
- [Add Via Dialog Box for Reshape Interconnect](#)

Commands for Reshaping Paths or Path Segments

You can change the shape of paths and path segments using one of the following **Edit** commands.

Command	Function/Purpose
Split	Separates the interconnect into two separate paths (or two unconnected series of path segments). See Splitting Objects .
Chop	Separates the interconnect into two separate paths (or two unconnected series of path segments), copying the removed portion to the clipboard. See Chopping Objects .
Stretch	Reshapes the interconnect center lines, possibly creating or deleting path vertices and path segments. See Stretching Objects .
Reshape	Adds points or segments in the middle of a path or a series of connected path segments. See Reshaping Objects and Examples of Reshaping Interconnects .
Reshape Interconnect	Adds vias while adding points or segments to the interconnect, optionally with automatic loop removal (for information about automatic loop removal. See Using the Reshape Interconnect Command . The Reshape Interconnect command supports multiple layer objects when reshaping a path segment. It also changes the interconnect width.

Using the Reshape Interconnect Command

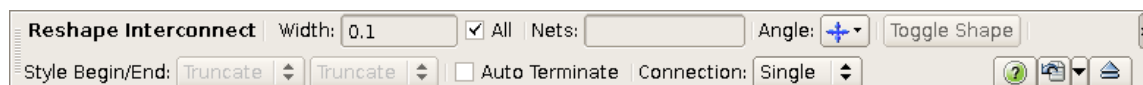
You use the **Reshape Interconnect** command to add points along a path (or add segments to a series of connected path segments). You can also add vias during the operation.

To use the **Reshape Interconnect** command:

1. Choose **Edit > Reshape > Interconnect**. The **Reshape Interconnect** toolbar opens.

Note:

The **Width** box and the **Defaults** button remain disabled until you select the route to reshape.



2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Reshape Interconnect** dialog box. The dialog box contains additional options.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

3. On the layout canvas, select a path or path segment.

Click on the centerline of the interconnect shape if no path or path segment is preselected.

In Infix mode, if no objects are preselected, the tool selects the object under the cursor. The cursor position is also the first point on the new route.

4. Click to enter new points on the path.

This creates a new path or pathSeg object based on the selected object type (path or path segment).

Right-click to toggle between the X-First and Y-First modes.

5. To add a via, right-click on at the location where you want to add the via, and choose **Add Via**. See [Add Via Dialog Box for Reshape Interconnect](#) to set the via options.

To ensure the paths fully cover the via metal layers, the `leInterconnectExtendPathEndToViaEnd` Tcl preference to set to true by default. See [Before You Start](#).

6. Complete the route in one of the following ways:

- Double-click to enter the final point.
- Click once on the centerline of the path or path segment you are reshaping.
- Click once on an object on the same net, if the layer purpose pairs of the objects you are joining match and you selected **Auto Terminate**.

If the tool prompts you to select one of two portions of the path or path segment to preserve, press Enter to select the highlighted portion or Shift-middle-click to select the other portion and then press Enter. See [Saving and Deleting Dangling Path Sections](#).

The newly created object (path or pathSeg) replaces the selected object after committing the operation.

The **Reshape Interconnect** command creates path vertices when reshaping a path and path segments when reshaping a series of connected segments, regardless of the current value of the `leUsePathSeg` option.

Note:

If you change the layer purpose pair while the command is active (with the Object/Layer Panel or the **Tap** command), the change affects the entire active path (or the entire series of connected path segments).

Instead of using the **Reshape Interconnect** toolbar, you can use the `ile::reshapeInterconnect` Tcl command. For more information, see “Layout Editing Tcl” in the *Custom Compiler Tcl Reference Manual*.

Saving and Deleting Dangling Path Sections

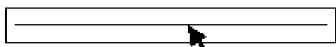
When reshaping interconnects, dangling path sections are typically deleted. This section explains how to keep dangling path sections.

When you reshape a path, the **Reshape Interconnect** command must delete one of two possible dangling sections unless you do one of the following:

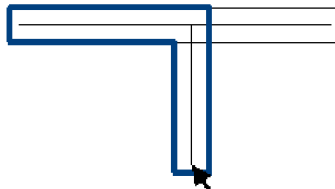
- Begin reshaping the path at one of the endpoints.
- End the command by clicking on the path you reshaped.

The command highlights one of the two possible sections after you enter the final point (B). To keep that section, press Enter. To keep the other section (C), press Shift+middle-click, and then press Enter.

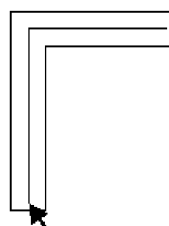
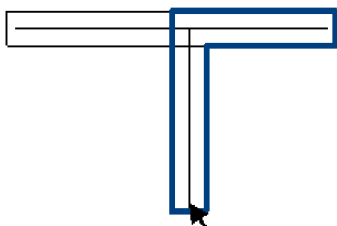
A. Starting reshape.



B. Reshaping.



C. Swapping the portion that will be eliminated. D. Committing.

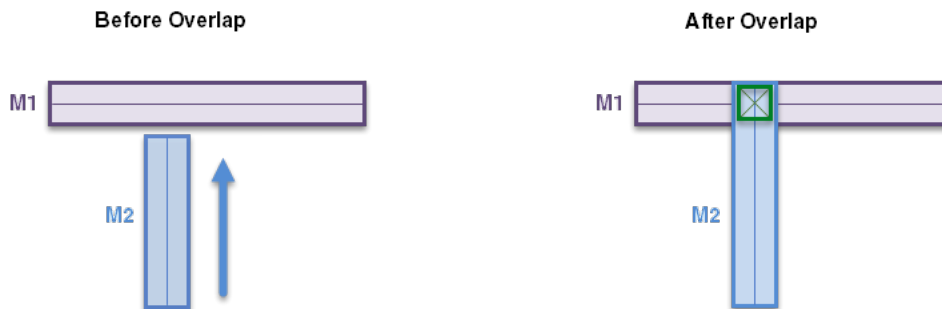


Inserting Vias When Reshaping Interconnects

When you reshape an interconnect such that it overlaps with another interconnect on another layer, a via is automatically added to connect the two paths.

The requirements for adding the vias automatically between overlapping interconnects are:

- The overlapping interconnects must be on the same net.
- You must set the `leUpdateConnectedViasAndEndStyles` Tcl preference to `true`.

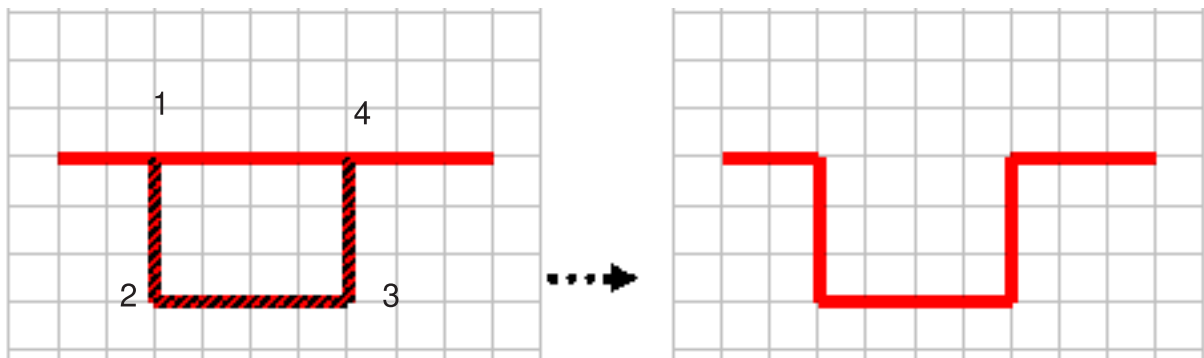


The via is automatically removed if the interconnects no longer overlap during a committed reshape operation.

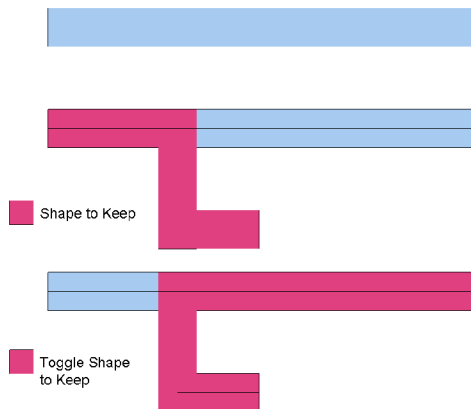
Examples of Reshaping Interconnects

To help you better understand how to use the **Reshape Interconnect** command and the results that can be achieved, this topic provides several example scenarios.

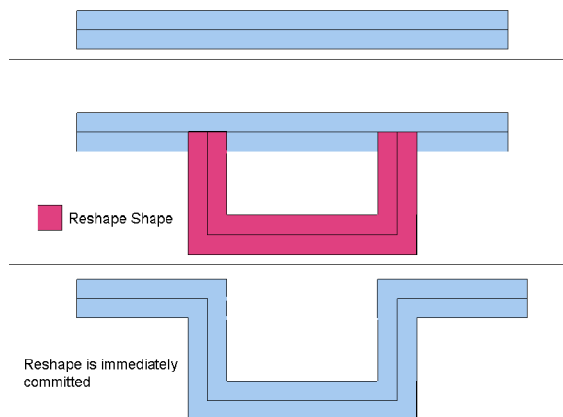
The following example shows the result of adding a jog to a path with the Reshape command.



The following example shows the result of drawing two segments (an “L” shape) while reshaping an interconnect. There are two possible resulting shapes.



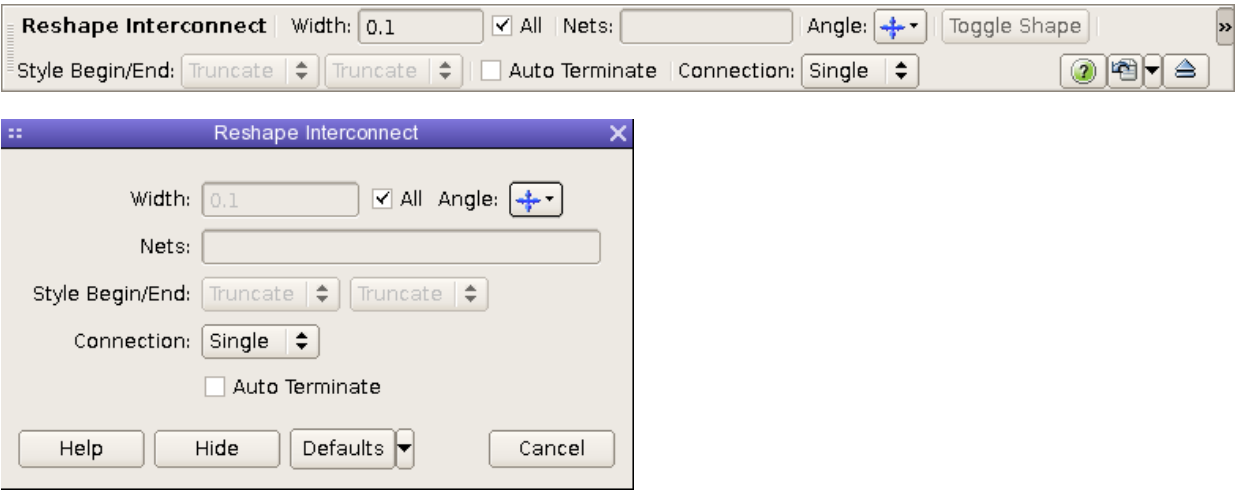
The following example shows a case when only one possible object results. During the reshaping operation, the newly added polyline shape touches the interconnect in two places, defining a jog. The tool shows the only possible resulting shape and waits for you to click **Apply**.



Reshape Interconnect Toolbar and Dialog Box

When using the **Reshape Interconnect** command, you can specify the options on the toolbar or in the equivalent dialog box.

This section provides a quick reference of the **Reshape Interconnect** toolbar and dialog box. For instructions on how to reshape interconnects, see [Using the Reshape Interconnect Command](#).



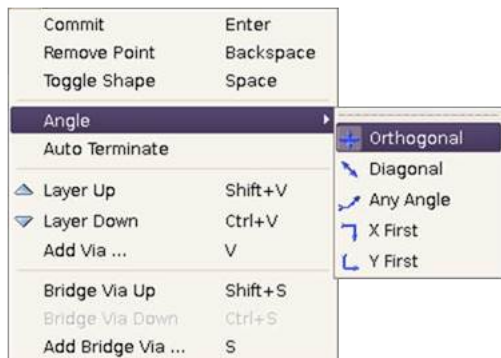
Option	Description
Width	Sets the width of the new segments. The default is the width of the route you select, but you can use the toolbar to change the value after you make your selection. If you change the width before inserting a new via along the path, the width change affects the entire path. A. Start reshape. B. Change width.
All	When enabled (default), applies the width to all paths or path segments. When disabled, only applies the width to the path or path segment under the cursor. Tcl preference: <code>leUpdateWidthForAll</code>
Nets	Displays the net name of the selected interconnect. You cannot edit the net name. The net name might change while the command is active if you make a connection to another net.

Option	Description
Angle	Controls the orientation of segments by Orthogonal, Diagonal, Any Angle, X First, or Y First. The default is Orthogonal. - If you are creating path segments: - The Diagonal option is automatically substituted for the Any Angle option. - With X First or Y First selected, adding a single point may create two path segments. See Angle Mode for Edges and Centerlines.
Begin/End Styles	Displays the style to use for the first and last points. You cannot change the endpoint style with Reshape Interconnect; use the Property Editor instead. The router chooses the end styles of intermediate path segments automatically, and adjusts the end styles of path segments that you extend if necessary. See Choosing End Styles.
Connection	Specifies the target wire objects (paths or path segments) to delete. None: Only removes the selected object. Single (default): Removes the selected object and the objects connected to the same layer. Branch: Removes the whole wire containing the selected object until the branch point, but preserves the via object at the branch point. Tcl preference: <code>leReshapeInterconnectMode</code>
Auto Terminate	Instructs the router to commit the route if you complete a connection by adding a point that overlaps an electrically equivalent shape or via.
Defaults	Resets all of the options to your Custom Compiler environment default values.
Eject	Opens a Reshape Interconnect dialog box which lets you define the options. The Hide button at the bottom of the dialog box closes the dialog box and displays the options in the toolbar. To switch between the toolbar and the dialog box while the command is active, use the Eject button and the Hide button in the dialog box or press the F3 shortcut key.

Reshape Interconnect Shortcut Menu and Shortcut Keys

While the **Reshape Interconnect** command is active, you can right-click to display the Reshape Interconnect shortcut menu.

This section provides a quick reference of the shortcut menu and shortcut keys. For instructions on how to reshape interconnects, see [Using the Reshape Interconnect Command](#).



Option	Description	Default Shortcut Key
Commit	Completes the route if it is valid.	Enter
Remove Point	Deletes the previously inserted point	Backspace
Toggle Shape	Toggles between alternative shapes.	Shift+T
Angle	Controls the orientation of segments by Orthogonal , Diagonal , Any Angle , X First , or Y First . The default is Orthogonal . - If you are creating path segments: - The Diagonal option is automatically substituted for the Any Angle option. - With X First or Y First selected, adding a single point may create two path segments. See Angle Mode for Edges and Centerlines.	Right-click to toggle the X First or Y First angle mode only if one of these modes is active.
Auto Terminate	Instructs the router to commit the route if you complete a connection by adding a point that overlaps an electrically equivalent shape or via.	
Layer Up	Inserts a single via at a layer above the current layer. For example, if you want insert a via two layers above the current layer, press Shift+V two times.	Shift+V
Layer Down	Inserts a single via at a layer below the current layer. For example, if you want to add a via three layers below the current layer, press Ctrl+V three times.	Ctrl+V
Add Via	Displays the Add Via dialog box. See Create Interconnect Menu and Shortcut Keys .	V
Bridge Via Up	Inserts a bridge via at a layer above the current layer.	Shift+S

Option	Description	Default Shortcut Key
Bridge Via Down	Inserts a bridge via at a layer below the current layer.	Ctrl+S
Add Bridge Via	Opens the Add Bridge Via dialog box. See Adding Custom Bridge Vias .	S

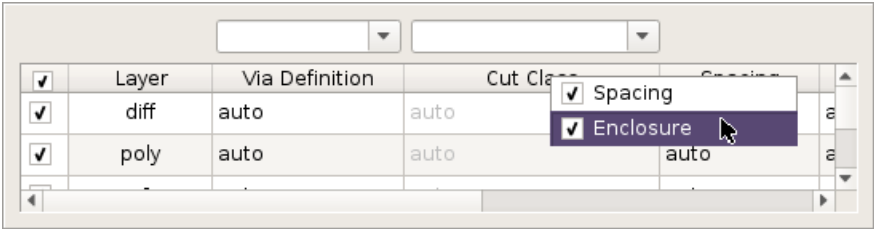
Add Via Dialog Box for Reshape Interconnect

When using the **Reshape Interconnect** command, you can right-click, specify the via definitions in the **Add Via** dialog box, and add vias.

This section provides a quick reference of the **Add Via** dialog box. For instructions on how to add vias while reshaping interconnects, see [Using the Reshape Interconnect Command](#).

Layer Table

To display additional columns in the table, right-click the table header, and select the columns you want to add to the table: **Spacing**, **Enclosure**.



Option	Description
--------	-------------

Filter boxes
(above the Layer table)

Filters the list of vias and cut classes based on the criteria you provide. The filters you specify are remembered within the same design session.

By default, the predefined **auto** filter is used. However, you can change the criteria by typing one or more strings in the box. Use a character space to separate the strings. For example:

bar diff po

After you type the filter strings, press Enter. The list of names that meet the criteria appear in the column. An ellipse (...) in a result indicates that multiple definitions match the filter criteria. To view the definitions, rest the pointer on the result.

	bar	auto
Layer	Via Definition	Cut Class
M4	M4_M5xHBAR	auto
M5	M5_M6xHBAR ...	auto
M6	M6_M7xHBA	M5_M6xHBAR M5_M6xVBAR
M7	M7_M8xHBAR ...	auto

The filter results depends on the **Filter** setting you selected in the **General Options** dialog box.

Layer
(Dialog box only) Displays only the bottom layer. You can specify whether to enabled each bottom layer. By default, all the bottom layers are enabled for via creation.

If **Stacked** mode is enabled, the table only shows the layers between bottom and top layers.

Via Definition
(Dialog box only) Lists via, via variant, and bridge via definitions. A via variant name contains the name of the via definition on which it is based.
The definitions are sorted in alphabetical order. The first name in the list is the default via.
This list might include oaViaDefArray objects. Selecting a predefined oaViaDefArray from the list is the only way to place an array of vias (a regular mosaic).
You can select the available definitions by clicking on the field. The list includes an **auto** definition when **Auto** mode is enabled.

Option	Description
Cut Class	(Dialog box only) Lists cut shapes defined in your technology database. You can select the available cut classes by clicking on the field. The list includes an <i>auto</i> cut class when Auto mode is enabled. When Manual non-stacked mode is enabled, you can modify the parameters of the selected via in the Parameters table. When Manual stacked mode is enabled, you can modify the spacing and the enclosure values for Layer1 and Layers by clicking on the field or by entering the desired values.
Spacing	Spacing between cuts in standard and custom built-in vias. The spacing applies to neighboring cut shapes that have positive parallel run lengths. You can specify one value, which applies to both the X and Y spacing directions. Alternatively, you can specify two values, the first value applies to the X direction and the second value applies to the Y direction. The spacing values you provide must be larger than the design-rule values. If the spacing values you provide create violations, the Layout Editor will automatically correct them. For all standard vias, if the <code>leViaHonorViaDefDefaultSpacing</code> Tcl preferences is <code>true</code>, the Layout Editor will automatically correct spacing values that are smaller than the defined cut spacing value. If a via variant is placed, the Layout Editor will ensure the placed via variant complies with the cut spacing value you specify and the via variant cut spacing value.
Layer <i>n</i> Enc	Via enclosure specifications. You can specify one, two, or four values: • One value specifies that the same distance is applied on all four sides of the enclosure. • When you specify two values, the first value is applied to the left and right sides of the enclosure, the second value applies to the top and bottom sides of the enclosure. • When you specify four values, the values are applied to the left, right, top, and bottom sides in the order encountered.

Define Via Size Manually

Cols/Rows	Uses the number of cut columns and rows that you specify to determine the via size (width and length).
Width/Height	Uses the width and length values that you specify to calculate the number of cuts that can fit within the area.

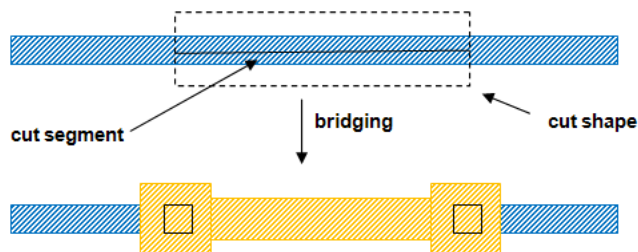
Bridging Interconnects

This section describes the following:

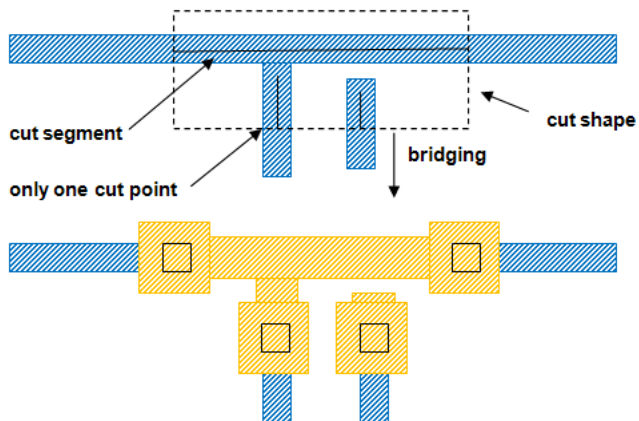
- [About Bridging Interconnects](#)
- [Bridging Leading Interconnects in Bus Configurations](#)
- [Using the Bridge Interconnect Command](#)
- [Bridge Toolbar](#)
- [Bridge Interconnect Menu and Shortcut Keys](#)

About Bridging Interconnects

A bridging operation lets you cut one or more existing interconnect shapes into two specified points and re-route the cut on a layer immediately above or below the original interconnect layer. The operation results in a new interconnect shape having the same type as the original and having the necessary vias to make the transition between the two layers. The following diagram shows the general case of two cut points.



There is a special case in which the cut shape will be allowed to cut the target interconnect in only one point, and that is when one of the end points of the interconnect is on the inside of the cut shape. The following diagram shows the special case of one cut point.



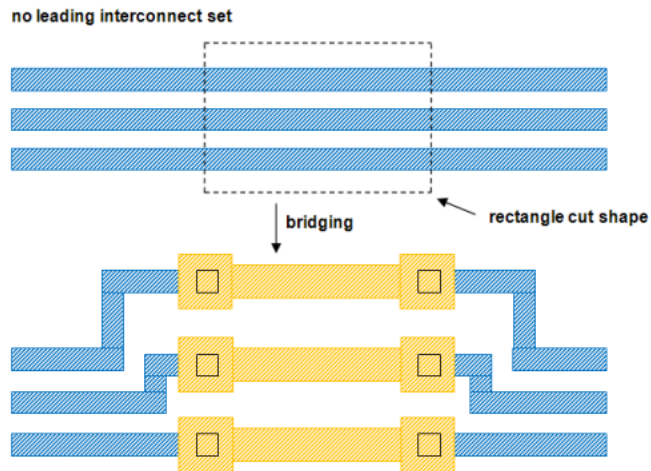
The following table lists the bridging interconnect terms and definitions.

Term	Description
Cut shape	Defines the cut point(s) on the target interconnect.
Cut segment	Segment defined by the two cut points on the same interconnect.
Target interconnect	Interconnect intersected by the cut shape.
Bridging interconnect	The interconnect that changes layers as a result of a bridging operation.
Bridging layer	The resulting layer.

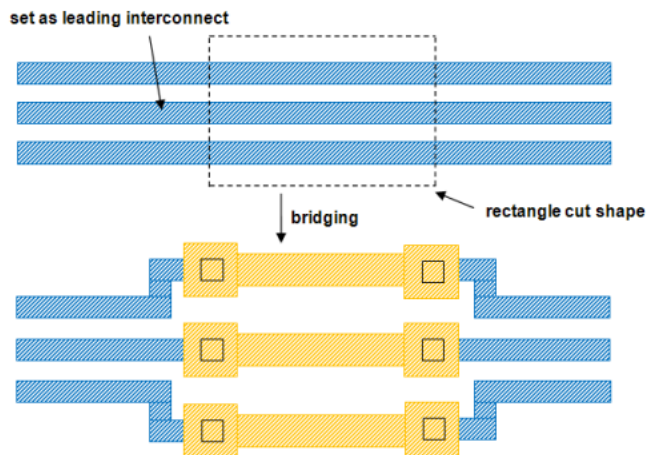
Bridging Leading Interconnects in Bus Configurations

For via staggering purposes, you can use a toggle option on the menu to set or unset the leading interconnect in a bus-like configuration.

The following diagram shows how vias are stacked when the leading interconnect has not been set.



The following diagram shows how vias are stacked when the leading interconnect has been set.



Using the Bridge Interconnect Command

To bridge interconnect objects:

1. Choose **Edit > Wire > Bridge Interconnect**.

The [Bridge Toolbar](#) opens.

2. Set the toolbar options you want to apply.

The command remains active.

3. Select the interconnect objects you want to bridge.

4. Specify the first cut point.

5. Specify the second cut point.

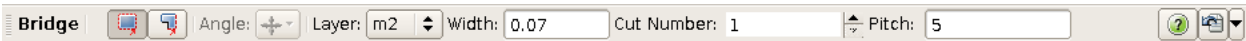
The bridging segment and the vias appear.

The command remains active.

6. Press Esc to end the command.

To use the `ile::bridgeInterconnect Tcl` command, see “Layout Editing Tcl” in the *Custom Compiler Tcl Reference Manual*.

Bridge Toolbar

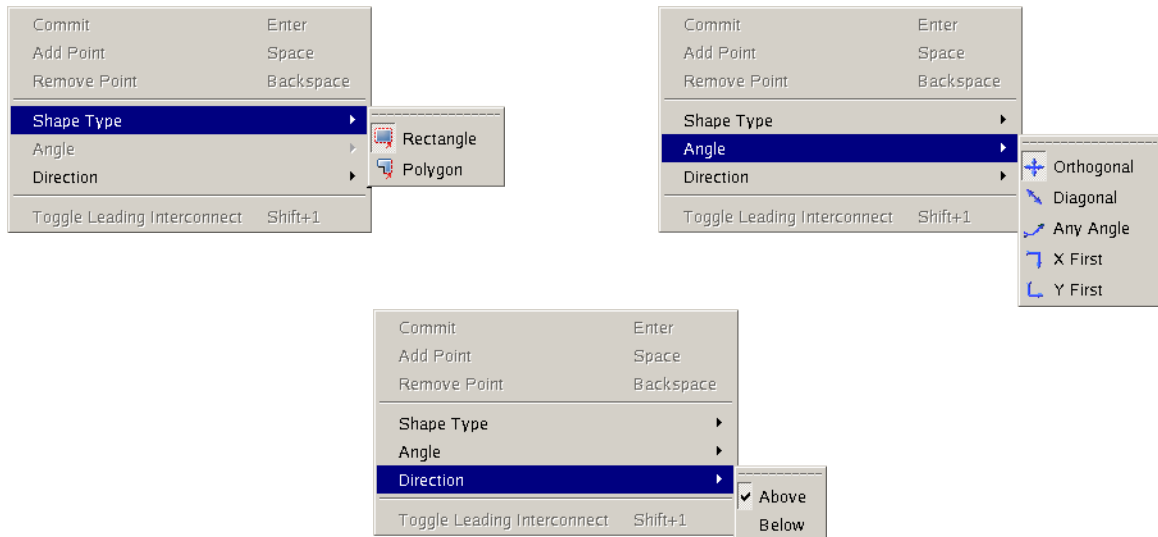


Option	Description
 	Switches between the following shapes: • Rectangle • Polygon
Angle	Specifies the snapping angle. The available angles are Orthogonal , Diagonal , Any Angle , X First , and Y First . The default is Orthogonal .
Layer	The layer to use for bridging the interconnects.
Width	The width of the current path or path segment. If you specify a value that is less than <code>minWidth</code> , the Layout Editor will use the <code>minWidth</code> value.
Cut Number	The number of via cuts to add.

Option	Description
Pitch	The center-to-center distance between vias on adjacent wires. When Pitch is 0, the Layout Editor uses the <code>minSpacing</code> value.
Defaults	Resets all of the options to your Custom Compiler environment default values.

Bridge Interconnect Menu and Shortcut Keys

When the **Bridge Interconnect** command is active, right-click to show the Bridge Interconnect menu, as shown.



Option	Description	Default Shortcut Key
Commit	Completes the shape.	Enter
Add Point	Adds a point, when prompted, for either a select or commit action.	Space
Remove Point	Deletes the previously inserted point	Backspace
Shape Type	Toggles between alternative shapes.	

Option	Description	Default Shortcut Key
Angle	Specifies which angle snapping mode to apply when defining the cut shape. The available angle snapping modes include Orthogonal , Diagonal , Any Angle , X First , and Y First . The default is Orthogonal .	Right-click to toggle the X First or Y First angle mode only if one of these modes is active.
Direction	Specifies the bridge direction.	
Toggle Leading Interconnect	Controls whether the interconnect under the mouse cursor is the leading interconnect.	Shift+1

Stacking Interconnects

This section describes how to use the **Edit > Wire > Stack Interconnect** command to add paths above and below selected paths. The topics are:

- [About Stacking Interconnects](#)
- [Before You Start](#)
- [Adding Stacked Interconnects](#)

About Stacking Interconnects

To reduce wire resistance, you can use the **Edit > Stack Interconnect** command to add tandem paths above or below selected paths.

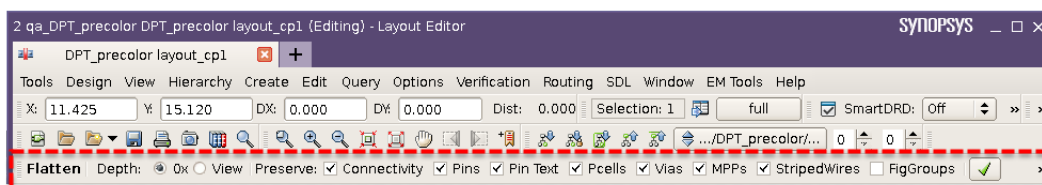
During the stacking operation:

- The stacked paths are added to specified layers on the same net.
- The stacked paths have the same width as the selected path. In addition, the stacked paths have the truncate end style.
- To prevent short errors, the stacked paths are only added to regions with available space.
- Via stacks are added automatically. If the stacked paths are not adjacent to the selected path, via stacks are added to the two ends of the selected path to ensure connection to the stacked paths.

Before You Start

If the selected path is wider than the value specified by the `maxWidth` constraint, you must:

1. Use the **Edit > Wire > Stripe Wire** command to split the existing wire (path) into parallel paths of narrower but identical width. For information about striping wires, see [Striping Wires](#).
2. Flatten the configuration groups created during step 1. To do this:
 - a. In the Layout Editor, choose **Edit > Hierarchy > Flatten** (or type `ile::flatten` in the Console). The **Flatten** toolbar appears.

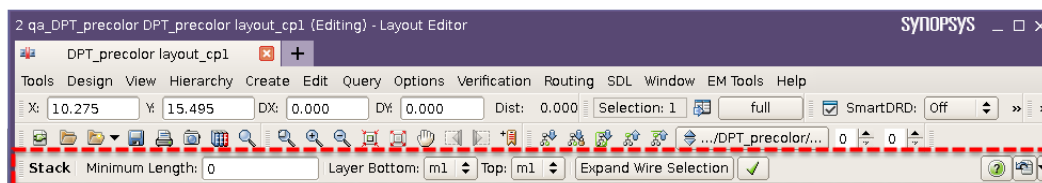


- b. On the **Flatten** toolbar, clear the **FigGroups** check box.
 - c. Click Apply .
3. Add stacked paths to each stripe of the striped wire.

Adding Stacked Interconnects

To add stacked interconnects:

1. Select the interconnect to which you want to add tandem paths and vias.
2. Choose **Edit > Wire > Stack Interconnect** (or press Ctrl+8). The **Stack** toolbar appears.



3. On the **Stack** toolbar, set the options:

Option	Description
Minimum Length	Specifies the minimum length of the stacked paths. If Minimum Length is 0 (default), then the minimum length of the stacked paths must be greater than the width of the selected path.
Layer — ◦ Bottom ◦ Top	Specifies the range of layers to use when adding the stacked paths: ◦ Bottom: The bottom-most layer in the range of layers. ◦ Top: The top-most layer in the range of layers.
Expand Wire Selection	Adds all the connected path segments to the selection set.

4. When you are finished, click  (**Apply**).

Striping Wires

This topic describes the following:

- [About Striped Wires](#)
- [Striping Wires Manually](#)
- [Editing Striped Wire Attributes](#)
- [Stacking Specified Striped Wires](#)
- [Converting Striped Wires Back to Fat Wires](#)
- [Stripe Toolbar Reference](#)

About Striped Wires

One way to address wire width and spacing requirements is to stripe a fat wire. This topic explains what striped wires are, when and why you might want to stripe wires, and other details to consider.

The information covered are as follows:

- [What is Striping](#)
- [Why is Striping Needed](#)
- [Wire Color, Width, and Spacing](#)

- [Limitations](#)
- [Examples Of Wire Striping](#)

What is Striping

Wire stripes are sets of orthogonal path segments whose combined widths and spacings satisfy the minimum spacing and maximum width constraints.

In addition to creating wire stripes, you can control whether the tool adds straps (path segments) across the endpoints of the wire stripes to ensure connectivity and to facilitate interfacing with pins of different sizes.

When path segments are created in different layers, the tool automatically replaces any existing fat wire vias or stacked vias with smaller vias and stacks.

Why is Striping Needed

Wire striping lets you convert fat wires to striped wires when your design constraints conflict with the physical manufacturing constraints; for example, when the wire width required for a particular current exceeds a layer's maximum width constraint.

You can also apply wire striping to avoid violating the width dependent separation rule. Striping a wide wire into smaller stripes ensures that the width-dependent separation rule is not violated.

Wire Color, Width, and Spacing

To ensure that wires are placed correctly in the design, you can snap the striped wires to tracks. When you snap striped wires to tracks:

- Each wire snaps independently to a different track.
- Each wire inherits the color of the track in compliance with the Color Propagation rules. If the tracks have the same color, the striped wires will have the same color. If the tracks have different colors, the striped wires will have different colors.
- Each wire's width depends on the striping policy that you select. The policy can result in different wire widths. For more information about the striping policies, see [Stripe Toolbar Reference](#).
- The spacing between the wires is determined by the minSpacing setting in the technology database. The spacing between the striped wires might vary.

Limitations

The following wire types are not supported:

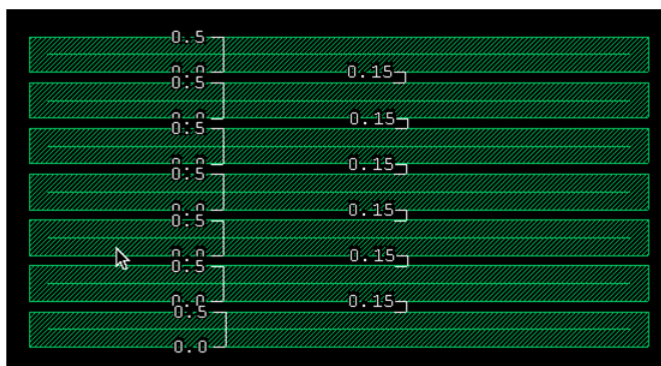
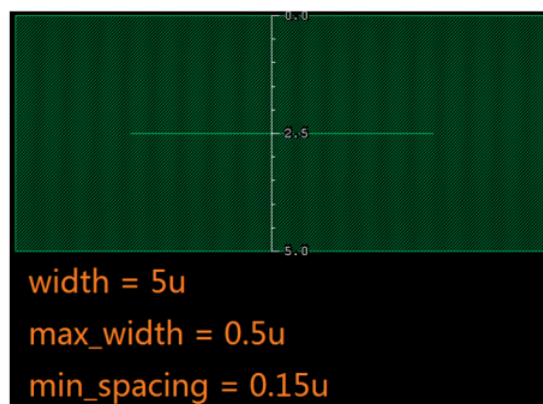
- 45° wire
- Any-angle wire
- Round end-style in path mode
- Fat wire with a custom via
- Fat wire with a via as the startpoint or endpoint

Examples Of Wire Striping

The following examples illustrate metal striping.

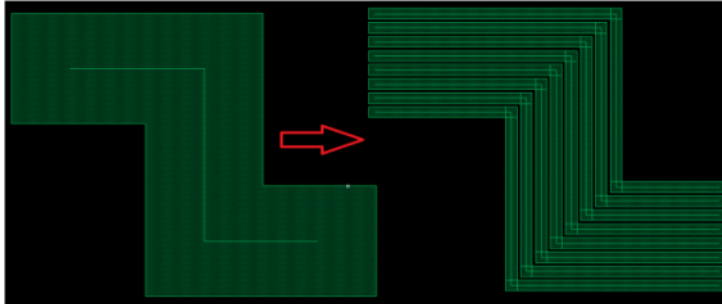
Wire Striping on a Simple Path

The following figure shows how striping works on a simple wide wire. Striping is created based on the `maxWidth` constraint in the technology file.



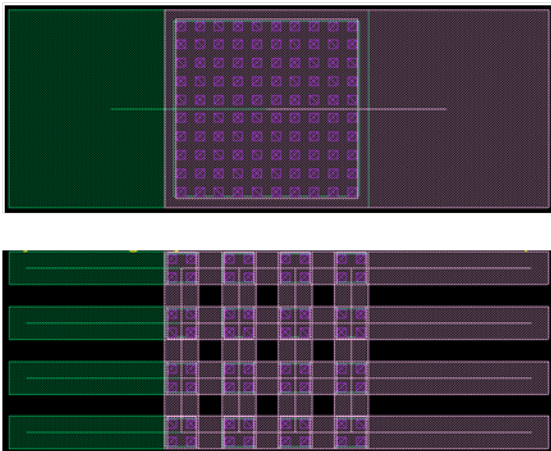
Wire Striping on an Orthogonal Path

You can stripe a wide path into smaller segments.



Wire Striping for Connected Fat Wires

You can stripe wide wires, add straps, and create vias for overlapping shapes.



Note:

When you stripe wires, the original geometry is deleted. Later, if you flatten a striped wire, reverting back to raw geometry, the result is the content of the striped wire, which can only be path segments and vias.

Striping Wires Manually

Wire striping lets you convert fat wires to striped wires when your design constraints conflict with the physical manufacturing constraints. You use the **Stripe Wire** command to convert fat wires to striped wires.

This topic describes the following:

- [Before You Start](#)
- [Procedure for Striping Wires](#)

Before You Start

Make sure the following constraints are defined in the technology database:

- `maxWidth`
- `minSpacing`

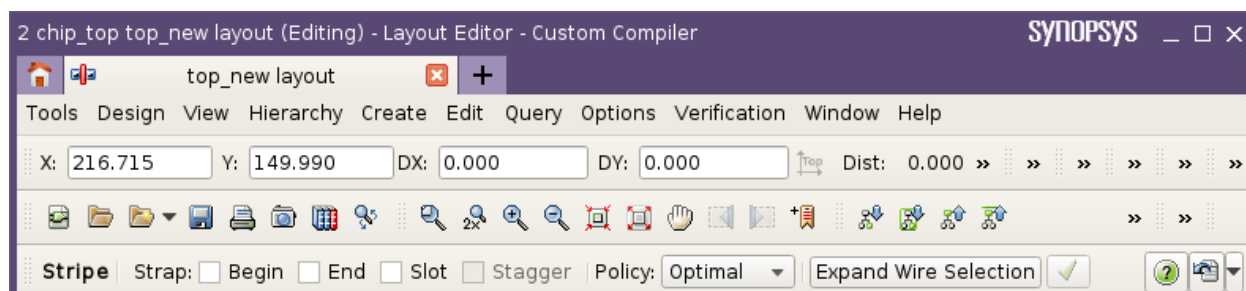
The Layout Editor uses the `maxWidth` and `minSpacing` constraints to determine the number of striped wires and the spacing between the wires. For information, see the *Custom Compiler Technology and Display Resource File Reference Manual*.

Procedure for Striping Wires

To manually stripe a fat wire:

1. In the Layout Editor, choose **Edit > Wire > Stripe Wire**.

The **Stripe** toolbar appears.



2. Specify the options you want to apply. See [Stripe Toolbar Reference](#).
3. Select one or more fat wires that violate the constraints in the technology file.

You can select one path segment of a set of connected path segments, and then click **Expand Wire Selection** on the toolbar to automatically select all the connected path segments.

4. Click **Apply**  to stripe the selected wire. The Layout Editor uses the constraints in the technology database to convert the wire.

Note:

By default, the Layout Editor creates the number of wire stripes and required spacings to match the footprint of the fat wire. To make changes after

creating the stripes, use the Property Editor. See [Editing Striped Wire Attributes](#).

5. When you are finished, press Esc to exit the wire striping mode.

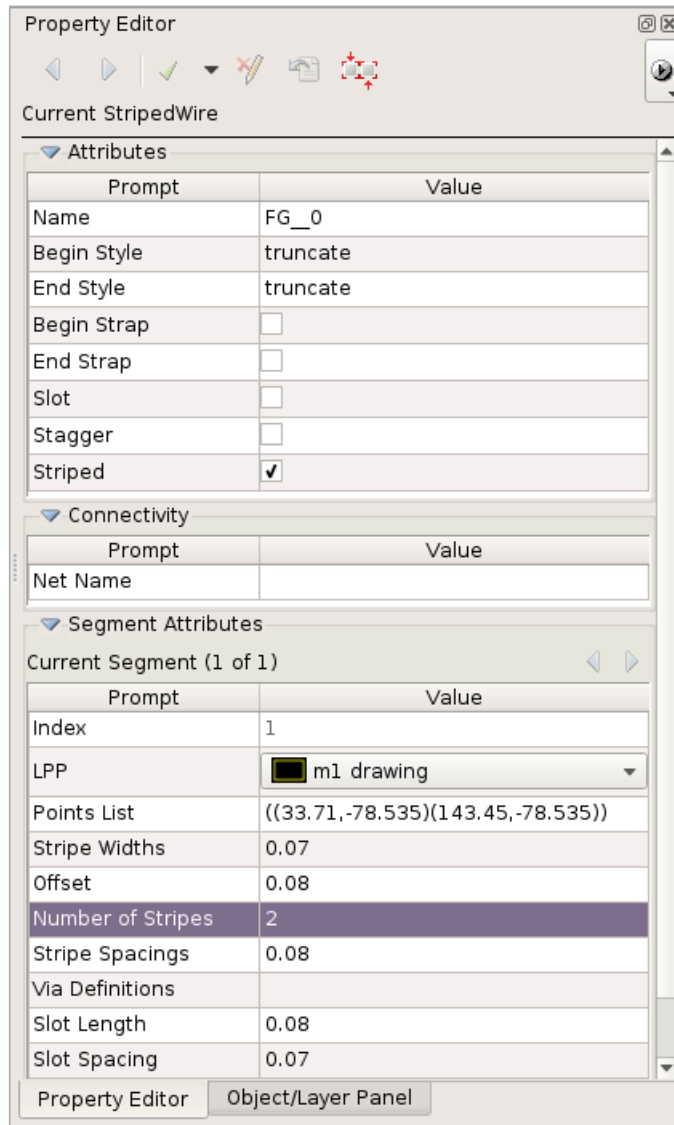
Editing Striped Wire Attributes

After you create striped wires, you can modify them by editing the attributes in the Property Editor.

To edit the striped wires attributes:

1. Open the Property Editor.
2. On the canvas, select the striped wires. The attributes of the selected striped wires display in the Property Editor.

3. Set the following attributes:



Property Editor

Current StripedWire

Attributes

Prompt	Value
Name	FG_0
Begin Style	truncate
End Style	truncate
Begin Strap	☐
End Strap	☐
Slot	☐
Stagger	☐
Striped	☒

Connectivity

Prompt	Value
Net Name	

Segment Attributes

Current Segment (1 of 1)

Prompt	Value
Index	1
LPP	m1 drawing
Points List	((33.71,-78.535)(143.45,-78.535))
Stripe Widths	0.07
Offset	0.08
Number of Stripes	2
Stripe Spacings	0.08
Via Definitions	
Slot Length	0.08
Slot Spacing	0.07

Property Editor Object/Layer Panel

Attribute	Description
Points List	The list of vertices for the selected wire segment.
Offset	The spacing before the first bridge.
Number of Stripes	The total number of stripes.
Stripe Spacings	The distance between the striped wires.

Attribute	Description
Via Definitions	The definitions of the vias and via variants used by the striped wires.
Slot Length	The length of the slots.
Slot Spacing	The width of the bridges between the striped wires.

Be aware that the adjusted values cannot violate the constraints in the technology database.

4. When you are finished, click Apply  .

Stacking Specified Striped Wires

You can stack one or more striped wires to meet design requirements. The procedure uses the **Stack Interconnect** command.

To stack individual striped wires:

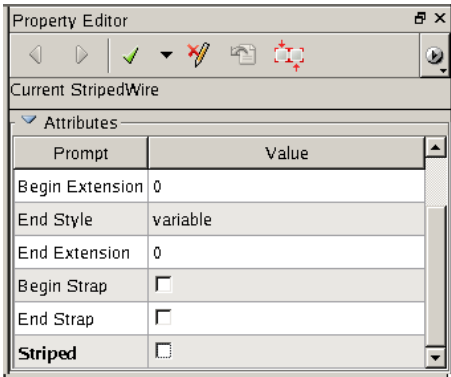
1. Start the **Stack Interconnect** command. (Type `ile::stackInterconnect` in the Console.)
2. On the layout canvas, select one or more striped wires you want to stack.
3. On the **Stack** toolbar, click **Apply** .

Converting Striped Wires Back to Fat Wires

You can convert a striped wire back to a fat wire by editing the striped wire attribute.

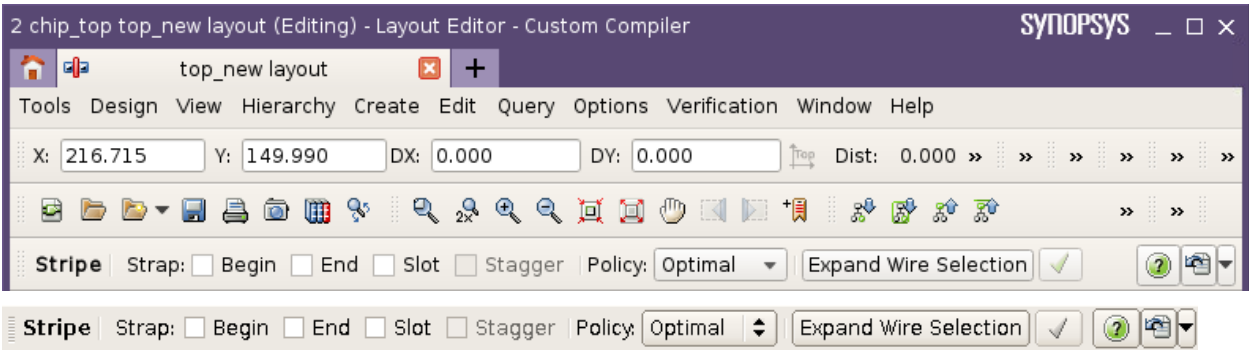
To convert a set of striped wires back to a fat wire:

1. Open the Property Editor.
2. On the canvas, select the striped wires that you want to convert back to a fat wire. The attributes of the striped wires display in the Property Editor.
3. Clear the **Striped** check box.

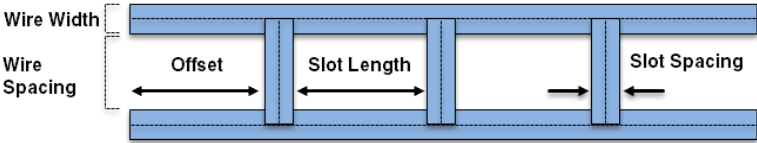
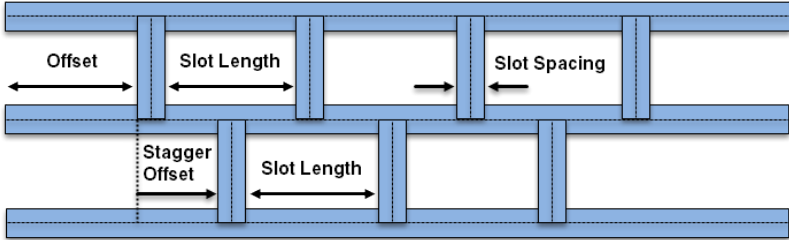


4. Click  (Apply).

Stripe Toolbar Reference



Option	Description
Strap	• Begin: Adds a strap at the beginning of the wire stripes. • End: Adds a strap at the end of the wire stripes.

Option	Description
Slot	Adds rectangular shapes perpendicularly to striped wires, forming bridges between the striped wires. Doing so prevents large power drops at terminals that are connected to the striped wires. The bridges are positioned between the centerlines of adjacent striped wires. The spaces between the shapes are called slots. An offset spacing is placed at the beginning of the striped wires. The width of the bridges is called slot spacing. By default, the slot length equals the stripe spacing, and the slot spacing equals the wire width. However, you can adjust the slot length and slot spacing values in the Property Editor. You can also adjust the offset spacing value. See Striping Wires Manually.  The diagram shows two horizontal blue striped wires. Vertical blue bars (bridges) connect them. Labels with arrows indicate: 'Wire Width' (the width of a wire), 'Wire Spacing' (the distance between wire centerlines), 'Offset' (the distance from the wire edge to the first bridge), 'Slot Length' (the length of a bridge), and 'Slot Spacing' (the distance between bridges).
Stagger	(Available only if you select Slot) Uses the stagger pattern when adding slots. The stagger offset is $\frac{1}{2} * (\text{slot length} + \text{slot spacing})$.  The diagram shows two horizontal blue striped wires with bridges. The top wire has bridges at regular intervals. The bottom wire's bridges are staggered. Labels indicate: 'Offset', 'Slot Length', 'Slot Spacing', and 'Stagger Offset' (the horizontal distance between the bridge of the top wire and the bridge of the bottom wire).
Policy	• Specifies the policy for striping wires: • Optimal: Uses the maxWidth constraint for stripe width. Smaller stripe width may be used if the original width is less than $2 * \text{maxWidth} + \text{min_spacing}$. Footprint can decrease but should not increase. reservednon-reserved • Footprint: Keeps the same footprint as the original path while maximizing the total metal width. reservednon-reserved • Width: Keeps the same overall width as the original path, increasing the footprint of the route. reservednon-reserved
Expand Wire Selection	Adds all the connected path segments to the selection set.
Apply Icon	Stripes the selected wire.

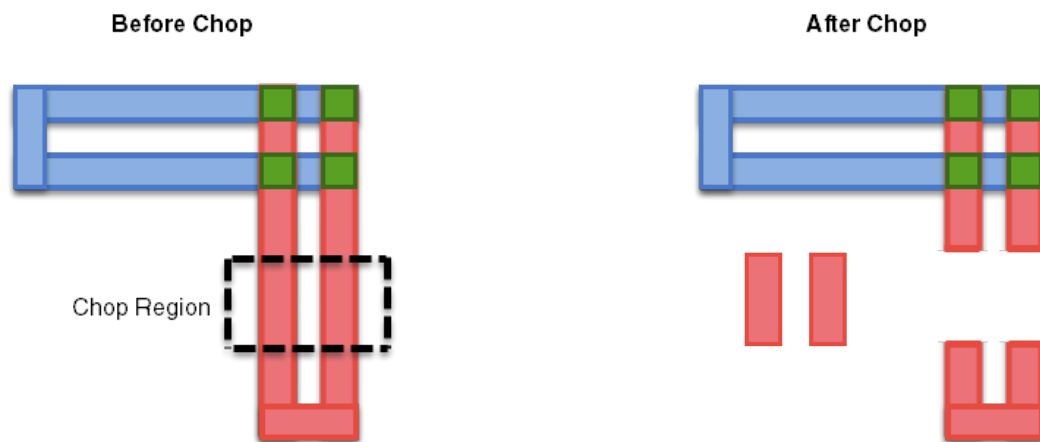
Chopping Striped Wires

You can use the **Edit > Clipboard > Chop** command to chop striped wires. See Using Chop to Yank and Delete a Region for instructions.

The chop operation changes the chopped ends of the striped wire as follows:

- No straps are added to the chopped ends, even if the original striped wires have straps at the begin and end. For information about straps, see [Striping Wires Manually](#).
- The end styles of the wires are truncated regardless of the end styles of the original wires. For information about wire (or path) end styles, see [Creating Interconnects](#).
- The ends of the wires have extension values of 0 regardless of the extension values of the original wires. For information about wire (or path) end extension values, see [Creating Interconnects](#).

The following example shows how striped wires are chopped. Before the chop operation, the striped wires have straps at the beginning and end. After the chop operation, the chopped ends do not have straps.



Trimming Wires

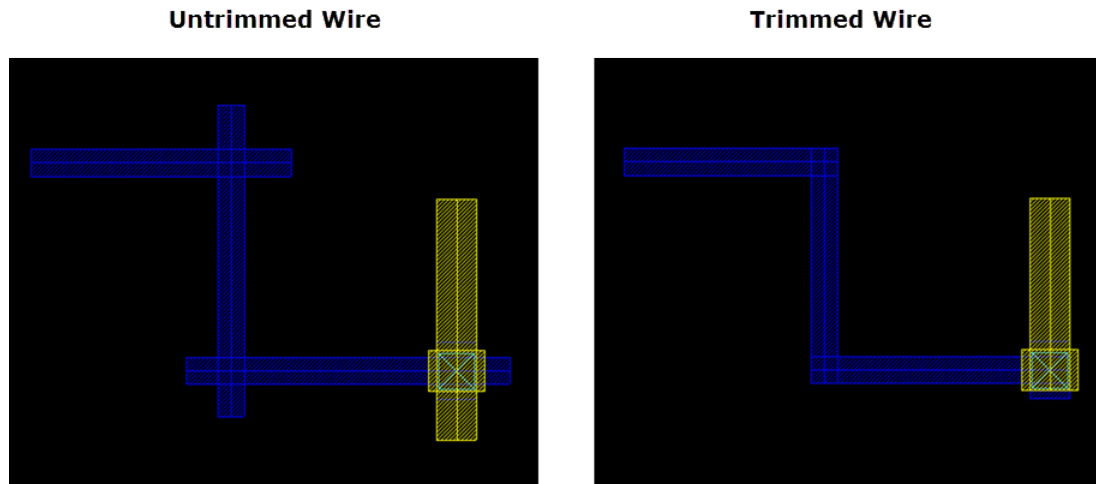
This topic describes the following:

- [About Trimming Wires](#)
- [Using the Edit > Wire > Trim Wire Command](#)
- [Trim Wire Toolbar](#)
- [Trim Wire Examples](#)

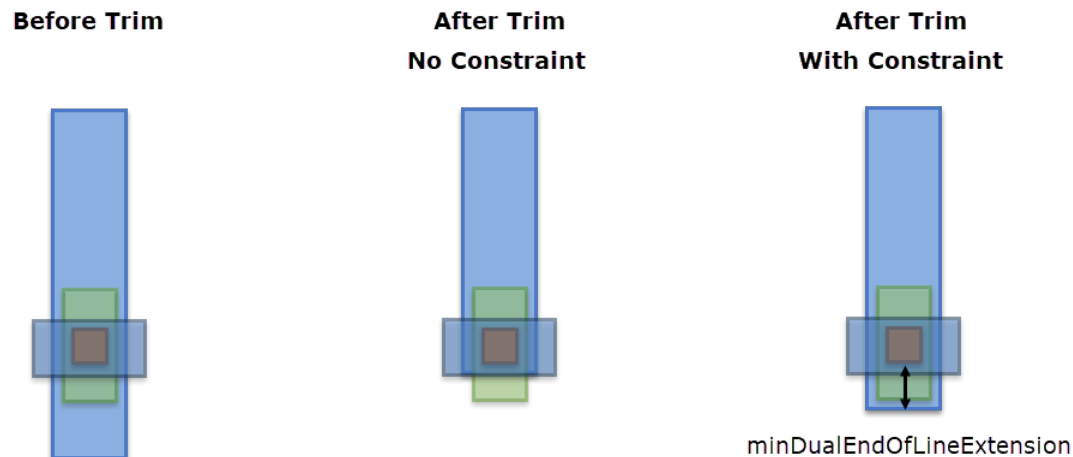
About Trimming Wires

After adding the routes, you might want to edit route segments to optimize the design. For example, you can remove dangling wire segments.

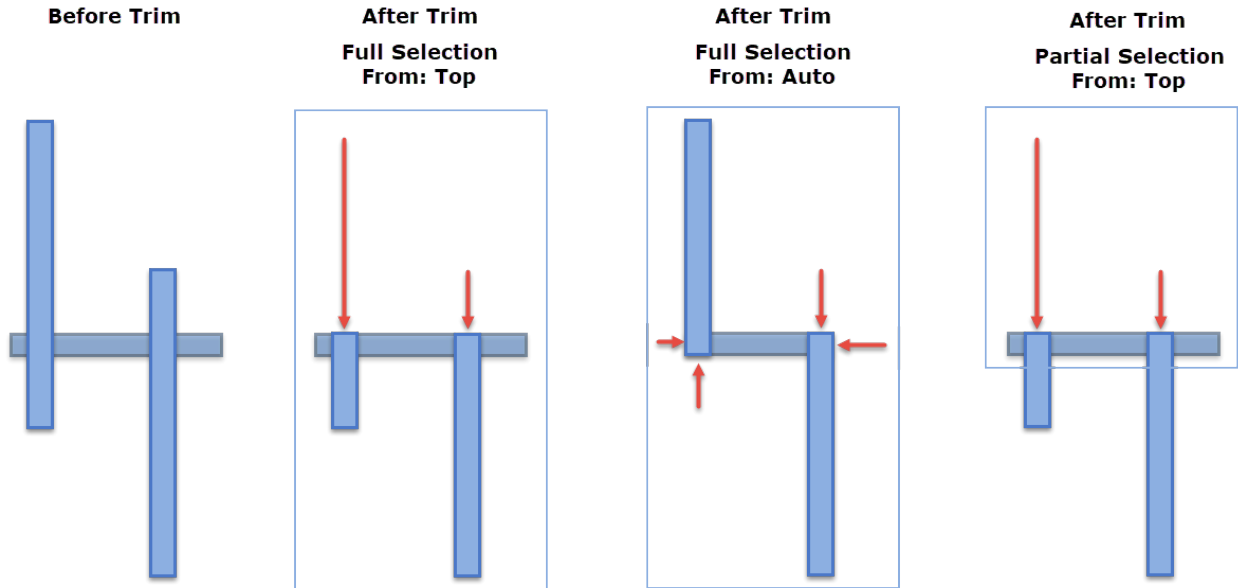
You use the **Edit > Wire > Trim Wire** command to remove undesirable extensions and dangling branches (path and path segments that do not form a closed circuit for current flow) at the intersection of paths and path segments.



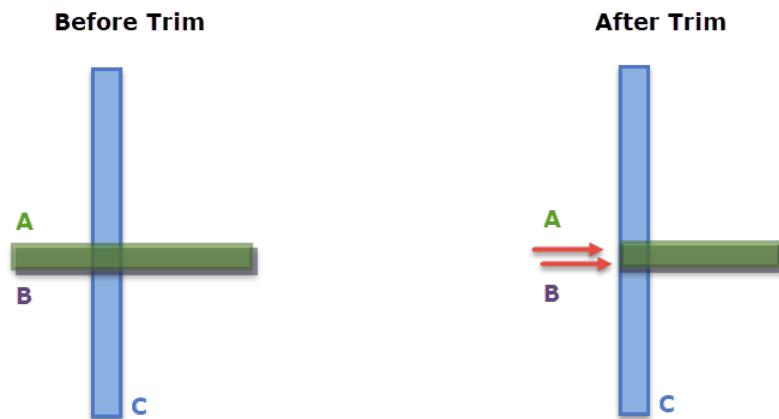
When trimming wires near vias, the results depend on whether the `minDualEndOfLineExtension` technology constraint is defined.



You can trim a single wire, or select and trim multiple wires simultaneously. When multiple wires are selected, the trimming results depend on whether you fully or partially select the wires, and the command options you set. For the descriptions of the options, see [Trim Wire Toolbar](#).



To trim overlapping wires (A and B) that have the same direction as shown, select both wires and the perpendicular wire (C).

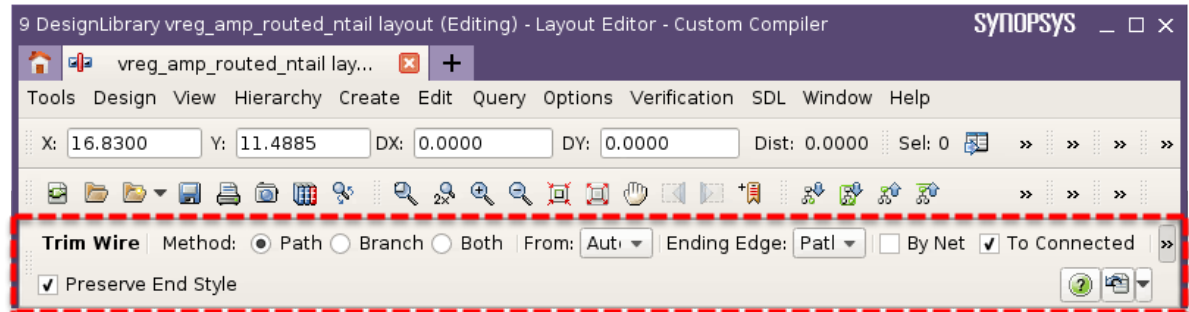


Using the Edit > Wire > Trim Wire Command

Before you start, be aware that the **Trim Wire** command supports both object preselection and postselection. Determine which method you want to use before you use the command. For more information about object selection, see [Preselecting and Postselecting Objects](#).

To trim wires:

1. Choose **Edit > Wire > Trim Wire**. The **Trim Wire** toolbar appears.



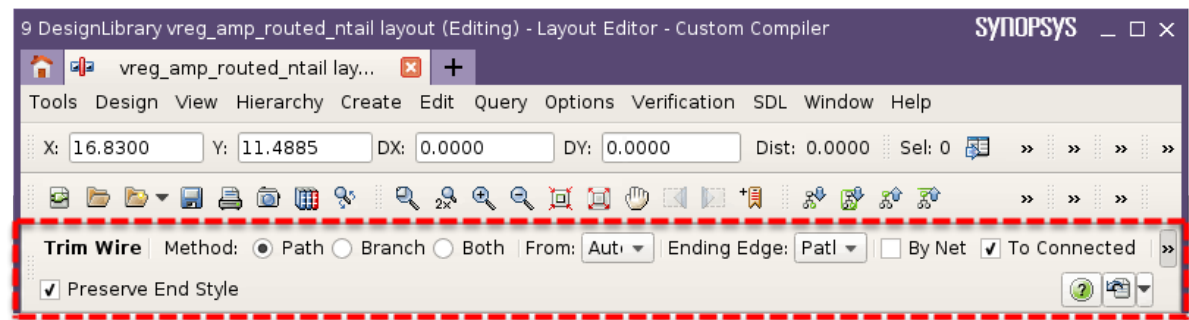
If you preselected objects, the Layout Editor trims the selected wires, and then exits the wire trimming mode.

2. (Optional) On the [Trim Wire Toolbar](#), set the wire trimming options.
3. If you are using the postselection method:
 - a. On the canvas, select the wires you want to trim.
 - b. When you are finished with trimming wires, press Esc to exit the wire trimming mode.

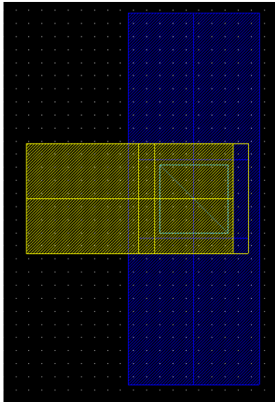
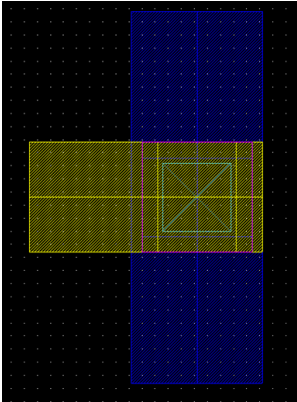
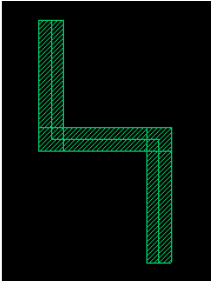
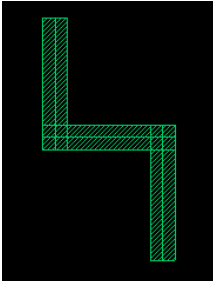
You can also right-click a wire on the canvas to trim it. The menu contains the same options as on the **Trim Wire** toolbar.

Trim Wire Toolbar

This section provides a quick reference of the **Trim Wire** toolbar. For instructions on how to generate multiple pins simultaneously, see [Using the Edit > Wire > Trim Wire Command](#).

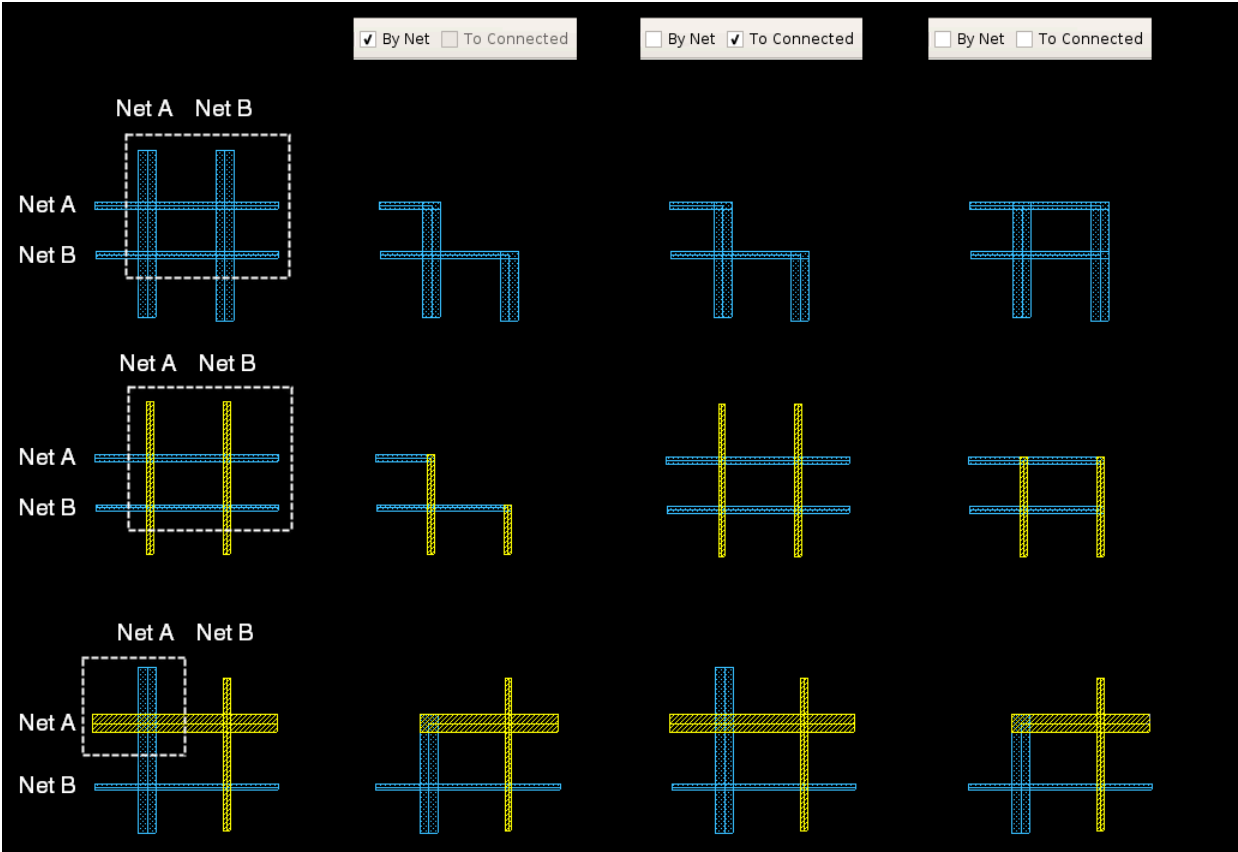


Option	Description
Method	Specifies what to trim: • Path: Trims paths and path segments only. This selection is only applicable if you fully or partially select paths or path segments on the canvas. • Branch: Removes dangling branches. This selection is applicable if you fully select paths, path segments, and vias. • Both: Trims paths and dangling branches. This selection is applicable if you fully select paths, path segments, and vias.
From	Specifies which end to trim: • Auto: Lets the Layout Editor determine which end to trim. If the path or path segment intersects with one other object, the short end is trimmed. If the path or path segment intersects with two or more objects, both ends are trimmed, and the trimmed ends have the well-formed end style regardless of whether you select Preserve End Style. • Left: Trims the end that is on the left side of the object, regardless of whether that end is shorter. • Bottom: Trims the end that is on the bottom side of the object, regardless of whether that end is shorter. • Right: Trims the end that is on the right side of the object, regardless of whether that end is shorter. • Top: Trims the end that is on the top side of the object, regardless of whether that end is shorter. For partially selected objects, the trim always occurs at the selected end regardless of the From selection.

Option	Description
Ending Edge	Ends the path or path segment at the edge of one of the following: • Path • Via  End at Path's Edge End at Via's Edge
By Net	(Method: Path, and Ending Edge: Path) Trims a path up to the nearest intersecting wires that are on the same net. If you do not select the option, the command will trim a path up to the nearest wires regardless of whether they are on the same net. See Trim Wire Examples.
To Connected	(Method: Path, and Ending Edge: Path) Trims a path up to the intersecting connected wires. This option is only available when By Net is turned off. See Trim Wire Examples.
Preserve End Style	Specifies whether the trim should retain the end style that was to be used before trimming the wires.  Preserved Ends Well-Formed Ends

Trim Wire Examples

The following examples show the results of **Trim Wire** with combinations of the **By Net** and **To Connected** settings. For the descriptions of the options, see [Trim Wire Toolbar](#).



Trim Wire Context-Sensitive Toolbar

The context-sensitive toolbar is displayed when the `ile::trimWire` command is activated followed by right-clicking.

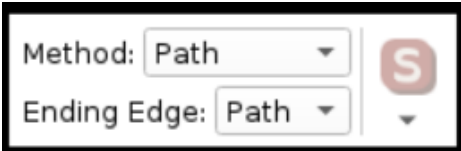


Table 3 *ile::trimWire* options

Options	Description
Method	Defines whether to use the trim path method or the trim branch method. Preference: <code>leTrimWireMethod</code>

Table 3 *ile::trimWire options (Continued)*

Options	Description
Ending Edge	Defines whether the path/pathseg objects end at the path edge or the via edge. This is useful when the neighbor is a via. This option is disabled when Method is set to Branch . Preference: <code>leTrimWireEndEdge</code>
Stack commands tool button	The Set Cursor action is disabled because the Set Cursor command does not support <code>ile::trimWire</code> .

Stretching Interconnects

You use different **Edit > Stretch** commands to stretch interconnects (wires). For instructions on using the command, see [Stretching Objects](#).

The results of a Stretch operation depends on the object, method, and stretch direction. This topic describes the behaviors when:

- [Stretching Paths and Path Segments](#)
- [Stretching Interconnects and Adding Vias Automatically](#)
- [Stretching Interconnects on Track Grids](#)
- [Stretching Striped Wires](#)

If you want to automatically merge overlapping collinear paths and path segments as you stretch them, set the `leAutoMergePaths` Tcl preference to `true`. See [Merging Paths and Path Segments Automatically](#).

Stretching Paths and Path Segments

Stretching paths or path segments reshapes their centerlines and possibly creates or deletes path vertices or path segments.

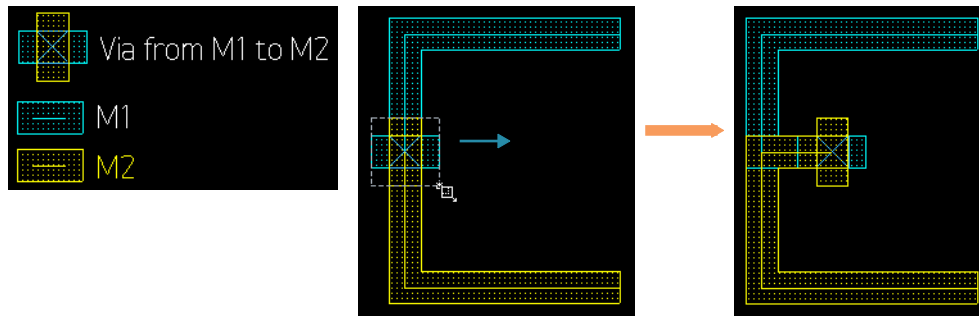
Note:

You can also modify paths and path segments with the **Reshape** and **Reshape Interconnect** commands. **Reshape Interconnect** lets you route new segments to replace an original segment. Stretch only lets you change the shape and position of the original segment.

If you select **Keep Connection**, and the stretched object is connected to a path, path segment, or via, the tool creates new vertices in the path (or new path segments) where needed to maintain connectivity.

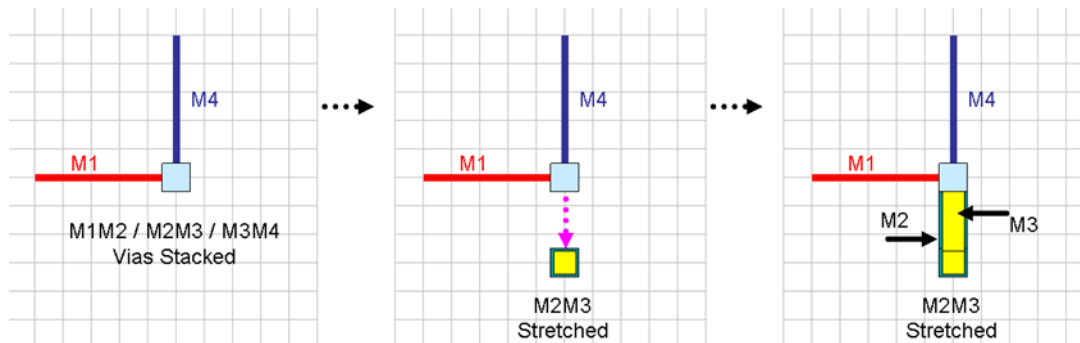
Normally, a fully selected object moves during a Stretch operation, but this is not always true of paths or series of connected path segments: if **Keep Connection** is active, they can be reshaped in order to maintain the connection.

If you stretch a via on an orthogonal path, the via maintains its shape and moves.



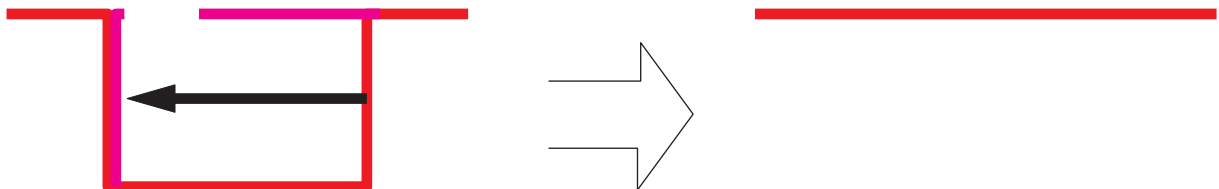
The tool adds orthogonal path segments or new path vertices to maintain the connection if **Keep Connection** is active. The tool also stretches the via stack together although you select only one via in the via stack.

To change the shape of a via, select only the edges and corners you want to stretch.

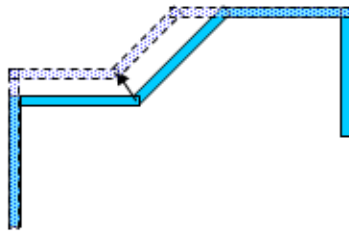


To change the width of a path or path segment, edit the object properties. Stretch never changes path widths, and never changes path segment widths, with one exception: if you change the orientation of a path segment, its width changes automatically to keep the path segment on grid.

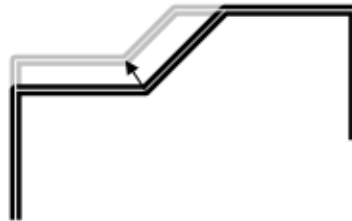
If stretching creates a loop in interconnect, the tool removes the loop automatically.



Stretching paths or a series of connected path segments yields equivalent results, except when you deselect **Lock Angles** and set the Angle mode to **Any Angle**. In this case, because path segments must be oriented at multiples of 45°, stretching constrains path segments to snap to a legal orientation. The following examples show the equivalent stretch results when the Lock Angles option is turned on.

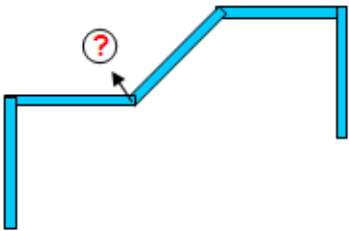


A Series of Connected Path

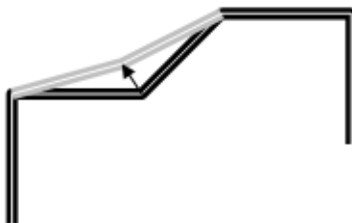


A Path with Several Points

A stretch can produce very different results for paths and path segments when you turn off **Lock Angles** and set the **Angle** mode to **Any Angle**.

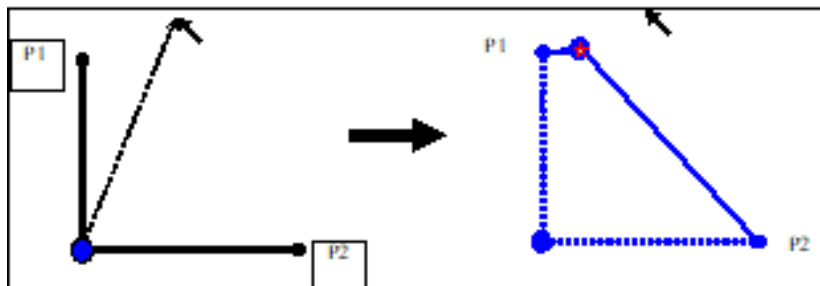
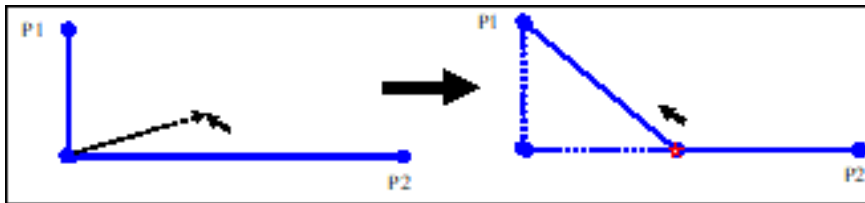


A Series of Connected Path
Segments

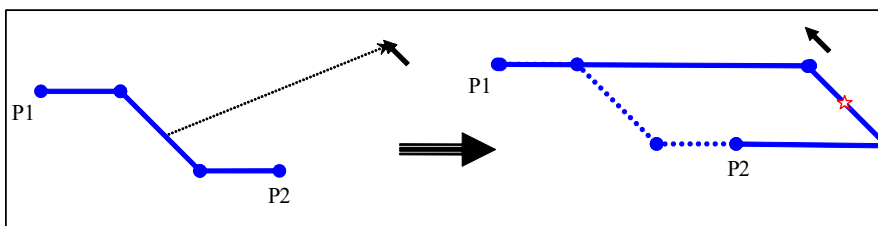


A Path with Several Points

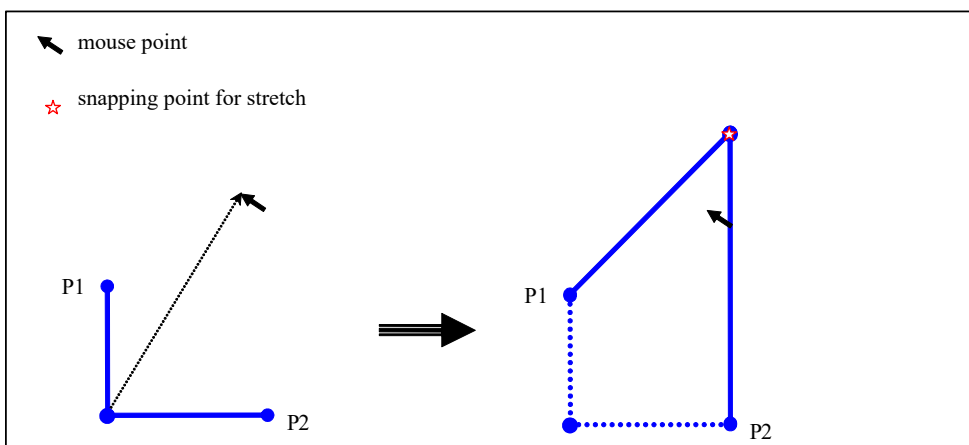
The following examples show the results of stretching path segment vertices when **Lock Angles** is Off.



The following examples show the results of stretching path segments when **Lock Angles** is deselected.

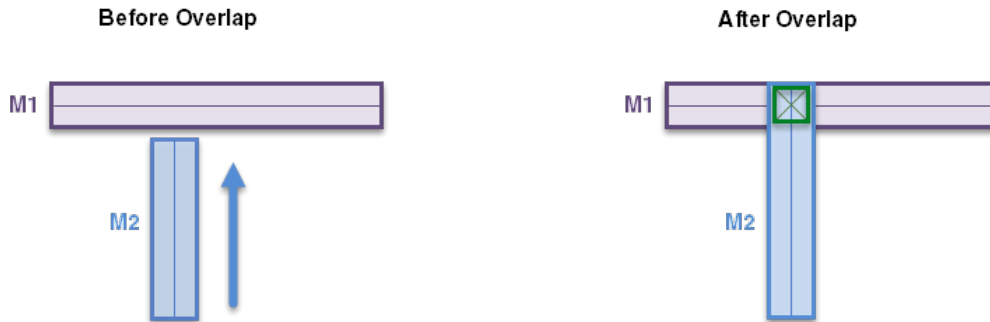


The following example shows the result of stretching a path vertex when **Lock Angles** is deselected.

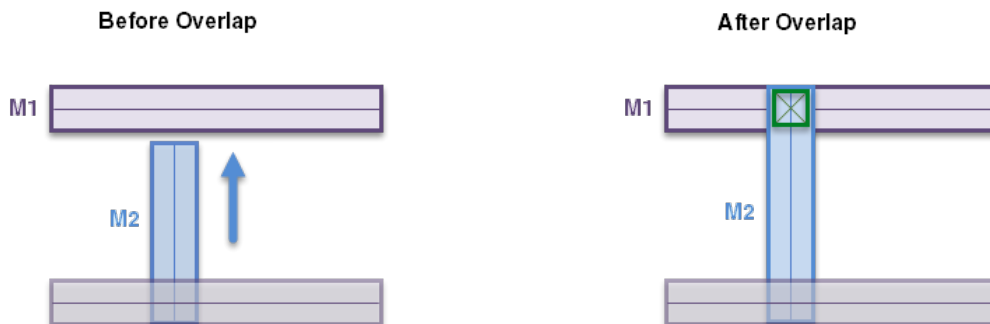


Stretching Interconnects and Adding Vias Automatically

When you stretch an interconnect such that it overlaps with another interconnect on another layer, a via is automatically added to connect the two interconnects.



Vias are added only when two paths overlap as a result of a stretching process. Vias are not added between existing overlapping paths.



Stacked vias can also be created. For example, if the existing interconnect is on m3 and the new interconnect is on m1, then via12 and via23 are added between m1 and m3.

The requirements for adding the vias automatically between overlapping interconnects are:

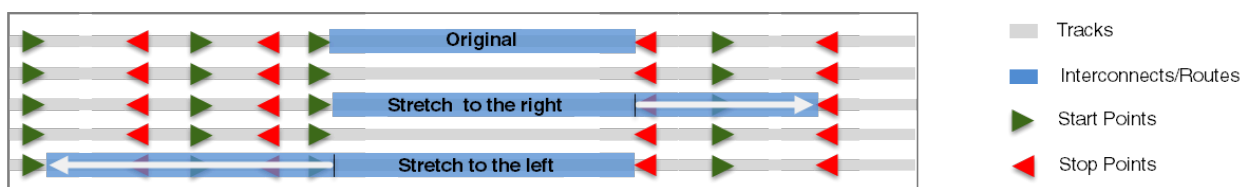
- The overlapping interconnects must be on the same net.
- You must set the `leUpdateConnectedViasAndEndStyles` Tcl preference to `true`.
- You must set the `leUpdateConnectedViasFullStacked` Tcl preference to `true`.

The via is automatically removed if the interconnects no longer overlap.

Stretching Interconnects on Track Grids

When you stretch interconnects that are snapped to track grids (tracks that have starting and stopping points), the interconnect will stretch and snap to the next applicable starting or stopping points.

Stretching Interconnects That Are Snapped to Track Grids



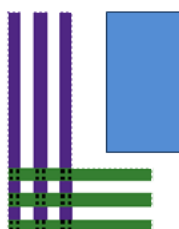
You can stretch an interconnect from one local track region to another local track region or to a global track only if compatible tracks are available. If no compatible track is available in the adjacent region, try to use a different routing layer.

To create interconnects, see [Creating Interconnects](#).

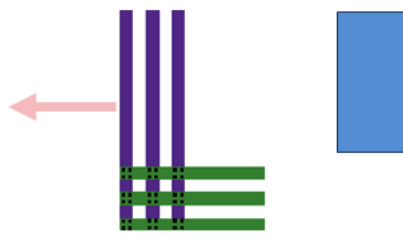
Stretching Striped Wires

When they are fully selected, striped wires move in the direction of the cursor without changing their shapes. In the following diagram, the striped wire is fully selected and then moved to the left. The shape of the striped wire does not change, but the distance between the striped wire and the blue object increases.

Before Stretching

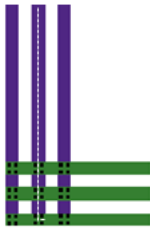


Stretching to the Left

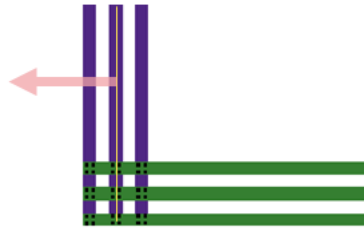


You can select the centerline of a striped wire and move only the wires that are parallel to that centerline. In the following diagram, the vertical centerline is selected and stretched to the left. The vertical wires that are parallel to the centerline move along the x-axis, thus increasing the length of the horizontal wires. To shorten the horizontal wires, move the vertical centerline to the right.

Before Stretching



Stretching to the Left



Moving Paths and Interconnects

You can use the **Move** command to change the location of paths and interconnects on the canvas. See [Moving Objects](#).

This topic describes the setup, behavior, and results of the path and interconnect move operations:

- [Adding Vias Between Overlapping Interconnects](#)
- [Changing LPPs When Moving Paths](#)

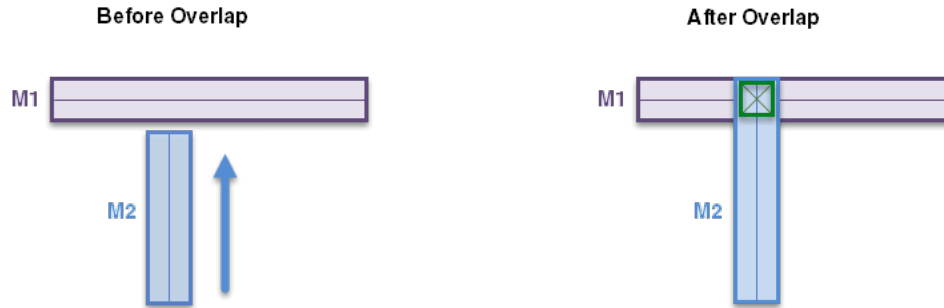
If you want to automatically merge overlapping collinear paths and path segments as you move them, set the `leAutoMergePaths` Tcl preference to `true`. See [Merging Paths and Path Segments Automatically](#).

Adding Vias Between Overlapping Interconnects

When you move an interconnect such that it overlaps with another interconnect on another layer, a via is automatically added to connect the two interconnects. Stacked vias are added if the interconnects are not on adjacent layers. For example, if the existing interconnect is on m3 and the new interconnect is on m1, then via1 is added between layers m1 and m2, and via2 is added between m2 and m3.

The requirements for adding the vias automatically between overlapping interconnects are:

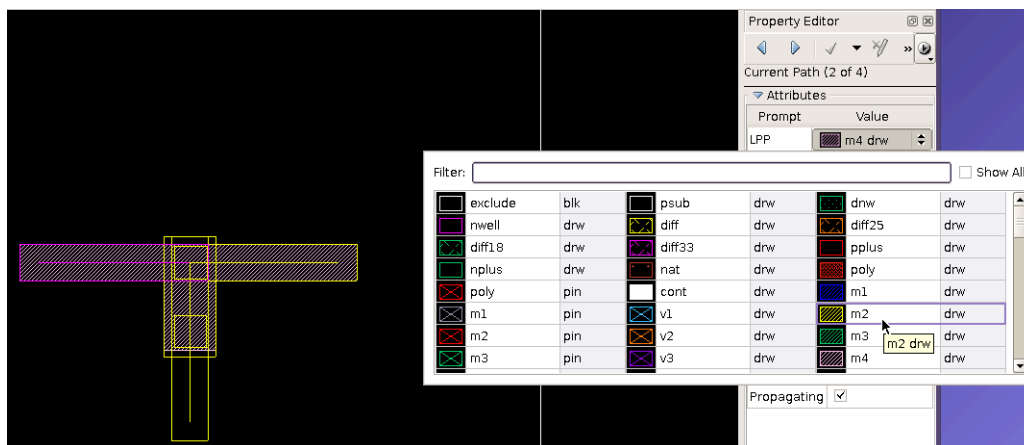
- The overlapping interconnects must be on the same net.
- You must set the `leUpdateConnectedViasAndEndStyles` Tcl preference to `true`. For more information, see the *Custom Compiler Tcl Reference Manual*.



The via is automatically removed if the interconnects no longer overlap.

Changing LPPs When Moving Paths

When moving paths, you can change the metal layer of the path object using the Property Editor.



Merging Paths and Path Segments Automatically

This topic describes the following:

- [About Merging Paths and Path Segments](#)
- [Requirements for Merging Paths and Path Segments Automatically](#)
- [Results of Path and Path Segment Merging](#)

About Merging Paths and Path Segments

When you [create](#) or edit a 0x path or path segment such that it overlaps with a collinear 0x path or path segment on the same layer and same width, the two shapes will merge and form one object.

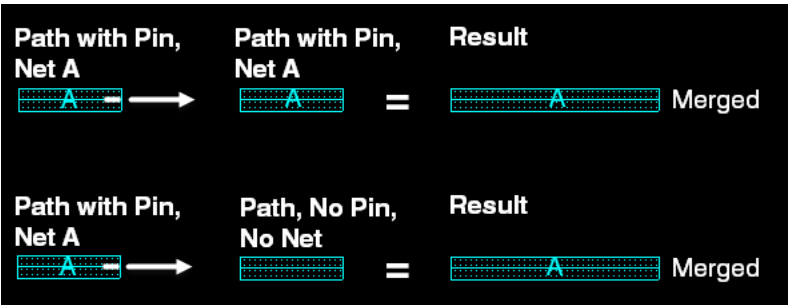
Note:

0x paths and path segments will not merge with paths and path segments on lower levels of the hierarchy.

Requirements for Merging Paths and Path Segments Automatically

Automatic merging of collinear paths and path segments occurs only if the following requirements are met:

- The `leAutoMergePaths` Tcl preference is set to `true`. By default, the Tcl preference is set to `false`.
- The overlapping paths and path segments have the same:
 - Net assignment
 - Width
 - Layer-purpose pair (LPP)
 - Color
- Paths and path segments that are connected to pins can merge if one of the following conditions is met:
 - Merging paths or path segments have pins that are on the same net.
 - One paths or path segments is not connected to any pins, and the path or path segment does not have a net assignment.

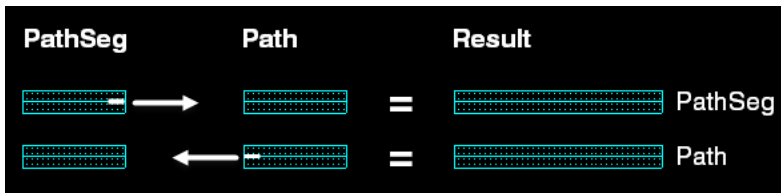


Results of Path and Path Segment Merging

This section describes the conditions that affect the automatic merging results.

The active or selected object has higher priority.

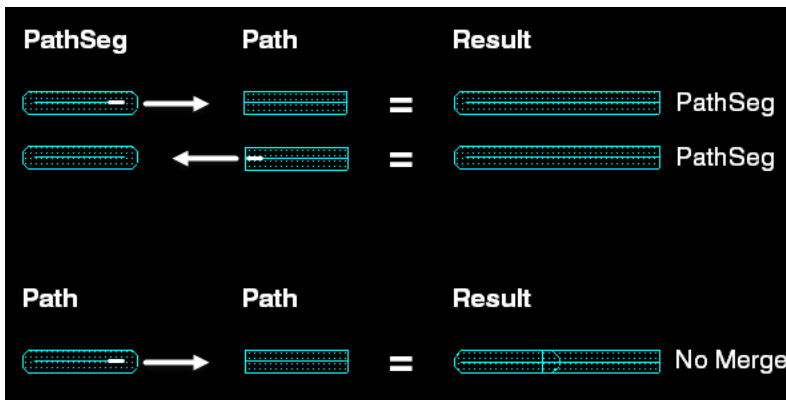
When merging a path segment and a path with the same end styles, the active or selected object has higher priority. For example, if you stretch the path segment so that it overlaps with the path, the resulting object is a path segment. If you stretch the path so that it overlaps with a path segment, the resulting object is a path.



The end styles affect the results.

Path segments can have different end styles, but paths cannot. Therefore, regardless of the active or selected object:

- If you merge a path segment with a path that has a different end style, the result will always be a path segment.
- You cannot merge a path that has the round end style with a path that has a different end style.



Merging path segments with bending paths always produces paths.

Regardless of the active or selected object, merging path segments with bending paths always results in paths.



Merge will not occur if the result is invalid.

If the merge will produce an invalid result, no merging will occur.



Stretching the Wire Width

To fix resistance constraint violations, you can use the **Edit > Stretch > Stretch Wire Width** command to widen wires.

This topic describes the following:

- [About Stretching Wire Widths](#)
- [Procedure for Stretching Wire Widths](#)

About Stretching Wire Widths

To reduce wire resistance, you can use the **Edit > Stretch > Stretch Wire Width** command to adjust the width of the wire that has violated the resistance constraint. You can stretch one side or both sides of the wire.

In addition to adjusting the wire width, **Stretch Wire Width** also lets you check the resistance during the modification and displays violations in real time.

Note:

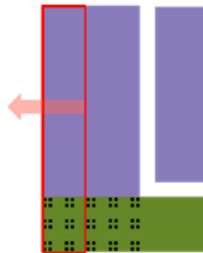
The minimum wire width you can set is the minWidth value set in the technology file. The maximum wire width you can set is 1000 um.

When you stretch the width of non-stripped wires, the width of the wire increases or decreases. In the following example, the **One Side** option is selected, and the left side of the wire is dragged to the left. Notice that both the wire width and the number of vias increased. To decrease the wire width in this case, drag the left side of the wire to the right. To move the vertical wire along the x-axis without changing its width, drag the wire's centerline to the left or right.

Before Stretching



Stretching the Left Side

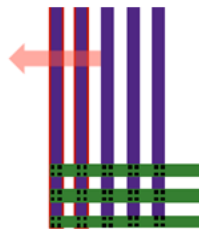


When you use **Stretch Wire Width** on striped wires, you can increase or decrease the number of wire stripes. In the following example, the **One Side** option is selected, and the left side of the wire is dragged to the left. Notice that the number of stripes increased. To decrease the number of stripes, you would drag the left side of the wire to the right.

Before Stretching



Stretching the Left Side

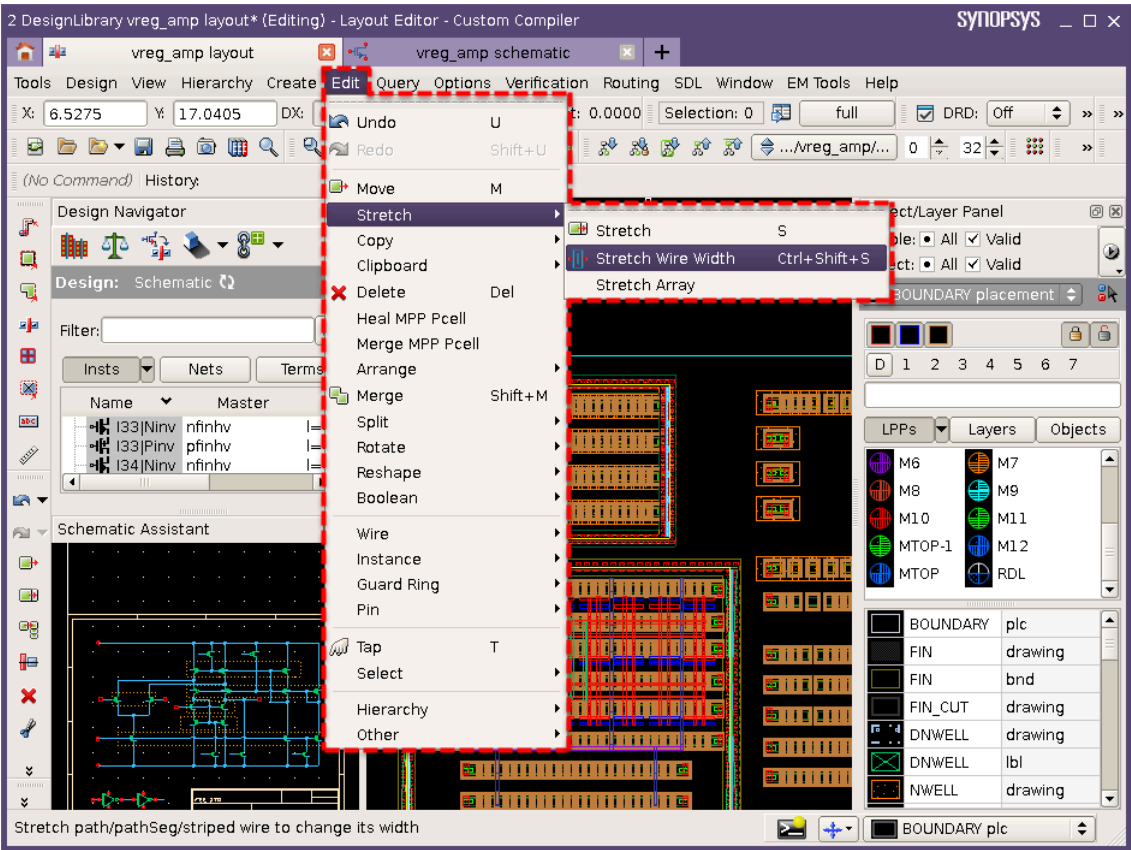


Procedure for Stretching Wire Widths

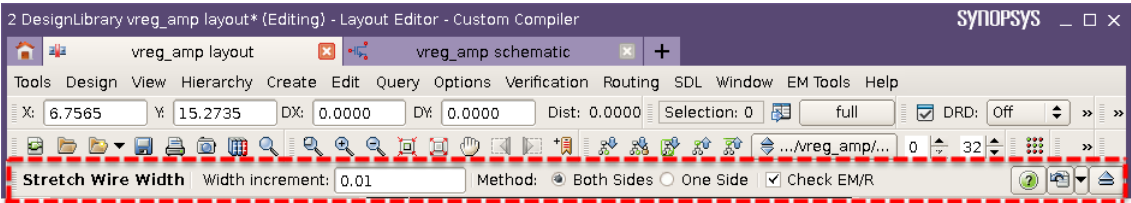
This section describes how to use the **Edit > Stretch > Stretch Wire Width** command to stretch wire widths. To use the `ile::stretchWidth` Tcl command, see the *Custom Compiler Tcl Reference Manual*.

To stretch the wire width:

1. In the Layout Editor, zoom in to display the wire whose width you want to stretch.
2. Choose **Edit > Stretch > Stretch Wire Width**.



The **Stretch Wire Width** toolbar appears.



3. On the **Stretch Wire Width** toolbar, set the following options:

Option	Description
Width Increment	Increase the wire width by the specified micrometer increments.
Method	Specifies which side of the wire to stretch: ◦ Both Sides: Stretches both sides of the wire equally. ◦ One Side: Stretches only one side of the wire. To specify the side to stretch, on the canvas, drag the side of the wire that you want to stretch.

Option	Description
Check EM/R	Checks the resistance while you make the width modifications and displays violations in real time.

4. On the canvas, select the wire that you want to stretch, and then drag upward to increase the width. As you do so, a tooltip displays the new width of the wire. The width should increase by the Width Increment you specified.

You might need to make other adjustments in your design. To modify the layout, see [Editing Objects](#).

5

Creating and Editing Bus Routes

This chapter describes how to create and edit bus routes (trunks) in the designs.

This chapter describes the following:

- [About Bus Routes](#)
- [Before You Start](#)
- [Creating Bus Routes](#)
- [Creating Bus Context-Sensitive Toolbar](#)
- [Extending Existing Bus Routes](#)
- [Adding Vias During Bus Creation](#)
- [Connecting Bus Bits on the Same Layer](#)
- [Continuing Bus Routes](#)
- [Compressing and Expanding Bus Routes](#)
- [Cutting Corners of Bus Routes](#)

About Bus Routes

A bus is a group of parallel routes that you add simultaneously. Individual routes in a bus are called bits.



You can use one of the following methods to add bus routes:

Routing Method	Description
Manually	Use the Create > Bus command to manually add bus routes. You run design-rule checks after the bus bits are placed. See Creating Bus Routes .
Interactively, with user guidance	Use the Custom Router to route bus bits while you provide some guidance. Design-rule checks can be run in real-time. See Interactive Bus Routing .

To add interconnects, see [Creating and Editing Interconnects](#).

To add interconnects or routes for conducting high-frequency alternating currents (AC), such as radio and microwave signals, see [Creating and Editing Transmission Lines](#).

Before You Start

Before you start to create bus routes, review the applicable prerequisites and be aware of default settings that might impact the results.

This topic describes the following:

- [Creating Paths or Path Segments in Bus Routes](#)
- [Displaying Detailed Information About the Bus Routes](#)
- [Starting Bus Routes From 0x Pins](#)
- [Extending Path Ends to Cover Vias](#)

Creating Paths or Path Segments in Bus Routes

To specify whether you create paths or path segments during the **Create Bus** operation, select the **Interconnect Object** option in the **Command** tab of the **Layout Design Options** dialog box. See Setting Layout Design Options.

Displaying Detailed Information About the Bus Routes

To display the resistance, capacitance, and length of the bus as your pointer moves on the canvas, select **Show Dynamic Measurement** on the **Cursor** tab of the **Layout Display Options** dialog box. See Setting Layout Display, Cursor, and Selection Options.

Starting Bus Routes From 0x Pins

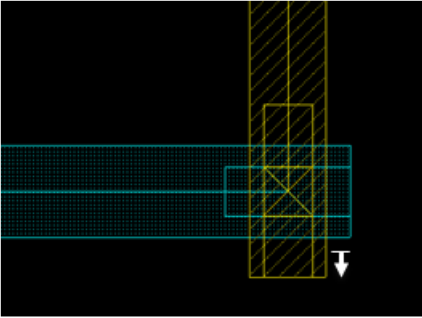
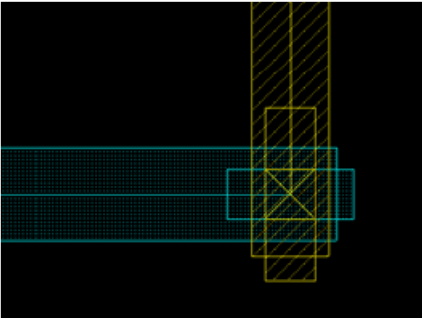
You can start the bus routes from pins on 0x to **Stop Level** + 1 hierarchy levels. To do this:

- Set the `leDrawInstPins` Tcl preference to `true`.
 - Make sure you set the desired **Stop Level** in the **Layout Design Options** dialog box. For information, see Setting Layout Design Options.
 - In the Object/Layer Panel, make sure the Instance Pin **Selectability** setting is on so that pins from 0x to **Stop Level** + 1 will be available. If **Selectability** is off, only pins on the 0x level are available when creating bus trunks.
-

Extending Path Ends to Cover Vias

Under some circumstances, the added via metal layers can be longer than the path ends, causing short-edge violations. To ensure the paths fully cover the vias, the `leInterconnectExtendPathEndToViaEnd` Tcl preference is set to `true` by default.

The following diagrams show the results of the `leInterconnectExtendPathEndToViaEnd` Tcl preference settings:

Preference Setting	Result
true (default)	
false	

This preference applies to vias that are added during the following command operations:

- **Create Interconnect**
- **Reshape Interconnect**
- **Create Bus**
- **Continue Bus**

In addition, the preferences is only applied when the pathend style is one of the following:

- **Truncate**
- **Extend**
- **Variable**

The preference does not apply to the **Round** pathend style.

See Also

- [Choosing End Styles](#)

Creating Bus Routes

You use the **Create > Bus** command to create bus routes manually. This topic describes the following:

- [About the Create > Bus Command](#)
- [Bus Routing Across Multiple LPPs](#)
- [Creating New Bus Routes](#)
- [Bus Toolbar and Create Bus Dialog Box Reference](#)
- [Create Bus Shortcut Menu and Shortcut Keys](#)

 [Show me](#) how to create bus routes.

About the Create > Bus Command

The **Create > Bus** command allows you to:

- Manually create bus routes on the same layer or across different layers. See [Bus Routing Across Multiple LPPs](#).
- Set the starting widths of the trunk bits. The bits can have uniform or different widths.
- Start the bus route from non-aligned pins.
- Automatically adjust the widths of the trunk bits during the bus creation process to connect properly to pins or other shapes. See [Bus Routing Across Multiple LPPs](#).
- Assign colors to the trunk bits. You should also be aware of the color propagation rules. For information, see Color Propagation.
- Lock the assigned colors.
- Add bus bits to extend the ends of two existing bus routes and connect them. See [Extending Existing Bus Routes](#).

Bus Routing Across Multiple LPPs

The **Create > Bus** command has an **Adjust Width > Taper** option that automatically adjusts the width of the bus bits to the width of the pins. When **Taper** is turned on, you have the option of selecting **Multiple LPPs** to create bus routes and connect to pins

across multiple layer-purpose pairs (LPPs). See [Bus Toolbar and Create Bus Dialog Box Reference](#).

When creating bus routes across multiple LPPs:

- As you bend (or turn) the bus bits, the spacing and width of the leading bit is applied across the remaining bits.
- After the bend, you can use the **Multiple Widths/Spacings** option to modify the width and spacing of the bits. See [Bus Toolbar and Create Bus Dialog Box Reference](#).

When connecting to pins across multiple LPPs:

- If pins with the same or different shapes overlap partially (have different positions), the bus bits can connect to the individual pins.

Overlapping Pins



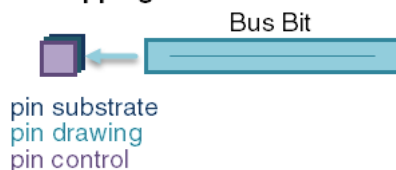
- If multiple identical pins overlap completely (same shape boundary, same positions, and same LPP), the shape with the pin (or terminal) has higher connection priority.

Overlapping Pins



- If multiple identical pins overlap completely (same shape boundary, same position, same layer number), but they have different layer purposes, the drawing purpose will have connection priority.

Overlapping Pins



- A source pin can only connect to a selected target pin that has the same net name and LPP. However, the set of source and target pins connected by the bus route does not

have to be in the same order. Jogs might be added automatically to honor spacing and other requirements. You can edit and adjust the routes as needed.

Source and Destination Pin Order

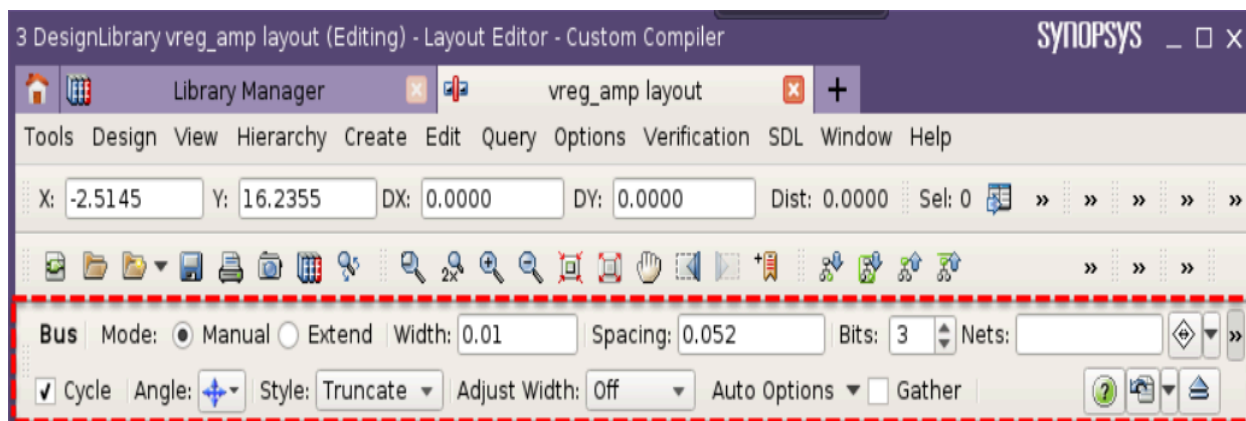


Creating New Bus Routes

To create a bus:

1. Choose **Create > Bus**, or press Ctrl+I.

The **Bus** toolbar appears. The cursor changes from  to .



To display the **Create Bus** dialog box, click  on the **Bus** toolbar. The options on the **Bus** toolbar are mostly identical to the options in the **Create Bus** dialog box.

2. Specify the options you want to apply. See [Bus Toolbar and Create Bus Dialog Box Reference](#).

To ensure that path widths fit pin widths, or to connect shapes that are not aligned, choose **Adjust Width > Taper** and make sure **Auto Options > Terminate** is disabled.

To create bus routes across multiple LPPs, select **Multiple LPPs**.

3. (Optional) In the Object/Layer Panel, select an interconnect LPP.

If you do not select an interconnect LPP, the Layout Editor will create the bus trunk using the nearest LPP.

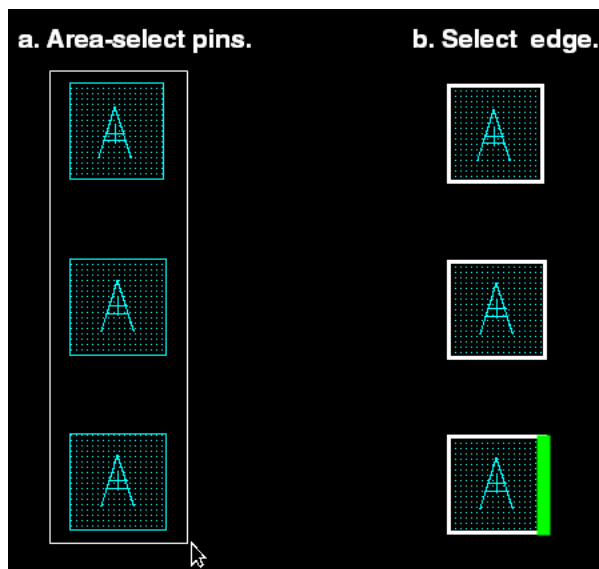
4. (Optional) To start from selected nets, in the Design Navigator, select the nets.

If the `leAutoPickUpNamesFromDN` Tcl preference is true, the net names automatically appear on the **Bus** toolbar. In addition, the number of selected nets appear in the **Bits** box on the toolbar.

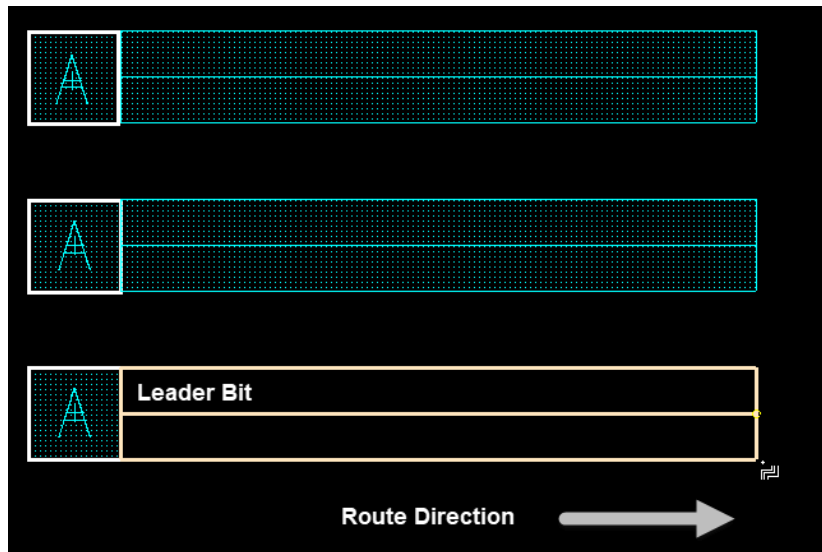
5. Click a location on the canvas to enter the starting point.

When connecting to pins, and if you selected **Adjust Width > Taper**, do the following to select a group of source pins as the starting points:

- a. Area-select the pins of interest. To do this, on the layout canvas, draw a rectangle to enclose the pins of interest. The selected pins are highlighted.
- b. Select the starting edge. The selected edge is highlighted.



6. Move the cursor on the canvas to direct the bus trunk.



7. Click to change direction on the same layer. Right-click to add via, change layer, change angle, and add or remove a point. For more information, see [Create Bus Shortcut Menu and Shortcut Keys](#) and [Adding Vias During Bus Creation](#).
8. When connecting to destination pins and you selected **Adjust Width > Taper**:
 - Select the first destination pin, and then Shift+click to select additional destination pins.
 - Select the first destination pin, and then Shift+drag to draw a rectangular area around the remaining destination pins.
9. Double-click to enter the last points and complete the paths.

The command remains active.

10. When you are finished, press Esc to quit the command.

For information on using the `ile::createBus` Tcl command, see “Layout Editing Tcl” in the *Custom Compiler Tcl Reference Manual*.

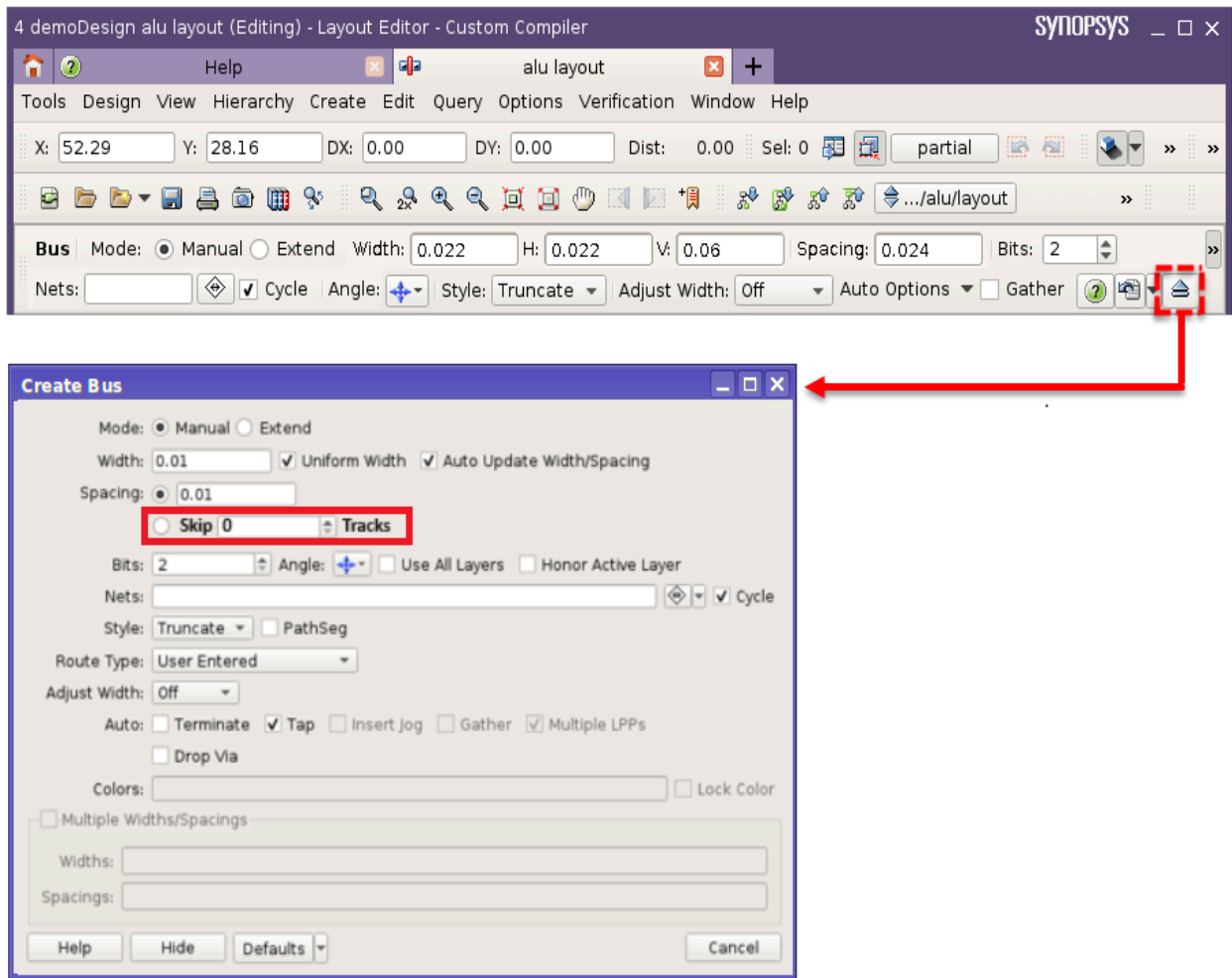
Bus Toolbar and Create Bus Dialog Box Reference

This section provides a quick reference of the **Bus** toolbar and the **Create Bus** dialog box:

- [Manual Mode](#)
- [Extend Mode](#)

For instructions on how to create bus routes, see [Creating Bus Routes](#). For instructions on how to extend and connect two existing bus routes, see [Extending Existing Bus Routes](#).

Manual Mode



The toolbar and dialog box contain the following options:

Option	Description
Mode	Mode in which you want to create bus: • Manual: Creates new bus routes. See Manual Mode. • Extend: Adds bus bits to connect the ends of two existing bus routes. See Extend Mode.

Option	Description
Width	Defines width per bus bit. The same width is applied to all bits. The default value is derived from the <code>minWidth</code> Layer constraint in the technology library. If <code>minWidth</code> is not defined in the technology library, then the Layout Editor will use the value set by the <code>leDefaultPathWidth</code> Tcl preference. The Layout Editor keeps track of the most recently used value for each layer purpose pair during the current session. When you begin creating a new interconnect on a previously used layer purpose pair, the toolbar defaults to the previous value. Changing the width changes the width of all bits from the first point of the interconnect or the last inserted via, whichever is first.
Width: H	(Available only if you specified the <code>spacingDirection</code> argument for the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraints in the technology library) Specifies the width of the current horizontal bus bits. The default value is derived from the <code>minWidth</code> or <code>allowedWidthRange</code> Layer constraint in the technology library, where the <code>spacingDirection</code> argument is <code>vertical</code>. If the width is not defined in the technology library, then the Layout Editor will use the value set by the <code>leDefaultPathWidth</code> Tcl preference. The Layout Editor keeps track of the most recently used value for each layer purpose pair during the current session. When you begin creating a new bus on a previously used layer purpose pair, the toolbar defaults to the previous value.

Option	Description
Width: V	(Available only if you specified the <code>spacingDirection</code> argument for the <code>minSpacing</code> or <code>allowedSpacingRange</code> layer constraints in the technology library) Specifies the width of the current vertical bus bits. The default value is derived from the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraint in the technology library, where the <code>spacingDirection</code> argument is <code>horizontal</code>. If the width is not defined in the technology library, then the Layout Editor will use the value set by the <code>leDefaultPathWidth</code> Tcl preference. The Layout Editor keeps track of the most recently used value for each layer purpose pair during the current session. When you begin creating a new bus on a previously used layer purpose pair, the toolbar defaults to the previous value.
Uniform Width	Enforces the same width for all path segments. Turning off the option permits different widths across different path segments.
Auto Update Width/Spacing	Uses the active layer's minimum width and spacing if you do not set the Width and Spacing options. Turning off the option uses the last-set values in the dialog box, as long as the values do not violate the active layer's minimum width and spacing.
Spacing	Defines the global centerline-to-centerline spacing between bits of the bus trunk. The same spacing is applied to the space between all bits in the bus trunk. The default value is derived from the <code>minSpacing</code> layer constraint in the technology library. If <code>minSpacing</code> is not defined in the technology library, then the Layout Editor will use the value set by the <code>leDefaultBusSpacing</code> Tcl preference.

Option	Description
Spacing: H	Specifies the spacing between the horizontal bus bits. Spacing H  The default value is derived from the <code>minSpacing</code> or <code>allowedWidthRange</code> layer constraint in the technology library, where the <code>spacingDirection</code> argument is <code>vertical</code>. If the technology library does not have the spacing defined, then the Layout Editor will use the value set by the <code>leDefaultBusSpacing</code> Tcl preference. The Layout Editor keeps track of the most recently used value for each layer purpose pair during the current session. When you begin creating a new bus on a previously used layer purpose pair, the toolbar defaults to the previous value.
Spacing: V	Specifies the spacing between the vertical bus bits. Spacing V  The default value is derived from the <code>minWidth</code> or <code>allowedWidthRange</code> layer constraint in the technology library, where the <code>spacingDirection</code> argument is <code>horizontal</code>. If the technology library does not have the spacing defined, then the Layout Editor will use the value set by the <code>leDefaultBusSpacing</code> Tcl preference. The Layout Editor keeps track of the most recently used value for each layer purpose pair during the current session. When you begin creating a new bus on a previously used layer purpose pair, the toolbar defaults to the previous value.
Skip Tracks	Specifies the number of tracks to skip during bus spacing to support multiple paths creation The default value is '0'.
Bits	Defines the number of bits. Note: The net name repeats and applies the count of the net name to the rest of the bus bit, if the bus bit index is larger than the count of net name. For example: • Example 1: Net name = "a", bits = 5 => resultant Net is "a a a a a" • Example 2: Net name = "a b", bits = 5 => resultant Net is "a b a b a"

Option	Description
Angle	Specifies the angle of the bus trunk when it changes direction: - Orthogonal: The new segments are 90° of the current trunk. - Diagonal: The new segments are 45° of the current trunk. X First: As you move diagonally from the previous point, horizontal segments are added first, followed by vertical segments. Y First: As you move diagonally from the previous point, vertical segments are added first, followed by horizontal segments.
Nets	Defines net names to be associated with the routing objects to be created.
 (Synchronize)	Copies the names of selected nets in the Design Navigator and pastes them in the Nets box. The order of the net names in the Nets box matches the order of the selected nets in the Design Navigator.
Cycle	Cycles bus bits width among the following three cases: - Width of previous segment - Minimum constraint compliant width - user defined width
Style	(Available when only when you clear the pathSeg check box) Specifies the end styles of the paths: Extend Truncate
Begin/End Styles	(Available when only when you select pathSeg .) Specifies the end styles of the path segments. The first menu defines the begin style of the bus bits. <i>Extend</i> <i>Truncate</i> The second menu defines the end style of the bus bits. <i>Extend</i> <i>Truncate</i>
PathSeg	Specifies whether the bus bits are paths or path segments. Select the check to create path segments. Clear the check box to create paths.
Route Type	The type of route (for example, signal, PG, clock, and so on). Choose the route type that is applicable to the path you are creating. This options is used primarily in analog and digital co-designing.

Option	Description
Adjust Width	Adjusts the width. • Off: Bus bit width is not adjusted automatically. • Narrow: Adjust width to the narrow. dimension of the tapped rectangular object • Wide: Adjust width to the wide dimension of the tapped rectangular object. • Taper: Adjust width to the dimension of the rectangular object facing the bus. When Taper is enabled, you can set the <code>leTaperToCoverPin Tcl</code> preference to fully cover the pin or cover to the edges of the pin. If the <code>leTaperFromSelectedEdge Tcl</code> preference is set to <code>true</code>, the width of the bits will taper and match the width of the selected pins or their edges when you start to create the bus.
Auto Options	• Terminate: Defines if interconnect automatically commits. • Tap: Defines if interconnect runs an implicit tap at the first add point. • Insert Jog: Defines if jog can be inserted when Adjust Width is not set to Off.
Gather	Uses <code>minSpacing</code> to set the distance between two adjacent bits. This option is not available when you are connecting to pins on different layer-purpose pairs (LPPs).
Multiple LPPs	(Available only when you choose Adjust Width: Taper) Creates bus routes on multiple layer-purpose pairs (LPPs).
Colors	(Available only if the design contains colors) Assigns a color to the bus bits as follows: • Auto: Automatically assigns a color to the new shape based on the color propagation rule. If no propagation can be applied, then the shape or via is assigned the default color. • <colorName>: Assigns the selected color, <code><colorName></code>, to the newly created shape. The available color names are defined in the technology file. • Same: Assumes that nearby shapes have the same color and applies the <code>multiPatternSpacing</code> constraint. This selection is available only if you set the <code>leIncludeColorSame</code> preference to <code>true</code>. • No Color: Does not apply any color to the newly created shape. This selection is available only if you set the <code>leIncludeColorNone</code> preference to <code>true</code>.
Lock Color	(Available only if the design contains colors) Locks or unlocks the assigned color. Locking and unlocking a shape's color changes its ability to propagate its color to or inherit color from overlapping shapes. For more information, see Color Propagation Rules.

Option	Description
Multiple Widths/Spacings	(Dialog box only) Specifies different width and spacing values for the bus bits. Add a character space to separate adjacent values (for example, 0.1 0.2 0.3). Widths: If you specify 0, <code>minWidth</code> is used. Spacings: If you specify 0, <code>minSpacing</code> is used. Multiple Widths/Spacings will ensure that the paths and path segments comply with the <code>spacingDirection</code> that is specified by the <code>minWidth</code> and <code>minSpacing</code> constraints. If Bits is greater than the number of Widths or Spacings values, the Widths or Spacings values are repeated. For example, if the number of bits is 4, and you supply two width values (0.15 0.25), then the bit width assignments are 0.15, 0.25, 0.15, and 0.25. You cannot add vias automatically or manually when the bus bits have different widths and spacings. If you also specified Multiple Widths/Spacing values, the Layout Editor ignores any Width and Spacing values.
Defaults	Resets all of the options to your Custom Compiler environment default values.

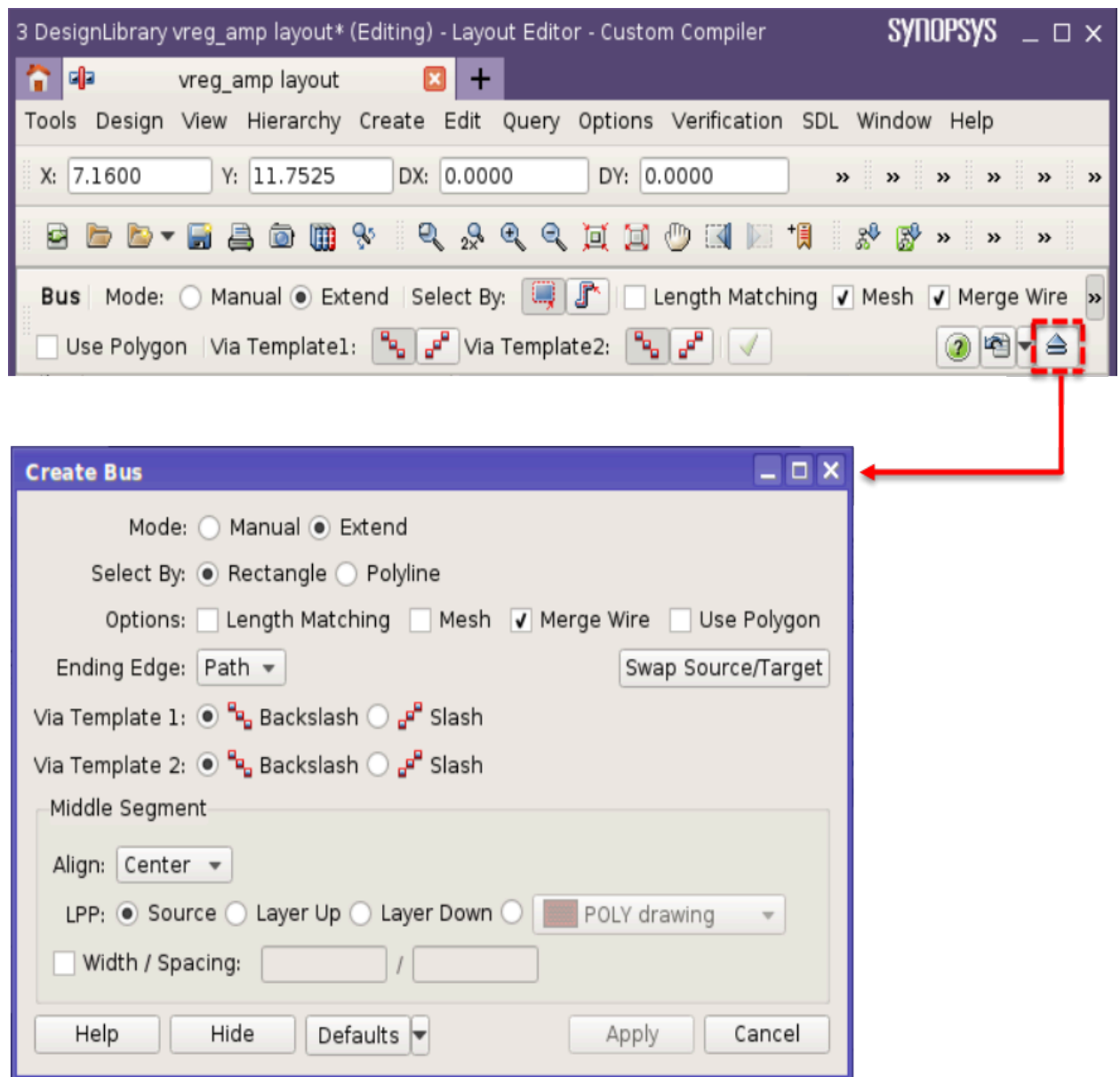
If you choose the **Adjust Width - Taper** option to taper the bits, the Layout Editor uses one of the following widths. To cycle forward through the widths in the order shown, press Ctrl+4. To cycle backwards through the widths, press Shift+4.

First and Last Segment Tapered on Pins	Other Segments
1. Pin width	1. NDR minimum width
2. NDR minimum width	2. Width you specified on the Bus toolbar
3. Foundry minimum width	3. Width of the previous segment
4. Width you specified on the Bus toolbar	

If **Taper** is on but no pins are tapered, or if taper is off, the Layout Editor uses one of the following widths. You can cycle through the widths, in the order shown:

First Segment	Other Segments
1. NDR minimum width	1. NDR minimum width
2. Width you specified on the Bus toolbar	2. Width you specified on the Bus toolbar
	3. Width of the previous segment

Extend Mode



The toolbar and dialog box contain the following options:

Option	Description
Mode	Mode in which you want to create bus routes: • Manual: Creates new bus routes. See Creating New Bus Routes. • Extend: Adds bus bits between two existing bus routes to connect them. See Extending Existing Bus Routes. Tcl preference: <code>leCreateBusMode</code>
Select By	Area-selection method: • Rectangle: The selection area is rectangular. • Polyline: The selection area is polygonal. Tcl preference: <code>leCreateBusSelectBy</code>
Length Matching	Keeps the excess path lengths after adding the bus extension. Tcl preference: <code>leBusExtendLengthMatching</code>
Mesh	Creates mesh-pattern connections when extending bus bits. See Extending Existing Bus Bits. Tcl preference: <code>leBusExtendMeshMode</code>
Merge Wire	Merges the wire shapes after adding the new bits to create continuous wire shapes. Tcl preference: <code>leBusExtendMergeWire</code>
Use Polygon	Adds polygon shapes instead of paths when adding the bits. Tcl preference: <code>leBusExtendUsePolygon</code>
Ending Edge	Specifies where the wire extension ends: • Path: At the edge of the overlapping path. • Via: At the edge of the via.
Swap Source and Target	Switches the source and target bus bits. (The source becomes the target, and the target becomes the source.)
Via Template 1	The via placement template to use in L-shaped extensions: •  Backslash: Cascades the vias to the right. •  Slash: Cascades the vias to the left. Tcl preference: <code>leBusExtendViaTemplate1</code>
Via Template 2	The via placement template to use in Z-shaped extensions: •  Backslash: Cascades the vias to the right. •  Slash: Cascades the vias to the left. Tcl preference: <code>leBusExtendViaTemplate2</code>

Middle Segment (dialog box only)

Option	Description
Align	Places the bus extension using one of the following methods: • Center: Aligns the center of the bus extension with the center of the selection area. • Specify: Positions the bus extension based on the position of the pointer. Tcl preference: <code>leBusExtendMiddleAlign</code>
Width	Width of the bits in the middle segment. Tcl preference: <code>leBusExtendWidth</code>
Spacing	Minimum spacing between the bits in the middle segment. Tcl preference: <code>leBusExtendSpacing</code>
LPP	Specifies the LPP for the bus extension. • Auto: Uses the lowest-numbered LPP between the source and target bit layers. • Target Layer: Uses the target bits' LPP. • Specify: Uses the LPP that you specify. Select the LPP you want to use. Tcl preference: <code>leBusExtendLPPMode</code>

Create Bus Shortcut Menu and Shortcut Keys

While the **Create > Bus** command is active, you can right-click to display the Create Bus shortcut menu. This section provides a quick reference of the shortcut menu and shortcut keys. For instructions on how to create bus trunks, see [Creating Bus Routes](#).

During the **Create Bus** operation, you can right-click to display the Create Bus menu. The shortcut menu that appears depends whether you are in the [Manual Mode](#) or the [Extend Mode](#).

Manual Mode

Commit	Num+Enter
Add Point	Space
Remove Point	Backspace
Angle	▶
Adjust Width	▶
Auto	▶
☐ Gather	
Cycle Leader	▶
Via Justification	▶
Reference	▶
Via Template	▶
▲ Layer Up	Shift+V
▼ Layer Down	Ctrl+V
Add Via ...	V

Extend Mode

Commit	Num+Enter
Toggle Via Template1	
Toggle Via Template2	
☒ Select Target	

Manual Mode

Command	Purpose	Default Shortcut Key
Commit	Completes the bus.	Enter
Add Point	Inserts a point.	Space
Remove Point	Deletes the previously entered point.	Backspace
Angle	Controls the drawing angle mode by Orthogonal , Diagonal , X First , or Y First .	Right-click to switch between the X First or Y First angle mode only if one of these modes is active.
Adjust Width	Adjusts the width of the bus bit: • Off: Does not automatically adjust the bus bit width. • Narrow: Adjusts the width to the narrow dimension of the tapped rectangular object. • Wide: Adjusts the width to the wide dimension of the tapped rectangular object. • Taper: Adjusts width to the dimension of the rectangular object facing the bus.	

Command	Purpose	Default Shortcut Key								
Auto	• Terminate: Defines if interconnect automatically commits. • Tap: Defines if interconnect runs an implicit tap at the first add point. • Insert Jog: Defines if jog can be inserted when Adjust Width is not set to Off. • Drop Via: Adds vias and connects to layers or pins above or below the routing layer. The connecting layers or pins must be on the same net as the routing layer. To prevent extra vias from being created during editing operations such as Copy or Move, turn off Vias and End Styles: Auto Update, or turn on Vias and End Styles: Auto Update and select End Styles. See the Command Tab in the Layout Design Options dialog box.									
Gather	Uses <code>minSpacing</code> to set the distance between two adjacent bits.									
Cycle Leader	Selects the bit to use as the leading bit. The leading bit is the bit that the Layout Editor uses to create the bus. When you cycle forward or backward, you are not changing the order of the bits. In the following example, the default leading bit is the left-most bit. You can cycle forward to select the second or third bit. You can also cycle backward if you want to return to the previous bit. <table><tr><td>Starting</td><td>Forward 1</td><td>Forward 2</td><td>Backward</td></tr><tr><td></td><td></td><td></td><td></td></tr></table>	Starting	Forward 1	Forward 2	Backward					Shift+2, Ctrl+2
Starting	Forward 1	Forward 2	Backward							
										
Via Justification	Specifies the position of the via relative to the metal paths above and below it: Upper Left, Center Left, Lower Left, Upper Center, Center Center, Lower Center, Upper Right, Center Right, or Lower Right. The default is Center Center. See Via Justification. This option works with the Reference option and only applies for cases with one via between adjacent metal layers; it does not affect via stacks. Tcl preference: <code>leViaJustification</code>									

Command	Purpose	Default Shortcut Key
Reference	Specifies whether Via Justification is based on the Cut Center (default) or the Metal Edge. See Positioning the Vias. This option works with the Via Justification option and only applies for cases with one via between adjacent metal layers; it does not affect via stacks. Tcl preference: <code>leViaJustificationMaterial</code>	
Via Template	The pattern to use for via placement. Choose a pattern:  Straight: Aligns the vias in a straight vertical line.  Backslash: Cascades the vias to the right.  Slash: Cascades the vias to the left.  Right Arrow: Arranges the vias in a horizontal V shape, where the bottom of the V points to the right.  Left Arrow: Arranges the vias in a horizontal V shape, where the bottom of the V points to the left.  Non Cross: Automatically arranges the vias and prevents physical wires on upper and lower layers from overlapping.	
Layer Up	Inserts a single via at a layer above the current layer. For example, if you want insert a via two layers above the current layer, press Shift+V two times. This option is not available when you are connecting to pins on different layer-purpose pairs (LPPs).	Shift+V
Layer Down	Inserts a single via at a layer below the current layer. For example, if you want to add a via three layers below the current layer, press Ctrl+V three times. This option is not available when you are connecting to pins on different layer-purpose pairs (LPPs).	Ctrl+V
Add Via	Opens the Add Via dialog box to place a via or via stack (For more information about adding a via stack, see Adding Vias During Bus Creation).	V

Extend Mode

Command	Purpose	Default Shortcut Key
Commit	Completes the bit extension operation.	Num+Enter
Toggle Via Template 1	Changes the via placement between Slash  and Backslash  in L-shaped extensions or the first bend of Z-shaped extensions.	
Toggle Via Template 2	Changes the via placement between Slash  and Backslash  in the second bend of Z-shaped extensions.	
Select Target	Adds selected objects to target (on) or source (off) during the extension operation.	

Creating Bus Context-Sensitive Toolbar

The context-sensitive toolbar is displayed when the `ile::createBus` command is activated followed by right-clicking.



Table 4 *ile::createBus options*

Options	Description
Width	Defines width of the current path or pathSeg object. Preference: <code>leWidth</code> Note: The width text input is automatically in focus after the CST is displayed.
Nets	Defines net name(s) to be associated with the routing objects that are to be created. Preference: <code>leNetName</code>
Sync button	Synchronizes the net name(s) selected in Design Navigator. Preference: <code>leDNButton</code>

Table 4 *ile::createBus options (Continued)*

Options	Description
Cycle	Defines if first net name in the net name field is removed or remains after object is committed. Preference: <code>leCycleName</code>
Use All Layers	Enables all layers for interconnect creation. Preference: <code>leUseAllLayers</code>
Bits	Defines number of bits. <code>leBusBits</code>
Spacing	<code>leBusUseSkipTracks = false</code> Defines global spacing between bits of bus. Preference: <code>leBusSpacing</code> Note: - If technology file does not have a <code>minSpacing</code> value defined, the default spacing uses the value set in the <code>leDefaultBusSpacing</code> preference. - The centerline-to-centerline pitch of the bus bits is defined as: <code>bus bit width + bus spacing</code> <code>leBusUseSkipTracks = true</code> Defines number of tracks skipped when calculate bus spacing. Preference: <code>leBusSkipTracks</code>
PathSeg	Enables pathSeg object for interconnect creation. Preference: <code>leUsePathSeg</code>
Drop Via	Supports auto drop via in ON/OFF state. Preference: <code>leInterconnectDropVia</code>
Color	Defines the color name and the color enumerator to be associated with interconnect paths / pathSegs. Preference: <code>leColorName</code> Note: - This widget is enabled only when the selected layer has more than one color. In addition to tech-defined colors, “auto”, “none” or “same” can also be shown in the option list. “None” and “same” are controlled by preferences <code>leIncludeColorNone</code> and <code>leIncludeColorSame</code> respectively. “Auto” is always shown. It means LE commands determine the color automatically by tap or colored track. If both cannot determine color, tech defined default color is used. The only exception is, if no default color is defined and “none” is prohibited by preference, command uses the first legal color. - This option is invisible if the current layer has no color.

Table 4 *ile::createBus options (Continued)*

Options	Description
Lock Color	Defines if interconnect color is locked. Note: • If this option is set to true, the color on interconnect is locked. • If the current layer has no color, this option is invisible.
Stack commands tool button	The Set Cursor action is disabled when you have not specified the first point.

Extending Existing Bus Routes

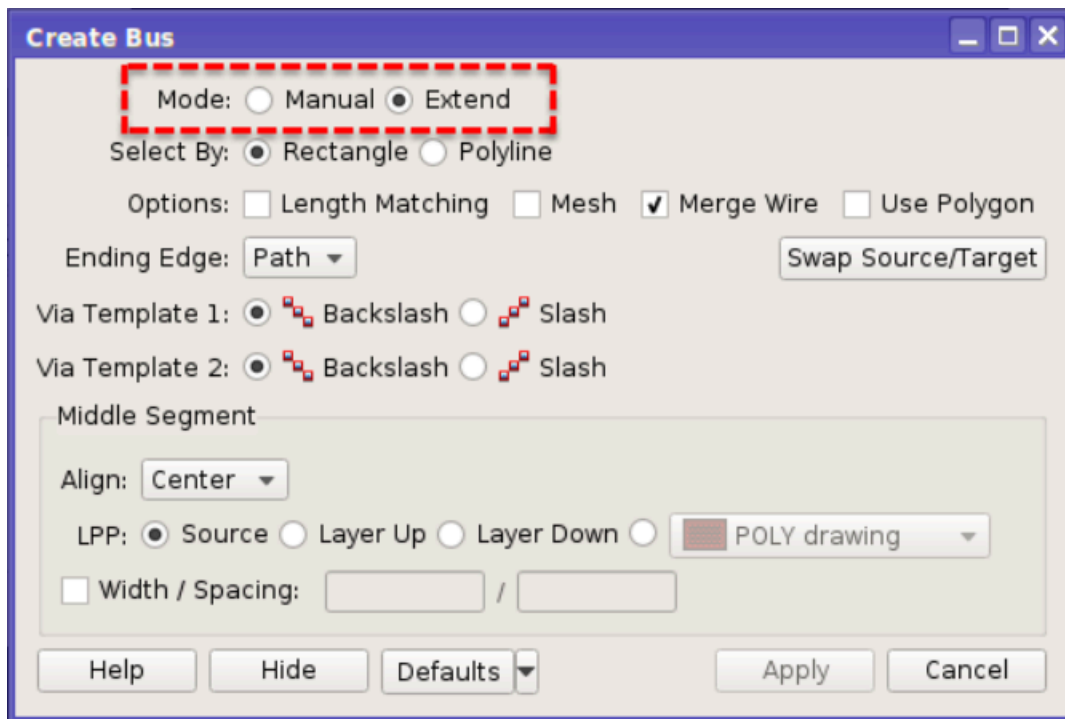
This topic describes the following:

- [About the Create Bus Extend Mode](#)
- [Extending Existing Bus Bits](#)

See also [Creating Bus Routes](#).

About the Create Bus Extend Mode

The **Create Bus** command has an **Extend** mode for extending the ends of two existing bus routes and connecting them.



Bus extensions can only occur when the following requirements are met:

- The bus bits to be extended are supported objects: path, path segments, rectangles, and polygons.

If both path and path segments are within the same selection for the operation, the command will check the `leUsePathSeg` Tcl preference setting.

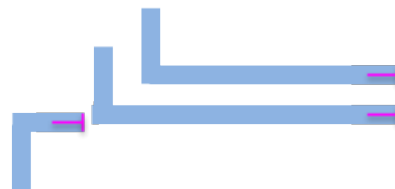
If a connection to a path segment will result in a non-supported angle, paths will be used instead.

- The selected ends within the source group or the target group do not overlap.

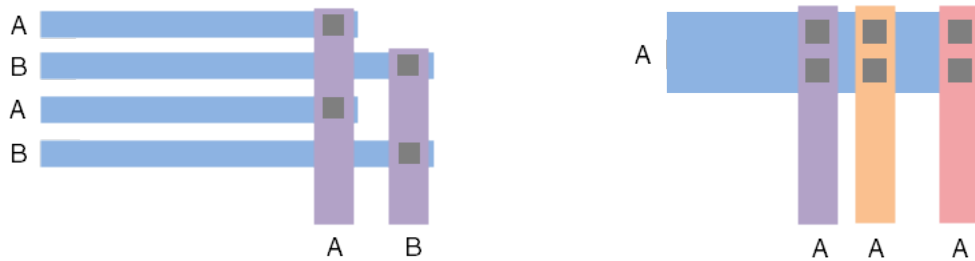
No Overlapping



Overlapping

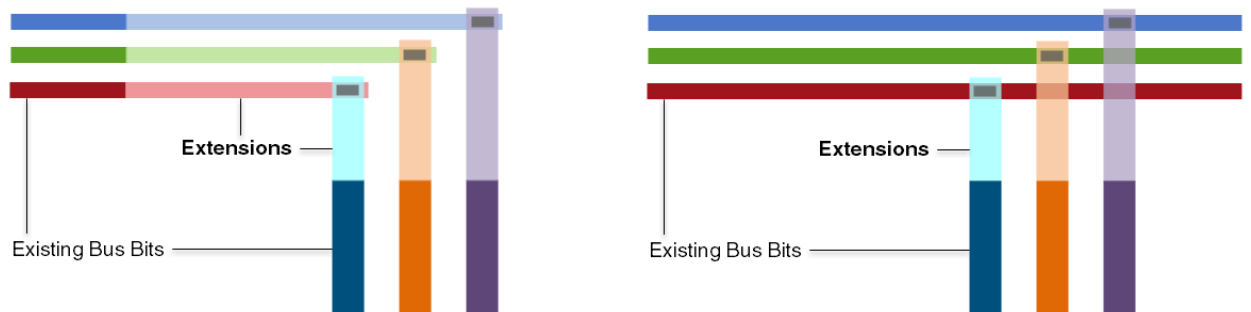


The two bus routes do not have to have the same number of bits. The widths of bus bits and the spacing between them do not have to be identical. The extension bits can be on different layers, and they do not all necessarily have to have net assignments.

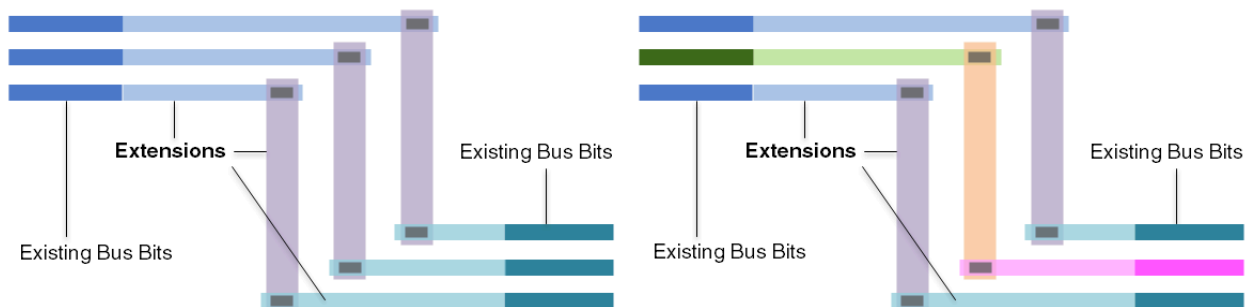


Bus extensions are added as follows:

- If the two bus routes are perpendicular to each other, an L-shaped or T-shaped extension is created.



- If the two bus routes are parallel but offset from each other, a Z-shaped extension is created.

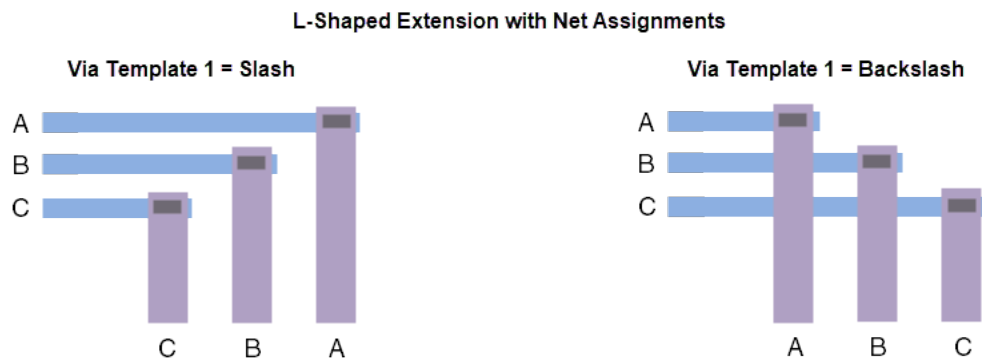


- If the extensions must traverse different layers, vias are automatically added between the layers.

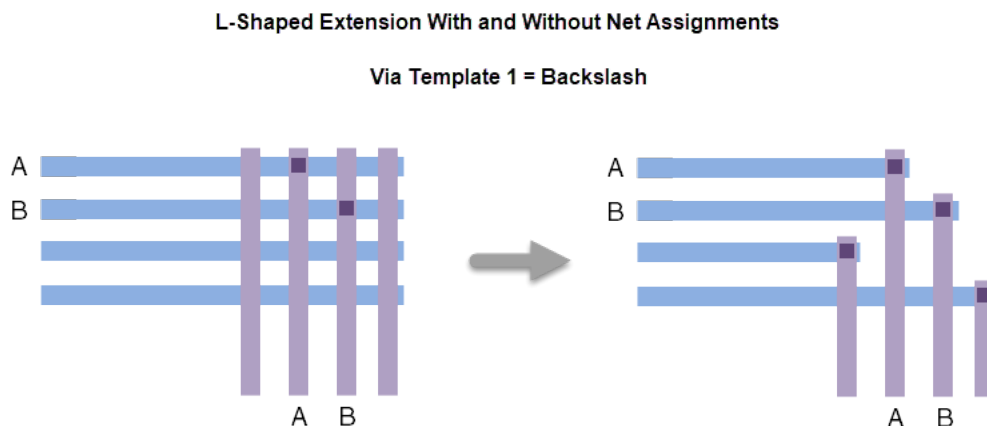
The bus extension results depend on the order in which the bits are connected and the **Via Template** setting. When the bus extension consist of both netted and unnetted bits,

the command always connects bits with net assignments first before determining the connections of unnetted bits.

In the following example, the L-shaped extension consist of wires that have net assignments, and the number of source and target bits are the same. The command uses the **Via Template 1** setting (Slash or Backslash) to generate the result shown.



In the next example, some of the L-shaped extension wires do not have net assignments. The command first uses the net information and the **Via Template 1** setting to connect the netted wires. The command then uses the **Via Template 1** setting to complete the remaining connections. In this example, **Via Template 1** is **Backslash**.



When Z-shaped extensions consist of netted and unnetted wires, the command processes one bend at a time. Within each bend, the command first processes netted wires before processing unnetted wires. **Via Template 1** is applied to the first bend, **Via Template 2** is applied to the second bend.

The command also uses wire-indexing to determine the results:

- Horizontal index assignment always starts from the bottom-most wire.
- Vertical index assignments are based on the Via Template setting: Slash assigns indexes from the left-most wire, Backslash assigns indexes from the right-most wire.

The following example demonstrates how the command generates the results for a Z-shaped extension. In the example, **Via Template 1** is **Slash**, and **Via Template 2** is **Backslash**.

- Within the first bend, horizontal paths are indexed from the bottom-most wire. Because **Via Template 1** is **Slash**, vertical wires are indexed from the left.

The two horizontal wires that are on net A are indexed 2 and 4. Vias can only be placed at the index-2 and index-4 intersections. Notice that the resulting via pattern is Slash.

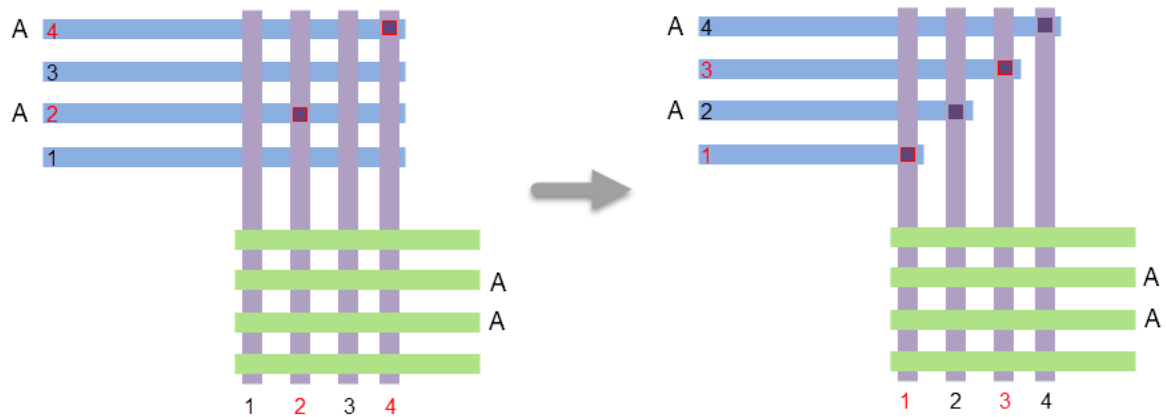
Next, for the unnetted wires, vias are added at the intersections of the two index-1 and index-3 wires. The resulting via pattern is Slash.

- Within the second bend, the netted wires are indexed first. Horizontal paths are indexed from the bottom-most wire. Because **Via Template 1** is **Backslash**, vertical wires are indexed from the right. Vias are placed at the index-1 and index-2 intersections. The resulting via pattern is backslash.

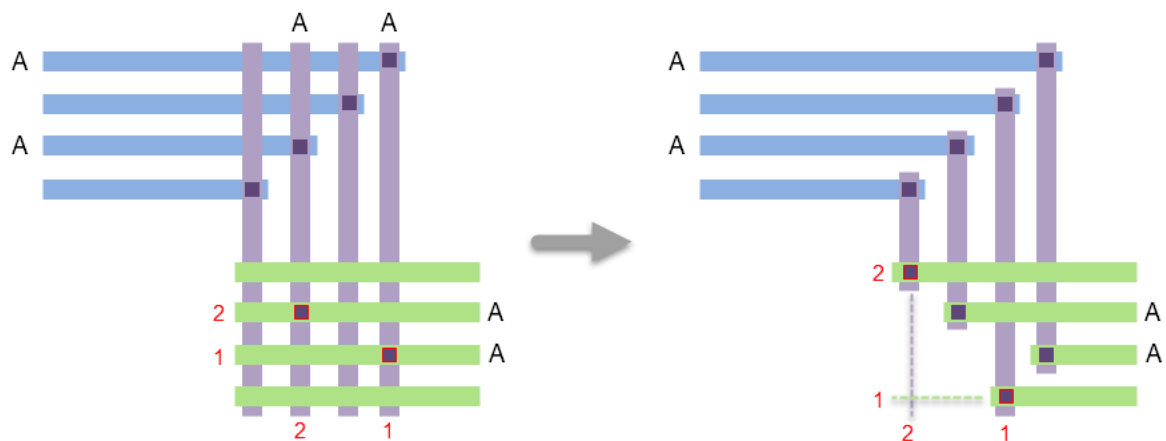
Next, the unnetted wires are indexed. Horizontal paths are indexed from the bottom-most wire. Because **Via Template 1** is **Backslash**, vertical wires are indexed from the right. Vias are placed at the index-1 and index-2 intersections. The resulting via pattern is backslash.

Z-Shaped Extension With and Without Net Assignments

Via Template 1 = Slash



Via Template 2 = Backslash

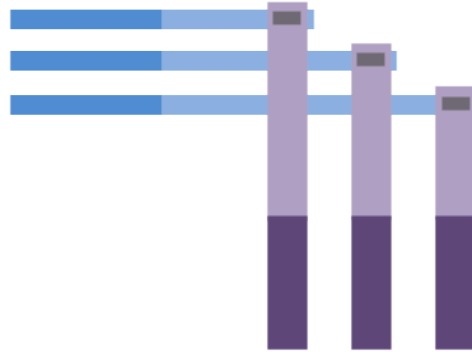


When the extending wires must traverse different layers, they will always bend orthogonally. However, if the extending wires are on the same layer, non-orthogonal bends are possible.

Same Layer, Any Angle



Different Layers, Orthogonal Only



Extending Existing Bus Bits

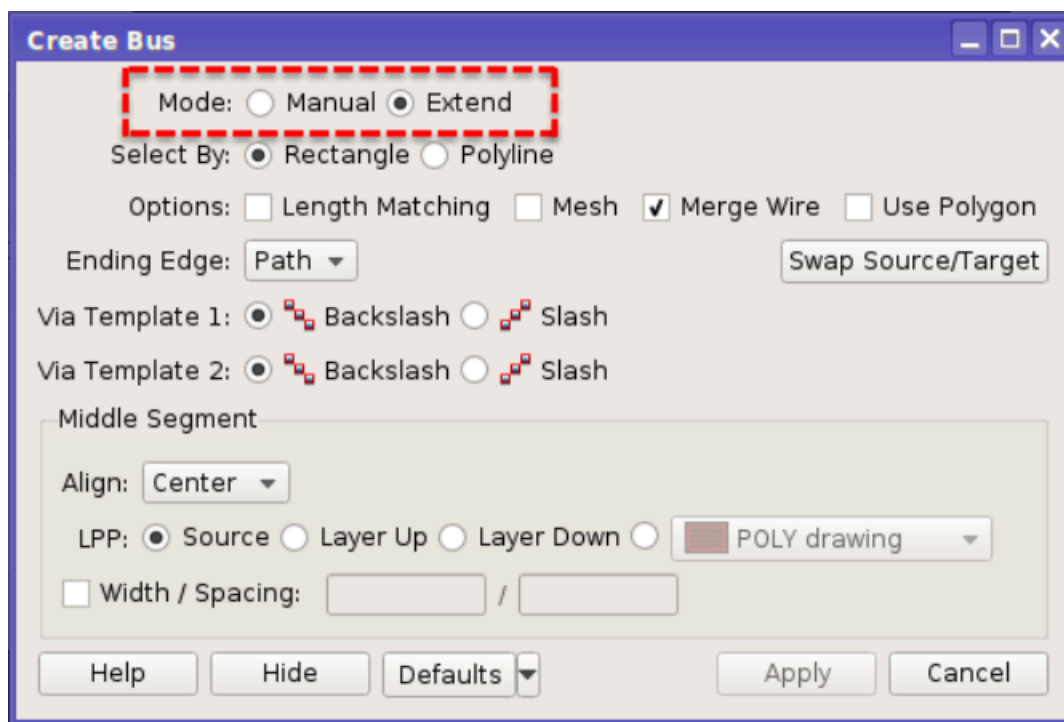
You can extend the ends of existing bus bits and connecting them.

This section describes how to extend existing bus bits. For a quick reference of the **Create Bus Extend Mode** toolbar and dialog box, see [Bus Toolbar and Create Bus Dialog Box Reference](#).

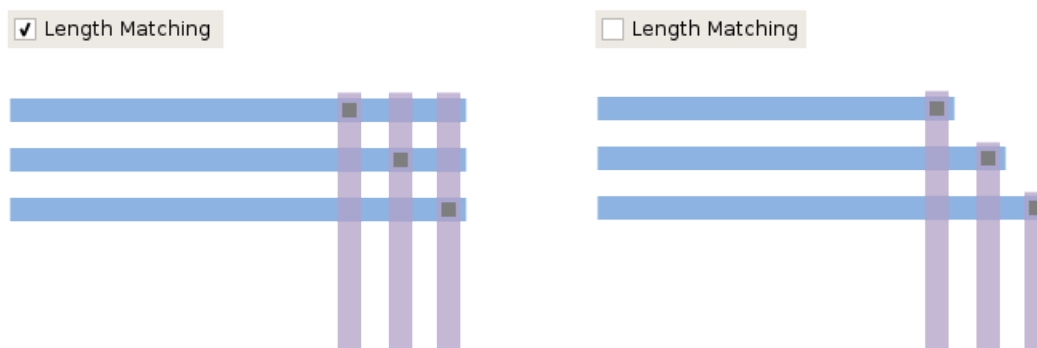
 [Show me](#) how to create and extend bus routes.

To extend existing bus bits to connect them:

1. In the Layout Editor, choose **Create > Bus**.
2. On the **Bus** toolbar or in the **Create Bus** dialog box, select **Mode: Extend**.

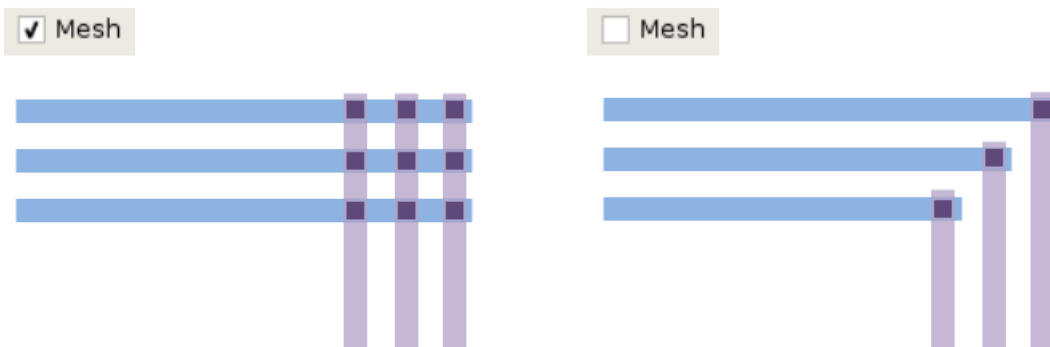


3. Specify the area-selection method to use when selecting the bus bits you want to extend:
 - **Rectangle:** Draws a rectangle during the area-selection process.
 - **Polyline:** Lets you use polylines to create a selection area.
4. (Optional) Select **Length Matching** if you want to keep excess path sections to ensure wire length matching. Clear the check box to remove the excess path sections (for example, to balance parasitic capacitance).



5. Select **Mesh** if you want to create a mesh pattern when extending the bus bits.

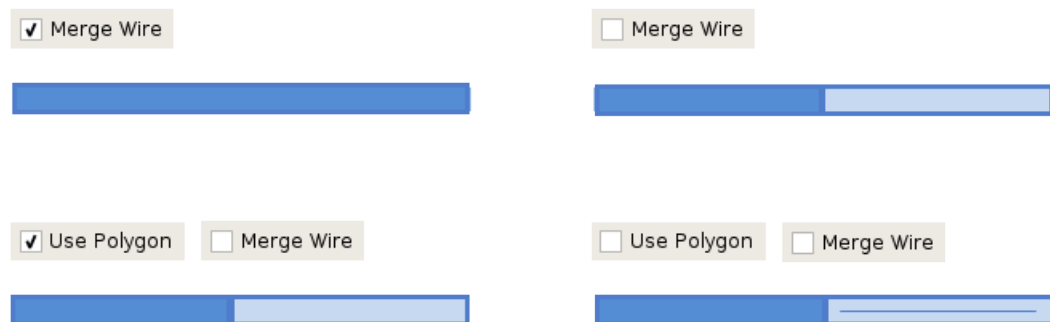
To create the mesh pattern, the bus bits you are connecting must be on the same net (or both have no net assignments). The connecting bits can be on the same layer or on different layers.



6. (Optional) Select one or more of the following:

- **Merge Wire:** Merges each extension with the existing wire so that they form one wire shape. To keep the extensions and the existing bits are distinct shapes, clear the check box.
- **Use Polygon:** Creates polygons to extend the existing bus bits. To create paths, clear the check box.

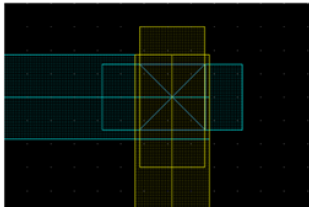
See [About the Create Bus Extend Mode](#).



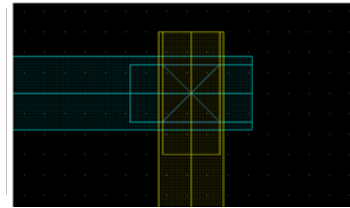
7. Select the **Ending Edge**:

- **Path:** Ends the wire extension at the edge of the overlapping path.
- **Via:** Ends the wire extension at the edge of the via.

Path



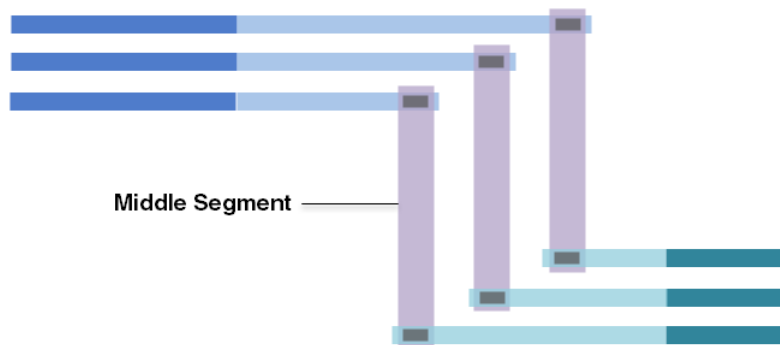
Via

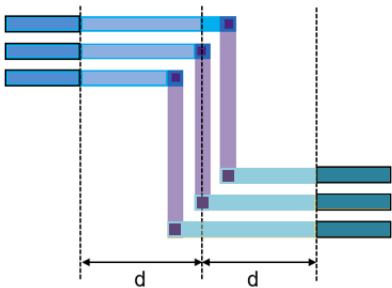
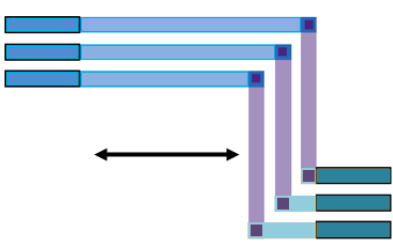


8. To switch the source and target bus bits, click **Swap Source/Target**. The source becomes the target, and the target becomes the source.
9. Select the **Via Template** you want to use for bus extensions.
 - **Via Template 1**: The via template to use in L-shaped extensions and the first bend in Z-shaped extensions.
 - **Via Template 2**: The via template to use in the second bend in Z-shaped extensions.

See [About the Create Bus Extend Mode](#) for different types of extensions. See [Adding Vias During Bus Creation](#) for the via template descriptions.

10. (Z-shaped extensions and dialog box only) In the **Middle Segment** section, define the following for the middle segment of Z-shaped extensions:

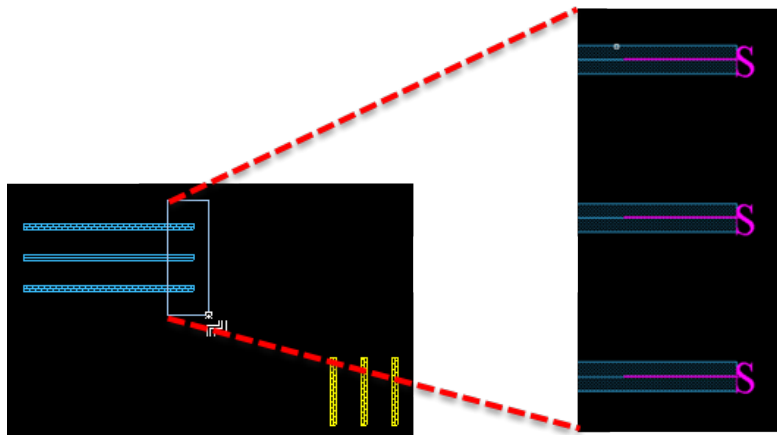


Option	Description
Align	Placement of the middle segment: ◦ Center: (Default) Aligns the center of the middle segment with the center of the selection area. ◦ Specify: Positions the bus extension based on the position of the pointer. See Step 11c. Center  Specify 
LPP	Specifies the layer-purpose pair (LPP) on which the middle segment is placed. ◦ Source: Uses the LPP of the source bits (the first bits you select). See Step 11a. ◦ Layer Up: Uses the LPP above the source bits. ◦ Layer Down: Uses the LPP below the source bits. ◦ <LPP>: Uses the LPP that you specify. Select the LPP you want to use.
Width	Width of the bits in the middle segment. If the value you specify violates the default, a value closest to the default will be used. Default: Minimum width defined by the design constraint Tcl preference: <code>leBusExtendWidth</code>
Spacing	Minimum spacing between the bits in the middle segment. If the value you specify violates the default, a value closest to the default will be used. Default: Minimum spacing defined by the design constraint Tcl preference: <code>leBusExtendSpacing</code>

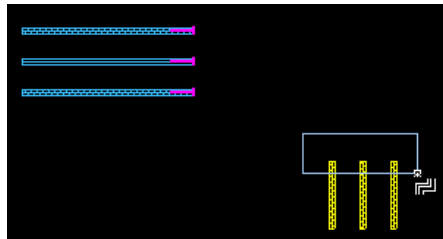
11. On the canvas:

- a. Area-select the ends of the first bus route. To exclude individual bits from the selection, Ctrl+click the bits.

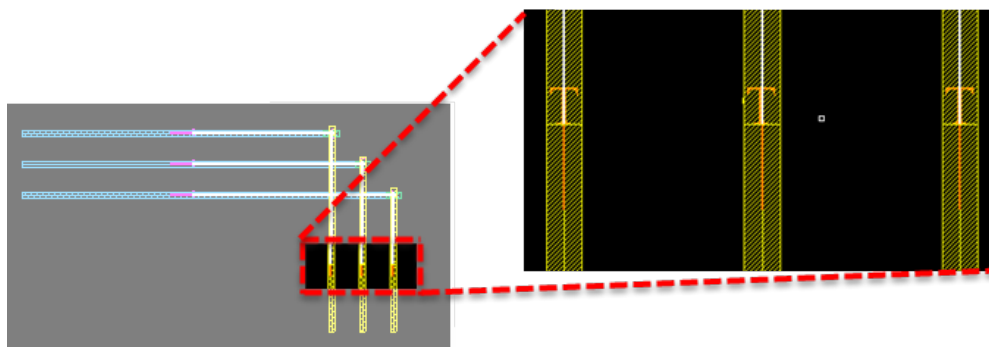
Short highlighting lines appear at the wire-ends, indicating they are selected. In addition, the letter S appears at each wire-end to indicate they are the source bits.



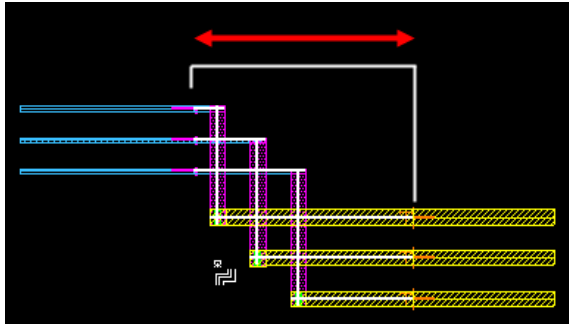
- b. Area-select the ends of the second bus route. To exclude individual bits from the selection, Ctrl+click the bits.



The command automatically adds the wire extensions between the source and target bits. Short highlighting lines appear at the target wire-ends with the letter T to indicate they are the target bits.

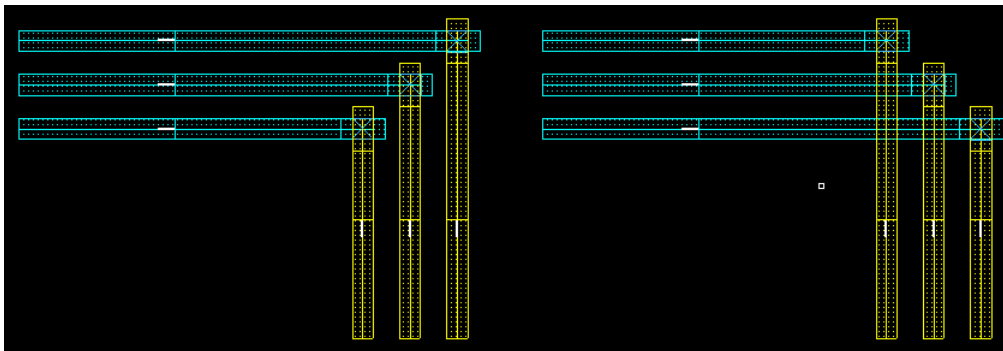


- c. (Z-shaped extensions only) If you chose **Align: Specify**, on the layout canvas, move the middle segment to the desired position.



If you chose **Align: Center**, the middle segment is placed at the midpoint between the selected bits.

12. (Bits with no net assignments) To choose a different result, right-click anywhere on the canvas and choose **Toggle Via Template 1** (L-shaped extensions, or the first bend in Z-shaped extensions) or **Toggle Via Template 2** (second bend in Z-shaped extensions only). The via positions change between  and .



13. When you are finished, right-click anywhere on the canvas and choose **Commit**. Alternatively, press Enter.

Adding Vias During Bus Creation

To add vias while creating bus routes:

1. On the canvas, at the location where you want to add the via, right-click and choose **Add Vias** in the shortcut menu. The **Add Via** dialog box opens.

Layer	Via Definition	Cut Class	Spacing	Layer1 Enc	Layer2 Enc	Metal Color	Cut Color
POLY							
M0POLY	POLY	auto	auto	auto	auto	Not Colorable	Not Colorable
MODIFF1	M0POLY	auto	auto	auto	auto	Not Colorable	Not Colorable
MODIFF2	auto	auto	auto	auto	auto	Not Colorable	Not Colorable
M1	auto	auto	auto	auto	auto	auto	Not Colorable
M2	auto	auto	auto	auto	auto	auto	auto
M3	auto	auto	auto	auto	auto	auto	auto

If you do not see all the columns in the table, right-click on any column heading and select the columns you want to show.

2. In the **Via Setting** list, choose one of the following:

Argument	Description
Local	The Via Definition and Cut Class settings you specify in this dialog box apply only to the Create Interconnect > Add Via command.

Argument	Description
As Auto Via	The Via Definition and Cut Class settings you specify in this dialog box are synchronized with the Create Via (Auto mode) command.

3. Set the following options in the dialog box.

Option	Description																				
Filter boxes (above the Layer table)	Filters the list of vias and cut classes based on the criteria you provide. The filters you specify are remembered within the same design session. By default, the predefined auto filter is used. However, you can change the criteria by typing one or more strings in the box. Use a character space to separate the strings. For example: bar diff po After you type the filter strings, press Enter. The list of names that meet the criteria appear in the column. An ellipse (...) in a result indicates that multiple definitions match the filter criteria. To view the definitions, rest the pointer on the result. bar auto X <table><tr><th>☒</th><th>Layer</th><th>Via Definition</th><th>Cut Class</th></tr><tr><td>☒</td><td>M4</td><td>M4_M5xHBAR</td><td>auto</td></tr><tr><td>☒</td><td>M5</td><td>M5_M6xHBAR ...</td><td>auto</td></tr><tr><td>☒</td><td>M6</td><td>M6_M7xHBAR M5_M6xHBAR M5_M6xVBAR</td><td></td></tr><tr><td>☒</td><td>M7</td><td>M7_M8xHBAR ...</td><td>auto</td></tr></table> The filter results depends on the Filter setting you selected in the General Options dialog box.	☒	Layer	Via Definition	Cut Class	☒	M4	M4_M5xHBAR	auto	☒	M5	M5_M6xHBAR ...	auto	☒	M6	M6_M7xHBAR M5_M6xHBAR M5_M6xVBAR		☒	M7	M7_M8xHBAR ...	auto
☒	Layer	Via Definition	Cut Class																		
☒	M4	M4_M5xHBAR	auto																		
☒	M5	M5_M6xHBAR ...	auto																		
☒	M6	M6_M7xHBAR M5_M6xHBAR M5_M6xVBAR																			
☒	M7	M7_M8xHBAR ...	auto																		
Layer	(Dialog box only) Displays only the bottom layer. You can specify whether to enabled each bottom layer. By default, all the bottom layers are enabled for via creation. If Stacked mode is enabled, the table only shows the layers between bottom and top layers.																				
Via Definition	(Dialog box only) Lists via, via variant, and bridge via definitions. A via variant name contains the name of the via definition on which it is based. The definitions are sorted in alphabetical order. The first name in the list is the default via. This list might include oaViaDefArray objects. Selecting a predefined oaViaDefArray from the list is the only way to place an array of vias (a regular mosaic). You can select the available definitions by clicking on the field. The list includes an <i>auto</i> definition when Auto mode is enabled.																				

Option	Description
Cut Class	(Dialog box only) Lists cut shapes defined in your technology database. You can select the available cut classes by clicking on the field. The list includes an <i>auto</i> cut class when Auto mode is enabled. When Manual non-stacked mode is enabled, you can modify the parameters of the selected via in the Parameters table. When Manual stacked mode is enabled, you can modify the spacing and the enclosure values for Layer1 and Layers by clicking on the field or by entering the desired values.
Spacing	Spacing between cuts in standard and custom built-in vias. The spacing applies to neighboring cut shapes that have positive parallel run lengths. You can specify one value, which applies to both the X and Y spacing directions. Alternatively, you can specify two values, the first value applies to the X direction and the second value applies to the Y direction. The spacing values you provide must be larger than the design-rule values. If the spacing values you provide create violations, the Layout Editor will automatically correct them. For all standard vias, if the <code>leViaHonorViaDefDefaultSpacing</code> Tcl preferences is <code>true</code>, the Layout Editor will automatically correct spacing values that are smaller than the defined cut spacing value. If a via variant is placed, the Layout Editor will ensure the placed via variant complies with the cut spacing value you specify and the via variant cut spacing value.
Layer <i>n</i> Enc	Via enclosure specifications. You can specify one, two, or four values: - One value specifies that the same distance is applied on all four sides of the enclosure. - When you specify two values, the first value is applied to the left and right sides of the enclosure, the second value applies to the top and bottom sides of the enclosure. - When you specify four values, the values are applied to the left, right, top, and bottom sides in the order encountered.

4. Choose the **Template**, or the via placement pattern:

Setting	Description
Template	 Straight: Aligns the vias in a straight vertical line.  Backslash: Cascades the vias to the right.  Slash: Cascades the vias to the left.  Right Arrow: Arranges the vias in a horizontal V shape, where the bottom of the V points to the right.  Left Arrow: Arranges the vias in a horizontal V shape, where the bottom of the V points to the left.  Non-Cross: Arranges the vias so that the interconnects do not overlap or cross one another.

5. (Optional) Select **Define Via Size Manually**, and select the method to use to determine the via size:

Cols/Rows	Uses the number of cut columns and rows that you specify to determine the via size (width and length).
Width/Height	Uses the width and length values that you specify to calculate the number of cuts that can fit within the area.

6. When you are finished, click **OK** to close the dialog box. You can continue creating bus trunks.

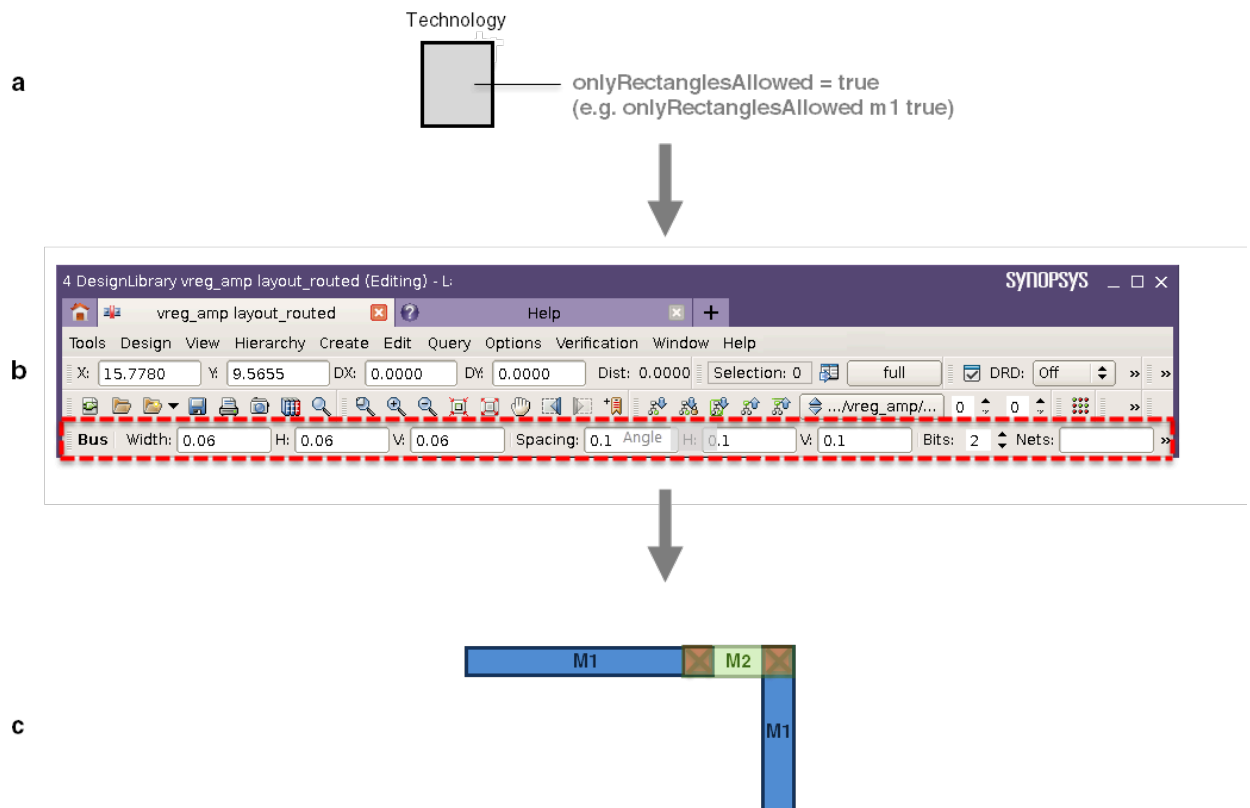
Connecting Bus Bits on the Same Layer

If the `onlyRectanglesAllowed` layer constraint in the technology is set to *true* for specified layers ([Figure 5a](#)):

- The **Angle** option is disabled on the **Interconnect** toolbar ([Figure 5b](#)).
- The bus bits that you create on a specified layer can only be placed horizontally or vertically. The bus bits cannot turn and change direction while on the same layer.
- To connect orthogonal shapes on the same layer, you must add vias and connecting shapes on layers above or below the orthogonal shapes ([Figure 5c](#)).

- When adding vias, you can choose only the  Slash and  Backslash via templates to turn and change direction. For information about the available **Via Justification** options, see [Interconnect Toolbar and Create Interconnect Dialog Box](#).
- If the connecting trunks do not change directions, the bit pitch of the second trunk remains the same as the bit pitch in the first trunk. However, if the bus changes layers, and the first layer's bit pitch is smaller than permitted by the second layer's spacing rules, the Layout Editor will not permit you to add vias with the  Straight via template and connect to the second layer.

Figure 5 Connecting Orthogonal Bits on the Same Layer



For information about the `onlyRectanglesAllowed` layer constraint, see the *Custom Compiler Technology and Display Resource Reference Manual*.

Continuing Bus Routes

This section covers the following topics:

- [About Continuing Bus Routes](#)
- [Using the Continue Bus Command](#)
- [Continue Bus Toolbar and Dialog Box](#)
- [Continue Bus Shortcut Menu](#)

For information about creating buses, see [Creating Bus Routes](#).

About Continuing Bus Routes

You can use the **Continue Bus** command to extend (or continue) existing bus routes. You can continue a subset of the bits in the trunk, or you can continue the entire trunk. The continuation bits inherit the properties and settings of the existing bits, such as width and spacing.

The Continue Bus operation lets you extend the bits from the start or stop end of the bus. When you continue bus bits, you can add points, change layers, add vias, or change directions just as you would if you were creating the bus route.

Using the Continue Bus Command

To extend an existing bus routes:

1. Zoom in the design and make sure the end points of the bus bits are visible and accessible in the viewport.
2. Choose **Edit > Wire > Continue Bus**. The **Continue Bus** toolbar displays.



3. (Optional) On the toolbar, click Eject  or press F3 to display the **Continue Bus** dialog box. The dialog box contains additional options.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

4. (Optional) Set the options on the toolbar or in the dialog box. See [Continue Bus Toolbar and Dialog Box](#).
5. On the canvas, select the end points from which you want to continue the route. You can select the ends of one or more bits. See [Working With Objects](#).

6. Click anywhere on the canvas, and then move the mouse to extend the route.
7. To add points, layer up or down, or add vias, right-click and choose the command you want to use. See [Continue Bus Shortcut Menu](#).

To ensure the paths fully cover the via metal layers, the `leInterconnectExtendPathEndToViaEnd` Tcl preference to set to true by default. See [Before You Start](#).

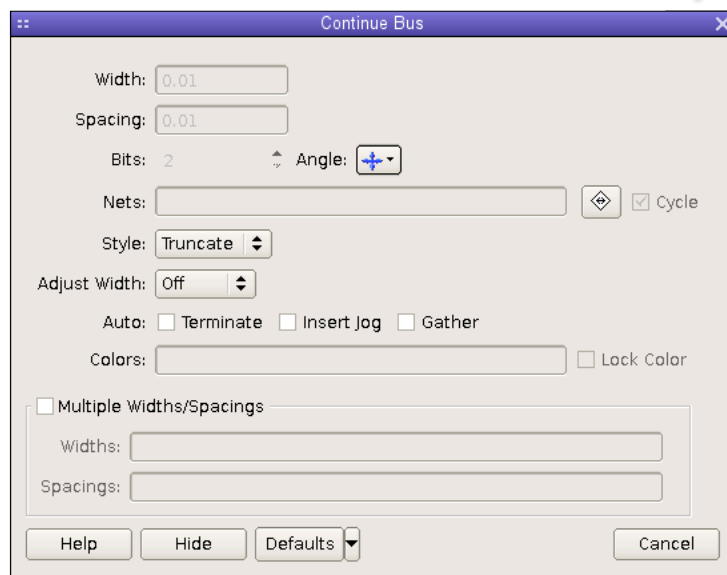
8. When you are finished, double-click to end the bus route.

Continue Bus Toolbar and Dialog Box

The **Continue Bus** toolbar and dialog box contain the same options as the **Create Bus** toolbar and dialog box. For the description of the options, see [Bus Toolbar and Create Bus Dialog Box Reference](#).

Notice that the following **Continue Bus** dialog box fields are pre-populated with values of the existing bus trunk, and you cannot change them:

- Width
- Spacing
- Bits
- Net
- Auto



Continue Bus Shortcut Menu

While the **Continue Bus** command is in operation, you can right-click and display the **Continue Bus** shortcut menu, which contains a subset of the commands in the **Create Bus** shortcut menu. For the description of the commands, see [Bus Toolbar and Create Bus Dialog Box Reference](#).

Commit	Num+Enter
Add Point	Space
Remove Point	Backspace
Angle	▶
Adjust Width	▶
Auto	▶
☐ Gather	
Cycle Leader	▶
Via Justification	▶
Reference	▶
Via Template	▶
▲ Layer Up	Shift+V
▼ Layer Down	Ctrl+V
Add Via ...	V

Compressing and Expanding Bus Routes

This section covers the following topics:

- [About Compressing and Expanding Bus Routes](#)
- [Compacting Bus Bits](#)
- [Using the Fan-In/Out Mode](#)
- [Adding Jogs in Bus Bits \(Ladder Mode\)](#)
- [Compress/Expand Bus Toolbar and Dialog Box Reference](#)

For information about creating buses, see [Creating Bus Routes](#).

About Compressing and Expanding Bus Routes

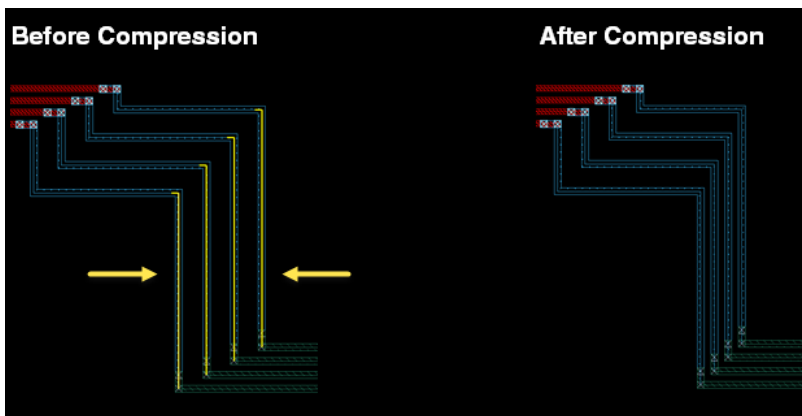
You can use the Layout Editor **Edit > Wire > Compress/Expand Bus** command to change the spacing between bus bits to comply with spacing requirements.

This topic describes the available modes you can specify when compressing or expanding bus routes:

- [Compact Mode](#)
- [Fan-In/Out Mode](#)
- [Ladder Mode](#)

Compact Mode

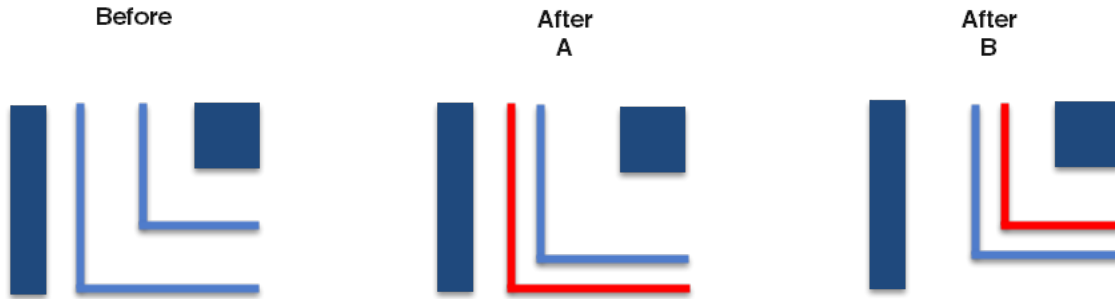
The **Compact** mode reduces the spacing between adjacent bits. Compaction is applied only to selected bits.



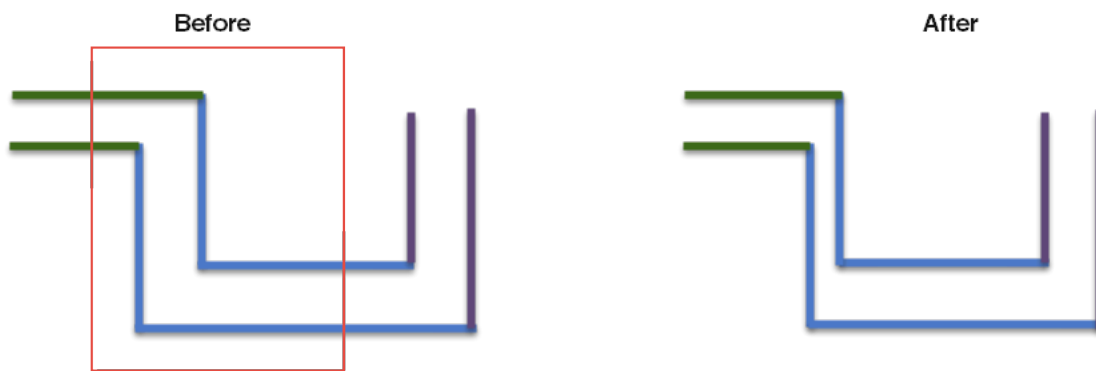
During the compacting operating:

- The spacing of the follower bits are updated to match the spacing requirements of the selected leader bit.
- The follower bits are moved closer to the selected leader bit.
- Only segments that are completely within the selection area are compacted.
- To retain pin position and connection, the Layout Editor might jog the bits near the pins.
- Bits that are connected to pins are not moved.
- Obstacles that are nearby the selected bits are ignored so that the compaction can proceed.
- The compaction is not completed if 0x obstacles exist on the same layer as and are between the selected bits.
- Bit connections can become non-well-formed after compaction.

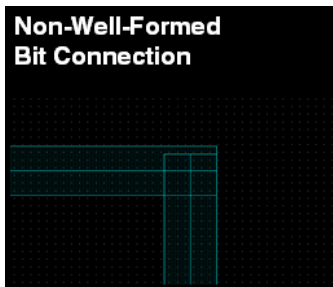
The following example show how compaction results are based on the selected red leader bit. In A, the blue follower bit is moved lower. In B, the follower bit is moved higher.



In the following example, only the blue vertical bits are completely enclosed in the red selection area. So, only the blue vertical segments are compacted.

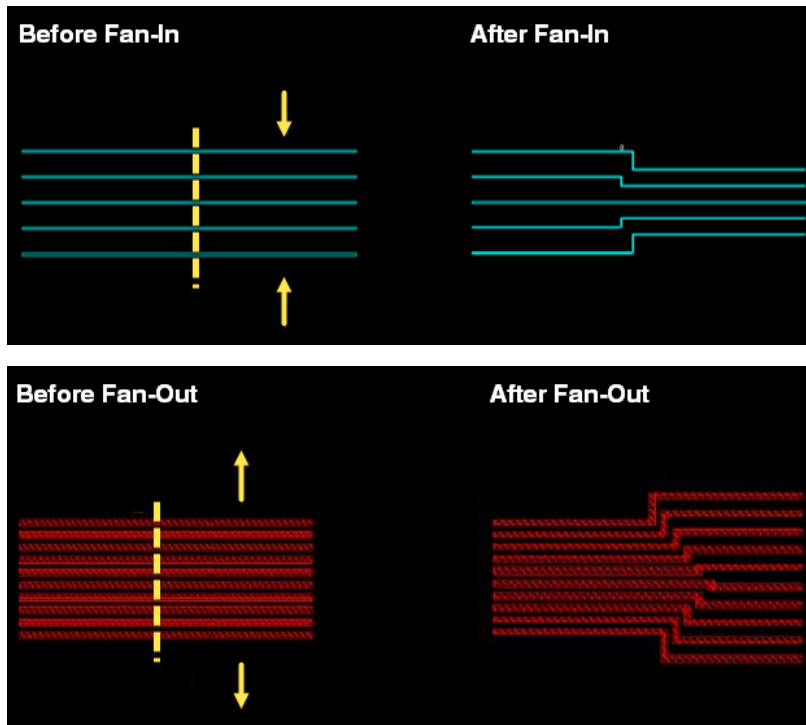


The following is an example of a bit connection that becomes not well-formed after compaction.



Fan-In/Out Mode

The **Fan-In/Out** mode can reduce or increase the spacing between bus bits at mid-path.

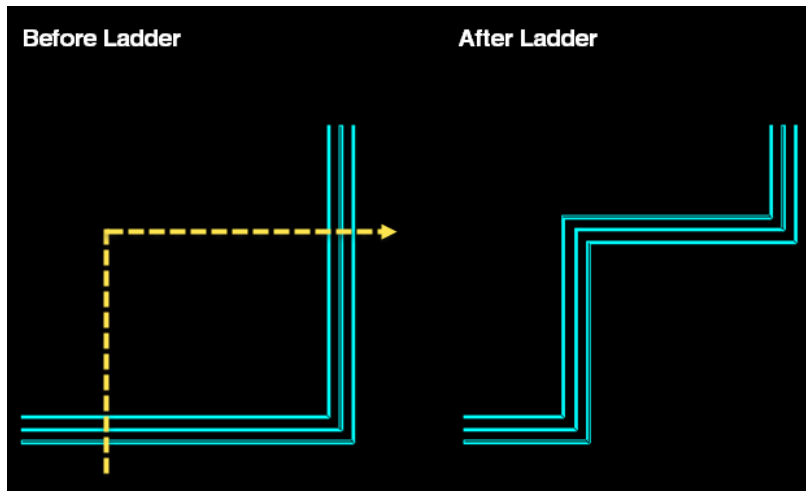


The fan-in and fan-out mode:

- Processes selected bus bits that have different widths, spacing, and routing layers.
- Uses the spacing value you specify. If you do not specify the spacing value, the command will use the default spacing defined in the technology database.
- Uses the routing layer that you specify. If you do not specify a layer, the command will use the layer of the first bit (the bit closest to where you clicked on the canvas to start the operation. See [Step 6](#) in [Compacting Bus Bits](#).

Ladder Mode

The **Ladder** mode adds one or more jogs in selected orthogonal bus bits. In the following example, one jog is added.

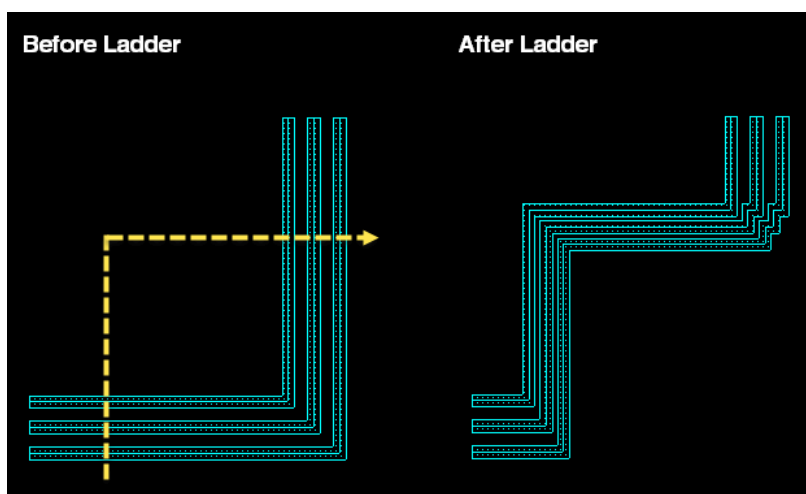


You can let the Layout Editor determine the maximum number of jogs, or you can specify the maximum number of jogs. See [Adding Jogs in Bus Bits \(Ladder Mode\)](#).

The maximum number of jogs that are added depends on the following:

- Compliance with spacing constraints determines the total number of jogs added.
- If the bit-to-bit spacing is uniform in the orthogonal bus route, one jog is added.
- When the bit spacing you specify for the jogs is different from the bit spacing in the original bus bits, additional jogs are added.

In the following example, the jog bit spacing ($0.3\text{ }\mu\text{m}$) is less than the bit spacing in the original bus route ($0.5\text{ }\mu\text{m}$). As a result, up to three jogs are added.

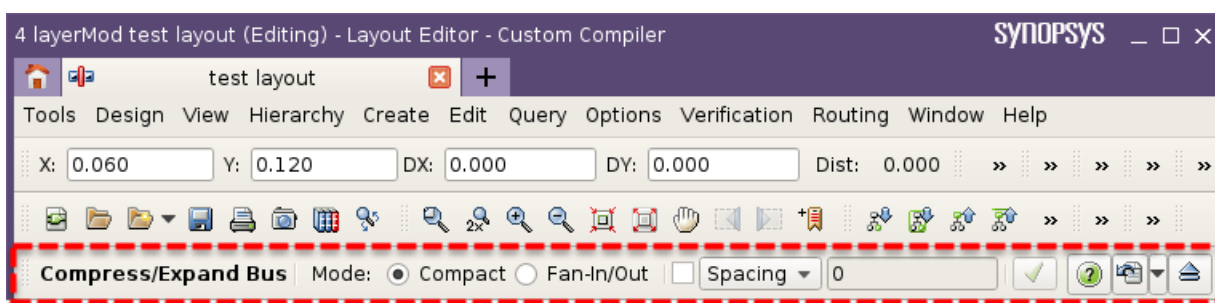


Compacting Bus Bits

The **Edit > Wire > Compress/Expand Bus** command has a **Compact** mode that lets you reduce the spacing between adjacent bits. Compaction is applied only to selected bits.

To compact bus bits:

1. Zoom in the design and make sure the bus bits are visible and accessible in the viewport.
2. Choose **Edit > Wire > Compress/Expand Bus**. The **Compress/Expand Bus** toolbar displays.



3. On the **Compress/Expand Bus** toolbar, select **Compact**.
4. (Optional) If you want to specify the compressed distance between the bits, select **Spacing** and type the distance value in the box.

If you do not specify the compressed distance, the `minSpacing` value from the technology database will be used.

5. Click **Spacing**, and choose how you want the compressed distance to be applied:



6. On the canvas, select at least two bus bits. See Working With Objects. By default, the lowest horizontal or left-most vertical bit is the leader bit, and all other bits are follower bits.

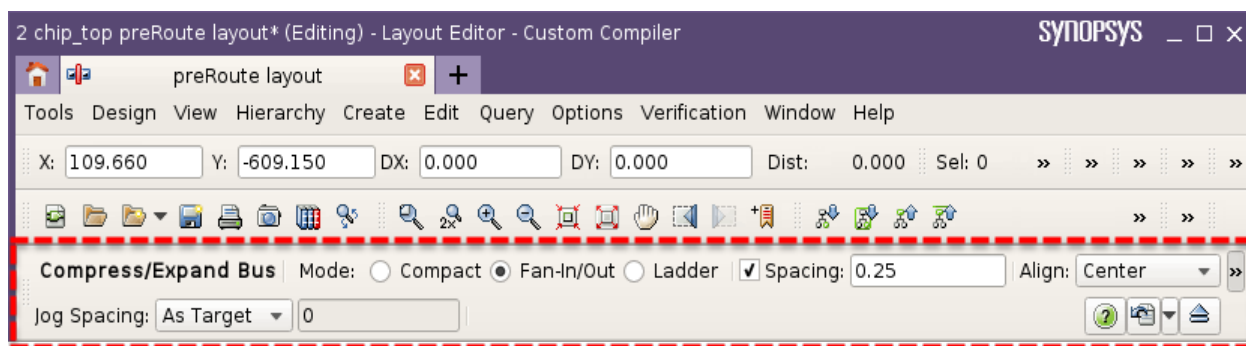
7. (Optional) To change the leader bit, press 1 to cycle to a different bit.
8. When you are ready, click  to compress the bus bits. The follower bits are moved closer or farther to the leader bit based on the Spacing you specified.

Using the Fan-In/Out Mode

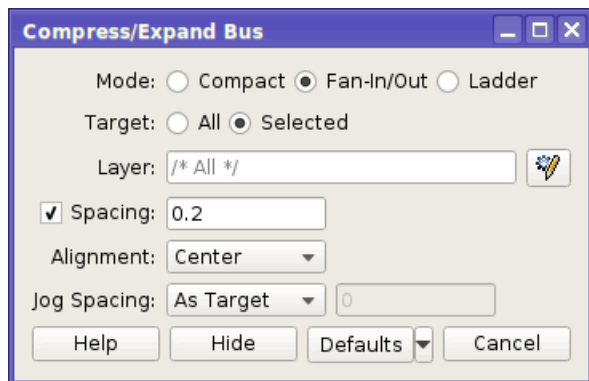
The **Edit > Wire > Compress/Expand Bus** command has a **Fan-In/Out** mode that lets you reduce or increase the spacing between bus bits mid-path.

To use the **Fan-In/Out** mode:

1. Zoom in the design and make sure the bus bits are visible and accessible in the viewport.
2. Choose **Edit > Wire > Compress/Expand Bus**. The **Compress/Expand Bus** toolbar displays.

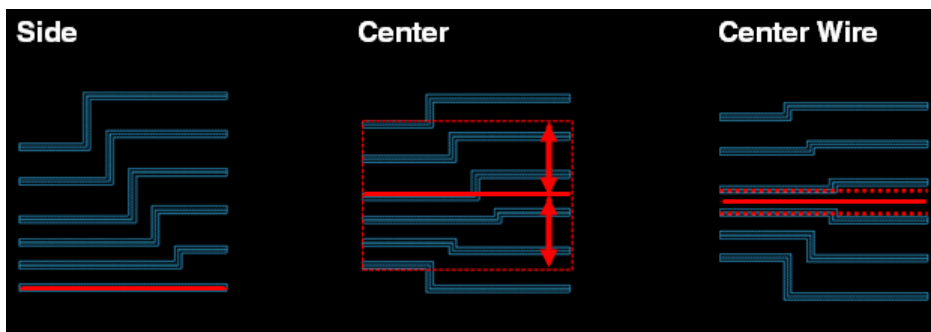


3. Click **Eject** . The **Compress/Expand Bus** dialog box opens and displays additional options.

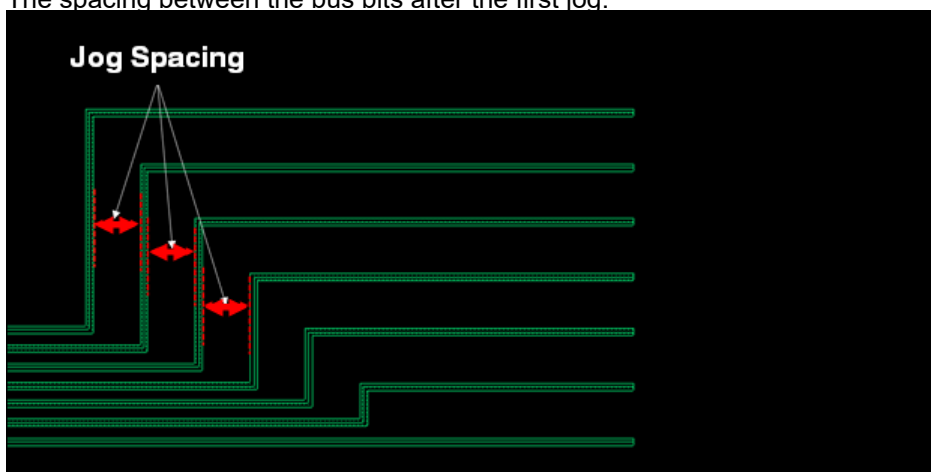


4. Set the following options:

Option	Description
Mode	The compress or expand mode to use: ◦ Compact: Reduces spacing between selected bus bits. ◦ Fan-In/Out: Reduces or increases bus bits at mid-path. ◦ Ladder: Adds jogs in bus bits.
Target	Specifies the applicable bus bits for the operation: ◦ All: Selects all bus bits on the layout canvas. ◦ Selected: Lets you select the bus bits on the layout canvas.
Layer	Opens the Select Layer dialog box and lets you select the routing layers that are applicable for the operation.
Spacing	Specifies the bit-to-bit spacing of compressed or expanded bits.
Alignment	The anchor position from which the spacing increases or decreases: ◦ Side: The bit from which you start the operation. ◦ Center: The center of the selection area. ◦ Center Wire: The center wire.



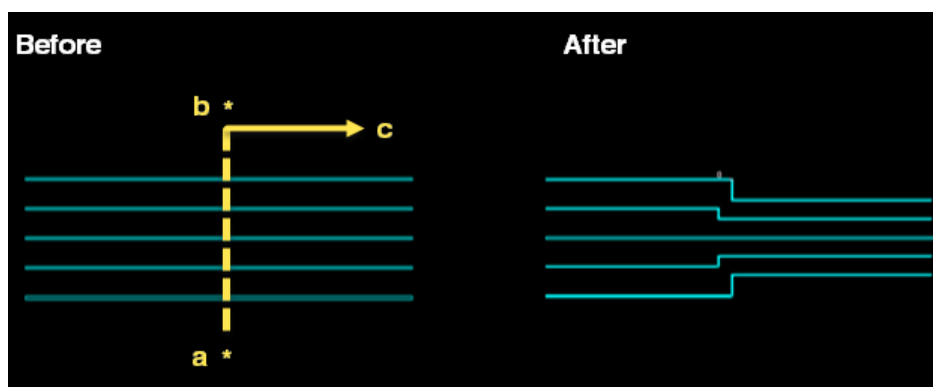
Jog Spacing The spacing between the bus bits after the first jog.



5. If you selected **Target: Selected**, on the layout canvas, select the bus bits.
6. On the layout canvas:
 - a. Click the canvas and draw a line that is perpendicular to the bits to indicate where you want the fan-in or fan-out to begin.
 - b. Click to turn, and draw a line that is parallel to the bits to indicate the direction of the fan-in or fan-out.

If the spacing value you provided is less than the original spacing value, fan-in will result. If the spacing value you provided is greater than the original spacing value, fan-out will result. In the following example, the spacing value specified is less than the original value, so a fan-in structure is created.

- c. Click a third time to complete the operation.



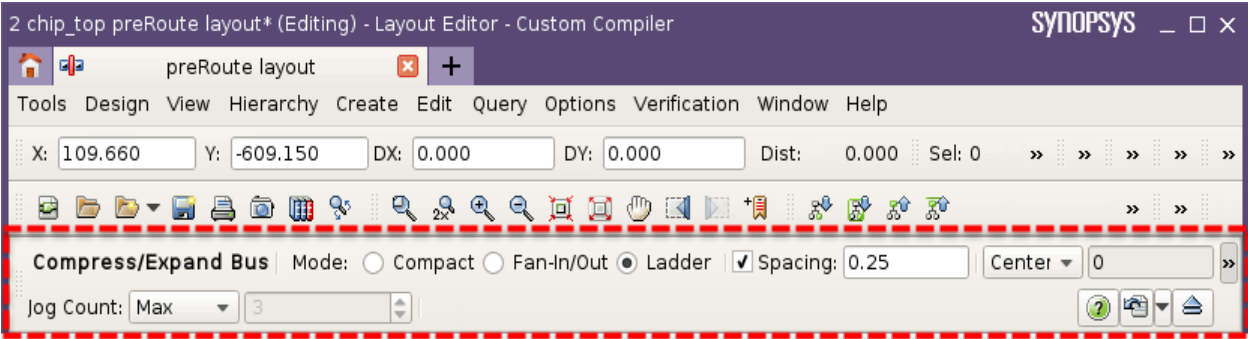
Adding Jogs in Bus Bits (Ladder Mode)

The **Edit > Wire > Compress/Expand Bus** command has a **Ladder** mode that lets you add jogs in bus bits.

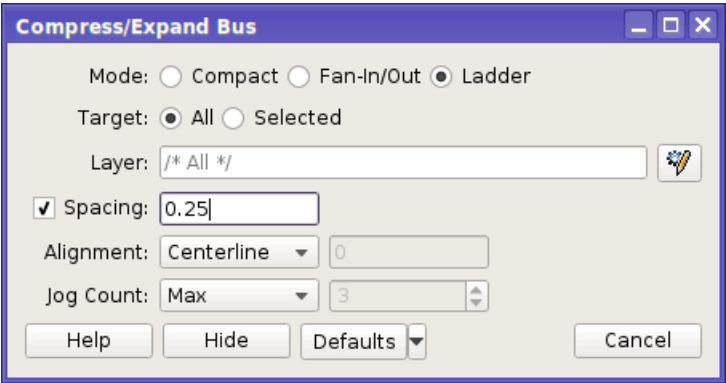
Before you start, review [Ladder Mode](#) to understand the factors that can impact the number of jogs added.

To use the **Ladder** mode to add jogs in bus bits:

1. Zoom in the design and make sure the bus bits are visible and accessible in the viewport.
2. Choose **Edit > Wire > Compress/Expand Bus**. The **Compress/Expand Bus** toolbar displays.



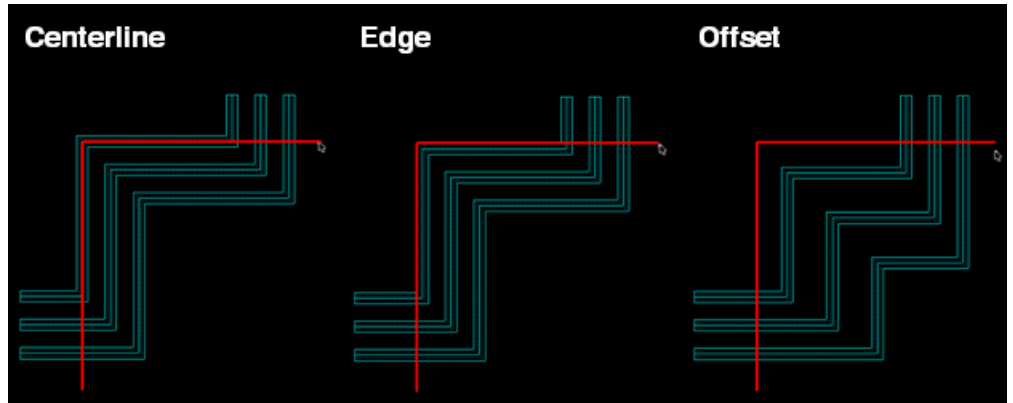
3. Click **Eject** . The **Compress/Expand Bus** dialog box opens and displays additional options.



4. Set the following options:

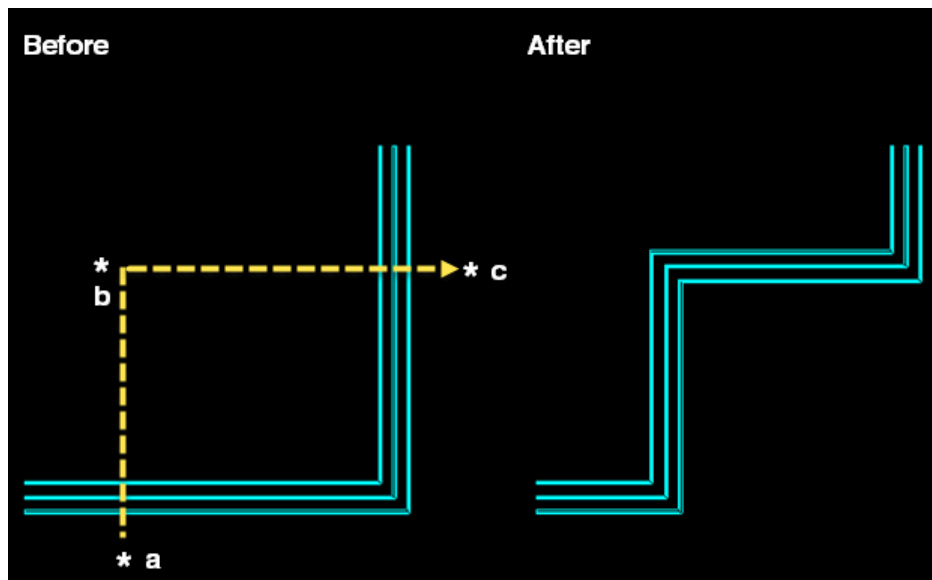
Option	Description
Mode	The compress or expand mode to use: ◦ Compact: Reduces spacing between selected bus bits. ◦ Fan-In/Out: Reduces or increases bus bits at mid-path. ◦ Ladder: Adds jogs in bus bits.
Target	Specifies the applicable bus bits for the operation: ◦ All: Selects all bus bits on the layout canvas. ◦ Selected: Lets you select the bus bits on the layout canvas.
Layer	Opens the Select Layer dialog box and lets you select the routing layers that are applicable for the operation.
Spacing	Specifies the bit-to-bit spacing in jog segments.

Option	Description
Alignment	The position of the reference line used to determine where the jogs begin and end: ◦ Centerline: The centerline of the inner-most bit. ◦ Edge: The edge of the inner-most bit. ◦ Offset: A specified distance offset from the inner-most bit.
Jog Count	The number of jogs to add: ◦ Max: Adds the maximum number of jogs permitted. ◦ Specify: Adds up to the number of jogs you specify.



5. If you selected **Target: Selected**, on the layout canvas, select the bus bits.

6. On the layout canvas, draw reference lines as follows:
 - a. Click the canvas and draw a line that is perpendicular to the bits to indicate where you want the jogs to start.
 - b. Click to turn, and draw a line to indicate where the jogs should end.
 - c. Click to commit the jogs.



Compress/Expand Bus Toolbar and Dialog Box Reference

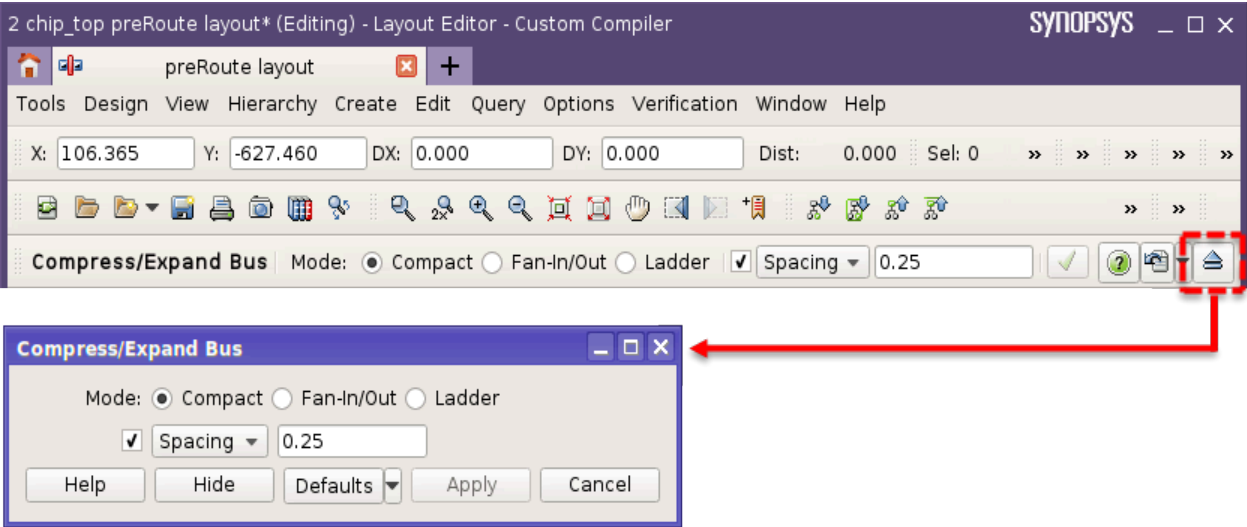
This section presents a quick reference of the **Compress/Expand Bus** toolbar and dialog box.

The contents of the **Compress/Expand Bus** toolbar and dialog box depend on the mode you select:

- [Compact Mode](#)
- [Fan-In/Out Mode](#)
- [Ladder Mode](#)

Compact Mode

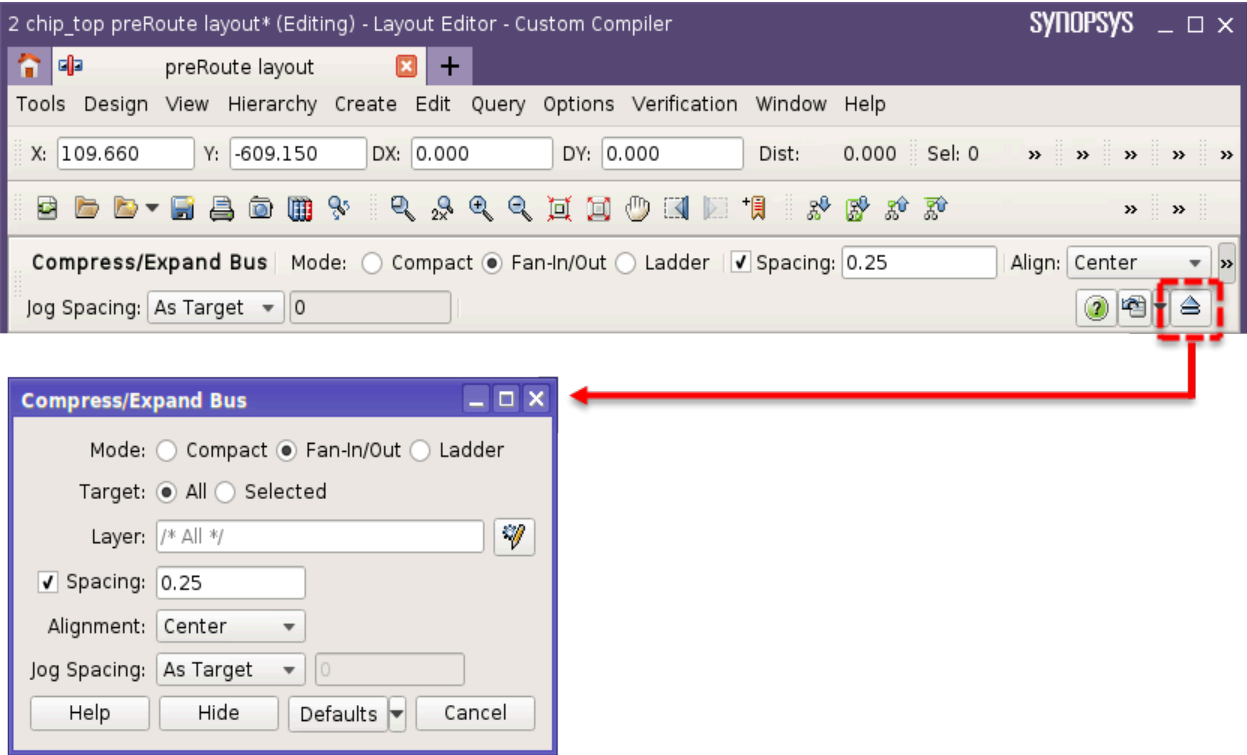
This section presents a quick reference of the **Compress/Expand Bus** toolbar and dialog box in the **Compact** mode. For instructions, see [Compacting Bus Bits](#).



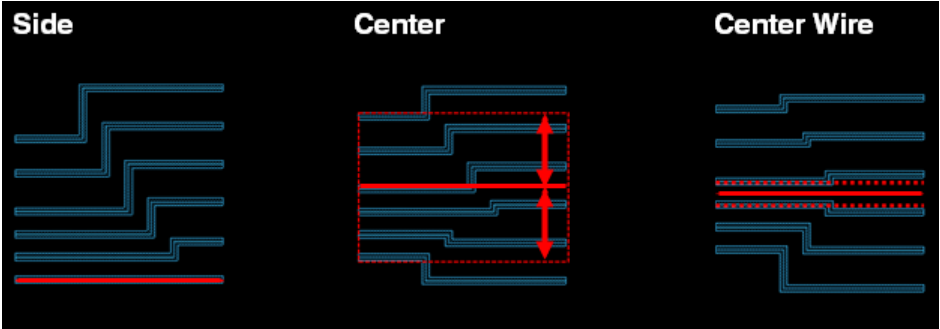
Option	Description
Mode	The compress or expand mode to use: • Compact: Reduces spacing between selected bus bits. • Fan-In/Out: Reduces or increases bus bits at mid-path. • Ladder: Adds jogs in bus bits.
Spacing	Specifies the bit-to-bit spacing after compression. Choose how you want the compressed distance to be applied: Spacing Pitch

Fan-In/Out Mode

This section presents a quick reference of the **Compress/Expand Bus** toolbar and dialog box in the **Fan-In/Out** mode. For instructions, see [Using the Fan-In/Out Mode](#).



Option	Description
Mode	The compress or expand mode to use: • Compact: Reduces spacing between selected bus bits. • Fan-In/Out: Reduces or increases bus bits at mid-path. • Ladder: Adds jogs in bus bits.
Target	Specifies the applicable bus bits for the operation: • All: Selects all bus bits on the layout canvas. • Selected: Lets you select the bus bits on the layout canvas.
Layer	Opens the Select Layer dialog box and lets you select the routing layers that are applicable for the operation.
Spacing	Specifies the bit-to-bit spacing of compressed or expanded bits.

Option	Description
Alignment	The anchor position from which the spacing increases or decreases: • Side: The bit from which you start the operation. • Center: The center of the selection area. • Center Wire: The center wire.
	

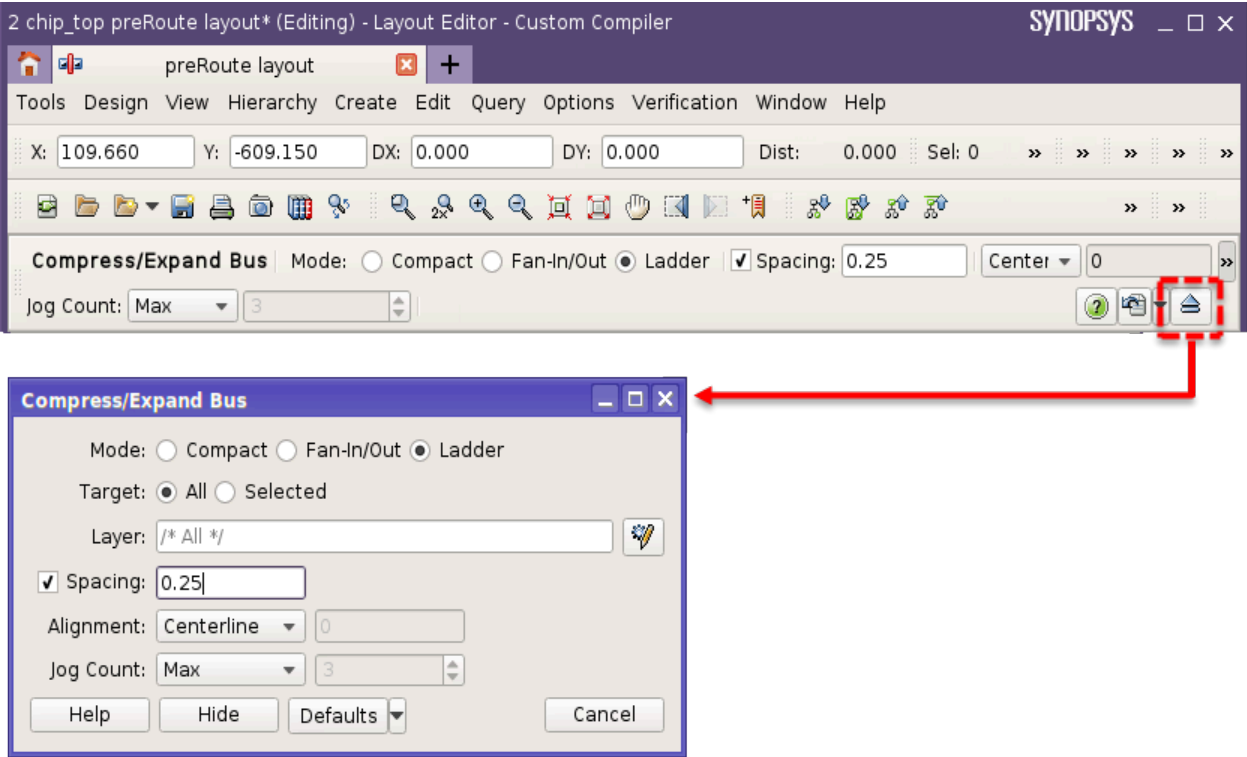
Jog Spacing

The spacing between the bus bits after the first jog.



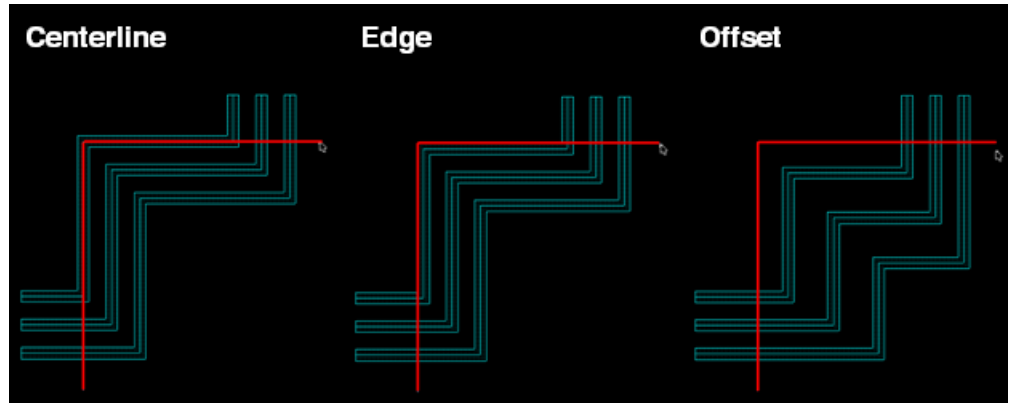
Ladder Mode

This section presents a quick reference of the **Compress/Expand Bus** toolbar and dialog box in the **Ladder** mode. For instructions, see [Adding Jogs in Bus Bits \(Ladder Mode\)](#).



Option	Description
Mode	The compress or expand mode to use: • Compact: Reduces spacing between selected bus bits. • Fan-In/Out: Reduces or increases bus bits at mid-path. • Ladder: Adds jogs in bus bits.
Target	Specifies the applicable bus bits for the operation: • All: Selects all bus bits on the layout canvas. • Selected: Lets you select the bus bits on the layout canvas.
Layer	Opens the Select Layer dialog box and lets you select the routing layers that are applicable for the operation.
Spacing	Specifies the bit-to-bit spacing in jog segments.

Option	Description
Alignment	The position of the reference line used to determine where the jogs begin and end: • Centerline: The centerline of the inner-most bit. • Edge: The edge of the inner-most bit. • Offset: A specified distance offset from the inner-most bit.
Jog Count	The number of jogs to add: • Max: Adds the maximum number of jogs permitted. • Specify: Adds up to the number of jogs you specify.



Cutting Corners of Bus Routes

When using the **Edit > Other > Cut Corner** command on bus routes, paths might be converted to path segments so that they can remain on the track. You can optionally change the objects back to paths if having off-track paths is acceptable.

To use the **Cut Corner** command on bus routes:

- See Cutting Object Corners.

6

Creating and Editing Vias

This chapter describes how to create and edit vias in the designs.

This chapter describes the following:

- [About Vias](#)
- [Bypassing the System's Standard Via Parameter Calculations](#)
- [Creating Individual Vias Manually](#)
- [Creating Individual Vias Automatically](#)
- [Creating Arrays of Vias](#)
- [Creating Stacked Vias](#)
- [Stretching Vias](#)
- [Replacing Vias](#)
- [Adjusting Vias](#)
- [Modifying Via Attributes and Properties](#)
- [Freezing and Unfreezing Vias](#)
- [Creating and Editing Built-In Custom Vias](#)

See also [Adding Vias Between Interconnects](#), [Adding Default Bridge Vias](#), and [Adding Custom Bridge Vias](#).

About Vias

This topic describes the following:

- [Via Description](#)
- [Types of Vias](#)

- [Methods of Adding Vias](#)
- [Via Justification](#)

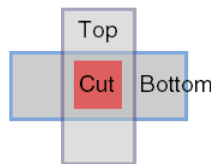
Via Description

A vertical interconnect access, or via, consists of a top metal layer, a bottom metal layer, and one or more cut shapes between the two layers.

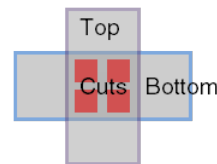
**Via Components
(Side View)**



**Via with One Cut Shape
(Top View)**

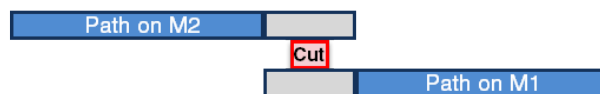


**Via with Multiple Cut Shapes
(Top View)**

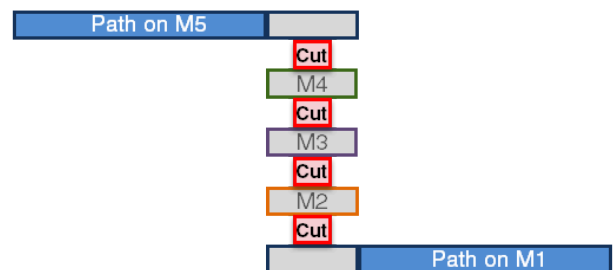


The purpose of a via is to connect two paths that are on different layers. You can create a via to connect two paths on adjacent layers. You can also create multiple vias to connect two paths that are several layers apart.

**Via Connecting Paths on Adjacent Layers
(Side View)**



**Vias Connecting Distant Paths
(Side View)**



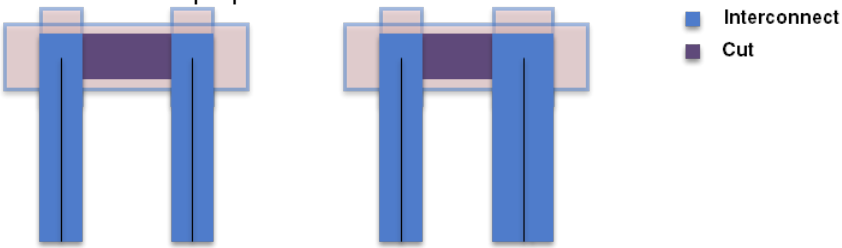
Vias in your design must be based on via definitions in the technology database. See [Via Definitions Section](#) in the technology file.

In advanced node designs (such as those containing FinFETs), you can assign colors to the cut layers when creating or modifying vias. Colors can also be propagated to cuts based on the Color Propagation rules. Color assignments are possible if the color names are defined in the technology database.

Types of Vias

The types of vias you can define in the technology are:

Type of Via	Description
Standard	A standard via has a predefined set of via parameters that can be tailored to make the proper connection in a variety of situations. A standard via is defined in the technology file, but does not have a corresponding master design; therefore, standard vias do not appear in the Custom Compiler Library Manager. When you place a standard via, you can optionally create an array of vias.
Custom	A custom via is similar to an instance because it references a master design in a library. Custom vias are often parameterized cells. The technology file entry for a custom via references the master design; therefore, custom vias do appear in the Custom Compiler Library Manager. When you place a custom via, the tool sets all of the via parameters to their default values. You cannot create custom via arrays; you can only place individual instances of a custom via. Note: If you place a custom via using the Create Instance command instead of the Create Via command, the tool and the design database does not recognize the instance as a via. Always use the Create Via command to place custom vias. Custom vias that have extended functions (also known as built-in custom vias) can only be created and deleted using the <code>db::generateBuiltInVias</code> Tcl command only. You can edit built-in custom vias using the Property Editor and the available Edit commands. See Creating and Editing Built-In Custom Vias.
Variant	A via variant is based on either standard or custom via definitions. However, the variants are parameterized. Standard vias and custom vias must be defined in the technology database. See the Via Definitions Section in the technology file. Via variants can be defined in the technology database, or you can use the <code>db::createStdViaVariant</code> Tcl command to define design-specific standard via variant definitions.

Type of Via	Description
Bridge	Instead of using two vias and a rectangle to bridge and connect two wires on adjacent tracks, you can use bridge vias to define and create a single via for the same purpose. 
	Same Width Different Widths

Methods of Adding Vias

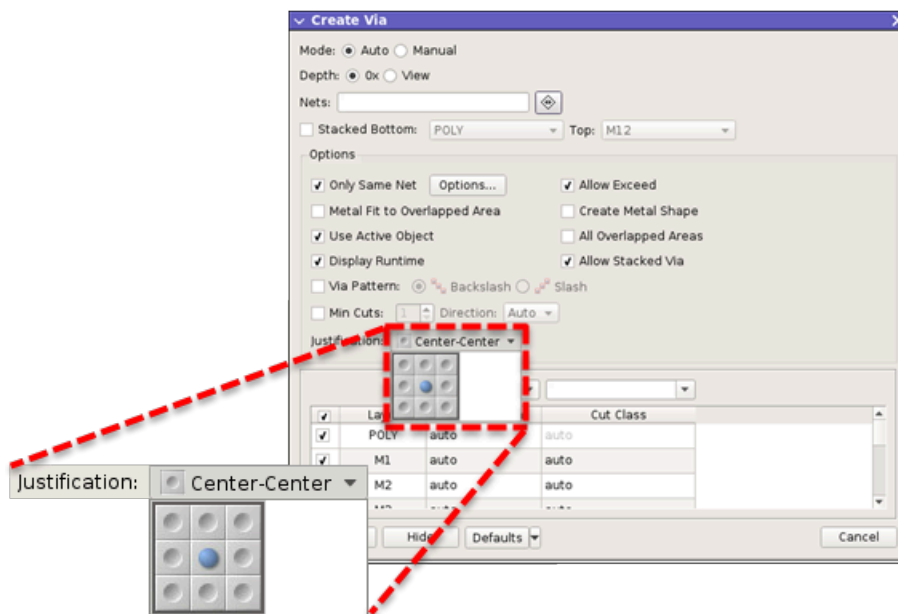
You can use one of the following methods to create or add vias in your design:

Method	Description
Create a via manually	You determine the via definition to use and manually create the vias. Use this method if want to connect existing metal shapes. See Creating Individual Vias Manually .
Create multiple vias automatically	The Layout Editor determines the via definitions to use and automatically creates vias at the location or within the region that you define. Use this method if want to connect existing metal shapes. See Creating Individual Vias Automatically .
Add vias as you create interconnects (routes)	While you are creating interconnects or routes, you can right-click and add vias. Use this method if you want to add vias as you create interconnects. See Adding Vias Between Interconnects .
Add vias as you create bus routes	While you are creating bus routes, you can right-click and add vias. See Adding Vias During Bus Creation .

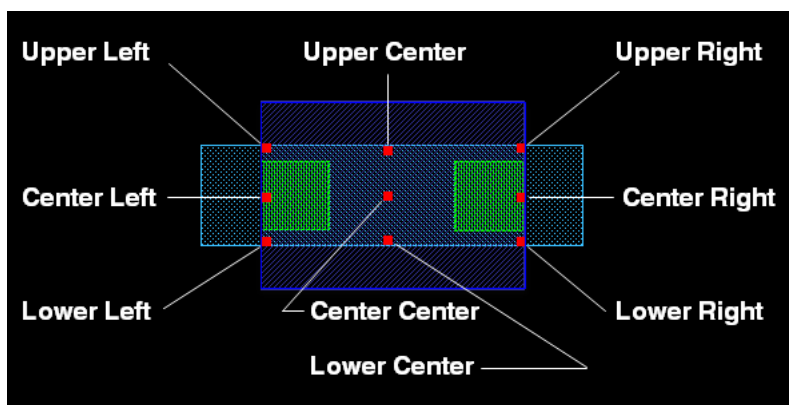
Except for the built-in custom vias, you can use the `ile::createVia` and `le::createVia` Tcl commands instead of using graphical user interface to create vias.

Via Justification

When setting up automatic or manual via creation, you have the option of specifying the via **Justification**, the position of the via between the metal wires. See [Via Toolbar and Create Via Dialog Box \(Manual Non-stacked Mode\)](#) and [Via Toolbar and Create Via Dialog Box Reference \(Auto Mode\)](#).

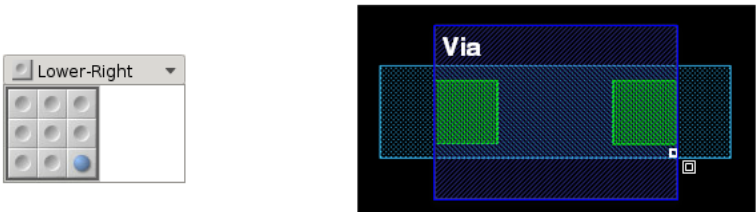


The **Justification** option specifies one of nine locations where the via layers overlap:



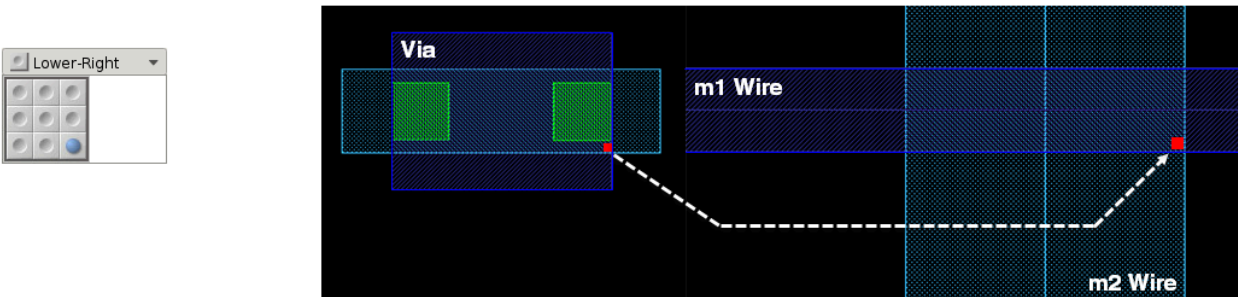
When creating vias manually, the **Justification** setting corresponds to the via origin, which coincides with the pointer position as you manipulate the via on the canvas.

Manual Mode

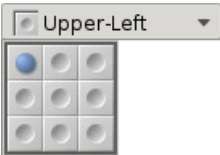
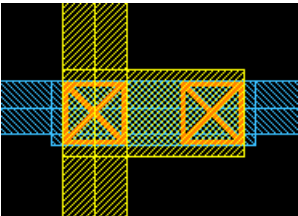
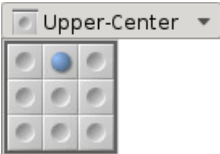
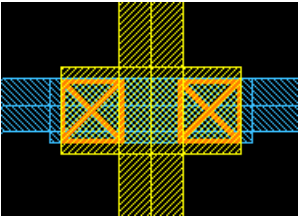


When creating vias automatically, the via **Justification** location snaps to the justification location of the intersecting wires.

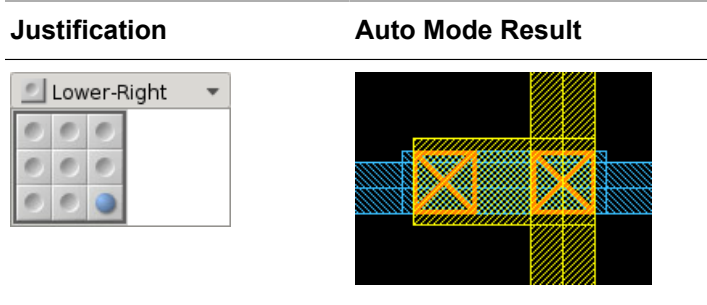
Auto Mode



The following table lists the nine via **Justification** settings and the corresponding automatic via positions at the juncture between two intersecting paths.

Justification	Auto Mode Result
	
	

Justification	Auto Mode Result
Upper-Right	
Center-Left	
Center-Center	
Center-Right	
Lower-Left	
Lower-Center	



Under the following circumstances, the **Center-Center** via justification will be applied regardless of the **Justification** setting:

- When creating stacked vias to connect paths that are multiple layers apart, and the Layout Editor is not able to align the via based on the Justification specification across all layers.
- (Auto Via mode only) When creating vias between collinear paths (two paths that are stacked), the Layout Editor is unable to align the via with one of the paths based on the Justification specification.
- (Auto Via mode only) When creating vias between two paths and one of the paths is not well formed. The Layout Editor is unable to align the via with the non-well-formed path.
- (Auto Via mode only) When `leAutoViaEnvAware` is turned on, and to prevent a DRC violation, the Layout Editor is unable to align the via based on the Justification specification.

Bypassing the System's Standard Via Parameter Calculations

You can bypass the system and perform your own standard via parameter calculations. Doing so lets you fully control every parameter calculation during automatic via creation and editing.

The setup requires the following:

- Define the Tcl procedure as a via property that has a string value.
- Name the property `StdViaParameterizeFunc`. During the auto via creating or editing operation, the `StdViaParameterizeFunc` property tells the Layout Editor to bypass the system and call the Tcl procedure to calculate and update the standard via parameters.

In the Tcl procedure, set the function signature to `userFunc $data`, where:

- `data` is `{(keywordString_1 value_1) ... (keywordString_n value_n)}`.
- `value_i` can be any value type, such as string, int, double, or list.

`data` must contain the following data that specify the via placement environment:

Input Data	Description
<code>StdVia</code>	oaStdVia ID that is used to obtain the standard via definition, via layer numbers, and other information for calculating and updating the standard via definition.
<code>is_viadev_specified</code>	Flag that indicates whether to bypass the system and use the Tcl procedure you provided to update the standard via definition. Set the flag to <code>true</code> to bypass the system and use the Tcl procedure. Set the flag to <code>false</code> to let the system update the standard via definition.
<code>overlap_box</code>	Overlapping region between layer1 and layer2. The via will be automatically placed within this region.
<code>layer1_path_width</code>	Width of layer1. 0 indicates no layer1 present. Default: 0
<code>layer2_path_width</code>	Width of layer2. 0 indicates no layer2 present. Default: 0
<code>layer1_path_direction</code>	Direction the layer1 wire. The direction is only applicable if <code>layer1_path_width</code> is greater than 0.
<code>layer2_path_direction</code>	Direction of the layer2 wire. The direction is only applicable if <code>layer2_path_width</code> is greater than 0.
<code>layer1_color</code>	Color of layer1.
<code>layer2_color</code>	Color of layer2.
<code>can_adjust_origin</code>	Flag that indicates whether you are permitted to adjust the via origin during the create and edit operations. Set the flag to <code>true</code> to permit via origin adjustment.

If you select a standard via definition to create the via automatically, the system will call the Tcl procedure to parameterize the standard via. If you do not select a standard via definition, the system will attempt to use the default standard via definition, if any. However, if the `leChooseCustomViaFirst` Tcl preference is set to `true`, the system will try to use the custom via definition first.

The system requires a return value from the Tcl procedure to determine whether the via parameterization succeeded:

- `0` or `false`: The system was unable to create a via at the specified location, or the system was unable to get the available via parameters.
- `1` or `true`: The via parameters are updated in the Tcl procedure, and the system created the via successfully.

Creating Individual Vias Manually

When creating individual vias manually, you determine the via definition, set the options, and manually add the vias in the design.

This topic describes the following:

- [Creating Individual Vias Manually at Specific Locations](#)
- [Via Toolbar and Create Via Dialog Box \(Manual Non-stacked Mode\)](#)
- [Via Shortcut Menu and Shortcut Keys](#)

Before you create individual vias manually, you can set the `leManualViaCalculateLayerWidth` Tcl preference to `true` to calculate the pseudo layer width based on the preferred routing direction and the columns, rows, and spacing settings of the cut via. For information about the Tcl preference, see the *Custom Compiler Tcl Reference Manual*.

To add individual vias automatically, see [Creating Individual Vias Automatically](#).

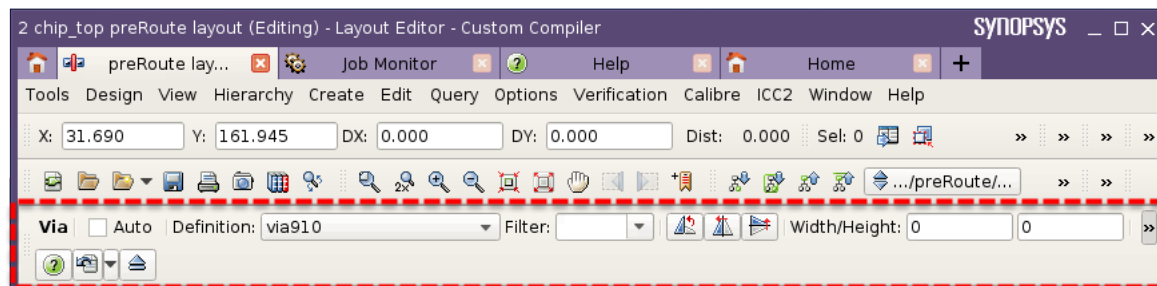
To add vias while creating interconnects, see [Adding Vias Between Interconnects](#).

Creating Individual Vias Manually at Specific Locations

In the **Create Via Manual** mode, you can add individual vias at specific locations on the canvas.

To create individual vias manually at specific locations:

1. In the Layout Editor, choose **Create > Via**. The **Via** toolbar appears.

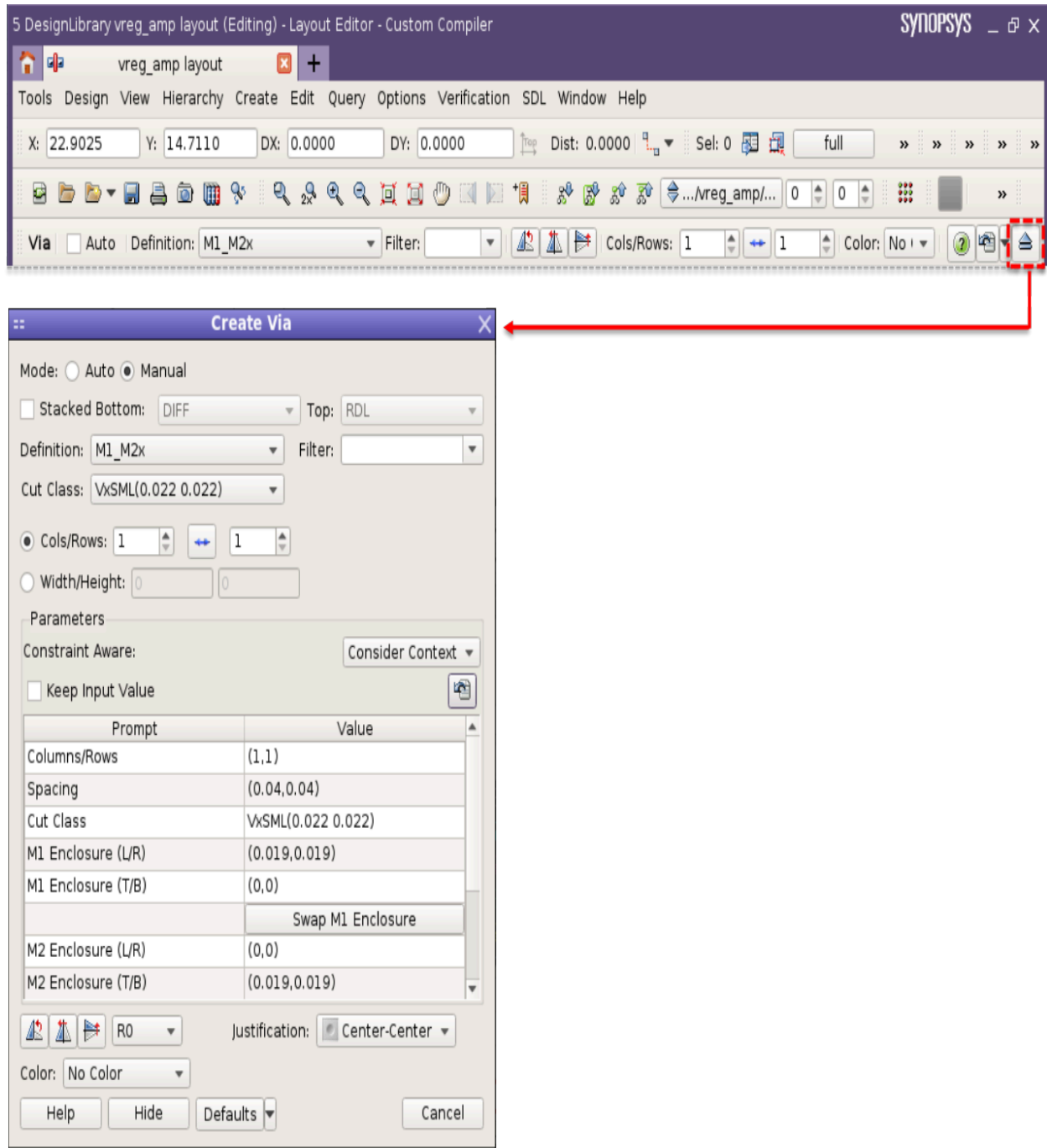


2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Create Via** dialog box. The dialog box contains additional options.
To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.
3. On the toolbar, clear the **Auto** check box. If you are in the dialog box, select **Manual**.
4. In the **Definition** list, choose the via definition you want to use. If the list is long, type the first few characters of the via definition name in the **Filter** box to quickly find the via definition.
5. (Optional) Specify the remaining options on the toolbar or in the dialog box. For example, you can select the cut class. See [Via Toolbar and Create Via Dialog Box \(Manual Non-stacked Mode\)](#).
6. On the layout canvas:
 - To create vias based on the **Col/Rows** or **Width/Height** values you specified, click a location where you want to place via.
 - To create vias without complying with the **Col/Rows** or **Width/Height** values, draw a rectangular region. The origin is based on the **Justification** setting.
7. (Optional) Repeat steps [Step 4](#) to [Step 6](#) and create additional vias.
8. When you are finished, press Esc to quit the command.

Via Toolbar and Create Via Dialog Box (Manual Non-stacked Mode)

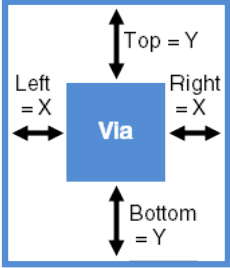
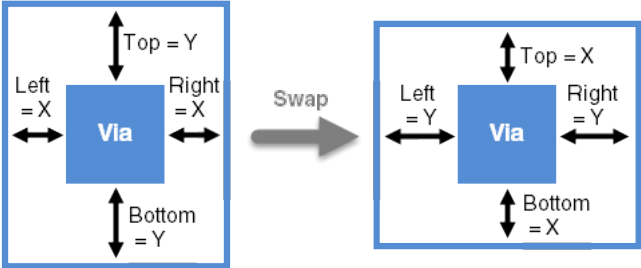
This section provides a quick reference of the **Via** toolbar and the **Create Via** dialog box in the **Manual** mode, when the **Stacked** option is turned off. For instructions on how to create individual vias manually, see [Creating Individual Vias Manually at Specific Locations](#).

For the quick reference of the **Stacked** mode options, see [Via Toolbar and Create Via Dialog Box Reference \(Manual Stacked Mode\)](#).



Option	Description
Auto/Manual	Specifies the via-creation mode: Auto or Manual. When Auto is selected, you can drag your cursor to define a region on the canvas. The Layout Editor determines via creation and placement. When Manual is selected (default), you can specify the via options and place the via. (On the toolbar, clear the Auto check box to specify the manual via creation mode.)
Stacked	(Dialog box only) When enabled, turns on the stacked via mode for standard vias and standard via variants. See Creating Stacked Vias.
Definition	Displays only the via, via variant, or bridge via definitions that meet the filter criteria specified in the Filter box.
Filter	Type the via, via variant, or bridge via definition name in the Filter box. The Recently Used history list saves up to 10 user-specified filter strings. The By Bottom Layer list lets you choose the name of the layer below the via.
Cut Class	(Dialog box only) The standard via's dimensions. Choose one from the list. The cutClasses are defined in the constraints section of the technology database. The list includes an <i>auto</i> cut class when Auto mode is enabled. When Manual non-stacked mode is enabled, you can modify the parameters of the selected via in the Parameters table. When Manual stacked mode is enabled, you can modify the spacing and the enclosure values for Layer1 and Layers by clicking on the field or by entering the desired values.
Cols/Rows	Cols specifies the number of via columns for standard vias. For custom vias, the value is fixed at 1. Rows specifies the number of via rows to create for standard vias. The tool creates equal numbers of rows and columns, or one more row than the number of columns, whichever more closely matches the <code>minNumCut</code> rule. You can switch the Cols and Rows values using the Swap Number Between Column and Row  button. Tcl preferences: <code>leViaCols</code> and <code>leViaRows</code>
Width/Height	The dimensions of the area in which you will add standard vias. The rows and columns of vias will be calculated automatically. Type the width (x value) in the first box. Type the height (y value) in the second box. Both values are in μm. Tcl preferences: <code>leViaCreateMethod</code>, <code>leViaRegionWidth</code>, <code>leViaRegionHeight</code>
Parameters	(Dialog box only) A table showing all of the parameters available for the selected via.

Option	Description
Constraint Aware	Specifies whether to consider the via's surroundings and rule compliance when determining the parameter values: • Off: The Layout Editor does not adjust the vias automatically. You can enter values even if the values might cause DRC violations. • Ignore context: The Layout Editor computes the parameters as if the via is placed in free space in the design. The Layout Editor does not consider the via's immediate surroundings (or context). • Consider context: The Layout Editor computes the parameters based on the location of the via. The via's immediate surroundings (or context) are considered for adjusting the parameters to ensure DRC-clean results. Tcl preference: <code>leEditViaParamConstraintAwareMode</code>
Keep Input Value	(Dialog box only) Retains the following parameter values you specify for each via definition: Cut Class, Cols/Rows, Enclosure, Spacing. If you choose a different via definition, or if you exit the dialog box and return, the last via definition settings will persist. This option is only available for creating non-stacked vias.
Reset to Default Value 	(Dialog box only) Resets all parameters to their default values. This option is only available for creating non-stacked vias.
Columns/Rows	(Dialog box only) The number of columns and rows of cut shapes in the via. To swap the Columns and Rows values, click one of the values and click the Swap Columns and Rows Values  button that appears between the Columns and Rows values.
Spacing	(Dialog box only) The X and Y distance between the cuts. If the via contains multiple cut arrays, Spacing is the distance between the arrays. By default, the cut array spacing values are based on the <code>minLargeViaArraySpacing</code> constraint.
Cut Width	(Dialog box only) The width of the cut.
Cut Height	(Dialog box only) The height of the cut.

Option	Description
<layer1/2> Enclosure (L/R)	(Dialog box only) The amount to extend Layer1 or Layer 2 beyond the cut edges in four directions: left (L), right (R), top (T), and bottom (B).
<layer1/2> Enclosure (T/B)	By default, the Layer2 Enclosure values are derived from the Layer1 Enclosure values.
	 (Standard vias only) To transpose the X and Y values of the layer enclosure, click Swap <layer> Enclosure. The button is available only if all standard vias share the same enclosure type. 
Origin Offset	(Dialog box only) The X and Y distance from the design origin.
Cut LPP	(Dialog box only) The layer-purpose pair of the cut layer.
Rotate	Rotates the shape 90° counter-clockwise. To rotate 180° or 270°, repeat the command. Tcl preference: <code>leCreateOrientation</code>
Flip Horizontal	Mirrors the shape around the X-axis before placement. Tcl preference: <code>leCreateOrientation</code>
Flip Vertical	Mirrors the shape around the Y-axis before placement. Tcl preference: <code>leCreateOrientation</code>
Orientation	(Dialog box only) Specifies the orientation for the instance you want to place. Tcl preference: <code>leCreateOrientation</code>

Option	Description
Justification	(Dialog box only) Specifies the via origin, which coincides with the pointer position as you manipulate the via on the canvas. See Via Justification . Tcl preference: <code>leCreateViaJustification</code>
Color	(Manual non-stacked mode only) Assigns a color to the selected via (cut layer shape) as follows: • Auto (default): Automatically assigns a color to the new via based on the color propagation rule. If no propagation can be applied, then the shape or via is assigned the default color. See Color Propagation. • <colorName>: Assigns the selected color, <colorName>, to the newly created via. The available color names are defined in the technology file. • Same: Assumes that nearby shapes have the same color and applies the <code>multiPatternSpacing</code> constraint. This selection is available only if you set the <code>leIncludeColorSame</code> preference to <i>true</i>. • No Color: Does not apply any color to the newly created via. This selection is available only if you set the <code>leIncludeColorNone</code> preference to <i>true</i>.

Via Shortcut Menu and Shortcut Keys

While the **Create > Via** command is active, you can right-click to display the Create Via shortcut menu.

This section provides a quick reference of the shortcut menu and shortcut keys. For instructions on how to create individual vias, see [Creating Individual Vias Manually](#) or [Creating Individual Vias Automatically](#).

Note:

This menu is not available for manual stacked-via creation.

	Rotate	R
	Flip Horizontal	
	Flip Vertical	
	Decrease Rows	Shift+Down
	Increase Rows	Shift+Up
	Decrease Cols	Shift+Left
	Increase Cols	Shift+Right

Command	Purpose	Default Shortcut Key
Rotate	Rotates the via 90 degrees counter-clockwise. To rotate 180 or 270 degrees, repeat the command.	R
Flip Horizontal	Mirrors the via around the Y-axis.	Shift-right-click
Flip Vertical	Mirrors the via around the X-axis.	Ctrl-right-click
Decrease Rows	Decreases the number of via rows by 1.	Shift+Down arrow
Increase Rows	Increases the number of via rows by 1.	Shift+Up arrow
Decrease Cols	Decreases the number of via columns by 1.	Shift+Left arrow
Increase Cols	Increases the number of via columns by 1.	Shift+Right arrow

Creating Individual Vias Automatically

You can use the **Create Via Auto** mode to add individual vias in your designs automatically.

This topic describes the following:

- [About Creating Individual Vias Automatically](#)
- [About the Use Active Object and All Overlapped Areas Options](#)
- [Before You Start](#)
- [Creating Individual Vias Automatically by Object Selection](#)
- [Creating Individual Vias Automatically Within Specified Region](#)
- [Creating Vias in the Intersection Mode](#)
- [Via Toolbar and Create Via Dialog Box Reference \(Auto Mode\)](#)
- [Via Shortcut Menu and Shortcut Keys](#)

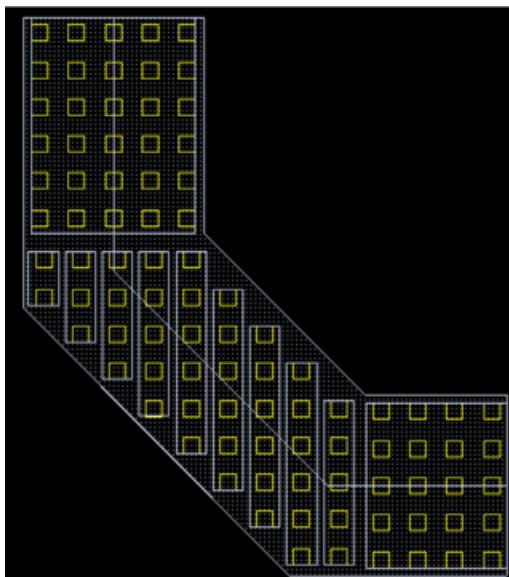
To create vias automatically while stretching interconnects, see [Stretching Interconnects and Adding Vias Automatically](#).

To create vias manually, see [Creating Individual Vias Manually](#).

To add vias while creating interconnects, see [Adding Vias Between Interconnects](#).

About Creating Individual Vias Automatically

When creating individual vias in the automatic mode (or Auto Via mode), you select an object to define a location or you draw and define a region in your design. The command then determines the appropriate options to use and automatically places the vias, even within defined via locations that form non-Manhattan regions.



See [Creating Individual Vias Automatically by Object Selection](#) and [Creating Individual Vias Automatically Within Specified Region](#).

When creating vias automatically, all the layers on the connection path comply with the following rules:

- The command does not create vias at blockages.
- If a blockage is nearby, the command uses the spacing and effective width parameters to determine the minimum spacing around the blockage.

If no spacing or effective width parameter is available, the command uses the minimum width of the blockage to determine the spacing to use.

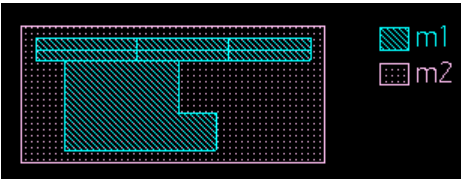
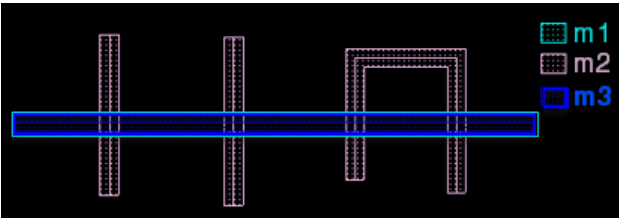
The Layout Editor does not create vias if doing so results in shorts.

About the Use Active Object and All Overlapped Areas Options

In the **Create Via (Auto mode)** dialog box, different combinations of the **Use Active Object** and **All Overlapped Areas** options can generate different results.

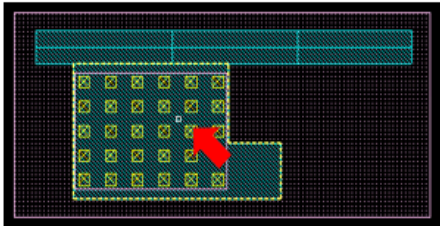
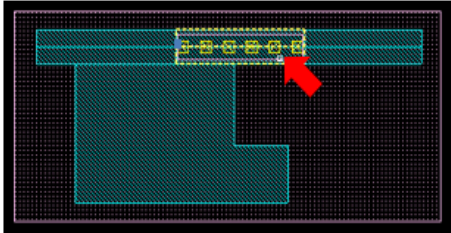
The following table shows the possible results based on the different selection combinations. In the Option Selections and Results table, the red arrow represents the pointer. The yellow highlighting indicates the active object.

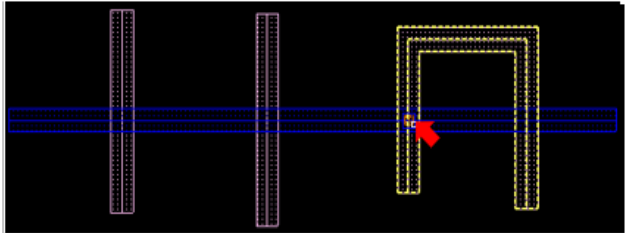
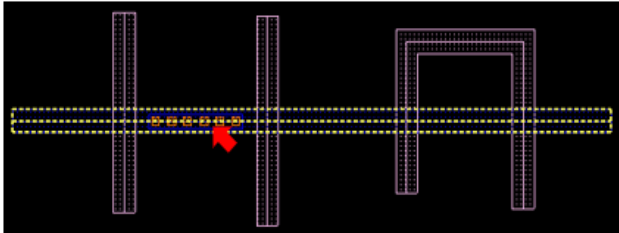
For instructions on how to set the options, see [See Creating Individual Vias Automatically by Object Selection](#) and [Creating Individual Vias Automatically Within Specified Region](#). See also [Via Toolbar and Create Via Dialog Box Reference \(Auto Mode\)](#).

Original Scenario		Description
1		• m1 path abuts m1 polygon. • m1 path and m1 polygon overlap with m2 rectangle.
2		• m1 and m3 paths overlap. • m1 path is under m3 path. • Both m1 and m3 paths overlap with m2 paths.

Option Selections and Results

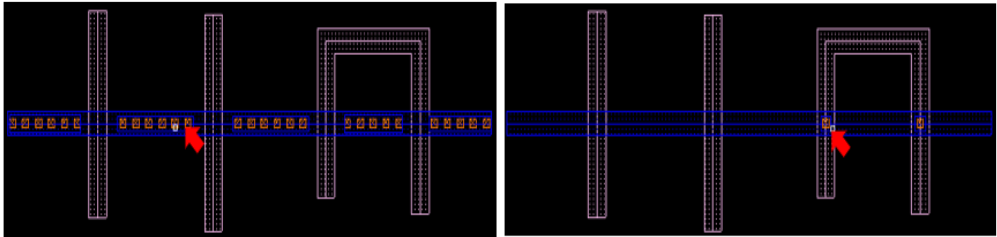
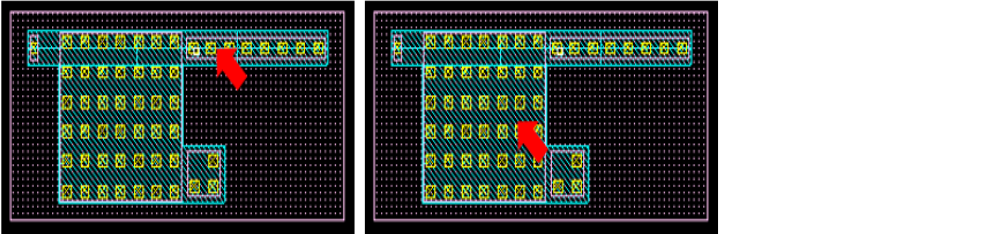
☒ Use Active Object ☐ All Overlapped Areas



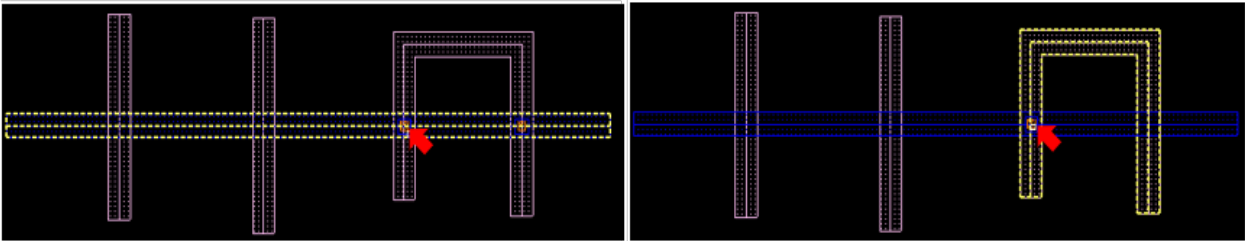
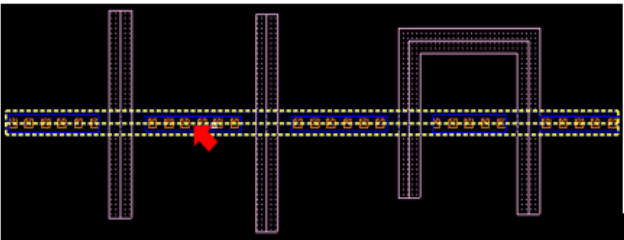
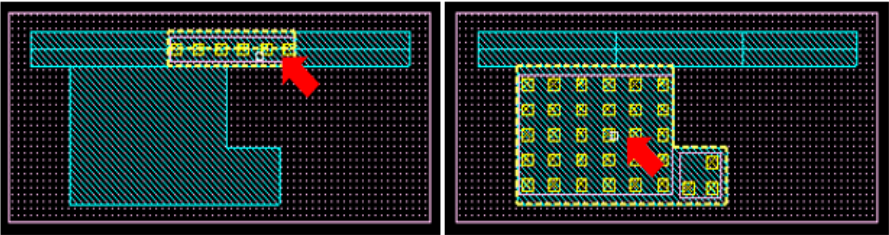


☐ Use Active Object ☒ All Overlapped Areas

Option Selections and Results

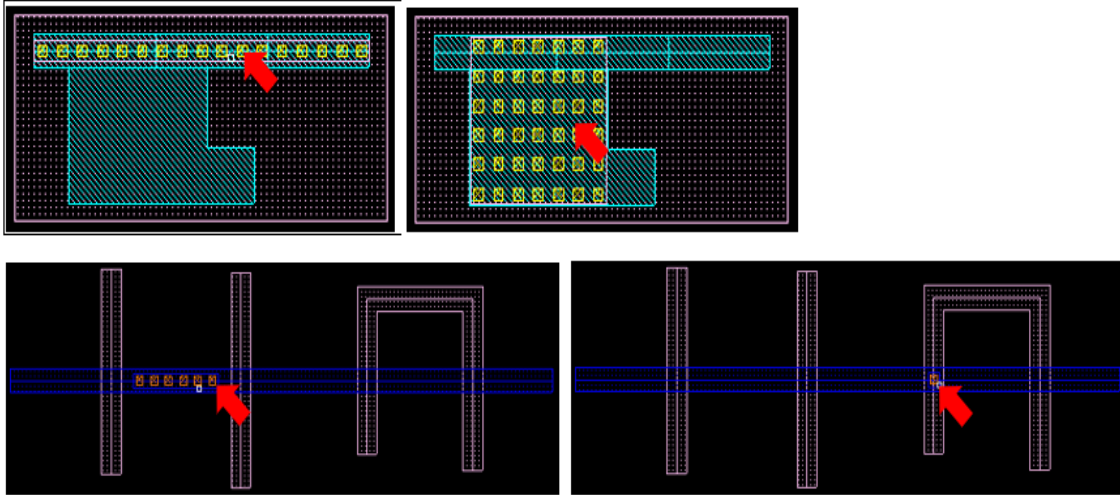


☒ Use Active Object ☒ All Overlapped Areas



☐ Use Active Object ☐ All Overlapped Areas

Option Selections and Results



Before You Start

You can set Tcl preferences to broaden rule compliance checks during via creation. You can also set Tcl preferences to automatically add bridge vias and ensure proper net propagation in designs with feedthrough shapes and layers. You must set the Tcl preferences prior to creating the vias.

This topic describes the following:

- [Setting Up for Rule Compliance Checks Within the Surrounding Area](#)
- [Setting Up for Automatic Bridge Via Creation](#)

Setting Up for Rule Compliance Checks Within the Surrounding Area

Auto Via creates vias that comply with design rules. If you also want to quickly check the surrounding area for rule compliance during the Auto Via operation, set the `leAutoViaEnvAware` Tcl preference to `true`.

Tcl Preference	Description
<code>leAutoViaEnvAware</code>	Lets Auto Via check for rule compliance within the surrounding area.

When `leAutoViaEnvAware` is `true`, vias might not be added even when a matching `cutClass` candidate is found. This is to ensure sufficient spacing between neighboring cuts. To add the smallest possible cut among all applicable self-aligned `cutClass` candidates and to avoid design rule errors when `leAutoViaEnvAware` is `true`, set the `leCutClassAlignPreferSmall` Tcl preference to `true`.

By default, vias are sometimes added offset from the center of the intersecting routes to avoid design rule errors. To always add vias at the center of the intersecting routes, be sure to define the `minCenterlineExtension` layer pair constraint.

Setting Up for Automatic Bridge Via Creation

Instead of using two vias and a rectangle to connect two wires on adjacent tracks, you can use bridge vias to define and create a single via for the same purpose. You can use the **Create Via (Auto mode)** command to automatically add bridge vias.

To set up for automatic bridge via creation:

1. In the `Via Definitions` Section of the technology file, define `bridgeVias`.

The bridge via definition is based on a standard via, standard via variant, custom via, or custom via variant definition.

2. Set the `leAutoViaAutoBridgeVia` Tcl preference.

- `true`: Creates bridge vias when possible. Otherwise, the command adds standard vias.

If standard vias are added, the **Create Via (Auto mode)** options, such as **Allow Exceed**, are applicable. The `leAutoViaEnvAware` Tcl preference setting is also applied.

The command cannot alter standard-via parameters. If the vias will result in rule violations, the command will not create the via.

- `false`: Creates bridge vias only if the `bridgeVias` definition name or the `bridgeVias` variant name is selected in the **Create Via (Auto mode)** dialog box.

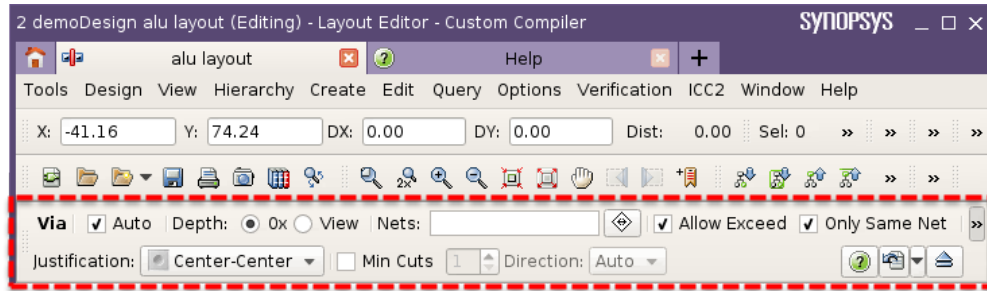
See Also

- [Adding Default Bridge Vias](#)
- [Adding Custom Bridge Vias](#)

Creating Individual Vias Automatically by Object Selection

To place vias automatically by object selection:

1. Choose **Create > Via**. The Via toolbar appears.



2. (Optional) On the toolbar, click Eject  or press F3 to display the **Create Via** dialog box. The dialog box contains additional options.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

3. Select **Auto**.
4. (Optional) Specify the remaining options on the toolbar or in the dialog box. See [Via Toolbar and Create Via Dialog Box Reference \(Auto Mode\)](#).
5. On the layout canvas, click an object to specify a via location. The command creates a via at the location.

Note:

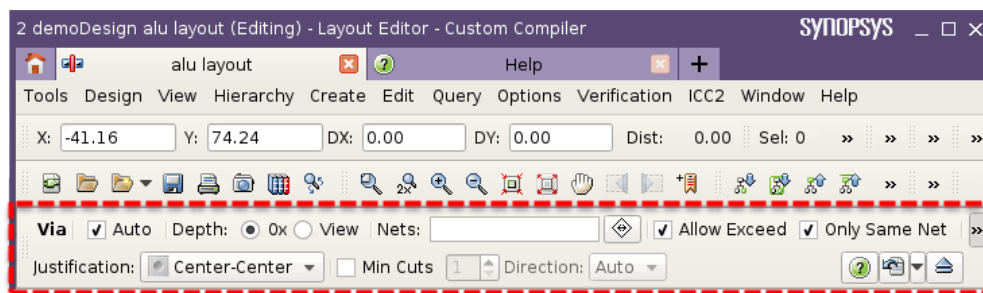
Via placement is neither DRC correct nor LVS correct at this point. If a large area is passed to the command, contacts may be created in undesirable locations; for example, poly-metal or diff-metal of well contacts.

6. When you are finished, press Esc to quit the command.

Creating Individual Vias Automatically Within Specified Region

To place vias automatically by region:

1. Choose **Create > Via**. The Via toolbar appears. The mouse pointer changes from  to .



2. (Optional) On the toolbar, click Eject  or press F3 to display the **Create Via** dialog box. The dialog box contains additional options.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

3. Select **Auto**.
4. (Optional) Specify the remaining options on the toolbar or in the dialog box. See [Via Toolbar and Create Via Dialog Box Reference \(Auto Mode\)](#).
5. On the layout canvas, draw a rectangular region in which you want to create the vias.

The command creates vias between adjacent layers within the region. The operation starts from the bottom-left part of the region and ends at the top-right part of the region. The command connects the uppermost interconnect layer first, and then connects downward through the layers.

Note:

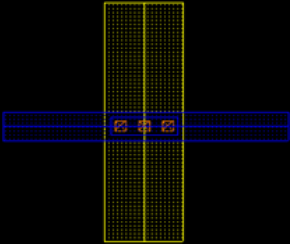
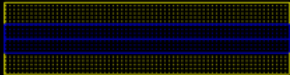
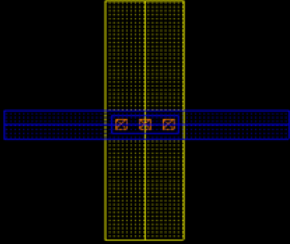
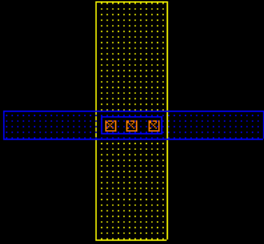
Via placement is neither DRC correct nor LVS correct at this point. If a large area is passed to the command, contacts may be created in undesirable locations; for example, poly-metal or diff-metal of well contacts.

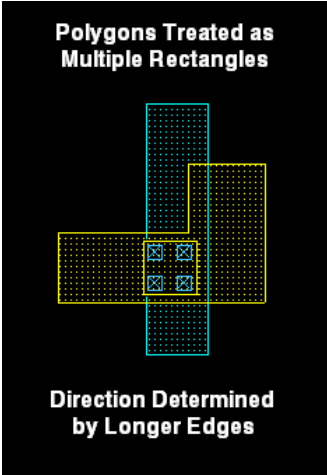
6. When you are finished, press Esc to quit the command.

Creating Vias in the Intersection Mode

You have the option of adding vias only when two paths or two Manhattan shapes of different directions intersect. To turn on this mode, set the `leAutoViaOnIntersectionOnly` Tcl preference to true.

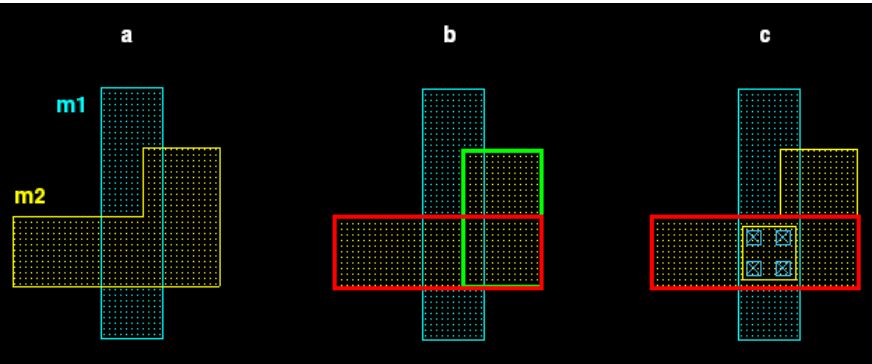
The results of the preference setting are as follows:

Scenario	leAutoViaOnIntersectionOnly true	leAutoViaOnIntersectionOnly false
Paths	Different Directions  Same Direction 	Different Directions  Same Direction 
Rectangular Shapes	Direction Determined by Longer Edges  Horizontal Direction If Square Shapes	

Scenario	leAutoViaOnIntersectionOnly true	leAutoViaOnIntersectionOnly false
Polygons	 Polygons Treated as Multiple Rectangles Direction Determined by Longer Edges	

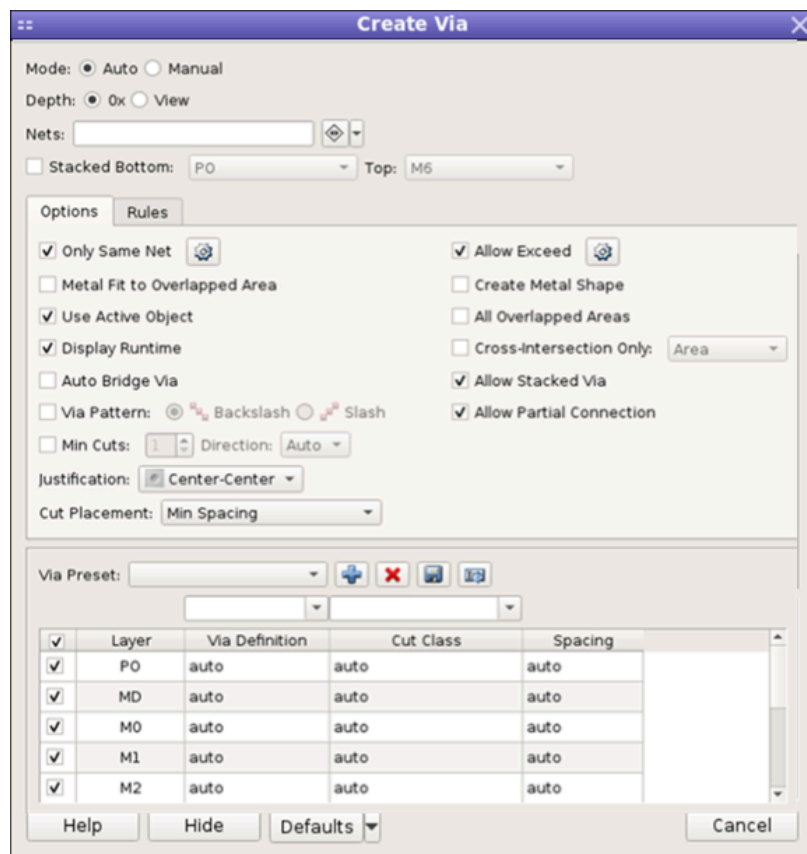
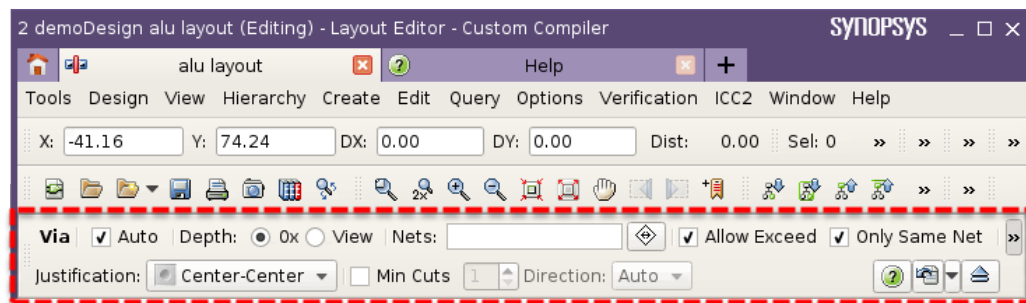
Polygons are treated as multiple rectangles, and vias are added based on the direction of the individual rectangular components relative to the intersecting path or rectangle. In the following example:

- The m2 polygon overlaps with the m1 rectangle (a).
- The m1 rectangle is vertical.
- The m2 polygon is treated as two rectangles (b). The red rectangular component is horizontal, and the green rectangular component is vertical.
- When `leAutoViaOnIntersectionOnly` is true, vias are added between the vertical m1 path and the red rectangular component of the m2 polygon, because they are in different directions. The green rectangular component is not considered, because it is in the same direction as the intersecting m1 rectangle.



Via Toolbar and Create Via Dialog Box Reference (Auto Mode)

This section provides a quick reference of the **Via** toolbar and the **Create Via** dialog box in the **Auto** mode.



The **Create Via - Auto Mode** dialog box consists of the following:

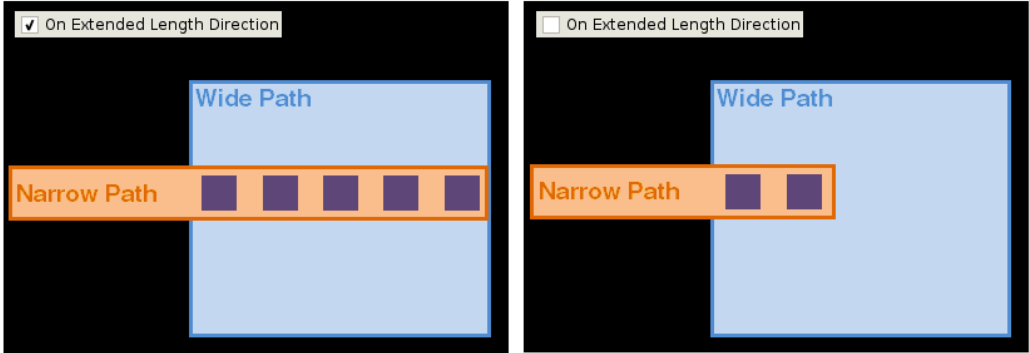
- [General Options](#)
- [Options Tab](#)

- [Rules Tab](#)
- [Layer Table](#)

General Options

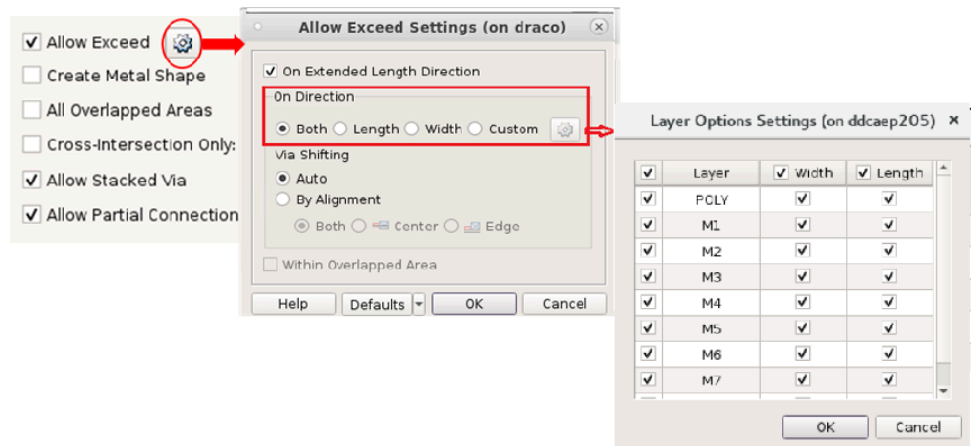
Option	Description
Auto (toolbar) Mode (dialog box)	Specifies the via-creation mode: Auto or Manual . When Auto is selected, you can drag your cursor to define a region on the canvas. The Layout Editor determines via creation and placement. When Manual is selected (default), you can specify the via options and place the via.
Depth	Determines how far the Layout Editor looks for layer overlaps. Select 0x to look only at the top layer. Select View to look at the stop level you set.
Nets	Specifies the assigned nets.
Stacked	(Dialog box only) When enabled, turns on the stacked via mode for standard vias and standard via variants. Only routing layers between the start layer defined by Bottom and the end layer defined by Top are available for stacked via creation. Via definitions defined on layers between bottom and top layers can be used to create stacked vias. When disabled (default), turns off the stacked via mode. Tcl preferences: <code>leViaStackedAuto</code> (for Auto mode) and <code>leViaStackedManual</code> (for Manual mode)
Bottom	(Dialog box only) Bottom displays the start layer for stacked vias. Select Bottom , and then choose a start layer from the valid routing layers. Tcl preferences: <code>leViaStartLayerAuto</code> (for Auto mode) and <code>leViaStartLayerManual</code> (for Manual mode)
Top	(Dialog box only) Top displays the end layer for stacked vias. Select Top , and then choose an end layer from the valid routing layers. Tcl preferences: <code>leViaEndLayerAuto</code> (for Auto mode) and <code>leViaEndLayerManual</code> (for Manual mode)

Options Tab

Option	Description
Only Same Net	This option appears only when you set the <code>leAutoViaSameNetOnlyVisible</code> Tcl preference to <code>true</code>. Creates vias to connect shapes on the same net. Clear the Only Same Net check box to create vias regardless of the connecting shapes' net assignments. Tcl preference: <code>leAutoViaSameNetOnly</code> Click  to select the following: • No Net Matches Any Net: Adds via to connect a shape with no net and a shape that is assigned any net. Tcl preference: <code>leAutoViaSameNetOnlyNoNetAnyNet</code> • No Net Matches No Net: Adds via to connect two shapes with no net. Tcl preference: <code>leAutoViaSameNetOnlyNoNetNoNet</code> • Check Nets in Hierarchy: Considers shapes that have the same net as terminals in lower hierarchy levels. Clear the check box to consider unnetted shapes in lower hierarchy levels as 0x unnetted shapes. Auto Via considers shapes with internal nets in lower hierarchy levels as netted shapes whose nets do not match any 0x net. Tcl preference: <code>leAutoViaCheckHierarchyNet</code>
Allow Exceed	Determines whether the newly created via can exceed the overlapping region. This option does not apply to overlapping regions that form non-Manhattan shapes. Click  to open the Allow Exceed Settings dialog box and set the following: • On Extended Length Direction • On Direction • Via Shifting • Within Overlapped Area Tcl preference: <code>leAutoViaAllowExceed</code>
On Extended Length Direction	(Allow Exceed must be selected) When a narrow shape intersects with a larger shape, extends the narrow shape along its length and adds vias across the intersecting area.  Tcl preference: <code>leAutoViaExtendLengthDirection</code>

Option	Description
--------	-------------

On Direction (Allow Exceed must be selected) A total of four settings, which are controlled by `leAutoViaAllowExceedOnDir` preference, can be specified:

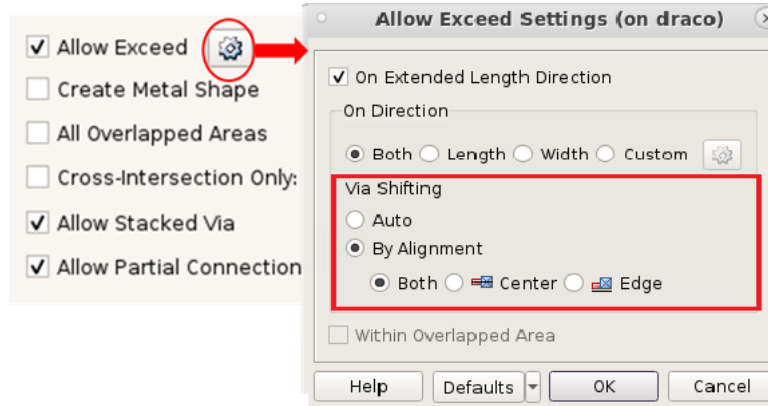


Options:

- **Both**: Allows exceed on both width and length directions.
- **Length**: Allows exceed on length direction but not width direction.
- **Width**: Allows exceed on width direction but not length direction.
- **Custom**: Specifies width and length exceed by layers.

The **Layer Options Settings** dialog box is displayed if you choose **Custom**. You can also modify the preference `leAutoViaAllowExceedOnDirByLayers` directly to update the table in the **Layer Options Settings** dialog.

Option	Description
Via Shifting	(Allow Exceed must be selected) Auto Via supports via alignment types, which creates a shifted and aligned via with user settings. This is mainly to maximize routing resources during place and route stage.



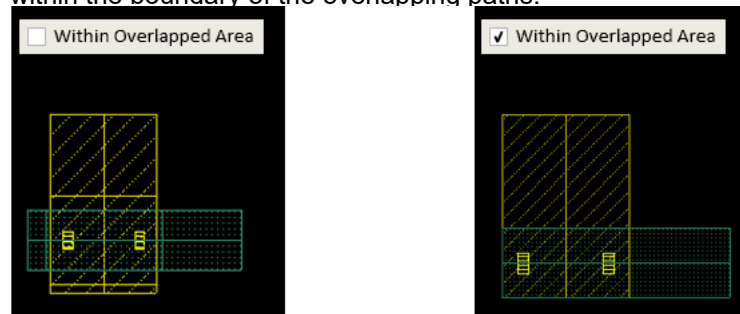
Select one of the following options:

- **Auto**: Selected, by default. Auto Via automatically calculates offset and shifts via.
- **By Alignment**: If selected, Auto Via considers `leAutoViaShiftingAlignmentType` for via creation. If the **Both** option is selected, the **Center** alignment type takes precedence over the **Edge** alignment type. This preference is involved only when layer1 or layer2 of the via enclosure exceeds on wire width direction.

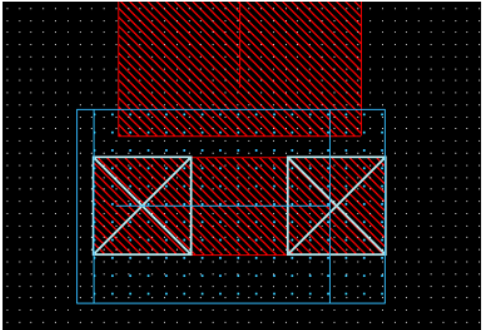
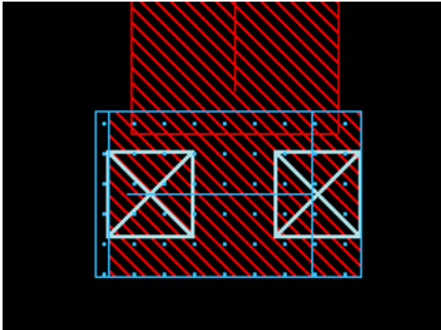
Tcl preference: `leAutoViaShiftingMode`

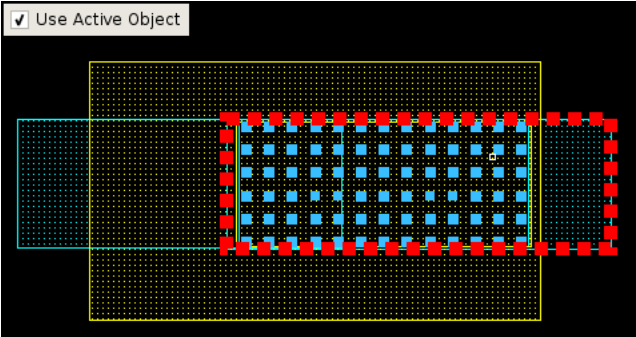
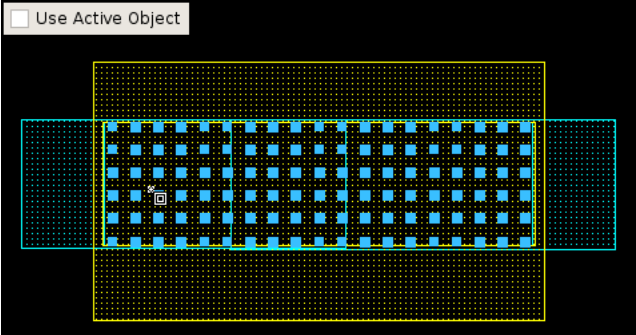
Within Overlapped Area

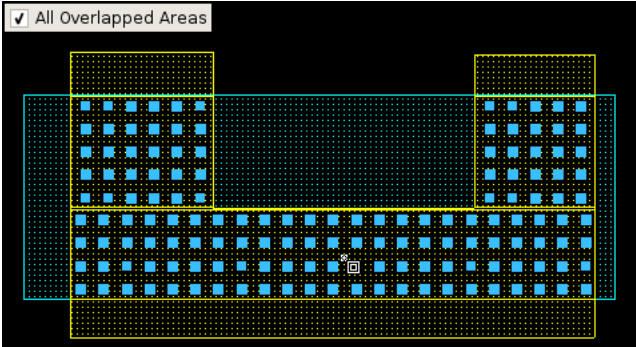
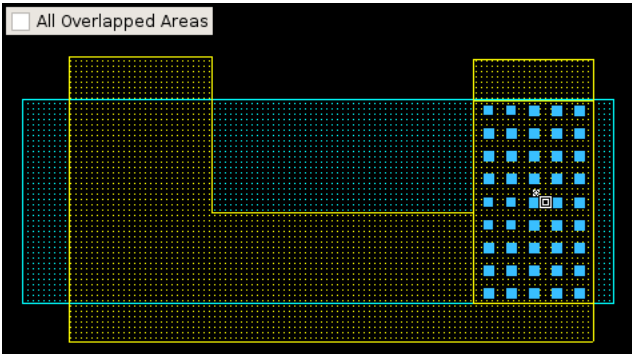
(**Allow Exceed** must not be selected) Ensures that the metal layers of the via remain within the boundary of the overlapping paths.

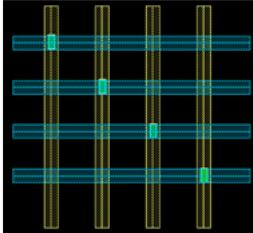
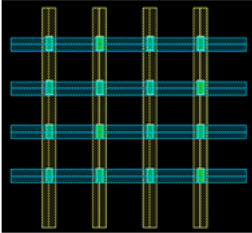


Tcl preference: `leAutoViaWithinOverlappedArea`

Option	Description
Metal Fit to Overlapped Area	(Dialog box only) Fits the metal shapes of the newly created via to fully cover the overlapped region. This option does not apply to overlapping regions that form non-Manhattan shapes. ☐ Metal Fit to Overlapped Area  ☒ Metal Fit to Overlapped Area  In Fusion Compiler, IC Compiler II, and IC Compiler co-design flows, this option does not apply when <code>lePreserveViaDefParameters</code> Tcl preference is <code>true</code> (default in co-design flows). Tcl preference: <code>leViaMetalFitToOverlappedArea</code>
Create Metal Shape	(Dialog box only) Determines whether to create metal shapes for layers without any metal shapes. The shape size is the same as the bounding box of the merged bounding box for each same layer. This option does not apply to overlapping regions that form non-Manhattan shapes. Tcl preference: <code>leViaCreateMetalShapes</code>

Option	Description
Use Active Object	(Dialog box only) Creates vias between overlapping shapes within the boundary of the active object. The default is false.  Clear the check box to create vias between overlapping shapes within the largest possible region under the pointer. The region depends on the All Overlapped Areas setting.  Tcl preference: <code>leViaUseActiveObject</code>. Default is false.

Option	Description
All Overlapped Areas	(Dialog box only) Creates vias between overlapping shapes within the largest possible region under the pointer. This option is available only when you clear the Use Active Object check box.  Clear the check box to create vias between overlapping shapes within the largest rectangular region under the pointer.  Tcl preference: <code>leAutoViaAllOverlappedAreas</code>
Display Runtime	(Dialog box only) Displays the vias on the canvas as you area-select (draw) the region for via creation. Displaying the vias while you draw the region can impact performance, especially when you are creating a large number of vias. Clear the check box to display the vias after you have finished drawing the area.
Allow Stacked Via	(Dialog box only) Creates stacked vias. Clear the check box to turn off stacked-via creation for faster performance, such as when the design contains a large number of overlapping shapes.

Option	Description
Via Pattern	(Dialog box only) Adds vias based on the pattern you select:  Backslash: Cascades the vias to the right.  Slash: Cascades the vias to the left. Select this option if you want to add a large array of vias. To add the vias, drag and draw a rectangular region to indicate where to add the vias. ☐ Via Pattern: ☒ Via Pattern: ☒  Backslash ☐  Slash  This option does not apply to overlapping regions that form non-Manhattan shapes. Tcl preference: <code>leAutoViaUsePattern</code>, <code>leAutoViaPatternStyle</code>
Min Cuts	Adds the minimum the number of cuts that you specify. This option does not apply to overlapping regions that form non-Manhattan shapes. Tcl preference: <code>leAutoViaMinCuts</code>
Directions	(Dialog box only) Specifies the direction to apply to the cuts: Auto (automatically determined), H (horizontal), or V (vertical). Tcl preference: <code>leAutoViaMinCutsDirection</code>
Justification	Specifies the position of the via between the intersecting wires. Note: Justification is ignored for single-sided self-aligned vias. See <code>selfAlignedViaExtension</code>. Tcl preference: <code>leAutoViaJustification</code>

Custom Compiler™ Layout Editor User Guide for Connecting Objects
W-2024.09

Rules Tab

Create Via

Mode: ☒ Auto ☐ Manual

Depth: ☒ Dk ☐ View

Nets:

☐ Stacked Bottom: POLY Top: M1.2

OptionsRules

☒ Environment Aware

☒ Same Cut Layer Spacing

☒ Same Wire Layer Spacing

☒ Forbidden End-to-End Rules

☒ Different Cut Layer Spacing

☒ Different Wire Layer Spacing

☒ Cut And Wire Layer Spacing

☒ Maximum Vias in Stack

☒ Voltage Dependent Rules

☒ Nonvisible Layers

☒ Notification (Forbidden End-to-End Rules)

☐ Create Markers

Via Preset:

☒	Layer	Via Definition	Cut Class	Spacing
☒	POLY	auto	auto	auto
☒	MODIFF2	auto	auto	auto
☒	M1	auto	auto	auto
☒	M2	auto	auto	auto
☒	M3	auto	auto	auto

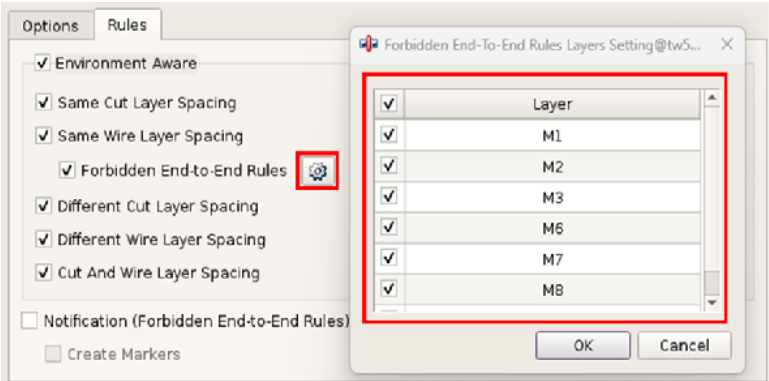
Help

Hide

Defaults

Cancel

Option	Description
Environment Aware	Uses the Create Via (Auto) command to check the surrounding area for rule compliance during via creation. No vias are added if violations would result from the create via operation. You must select one or more of the following rules: • Same Cut Layer Spacing (minViaSpacing, minCutClassSpacing, allowedCutSpacing, generalCutClassSpacing, and so on) • Same Wire Layer Spacing (allowedSpacingRange, minVoltageSpacing, minSpacing, and so on) • Forbidden End-to-End Rules (allowedEndToEndSpacing) Click  to open the Forbidden End-to-End Rules Layers Settings dialog box.



This option enables or disables a specific layer for checking forbidden end-to-end rules.
The default is true.
If the option is false, the corresponding layer does not check the forbidden end-to-end rules.

Note:

- The option appears only if you set the `_SYNOPTSYS_CUSTOM_ALLOW_IGNORE_FORBIDDEN_ETE` environment variable.
- The **Forbidden End-to-End Rules Layers Settings** dialog box only displays the metal layers that have `allowedEndToEndSpacing` constraint defined in the technology file.
- The `leAutoViaIgnoreForbiddenETERulesLayers` preference controls every layer.
The preference value can be either empty or a string consisting of keywords separated by a whitespace.
For example:

```
db::setPrefValue
leAutoViaIgnoreForbiddenETERulesLayers -value "M1
M2"
```

To reset the preference, you can specify an empty string:

```
db::setPrefValue
leAutoViaIgnoreForbiddenETERulesLayers -value ""
```

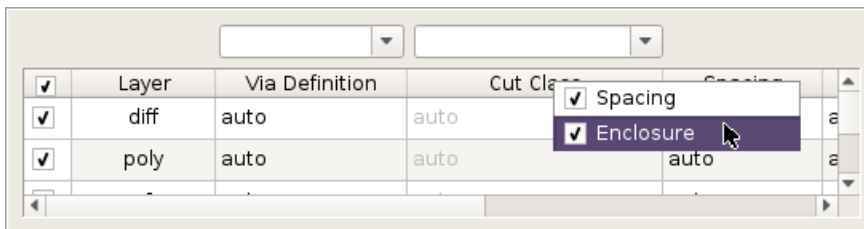
- Different Cut Layer Spacing (minViaClearance, minCutClassClearance, minClearance, and so on)
- Different Wire Layer Spacing (minVoltageClearance, minClearance, and so on)
- Cut and Wire Layer Spacing (minCutRoutingClearance, minClearance, and so on)
- Maximum Vias in Stack (viaStackLimit)
- Voltage Dependent Rules (minVoltageSpacing, minVoltageClearance, minCutRoutingClearance, and so on)

If you want the automatic via creation operation to consider shapes on invisible layers, select **Involves Invisible Layers**.

Option	Description
Notification (Forbidden End-to-End) Rules	When the option is selected, Auto Via displays the message for forbidden end-to-end rules on the console. The supported rules and message are: • Rule - <code>allowedEndToEndSpacing</code> • Message Type - Information • Message - The enclosures of some vias are extended because of forbidden end-to-end rules. Create Markers allow Auto Via to create markers and set the severity of the markers as <code>Information</code> on the extended enclosures after commit vias.
Create Markers	When the option is selected, Auto Via creates markers with severity as Info on the extended enclosures after commit vias when the enclosures of the vias are extended because of <code>allowedEndToEndSpacing</code>. Details of the marker: • Short Message: Extend enclosure • Long Message: Extend enclosure because of forbidden end to end rules

Layer Table

To display additional columns in the table, right-click the table header, and select the columns you want to add to the table: **Spacing, Enclosure**.



Option	Description																				
Filter boxes (above the Layer table)	Filters the list of vias and cut classes based on the criteria you provide. The filters you specify are remembered within the same design session. By default, the predefined auto filter is used. However, you can change the criteria by typing one or more strings in the box. Use a character space to separate the strings. For example: bar diff po After you type the filter strings, press Enter. The list of names that meet the criteria appear in the column. An ellipse (...) in a result indicates that multiple definitions match the filter criteria. To view the definitions, rest the pointer on the result. bar auto X <table><tr><th>☒</th><th>Layer</th><th>Via Definition</th><th>Cut Class</th></tr><tr><td>☒</td><td>M4</td><td>M4_M5xHBAR</td><td>auto</td></tr><tr><td>☒</td><td>M5</td><td>M5_M6xHBAR ...</td><td>auto</td></tr><tr><td>☒</td><td>M6</td><td>M6_M7xHBAR</td><td>M5_M6xHBAR M5_M6xVBAR</td></tr><tr><td>☒</td><td>M7</td><td>M7_M8xHBAR ...</td><td>auto</td></tr></table> The filter results depends on the Filter setting you selected in the General Options dialog box.	☒	Layer	Via Definition	Cut Class	☒	M4	M4_M5xHBAR	auto	☒	M5	M5_M6xHBAR ...	auto	☒	M6	M6_M7xHBAR	M5_M6xHBAR M5_M6xVBAR	☒	M7	M7_M8xHBAR ...	auto
☒	Layer	Via Definition	Cut Class																		
☒	M4	M4_M5xHBAR	auto																		
☒	M5	M5_M6xHBAR ...	auto																		
☒	M6	M6_M7xHBAR	M5_M6xHBAR M5_M6xVBAR																		
☒	M7	M7_M8xHBAR ...	auto																		
Layer	(Dialog box only) Displays only the bottom layer. You can specify whether to enabled each bottom layer. By default, all the bottom layers are enabled for via creation. If Stacked mode is enabled, the table only shows the layers between bottom and top layers.																				
Via Definition	(Dialog box only) Lists via, via variant, and bridge via definitions. A via variant name contains the name of the via definition on which it is based. The definitions are sorted in alphabetical order. The first name in the list is the default via. This list might include oaViaDefArray objects. Selecting a predefined oaViaDefArray from the list is the only way to place an array of vias (a regular mosaic). You can select the available definitions by clicking on the field. The list includes an <i>auto</i> definition when Auto mode is enabled.																				

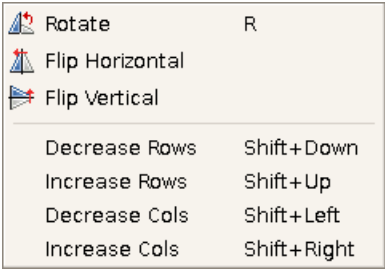
Option	Description
Cut Class	(Dialog box only) Lists cut shapes defined in your technology database. You can select the available cut classes by clicking on the field. The list includes an <i>auto</i> cut class when Auto mode is enabled. When Manual non-stacked mode is enabled, you can modify the parameters of the selected via in the Parameters table. When Manual stacked mode is enabled, you can modify the spacing and the enclosure values for Layer1 and Layers by clicking on the field or by entering the desired values.
Spacing	Spacing between cuts in standard and custom built-in vias. The spacing applies to neighboring cut shapes that have positive parallel run lengths. You can specify one value, which applies to both the X and Y spacing directions. Alternatively, you can specify two values, the first value applies to the X direction and the second value applies to the Y direction. The spacing values you provide must be larger than the design-rule values. If the spacing values you provide create violations, the Layout Editor will automatically correct them. For all standard vias, if the <code>leViaHonorViaDefDefaultSpacing</code> Tcl preferences is <code>true</code>, the Layout Editor will automatically correct spacing values that are smaller than the defined cut spacing value. If a via variant is placed, the Layout Editor will ensure the placed via variant complies with the cut spacing value you specify and the via variant cut spacing value.
Layer <i>n</i> Enc	Via enclosure specifications. You can specify one, two, or four values: • One value specifies that the same distance is applied on all four sides of the enclosure. • When you specify two values, the first value is applied to the left and right sides of the enclosure, the second value applies to the top and bottom sides of the enclosure. • When you specify four values, the values are applied to the left, right, top, and bottom sides in the order encountered.

Via Shortcut Menu and Shortcut Keys

While the **Create > Via** command is active, you can right-click to display the Create Via shortcut menu. This section provides a quick reference of the shortcut menu and shortcut keys. For instructions on how to create individual vias, see [Creating Individual Vias Manually](#) or [Creating Individual Vias Automatically](#).

Note:

This menu is not available for manual stacked-via creation.



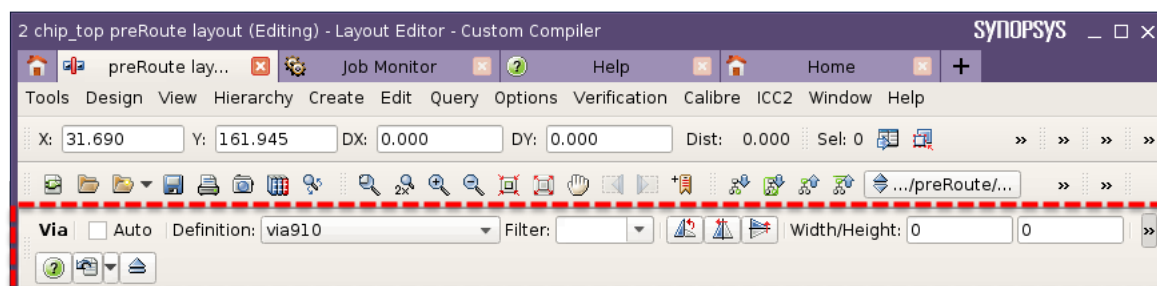
Command	Purpose	Default Shortcut Key
Rotate	Rotates the via 90° counter-clockwise. To rotate 180° or 270°, repeat the command.	R
Flip Horizontal	Mirrors the via around the Y-axis.	Shift-right-click
Flip Vertical	Mirrors the via around the X-axis.	Ctrl+right+click
Decrease Rows	Decreases the number of via rows by 1.	Shift+Down arrow
Increase Rows	Increases the number of via rows by 1.	Shift+Up arrow
Decrease Cols	Decreases the number of via columns by 1.	Shift+Left arrow
Increase Cols	Increases the number of via columns by 1.	Shift+Right arrow

Creating Arrays of Vias

You can interactively define the size of a via array and simultaneously place the array by dragging your cursor across the canvas. You can create arrays of vias either automatically or manually.

To define and place via arrays:

1. Choose **Create > Via**. The Via toolbar appears. The mouse pointer changes from  to .



2. (Optional) On the toolbar, click Eject  or press F3 to display the **Create Via** dialog box. The dialog box contains additional options.

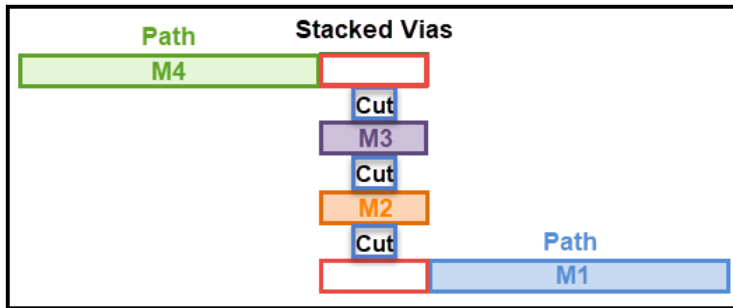
To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

3. Specify whether you want to create the via array automatically or manually:
 - On the toolbar, select or clear the **Auto** check box.
 - In the dialog box, select **Auto** or **Manual**.
4. On the layout canvas, draw a rectangular region in which you want to create the via array.
5. When you are finished, press Esc to quit the command.

Creating Stacked Vias

Stacked vias are multiple vias placed on top of one another. You use stacked vias to connect paths that are multiple layers apart.

In the following example, stacked vias connect the M1 path to the M4 path.



This topic describes the following:

- [Creating Stacked Vias Manually Within Shape Boundary](#)
- [Creating Stacked Vias Manually Within Specified Region](#)
- [Creating Stacked Vias Automatically](#)
- [Via Toolbar and Create Via Dialog Box Reference \(Manual Stacked Mode\)](#)

For information about creating stacked vias automatically while stretching interconnects, see [Stretching Interconnects and Adding Vias Automatically](#).

Creating Stacked Vias Manually Within Shape Boundary

You can create stacked vias within the shape boundary of an object using the Layout Editor **Create > Via** command in the **Manual** mode. The stacked vias can be standard vias or via variants.

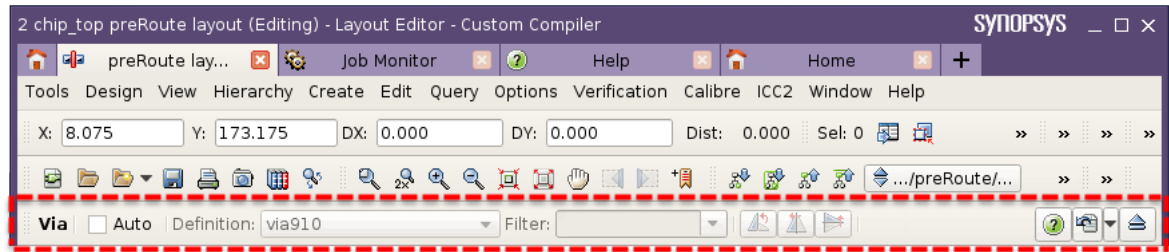
Only the following are supported during manual stack creation within shape boundaries:

- Top-level (0x) shapes
- Visible Nx shapes that are at or above the Stop Level you set.
- Manhattan shapes

Stacked vias are created as long as the placement complies with constraints and does not cause shorts. Results of the placement are also impacted by constraints and short avoidance.

To create stacked vias manually within the area of a shape:

1. In the Layout Editor, choose **Create > Via**. The **Via** toolbar appears.



2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Create Via** dialog box.

Create Via

Mode: ☐ Auto ☒ Manual

☒ Stacked Bottom: M1 Top: M12

☒ Specify: ☐ Cols/Rows: 1 1 ☒ From Shape ☐ By Region ☒ Rectangle ☐ Polygon

Options

☐ Same Metal Size ☐ Create Metal Shape

☐ Trace Same-Layer Shapes

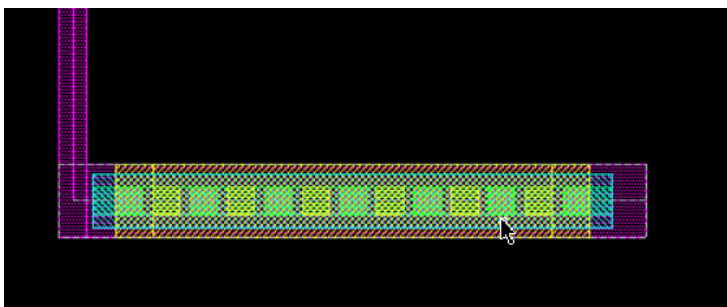
Via Definition	Cut Class	Spacing	Layer1 Enc	Layer2 Enc
M1_M2x	default	0.04 0.04	0.019 0	0 0.019
M2_M3x	default	0.04 0.04	0 0.019	0.019 0
M3_M4x	default	0.04 0.04	0.019 0	0 0.019
M4_M5x	default	0.04 0.04	0 0.019	0.05 0.011
M5_M6x	default	0.075 0.075	0.05 0	0 0.05
M6_M7x	default	0.075 0.075	0 0.05	0.05 0
M7_M8x	default	0.075 0.075	0.05 0	0 0.05
M8_M9x	VxSML(0.044 0.044)	0.075 0.075	0 0.05	0.1 0.025
M9_M10x	VxSML(0.45 0.45)	0.45 0.45	0.1 0.025	0.025 0.1
M10_M11x	VxSML(0.45 0.45)	0.45 0.45	0.025 0.1	0.1 0.025
M11_M12x	VxSML(0.45 0.45)	0.45 0.45	0.1 0.025	0.025 0.1

Help Hide Defaults Cancel

3. In the dialog box, select **Mode: Manual** and **Stacked**.
4. Choose the **Bottom** and **Top** layers for the stacked via. The shape must be on the Bottom or Top layer.
5. Select **Specify** and **From Shape**.
6. Select applicable **Options**. See [Via Toolbar and Create Via Dialog Box Reference \(Manual Stacked Mode\)](#).

The options you specify can affect the stacked via results. For example, selecting **Trace Same-Layer Shapes** shapes lets the command trace and consider shapes that are on the same layer as and touch the active shape.

7. (Optional) Modify the **Spacing** and enclosure values for Layer1 and Layer2 (**Layer1 Enc**, **Layer2 Enc**). See [Via Toolbar and Create Via Dialog Box Reference \(Manual Stacked Mode\)](#).
8. On the layout canvas, click the shape. The via is created within the area of the shape. The number of cut rows and columns are automatically determined.



9. When you are finished, press Esc to quit the command.

Creating Stacked Vias Manually Within Specified Region

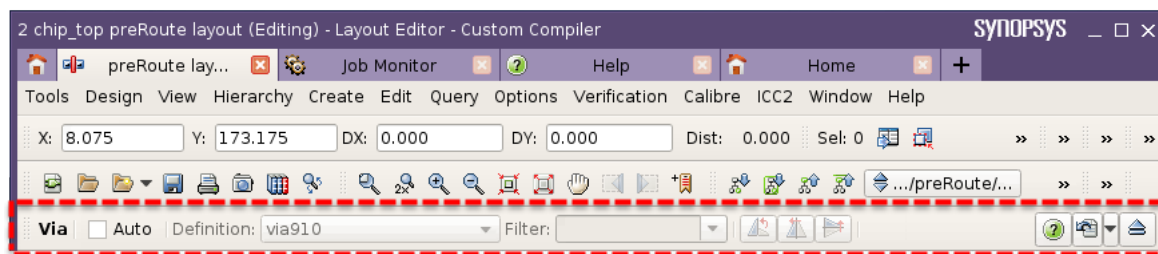
You can create vias within a specified region on selected objects using the Layout Editor **Create > Via** command in the **Manual** mode. The stacked vias can be standard vias or via variants.

Only the following are supported during manual via creation within a specified region:

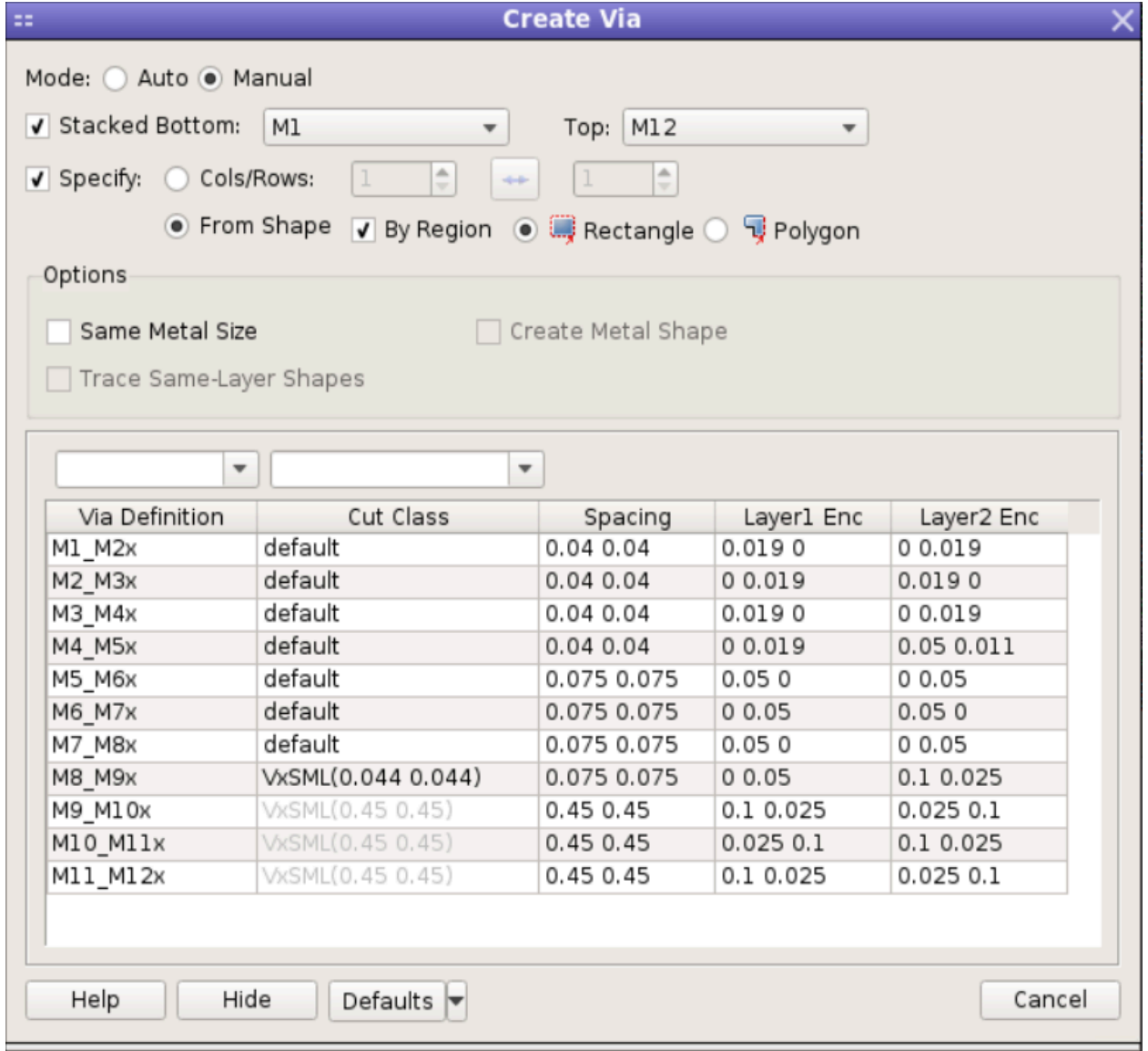
- Top-level (0x) shapes
- Manhattan shapes

To create stacked vias manually within a specified region:

1. In the Layout Editor, choose **Create > Via**. The **Via** toolbar appears.



2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Create Via** dialog box.



The **Create Via** dialog box is shown with the following settings:

- Mode:** ☐ Auto ☒ Manual
- Stacked Bottom:** M1 **Top:** M12
- Specify:** ☒ ☐ Cols/Rows: 1 ☐ From Shape ☒ By Region ☐ Rectangle ☐ Polygon
- Options:**
 - ☐ Same Metal Size ☐ Create Metal Shape
 - ☐ Trace Same-Layer Shapes
- Via Definition Table:**

Via Definition	Cut Class	Spacing	Layer1 Enc	Layer2 Enc
M1_M2x	default	0.04 0.04	0.019 0	0 0.019
M2_M3x	default	0.04 0.04	0 0.019	0.019 0
M3_M4x	default	0.04 0.04	0.019 0	0 0.019
M4_M5x	default	0.04 0.04	0 0.019	0.05 0.011
M5_M6x	default	0.075 0.075	0.05 0	0 0.05
M6_M7x	default	0.075 0.075	0 0.05	0.05 0
M7_M8x	default	0.075 0.075	0.05 0	0 0.05
M8_M9x	VxSML(0.044 0.044)	0.075 0.075	0 0.05	0.1 0.025
M9_M10x	VxSML(0.45 0.45)	0.45 0.45	0.1 0.025	0.025 0.1
M10_M11x	VxSML(0.45 0.45)	0.45 0.45	0.025 0.1	0.1 0.025
M11_M12x	VxSML(0.45 0.45)	0.45 0.45	0.1 0.025	0.025 0.1

Buttons: Help, Hide, Defaults, Cancel

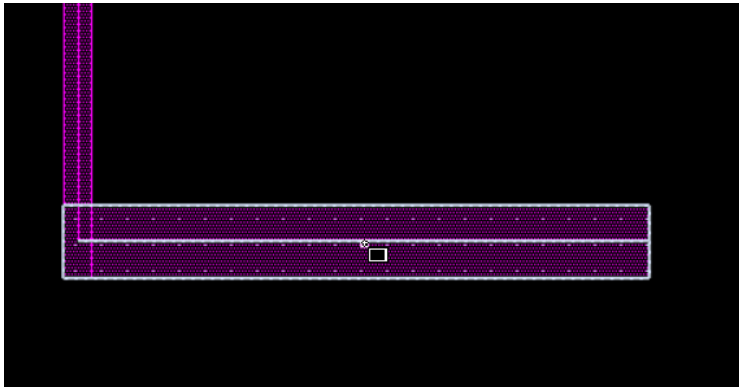
3. In the dialog box, select **Mode: Manual** and **Stacked**.
4. Choose the **Bottom** and **Top** layers for the stacked via.
5. Select **Specify**, **From Shape**, and **By Region**.

6. Select one of the following:
 - **Rectangle:** Creates vias within the rectangular region that you draw on the canvas.
 - **Polygon:** Creates vias within the polygon region you draw on the canvas.
7. (Optional) Set other **Options** in the dialog box. See [Via Toolbar and Create Via Dialog Box Reference \(Manual Stacked Mode\)](#).
8. (Optional) Modify the **Spacing** and enclosure values for Layer1 and Layer2 (**Layer1 Enc, Layer2 Enc**). See [Via Toolbar and Create Via Dialog Box Reference \(Manual Stacked Mode\)](#).
9. On the layout canvas, select the shapes for which you want to create the stacked vias.

To select multiple shapes, Shift+click the shapes. The shape boundaries are highlighted.

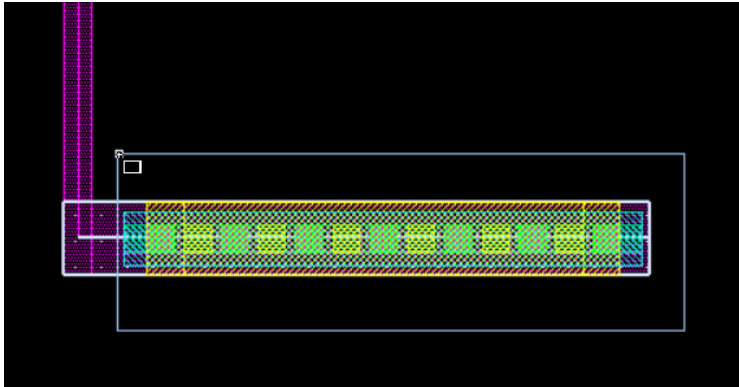
Note:

The multiple shapes that you select must be on the same layer. The layer must be between the **Bottom** and **Top** layers.



10. On the layout canvas, draw the rectangular or polygon region within which you want to place the stacked vias.

In the following example, the rectangular region is being drawn on the canvas, and the vias are displayed.

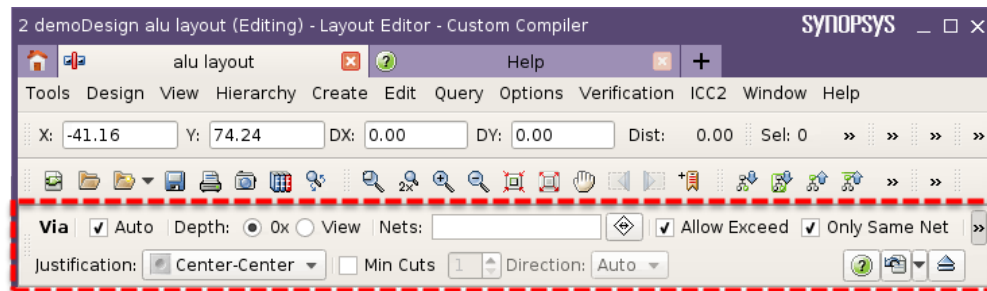


11. Click the canvas or press Enter to commit the vias.
12. When you are finished, press Esc to exit the command.

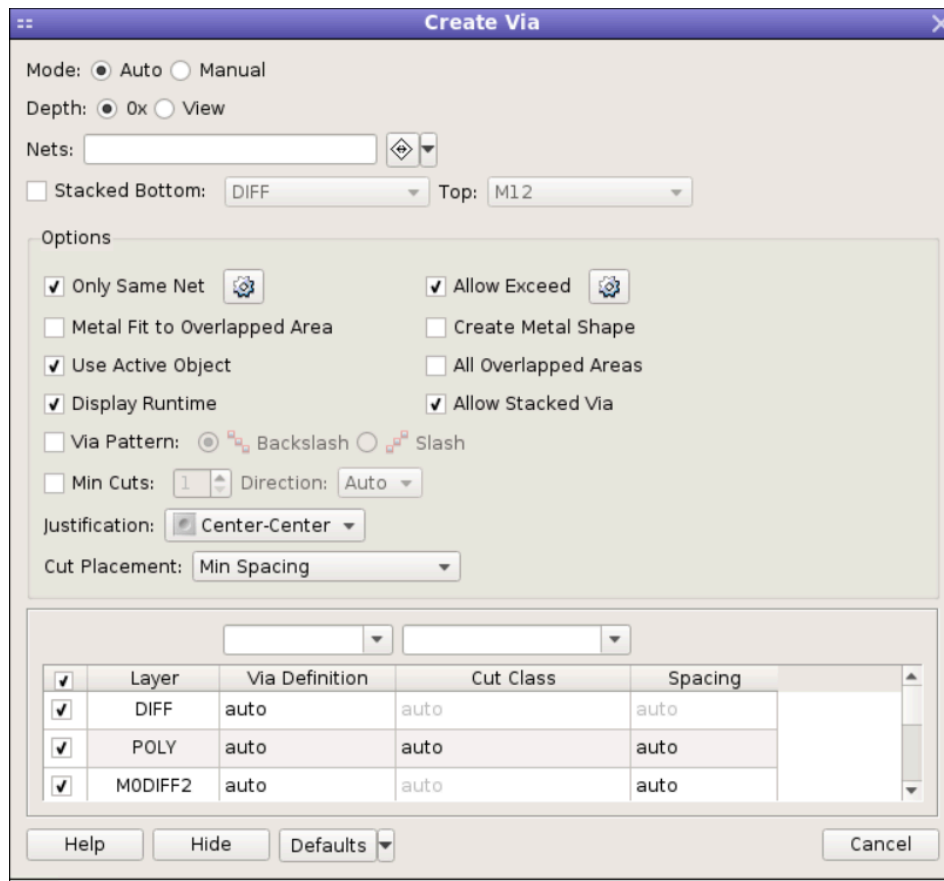
Creating Stacked Vias Automatically

To create stacked vias (standard vias or via variants) automatically:

1. Choose **Create > Via**. The Via toolbar appears.



2. (Optional) On the toolbar, click **Eject**  or press F3 to display the **Create Via** dialog box.

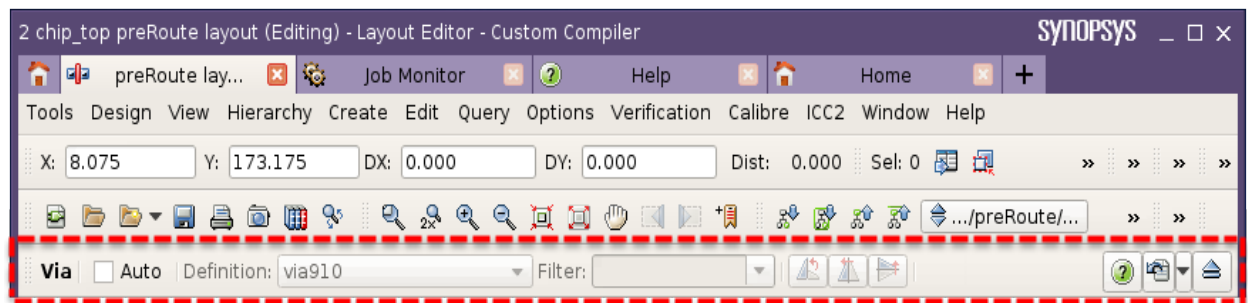


3. In the dialog box, select **Auto** and **Stacked**.
4. Choose the **Bottom** and **Top** layers for the stacked via.
5. (Optional) Set other **Options** in the dialog box. See [Via Toolbar and Create Via Dialog Box Reference \(Auto Mode\)](#).
6. On the layout canvas, do one or more of the following:
 - Click the overlapping shapes between which you want to create vias.
 - Draw a rectangular region to enclose overlapping shapes. Stacked vias will be created between overlapping shapes.
7. When you are finished, press Esc to quit the command.

Via Toolbar and Create Via Dialog Box Reference (Manual Stacked Mode)

This section provides a quick reference of the **Via** toolbar and the **Create Via** dialog box in the **Manual** mode, when the **Stacked** option is turned on. For instructions on how to create stacked vias manually, see [Creating Stacked Vias Manually Within Shape Boundary](#) and [Creating Stacked Vias Manually Within Specified Region](#).

For the quick reference of the toolbar and dialog box in the **Auto** mode (stacked and non-stacked vias), see [Via Toolbar and Create Via Dialog Box Reference \(Auto Mode\)](#).



Create Via

Mode: ☐ Auto ☒ Manual

☒ Stacked Bottom: M1

Top: M12

☒ Specify: ☐ Cols/Rows: 1

1

☒ From Shape ☒ By Region ☒ Rectangle ☐ Polygon

Options

☐ Same Metal Size ☐ Create Metal Shape

☐ Trace Same-Layer Shapes

Via Definition	Cut Class	Spacing	Layer1 Enc	Layer2 Enc
M1_M2x	default	0.04 0.04	0.019 0	0 0.019
M2_M3x	default	0.04 0.04	0 0.019	0.019 0
M3_M4x	default	0.04 0.04	0.019 0	0 0.019
M4_M5x	default	0.04 0.04	0 0.019	0.05 0.011
M5_M6x	default	0.075 0.075	0.05 0	0 0.05
M6_M7x	default	0.075 0.075	0 0.05	0.05 0
M7_M8x	default	0.075 0.075	0.05 0	0 0.05
M8_M9x	VxSML(0.044 0.044)	0.075 0.075	0 0.05	0.1 0.025
M9_M10x	VxSML(0.45 0.45)	0.45 0.45	0.1 0.025	0.025 0.1
M10_M11x	VxSML(0.45 0.45)	0.45 0.45	0.025 0.1	0.1 0.025
M11_M12x	VxSML(0.45 0.45)	0.45 0.45	0.1 0.025	0.025 0.1

Help

Hide

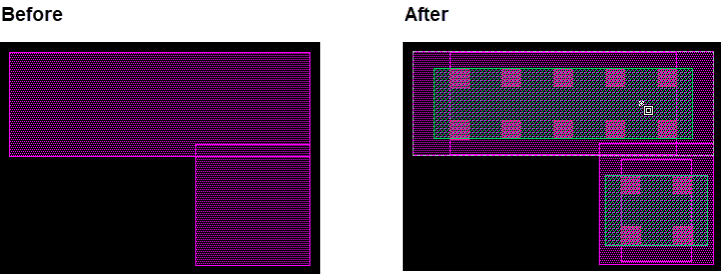
Defaults

Cancel

Options	Description
Mode	Specifies the via-insertion mode: Auto or Manual. When Auto is selected, you can drag your cursor to define a region on the canvas. The Layout Editor determines via creation and placement. When Manual is selected (default), you can specify the via options and place the via.

Options	Description
Stacked	Turns on the stacked via mode for standard vias and standard via variants. Only routing layers between the start layer defined by Bottom and the end layer defined by Top are available for stacked via creation. Via definitions defined on layers between bottom and top layers can be used to create stacked vias. When disabled (default), turns off the stacked via mode. Tcl preferences: <code>leViaStackedAuto</code> (for Auto mode) and <code>leViaStackedManual</code> (for Manual mode)
Stacked: Bottom	Bottom displays the start layer for stacked vias. Select Bottom, and then choose a start layer from the valid routing layers. Tcl preferences: <code>leViaStartLayerAuto</code> (for Auto mode) and <code>leViaStartLayerManual</code> (for Manual mode)
Stacked: Top	Top displays the end layer for stacked vias. Select Top, and then choose an end layer from the valid routing layers. Tcl preferences: <code>leViaEndLayerAuto</code> (for Auto mode) and <code>leViaEndLayerManual</code> (for Manual mode)
Specify: Cols/Rows	Cols specifies the number of via columns for standard vias. For custom vias, the value is fixed at 1. Rows specifies the number of via rows to create for standard vias. The tool creates equal numbers of rows and columns, or one more row than the number of columns, whichever more closely matches the <code>minNumCut</code> rule. You can switch the Cols and Rows values using the Swap Number Between Column and Row  button. Tcl preferences: <code>leViaManualStackedSpecifyMode</code>, <code>leViaCols</code>, and <code>leViaRows</code>
Specify: From Shape	Lets you create a via within the area of a shape (top or bottom). The number of cut rows and columns are automatically determined. Tcl preference: <code>leViaManualStackedSpecifyMode</code>
Specify: By Region	Lets you create vias within the area that you specify: • Rectangle: Creates vias within the rectangular region that you draw on the canvas. • Polygon: Creates vias within the polygon region you draw on the canvas.
Options: Same Metal Size	Determines whether the newly created via has the same geometry as metal shapes. In stacked vias, all metal shapes for different vias have the same geometry based on the largest shape among the vias. Tcl preference: <code>leViaSameMetalSize</code>

Options	Description
Options: Trace Same-Layer Shapes	Treats connected shapes as a single shape, and adds the via within the area of the single shape. This options is available only if you are in the Create Via Manual mode, and you select Specify: From Shape .



Tcl preference: `leViaTraceSameLayerShapes`

Options: Create Metal Shape	Determines whether to create metal shapes for layers without any metal shapes. The shape size is the same as the bounding box of the merged bounding box for each same layer. Tcl preference: <code>leViaCreateMetalShape</code>
--------------------------------	---

Filter boxes (above the Layer table)	Filters the list of vias and cut classes based on the criteria you provide. The filters you specify are remembered within the same design session. To specify the filter criteria, type one or more strings in the box. Use a character space to separate the strings. For example:
---	--

bar diff po

After you type the filter strings, press Enter. The list of names that meet the criteria appear in the column. An ellipse (... , not shown) in a result indicates that multiple definitions match the filter criteria. To view the definitions, rest the pointer on the result.

bar

X

bar

Via Definition	Cut Class
none	none
PO_M1x	BAR(0.022 0.04)
M0DIFF2_M1xHBAR	none
M1_M2xHBAR	VxBAR(0.022 0.05)
M2_M3xHBAR	VxBAR(0.022 0.05)
M3_M4xHBAR	VxBAR(0.022 0.05)
M4_M5xHBAR	VxBAR(0.022 0.05)
M5_M6xHBAR	VxBAR(0.044 0.088)
M6_M7xHBAR	VxBAR(0.044 0.088)
M7_M8xHBAR	VxBAR(0.044 0.088)
M8_M9x	VxBAR(0.044 0.088)

The filter results depends on the **Filter** setting you selected in the **General Options** dialog box. See Setting General Options.

Options	Description
Via Definition	Lists via, via variant, and bridge via definitions. A via variant name contains the name of the via definition on which it is based. The definitions are sorted in alphabetical order. The first name in the list is the default via. This list might include <code>oaViaDefArray</code> objects. Selecting a predefined <code>oaViaDefArray</code> from the list is the only way to place an array of vias (a regular mosaic). You can select the available definitions by clicking on the field.
Cut Class	Lists cut shapes defined in your technology database. You can select the available cut classes by clicking on the field. The list includes an <i>auto</i> cut class when Auto mode is enabled. When Manual non-stacked mode is enabled, you can modify the parameters of the selected via in the Parameters table. When Manual stacked mode is enabled, you can modify the spacing and the enclosure values for Layer1 and Layers by clicking on the field or by entering the desired values.
Spacing	The X and Y distance between the cuts.
Layer1 Enc Layer2 Enc	The amount to extend Layer1 or Layer 2 beyond the cut edges in the X and Y directions.

Rules	Description
Mode	Specifies the via-insertion mode: Auto or Manual. When Auto is selected, you can drag your cursor to define a region on the canvas. The Layout Editor determines via creation and placement. When Manual is selected (default), you can specify the via options and place the via.
Depth	0x; View
Nets	
Stacked: Bottom	Bottom displays the start layer for stacked vias. Select Bottom, and then choose a start layer from the valid routing layers. Tcl preferences: <code>leViaStartLayerAuto</code> (for Auto mode) and <code>leViaStartLayerManual</code> (for Manual mode)
Environment Aware	

Rules	Description
Same Cut Layer Spacing	
Different Cut Layer Spacing	
Different Wire Layer Spacing	
Cut and Wire Layer Spacing	
Maximum Vias in Stack	
Layer	
Via Definition	
Cut Glass	
Spacing	

Stretching Vias

This topic describes the following:

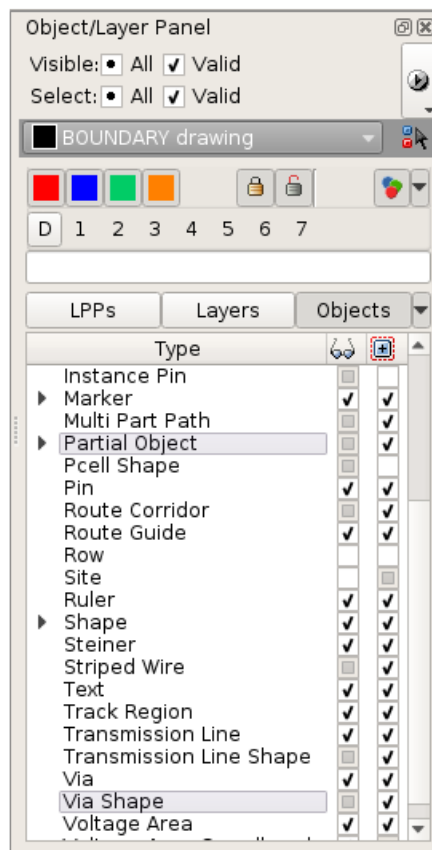
- [Stretching 1x Shapes in Standard Vias](#)
- [Stretching 1x Shapes in Built-in Custom Vias](#)

Stretching 1x Shapes in Standard Vias

You can directly stretch 1x shapes in standard vias at the top level after making the **Via Shape** selectable in the Object/Layer Panel.

To stretch 1x Layer1 and Layer2 shapes from the via master at the top level:

1. In the Object/Layer Panel, click the **Objects** tab, and enable the selectability of **Partial Object** and **Via Shape**.



2. Choose **Edit > Stretch > Stretch**.
3. On the **Stretch** toolbar, select **Via Stretch Method: Cuts and Enclosure** or **Best-Fit**.
For information about other available options, see Stretch Toolbar and Dialog Box.

The Best-Fit mode always uses minimum enclosure rules to calculate the number of cut columns and rows when you stretch vias in either the vertical or horizontal direction.



Original	Cuts and Enclosure	Best-Fit
	Stretch	
	Enclosure on the two sides are not equal	Enclosure on both sides are equal

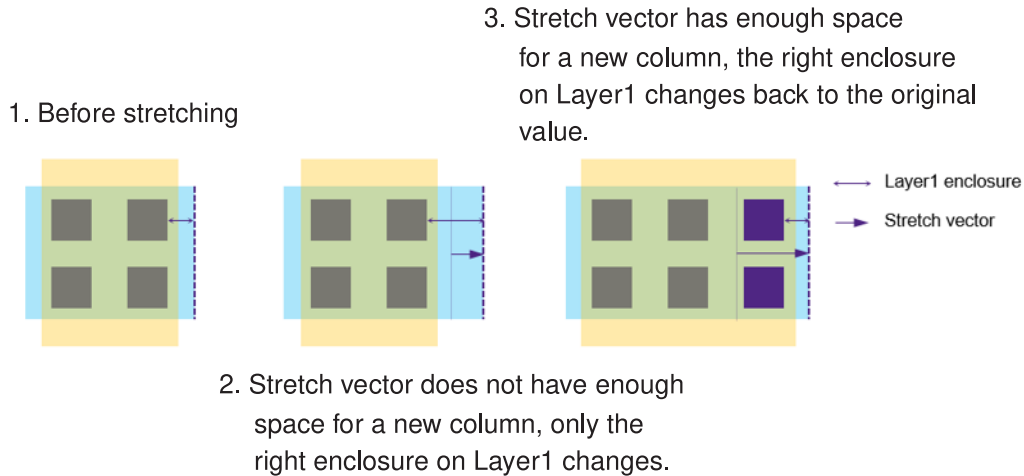
If the `leManualViaCalculateLayerWidth` Tcl preference is false (default), the wire widths are adjusted to match the stretched via metal layer widths during best-fit via stretch operations.

Constraint	Before Stretch	After Stretch
<pre>(minDualExtension m1 v1 (((0.005 0) (0.14 0) (0.205 0.07)) ((0.005 0) (0.14 0) (0.205 0.07))))</pre>		

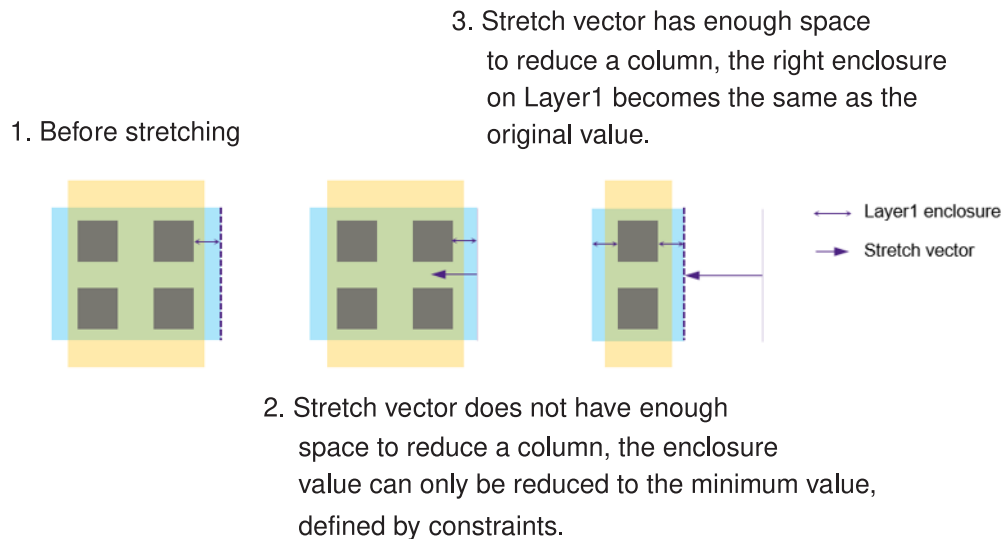
4. Select the 1x shape you want to stretch.

Only partial selection is supported. You can select both vertices and edges on layer1 and layer2 shapes but not on cut layer shapes.

In the example below, you select an edge of the Layer1 shape and stretch it to right. If the stretch vector has enough space to grow a new column, the tool changes the number of cuts and preserves the right enclosure value on Layer1.



When there are enclosure constraints defined, the enclosure values can be reduced to the minimum value defined by constraints. When reaching the minimum value, further stretching makes no change on the via until the stretch vector has enough space to reduce a column. In the example below, if the original right enclosure on Layer1 is greater than the constraint defined minimum value, stretching to the left can reduce the enclosure at first until the minimum value is reached. The via will not be changed until the stretch vector has enough space to reduce one column.



When the number of columns or rows reaches 1 in the stretching direction, the tool only reduces the enclosure values until a constraint defined minimum value is reached.

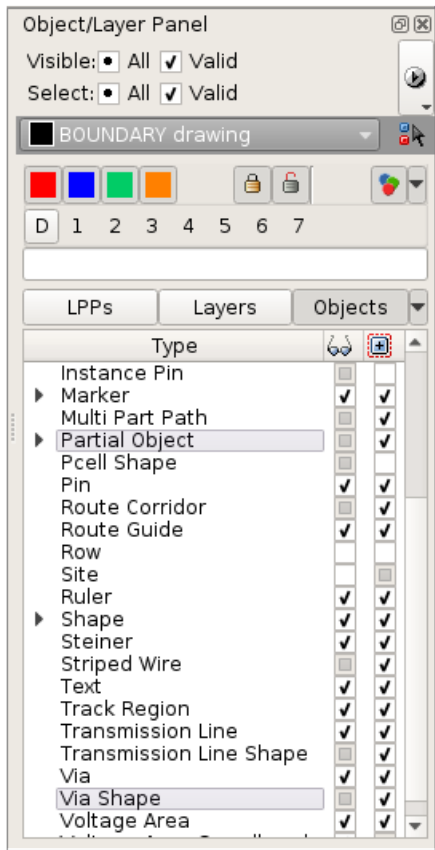
Stretching a vertex can result in changes in one direction or two depending on the angle mode. You can also stretch Layer1 and Layer2 shapes at the same time, in which case the enclosure values on both layers may change together.

Stretching 1x Shapes in Built-in Custom Vias

You can directly stretch 1x shapes in built-in custom vias at the top level after making the **Via Shape** selectable in the Object/Layer Panel.

To stretch 1x Layer1 and Layer2 shapes from the via master at the top level:

1. In the Object/Layer Panel, click the **Objects** tab, and enable the selectability of **Partial Object** and **Via Shape**.



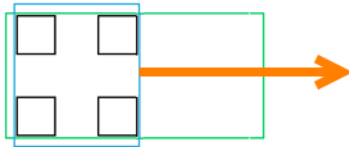
2. Choose **Edit > Stretch > Stretch**.

3. On the **Stretch** toolbar, set the **Via Stretch Method**. For more information, see Stretch Toolbar and Dialog Box.
4. Select the 1x shape you want to stretch.

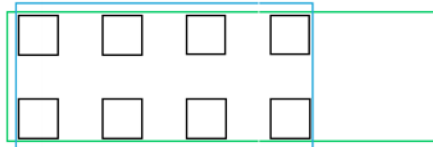
Only partial selection and the orthogonal angle mode are supported. You can select edges on Layer1 and Layer2 shapes but not on cut shapes. Cut shapes are automatically added or removed according to the editing results.

In the example below, you stretch an edge of a Layer2 shape to the right. Changes of the cut number and the Layer2 enclosure depends on the Via Stretch Method, as illustrated in the following figure. When **Cut Number** is selected, the tool increases the cut number while the Layer2 enclosure remains unchanged. When **Both** is selected, the tool changes both the cut number and the Layer2 enclosure after stretching.

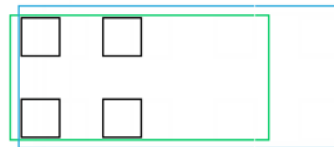
Stretching Layer2



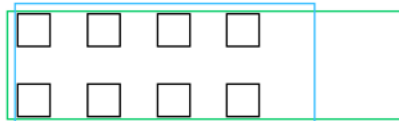
Via Stretch Method: Cut Number



Via Stretch Method: Enclosure

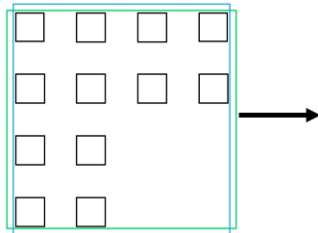


Via Stretch Method: Both

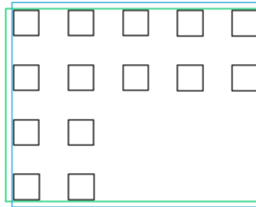


In the example below, you stretch an edge of a Layer1 shape to the right. Changes of the cut number and the Layer1 enclosures depends on the Via Stretch Method, as illustrated in the following figure.

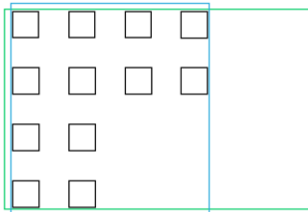
Stretching Layer1



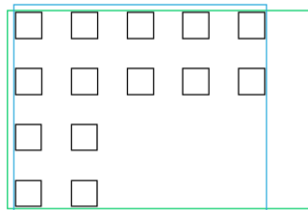
Via Stretch Method: Cut Number



Via Stretch Method: Enclosure



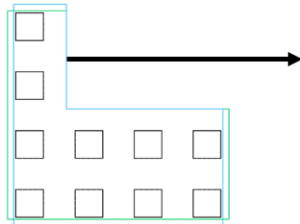
Via Stretch Method: Both



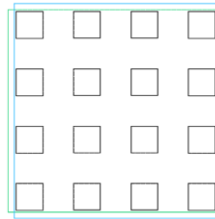
In the example below, you stretch an edge of a chopped Layer1 shape to the right. Changes of the cut number and the Layer1 enclosures depends on the Via Stretch Method, as illustrated in the following figure. When **Cut Number** is selected, the

tool heals the via by reverting the cut number and Layer1/Layer2 enclosures. When **Enclosure** is selected, the tool heals the via till the Layer1 enclosure becomes a rectangle after stretching.

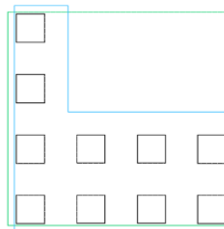
Stretching Layer1



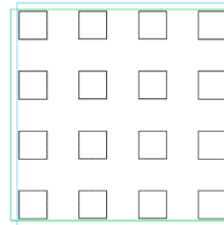
Via Stretch Method: Cut Number



Via Stretch Method: Enclosure



Via Stretch Method: Both



Replacing Vias

This section includes the following topics:

- [About Replacing Vias](#)
- [Using the Replace Via Command](#)
- [Replace Via Toolbar](#)

For information on using the `ile::replaceVia` Tcl command, see “Layout Editing Tcl” in the *Custom Compiler Tcl Reference Manual*.

About Replacing Vias

You can use the **Query > Replace Via** command to replace standard vias with a via using a user-specified number of cuts, while keeping the interconnect footprint and obstacles.

There are 5 possible solutions to grow the via during replacement: in place, up, down, left, and right. While keeping the footprint and avoiding obstacles, the Layout Editor chooses the direction on the basis of connecting paths. If one direction is not possible due to obstacles, the tool will try other directions one by one.

The column and row numbers of cuts depend on the grow direction. For the case of double cuts and triple cuts, if growing vertically, the row number is 1 and the column number is 2; while if growing horizontally, the row number is 2 and the column number is 1. For the case of 4 cuts, there could be two combinations: 1x4 (4x1) or 2x2. In order to keep the footprint, 1x4 is the first choice; otherwise, it is 2x2.

If there is no solution with the user-specified cut number due to obstacles, then the tool reduces the cut number by 1 and tries again. It is a recursive process until a solution can be found. If no solution can be found even for double cuts, the replacement fails with an error message.

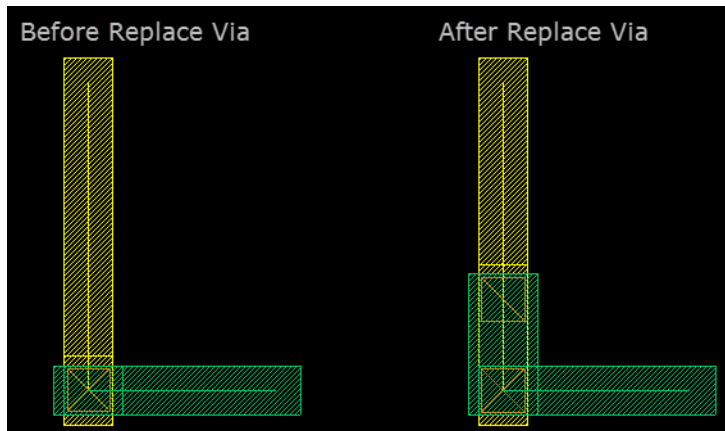
The tool keeps the via origin so the origin offset is adjusted accordingly. Changing the column and row numbers may require a change in the cut spacing due to the minimum adjacent spacing constraint. Other via parameters may also be changed to make the resulting via DRC clean.

The following sections describe several cases for via replacement:

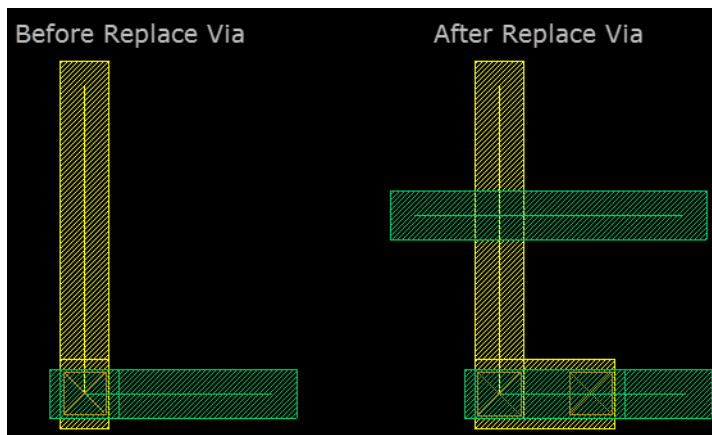
- [L-connection](#)
- [T-connection](#)
- [Other Cases](#)

L-connection

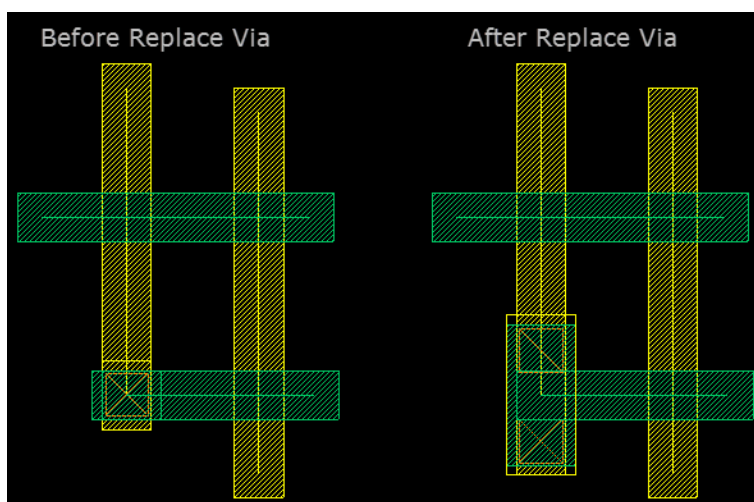
If two connected paths form an L-connection, the tool grows the via in the direction of the longer path first.



If the tool fails due to obstacles, the tool grows the via in the direction of another path.

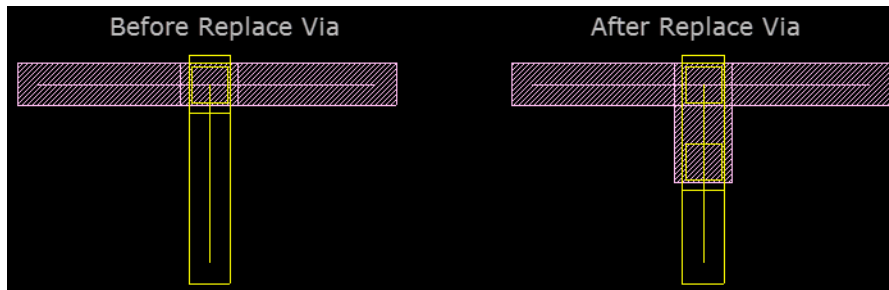


If there are more obstacles, the tool will try to create the vias in the opposite direction from the shorter path, and then in the opposite direction from the longer path.

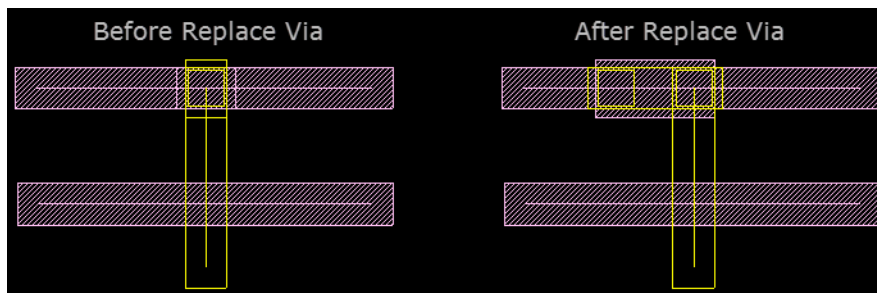


T-connection

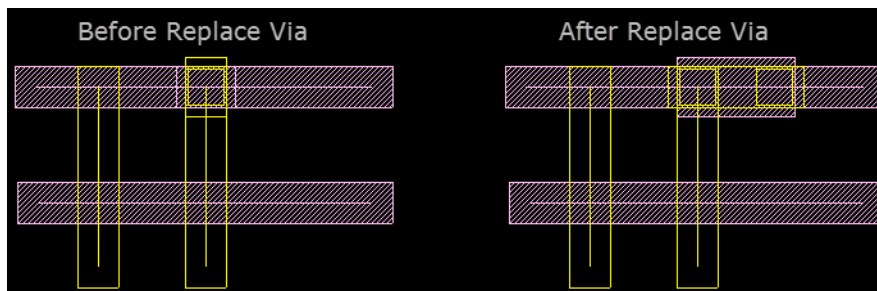
If two connected paths form a T-connection, the tool grows the via in the direction of the branch path first.



If the tool fails because of obstacles, then it grows in the direction of the trunk path.



If there are obstacles on the left and bottom, the tool grows the via in the direction of the trunk path to the right.



Other Cases

For other cases, the tool usually grows the via in place. If it is not possible to do so, the tool will then try the other 4 directions one by one until a solution can be found.

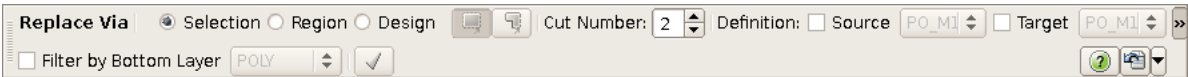
Using the Replace Via Command

To replace a single cut via into multiple cuts:

1. In the Layout Editor, choose **Query > Replace Via**. The [Replace Via Toolbar](#) appears.
2. On the layout canvas, select the standard via that you want to replace. If you preselected a standard via, the command recognizes it as the via object you want to replace.
3. Specify the number of cuts in the **Cut Number** box.
4. Click Apply  or press Enter.
5. When you are finished, press Esc to quit the command.

Replace Via Toolbar

This section provides a quick reference of the **Replace Via** toolbar. For instructions on replacing vias, see [Using the Replace Via Command](#).



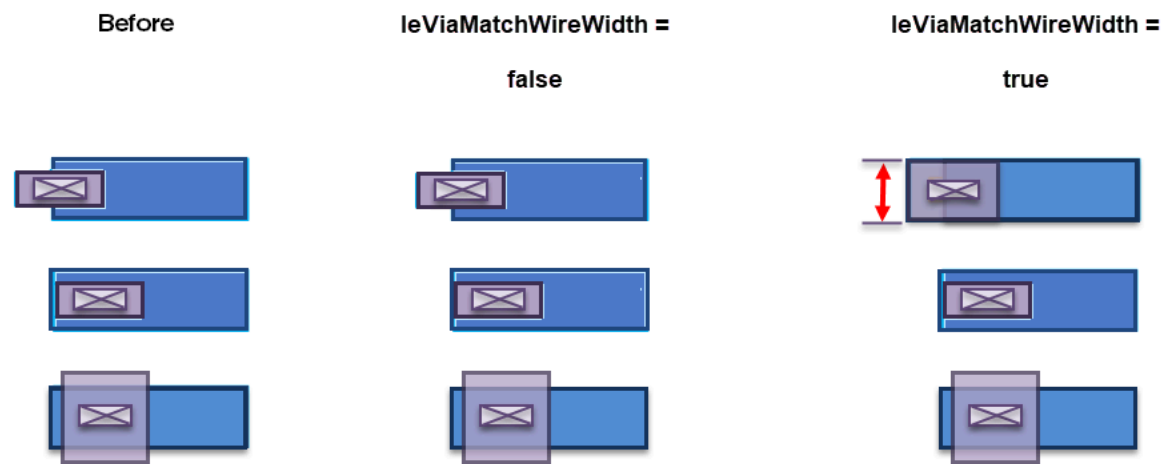
Option	Description
Mode	Specifies the replacement mode. • Selection (default): Works on the basis of selection. You need to select vias to replace. • Region: Works on the basis of a drawn region. The tool considers any standard vias that are covered by the region as the candidates for replacement. • Design: Works on the basis of the whole design. The tool considers any standard vias within the design as the candidates. Tcl preference: <code>leReplaceViaMode</code>
Region Type	Enables when Region is selected. You can specify to draw a rectangular (default) or polygonal area for replacement. Tcl preference: <code>leReplaceViaRegionType</code>
Cut Number	Defines the number of cuts. The default value is 2. Tcl preference: <code>leReplaceViaCutNum</code>
Source	Defines whether to specify the source via definition. (Off by default.) When Source and Selection are enabled, you can only select standard vias with that particular definition. Tcl preferences: <code>leReplaceViaSpecifySourceDef</code>, <code>leReplaceViaSourceDef</code>

Option	Description
Target	Defines whether to specify the target via definition. (Off by default.) When enabled, defines the target via definition for replacement. Tcl preferences: <code>leReplaceViaSpecifyTargetDef</code> , <code>leReplaceViaTargetDef</code>
Filter by Bottom Layer	Defines whether to filter source and target definitions using the bottom layer. (Off by default.) When enabled, the tool only shows the via definitions whose bottom layer matches this option in the Source and Target definition fields. Tcl preferences: <code>leReplaceViaFilterByBottomLayer</code> , <code>leReplaceViaBottomLayer</code>
Apply	Completes the operation.
Defaults	Resets all of the options to your Custom Compiler environment default values.

Adjusting Vias

You can set the `leViaMatchWireWidth` Tcl preference to `true` to expand the width of the via's metal layers (layer 1 and layer 2) to match the wire width. The adjustment occurs only if both of the following conditions are met:

- The via's metal layers (layer 1 and layer 2) are narrower than the wire.
- The via protrudes past the end of the wire.



Modifying Via Attributes and Properties

Note:

To freeze vias and prevent them from being modified, see [Freezing and Unfreezing Vias](#).

You use the Property Editor to modify via attributes and properties after placement.

In the Property Editor, when different custom via types have common parameters and properties with the same names, values, and value types, you have the option of displaying them in the **All Custom Vias** tab in the **Show Multiple-Item View** . Doing so allows you to modify the common parameters and properties from a central location. To display the common parameters in the **All Custom Vias** tab, set the `dePropertyEditorShowCommonAllRefs` Tcl preference to `true`.

By default, when you adjust any of the via parameters or the via definition, the Layout Editor checks supported constraints and considers overlapped shapes. You have the option of setting the `leEditViaParamConstraintAwareMode` Tcl preference to check constraints without considering overlapped shapes or ignore constraint checks.

To modify via attributes and properties:

1. Display the Property Editor.
2. On the layout canvas, select the via you want to modify. The via information appears in the Property Editor.

Note:

When displaying the default values of custom via parameters, the Property Editor first checks the CDF parameters before the PCell parameters.

Property Editor

Design: Layout

Current StdVia M1_M2x

Attributes

Prompt	Value
Definition Name	M1_M2x
Origin	(4.376,6.4565)
Orientation	R0
Route Status	normal
Route Type	User Entered
M1 Color	No Color
VIA2 Color	No Color
M2 Color	No Color
Via Ladder	☐

Connectivity

Prompt	Value
Net Name	
Anchor	☐

Parameters

Constraint Aware: Consider Context

Prompt	Value
Columns/Rows	(1,1)
Spacing	(0.04,0.04)
Cut Class	VxSML(0.022 0.022)
M1 Enclosure (L/R)	(0.019,0.019)
M1 Enclosure (T/B)	(0,0)
	Swap M1 Enclosure
M2 Enclosure (L/R)	(0,0)
M2 Enclosure (T/B)	(0.019,0.019)
	Swap M2 Enclosure
Origin Offset	(0,0)

Properties

Name	Value
Click to add	

Property Editor

Constraint Editor

Object/Layer Panel

3. In the **Attributes** section, change desired attribute values:

Attribute	Description
Definition Name	The name of the via, via variant, or bridge via definition. A via variant name contains the name of the via definition on which it is based. Choose a via definition or via variant from the list. Alternatively, click  to filter the list of definitions. Via, via variant, and bridge via definitions are set up in the technology database. For more information, see the Via Definitions Section in the <i>Custom Compiler Technology and Display Resource File Reference Manual</i>.
Origin	The location of the via (x, y).
Orientation	The orientation of the selected object. Choose one of the following: ◦ R0: Does not rotate the object. ◦ R90: Rotates the object counterclockwise by 90°. ◦ R180: Rotates the object counterclockwise by 180°. ◦ R270: Rotates the object counterclockwise by 270°. ◦ MY: Flips the object along the y-axis. ◦ MYR90: Flips the object along the y-axis, and then rotates the via counterclockwise by 90°. ◦ MX: Flips the object along the x-axis. ◦ MXR90: Flips the object along the x-axis, and then rotates the via counterclockwise by 90°. ◦ : Rotates the object counterclockwise by 90°. ◦ : Flips the object along the y-axis. ◦ : Flips the object along the x-axis.
Route Status	Indicator of whether the route can be modified: ◦ normal: The route or via can be modified. ◦ fixed: The route or via can be modified manually using the Edit commands or by direct manipulation on the canvas. The route or via cannot be modified by an application such as the router. ◦ locked: The route cannot be modified.
Route Type	How the route or via is treated by the Fusion Compiler or IC Compiler router. For information about these route types, see the Fusion Compiler user documentation or the IC Compiler user documentation. Note: A Custom Route route type indicates that the route shapes were added by the Custom Router.

Attribute	Description
<Layer_name> Color	The color assignments of the layer above the via, the cut layer, and the layer below the via. Click  to lock the color assignment. The lock prevents automatic changes, such as during color propagation.
<Via_layer> Color	
<Layer_name> Color	
Via Ladder	(Fusion Compiler or IC Compiler II designs only) The vias are stacked, starting from the pin layer and up to the layer at which the router connects.

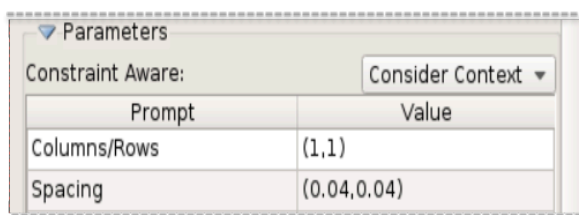
4. In the **Connectivity** section, change the desired connectivity information:

Connectivity Info	
Net Name	The name of the net on which the route or via is located.
Anchor	The indicator that the route or via is a connectivity anchor. The anchor controls the net propagation behavior. For more information, see Connectivity Anchor .

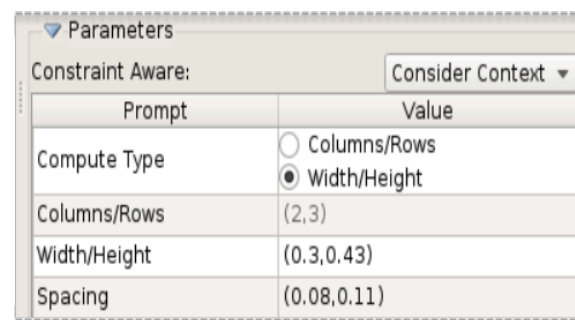
5. In the **Parameters** section, change the parameter values.

If you set `lePropertyEditorShowViaWidthHeight` to `true`, the **Compute Type** selections appear.

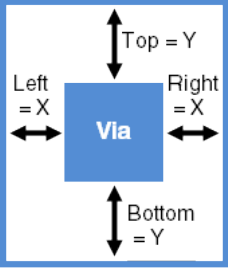
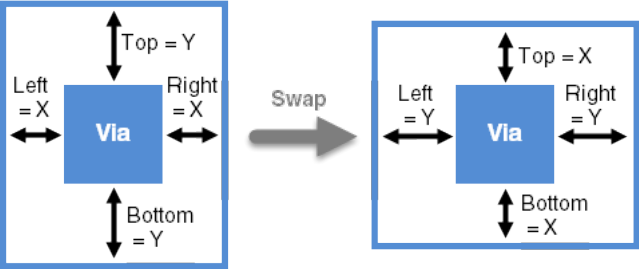
`lePropertyEditorShowViaWidthHeight = false`



`lePropertyEditorShowViaWidthHeight = true`



Parameter	Description
Constraint Aware	Specifies whether to consider the via's surroundings and rule compliance when determining the parameter values: ◦ Off: The Layout Editor does not adjust the vias automatically. You can enter values even if the values might cause DRC violations. ◦ Ignore context: The Layout Editor computes the parameters as if the via is placed in free space in the design. The Layout Editor does not consider the via's immediate surroundings (or context). ◦ Consider context: The Layout Editor computes the parameters based on the location of the via. The via's immediate surroundings (or context) are considered for adjusting the parameters to ensure DRC-clean results. Tcl preference: <code>leEditViaParamConstraintAwareMode</code>
Compute Type	(If <code>lePropertyEditorShowViaWidthHeight</code> is true) Specifies the method to use to determine the number of vias to place: ◦ Columns/Rows: Enables the Columns/Rows field. ◦ Width/Height: Enables the Width/Height field.
Columns/Rows	The number of columns and rows of cut shapes in the via. To swap the Columns and Rows values, click one of the values and click the Swap Columns and Rows Values  button that appears between the Columns and Rows values.
Width/Height	(If <code>lePropertyEditorShowViaWidthHeight</code> is true) The dimensions of the area in which you will add standard vias. The rows and columns of vias will be calculated automatically. Type the width (x value) in the first box. Type the height (y value) in the second box. Both values are in μm.
Spacing	The X and Y distance between the cuts. If the via contains multiple cut arrays, Spacing is the distance between the arrays. By default, the cut array spacing values are based on the <code>minLargeViaArraySpacing</code> constraint.
Cut Class	The standard via's dimensions. Choose one from the list. Cut Class is only visible if one or more cut classes are defined in the constraints section of the technology database. For more information, see <code>cutClass</code> in the <i>Custom Compiler Technology and Display Resource File Reference Manual</i>. Cut Class is also displayed in the Attributes section (Step 3). If you change the Cut Class value in the Parameters section, the Cut Class value in the Attributes section automatically updates.

Parameter	Description
Layer1 Enclosure Layer2 Enclosure	The amount to extend Layer1 or Layer 2 beyond the cut edges in four directions: left (L), right (R), top (T), and bottom (B). By default, the Layer2 Enclosure values are derived from the Layer1 Enclosure values.  (Standard vias only) To transpose the X and Y values of the layer enclosure, click Swap <layer> Enclosure. The button is available only if all standard vias share the same enclosure type. 
Origin Offset	The X and Y distance from the design origin.
Cut LPP	The layer-purpose pair of the cut layer.

- In the **Properties** section, change the desired property values. You can also add a new property or delete an existing property.
- When you are finished, click **Apply Changes**  to save the changes.

Freezing and Unfreezing Vias

When a design nears completion, you might want to fix certain components to prevent them from being re-evaluated or modified each time the design is opened. This topic describes how to freeze vias so that the via parameters cannot be modified. To freeze PCells, see Enabling PCell Freezing.

The ways you can freeze and unfreeze vias are:

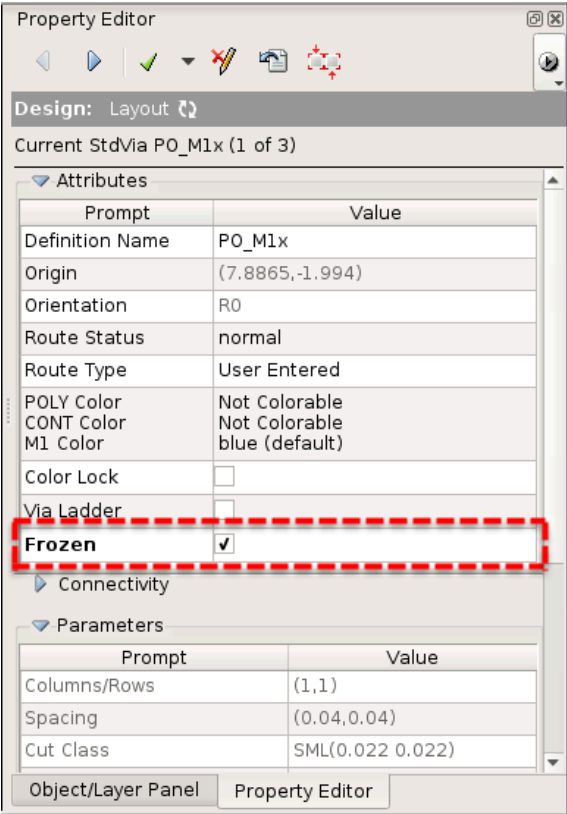
- [Freezing and Unfreezing Vias Using the Property Editor](#)
- [Freezing and Unfreezing Vias Using Tcl Commands](#)

Freezing and Unfreezing Vias Using the Property Editor

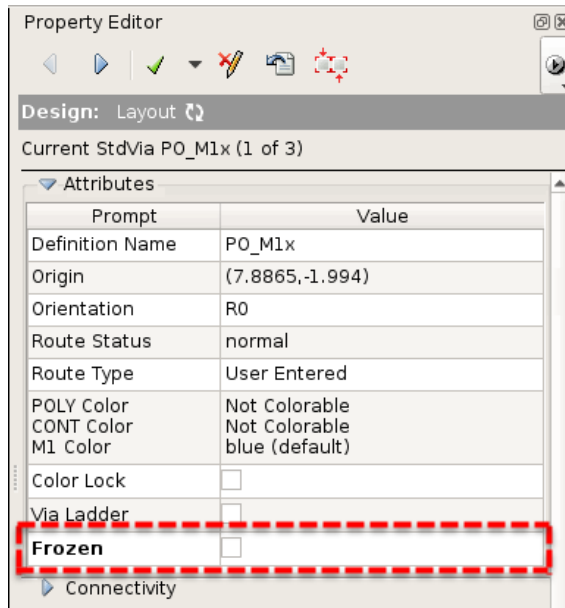
To freeze or unfreeze vias using the Property Editor:

1. Set the `dbPcellFreezeEnabled` Tcl preference to `true` to display the **Frozen** check box in the Property Editor.
2. Display the Property Editor.
3. On the layout canvas, select the via or vias you want to modify. The information about the vias appear in the Property Editor.
4. In the **Attributes** section, select **Frozen**.

The **Origin** and **Orientation** settings become gray, indicating that they cannot be modified. In the **Parameters** section, the parameter names and values also become gray, indicating that they cannot be modified.



5. To unfreeze the via or vias, clear the **Frozen** check box. To modify the parameters, see [Modifying Via Attributes and Properties](#).



Freezing and Unfreezing Vias Using Tcl Commands

To freeze or unfreeze vias using Tcl commands:

- In the Console, type the following commands:

```
set via [de::getSelected -design [ed]]
db::setAttr via.object.frozen -value <true | false>
db::getAttr via.object.frozen (value <true | false>)
db::getAttr via.object.header.frozen (value <true | false>)
```

Creating and Editing Built-In Custom Vias

This topic describes the following:

- [Creating and Deleting Built-In Custom Vias](#)
- [Modifying Built-in Custom Via Attributes and Properties](#)
- [Editing Built-in Custom Vias](#)
- [Chopping Operations That Result in Built-in Custom Vias](#)

- [Healing Built-in Custom Vias](#)
- [Defining Cut LPPs for Built-In Custom Vias](#)

Creating and Deleting Built-In Custom Vias

A built-in custom via is a custom via that has extended functions to support advanced features such as chopping and coloring.

To create built-in custom vias, use the `db::generateBuiltInVias` Tcl command. Because built-in custom vias are based on standard via definitions, the newly created built-in via names have the same name base as the original standard vias. For example, if the original standard via name is M1_M2, the corresponding built-in custom via name would be M1_M2_c.

To delete built-in custom vias, use the `db::deleteBuiltInVias` Tcl command.

You can modify a built-in custom via's properties using the Property Editor, and edit the via object using some of the available **Edit** commands, such as **Chop**.

Modifying Built-in Custom Via Attributes and Properties

You can select and query a built-in custom via or its 1x shapes and modify the attributes and properties using the Property Editor. For example, you can use this method to change a built-in custom via to a standard or custom via.

Except for the via definition, the attributes and properties of built-in custom vias are similar to those of standard vias. The custom via definition name contains a system-reserved suffix `_c`. For example, the tool creates a built-in custom via M1_M2_c based on its original standard via M1_M2. When querying a built-in custom via, the tool lists all vias including the referenced technology database in the Definition attribute list. For help on modifying vias, see [Modifying Via Attributes and Properties](#).

Built-in custom vias contain PCell parameters for custom via definition and CDF parameters of high level variation of PCells. Changing parameters of built-in custom vias in the Property Editor modifies the PCell parameters of built-in custom vias.

Editing Built-in Custom Vias

This section lists ways you can edit built-in custom via objects.

- You can use the **Stretch** command (in orthogonal mode only) to stretch or align the Layer1/Layer2 shapes within a built-in custom via with any of the via stretch method. See [Stretching 1x Shapes in Built-in Custom Vias](#).
- You can use the **Chop** command to chop Layer1, Layer2, and cut layer within a built-in custom vias. The default choppable LPP includes Layer1 and Layer2. For help on setting choppable layers, see Chop Toolbar.

When you select multiple vias at 0x level, the choppable layers of each via are shown in the **Choppable Layers** dialog box. Choppable layers of vias that are beyond 1x will not be included in the dialog box. You can specify choppable layers using the **Chop Layer** button on the **Chop** toolbar. See [Chopping Operations That Result in Built-in Custom Vias](#).

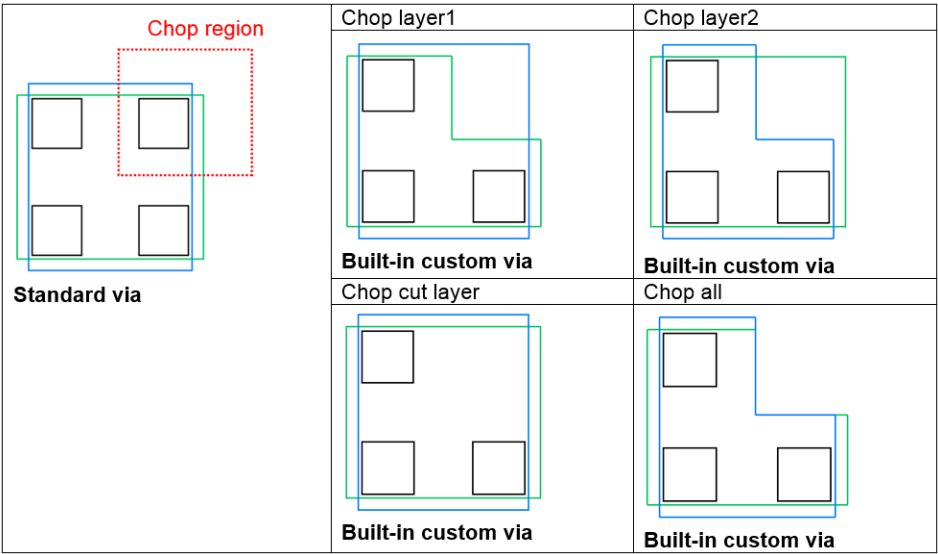
- You can use the **Heal** command to heal Layer1, Layer2, and cut layer within a built-in custom via with rectangle or polygon modes. See [Healing Built-in Custom Vias](#).
- You can use the Constraint Manager to edit constraints. Built-in custom vias should not be listed in the viaDef when you edit the oaViaDefArray and oaViaTopologyArray values.

Chopping Operations That Result in Built-in Custom Vias

When you chop a standard via, if any of the unsupported actions occurs (assuming the built-in custom via has been defined), the chop operation will result in a built-in custom via. The following actions are not supported for standard vias:

- Only chop the cut layer
- The chop will result in non-rectangular Layer1/Layer2 shapes or removing cuts from a full cut array (full row * column cut array)

The following diagram shows some examples when you chop the Layer1, Layer2, and cut layer shapes of a standard via within the chop region, the chop results in non-rectangular shapes or cut removal, thus the tool converts it to a built-in custom via after chopping. Parameters on the standard via are completely converted to the built-in custom via. The tool removes extra cuts if the resultant via violates the layer to cut enclosure value.

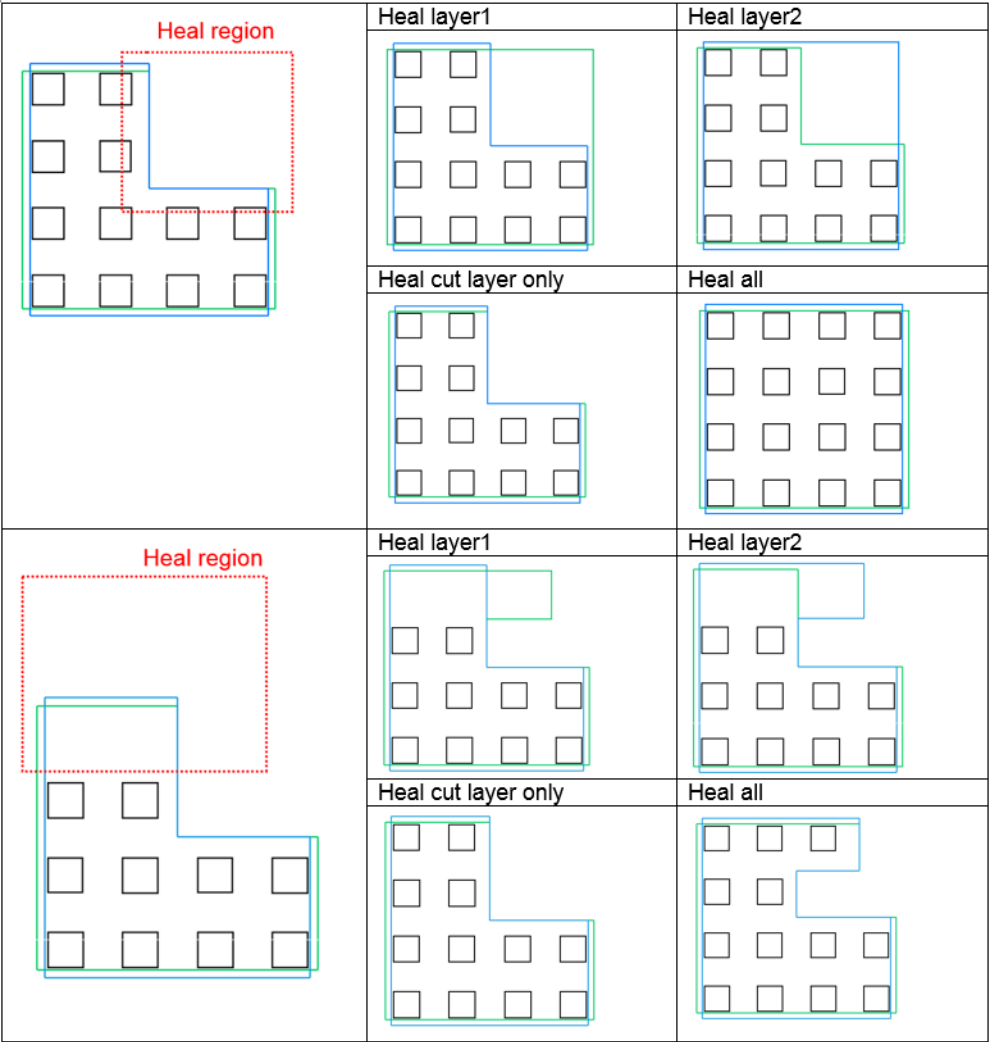


You can chop Layer1, Layer2, and cut layer shapes in built-in custom vias using all angle modes and shape types (such as rectangle, polygon, or polyline) for chopping. If no choppable layer is specified, no action will be performed on the selected via. Chopping Layer1 or Layer2 shapes implies cut removal. Therefore, selecting Layer1 and Layer2 to chop implies chopping all layers.

The following diagram shows some examples of how to chop built-in custom vias.



The following diagram shows some examples on how to heal built-in custom vias.



Defining Cut LPPs for Built-In Custom Vias

You can use the `cutLPP` CDF parameter to define cut layer-purpose pairs (LPPs) for custom vias. The `cutLPP` values must be in the following format:

((cutLayer1 cutPurpose1) (cutLayer2 cutPurpose2) ... (cutLayer n cutPurposen))

7

Creating and Editing Transmission Lines

This chapter describes how to create and edit transmission lines in the designs.

This chapter describes the following:

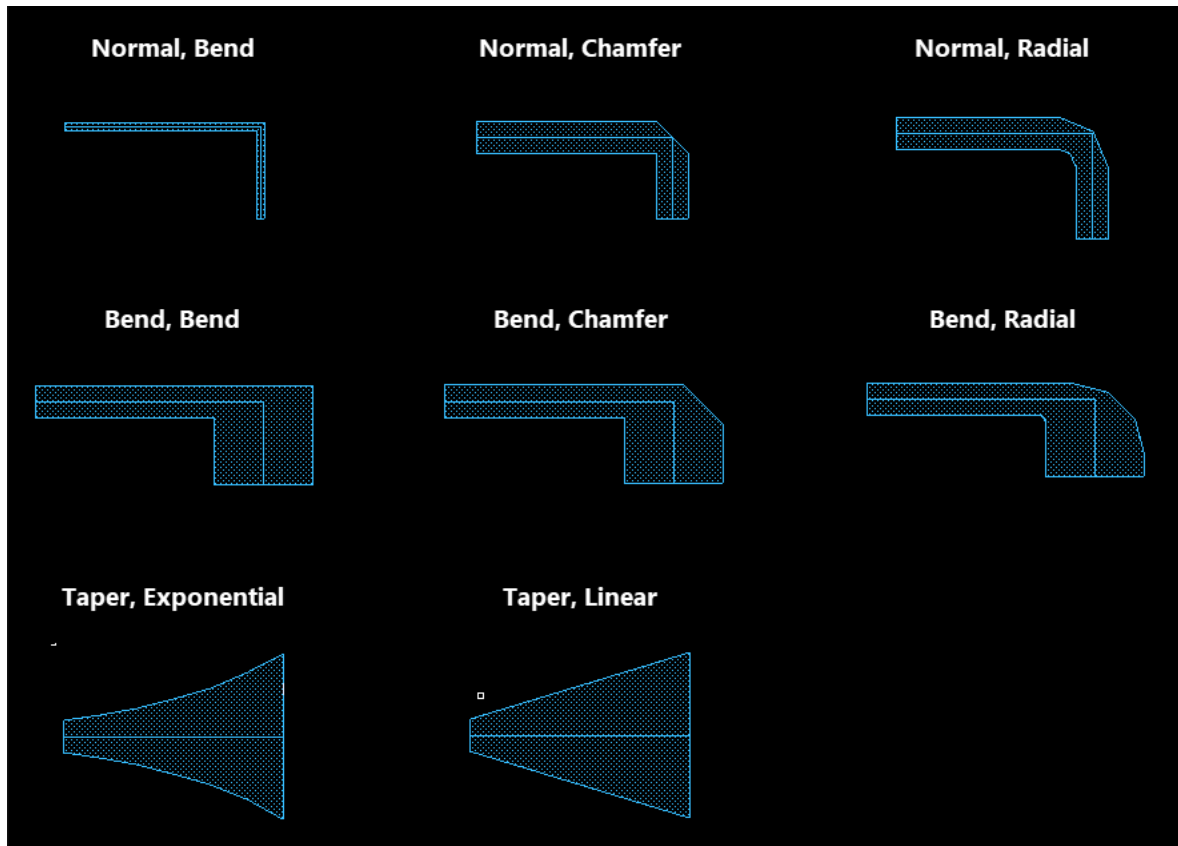
- [About Creating Transmission Lines](#)
- [Creating Transmission Lines](#)
- [Editing Transmission Lines](#)
- [Create Transmission Line Toolbar and Dialog Box Reference](#)

About Creating Transmission Lines

A transmission line is a type of interconnect or route that conducts high-frequency alternating currents (AC), such as radio and microwave signals. Transmission lines are placed on a single layer, so vias are not required.

You can use the **Create > Transmission Line** command to add transmission lines in your designs. Because transmission lines are editable PCells, you can adjust parameters to control the line size, shape, and other characteristics.

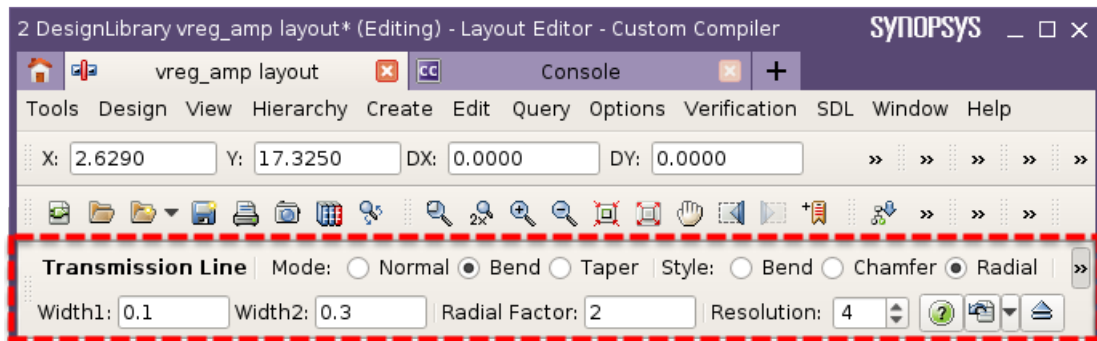
The following examples show how different parameters can affect the appearance of the transmission lines. The labels above each example correspond to the option names you can select when you create the transmission lines. See [Creating Transmission Lines](#) or [Create Transmission Line Toolbar and Dialog Box Reference](#) for details.



Creating Transmission Lines

To create a transmission line:

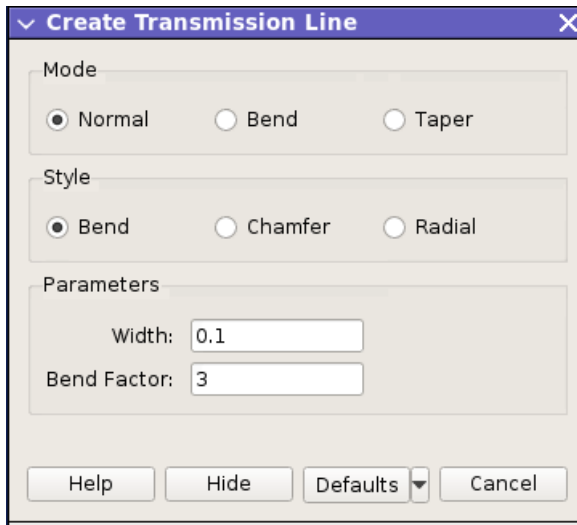
1. In the Object/Layer Panel, select a layer-purpose pair (LPP). If you do not select a LPP, the Layout Editor will create the transmission line using the nearest LPP.
2. Choose **Create > Transmission Line**. The **Transmission Line** toolbar appears.



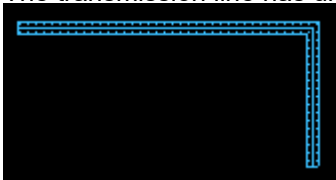
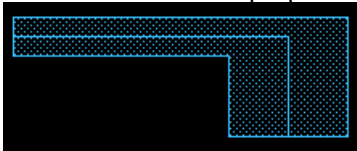
3. (Optional) On the toolbar, click  to display the **Create Transmission Line** dialog box.

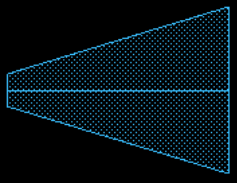
The toolbar and the dialog box contain identical options. However, depending on the options you select, the toolbar and dialog box contents can vary.

To hide the dialog box and return to the toolbar, click **Hide** in the dialog box.

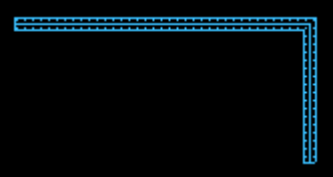
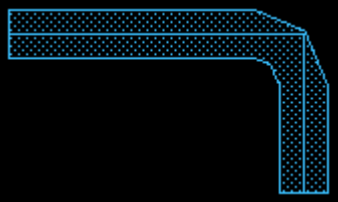


4. Select the **Mode** to specify the type of transmission line you want to draw. When you select a **Mode**, the contents of the dialog box change.

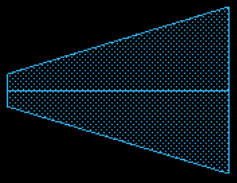
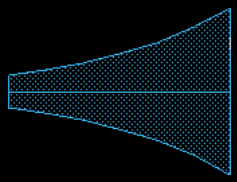
Mode	Description
Normal	The transmission line has uniform width throughout. 
Bend	The transmission line has one width in one direction and a different width in the perpendicular direction. 

Mode	Description
Taper	The ends of the single-direction (horizontal or vertical) transmission line has two different widths.  A diagram showing a transmission line that tapers from a narrow width on the left to a wider width on the right. The tapering is represented by a series of small, closely spaced, parallel lines forming a triangular shape.

5. (**Mode: Normal** or **Mode: Bend**) Select one of the following **Styles**:

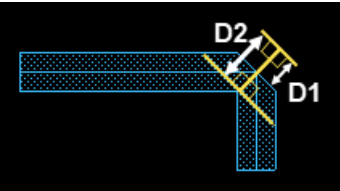
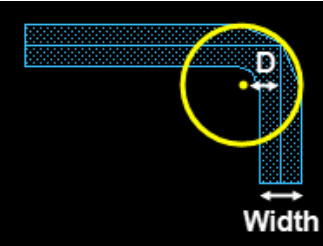
Style (Normal and Bend Modes)	Description
Bend	The outer bend corner is 90°.  A diagram showing a transmission line that bends at a 90-degree angle. The bend is sharp, and the outer corner is a perfect 90-degree angle.
Chamfer	The outer bend corner is cut away, forming a beveled edge.  A diagram showing a transmission line that bends at a 90-degree angle. The outer corner is cut away, forming a beveled edge. The bevel is represented by a series of small, closely spaced, parallel lines forming a triangular shape.
Radial	The outer bend is curved.  A diagram showing a transmission line that bends at a 90-degree angle. The outer corner is curved, following the radius of the bend. The curvature is represented by a series of small, closely spaced, parallel lines forming a circular arc.

6. **(Mode: Taper)** Select one of the following **Styles**:

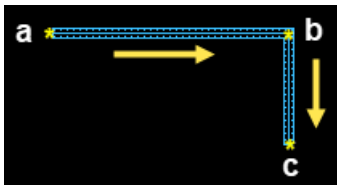
Style (Taper Mode)	Description
Linear	The ends of the single-direction path have two different widths. The sides of the path are linear. 
Exponential	The ends of the single-direction path have two different widths. The sides of the path form curves. 

7. Set the following **Parameters**:

Parameter	Description
Width	(Mode: Normal) Width of the path.
Width1, Width2	(Mode: Bend) Width of the path in one direction and a different width in the perpendicular direction.
Bend Factor	(Mode: Normal and Mode: Bend) (Style: Bend) The threshold value used to determine whether the outer bend corner is chamfered. When Width > Bend Factor, the outer bend corner will be chamfered based on the Chamfer Factor parameter that appears.

Parameter	Description
Chamfer Factor	(Mode: Normal and Mode: Bend) (Style: Bend and Style: Chamfer) The amount of the outer bend corner that is cut. Chamfer Factor is calculated as follows: Chamfer Factor = D1/D2 
Radial Factor	(Mode: Normal and Mode: Bend) (Style: Radial) The size of the outer bend curvature. Radial Factor is used to calculate the distance (D) between the path's centerline to the center of the arc. $D = \text{Radial Factor} * \text{Width}$ 
Resolution	(Mode: Normal and Mode: Bend) (Style: Radial) The number of line segments that form the curve. The larger the Resolution value, the smoother the curve. 

8. On the canvas, draw the transmission line:
 - a. Click to start drawing, and move the cursor to direct the line.
 - b. Click to change direction.
 - c. Double-click to enter the last point and complete the transmission line.



9. When you are finished, press Esc to quit the command.

For information on using the `ile::createTransmissionLine` Tcl command, see “Layout Editing Tcl” in the *Custom Compiler Tcl Reference Manual*.

Editing Transmission Lines

This topic describes the ways in which you can edit transmission lines:

- [Using Editing Commands](#)
- [Using the Property Editor](#)

Using Editing Commands

You can use the following **Edit** commands to modify transmission lines:

- Move
- Copy
- [Stretch](#)

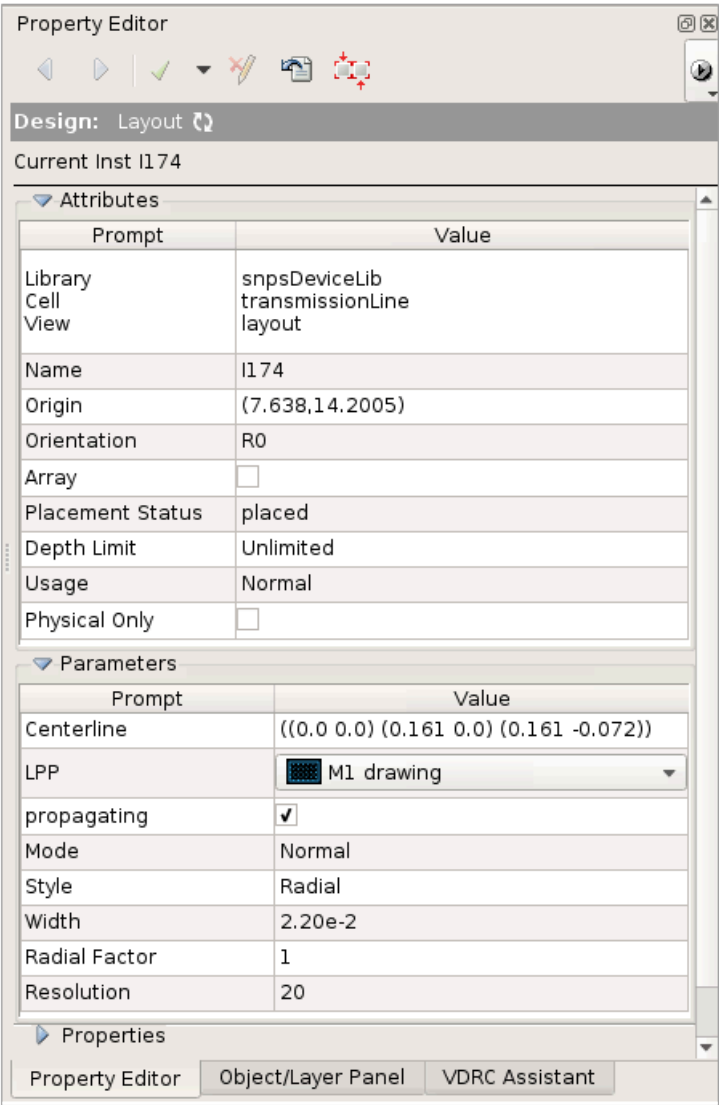
For details, see the topics that describe the commands.

Using the Property Editor

You can modify transmission lines using the Property Editor.

To modify a transmission line using the Property Editor:

1. Display the Property Editor.
2. On the canvas, select the transmission line you want to modify. The information about the selected transmission line appears in the Property Editor.



3. In the Property Editor, modify the transmission line attributes.

For information about the **Attributes** section, see Editing Instance Properties.

The **Parameters** section contains the following options:

Parameter	Description
Centerline	The coordinates of the path's centerline.
LPP	The layer-purpose pair on which the transmission line is created.
Propagating	The indicator that the path segment propagates connectivity.

The remaining **Parameters** are the same as the options that you set when you created the transmission line. For the descriptions of these parameters, see [Creating Transmission Lines](#).

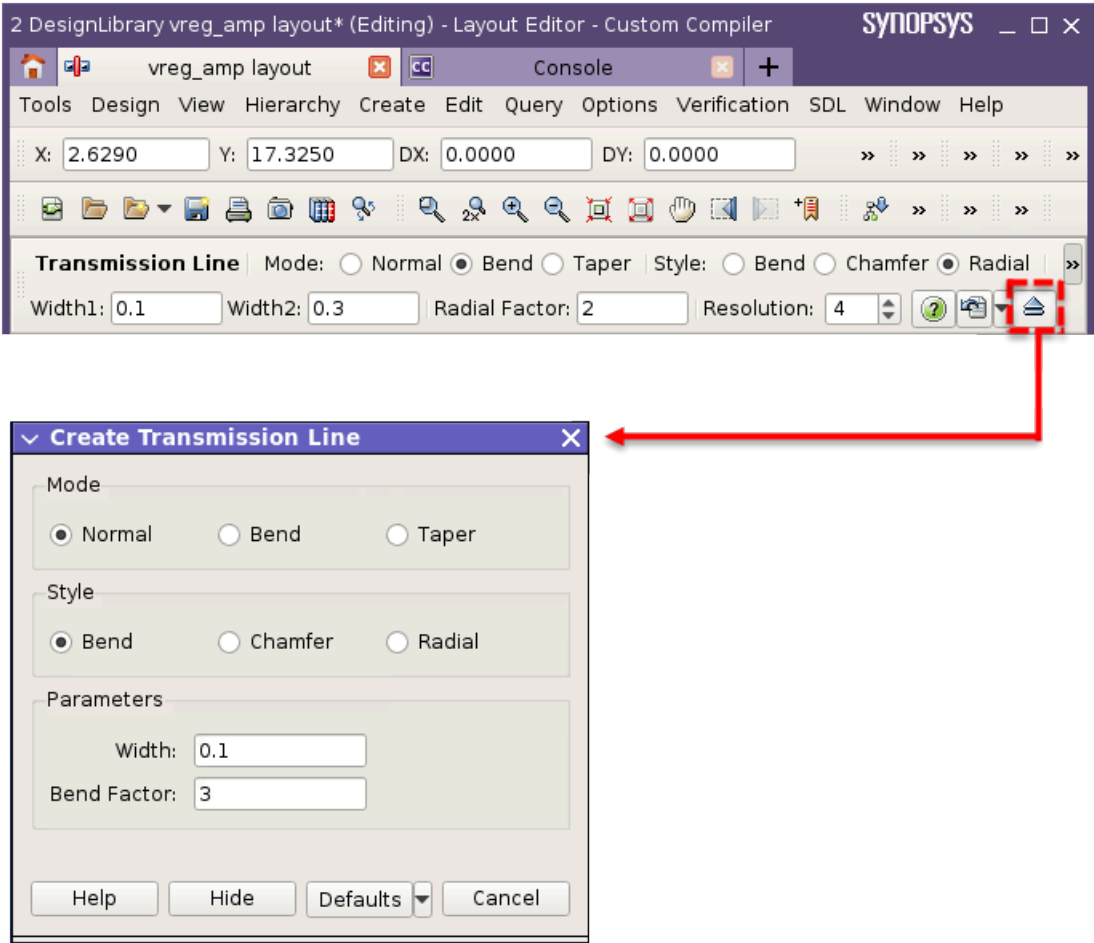
4. When you are finished, click **Apply Changes**  at the top of the Property Editor. The Layout Editor saves the changes.

Create Transmission Line Toolbar and Dialog Box Reference

This section provides a quick reference of the **Create Transmission Line** toolbar and dialog box. For instructions on how to create transmission lines, see [Creating Transmission Lines](#).

Note:

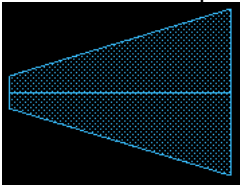
The contents of the toolbar and dialog box changes when you select different options.

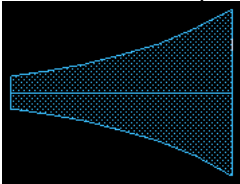


Mode:

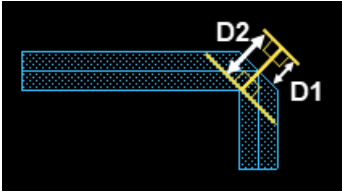
Mode	Description
Normal	The transmission line has uniform width throughout.
Bend	The transmission line has one width in one direction and a different width in the perpendicular direction.
Taper	The two ends of a single-direction transmission line has two different widths.

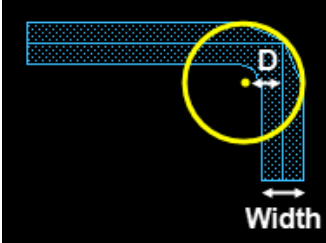
Style:

Style (Normal and Bend Modes)	Description
Bend	The outer bend corner is 90°.  However, when the wire Width is greater than the Bend Factor, the outer corner chamfers. See Parameters.
Chamfer	The outer bend corner is cut away, forming a beveled edge.  The Chamfer Factor determines the amount of the corner that is part of the beveled edge. See Parameters.
Radial	The outer bend is curved.  The Radial Factor determines the size of the curvature. See Parameters.
Linear	The ends of the single-direction path have two different widths. The sides of the path are linear. 

Style (Normal and Bend Modes)	Description
Exponential	The ends of the single-direction path have two different widths. The sides of the path form curves. 

Parameters:

Mode	Description
Width	(Mode: Normal) Width of the path.
Width1, Width2	(Mode: Bend) Width of the path in one direction and a different width in the perpendicular direction.
Bend Factor	(Mode: Normal and Mode: Bend) (Style: Bend) The threshold value used to determine whether the outer bend corner is chamfered. When Width > Bend Factor, the outer bend corner will be chamfered based on the Chamfer Factor parameter that appears.
Chamfer Factor	(Mode: Normal and Mode: Bend) (Style: Bend and Style: Chamfer) The amount of the outer bend corner that is cut. Chamfer Factor is calculated as follows: Chamfer Factor = D1/D2 

Mode	Description
Radial Factor	(Mode: Normal and Mode: Bend) (Style: Radial) The size of the outer bend curvature. Radial Factor is used to calculate the distance (D) between the path's centerline to the center of the arc. $D = \text{Radial Factor} * \text{Width}$ 
Resolution	(Mode: Normal and Mode: Bend) (Style: Radial) The number of line segments that form the curve. The larger the Resolution value, the smoother the curve. 

8

Working With Cut Metal Shapes and Erasing Layers

This chapter describes concepts and instructions for working with cut metal shapes and erasing layers in advanced node designs containing FinFET components.

This chapter describes the following:

- [Generating and Deleting Cut Metal Shapes](#)
- [Erasing Layers](#)

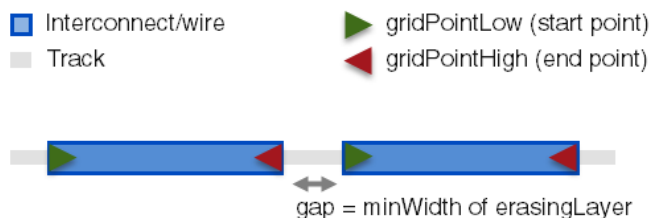
Generating and Deleting Cut Metal Shapes

This topic describes the following:

- [About Cut Metal Shapes](#)
- [Before You Start](#)
- [Generating Cut Metal Shapes](#)
- [Deleting Cut Metal Shapes](#)

About Cut Metal Shapes

In designs that originate from the Custom Compiler environment or the digital tool (Fusion Compiler or IC Compiler II tool), where metal shapes need to be closer together than permitted by spacing constraints, you can use gridded tracks and keepout constraints to place the metal shapes close together.

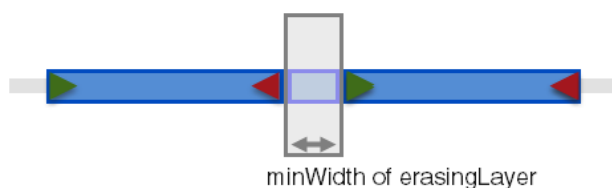


To prevent spacing violations, you can use the **Generate Cut Metal** command to fill the gaps that are equal to the `minWidth` of the erasing layer. When filling the gaps, the command creates the following shapes:

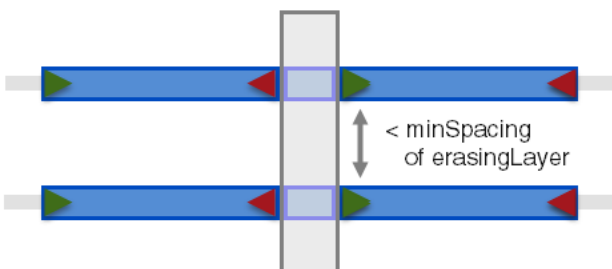
- Erasing (or cut metal) shape that is on the erasing layer. By default, the width of the erasing shape is the `minWidth` of the erasing layer. However, you have the option of specifying the width and length of the erasing shape in the command settings. See [Generating Cut Metal Shapes](#).
- Metal shape that connects the existing metal shapes.

In Custom Compiler designs, the connecting metal shape has the same layer-purpose pair as the two existing metal shapes. In Fusion Compiler or IC Compiler II designs, the connecting metal shape is on the same layer as the two existing metal shapes and it has the same purpose as the shape below the cut metal.

-  Erasing shape
-  Metal shape beneath erasing shape



If two adjacent erasing shapes are closer than `minSpacing` for the erasing layer, the shapes will be merged to form one shape.



The **Generate Cut Metal** command will fill the gaps only if:

- The metal shapes that flank the gaps are aligned, have the same layer-purpose pair, and are assigned the same color.
- The distance between the shapes is equal to the `minWidth` of the erasing layer.

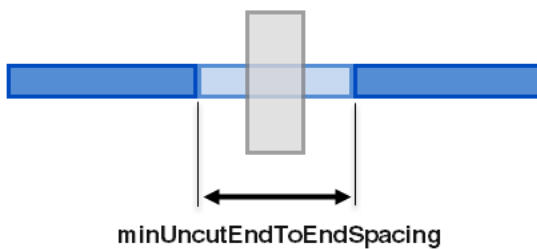
If you are editing a Fusion Compiler or IC Compiler II design in the Custom Compiler environment, the **Generate Cut Metal** command first removes existing erasing shapes that were created in the Fusion Compiler or IC Compiler II tool before filling the gaps.

After the generate operation, the generated metal shapes that are below the erasing shapes become non-propagating. For information about connectivity propagation, see [Propagating Connectivity](#).

In Custom Compiler designs that comprise of standard cells, and the metal shapes are placed without gridded tracks or keepout constraints, the **Generate Cut Metal** command will:

- Look for gaps that are less than `minUncutEndToEndSpacing`.
- Fill the gaps by adding erasing shapes and connecting metal shapes.
- Align the centerline of the erasing shape with the centerline of the standard cell boundary.

- Interconnect/wire
- Metal shape (extension)
- Erasing shape



You can use the **Delete Cut Metal** command to remove the generated shapes in the gaps in the Custom Compiler designs, except for those that were added manually. In Fusion Compiler or IC Compiler II designs, the command removes the erasing shapes that were created using the Fusion Compiler or IC Compiler II tool.

Before You Start

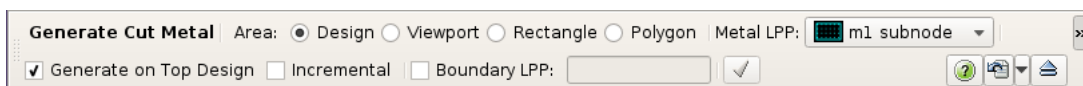
Make sure you specify the `erasingLayer` constraint in the technology library. During the **Generate Cut Metal** operation, the cut-metal shapes are created on the defined erasing layers.

Place routes and use gridded tracks. For instructions, see [Creating Interconnects](#) and [Routing with the Custom Router](#).

Generating Cut Metal Shapes

To generate cut metal shapes:

1. In the Layout Editor, choose **Edit > Other > Generate Cut Metal**. The **Generate Cut Metal** toolbar appears.

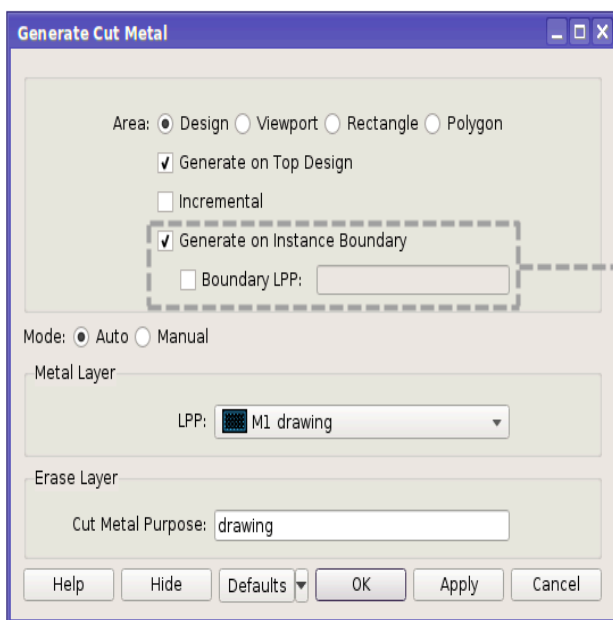


2. (Optional) On the **Generate Cut Metal** toolbar, click **Eject**  to display the **Generate Cut Metal** dialog box.

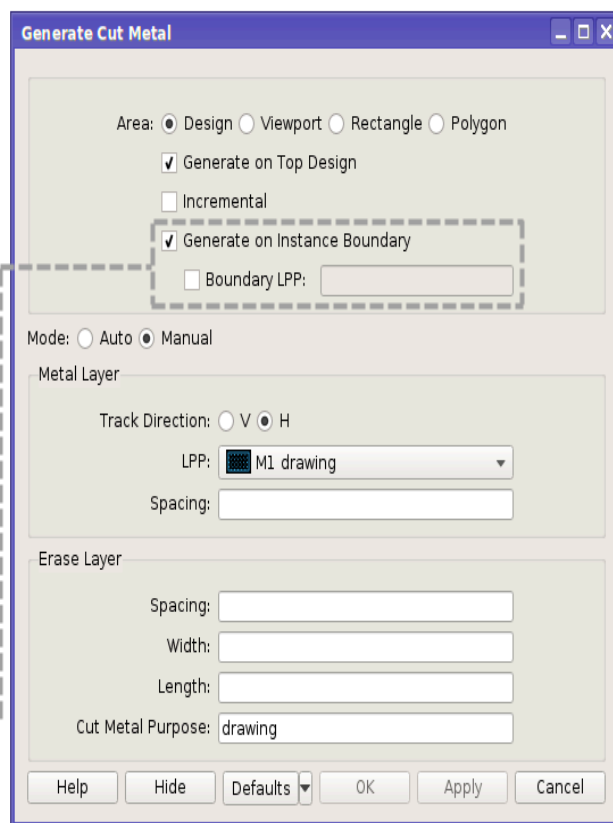
Custom Compiler Designs

Auto

Manual



☐ Only Generate Shapes Below Cut
Fusion Compiler or IC Compiler II Designs



3. On the toolbar or in the dialog box, set the basic options:

Option	Description
Area	Generates the erasing shapes and metal shapes within the specified area: ◦ Design: Across the entire design. ◦ Viewport: Within the viewport. ◦ Rectangle: Within the rectangular region you specify. ◦ Polygon: Within the polygonal region you specify. If you select Rectangle or Polygon, the Layout Editor will prompt you to draw the rectangle or polygon on the canvas. Tcl preference: <code>leGenerateCutMetalArea</code>
Generate on Top Design	Generates the shapes on the top (0x) hierarchy level. Tcl preference: <code>leGenerateCutMetalOnTop</code>
Incremental	Keeps existing metal shapes and erasing shapes without modifying them, and generates shapes in remaining gaps. If you do not select the option, the command updates any existing shapes in the gaps. Tcl preference: <code>leGenerateCutMetalIncrementalMode</code>
Generate on Instance Boundary	Generates cut metal shapes on instance boundary, using the boundary LPP. Clear the check box to generate cut metal shapes on tracks. If tracks do not exist, the cut metals shapes are generated at the center of the design. Tcl preference: <code>leGenerateCutMetalGenerateInstBoundary</code>
Boundary LPP	(Custom Compiler designs) Uses the specified layer-purpose pair to determine instance boundaries. If you do not select the option, the Layout Editor will use the <code>leInstBoundary</code> Tcl preference specification. Tcl preference: <code>leGenerateCutMetalSpecifyBoundaryLPP</code>
Only Generate Shapes Below Cut	(Fusion Compiler or IC Compiler II designs) Generates only the metal shapes that fill the gaps, but does not generate the erasing shapes. Tcl preference: <code>leGenerateCutMetalShapeBelowCut</code>

4. (Dialog box only) Select one of the following modes:

- **Auto:** The command automatically generates the cut metal shapes based on the technology constraints, and the **Metal Layer** LPP and the **Erase Layer** purpose information you provide.
- **Manual:** You must supply additional details about the **Metal Layer** and **Erase Layer**.

5. (Dialog box only) Set the **Metal Layer** options:

Option	Description
Track Direction	(Manual Mode only) Generates shapes on the tracks with the selected direction: ◦ V: Vertical ◦ H: Horizontal If the preferred direction is not specified in the technology database, the vertical direction is used. The preferred direction you specify in the dialog box overrides the specification in the technology database. Tcl preference: <code>leGenerateCutMetalTrackDir</code>
LPP	Generates the shapes on the specified layer-purpose pair (LPP). Tcl preference: <code>leGenerateCutMetalLPP</code>
Spacing	(Manual Mode only) Specifies the minimum distance between the wire ends. By default, the Spacing is <code>minUncutEndToEndSpacing</code>. Tcl preference: <code>leGenerateCutMetalSpacing</code>

6. (Dialog box only) Set the **Erase Layer** options:

Option	Description
Spacing	(Manual Mode only) Specifies the minimum distance between adjacent erasing shapes. The default value is minimum spacing specified for the erasing layer in the technology database. Tcl preference: <code>leGenerateCutMetalEraseSpacing</code>
Width	(Manual Mode only) Specifies the minimum width of the erasing shape. The default value is the minimum width specified for the erasing layer in the technology database. Tcl preference: <code>leGenerateCutMetalEraseWidth</code>

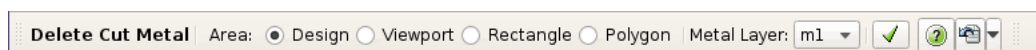
Option	Description
Length	(Manual Mode only) Uses the erasing layer's length to calculate the erasing shape length. The default value is the erasing layer's length that is defined in the technology database. Tcl preference: <code>leGenerateCutMetalEraseLength</code>
Cut Metal Purpose	Specifies the purpose of the erasing shape. The default is: ◦ <i>Custom Compiler design</i>: drawing ◦ <i>Fusion Compiler or IC Compiler II design</i>: cutShape For Custom Compiler designs, if you do not specify the Cut Metal Purpose , the Layout Editor will use the erasing layer purpose defined in the technology database. If the cut metal purpose is not defined in the technology database, the drawing purpose is used by default. Tcl preference: <code>leGenerateCutMetalCutPurpose</code>

7. When you are finished, click  on the toolbar or **Apply** in the dialog box.

Deleting Cut Metal Shapes

To remove the overlapping erasing and metal shapes:

1. In the Layout Editor, choose **Edit > Other > Delete Cut Metal**. The **Delete Cut Metal** toolbar appears.



2. Set the following options:

Option	Description
Area	Applies the command to the specified area: ◦ Design: Across the entire design. ◦ Viewport: Within the viewport. ◦ Rectangle: Within the rectangular region you specify. ◦ Polygon: Within the polygonal region you specify. If you select Rectangle or Polygon , the Layout Editor will prompt you to draw the rectangle or polygon on the canvas. Tcl preference: <code>leGenerateCutMetalArea</code>

Option	Description
Metal Layer	Deletes the generated shapes on the specified metal layer. Tcl preference: <code>leDeleteCutMetalLayer</code>
Cut Metal Purpose	(Fusion Compiler or IC Compiler II designs) Deletes the purpose of the erasing shape created by the Fusion Compiler or IC Compiler II tool. The default is <code>cutShape</code> . Tcl preference: <code>leGenerateCutMetalCutPurpose</code>

3. When you are finished, click  on the toolbar or **Apply** in the dialog box.

Erasing Layers

The Layout Editor **Erase Layer** command cuts a single shape into multiple smaller shapes. The technique is used in advanced technology nodes to create smaller spacing between objects.

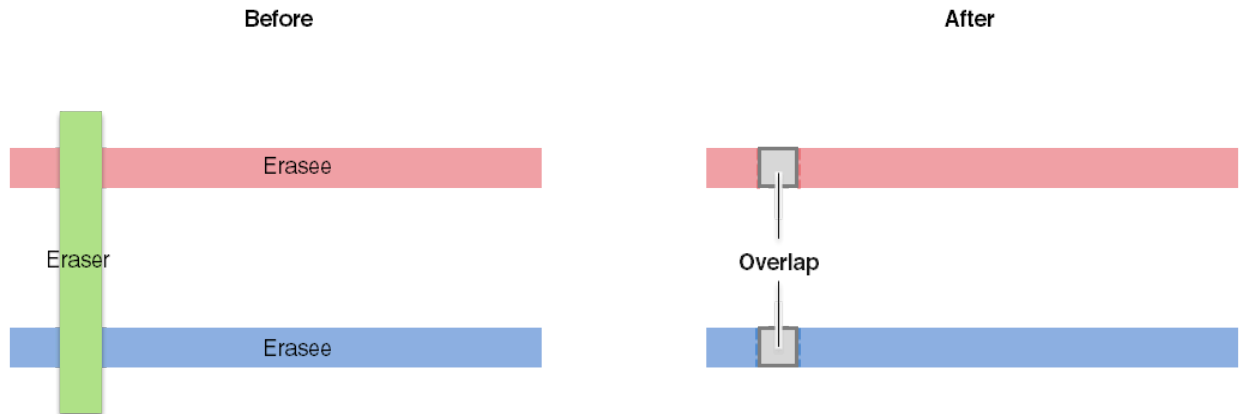
This topic describes the following:

- [About Erasing Layers](#)
- [About Hierarchical Erase-Layer Operations](#)
- [Before You Start](#)
- [Setting Up Tcl Functions to Erase Layers in PCells](#)
- [Using the Edit > Other > Erase Layer Command](#)

About Erasing Layers

In advanced technology nodes, shapes need to be closer together than permitted by defined constraints. To achieve the desired spacing without violating constraints, you can use the Layout Editor **Edit > Other > Erase Layer** command to add a new layer and use it to create cuts (or the desired spacing) in shapes.

In the following diagram, red and blue shapes are placed on the canvas. To create the desired spacing in the shapes, a green shape is added to overlap with the red and blue shapes at the locations where the spacing is desired. The green shape is then removed using the **Erase Layer** command. During the removal process, the areas that overlap with the red and blue shapes are also removed, thus creating the desired spacing. In the example, the green shape is the *eraser*, and the red and blue shapes are the *erasees*.



During the erasing layer operation, the Layout Editor looks for overlapping shapes and proceeds as follows:

- If the erasees are 0x shapes, remove the eraser shape and overlapping regions.
- If the erasees are lower level shapes in parameterized cells (PCells), call the user-defined Tcl function to process the overlapping regions. For more information, see [Setting Up Tcl Functions to Erase Layers in PCells](#).

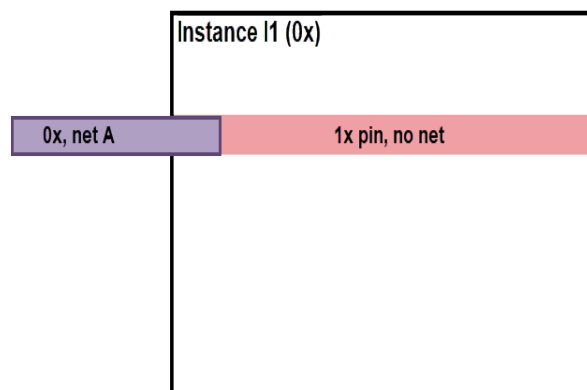
After the erase layer operation, the remaining erasee shapes become non-propagating. For information about connectivity propagation, see [Propagating Connectivity](#).

About Hierarchical Erase-Layer Operations

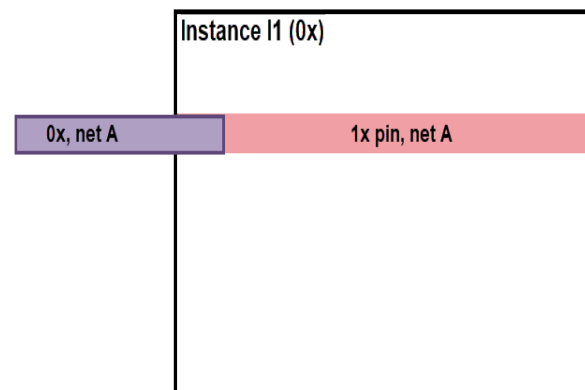
Net propagation in erase-layer operations depend on whether you turned on the Connectivity Engine and set the `leHonorErasingLayerConnectivity` Tcl preference to true. The results also depend on the properties of adjacent shapes.

When working with PCells, net propagation relies on the Connectivity Engine to trace nets through hierarchies. In the following example, the 0x shape propagates its net to the 1x pin in the I1 instance when the two shapes connect.

Before Propagation



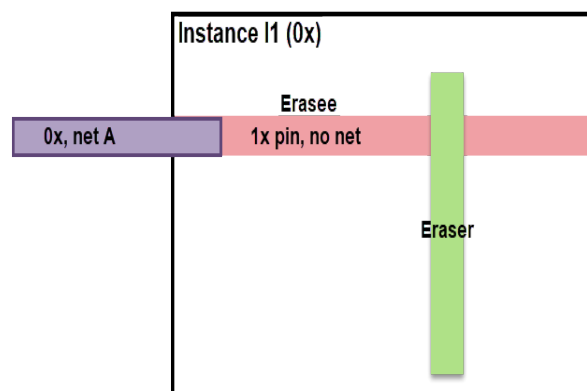
After Propagation



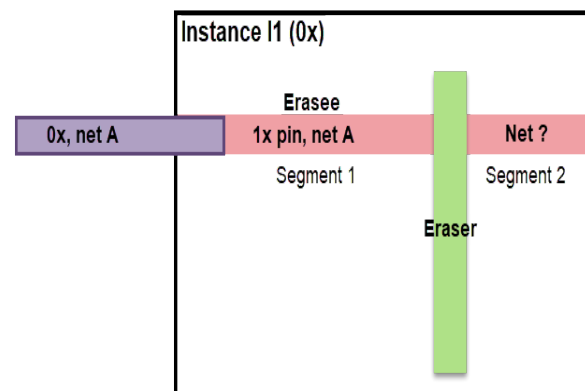
When erasing shapes in PCells, net propagation also relies on the Connectivity Engine to trace nets through hierarchies. In addition, the `leHonorErasingLayerConnectivity` Tcl preference must be set to true.

In the following example, the 0x shape propagates its net to the first segment of the 1x pin in instance I1. The net assignment of Segment 2 depends on its surroundings. If nothing connects to or touches it, Segment 2 will continue to have no net assignment.

Before Propagation

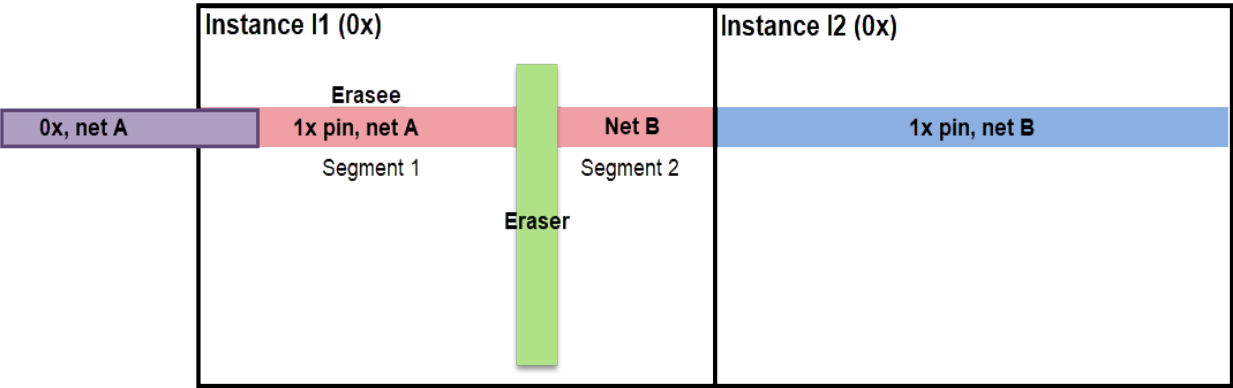


After Propagation



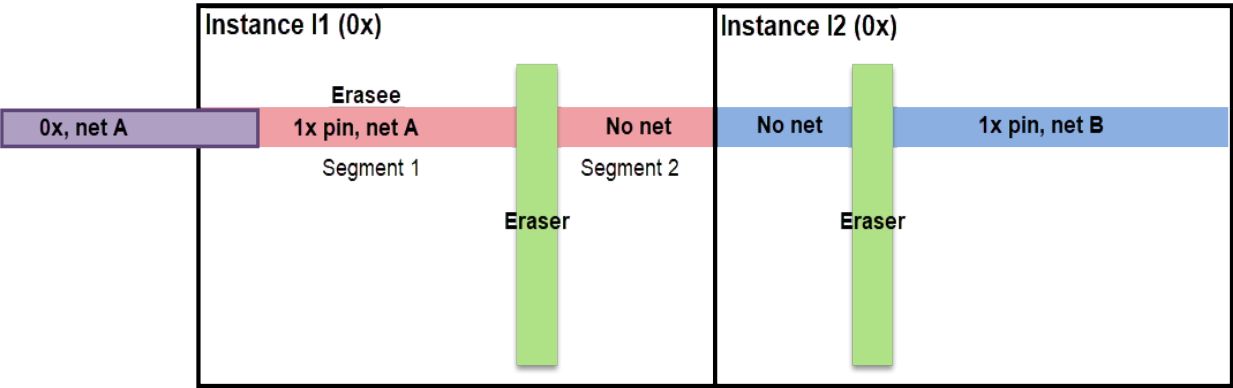
If Segment 2 touches instance I2 as shown, net B is propagated to Segment 2.

After Propagation



If the pin shape in I2 is cut by an eraser shape as shown, Segment 2 remains without a net assignment.

After Propagation



Before You Start

To ensure successful operation and to optimize the results of erasing layers, you must correctly define constraints in the technology database and set the relevant Tcl preferences.

Defining the eraseLayer Technology Constraint

Define the eraser layers in the `erasingLayer` constraint section of the technology file. During the erase layer operation, the Layout Editor automatically removes the layer you specified in the `erasingLayer` constraint.

Turning On the `leAutoEraseLayer` Tcl Preference

Set the `leAutoEraseLayer` Tcl preference to `true` to control the event-driven callbacks. Setting the preference to `true` turns on the internal callback that listens for modification events from the Layout Editor.

To set the preference to `true`, type the following in the Console:

```
db::setPrefValue leAutoEraseLayer -value true
```

Setting the `leSmartEraseLayer` Tcl Preference

To manipulate the erasee shapes as one shape, set the `leSmartEraseLayer` Tcl preference to `true`. Otherwise, the Layout Editor will treat the erasee shapes as separate shapes.

Setting Up Hierarchical Erase-Layer Operations

For erasing layers in PCells, set the following Tcl preferences:

- `leEnableCE`: Set the preference to `true` to turn on the Connectivity Engine.
- `leHonorErasingLayerConnectivity`: Set the preference to `true` to use hierarchical net tracing during erase layer operations.

See Also

- [Setting Up Tcl Functions to Erase Layers in PCells](#)

Defining Feedthrough Layers

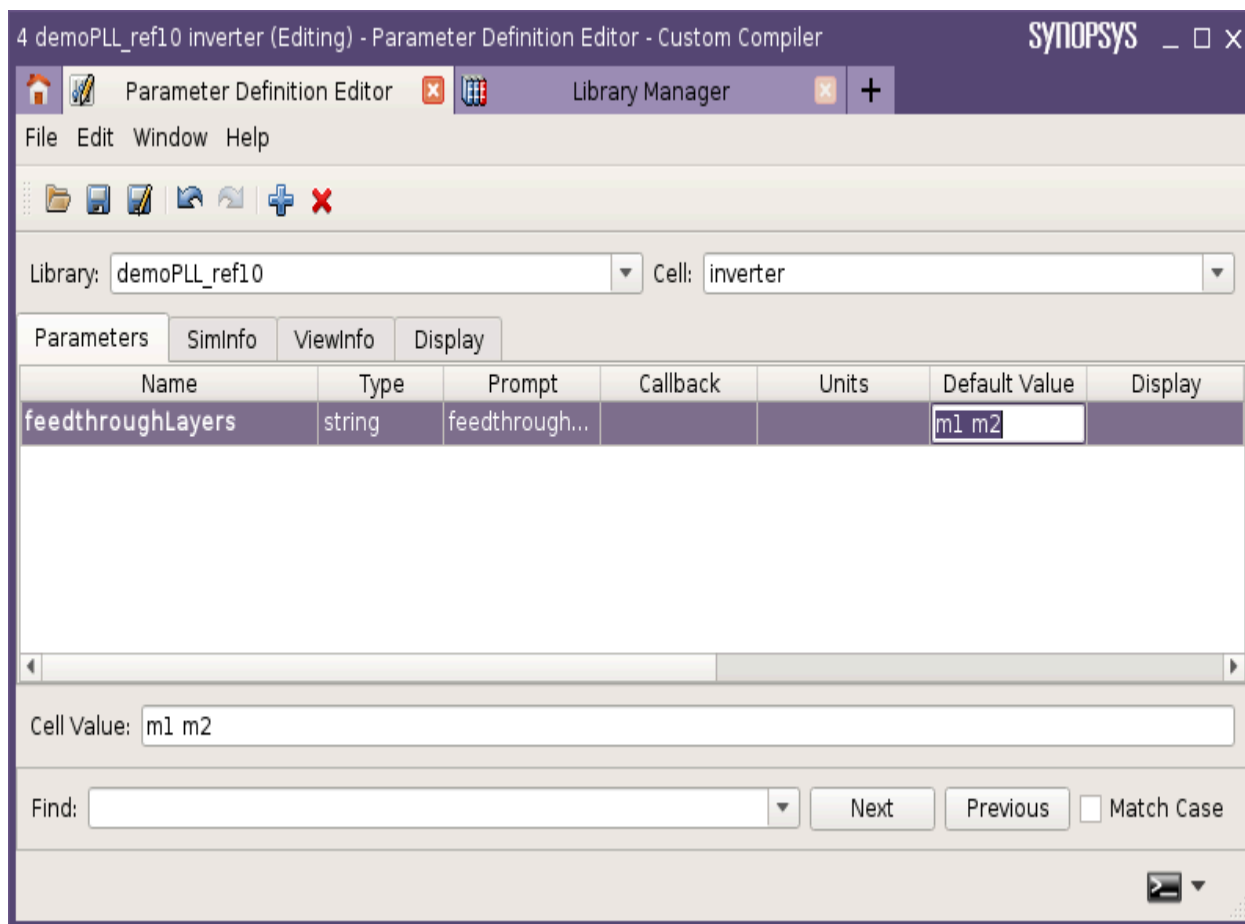
To extend net propagation to unnetted Nx objects during erasing layer operations, you must add the `feedthroughLayers` parameter to the CDF file, and then assign the feedthrough layers to the parameter. You can do both using the Parameter Definition Editor.

The `feedthroughLayers` parameter is supported by both libraries and cells. During erasing layer operations, the command looks for feedthrough layer specifications in cells before checking libraries.

To add the `feedthroughLayers` parameter and specify the feedthrough layers:

1. Display the Parameter Definition Editor. See [Displaying the Parameter Definition Editor Tool](#).
2. Add the `feedthroughLayers` parameter. See [Adding Parameters](#).
3. In the **Default Value** column, specify the layers that can be used for feedthroughs. When specifying multiple layers, add a character space between the layer names.

In the following example, m1 and m2 layers are specified as feedthrough layers.



Setting Up Tcl Functions to Erase Layers in PCells

This section describes the requirements for erasing layers inside PCells:

- [Creating the Erasing Layer User-Defined Tcl Function](#)
- [Setting the leAutoEraseLayer Tcl preference](#)

Creating the Erasing Layer User-Defined Tcl Function

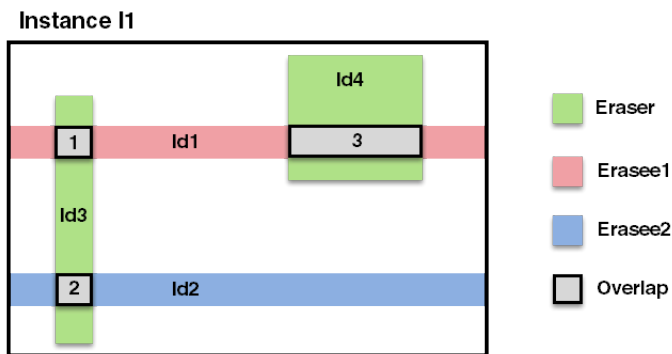
The user-defined erase layer function has the following Tcl syntax:

```
proc eraselayer_function_name {{InstId oaInstId}} {erasingRegions
  list{erasePointList1... erasePointListn}} {Erasers
  list{eraserShapeId1... eraserShapeIdn}} {Erasees list{erasee1...
  eraseen}}}
```

Where

{argument_key_value_pair}	Description
{InstID oaInstId}	Key: InstId Value: OA instance Id.
{ErasingRegions list{erasePointList1 ... erasePointListn}}	Key: ErasingRegions Value: List of overlapping shapes that will be erased.
{Erasers list{eraserShapeId1...eraserShapeIdn}}	Key: Erasers Value: List of shapes that will perform the erasing.
{Erasees list{erasee1...eraseen}}	Key: Erasees Value: List of shapes that will be cut (erased). If multiple erasers overlap with a single erasee, the erasee should be listed multiple times.

In the following diagram, the example shows the shapes within instance I1. The green shapes are the erasers. The red and blue shapes are the erasees. Overlapping regions 1, 2, and 3 are being erased. Each overlapping region can be described by points. For example, overlapping region 1 is a rectangle that is described by the following: {pt11 pt12 pt13 pt14}.



The Tcl code for the erasing operation is as follows:

```
eraseLayerNewFunction {{InstID I1} {erasingRegions {{pt11 pt12 pt13 pt14}
{pt21 pt22 pt23 pt24} {pt31 pt32 pt33 pt34}}} {Erasers {{Id3} {Id3}
{Id4}}} {Erasees {Id1 Id2 Id1}}}
```

Notice that because erasers Id3 and Id4 both erase different parts of erasee1, Id1 appears twice in the Erasees list.

If you want to specify less information for the function, you can use the following syntax:

```
proc eraselayer_function_name {oaScalarInst list{erasePointList} }
```

Where the arguments are:

`oaScalarInst`

The Tcl variable name used for cuttee object

`list{erasePointList}`

The list of pointList for the actual overlaps between cutter and cut. The list has been transformed from 0x coordinates to 1x. If the list is empty, it means there is no erasure.

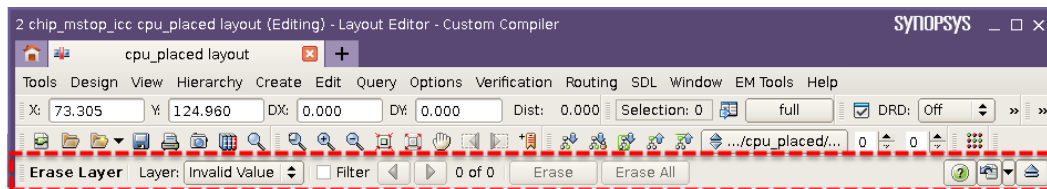
Setting the `leAutoEraseLayer` Tcl preference

Whichever syntax you use, make sure you set the `leAutoEraseLayer` Tcl preference to `true`. See [Before You Start](#).

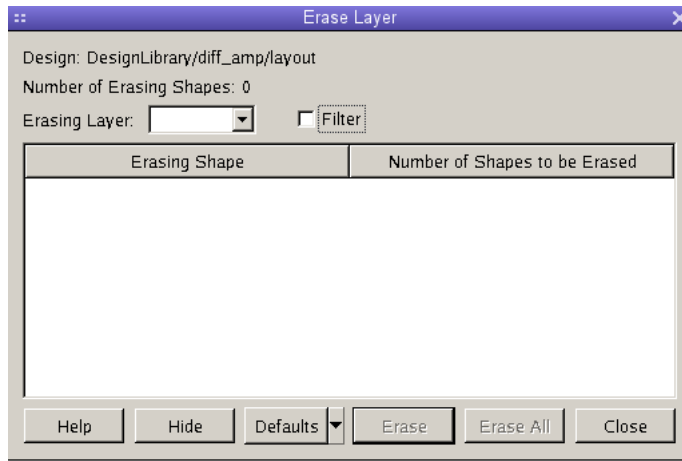
Using the Edit > Other > Erase Layer Command

To use the **Edit > Other > Erase Layer** command:

1. In the Layout Editor window, choose **Edit > Other > Erase Layer**. The **Erase Layer** toolbar appears.



2. (Optional) To display the **Erase Layer** dialog box, click . The options on the toolbar and in the dialog box are nearly identical.



3. Select the layer that will perform the erasing using one of the following methods:
 - On the toolbar, select the **Layer**.
 - In the dialog box, select the **Erasing Layer**.
4. (Optional) Select **Filter** to exclude overlapping shapes that were previously edited by the **Erase Layer** command.
5. Choose the layer that contains the shapes you want to cut using one of the following methods:
 - On the toolbar, click  (**Next Erase Set**) or  (**Previous Erase Set**).
 - In the dialog box, choose the layers from the **Erasing Shape** list.
6. Click **Erase** to erase the selected layers. To erase all layers in the list, click **Erase All**.

9

Creating Abstract Views From Designs

This chapter describes how to generate abstract views from a layout design.

This chapter describes the following:

- [About Abstract Views](#)
- [Generating Abstract Views From Console](#)
- [Generating Abstract Views From Layout Editor](#)
- [Managing Abstract Generator Option Sets](#)
- [Running Abstract Generator in Batch Mode](#)
- [Specifying the Routing Layers to Extract](#)

Note:

For information about creating soft abstract views from layouts, see *Creating Soft Abstract Views From Layouts* in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

For information about creating layout views from symbols, see *Creating Layouts From Symbols* in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

For information about creating symbol views from schematics, see *Creating Symbols From Schematics* in the *Custom Compiler Schematic Editor User Guide*.

About Abstract Views

Abstract views are simplified versions of designs that contain primarily components for routing. You can create abstract views of large and complex designs to reduce the length of routing time.

Use the **Create > Abstract** command to generate a simplified, abstract view of a design. The abstract generation option set you specify controls the abstract generation process for each layer. Color information in the design is preserved during abstract generation.

This topic describes the following:

- [Requirements](#)
- [Exclusions](#)
- [Objects Supported in Abstract Creation](#)
- [Attributes Used to Create Abstract Views](#)
- [Blockage Shapes](#)

Requirements

Before you create abstract views from designs:

- Make sure the design has a PR boundary or Snap boundary.
- Check that the design does not contain any unbound masters.

To extract only a specific set of routing layers or routing layers that are not defined by the foundry's `validRoutingLayers` constraint, you can specify `validRoutingLayer` in a constraint group, and make sure the `leAbstractConstraintGroups` Tcl preference specifies that constraint group.

Tcl preference:

`db::setPrefValue leAbstractConstraintGroups -value "leAbstractConstraintGroup"`



```
Technology File:

...
constraintGroups(
...
  (leAbstractConstraintGroup
    (validRoutingLayers (...))
  );leAbstractConstraintGroup
...
);constraintGroups
```

The abstract creation process extracts supported objects on valid routing layers, which include poly, cut, metal, contactless metal, and MIM layers. If you do not specify `validRoutingLayers` in the constraint group, the abstract creation process will use the valid routing layers defined in the following, in the order shown:

1. Design
2. Technology

Exclusions

When creating an abstract view, the tool excludes the following:

- Terminals without pins
- Pins that have no pin figures on an interconnect layer purpose pair
- MIMcap layers
- Blockages that you specified using the `leIgnoreBlockageTypesForAbstractGenerator` Tcl preference
- Blockages layers that you specified using the `leAbstractExcludedBlockageLayers` Tcl preference

For example, to exclude TPC, MPC, and BPC from the abstract generator process, you would set the preference as follows:

```
db::setPrefValue leAbstractExcludedBlockageLayers -value "TPC MPC BPC"
```

Objects Supported in Abstract Creation

This section describes the OpenAccess objects that are supported during abstract creation. Abstract creation ignores all other types of objects.

Table 5 *Objects Used to Create Abstracts*

Object	Description
oaTerm	Each terminal appears in the abstract, except terminals without pins.
oaNet	Each net attached to a terminal appears in the abstract, including nets without shapes. Net attributes also appear in the abstract. The signal type attribute is especially important for abstract generation; see Attributes Used to Create Abstract Views .
oaPin	Each pin appears in the abstract, except pins without shapes. oaVia pins that are on selected layers will be extracted. oaStdVia pins will be copied directly. oaCustomVia pin shapes will be extracted. The accompanying pin text labels will be copied. For pins without text labels, the abstract creation process will generate new text labels based on the terminal names. Pin shapes that are cut by shapes on erasing layers will be included in the abstract design.
oaPRBoundary	The place-and-route boundary, if it exists, appears in the abstract.

Table 5 Objects Used to Create Abstracts (Continued)

Object	Description
oaAreaHalo	The halo (if any) of the place-and-route boundary for the top-level design appears in the abstract. Halos for lower-level instances do not appear in the abstract.
oaSnapBoundary	The snap boundary, if it exists, appears in the abstract.
oaGroup	Groups appear in the abstract, unless they would be empty.
oaTextDisplay	Labels attached to pin shapes appear in the abstract. All other types of text objects do not appear in the abstract.
oaPinFig	Certain pin shapes appear in the abstract if they are attached to a pin and appear at the top level of the design: - Pin shapes that are paths, path segments, polygons, rectangles, and vias. - Pin shapes that are oaEllipse objects, which become polygons in the abstract. The <code>leNumPoints</code> option sets the number of vertices on the resulting polygons, but you can override the option value by modifying the <code>AGPSet</code> object with Tcl commands. Pin shapes that meet the requirements above, but are not attached to a pin or are not at the top level of the design hierarchy, appear in the abstract as routing blockages rather than pin shapes. These blockages might be merged with adjacent blockages or a cover blockage. See Blockage Shapes. If pin shapes meet the requirements above, the shapes merge if they: - Share the same pin object - Abut or overlap each other - Are strongly connected Pin shapes that are oaArc, oaDonut, oaDot, or oaLine objects do not appear in the abstract.
Shapes on interconnect layers	All shapes on interconnect layers, except for pins at the top level of the design, are merged and converted to layer blockages (oaLayerBlockage) of the routing type on the corresponding layer. See Blockage Shapes. During abstract creation, objects on valid routing layers are extracted. See Requirements.
Instances	Abstract generation flattens all instances down to the bottom of the design hierarchy. Instances appear in the abstract as blockages on the appropriate layers.
Vias	Abstract generation flattens all vias at all levels of the design hierarchy. Vias appear in the abstract as routing blockages.

Table 5 Objects Used to Create Abstracts (Continued)

Object	Description
Existing blockages	Existing blockages merge with other blockages of the same type and appear in the abstract. Effective width property values on existing blockages also appear in the abstract. See Blockage Shapes . You can specify the blockages to ignore using the <code>!ignoreBlockageTypesForAbstractGenerator</code> Tcl preference.

Attributes Used to Create Abstract Views

During abstract generation, the tool uses the following design data attributes related to placement and routing:

Table 6 Attributes Used to Create Abstracts

Attribute	Description
Pin Type	Differentiates power, ground, clock, and other pins. This attribute is the value of the <code>oaSigType</code> property on the net associated with the pin. The default value is <i>signal</i> . Use the Property Editor to change the value. The valid values are: • <code>signal</code> (<code>oacSignalSigType</code>) • <code>power</code> (<code>oacPowerSigType</code>) • <code>ground</code> (<code>oacGroundSigType</code>) • <code>clock</code> (<code>oacClockSigType</code>) • <code>tieoff</code> (<code>oacTieoffSigType</code>) • <code>tieHi</code> (<code>oacTieHiSigType</code>) • <code>tieLo</code> (<code>oacTieLoSigType</code>) • <code>analog</code> (<code>oacAnalogSigType</code>) • <code>scan</code> (<code>oacScanSigType</code>) • <code>reset</code> (<code>oacResetSigType</code>)
Pin Access Direction	Defines the directions from which the router may approach a pin. You can set this attribute while creating pins with the Create Pin command, or later with the Property Editor.
Cell Type	Defines the type of cell, which affects abstract generation. Abstract generation treats core cells as standard cells.

Blockage Shapes

If a layer has been selected in the options set, the tool merges blockages that share the following:

- Layer purpose pair
- Type
- Effective width

If the layer has not been selected in the option set, no blockages are created for that layer.

Blockages for interconnect shapes do not have a value for effective width. Place-and-route tools normally use the minimum spacing rules for the corresponding layer when blockages have no value for effective width.

The tool follows these steps to merge blockages:

1. Increase the size of each blockage shape by the Oversize value specified in the option set template.
2. Merge the oversized shapes that abut or overlap.
3. Shrink the resulting shapes by the Oversize value.

If the option set template specifies that the layer should have a cover blockage, the tool follows these additional steps:

1. Cover the entire design (using the PR boundary if it exists, or otherwise the snap boundary) with a blockage.
2. Cut routing channels in the cover blockage to allow access to each pin, based on its pin access direction property.

The blockages generated from the shapes and instances in the design data are never cut to provide pin access; only the cover blockage is cut.

Generating Abstract Views From Console

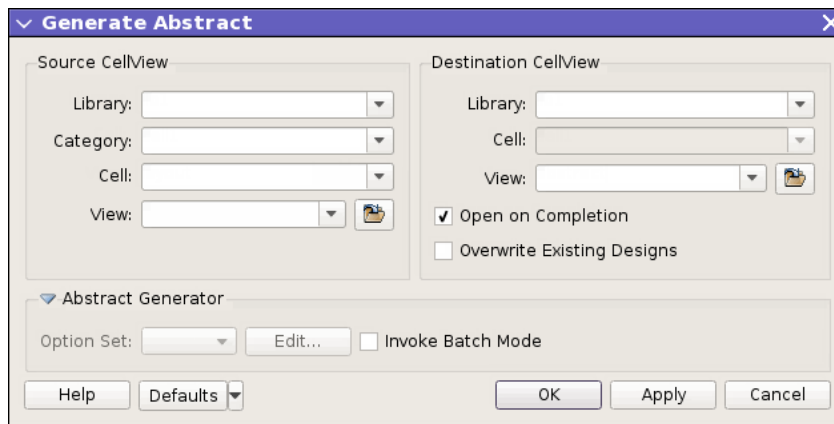
This topic describes how to use the **Tools > Abstract Generator** command to generate abstract views for multiple cells of a design library.

Alternatively, you can use the `db::showGenerateAbstract` Tcl command to generate abstract views for multiple cells. For more information, see Database Tcl in the *Custom Compiler Tcl Reference Manual*.

To create abstract views for multiple cells of a design:

1. Choose **Console** on the Custom Compiler **Home** page.
2. Choose **Tools > Abstract Generator**.

The **Generate Abstract** dialog box opens.



3. In the **Source CellView** area, select the **Library**, **Category**, **Cell**, and **View** of the design for which you want to create an abstract view. You can select multiple cells from the **Cell** list. In addition, you can select a category of cells to perform abstract generation with the same option set. When selecting multiple categories, cells in these categories will be the source cells.
4. In the **Destination CellView** area:
 - From the **Library** list, select the library in which you want to store the abstract design.
 - From the **Cell** list, show the cell that you selected from the source.
 - From the **View** list, select **abstract**.
 - Enable the **Open on Completion** option if you want to open the abstract design after the tool creates it.
 - Enable the **Overwrite Existing Designs** option if you want to overwrite existing abstract designs on the destination.
5. In the **Abstract Generator** area, select an **Option Set** for the abstract view.

An option set specifies whether you want to flatten the design, extract antenna ratio information, and other options. If you want to create new or modify existing option sets, see [Managing Abstract Generator Option Sets](#).

6. Click **OK** to generate the abstract view using the selected option set.

The tool adds the new abstract design to the specified library.

Generating Abstract Views From Layout Editor

This topic describes how to use the **Create > Abstract** command to generate the abstract view for a single cell in a library.

Note:

The **Create > Abstract** command is hidden if the design library is read-only or if you are in a read-only session.

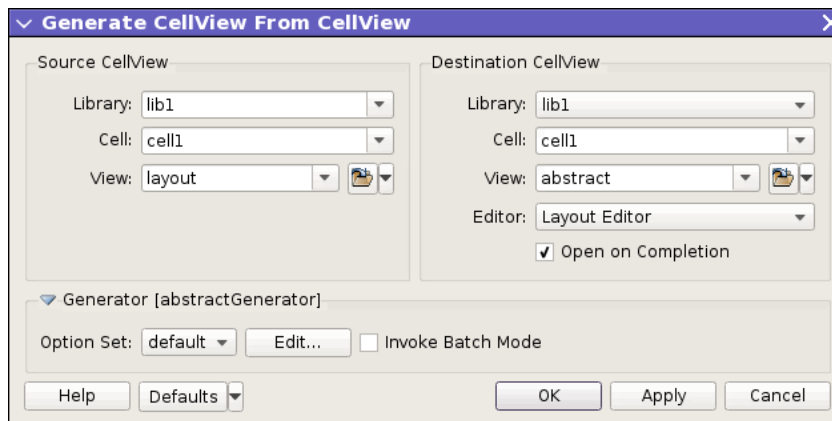
Alternatively, you can use the `le::generateAbstract` Tcl command to generate abstract views of designs. For more information, see Layout Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

 [Show me](#) an example of how to create an abstract view and export LEF for it.

To create an abstract view of a design:

1. In the Layout Editor, open the design for which you want to create an abstract view.
2. Choose **Create > Abstract**.

The **Generate CellView From CellView** dialog box opens.



3. In the **Source CellView** area, select the **Library**, **Cell**, and **View** of the design for which you want to create an abstract view.

4. In the **Destination CellView** area:

- From the **Library** list, select the library in which you want to store the abstract design.
- From the **Cell** list, select the cell category.
- From the **View** list, select **abstract**.
- From the Editor list, set **Layout Editor** as the default editor for *maskLayout* view type. When set, you can edit the abstract generation parameters in the **Generator** area.
- Enable the **Open on Completion** option if you want to open the abstract design after the tool creates it.

5. In the **Generator** area, select an **Option Set** for the abstract view.

An option set specifies whether you want to flatten the design, extract antenna ratio information, and other options. If you want to create new or modify existing option sets, see [Managing Abstract Generator Option Sets](#).

6. Click **OK** to generate the abstract view using the selected option set.

The tool adds the new abstract design to the specified library.

The tool ignores any layers that do not appear in the option set template: those layers do not become blockages or appear in the abstract.

For layers that appear in the option set template, but do not have values for all of the available options, the tool uses the default values described in [Managing Abstract Generator Option Sets](#).

Managing Abstract Generator Option Sets

You can create your own option sets for use during future Custom Compiler sessions. You can also modify, copy, and remove option sets.

This topic describes the following:

- [Displaying the Abstract Generator Options Dialog Box](#)
- [Creating New Option Sets](#)
- [Saving Option Sets to Files](#)
- [Loading Option Sets From Files](#)
- [Copying Existing Option Sets](#)

- [Modifying Existing Option Sets](#)
- [Removing Option Sets](#)
- [Generating Cellviews With Specific Options to P/G Pins and Signal Pins](#)
- [Extracting Antenna With GDS Mode](#)
- [Setting Run Directory for PVE](#)

Displaying the Abstract Generator Options Dialog Box

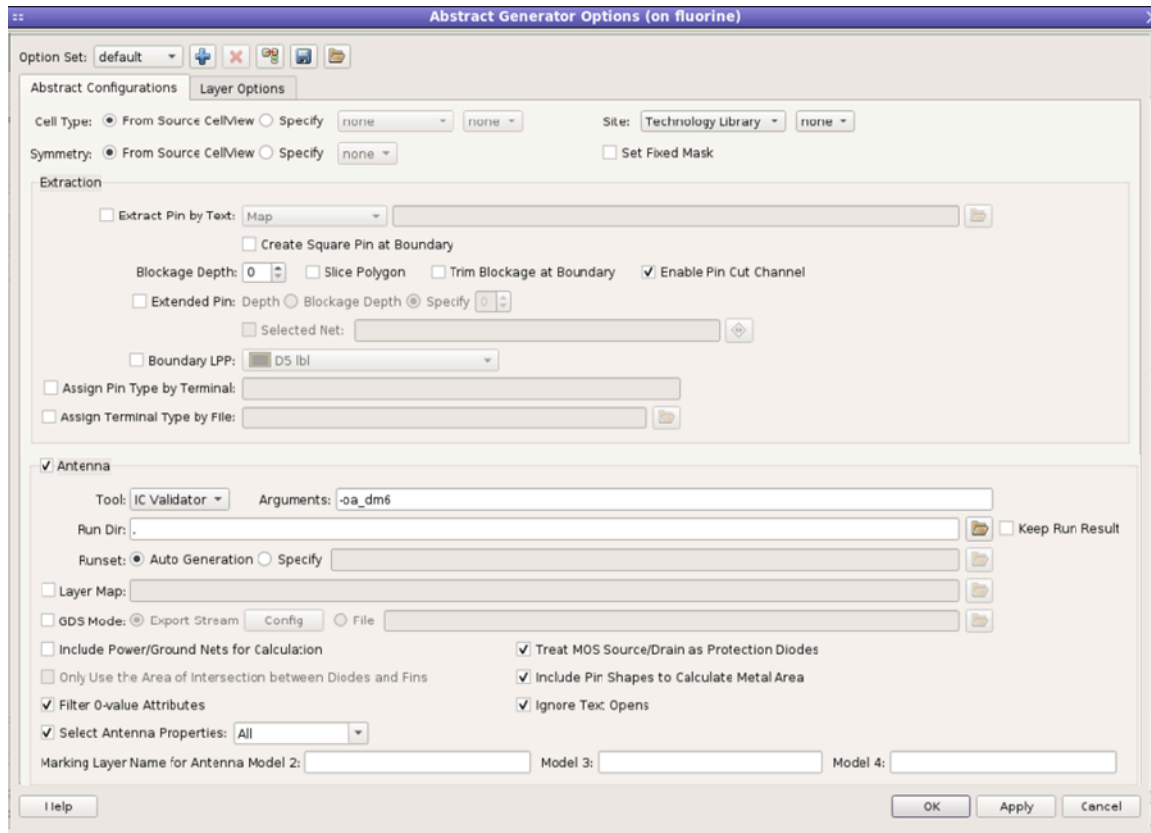
You create and manage option sets in the **Abstract Generator Options** dialog box.

To display the **Abstract Generator Options** dialog box:

1. In the Layout Editor, choose **Create > Abstract**. The **Generate CellView From CellView** dialog box opens.
(Optional) Click **Console** on the Custom Compiler **Home** page, and then choose **Tools > Abstract Generator**. The **Generate Abstract** dialog box opens.
2. In the **Generate CellView From CellView** dialog box, click **Edit** in the **Generator** area.
(Optional) In the **Generate Abstract** dialog box, click **Edit** in the **Abstract Generator** area.

The **Abstract Generator Options** dialog box opens.

Alternatively, you can use the `db::showAbstractGeneratorOptions` Tcl command to open the **Abstract Generator Options** dialog box.



3. Create a new option set. See [Creating New Option Sets](#).
4. Enable the **Invoke Batch Mode** option if you want run the abstract generation process in batch mode. See [Running Abstract Generator in Batch Mode](#) for more details.

Creating New Option Sets

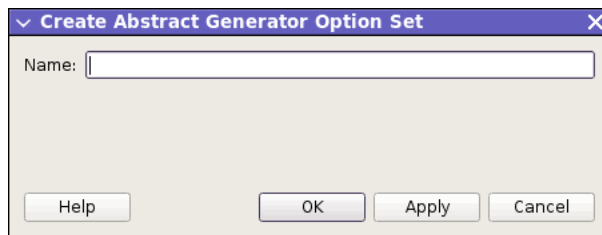
To create a new option set:

1. At the bottom of the **Abstract Generator Options** dialog box, choose **Create** from the **Manage Sets** list.

Alternatively, you can use the `db::createGenOptionSet` Tcl command to create a new option set. For more information, see “Database Tcl” in the *Custom Compiler Tcl Reference Manual*.

The **Manage Sets** list lets you create, copy, load, save, or delete option set templates. For instructions, see [Managing Abstract Generator Option Sets](#).

The **Create Abstract Generator Option Set** dialog box opens.



2. In the **Name** box, type a name for the new option set, and then click **OK**. The new name appears in the **Option Set** list, located at the top of the **Abstract Generator Options** dialog box.

The **Option Set** list specifies which option set template to use.

3. Edit the options in the following tabs in the **Abstract Generator Options** dialog box:

- [Abstract Configurations Tab](#)
- [Layer Options Tab](#)

4. Before you create abstract views from the design, you can use the following Tcl commands to set the cell symmetry properties in the layout view:

```
set block [oa::getTopBlock [ed]]  
  
oa::setSymmetry $block [oa::Symmetry "any"]
```

If you do not set the properties in the layout view, you can manually specify the symmetric orientation using **Symmetry** in the dialog box.

5. Click **OK** or **Apply** to save the option set to memory. To save the option set permanently to a file, see [Saving Option Sets to Files](#).

Caution:

If you do not save the option set to a file, you lose your changes when you exit the tool.

Abstract Configurations Tab

The **Abstract Configurations** tab contains the following options:

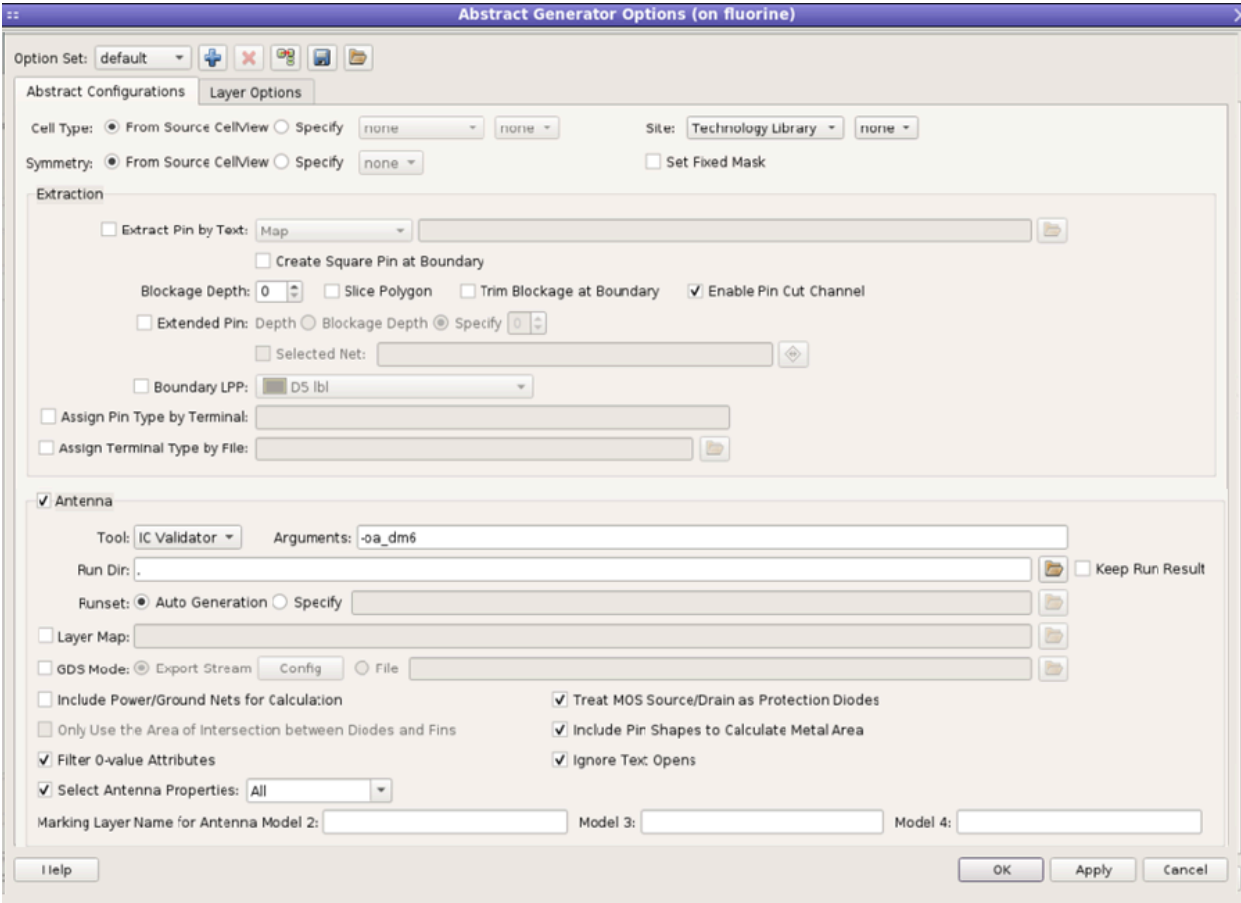


Table 7 Abstract Configurations Tab Options

Option	Description
Cell Type	Specifies the design cell type: • From Source Cellview: Automatically uses the cell type of the source design. • Specify: Lets you manually specify the cell type. Choose from the list of available OA cell types. Tcl preferences: <code>leAbstractUseSourceCellType</code>, <code>leAbstractCellSubType</code>

Table 7 Abstract Configurations Tab Options (Continued)

Option	Description
Symmetry	Specifies the cell symmetric properties: • From Source Cellview: Automatically uses the cell symmetry properties of the source design. • Specify: Lets you manually specify the cell's symmetric orientation. Choose from the list of available orientations: • none: The cell is not symmetric. • x: The cell is symmetric about the x-axis. • y: The cell is symmetric about the y-axis. • xy: The cell is symmetric about both the x- and y-axis. • any: The cell has rotational symmetry. Tcl preferences: <code>leAbstractUseSourceSymmetry</code>, <code>leAbstractSymmetry</code>
Site	Uses the site name from Technology Library or Specify to set the site attribute on abstract cellviews. • Technology Library: Predefined site definitions in the technology library. When selected, if no site definition in the technology library, only none is listed. The default is none, which means no site name is written to the abstract view. • Specify: Predefined site definitions in a directory. When specified, sets the specified value (if exists) as the site value written to the abstract view.
Set Fixed Mask	Sets fixed mask for MACRO in the LEF file by creating the <code>IntegrationColorModel</code> constraint with value 1 to abstract designs. If the <code>IntegrationColorModel</code> constraint is defined with non-zero value in the technology library, mask shifting is not allowed at all, then this option is disabled. Tcl preference: <code>leAbstractSetFixedMask</code>
Extraction	Specifies settings for extraction.

Table 7 Abstract Configurations Tab Options (Continued)

Option	Description
Extract Pin by Text	Preserves 0x (top-level) non-pin shapes that are overlapped with text as pin shapes, based on the mapping information in one of the following: • Technology Library: The mapping information is defined in <code>pinTextLayers</code> under Layer Rules Section. • Map: The mapping file that you specify. Use the Tcl preference <code>leAbstractTextToPinDepth</code> to specify shapes hierarchy depth to be extracted as pins, not limited to 0x shapes only. If the depth is greater than the blockage depth, uses the blockage depth. Note: This option requires a PinExtractionFile mapping information in the format: <pre>MI drawing M1 text M1 pin</pre> where: <code>M1 drawing</code> specifies the source shape layer and purpose. <code>M1 text</code> specifies the text layer and purpose. <code>M1 pin</code> specifies the output pin layer and purpose. The abstract generator queries the 0x M1 drawing shapes overlapping with the M1 text and then creates the Pin for these 0x shapes on the M1 pin in the abstract view. In the abstract view, the pin name for these pins would be the same as the text name in the source layout view. Tcl preferences: <code>leAbstractPinExtractionMode</code>, <code>leAbstractPinExtractionOption</code>, <code>leAbstractPinExtractionFile</code>
Create Square Pin at Boundary	Generates a square pin on the cell boundary if a pin touches the cell boundary and Extended Pin is disabled. Note that if the pin touches multiple edges of the cell boundary (for example: pass-through pins), no square pin is generated. This option is turned off by default.
Blockage Depth	Flattens to the depth that you specify. This option works with the Cover option in the Layer Options Tab. See Specifying the Routing Layers to Extract. Tcl preference: <code>leAbstractBlockageDepth</code>
Slice Polygon	Slices blockage polygon shapes into rectangles or trapezoids. Tcl preference: <code>leAbstractSlicePolygon</code>
Trim Blockage at Boundary	Limits blockage shapes within the cell boundary. Tcl preference: <code>leAbstractTrimBBdry</code>

Table 7 Abstract Configurations Tab Options (Continued)

Option	Description
Enable Pin Cut Channel	When enabled (default), cuts a boundary channel per pin access direction when creating cover blockages. Tcl preference: <code>leAbstractEnablePinCutChannel</code>
Extended Pin	Creates a pin island for a 0x (top-level) pin when the pin has connected shapes. Depth: Specifies the pin island depth with one of the following methods: • Blockage Depth: Uses the Blockage Depth value. • Specify: Specifies the Extended Pin Depth value that is equal to or less than the blockage depth. When Extended Pin is enabled, the abstract generator checks the <code>leUsePurposesForAbstractGenerator</code> Tcl preference. If <code>leUsePurposesForAbstractGenerator</code> is not set, then <code>leIgnorePurposes</code> is used to ignore connectivity. Selected Net (off by default) lets you specify the nets to extract extended pins. You can specify net names directly in the text field, or select nets on the Design Navigator using the selection button  to copy net names. Tcl preferences: <code>leAbstractExtendedPinMode</code>, <code>leAbstractExtendedPinDepthMode</code>, <code>leAbstractExtendedPinSpecifyDepth</code>
Boundary LPP	Generates a <code>prBoundary</code> object by shapes on the top design with a specified LPP object in abstract view. For example: A BoundaryLPP PM drawing uses shapes in {PM drawing} LPP to generate <code>prBoundary</code> in abstract view.
Assign Pin Type by Terminal	Assigns pin types (<code>none</code>, <code>core</code>, or <code>bump</code>) by terminal name. When creating pins on cell boundary, set the Tcl preference <code>leAbstractAssignPinTypeForBoundaryPin</code> to <code>true</code> to specify the terminal name for <code>core</code> and <code>bump</code> pins with paired brackets, for example: <code>{VDD* core} {VSS* bump}</code>. Wildcard expression is supported. To assign pin types for all pins that match the terminal name, set the Tcl preference <code>leAbstractAssignPinTypeForBoundaryPin</code> to <code>false</code>.

Table 7 Abstract Configurations Tab Options (Continued)

Option	Description
Assign Terminal Type by File	Assigns terminal type by netlist files. Click the Browse button to select one or more files. The tool reads port types of netlist using the <code>db::importText</code> Tcl command and sets to the terminal type (input, output, inout) in the abstract view. Netlist files can be CDL or Verilog files. Each SUBCKT (for CDL files) or module (for Verilog files) is in different text views after importing files. The tool gets the corresponding SUBCKT or module by cell name, to query the required terminal type in the text view. The terminal type mapping is listed in the log file, <i>abstract_generator.log</i>, which shows the file name to get the corresponding terminal type. If you specify more than one netlist file, the tool uses the first module when duplicate modules exist. Duplications are checked and a warning is reported. For the following CDL file, in cellview AND2, the output type is terminal O, the input type is terminal A/B/GND/VDD. <pre>.subckt AND2 O A B GND VDD *.PININFO O:O A:I B:I GND:I VDD:I</pre> For the following Verilog module, in cellview test1, A/B/C will be input type, and OUT will be output type. <pre>module test1 (A, B, C, OUT); input [7:0] A; input [7:0] A; input B; input [2:0] C; output [7:0] OUT; endmodule</pre> The abstract generator also supports assign terminal type by side file also. Here is the syntax for the supported terminal types: input, output, inputOutput, switch, jumper, unused, and tristate. <pre>#module name #term type term names Cell1 Input N1 N2 Output N3 N4 Cell2 Input N1 N2 Output N3 N4</pre>
Antenna	Extracts antenna data (for example, maximum antenna ratios) for terminals in the abstract design. Tcl preference: <code>leAbstractAntennaMode</code>

Table 7 Abstract Configurations Tab Options (Continued)

Option	Description
Tool	Specifies the physical verification tool to use for antenna data extraction. Select either Hercules or IC Validator. Tcl preference: <code>leAbstractPVEToolName</code>
Arguments	Specifies the PVE options for the tool you selected. Tcl preference: <code>leAbstractPVEOptions</code>
Run Dir	Specifies the directory of the temporary generated files. The verification tool automatically creates layer mapping and runset files. Tcl preference: <code>leAbstractPVERunDir</code>
Keep Run Result	Retains the generated files in the folder you selected in the Run Dir box. See Setting Run Directory for PVE for setting the sub directory name. Tcl preference: <code>leAbstractKeepRunResult</code>
Runset	Customizes the runset for antenna property extraction. • Auto Generation: Automatically generates the runset. • Specify: Specifies the custom runset file in absolute path. Tcl preferences: <code>leAbstractAutoGenerateRunset</code>, <code>leAbstractCustomRunsetFile</code>
GDS Mode	When disabled (default), generates a runset with OpenAccess mode. When enabled, select Export Stream (default) to run an Export Stream process to create a GDS file for the current cellview first, and then generates a runset for the created GDS file; otherwise, select File to specify an existing GDS file. You can set Export Stream options using the Config button, or use the default options instead. If you specify a layer map file in Export Stream, the tool uses the user-specified file for exporting, otherwise it uses the auto generated layer map by the abstract generator. See Extracting Antenna With GDS Mode for an example of a partial ICV runset. Using <code>xt::createJob</code>, the tool creates a job to run the PVE process with the generated runset, and lets you check the job status from the job monitor. The tool saves the antenna results to an abstract view after the job finishes, and shows a message on the Console, without opening the result cellview in the layout even if Open on Completion is turned on. Tcl preference: <code>leAbstractRunAsGDSMode</code>
Layer Map	This option is enabled only when you specify a custom runset file. When a layer map file is specified, the physical verification tool applies the layer mapping in the file. Tcl preferences: <code>leAbstractCustomLayerMapFile</code>, <code>leAbstractUseCustomLayerMap</code>

Table 7 Abstract Configurations Tab Options (Continued)

Option	Description
Include Power/Ground Nets for Calculation	Extracts antenna information for power and ground nets. You can generate cellviews with specific options to power/ground pins and other signal pins. For more information, see Generating Cellviews With Specific Options to P/G Pins and Signal Pins. Tcl preference: <code>leAbstractIncludePGNets</code>
Treat MOS Source/Drain as Protection Diodes	Treats MOS sources and drains as protection diodes. This option works with the Auto Generation option in Runset. Tcl preference: <code>leAbstractUseMosSourceDrain</code>
Only Use the Area of Intersection between Diodes and Fins	Uses only the intersecting area between diodes and fins for the calculation. This option works with the Auto Generation option in Runset. Tcl preference: <code>leAbstractUseIntersectionAreaWithFin</code>
Include Pin Shapes to Calculate Metal Area	Includes pin shapes for calculating metal areas. This option works with the Auto Generation option in Runset. Tcl preference: <code>leAbstractIncludePinArea</code>
Filter 0-value Attributes	Filters 0-value antenna values. This option is turned <i>on</i> by default.
Ignore Text Opens	This option is available only when Tool is IC Validator. When enabled (default), ignores text opens in connection. That is, text opens are not renamed or merged, <code>opens = opens = IGNORE</code> is set to <code>text_net</code> function in the runset. When disabled, the IC Validator merges open nets if possible.
Select Antenna Properties	Selects the properties to be generated during antenna extraction. Default is all properties.
Marking Layer Name for Antenna Model 2/3/4	Specifies the name of the marking layers for antenna models 2, 3, and 4. This option works with the Auto Generation option in Runset. The antenna models provide different antenna ratio constraints based on different oxide thicknesses. Model 1 specifies the built-in maximum ratio. Models 2, 3, and 4 specify additional ratios based on different oxide thicknesses. For more information about the antenna ratios and antenna layer constraints, see antenna in the <i>Custom CompilerTechnology and Display Resource File Reference Manual</i>. Tcl preferences: <code>leAbstractAntennaModel2LayerName</code>, <code>leAbstractAntennaModel3LayerName</code>, <code>leAbstractAntennaModel4LayerName</code>

Layer Options Tab

The **Layer Options** tab contains the following options:

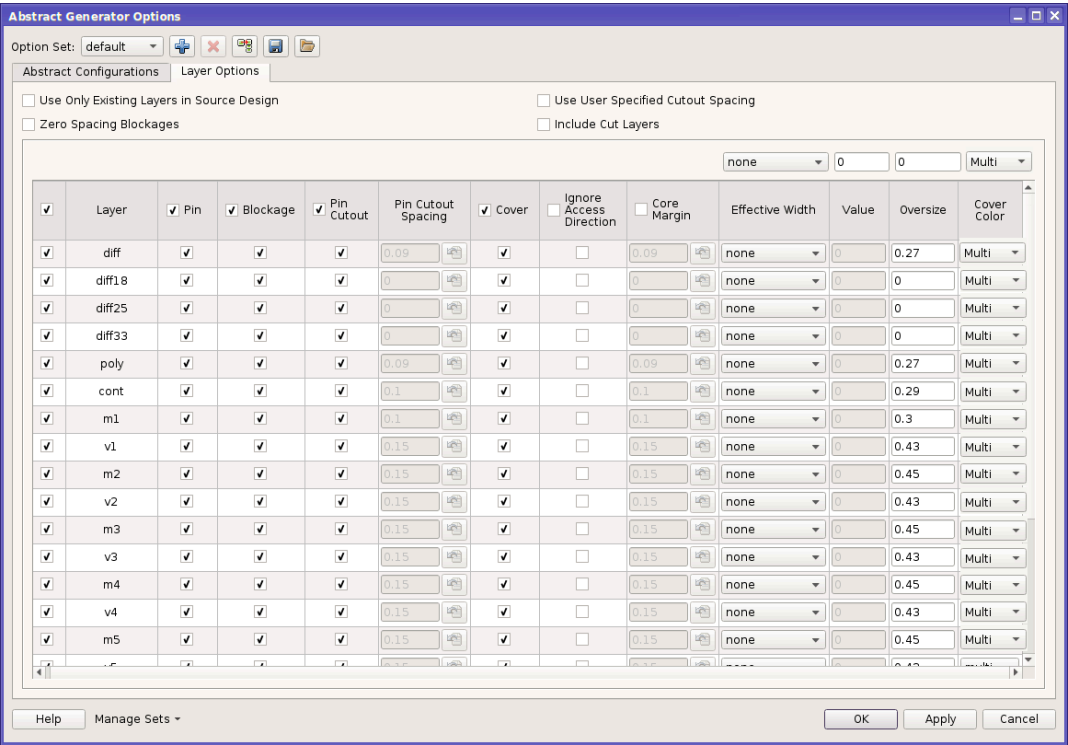


Table 8 Layer Options Tab Settings

Option	Description
Use Only Existing Layers in Source Design	Generates cover blockage with existing layers in the top block of the source design and the hierarchical instances based on the Blockage Depth . Tcl preference: <code>leAbstractUseExistingLayer</code>
Use User Specified Cutout Spacing	Specifies cutout spacing for each layer. When enabled, the Pin Cutout Spacing column is available in the table for you to specify pin cutout spacings. If you specify the horizontal and vertical spacings for special pins (<code>SpecialPinSpacing</code>) or the horizontal and vertical spacings for signal pins (<code>SignalPinSpacing</code>) using Tcl preferences <code>leAbstractSpecialPinHSpacing</code> , <code>leAbstractSpecialPinVSpacing</code> , <code>leAbstractSignalPinHSpacing</code> , <code>leAbstractSignalPinVSpacing</code> , the <code>SpecialPinSpacing</code> or <code>SignalPinSpacing</code> value is honored. See also Generating Cellviews With Specific Options to P/G Pins and Signal Pins .

Table 8 Layer Options Tab Settings (Continued)

Option	Description
Zero Spacing Blockages	Specifies the effective width of 0 for all blockage layers during abstract generation. Doing so permits the distance between blockages and pins to be 0. Generates zero spacing blockages for the layer when its minSpacing is 0 (including no constraint or the value is 0). When LEF is exported for the abstract, SPACING 0 attr is created for the blockages. For example: <pre>OBS LAYER M1 SPACING 0 ; POLYGON ... END</pre> Tcl preference: <code>leAbstractZeroBlockageSpacing</code>
Include Cut Layers	Considers cut layers in abstract generation. Cut layers from the <code>validRoutingLayer</code> constraint are listed in the table and they can be sorted according to the layer order when the Include Cut Layers option is enabled. Tcl preference: <code>leAbstractIncludeCutLayer</code>
Use (check box)	Includes the layer in the abstract design.
Layer	Identifies the layer.
Pin	Creates pin shapes by each layer.
Blockage	Creates blockage shapes by each layers when the <code>leAbstractExcludedBlockageLayers</code> Tcl preference is set to <code>false</code>.
Pin Cutout	Cuts out pin shapes when generating blockages.
Pin Cutout Spacing	Specifies pin cutout spacings using one or two numbers. Use one number if both horizontal and vertical spacings are equal. When using two numbers, the first number specifies the horizontal spacing, while the second specifies the vertical spacing. The default pin cutout spacing is the minimum spacing (<code>minSpacing</code>) for each layer. To reset the value to <code>minSpacing</code>, click Restore. This column is available only when Pin Cut and Use User Specified Cutout Spacing are enabled.

Table 8 Layer Options Tab Settings (Continued)

Option	Description
Cover	When enabled (default), creates blockages over the cell boundary for the specified layer and performs cutoff for pin access. The default color of the cover blockage is multi, which means different color objects are covered, and all other blockages on the specified layer also use multi color to merge blockages. If you do not want to create any blockage for specific layers, specify those layers using the <code>leAbstractExcludedBlockageLayers</code> Tcl preference. See also Exclusions. This option works with the Blockage Depth option in the Abstract Configurations Tab. See Specifying the Routing Layers to Extract. When disabled, creates shape-specific blockages on interconnect layers. Generated blockages are always merged according to the user-defined Oversize value. No cutoffs are done in blockages for pin access, and blockages take colors from parent objects in the source design. The corresponding Ignore Access Direction is also disabled.
Ignore Access Direction	When enabled, ignores the given pin direction and create cutoffs based on the placement of the source design pin shapes. When disabled (default), follows the given value of pin access direction attribute to trim blockages around the pin shape.
Core Margin	Enter the core margin to control the shrinking of routing blockage in each direction. • One margin value to shrink on all four sides. • Two margin values separated by space to shrink from left/right and Bottom/Top. • Four margin values separated by space to shrink left, right, bottom and top direction respectively. By default, Core Margin is turned off and the default value is the <code>minSpacing</code> value of the corresponding layer from the technology database.
Effective Width, Value	Applies the specified <code>effectiveWidth</code> value. These columns are available only when Cover is enabled.
Oversize	Specifies the distance to expand each edge during the merging of blockage shapes.
Cover Color	Sets colors for cover blockages using the value from color 1 to N (supported colors for the specified layer), Multi (default), Black, and Grey. This column is available only when Cover is enabled.

Saving Option Sets to Files

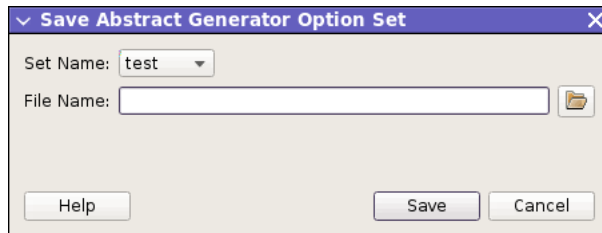
You must save option sets permanently to a file to reuse them later. The tool saves the option set data to an XML file.

To save an option set:

1. In the **Abstract Generator Options** dialog box, click  on top.

Alternatively, you can use the `db::saveGenOptionSets` Tcl command to save an option set.

The **Save Abstract Generator Option Set** dialog box opens.



2. In the **Set Name** list, choose the option set you want to save.
3. In the **File Name** box, type the name you want to assign to the option set file.
4. Click  and select the folder in which you want to save the option set file.
5. Click **Save**. The tool saves the option set data in an XML file.

Loading Option Sets From Files

In the **Abstract Generator Options** dialog box, you must load option sets so that they appear in the **Option Set** list. You can load an option set if you saved it permanently to a file. For instructions, see [Saving Option Sets to Files](#).

To load an option set:

1. In the **Abstract Generator Options** dialog box, click  on top.

Alternatively, you can use the `db::loadGenOptionSets` Tcl command to load an existing option set.

The **Load Abstract Generator Option Set** dialog box opens.

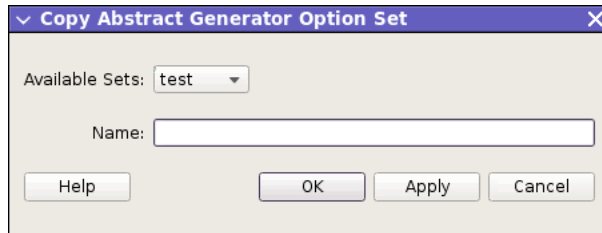
2. Locate and select the option set XML file, and then click **Load**. The tool imports and displays the option set in the **Abstract Generator Options** dialog box.

Copying Existing Option Sets

You can copy an existing option set, modify it, and then save it as a new option set.

To copy an existing option set:

1. In the **Abstract Generator Options** dialog box, click  on top. The **Copy Abstract Generator Option Set** dialog box opens.



2. From the **Available Sets** list, choose the option set you want to copy.
3. In the **Name** box, type a name for the new option set.
4. Click **OK**. The new name appears in the **Abstract Generator Options** dialog box.
5. Edit the options in the dialog box. For the description of each option, see [Abstract Configurations Tab](#) and [Layer Options Tab](#).
6. Click **OK** or **Apply** to save the option set in memory. To save the option set permanently to a file, see [Saving Option Sets to Files](#).

Caution:

If you do not export the option set to a file, you lose your changes when you exit the tool.

Modifying Existing Option Sets

To modify an existing option set template:

1. In the **Abstract Generator Options** dialog box, choose the **Option Set** you want to edit. If the option set is not in the list, load it first. See [Loading Option Sets From Files](#).
2. Edit the options in the dialog box. For the description of each option, see [Abstract Configurations Tab](#) and [Layer Options Tab](#).

3. Click **OK** or **Apply** to save the option set in memory. To save the option set permanently to a file, see [Saving Option Sets to Files](#).

Caution:

If you do not export the option set to a file, you lose your changes when you exit the tool.

Removing Option Sets

You can remove an option set from the **Option Set** list. Removing the option set from the **Option Set** list does not delete the option set XML file.

To remove an option set from the **Option Set** list:

1. In the **Abstract Generator Options** dialog box, choose the **Option Set** you want to delete. If the option set is not in the list, load it first. See [Loading Option Sets From Files](#).
2. From the **Manage Sets** list, choose **Delete**. A confirmation dialog box opens.
3. Click **Delete** to remove the selected option set.

You can also use the following Tcl script to remove an option set from memory:

```
set AGPSetObject [db::getGenOptionSets name  
-type abstractGenerator]  
db::destroy $AGPSetObject
```

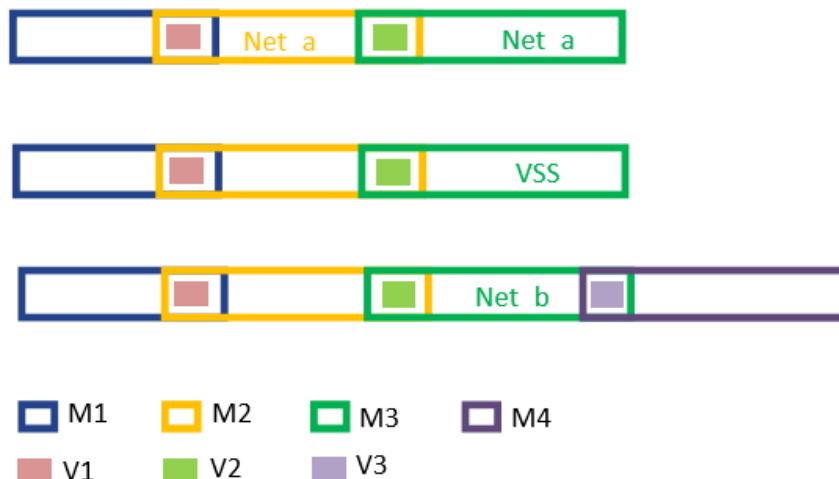
Generating Cellviews With Specific Options to P/G Pins and Signal Pins

You can generate cellviews with specific options to power/ground (P/G) pins and other signal pins for further power/ground analysis. The specific options include metal top levels for pin extraction and pin cutout options for blockage extraction. Use the Tcl preference `dbSigTypePowerFilter` to define special nets for power/ground nets, or use the Tcl preference `dbSigTypeGroundFilter` to query net's signal type from the source design. Other nets are treated as signal nets.

The specific options are described as follows:

- `SpecialNetTopLayerNum / SigNetTopLayerNum n`: Search top *n* metal layers from pin's layer as extended pins for power/ground nets or normal signal nets. You indicate top *n* metal layers from pin's layer, including cuts between metal layers if cut layers are specified to extract pins. The default is 0, and the tool doesn't check the top layer number for pins.

If multiple pins, which are not extended pins, are in the same connection island, that is, multiple texts with different layers are placed to extract pins, or multiple pins on different layers in the same connection, the tool starts top layer checking from pins with the higher layer during abstract generation. In the figure shown, there are three connection islands and shapes with label are pins.



For M3 and M2 pins with `Net_a`, if `SigNetTopLayerNum` is 1, only the M3 pin is extracted; if `SigNetTopLayerNum` is 2, M3 and M2 pins are extracted.

For VSS net, it belongs to the special net (ground net), and it refers `SpecialNetTopLayerNum` for pin extraction.

For `Net_b`, the pin is on M3 but connects to an M4 shape, and thus the layer that is upper than M3 will not be extracted as a pin.

- `EnablePinCutChannel false`: Turns off pin cutout channel for cover blockages. The default is `true`.
- `SpecialPinSpacing / SignalPinSpacing h v`: Specify special or signal pin cutout spacing for horizontal (H) and vertical (V) directions. You can specify one value if the H/V spacings are equal. The default pin cutout blockage spacing is `minwidth + minspacing` from technology library.
- `TopPowerMetalPinLPP / TopSignalMetalPinLPP ctmdmy drawing`: Create mask on the top layer metal for power/ground or signal pins. The mask is the same with top layer metal pins except LPP is different.

In the example below, you specify the options to extract power/ground pins with top 2 metal layers, other pins with top metal layer only, and create mask on {PM drawing} for top layer pin shapes for identifying. You also specify pin cutout channel, cutout spacing for power/ground nets and other nets.

```
SpecialNetTopLayerNum 2
SigNetTopLayerNum 1
EnablePinCutChannel false
SpecialPinSpacing 0.04 0.03
SignalPinSpacing 0.02 0.01
TopPowerMetalPinLPP PM drawing
```

The following figure shows the function of `EnablePinCutChannel` and `SpecialPinSpacing/SignalPinSpacing`.



Generally, a cover blockage is cut out by pin's direction or the nearest edge when **Ignore Access Direction** is enabled. When `EnablePinCutChannel` is turned off, no channel cut in this case. If the `SignalPinSpacing` or `SpecialPinSpacing` is specified, the spacing is used for cutout spacing, instead of using constraints from the tech rule. Beside specifying in the option set, you can use the Tcl preferences `leAbstractSpecialPinHSpacing`, `leAbstractSpecialPinVSpacing`, `leAbstractSignalPinHSpacing`, and `leAbstractSignalPinVSpacing` to specify pin spacings. The preference values are used if these spacing values are not specified in the option set.

Extracting Antenna With GDS Mode

The **GDS Mode** option lets the abstract generator run an Export Stream process to create a GDS file for the current cellview first, and then generates a runset for the created GDS file. Alternatively, you can select **File** to specify an existing GDS file.

The following example shows a partial ICV runset for GDS mode. The format is specified as GDSII, and the assignment is to specify the layer number in the layer map file, including the data type number. Logical information, such as pins, is not preserved in the GDS file. The abstract generator uses `-convertPinToText` to generate pin texts for antenna extraction during stream exporting.

```
#include <icv.rh>

library (
  library_name = ".cell.gds",
  format = GDSII,
  cell = "*"
)
```



```
);

error_options (
  error_limit_per_check = ERROR_LIMIT_UNLIMITED
);

tmp_OD_0_0 = assign( {{1, 0}} );
tmp_PO_0_0 = assign( {{2, 0}} );
tmp_PO_0_1 = assign( {{2, 1}} );

METAL1_1_0 = assign( {{9, 19}} );
METAL1_2_0 = assign( {{9, 38}} );
METAL1_0_0 = assign( {{9, 0}} );
METAL1_PIN_TEXT = assign_text( {{9}} );
```

Setting Run Directory for PVE

When running physical verification tools, the abstract generator creates a sub directory to save the layer map, the runset file, and the log files generated by the tool. The sub directory is cleared when the **Keep Run Result** option is turned off.

Setting the Tcl preference `leAbstractAntennaRunDirNameTemplate` lets you specify the template of the sub directory name. The default preference value is `.antenna.%c.%t`. See the following table for the supported tokens and substitution.

Token Name	Token Code	Substitution
Library	%l	Library name
Cell	%c	Cell name
View	%v	View name
Tool	%t	Tool name
User	%u	User name
Date	%d	YearMonthDay, for example: 20211208
Time	%T	HourMinuteSecond, for example: 120850

For example, when running antenna extraction by ICV for cellview `test_lib/test_cell/layout`, with the Tcl preference `leAbstractAntennaRunDirNameTemplate` set as `.antenna.%c.%t`, the files and logs generated by the physical verification tool are saved in `RunDir/.antenna.test_cell.icv`. Besides, the shell pid of the physical verification tool is saved as `<tool>.pid.<pid_number>`, for example: `icv.pid.1234`.

Running Abstract Generator in Batch Mode

You can run the abstract generator in batch mode or run a standalone executable of abstract generator directly.

To run the abstract generation process in batch mode, enable the **Invoke Batch Mode** option in the **Generate CellView From CellView** or **Generate Abstract** dialog boxes.

Alternatively, you can run a standalone process on terminal using the executable `abstract_generator`:

Syntax

```
abstract_generator -srcLib lib -srcCell cells -srcView view [-destLib destLib] [-destCell destCell] [-destView view] [-optionSet name] [-optionSetFile file] [-overwrite 0|1] [-prefFile arg] [-version] [-help]
```

Argument	Description
-srcLib lib	Source library name.
-srcCell cells	Source cell name. Multiple cells can be specified.
-srcView view	Source view name.
-destLib destLib	Destination library name.
-destCell destCell	Destination cell name. Multiple cells can be specified.
-destView view	Destination view name. Default: abstract.
-optionSet name	Option set name.
-optionSetFile file	Option set file in XML format.
-overwrite 0 1	Overwrites the existing view.
-prefFile arg	Preference file in XML format.
-version	Version information.
-help	Help messages.

Specifying the Routing Layers to Extract

The following options affect the routing layers that are extracted during abstract generation:

Option (Location)	Description
Blockage Depth (Abstract Configurations tab)	Select a value > 0 so that routing layers below 0x are searched. To search all layers, set the option to the lowest hierarchy level (32). See Abstract Configurations Tab .
Cover (Layer Options tab)	Clear the check box for the routing layers you want to include. This option depends on Blockage Depth . The Blockage Depth value must include the Cover layer for the setting to be effective. See Layer Options Tab .

For example, if you want to create routing blockages based on the existing design, set **Blockage Depth** to 32, and clear the **Cover** check box for all layers.

10

Reporting and Locating Short Circuits

The Custom Compiler environment provides features you can use to report and locate short circuits in your design.

This chapter describes the following topics:

- [Methods for Reporting and Locating Short Circuits](#)
- [Checking and Reporting Shorts Using the `le::checkShorts` Tcl Command](#)
- [Tracing Path Segments Using Highlight Path](#)

Methods for Reporting and Locating Short Circuits

A short circuit (or short) can occur when path segments on the same layer but belonging to different nets touch, causing the nets to connect. A short circuit can also occur across the design hierarchy when certain conditions are met. See [Short Circuits](#).

The Layout Editor provides two methods for locating or reporting shorts:

Method	Description
<code>le::checkShorts</code> Tcl command	Checks a design and reports shorts. See Checking and Reporting Shorts Using the <code>le::checkShorts</code> Tcl Command and the <code>le::checkShorts</code> description. Use the command if you do not know whether shorts exist in the design.
Highlight Path	Highlights path segments so that you can trace the paths and locate shorts. See Tracing Path Segments Using Highlight Path . Use the command if you know shorts exist in the design, and you want to locate them.

Checking and Reporting Shorts Using the `le::checkShorts` Tcl Command

Throughout the design stages, you can use the `le::checkShorts` Tcl command to check for shorts at the top level or across the entire design hierarchy. This topic describes how to use the command to check for and report shorts in your design.

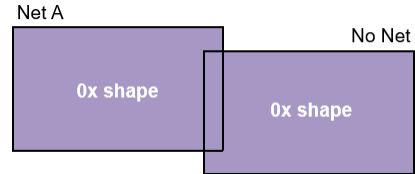
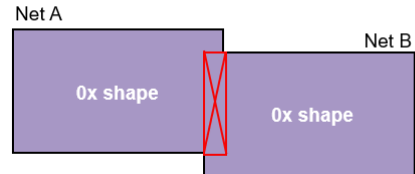
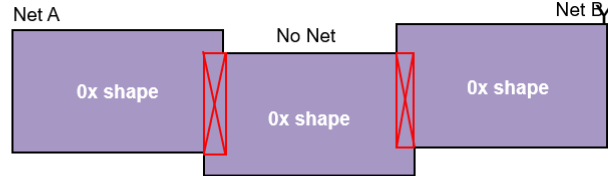
To check for shorts in your design:

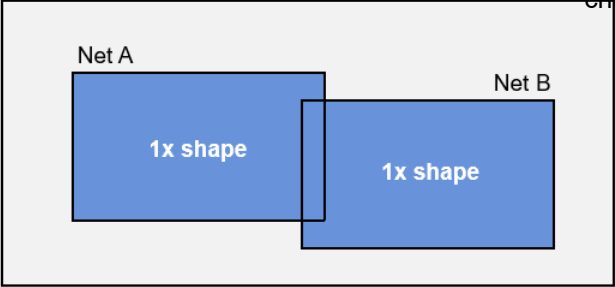
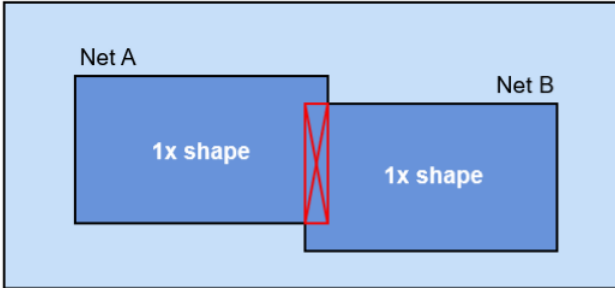
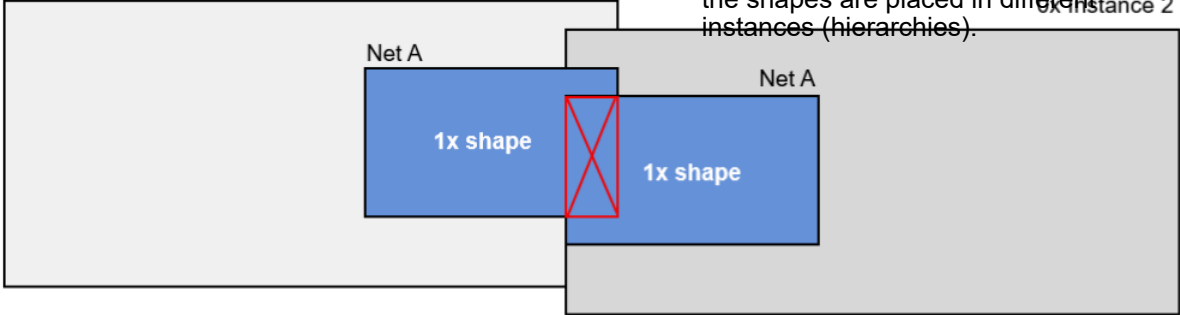
- 1. In the Console or in a Tcl script, use the following Tcl command:

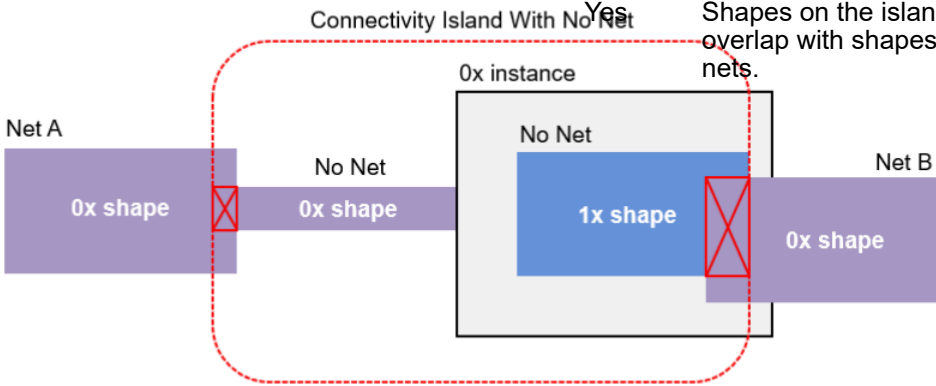
```
le::checkShorts -design <oaDesign> [-checkEntireHierarchy top | full]
[-depth <int>] [-skipCellsWithNoConnectivity true | false]
```

- 2. View the shorts in the Error Viewer Assistant.

The command reports shorts for the following conditions. For details, see [Short Circuits](#).

Condition	Short?	Reason
	No	Both 0x shapes are on M1, and one of the shapes does not have any net assignment.
	Yes	Although both 0x shapes are on M1, the two shapes have different nets.
	Yes	Although all three 0x shapes are on M1, two of the overlaps shapes have different nets.

Condition	Short?	Reason
0x instance 	Not checked	The command does not check for shorts inside instances. To determine whether a short exists, run the command in the instance master.
Instance Master 	Yes	The command detects shorts when checking the instance master. Although both 1x shapes are on M1, they have different nets.
0x Instance 1 	Yes	Although both 1x shapes are on M1, the shapes are placed in different instances (hierarchies).

Condition	Short?	Reason
	Yes	Shapes on the island with no net overlap with shapes that have different nets.

After running `le::checkShorts` Tcl command, you can use the **Highlight Path** command to trace segments and locate the shorts. See [Tracing Path Segments Using Highlight Path](#).

Tracing Path Segments Using Highlight Path

After you determine that your design has shorts, you can use the Layout Editor **Highlight Path** command to trace path segments and locate the shorts. Tracing path segments can be especially helpful for locating shorts in densely populated design areas.

This topic describes the following:

- [About Highlight Path](#)
- [Ignoring Instances](#)
- [Highlighting Paths](#)
- [Highlight Path Toolbar and Dialog Box](#)

See Also

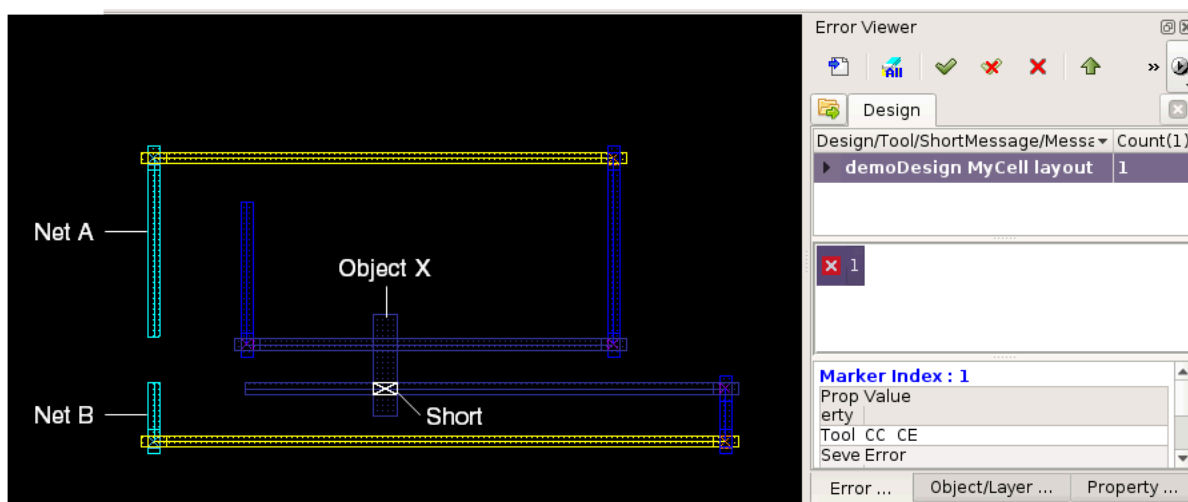
- [Checking and Reporting Shorts Using the `le::checkShorts` Tcl Command](#)

About Highlight Path

You can use **Highlight Path** if you are aware that short circuits (or shorts) exist in the design, and you want to locate the shorts. You can use the command to trace paths to sources of shorts on the same layer or across the design hierarchy.

A short occurs when path segments on the same layer but belonging to different nets touch, causing the nets to connect. A short can also occur under other circumstances in the design hierarchy. See [Short Circuits](#).

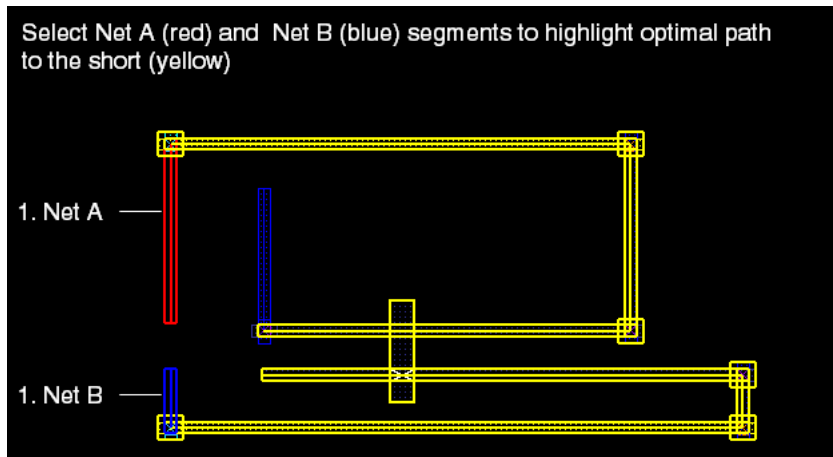
In the following example, a short circuit is created when an object (X) overlaps with segments on nets A and B. The connection occurs because object X is on the same layer as the two segments.



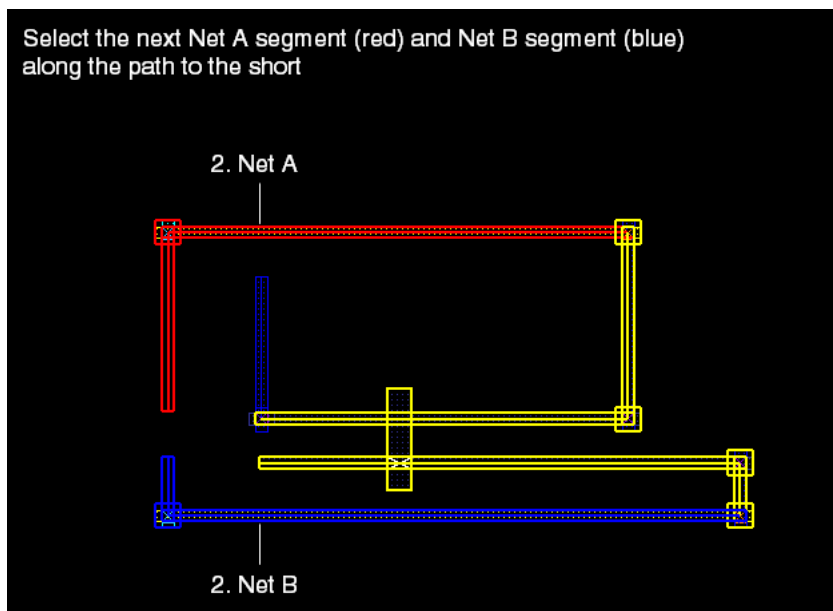
The example shown is extremely small with very short nets, so you can easily locate the short circuit. In complex and congested designs, locating short circuits might be difficult. The short circuit might be hidden behind other components and is difficult to access. Also, layouts that are generated without SDL might lack connectivity, and the Layout Editor might not catch the short circuit.

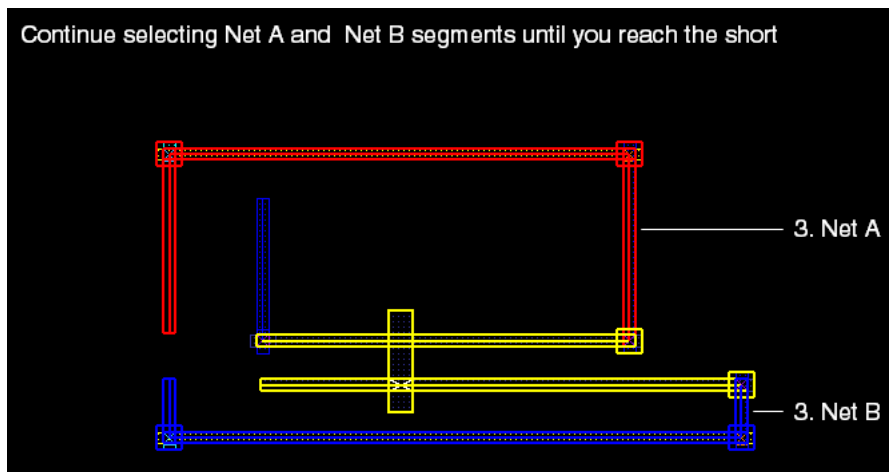
To overcome the limitations and to facilitate the search for short circuits across the design hierarchy, you can use the **Query > Highlight > Path** command. The command lets you highlight and visually trace path segments until you reach the short circuit.

During the Highlight Path operation, you select a segment on each of the two nets to indicate where you want to start and end the search. **Highlight > Path** highlights the selected segments in different colors so that you can distinguish them. With the start and end points identified, the command is able to highlight an optimal path between the points, containing the short circuit.



From the start and end points, you can visually trace (or select) the path segments on each of the nets until you reach the short circuit. As you select the path segments, the command highlights them using the assigned net colors, indicating that you have already inspected those segments.





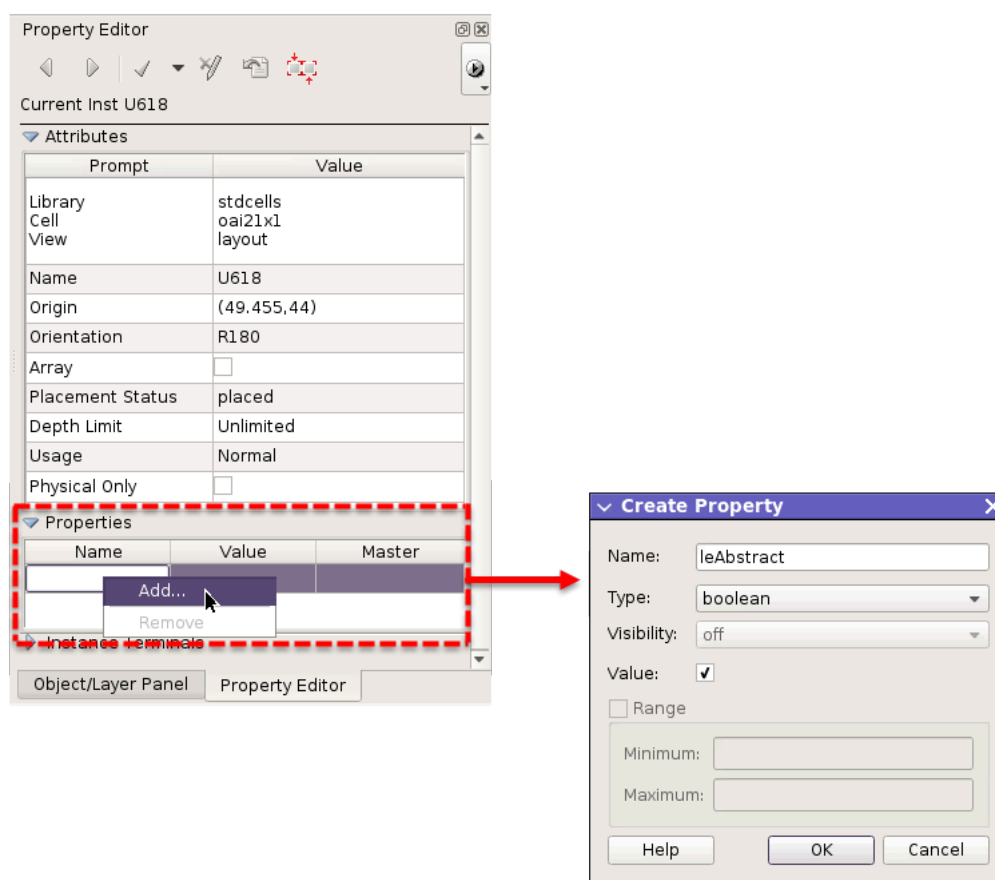
Ignoring Instances

You have the option of simplifying the search and ignoring the paths that are in instances during the Highlight Path operation. Doing so reduces the search area and time, especially if you are sure that the short circuits are not occurring within the instances.

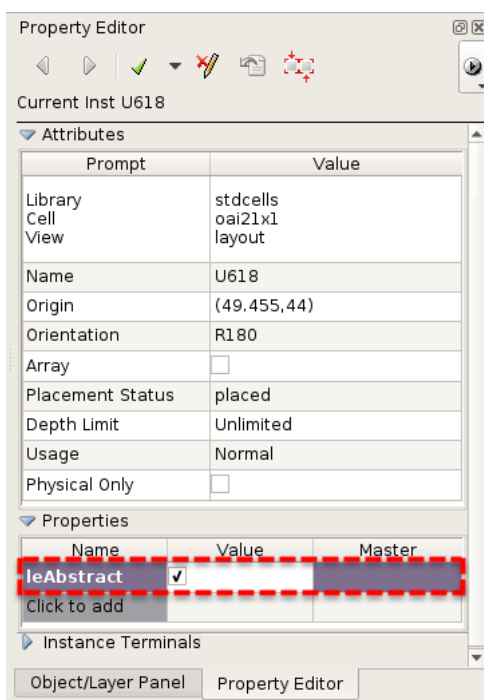
If you want to ignore instances and treat them as abstract objects, you must add and turn on the `leAbstract` property for the instances.

To add and turn on the `leAbstract` property for an instance:

1. In the Layout Editor, display the Property Editor.
2. On the canvas, select the instance. Information about the instance appears in the Property Editor.
3. In the Property Editor, locate and expand the **Properties** section, right-click an empty row, and choose **Add**. The **Create Property** dialog box opens.



4. Set the following options in the **Create Property** dialog box:
 - **Name:** Type `leAbstract`.
 - **Type:** Choose boolean.
 - **Visibility:** Choose Off.
 - **Value:** Select the check box to ignore instances during the Highlight Path operation by default.
5. When you are finished, click **OK** to return to the Property Editor. The `leAbstract` property appears, and it is turned on by default.



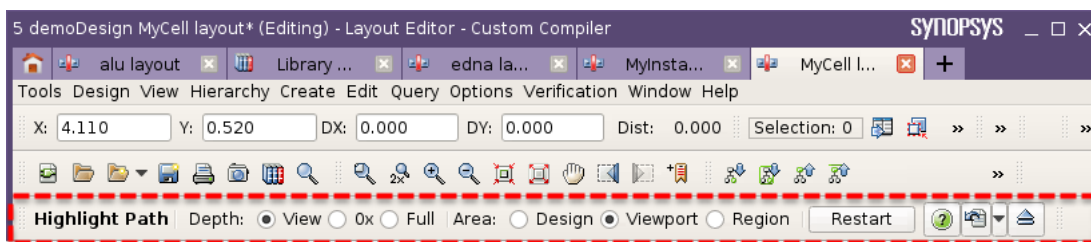
6. On the Property Editor toolbar, click **Apply Changes** .

Highlighting Paths

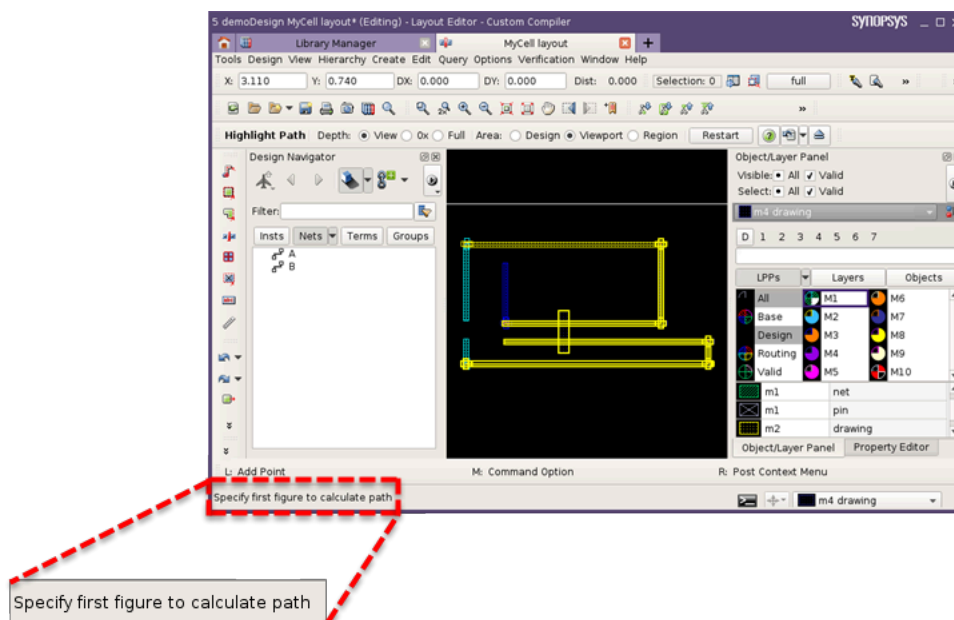
Before you start, turn on the Connectivity Engine in the Setting Layout Design Options dialog box.

To highlight the paths on the nets and locate the short circuit:

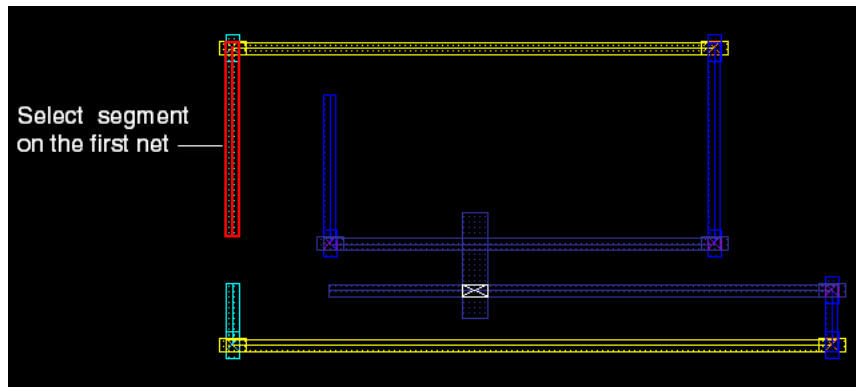
1. In the Layout Editor, choose **Query > Highlight > Path**. The **Highlight Path** toolbar appears.



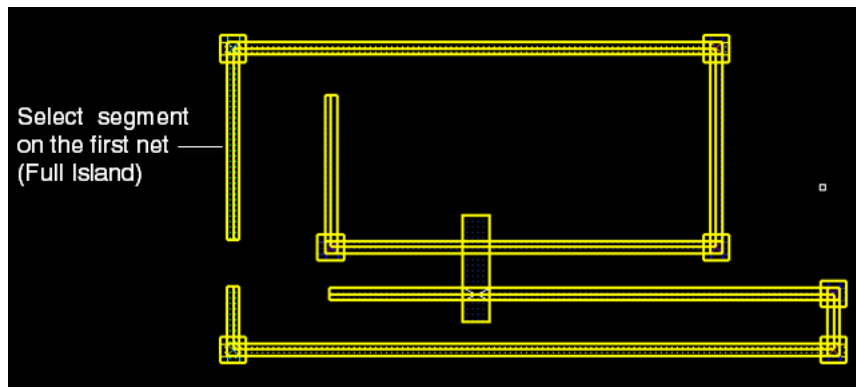
2. Set the options on the toolbar. Alternatively, click **Eject**  to display the **Highlight Path** dialog box and set additional options. See [Highlight Path Toolbar and Dialog Box](#).
3. Notice that the status bar displays step-by-step instructions. You can refer to the status bar message at any time during the operation for guidance.



4. On the canvas, select a path segment on the first net to indicate the start point for the search. The segment is highlighted with the color you selected in the **Highlight Path** dialog box. By default, the color is red.

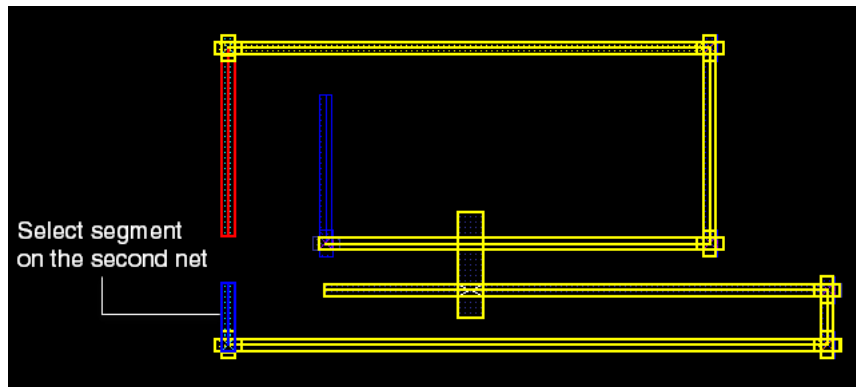


If you selected **Full Island** in the **Highlight Path** dialog box ([Highlight Path Toolbar and Dialog Box](#)), all shapes connected to the selected segment are highlighted. By default, the highlight color is yellow.



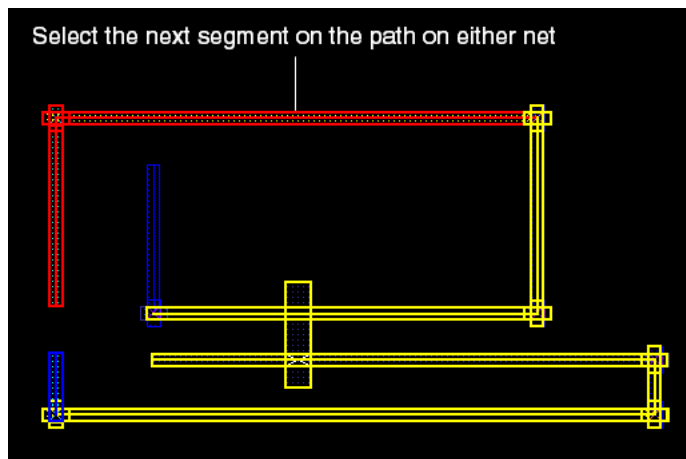
5. Select a path segment on the second net to indicate the end point for the search. The segment is highlighted with the color you selected in the **Highlight Path** dialog box. By default, the color is blue.

As soon as you select a segment on the second net, the command determines an optimal path that connects the two segments and contains the short circuit. The optimal path is highlighted yellow by default.



6. Select the next segment on the first net. If you prefer to select the next segment on the second net, press the Space key to switch nets.

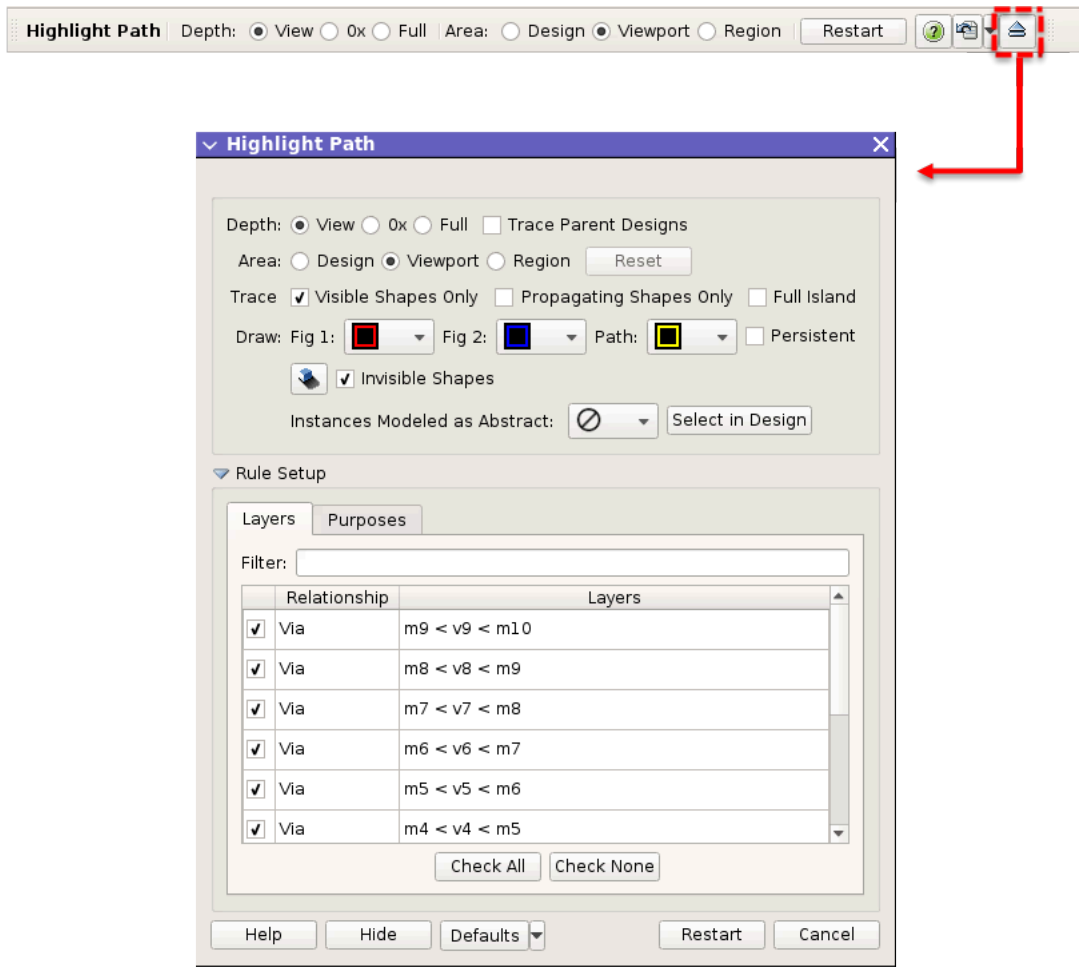
As you select the next segment, it is highlighted to indicate that you have inspected it.



7. Repeat [Step 6](#) until you locate the short circuit.

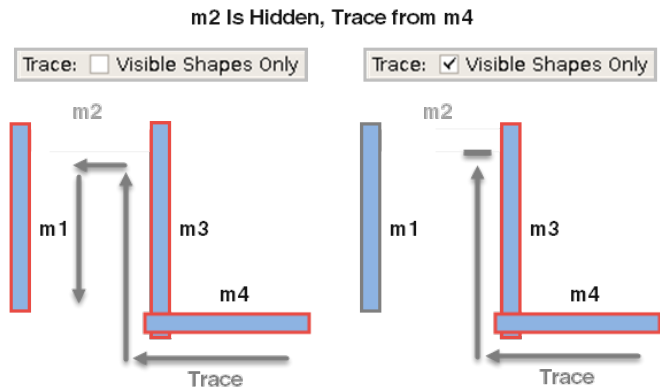
Highlight Path Toolbar and Dialog Box

This topic describes the options on the **Highlight Path** toolbar and in the **Highlight Path** dialog box. The toolbar contains a subset of options that are in the dialog box.



Option	Description
Depth	Specifies the stop level of the view: • View: Highlights the shapes up to the specified stop level. • 0x: Highlights the top-level shapes only. • Full: Highlights shapes in the entire design.
Area	Specifies the viewing area: • Design: Highlights the shapes in the entire design. • Viewport: Highlights only the shapes that are within the viewport. • Region: Highlights the shapes that are within the specified area. After you select Region, draw the area of interest on the canvas. Click Reset to remove the region you drew.

Option	Description
Trace Parent Designs (dialog box only)	Traces connected shapes based on one of the following selections: • Visible Shapes Only: Traces shapes on visible layer-purpose pairs (LPPs) only. Clear the check box to trace shapes on all layers. • Propagating Shapes Only: Traces and highlights connectivity-propagating shapes only. Clear the check box to trace and highlight all shapes. • Full Island: Highlights all shapes connected to the selected start segment. Full Island highlighting lets you see the affected path in its entirety. Clear the check box if you want to highlight only the shapes between the start and end segments. The following diagram shows the different highlighting results based on the Visible Shapes Only setting.



Option	Description
Draw (dialog box only)	Specifies the highlight color to use for: Fig 1: • Segments on the first net. • Tcl preference: leHighlightPathPacket1 • Default: highlight2 display packet (red) Fig 2: • Segments on the second net. • Tcl preference: leHighlightPathPacket2 • Default: highlight4 display packet (blue) Path: • Path between the start and end segments. • Tcl preference: leHighlightPathPacket Default: highlight1 display packet (yellow) • Default: highlight1 display packet (yellow) If you want to retain the Path highlighting after you exit the command and close the design, select Persistent. You can use Manage Highlights to control the Persistent setting.  Dims non-highlighted nets to increase the visual contrast between highlighted and non-highlighted nets. Select Invisible Shapes to highlight shapes that are on layers that are not currently visible.
Instances Modeled as Abstract (dialog box only)	Highlights the boundaries of the ignored instances with the color that you select. Click Select in Design to select the ignored instances on the canvas. The color used to indicate the selection is determined by the select1 display packet. For instructions on how to treat instances as abstract objects, see Ignoring Instances.
Rule Setup (dialog box only)	See Highlighting Connected Objects.
Restart	Removes all current highlighting from the Highlight Path operation.